MINISTRY OF EDUCATION OF THE REPUBLIC OF BELARUS
EDUCATIONAL INSTITUTION
«BREST STATE TECHNICAL UNIVERSITY»
Department of Intelligent Information Technologies

«I admit to the defense»
Head of the department
_____ В.А.Головко
«____»_____ 2027

Application for constructing a maritime vessel’s course based on neural-network classification of geographic zones

EXPLANATORY NOTE TO THE DIPLOMA PROJECT
ДР.ИИ2025.250978-11 81 00

Sheets: 95

Supervisor	А.А.Козинский
Completed	Li Haoran
Standard control	А.А.Козинский

2027

ABSTRACT

Explanatory note: 95 pages, 5 sections, 25 figures, 46 tables, 4 appendices, 42 references. Graphic material: 5 sheets of A1 format.

MARITIME ROUTE PLANNING, NEURAL NETWORK, CLASSIFICATION OF GEOGRAPHIC ZONES, AIS DATA, COST MAP, A* ALGORITHM, DYNAMIC PROGRAMMING, BELLMAN FUNCTION, GENETIC ALGORITHM, FITNESS FUNCTION.

The aim of the work is to develop an application that constructs a safe and economically reasonable course for a maritime vessel. A residual multilayer perceptron, trained on a corpus of eleven sea regions from which 5,427 mislabelled points were removed, classifies a geographic point into one of four zones; the zones form a traversal cost map, over which A*, Dijkstra, dynamic programming (Bellman function) or a genetic algorithm (fitness function) searches the route. The classifier reaches an accuracy of 96.95 % on 22,712 test points. On six benchmark voyages and one voyage in each of the eleven sea regions of the corpus the three exact methods return the same optimal cost, the genetic algorithm finds a navigable route on fifteen of them, and every great-circle route crosses land. A test on 100 random voyages, checked against an independent land mask, found 19 courses that cross land, mostly where the corpus samples the coast too sparsely; an automatic check of every route against that coastline, built into the planner, corrects 19 of them.

Faculty EIS Department IIT

APPROVED
Head of the
department _____
(signature)
“ ” _____ 2026

ASSIGNMENT
for the diploma project

Student Li Haoran

1. Topic of the project Application for constructing a maritime vessel’s course based on neural-network classification of geographic zones

Approved by the order of the university of _____2026 No. ___

2. Deadline for the submission of the completed project by the student 1.01.2027

3. Initial data for the project

General requirements for the application:

- The aim of the work is to develop an application for constructing an optimal and safe route of a maritime vessel on the basis of a classification of geographic zones using neural-network methods.
- The work is carried out using the Python programming language.
- The neural network is implemented with the TensorFlow, Keras or PyTorch libraries.
- The data source is a prepared data set of AIS data (Automatic Identification System), labelled with four classes of geographic zones:
  OPEN_SEA (open sea);
  COASTAL_SEA (coastal sea);
  NEAR_COAST (shore — land not further than 5 km from the coast);
  COASTLINE (land remote from the sea).

Functional requirements for the application:

- Loading of AIS data or of the coordinates of the route points.
- Classification of geographic zones using a pre-trained neural network.
- Construction of a traversal cost map on the basis of the classification (a weight for each zone).
- Search of the optimal route using the A* or the Dijkstra algorithm.
- Visualisation of the constructed route on a map.
- Comparison of the obtained route with the shortest-distance route.
- Visualisation of the constructed route on a map.
- Comparison of the obtained route with the shortest-distance route.
- Export of the results to the GeoJSON, CSV and PDF formats.

Requirements for the database:

- The kind of the database and the means of its implementation are determined by the developer.
- Stored data: coordinates of the points, classified zones, constructed routes, parameters of the routes (algorithm, total cost, execution time).

4. Contents of the explanatory note (list of the questions to be developed)

Introduction

1. System analysis and statement of the problem of creating an application for constructing the course of a maritime vessel
   1.1. Analysis of the subject area and review of existing solutions
   1.2. Statement of the problem of creating an application for constructing the course of a maritime vessel
2. Design of the application for constructing the course of a maritime vessel
   2.1. Development of the architecture of the application
   2.2. Design of the database
   2.3. Development of the model of classification of geographic zones
   2.4. Development of the route construction algorithms

3. Implementation and testing of the application
   3.1. Implementation of the module for loading and pre-processing AIS data
   3.2. Implementation of the neural network for the classification of geographic zones
   3.3. Implementation of the path search algorithms
   3.4. Implementation of the route visualisation module
   3.5. Integration of all modules and testing

4. Technical and economic justification
   4.1. Calculation of the development costs
   4.2. Evaluation of the economic effectiveness

5. Energy and resource saving
   5.1. Analysis of the energy consumption
   5.2. Recommendations for reducing the energy costs

Conclusion

List of abbreviations

List of references

Appendix A. User guide
Appendix Б. Program code of the application
Appendix B. Installation and configuration guide

5. List of the graphic material (with an exact indication of the obligatory drawings and charts):
   1. Statement of the problem of creating an application for constructing the course of a maritime vessel (poster – A1 format)
   2. Architecture of the application (poster – A1 format)
   3. Test results (screenshots of the interface, examples of constructed routes, comparison, poster – A1 format)
   4. Flowchart of the algorithm of classification of geographic zones using a neural network (drawing – A1 format)
   5. Flowchart of the route construction algorithm (drawing – A1 format)

6. Consultants on the project (with an indication of the sections of the project relating to them)
   – on ESPD: [full name, position]
   – on the economic section: [full name, position]

7. Date of issue of the assignment 01 September 2026

8. Calendar schedule of the work on the project for the whole period of design (with an indication of the deadlines and the labour intensity of the individual stages)

   Section 1 (System analysis): 01.09.2026 – 20.09.2026 15%
   Section 2 (Design): 21.09.2026 – 20.10.2026 20%
   Section 3 (Implementation and testing): 21.10.2026 – 10.12.2026 40%
   Section 4 (Economics): 11.12.2026 – 20.12.2026 5%
   Section 5 (Energy saving): 21.12.2026 – 25.12.2026 5%
   Preparation of the project: 26.12.2026 – 15.01.2027 10%
   Preparation for the defence: 16.01.2027 – 31.01.2027 5%

Supervisor _____
(signature)

The assignment is accepted for execution
(date) 01.09.2026

(signature of the student) _____

CONTENTS

INTRODUCTION..... 6
1 SYSTEM ANALYSIS AND STATEMENT OF THE PROBLEM.....8
1.1 Analysis of the subject area and review of existing solutions.....8
1.2 Statement of the problem.....11
2 DESIGN OF THE APPLICATION.....15
2.1 Design of the application architecture.....15
2.2 Design of the database.....18
2.3 Design of the geographic-zone classification model.....20
2.4 Design of the route construction algorithms.....26
3 IMPLEMENTATION AND TESTING OF THE APPLICATION.....33
3.1 Implementation of the AIS loading and pre-processing module.....33
3.2 Implementation of the neural network for zone classification.....34
3.3 Implementation of the path search algorithms.....38
3.4 Implementation of the visualisation module.....42
3.5 Integration of the modules and testing.....44
4 TECHNICAL AND ECONOMIC JUSTIFICATION.....56
4.1 Calculation of the development costs.....56
4.2 Evaluation of the economic effectiveness.....57
5 ENERGY AND RESOURCE SAVING.....61
5.1 Analysis of energy consumption.....61
5.2 Recommendations for reducing energy consumption.....62
CONCLUSION.....65
LIST OF ABBREVIATIONS.....68
LIST OF REFERENCES.....69
Appendix A. User guide.....73
Appendix B. Installation and configuration guide.....80
Appendix C. Results of the test on 100 random voyages.....85
Appendix D. Program text (separate document, 210 sheets).....96

INTRODUCTION

Maritime transport carries roughly four fifths of world trade by volume, and the fuel burned on that trade is both the largest single item in a ship operator's variable costs and a significant source of greenhouse-gas emissions [28]. The route a vessel actually sails is therefore not a neutral technical detail: it determines fuel consumption, passage time, charter costs, the exposure of the ship to navigational hazards and, ultimately, the safety of the crew and the cargo.

The naive answer to the question "what is the best course between two ports?" is the great circle — the shortest line on the surface of a sphere. For an aircraft at cruising altitude this answer is usually right. For a ship it is almost always wrong, because the shortest line on a sphere takes no account of the fact that most of that sphere is land. A great circle drawn from Kiel to Tallinn crosses Denmark and Sweden; a great circle from Odesa to Novorossiysk crosses the Crimean peninsula. A practical planner must therefore decide, for every point it considers, whether that point is water a vessel may sail through, water it should sail through with caution, or land it must avoid entirely.

Classically that decision is taken by intersecting the candidate route with vector coastline and depth-contour data from electronic navigational charts, as required by the guidelines for voyage planning [1] and by the safety-of-navigation chapter of the SOLAS convention [2]. That approach is accurate where such charts exist and are current, but it is expensive to license and heavy to process, and contour data is sparse in poorly surveyed waters.

This project takes a different route to the same decision. Instead of intersecting geometry, it learns the decision from data. A corpus of 156,838 geo-referenced points, each labelled with one of four geographic zones and each carrying bathymetric and infrastructure attributes, is used to train a neural network that maps a coordinate to a zone. The zones are then given traversal weights, the weights form a cost map over a geographic lattice, and a search — A* [3], Dijkstra [4], dynamic programming [39] or a genetic algorithm [31] — finds the cheapest path across that map. The result is a route that is, by construction, navigable over its whole length, and that can be compared quantitatively with the shortest-distance route it replaces.

The relevance of the work follows from three converging trends. First, open bathymetric and geospatial data — the NOAA and GEBCO grids [24, 25] and OpenStreetMap extracts [26] — have reached a coverage and quality that makes them a viable substitute for licensed chart products in a decision-support role. Second, AIS transponders have made vessel-track data abundant, so a corpus of real positions with real environmental context can be assembled without a dedicated survey. Third, a small neural network can now classify a whole planning lattice of tens of thousands of points in a fraction of a second on an ordinary processor.

The aim of the project is to develop an application for constructing the course of a maritime vessel on the basis of a neural-network classification of geographic zones.

To reach that aim the following tasks were set:

1) analyse the subject area and review existing approaches to maritime route planning and to the classification of geographic zones;
2) formulate the functional and non-functional requirements of the application;
3) design the architecture of the application and the structure of its database;
4) design and train a neural-network model that classifies a geographic point into one of four zones;
5) design and implement the construction of a traversal cost map and the search algorithms that operate over it, including dynamic programming and a genetic algorithm, and compare their evaluation functions;
6) implement the modules for loading and pre-processing AIS data, for classification, for path search, for visualisation and for export;
7) integrate the modules, test the resulting application and evaluate it experimentally against the shortest-distance route;
8) perform the technical and economic justification of the development;
9) analyse the energy consumption of the solution and give recommendations for reducing it.

The object of the study is the process of planning the course of a maritime vessel between two geographic points.

The subject of the study is a method of constructing a safe and economically reasonable route based on a neural-network classification of geographic zones.

The scientific and practical novelty of the work lies in the combination of a learned zone classifier with a classical optimal graph search, in such a way that the learned component supplies the cost model while the search retains its formal guarantee of optimality. The admissibility of the heuristic is preserved by deriving it from the minimum zone weight, and this was confirmed experimentally: on every test voyage A*, Dijkstra and dynamic programming returned exactly the same optimal cost. The same step cost is used as the basis of two evaluation functions — the Bellman function of dynamic programming and the fitness function of a genetic algorithm — which makes the two approaches directly comparable.

The explanatory note consists of an introduction, five sections, a conclusion, a list of abbreviations, a list of references and four appendices. The first section contains the system analysis and the statement of the problem, the second the design of the application, the third its implementation and testing, the fourth the technical and economic justification, and the fifth the analysis of energy consumption. Appendix A is the user guide and appendix B the installation and configuration guide; appendix C records the test of the application on 100 random voyages; appendix D, issued as a separate document, contains the text of the program.

1 SYSTEM ANALYSIS AND STATEMENT OF THE PROBLEM

1.1 Analysis of the subject area and review of existing solutions

1.1.1 The voyage planning process

Voyage planning is regulated internationally by IMO Resolution A.893(21) [1], which divides the process into four stages: appraisal, planning, execution and monitoring, and by chapter V of the SOLAS convention [2]. The present work addresses the second stage — planning — in which the navigator lays down the intended track on the chart, taking into account the draught of the vessel, the depth of water along the track, the proximity of the shore, traffic separation schemes, weather and the commercial constraints of the voyage.

In the planning stage the navigator repeatedly answers one elementary question: is this particular piece of water safe for this particular ship? Everything else — the choice of waypoints, the decision to take one strait rather than another, the acceptance of a longer track in exchange for deeper water — follows from a sequence of such answers. Automating voyage planning therefore means, first of all, automating that elementary judgement, and only then searching over the space of tracks that it admits.

It is important to state the boundaries of the present work clearly. The application developed here is a decision-support tool. It does not replace the officially approved passage plan, it does not use certified electronic navigational charts, and it must not be used as the sole basis for navigation. Its purpose is to produce a reasoned first approximation of a course, together with a quantitative comparison against the naive shortest-distance route, which a navigator can then refine against official charts and local knowledge.

1.1.2 Approaches to route construction

Three families of approaches to automated maritime route construction can be distinguished in the literature and in practice.

Geometric approaches treat the sea as a polygon with holes and compute a shortest path in the free space, typically over a visibility graph or a navigation mesh built from the coastline. They are exact with respect to the geometry they are given and they produce smooth tracks with few waypoints. Their weakness is that the geometry has to come from somewhere: high-quality vector coastlines are expensive, their generalisation level has to match the scale of

the voyage, and incorporating a continuous quantity such as depth into a polygon model requires slicing the sea into many depth bands, which multiplies the size of the geometry.

Raster approaches overlay a regular lattice on the area of interest, assign a traversal cost to every cell and run a graph search. They are simple, they accept any number of continuous cost factors, and they degrade gracefully where data is sparse. Their weakness is resolution: a strait narrower than one cell is invisible to the planner, and halving the cell size quadruples the number of cells. The present work uses a raster approach and addresses the resolution problem by automatic refinement, described in subsection 2.4.4.

Evolutionary and multi-objective approaches encode a route as a chromosome and optimise it against several objectives at once — fuel, time, safety, comfort — producing a Pareto front of candidate routes rather than a single answer [28]. They handle objectives that a scalar cost cannot express, but they give up the guarantee of optimality, their run time is hard to bound, and the result depends on the random seed. They are best suited to weather routing over several days, where the objectives genuinely conflict.

The present work deliberately stays in the second family and keeps the formal guarantee of optimality [5, 6], because the novel component — the learned cost model — is easier to evaluate when the search around it is exact. If the route is suboptimal it is the cost model that must be blamed, not the search.

1.1.3 Existing software

Table 1.1 compares representative existing solutions with the application developed in this project.

Table 1.1 – Comparison of existing solutions

Solution	**Class**	**Cost model**	**Optimality**	**Availability**
ECDIS route check (e.g. Transas, Wärtsilä)	Commercial, certified	Vector ENC geometry and depth contours	Checks a manual route; does not optimise	Licensed, ship-installed
Weather routing services (e.g. StormGeo, DTN)	Commercial service	Weather, currents, fuel model	Heuristic / multi-objective	Subscription
OpenSeaMap + generic GIS shortest path	Open source	Coastline polygons only	Exact over the given geometry	Free, no depth model
searoute / sea-distance libraries	Open source library	Pre-built maritime network graph	Exact over a fixed network	Free, fixed network, no local detail
Application developed in this project	Open, research / teaching	Learned four-zone classification from bathymetry and OSM context	Exact (A* with an admissible heuristic)	Free, runs offline on an ordinary computer

Two gaps follow from the comparison. Certified systems check a route that the navigator has already drawn, rather than proposing one, and they are not available outside a licensed installation. Open solutions that do propose a route either ignore depth entirely, working from a coastline polygon, or are restricted to a fixed pre-built network of ports and legs, which cannot

express an arbitrary pair of coordinates. The application developed here fills that gap: it proposes a route between arbitrary points, it takes depth and shore proximity into account through a learned model, and it runs without a licence on an ordinary computer.

1.1.4 Classification of geographic zones

The four zones required by the assignment are defined in table 1.2.

Table 1.2 – Definition of the geographic zones

Code	**Meaning**	**Navigable**	**Median depth / height in the corpus as delivered**
OPEN_SEA	Open sea — deep water far from any shoreline	yes	-2156 m
COASTAL_SEA	Coastal sea — navigable water close to the shore	yes	-27 m
NEAR_COAST	Near coast — land not further than 5 km from the shoreline	no	+10 m
COASTLINE	Inland — land remote from the sea	no	+216 m

The two water classes are separated by depth and by distance from the shore; the two land classes are separated by distance from the shore alone. The distinction between them matters to the planner for different reasons. OPEN_SEA and COASTAL_SEA both admit a vessel but carry different risk, so they receive different finite weights. NEAR_COAST and COASTLINE are both impassable, so their weights are identical; they are nevertheless kept apart because the boundary between them is what defines the shoreline, and a model that can place that boundary correctly is a model that understands the coast.

Figure 1.1 shows why the task is not trivial. The two extreme classes are separated by thousands of metres of water column and could be told apart by a threshold on depth alone. The two middle classes, however, occupy almost the same narrow band around sea level: in the corpus as delivered the fifth-to-ninety-fifth percentile range of COASTAL_SEA is -98 … 5 m and that of NEAR_COAST is -6 … 81 m. These two ranges overlap, so no threshold on depth can separate them, and the classifier must draw on spatial context instead.

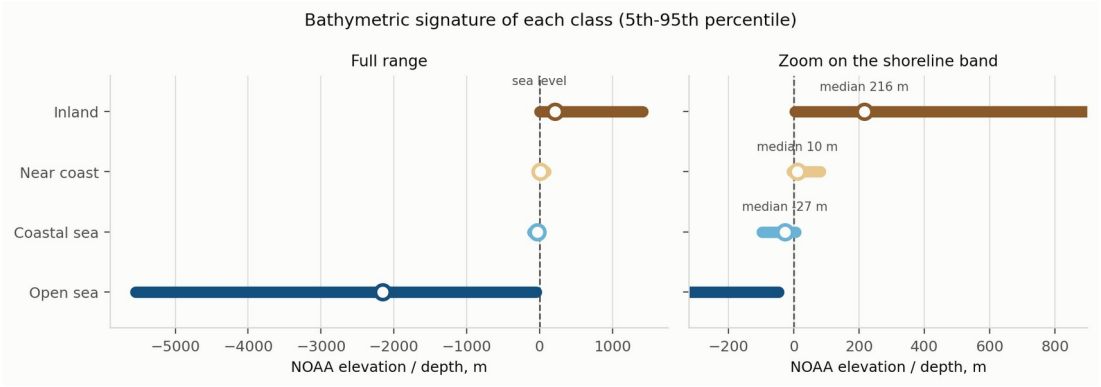


Figure 1.1 – Bathymetric signature of the four classes. Left: the full range. Right: a zoom on the shoreline band, where the two middle classes overlap

1.2 Statement of the problem

1.2.1 General statement

It is required to develop an application that, given a departure point and a destination point on the surface of the Earth, constructs a route between them that lies entirely within navigable water, minimises a traversal cost derived from a neural-network classification of geographic zones, and presents the result together with a quantitative comparison against the shortest-distance route.

Formally, let a geographic point be a pair q = (φ, λ) of latitude and longitude. Let Z = {OPEN_SEA, COASTAL_SEA, NEAR_COAST, COASTLINE} be the set of zones and let f: q → Z be a classifier. Let w: Z → $\mathbb{R}^+ \cup \{\infty\}$ assign a traversal weight to each zone. For a route R = (q_0, q_1, …, q_k) the cost is defined by formula (1.1):

$$C(R) = \Sigma_i \, d(q_{i-1}, q_i) \cdot ½ \cdot [\, w(f(q_{i-1})) + w(f(q_i)) \,], \qquad (1.1)$$

where d is the great-circle distance. The problem is to find a route R* from a given q_{start} to a given q_{end} that minimises C(R) subject to $w(f(q_i)) < \infty$ for every intermediate point. Because the departure and destination are berths chosen by the user, and a quay legitimately lies in a shore zone, the constraint is imposed on the interior of the route only.

1.2.2 Functional requirements

The functions the application shall provide are listed in table 1.3.

Table 1.3 – Functional requirements

ID	Requirement	Realised in
F1	Load AIS data or route-point coordinates from GeoJSON, CSV or JSON	Subsection 3.1
F2	Classify geographic zones with a pre-trained neural network	Subsections 2.3, 3.2
F3	Build a traversal cost map from the classification, with a weight per zone	Subsections 2.4.1, 3.3
F4	Search the optimal route with the A* or the Dijkstra algorithm; in addition, dynamic programming and a genetic algorithm	Subsections 2.4.2–2.4.3, 2.4.6–2.4.7, 3.3
F5	Visualise the route on an interactive map	Subsection 3.4
F6	Compare the route with the shortest-distance (great-circle) route	Subsections 2.4.5, 3.5.3
F7	Export the results to GeoJSON, CSV and PDF	Subsection 3.4.4
F8	Store points, classified zones, routes and route parameters in a database	Subsections 2.2, 3.5.1
F9	Show the progress of classification and search in real time	Subsection 3.4.2

1.2.3 Non-functional requirements

The non-functional requirements and the way each of them was verified are collected in table 1.4.

Table 1.4 – Non-functional requirements and their verification

ID	Requirement	Target	Achieved
N1	Classification accuracy on a held-out test set	≥ 90 %	96.95 %
N2	Macro F1 over the four classes	≥ 0.90	0.9695
N3	Planning time for a voyage of about 1 000 km	≤ 5 s	≤ 2.9 s on the benchmark voyages; 2.1 s on average, up to 53 s on the random test
N4	Interior of the produced route free of land	100 %	17 of 17 test voyages by the classifier; on the random test by an independent land mask 71 of 90 routes without the coastline check and 90 of 90 with it (subsections 3.5.6, 3.5.7)
N5	Optimality of the search	exact	A*, Dijkstra and dynamic programming agree, deviation 0 %
N6	Operation without a network connection	required	on-board chart, vendored client libraries
N7	Hardware	ordinary computer, no GPU	CPU only, 4 GB RAM
N8	Automated test coverage of the modules	all modules	101 tests, all passing

1.2.4 Input data

The source of training data is a labelled corpus of AIS observations supplied in GeoJSON format. It contains 156,838 point features collected in 28 recording sessions and spanning latitudes from -40.1° to 67.1° and longitudes from -87.9° to 168.3°. Every feature carries the attributes listed in table 1.5.

Table 1.5 – Attributes of a point in the source corpus

Attribute	**Type**	**Meaning**
timestamp	string	time of the observation (ISO 8601)
latitude, longitude	real	geographic coordinates, degrees
elevation	real	surface elevation, m (may be absent)
class_code	string	zone label: one of the four classes
class_name	string	human-readable name of the zone
source_session	integer	identifier of the recording session
noaa_depth	real	NOAA depth or height, m (negative below sea level)
noaa_type	string	“depth” or “height” — which quantity was sampled
osm_piers … osm_industrial	integer	nine counters of OpenStreetMap objects near the point

The corpus is moderately imbalanced: OPEN_SEA accounts for 36.3 % of the points, COASTAL_SEA for 24.9 %, NEAR_COAST for 20.4 % and COASTLINE for 18.4 %. In the corpus as delivered, elevation is missing for 7,066 points and the NOAA sample for 314 points; both gaps are handled by the feature extractor rather than by discarding the records, as described in subsection 2.3.3.

The corpus was delivered in two releases. The second release, supplied by the supervisor, keeps every point of the first one and adds 4,000 points of the Aegean Sea — 1,000 of each class, spread evenly over the rectangle from 34.4° to 41.3° N and from 19.1° to 30.9° E. Together with the 2,347 points of the first release that lie in the same rectangle they form the Aegean Sea region of 6,347 points. All numbers in this note refer to the second release.

The points were not collected in one sea but in eleven separate regions of the world ocean, listed in table 1.6. The regions were identified by clustering the coordinates of the corpus, the Aegean Sea being the region added in the second release, and are described in the configuration by rectangles of latitude and longitude; they are used to report the accuracy of the model region by region (subsection 3.2.6) and to build example routes in every region (subsection 3.5.5).

Table 1.6 – Sea regions of the corpus (after cleaning)

Region	Points	OPEN_SEA	COASTAL_SEA	NEAR_COAST	COASTLINE
North Europe (Baltic and North Sea)	29,693	6,333	9,790	6,616	6,954
South China Sea	19,442	11,516	4,031	2,310	1,585
Japan and North-West Pacific	18,425	11,137	2,329	2,285	2,674
Southern Africa (Cape of Good Hope)	15,770	5,563	5,227	3,128	1,852
Black Sea and Eastern Mediterranean	13,670	4,704	2,864	2,905	3,197
Persian Gulf and Arabian Sea	13,351	6,147	2,548	2,267	2,389
Red Sea and Gulf of Aden	11,162	1,782	3,452	3,150	2,778
Central America (Caribbean coast)	9,721	2,468	1,936	2,860	2,457
Strait of Malacca and Andaman Sea	8,386	3,329	2,091	1,743	1,223
Aegean Sea	6,124	1,413	1,615	1,781	1,315
Bay of Bengal	4,695	1,199	1,098	1,200	1,198
Other (inland samples)	972	63	0	0	909

Coverage outside these regions is thin, and the model should not be expected to classify distant seas reliably. A small share of the labels contradicts the measured height of the point — for example a coastal-sea point several metres above sea level. Such points are removed before training, as described in subsection 2.3.7.

The OpenStreetMap counters are sparse: fewer than one point in a hundred has a non-zero value in any of the nine columns. They are nevertheless retained, because where they are non-zero they are highly informative — a pier or a lighthouse is direct evidence of a shoreline — and because the feature extractor aggregates them over a neighbourhood, which turns a sparse point attribute into a dense contextual one.

1.2.5 Conclusions on the chapter

The analysis of the subject area shows that the practical difficulty of maritime route planning lies not in the search itself, which is a solved problem, but in the cost model that the search operates on. Existing open solutions either omit depth or are confined to a fixed network of legs; certified systems verify a route rather than propose one. The problem has therefore been stated as the construction of a learned four-zone cost model together with an exact search over it, and the functional and non-functional requirements of such an application have been formulated in tables 1.3 and 1.4.

2 DESIGN OF THE APPLICATION

2.1 Design of the application architecture

2.1.1 Architectural style and its justification

The application is built as a layered monolith with a single-page web client. Five layers are distinguished, each depending only on the layers below it: presentation, service, domain, data and export.

The responsibilities of the five layers are summarised in table 2.1. A layered monolith was chosen rather than a set of services for three reasons. The application is used by one operator at a time, so there is no load to distribute. The classifier holds a reference index of about a hundred thousand samples in memory, and splitting it across processes would multiply that memory rather than save it. And the dominant cost of a planning request is a single tight loop — classify the lattice, then search it — which is latency-sensitive and would only suffer from a network hop in the middle. A web client was nevertheless chosen over a desktop toolkit, because the map is the centre of the interface and browser mapping libraries are far ahead of desktop equivalents, and because a browser client runs unchanged on the operating systems found on a ship’s bridge.

Table 2.1 – Layers of the application

Layer	**Responsibility**	**Principal modules**
Presentation	Map, controls, live progress display, charts	index.html, app.js, api.js, charts.js
Service	HTTP endpoints, WebSocket channel, chart tiles	web/app.py, web/tiles.py
Domain	Classification façade, planning orchestration, cost map, search	model/inference.py, routing/planner.py, routing/coast_check.py, routing/cost_map.py, routing/astar.py, routing/dijkstra.py, routing/dynamic_programming.py, routing/genetic.py
Data	Loading, validation, feature engineering, persistence	data/ais_loader.py, data/cleaning.py, data/features.py, data/dataset.py, storage/repository.py
Export	GeoJSON, CSV and PDF writers	export/exporters.py

2.1.2 Decomposition into modules

Figure 2.1 shows the layered structure together with the request sequence of a single planning job and the schema of the database.

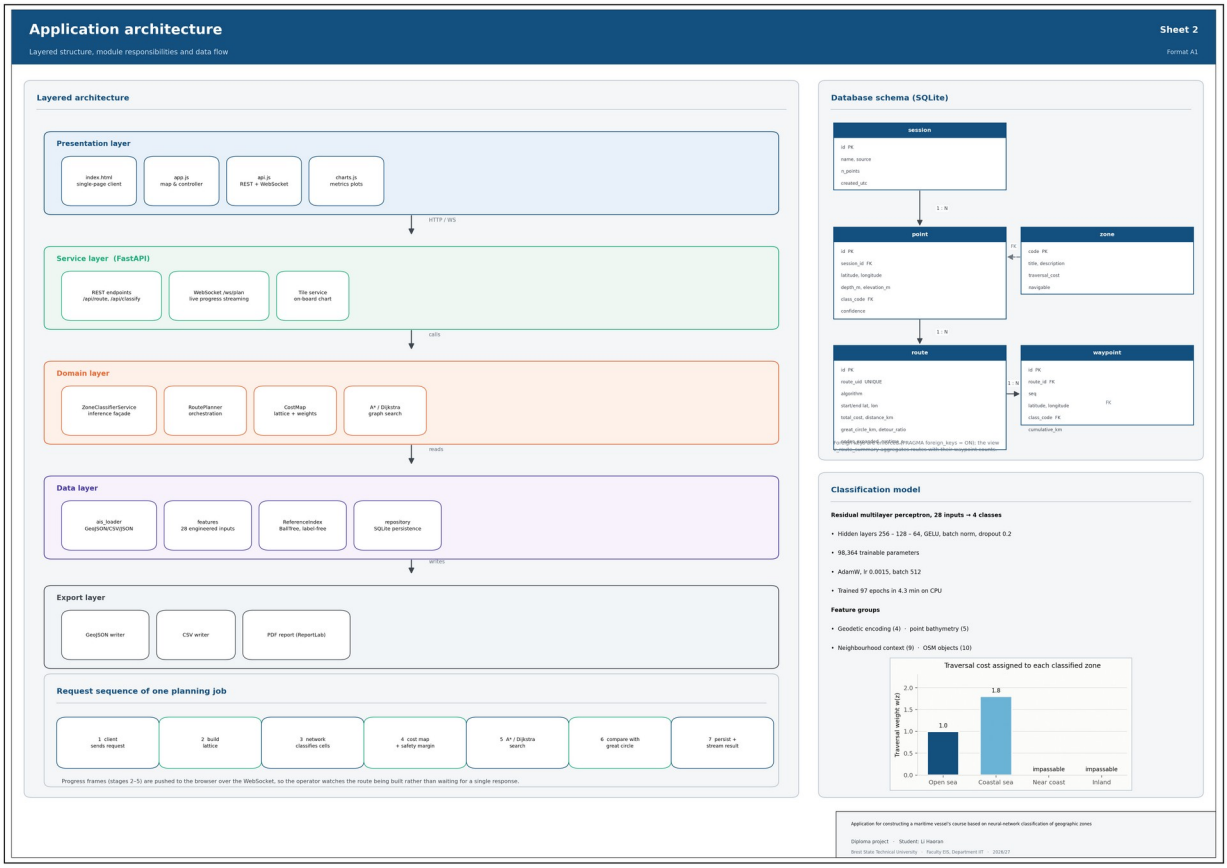


Figure 2.1 – Architecture of the application (sheet 2 of the graphic material)

The dependency rule is enforced by construction: a module of the domain layer never imports from the service layer, and the data layer knows nothing about either. All configurable values — paths, the zone taxonomy, the traversal weights, the hyper-parameters of the network and of the planner — are collected in a single module, config.py, so that no other module contains a hard-coded path, class index or weight.

2.1.3 The planning request sequence

A planning request passes through seven stages:

1) the client sends the departure, the destination and the planning parameters over the WebSocket;
2) the service builds a geographic lattice around the two points with a margin;
3) the classifier evaluates every cell centre of the lattice in batches;
4) the predicted zones are mapped to traversal weights and a safety margin is applied along the shore;
5) the chosen method — A*, Dijkstra, dynamic programming or the genetic algorithm — searches the cheapest path between the snapped endpoints;

6) the great circle between the same endpoints is evaluated on the same cost map for comparison;
7) the route is persisted in the database and streamed back to the client.

Stages two to five push progress frames to the client, so that the operator sees the lattice being classified and the search front advancing rather than waiting for a single response. The channel follows RFC 6455 [22] and complements the REST interface [23]. The design of that channel is described in subsection 2.1.4.

2.1.4 The real-time interaction channel

Interactivity is a requirement of the assignment, and it is not satisfied by a request-response interface alone: classifying a large lattice and searching it can take a second or more, during which a plain HTTP request would leave the interface frozen with no indication of progress. A WebSocket channel was therefore designed alongside the REST interface.

The channel carries the message types listed in table 2.2. The planning job runs in a worker thread; progress callbacks from that thread are marshalled onto the event loop through a queue, and frames are throttled to a configurable minimum interval so that a fast search cannot flood the client with thousands of messages.

Table 2.2 – Frame types of the real-time channel

Frame	**Direction**	**Content**
hello	server → client	protocol version and the zone palette
plan	client → server	departure, destination and planning parameters
classify	client → server	coordinates to classify (live pointer read-out)
started	server → client	acknowledgement, chosen algorithm and resolution
progress	server → client	stage, fraction complete, cells done or nodes expanded
overlay	server → client	down-sampled classification of the lattice
result	server → client	the complete route plan
classified	server → client	zone, confidence and depth of the queried points
error	server → client	a human-readable description of the failure
ping / pong	both	keep-alive

The client reconnects automatically with exponential back-off, and it queues frames sent while the socket is down. If the socket cannot be established at all, the client falls back to the equivalent REST endpoint, so the application remains usable — only without live progress — behind a proxy that forbids WebSocket upgrades.

2.2 Design of the database

2.2.1 Choice of the database management system

The assignment leaves the choice of the database to the developer. Three candidates were considered and compared in table 2.3.

Table 2.3 – Comparison of the candidate database management systems

Criterion	**SQLite**	**PostgreSQL + PostGIS**	**Files (JSON / CSV)**
Administration	none — a single file	server, users, backups	none
Transactional integrity	full ACID	full ACID	none
Foreign keys, views	yes	yes	no
Spatial operators	through an extension	native and rich	no
Deployment on a ship	trivial	heavy	trivial
Concurrency needed here	one operator	many clients	—
Suitability for this project	high	excessive	insufficient

SQLite was chosen. The application is a single-operator tool that must run on an ordinary computer, possibly offline and possibly with no administrator available; SQLite requires no server, stores the whole database in one file that can be copied as a backup, and still provides full transactional SQL with foreign keys, indexes and views. The spatial operators of PostGIS are not needed, because the only spatial query the application performs — the nearest-neighbour lookup in the reference index — is served by an in-memory ball tree that is far faster than a database round trip. Should a multi-user deployment ever be required, the repository pattern used in the data layer confines the change to a single module.

2.2.2 Logical model

Six relations and one view make up the schema, as table 2.4 shows. The zone table is a reference relation seeded at start-up from the configuration, so that the traversal weights stored in the database can never drift away from the weights the planner actually uses.

Table 2.4 – Relations of the database

Relation	Purpose	Principal attributes
session	One ingestion of AIS data or one interactive session	id, name, source, n_points, created_utc
point	A geographic point with its classification result	id, session_id, latitude, longitude, depth_m, class_code, confidence
zone	Reference table of the four zones	code, title, description, traversal_cost, navigable, colour
route	A planned route with its aggregated parameters	id, route_uid, algorithm, start/end coordinates, total_cost, distance_km, great_circle_km, detour_ratio, nodes_expanded, runtime_s, feasible
waypoint	An ordered point of a route	id, route_id, seq, latitude, longitude, class_code, cumulative_km, bearing_deg
model_run	Traceability of the deployed classifier	id, version, accuracy, macro_f1, cohen_kappa, parameters, trained_utc
v_route_summary	View: routes with their waypoint counts	route_uid, algorithm, distance_km, detour_ratio, n_waypoints

The attributes stored for a route are exactly those required by the assignment — the algorithm used, the total cost and the execution time — together with the quantities needed to reproduce and to compare the route: the endpoints, the lattice resolution, the number of expanded nodes, the great-circle distance and the serialised comparison against it.

2.2.3 Integrity and normalisation

The schema is in the third normal form. Every relation has a surrogate primary key; route additionally carries a unique business key, route_uid, which is the identifier exposed to the client and used in exported file names. Referential integrity is declared and enforced — the connection sets PRAGMA foreign_keys = ON — with cascade deletion from route to waypoint and from session to point, so that removing a route cannot leave orphaned waypoints behind.

Domain constraints are declared in the schema rather than only in the application: latitude is constrained to [−90, 90], longitude to [−180, 180], confidence to [0, 1], the algorithm to the five supported values and the boolean columns to zero or one. Non-finite values — an infinite cost on an infeasible route, a not-a-number detour ratio — are converted to NULL before insertion, so that SQL aggregates over these columns remain meaningful.

Three indexes support the access patterns that actually occur: point by session, point by coordinates, and route by creation time for the history list. No further index was added, because the remaining queries are either primary-key lookups or full scans of a small relation.

2.3 Design of the geographic-zone classification model

2.3.1 Formulation of the learning task

The task is a four-way classification of a geographic point. The natural input is the pair of coordinates, but coordinates alone carry almost no information about navigability: the same latitude and longitude would have to be memorised individually, and the model would not generalise one metre beyond the training corpus. The design problem is therefore the design of the features, not the design of the network.

Three requirements were imposed on the feature set. It must be computable for an arbitrary coordinate, not only for a point of the corpus, because the planner has to classify lattice cells that were never observed. It must not leak the target, directly or indirectly. And it must be reproducible in a deployment where the corpus is absent, so that the same features can be derived from public data sources.

2.3.2 The reference index

These requirements are satisfied by a label-free reference index. The index stores, for every training point, only its coordinates, its bathymetric sample, its elevation and its nine OpenStreetMap counters — never its class label. It is queried by a ball tree under the haversine metric, which returns the k nearest samples to an arbitrary coordinate in logarithmic time [16], the distance being computed by the haversine formula [17].

The index plays exactly the role that a NOAA or GEBCO bathymetry raster and an OpenStreetMap extract would play in a production deployment: it answers the question “what does the sea floor look like around this point, and what man-made objects are near it?”. Using it at inference time is therefore not a shortcut but a stand-in for a public data source, and the same features can be computed anywhere on Earth once that source is attached.

Three measures guarantee that no target information leaks through the index:

- the index stores no labels, so a leak would have to be indirect;
- the index is built from the training split alone — validation and test points are never inserted into it, so a test point cannot retrieve itself or a neighbour that was chosen with knowledge of its label;
- during the construction of the training features a point never queries itself: the nearest neighbour, which would be the point itself at distance zero, is discarded.

The corpus is split before the index is built, which is the order that makes the second measure possible. The split is stratified by class, in the proportion 70 : 15 : 15, giving 105,987 training, 22,712 validation and 22,712 test points.

2.3.3 The feature vector

Twenty-eight features in four groups are derived for every point.

Table 2.5 – Groups of input features

Group	Count	Features and their motivation
Geodetic encoding	5	sin φ, cos φ, sin λ, cos λ and the normalised absolute latitude. Trigonometric encoding is used instead of raw degrees so that the representation is continuous across the antimeridian and across the poles; a raw longitude would place −179° and +179° at opposite ends of the input range although they are neighbours.
Point measurements	5	signed logarithm of the bathymetric sample, its missing-value indicator, the height/depth type flag, signed logarithm of the elevation, its missing-value indicator. The signed logarithm $\operatorname{sign}(x)\cdot\ln(1+\lvert x\rvert)$ compresses a range of almost ten kilometres into a span the network can use while preserving the sign, which is the single most important bit of the input.
Neighbourhood context	9	mean, standard deviation, minimum, maximum and range of the depth of the eight nearest samples; the fraction of those samples that lie above sea level; agreement between the sign of the point and the sign of its neighbourhood; the logarithm of the distance to the nearest sample; the vertical gradient. This group is what separates the two middle classes: a point at −5 m surrounded by land is coastal water, the same depth surrounded by water is not.
OpenStreetMap objects	9	marine infrastructure (piers, lighthouses, breakwaters, harbours), natural coastal objects (beaches, cliffs) and inland water (rivers, lakes), each taken at the point and averaged over the neighbourhood; industrial objects at the point; the total and the density of objects in the neighbourhood.

The missing-value indicators deserve a comment. Rather than discarding the 6,023 points with no elevation and the 314 with no bathymetric sample, the extractor imputes the value from the interpolated neighbourhood and raises a binary flag. The network therefore learns both from the imputed value and from the fact that it was imputed, which is strictly more information than either discarding the record or imputing silently.

2.3.4 Architecture of the network

The architecture and the hyper-parameters adopted are collected in table 2.6. The input is a short, dense, tabular vector, which rules out convolutional and recurrent architectures; a multilayer perceptron is the appropriate family [7]. A plain perceptron was found to converge slowly on the boundary between the two middle classes, so residual blocks [8] were adopted: each block applies a linear map, batch normalisation [9], a GELU non-linearity [10] and dropout [12], and adds a projected skip connection.

Table 2.6 – Architecture and hyper-parameters of the classifier

Parameter	**Value**	**Justification**
Architecture	Residual MLP	tabular input; skip connections speed convergence on the hard boundary
Input dimension	28	see table 2.5
Hidden layers	256 – 128 – 64	a narrowing funnel; wider layers gave no measurable gain
Activation	GELU	smooth, avoids dead units
Normalisation	Batch normalisation	the input features differ in scale by orders of magnitude
Dropout	0.2	regularisation; higher values under-fit
Output	4 logits → softmax	four mutually exclusive classes
Trainable parameters	98,364	small enough for sub-millisecond batched inference on a CPU
Loss	Cross-entropy, class-weighted, label smoothing 0.02	the corpus is imbalanced; smoothing limits over-confidence
Optimiser	AdamW [11], lr 0.0015, weight decay 0.0001	decoupled weight decay regularises better than L2 in Adam
Batch size	512	a compromise between gradient noise and speed
Schedule	ReduceLROnPlateau, factor 0.5	halves the rate when validation loss stalls
Early stopping	patience 15 epochs	restores the best weights by validation loss
Random seed	42	reproducibility

Class weighting is applied as the inverse of the class frequency, normalised to a mean of one, so that the under-represented COASTLINE class is not sacrificed to the over-represented OPEN_SEA class. Label smoothing [13] of 0.02 keeps the network from driving its logits to saturation, which matters here because the confidence of the prediction is displayed to the operator and should be informative rather than uniformly close to one.

2.3.5 Forward pass of the network

The classifier is a feed-forward network: the input vector passes through a batch-normalisation layer, three hidden blocks and a linear output layer, and each layer depends only on the one before it. A network of this kind with enough hidden neurons can approximate any continuous mapping on a bounded domain [34], which is the theoretical basis for using it to classify a vector of features. Before it enters the network, every feature is standardised to zero mean and unit variance with the statistics of the training split, so that depths in metres, distances in kilometres and object counts enter on the same scale.

A layer of m neurons computes an affine transformation of its input, as formula (2.1) shows:

$$z = W \cdot h + b \tag{2.1}$$

where $h \in \mathbb{R}^n$ is the input of the layer, $W \in \mathbb{R}^{m \times n}$ the matrix of weights, $b \in \mathbb{R}^m$ the vector of biases. Each neuron forms a weighted sum of all n inputs; the weights and biases are the parameters that training adjusts. They are initialised by the Kaiming uniform scheme, which chooses the spread of the initial weights so that the variance of the signal is preserved from layer to layer.

The output of the layer is normalised over the mini-batch — batch normalisation [9], formula (2.2):

$$\hat{z} = \gamma \cdot (z - \mu_B) / \sqrt{(\sigma^2_B + \varepsilon)} + \beta \tag{2.2}$$

where μ_B and σ^2_B are the mean and the variance of z over the current mini-batch of 512 examples, $\varepsilon = 10^{-5}$ prevents division by zero, and γ and β are a learned scale and shift. During training the statistics of the mini-batch are used and running averages are accumulated; at inference the running averages are used, so the result for a point does not depend on the other points of the batch.

The non-linearity is the Gaussian error linear unit GELU [10], formula (2.3):

$$GELU(z) = z \cdot \Phi(z) \tag{2.3}$$

where Φ is the distribution function of the standard normal distribution. GELU passes large positive values almost unchanged, suppresses large negative ones and is smooth in between, so the gradient never vanishes abruptly as it does for the rectified linear unit. Without a non-linearity a stack of layers would reduce to one affine map, and the classes could be separated only by hyperplanes.

During training, dropout [12] sets each output of the main path to zero with probability 0.2 and multiplies the remaining outputs by $1/(1 - 0.2)$; the network cannot rely on any single neuron, which reduces over-fitting. At inference dropout is switched off. The four operations form one residual block [8], whose output is given by formula (2.4):

$$h_{l+1} = Dropout(GELU(BN(W_l \cdot h_l + b_l))) + P_l \cdot h_l \tag{2.4}$$

where P_l is a linear projection of the input onto the width of the block. The widths $28 \rightarrow 256 \rightarrow 128 \rightarrow 64$ all differ, so each of the three blocks has its own projection. The term $P_l \cdot h_l$ gives the gradient a short path back to the earlier layers, which is what made the boundary between the two middle classes converge faster than in a plain perceptron.

The output layer maps the last hidden vector to four numbers $z_1, \ldots, z_4$ (logits), which the softmax function turns into class probabilities, formula (2.5):

$$p_k = exp(z_k) / \Sigma_j exp(z_j), \quad k = 1, \ldots, 4 \tag{2.5}$$

The predicted zone is the class with the largest probability, and that probability is shown to the operator as the confidence of the prediction. The whole network has 98,364 trainable parameters.

2.3.6 Training of the network

The network is trained by supervised learning: the zone y of every training point is known, and the parameters θ — all weights, biases, γ and β — are adjusted to minimise a loss that measures the disagreement between the predicted probabilities and the known zones. The loss is the class-weighted cross-entropy with label smoothing, formula (2.6):

$$L(\theta) = - \Sigma_i \Sigma_k \alpha_k \cdot q_{ik} \cdot ln\, p_{ik} / \Sigma_i \alpha_{yi} \tag{2.6}$$

where i runs over the examples of a mini-batch, k over the four classes, p_{ik} is the probability (2.5) of class k for example i, and q_{ik} is the smoothed target: $1 - \varepsilon + \varepsilon/4$ for the true class and $\varepsilon/4$ for the others, with $\varepsilon = 0.02$ [13, 38]. The class weight $\alpha_k = N / (4 \cdot N_k)$, normalised to a mean of one, where N is the number of training points and N_k the number of those of class k, compensates for the imbalance of the corpus.

The gradient of the loss with respect to every parameter is computed by back-propagation [33]: the chain rule is applied layer by layer from the output back to the input, so that all the partial derivatives are obtained at the cost of about one additional pass through the network. The norm of the gradient is clipped to 5.0, which protects the first epochs, when the loss is large, from excessive steps. The parameters are then updated by the AdamW optimiser [11, 29], formulas (2.7) and (2.8):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{2.7}$$

$$\theta_t = \theta_{t-1} - \eta \cdot [\, \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon) + \lambda \cdot \theta_{t-1} \,] \tag{2.8}$$

where g_t is the gradient at step t; m_t and v_t are running averages of the gradient and of its square with $\beta_1 = 0.9$ and $\beta_2 = 0.999$; $\hat{m}_t$ and $\hat{v}_t$ are the same averages corrected for their zero initialisation; $\delta = 10^{-8}$ prevents division by zero; $\eta = 0.0015$ is the learning rate and $\lambda = 0.0001$

the weight decay. Every parameter thus receives its own step size, inversely proportional to the typical size of its gradient. AdamW applies the weight decay to the parameters directly rather than through the gradient, which regularises better than an L2 term in the loss [11].

One pass over the 105,987 training points, in 208 mini-batches, is an epoch. After every epoch the loss is computed on the validation split, which takes no part in the updates. When the validation loss has not improved for more than 6 epochs the learning rate is halved; when it has not improved for 15 epochs training stops and the weights of the epoch with the lowest validation loss are restored (early stopping [37]). The test split is used once, for the final evaluation in subsection 3.2.3.

Besides the neural network the application uses two further groups of artificial-intelligence methods. The reference index of subsection 2.3.2 is a nearest-neighbour model: it describes the surroundings of an arbitrary point by the measurements of the closest known points. The route search of section 2.4 uses the problem-solving methods of AI [5] — heuristic search, dynamic programming and evolutionary search.

2.3.7 Cleaning of the training data

A corpus labelled from several sources contains labelling errors, and a classifier trained on them learns the errors together with the rule [32]. In the supplied corpus the most reliably detectable error is a contradiction between the class of a point and its measured height: a sea point that lies several metres above sea level, or a land point that lies below it. Every point carries two independent height measurements, both positive above sea level: the NOAA relief value (attribute noaa_depth) and the elevation recorded by the data collector (attribute elevation). Each of them is checked separately. With C_w = {OPEN_SEA, COASTAL_SEA} the set of water classes and C_l = {NEAR_COAST, COASTLINE} the set of land classes, a point is removed when condition (2.9) holds for at least one of its measurements:

$$(y \in C_w \ \text{and} \ h_k > \tau) \quad \text{or} \quad (y \in C_l \ \text{and} \ h_k < -\tau), \quad k \in \{noaa, elev\} \tag{2.9}$$

where y is the label, h_k the k-th measured height and τ a tolerance that absorbs tides and the vertical error of the relief models. A missing measurement is not checked, and a point without any measurement is kept. The number of points removed at different tolerances is given in table 2.7.

Table 2.7 – Points removed by the consistency check

Tolerance τ, m	**0**	**2**	**5**	**10**	**20**
Points removed	7,546	6,523	5,427	4,124	2,558
Share of the corpus	4.81 %	4.16 %	3.46 %	2.63 %	1.63 %

The tolerance of 5 m was adopted. A smaller value begins to remove genuine beach and tidal-flat points, whose height legitimately differs from zero by a metre or two; a larger one keeps obvious errors such as a coastal-sea point on a ten-metre bank. At 5 m the check removes 5,427 points, 3.46 % of the corpus: 1,297 OPEN_SEA and 2,111 COASTAL_SEA points above sea level, and 1,707 NEAR_COAST and 312 COASTLINE points below it. The conflict is revealed by the NOAA value alone for 3,900 points, by the collector's elevation alone for 332 and by both for 1,195. 151,411 points remain, which is exactly the number of clean points stated by the supervisor for the second release of the corpus; this confirms that the rule reproduces the intended cleaning. The check is applied before the corpus is split, so the validation and test splits are cleaned as well. Its effect on the model is analysed in subsection 3.2.5.

2.4 Design of the route construction algorithms

2.4.1 The cost map

The search operates on a regular lattice in geographic coordinates, built around the bounding box of the departure and the destination with a margin of two degrees. The margin matters: without it, a route that has to go around a peninsula lying outside the bounding box could not be found.

Every cell centre is classified, and the predicted zone is mapped to a traversal weight through table 2.8. The two land classes receive an infinite weight, which the implementation represents by a large finite constant so that the weights can be stored in an ordinary numeric array.

Table 2.8 – Traversal weights

Zone	**Weight w(z)**	**Interpretation**
OPEN_SEA	1.0	reference cost of one kilometre of open-water passage
COASTAL_SEA	1.8	navigable, but shallower, busier and closer to hazards; one kilometre here is treated as costly as 1.8 km of open sea
NEAR_COAST	∞	land — impassable
COASTLINE	∞	land — impassable

The ratio of 1.8 between coastal and open water is a policy parameter rather than a physical constant. It was chosen so that the planner accepts a detour of up to roughly eighty per cent of the length of a coastal leg in order to reach open water, which corresponds to the practice of standing well off a lee shore, while still allowing coastal passage where no open-sea alternative exists — as in the Danish straits, where the whole route is necessarily coastal. The

parameter is exposed in the configuration, and the effect of changing it is examined in subsection 3.5.4.

A safety margin is then applied: every water cell orthogonally adjacent to a land cell has its weight multiplied by 1.6. This produces a penalised belt one cell wide along every shoreline, so that the optimiser prefers to stand off the coast where it can, without forbidding coastal passage where it cannot. The belt is computed by an iterative dilation of the land mask, which costs a handful of array operations and no additional classification.

2.4.2 The cost of an edge and the A* heuristic

The lattice is treated as an eight-connected graph. The cost of moving from cell u to an adjacent cell v is the great-circle length of the step multiplied by the mean of the two cell weights, as formula (2.10) shows:

$$c(u, v) = d_{gc}(u, v) \cdot \tfrac{1}{2} \cdot [\, w(z_u) + w(z_v) \,] \tag{2.10}$$

Averaging the two weights rather than taking the weight of the destination makes the cost symmetric, so that the cost of a route does not depend on the direction in which it is sailed. The poly-line the search returns follows the lattice and is therefore stepped; where a smoother track is wanted it is simplified by the Douglas–Peucker algorithm [18] within a tolerance the operator sets, which removes the staircase without moving the route off navigable water.

For A* [3] the heuristic is the great-circle distance from the current cell to the goal, multiplied by the smallest weight present anywhere on the map; it is given by formula (2.11):

$$h(v) = w_{min} \cdot d_{gc}(v, goal) \tag{2.11}$$

This heuristic is admissible. Any route from v to the goal has a length of at least d_{gc}(v, goal), because the great circle is the shortest path on a sphere, and every kilometre of it costs at least w_{min}, because no cell weight is smaller. Hence h(v) never exceeds the true remaining cost, and A* is therefore guaranteed to return the optimal path. The heuristic is also consistent, since the triangle inequality holds for great-circle distances, so no node has to be re-expanded. The guarantee was verified experimentally: on all seventeen test voyages A* and Dijkstra returned identical costs, with a maximum deviation of 0 %.

2.4.3 Choice of the search algorithms

Both algorithms required by the assignment are implemented, as two independent procedures rather than as one procedure with the heuristic switched off, so that their node expansion counts and run times can be compared on identical cost maps. Following the recommendation of the supervisor, two further methods were added: dynamic programming and a genetic algorithm; they are designed in subsections 2.4.6 and 2.4.7. The properties of the four methods are compared in table 2.9.

Table 2.9 – Properties of the implemented search methods

Property	**Dijkstra**	**A***	**Dynamic programming**	**Genetic algorithm**
What is minimised	g(v)	f(v) = g(v) + h(v)	V(v), Bellman equation	fitness F of a whole route
Optimality	guaranteed	guaranteed, h admissible	guaranteed	not guaranteed
Time	O(E log V)	O(E log V), fewer nodes	O(K · E), K sweeps	O(G · P · L)
Result	one route	one route	cost-to-go of every cell	one route
Use in the application	verification	default	verification, comparison	comparison

Here V is the number of cells, E the number of moves between them, K the number of sweeps of value iteration, G the number of generations, P the size of the population and L the length of a route in cells.

The great-circle route is implemented as a fifth, degenerate “algorithm”, so that it can be requested, stored, displayed and exported through exactly the same interfaces as the four real ones, which keeps the comparison in subsection 2.4.5 free of special cases.

2.4.4 Automatic refinement of the lattice

A raster planner cannot see a channel narrower than one cell. At a resolution of 0.25° a cell is about sixteen kilometres wide at the latitude of Denmark, while the navigable channel of the Great Belt is narrower than that; the planner therefore reports, correctly with respect to its own model but wrongly with respect to the world, that no route exists from the North Sea into the Baltic.

This failure mode was discovered during testing and is addressed by automatic refinement. When the search finds no path, the planner does not report failure: it multiplies the lattice step by 0.55, widens the margin around the two ports by two degrees, rebuilds the cost map and searches again, up to three times, down to a floor of 0.04°. The widening lets the planner find a passage lying outside the first rectangle, such as the Skagerrak on a voyage from the North

Sea into the Baltic. Only if the finest lattice still admits no path is the request reported as infeasible, and the message then names the resolution that was reached, so that the operator can distinguish a genuinely landlocked request from a resolution limit. The refinement is reported to the client as a distinct progress stage, so that the additional waiting time is explained rather than merely experienced.

Refinement removes the failure in which no route is found. The opposite failure — a route that is found but crosses land which the classifier has taken for water — is caught by a second safeguard added after the test of subsection 3.5.6: every route is checked against an independent coastline, and a route that crosses land is searched again with the offending cells blocked or on a finer lattice. It is described in subsection 3.5.7.

2.4.5 Comparison with the shortest-distance route

The comparison required by the assignment is performed on the same cost map as the optimised route, which is what makes it meaningful: both routes are judged by the same classifier, at the same resolution, at the same moment.

The great circle between the endpoints is sampled at 160 points; each sample is located in the lattice and its zone is read off. The comparison then reports the length of both routes, the additional distance accepted by the optimised route, how many samples of the great circle fall on land, and a verdict. Samples are counted both in total and over the interior of the route — excluding the two berths — so that the figure is directly comparable with the corresponding count for the optimised route, whose endpoints are also berths.

A route that crosses land has no meaningful finite cost. Where that is the case the comparison reports the cost as undefined rather than as a large number derived from the impassability constant, and the interface and the PDF report show “not navigable” instead. This was a deliberate correction made during implementation: the first version reported a cost of some hundreds of millions, which is arithmetically consistent but conveys nothing to the operator.

2.4.6 Dynamic programming and the Bellman function

Every search method in the application rests on an evaluation function of the discrete step — the cost (2.10) of one move between neighbouring cells. How the methods turn this step cost into a decision is determined by a second function, built on top of it; designing that function is the main decision in each algorithm. In dynamic programming and the theory of optimal control this function is the Bellman function, or value function [39, 40]; in the genetic algorithm it is the fitness function.

In the dynamic-programming formulation the state of the vessel is a cell v of the lattice and a control is one of the eight moves to a neighbouring cell u. The value V(v) is the smallest total cost with which the destination can be reached from v. By Bellman's principle of optimality, every tail of an optimal route is itself optimal, so V satisfies the Bellman equation (2.12):

$$V(goal) = 0, \quad V(v) = \min_{u \in N(v)} [\, c(v, u) + V(u) \,] \tag{2.12}$$

where N(v) is the set of navigable neighbours of v and c(v, u) is the step cost (2.10). No move from or into a land cell is admitted — its step cost is infinite — so the value of a land cell stays infinite.

The equation is solved by value iteration. V is set to zero in the goal and to infinity everywhere else; each sweep replaces the value of every cell by the right-hand side of (2.12). After k sweeps V(v) equals the cheapest cost over routes of at most k moves, so the iteration stops as soon as a sweep changes nothing, and the values are then exact. A sweep updates all cells at once with array operations: for each of the eight directions the lattice of values is shifted by one cell and added to the lattice of step costs. The route is then read off the value function by the greedy policy (2.13):

$$v_{t+1} = \arg\min_{u \in N(v_t)} [\, c(v_t, u) + V(u) \,], \quad v_0 = start \tag{2.13}$$

Starting from the departure, the vessel always moves to the neighbour through which the remaining cost is smallest; with an exact V every move lowers the value, and the walk ends in the destination. The cost of the route equals V(start) and is the same optimum that A* and Dijkstra find. Unlike them, dynamic programming computes the cost-to-go of every cell, not only of the cells near the optimal route; that is more work for one query, but once V is known the optimal route to the same destination from any other departure is obtained without a new search.

2.4.7 The genetic algorithm and the fitness function

A genetic algorithm [30, 31] imitates natural selection. It keeps a population of candidate solutions, each encoded as a chromosome; evaluates every candidate by a fitness function; and builds the next generation from the fitter candidates by crossover, which combines parts of two parents, and mutation, which changes a child at random. The method is used for path planning on grids with obstacles [35] and for ship weather routing [36].

A chromosome here is a list of up to fourteen intermediate waypoints, each a cell of the lattice. The route it encodes is the poly-line from the departure through the waypoints to the destination; each straight segment is rasterised into a chain of neighbouring cells, so the decoded

route is a path in the same graph that the other methods search. The waypoints are kept ordered along the line from the departure to the destination, which rules out loops. The fitness of a chromosome χ is the cost of the decoded route, formula (2.14):

$$F(\chi) = \Sigma_{(u,\,v)\,\in\,route(\chi)}\; d_{gc}(u, v) \cdot ½ \cdot [\; \tilde{w}(z_u) + \tilde{w}(z_v) \;] \qquad (2.14)$$

where w̃ equals the zone weight w for water and a large finite value W = 200 for land. This is the main design decision of the algorithm. With an infinite weight every route that touches land would have the same fitness, and the population would receive no signal about which of two infeasible routes is closer to being feasible. With a finite W a route that crosses three land cells is worse than one that crosses one, so evolution moves towards navigable routes, while even a single land cell costs more than any navigable detour on the voyages considered. Because F uses the same step cost as the exact methods, the result can be compared directly with the optimum; the relative difference is the optimality gap of the algorithm.

The operators and parameters are listed in table 2.10. The segment repair mutation is specific to route planning: it finds the first stretch of land that a segment crosses and inserts a waypoint at the navigable cell nearest to its middle, so that the route bends around a headland step by step.

Table 2.10 – Operators and parameters of the genetic algorithm

Element	**Choice**
Initial population	80 chromosomes: the direct line and random lists of 1–6 waypoints from an ellipse around the two ports
Selection	tournament of 3: the fittest of three random individuals
Crossover	one-point, along the departure–destination axis, probability 0.9
Mutation	probability 0.35; one of five moves: segment repair, shift, insertion, deletion of a waypoint, moving a waypoint off land
Elitism	the 2 best individuals pass unchanged to the next generation
Termination	300 generations, or 60 generations without improvement
Land weight W	200
Random seed	42

The two evaluation functions are compared in table 2.11.

Table 2.11 – Bellman function and fitness function

Aspect	**Bellman function V(v)**	**Fitness function F(χ)**
Argument	a state: one cell	a whole route
Built from	step cost c(v, u), recursively	step cost with a finite land weight, summed
Treatment of land	excluded: infinite value	penalised: finite weight W
How it is used	greedy policy (2.13)	selection of parents
Result	exact optimum	approximate, depends on the seed
Extension to weather or fuel	only if the cost stays additive per step	any function of the whole route

Dynamic programming is exact but requires the cost to be a sum of independent step costs. The genetic algorithm gives no guarantee, but its fitness can be any function of the whole route — the number of course changes, the fuel for a speed profile, the weather along the track — without any change to the search. The experimental comparison is given in subsections 3.3.5 and 3.3.6.

2.4.8 Conclusions on the chapter

The architecture of the application has been designed as a five-layer monolith with a single-page client and a WebSocket channel for live progress. A relational schema of six relations and one view has been designed and justified, and SQLite has been selected as the storage engine. The classification model has been designed around a twenty-eight-component feature vector derived from a label-free reference index, with three explicit measures against target leakage; the forward pass and the training of the network have been described mathematically, and a rule for removing mislabelled training points has been designed. The cost model, four search methods — A*, Dijkstra, dynamic programming and a genetic algorithm — and the automatic refinement of the lattice have been designed, with the Bellman function and the fitness function as the evaluation functions of the last two, and the admissibility of the A* heuristic has been proved.

3 IMPLEMENTATION AND TESTING OF THE APPLICATION

The application is implemented in Python 3.11. The neural network uses PyTorch [14]; scikit-learn [15] supplies the standardiser, the ball tree and the evaluation metrics; FastAPI and uvicorn serve the HTTP and WebSocket interfaces; ReportLab produces the PDF report; the client is written in plain JavaScript with the Leaflet mapping library. The application comprises about 9,900 lines of Python, including the documentation comments that accompany every module, class and function, about 2,000 lines of client code and a SQL schema of 209 lines. The source code is published in a public Git repository together with installation instructions.

3.1 Implementation of the AIS loading and pre-processing module

3.1.1 Reading the source formats

The loader accepts GeoJSON, CSV and JSON. The corpus supplied for this project is a GeoJSON file of 119 MB, too large to parse comfortably as a single document, so the loader uses a streaming parser that yields one feature at a time; the peak memory is proportional to the number of records retained rather than to the size of the file. Where a feature carries coordinates both in its geometry and in its properties, the geometry is authoritative. CSV input is tolerant of the usual column spellings: lat, latitude, y and their longitude counterparts are all recognised.

3.1.2 Validation and cleaning

Every record passes a validation stage that drops rows with missing or out-of-range coordinates and reports what was dropped and why; on the supplied corpus no record was dropped. The 156,838 valid points then pass the consistency check of subsection 2.3.7, implemented in the module cleaning.py, which removes 5,427 mislabelled points and leaves 151,411. The same module assigns every point to its sea region; the region of every test point is stored with the prepared dataset so that the accuracy can later be reported region by region.

3.1.3 Feature extraction

The extractor is vectorised over the whole frame: the ball tree is queried once for all points, and the neighbourhood statistics are computed as array operations over the resulting

matrix. Preparing the dataset — loading, cleaning, splitting, building the reference index and extracting the twenty-eight features of every point — takes about sixteen seconds.

3.2 Implementation of the neural network for zone classification

3.2.1 The network

The network of subsections 2.3.4–2.3.5 is implemented as two PyTorch modules: a residual block, which applies formulas (2.1)–(2.4), and the classifier, which stacks three blocks and a linear output layer. Softmax (2.5) is applied outside the network, inside the loss function during training and explicitly at inference.

3.2.2 Training

Training ran for 97 epochs in 4.3 minutes on four CPU cores. The lowest validation loss (0.175327) was reached at epoch 82; fifteen epochs without improvement followed, early stopping ended the run, and the weights of the best epoch were restored. The dynamics of the run are shown in figure 3.1.

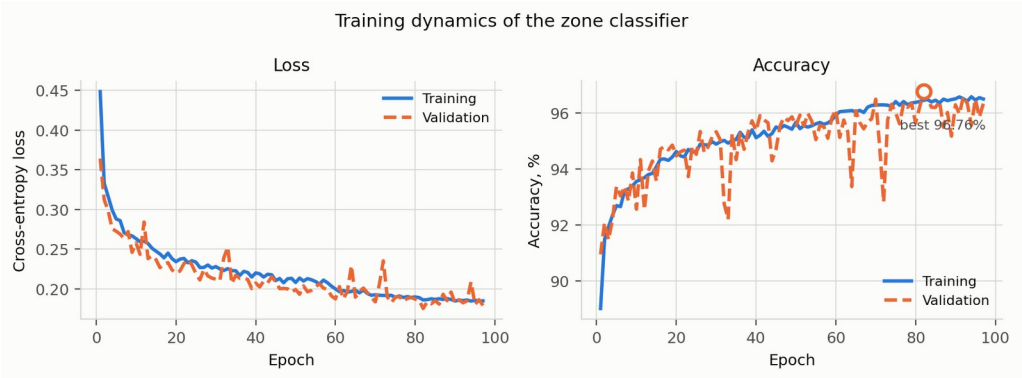


Figure 3.1 – Training dynamics: loss and accuracy on the training and validation splits

The curves show no over-fitting: the validation loss follows the training loss downwards and stays below it in 78 of the 97 epochs — an expected consequence of dropout and batch normalisation, which are active during training and switched off during evaluation. The best validation accuracy was 96.76 %.

3.2.3 Results of the classification

Table 3.1 reports the metrics on the held-out test split, which took no part in training, in validation or in the construction of the reference index.

Table 3.1 – Classification quality on the test set

Metric	**Value**
Number of test points	22,712
Accuracy	96.95 %
Macro F1	0.9695
Weighted F1	0.9695
Cohen’s kappa	0.9582
Macro ROC-AUC (one vs rest)	0.9985
Mean confidence of the predicted class	0.9504

Cohen’s kappa [19] of 0.9582 corrects for the agreement that would arise by chance on an imbalanced corpus; a value above 0.8 is usually read as almost perfect agreement. The macro ROC-AUC [20] of 0.9985 shows that the ranking produced by the network is nearly perfect even where the arg-max decision is wrong. The quality on each class is reported in table 3.2.

Table 3.2 – Per-class quality on the test set

Class	**Precision**	**Recall**	**F1**	**Support**
OPEN_SEA	0.9835	0.9639	0.9736	8,348
COASTAL_SEA	0.9433	0.9721	0.9575	5,548
NEAR_COAST	0.9703	0.9709	0.9706	4,537
COASTLINE	0.9768	0.9755	0.9761	4,279

3.2.4 Analysis of the errors

The distribution of the errors over the classes is shown in figure 3.2.

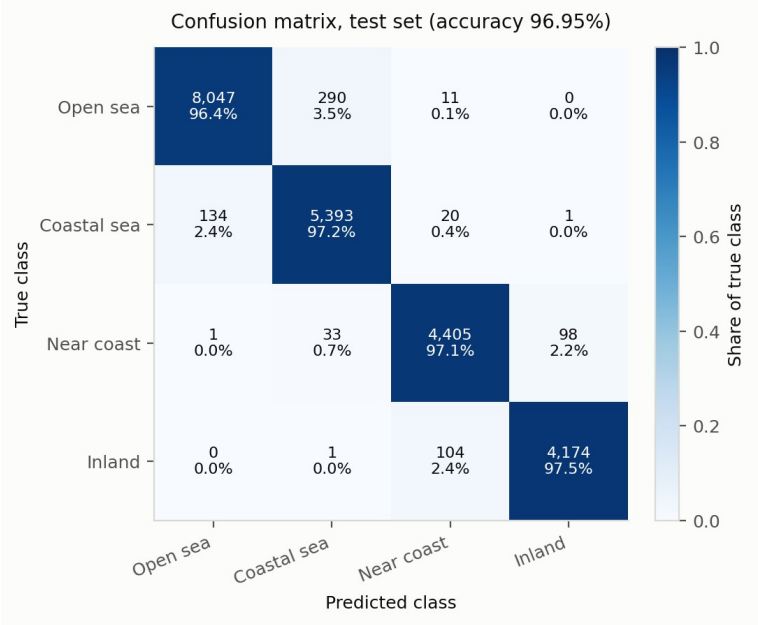


Figure 3.2 – Confusion matrix on the test set

The dominant error is the confusion of OPEN_SEA with COASTAL_SEA: 290 open-sea points were labelled coastal and 134 coastal points were labelled open sea. It is also the least consequential error, because both classes are navigable: it changes the weight of a cell from 1.0 to 1.8 or back, which may bend a route but cannot make it unsafe. The second group lies on the shoreline: 98 near-coast points were labelled inland and 104 inland points near-coast. Both classes are impassable, so this confusion has no effect on the routes.

The errors that matter are those that cross the water–land boundary. There are 32 water points classified as land and 35 land points classified as water, together 0.29 % of the test set. The dangerous direction, land classified as water, occurs in 0.15 % of cases. The safety margin of subsection 2.4.1 mitigates it: an isolated misclassified land cell is surrounded by correctly classified land, and the penalised belt around that land keeps the optimiser away from the hole.

3.2.5 Effect of the data cleaning

The effect of the cleaning of subsection 2.3.7 was measured by comparing the model trained on the uncleaned corpus with the model trained on the cleaned one (table 3.3). The first model was evaluated on all points of its own test split, and separately on the points that the cleaning rule keeps and on those it would remove.

Table 3.3 – Model trained before and after cleaning

Model and test points	Points	Accuracy	Water ↔ land errors
Uncleaned model, whole test split	23,526	96.74 %	189
of these: consistent labels	22,715	97.11 %	79
of these: contradictory labels	811	86.31 %	110
Cleaned model, cleaned test split	22,712	96.95 %	67

The 811 contradictory points, 3.4 % of the old test split, account for 110 of its 189 water–land errors: on these 110 points the model sided with the physical measurement against the wrong label, so the metric was penalising the model for being right. After cleaning, the overall accuracy barely changes, but the number of water–land errors falls to 67, 0.29 % of the test points; these are the errors that decide whether a route is safe.

3.2.6 Accuracy by sea region

A single overall accuracy can hide a region where the model is weak. The test split was therefore also evaluated region by region (table 3.4).

Table 3.4 – Accuracy of the classifier in the eleven sea regions

Region	Test points	Accuracy	Water ↔ land errors
North Europe (Baltic and North Sea)	4,533	97.95 %	5
South China Sea	2,857	93.45 %	5
Japan and North-West Pacific	2,747	99.20 %	1
Southern Africa (Cape of Good Hope)	2,353	94.14 %	3
Black Sea and Eastern Mediterranean	2,072	96.19 %	20
Persian Gulf and Arabian Sea	2,023	97.73 %	4
Red Sea and Gulf of Aden	1,663	99.46 %	4
Central America (Caribbean coast)	1,433	97.35 %	4
Strait of Malacca and Andaman Sea	1,315	98.02 %	3
Aegean Sea	868	97.93 %	7
Bay of Bengal	689	94.63 %	11
Other (inland samples)	159	100.00 %	0
All regions	22,712	96.95 %	67

The accuracy ranges from 93.5 % in the South China Sea, where the two water classes are hardest to tell apart, to 99.5 % in the Red Sea. The largest share of the water–land errors, 20 of 67, falls into the Black Sea and Eastern Mediterranean region, whose steep coasts leave only a narrow band of shallow water. In the Aegean Sea, the region added in the second release of the corpus, the model reaches 97.9 % on 868 test points with 7 water–land errors, although the many islands and narrow channels make it one of the most intricate coastlines of the corpus. Its share of water–land errors, 0.81 %, is nevertheless about 2.7 times the average of the test split and the third highest of all regions, after the Bay of Bengal and the Black Sea and Eastern Mediterranean.

3.3 Implementation of the path search algorithms

3.3.1 The cost map

The cost map is built by classifying the flattened lattice in batches, reshaping the predictions into the lattice, mapping zones to weights and dilating the land mask to produce the safety belt. The batch size is chosen so that a large lattice is classified in at least twenty batches of at least 1 024 cells, which gives the client a steady stream of progress frames. For the Kiel–Tallinn voyage at 0.25° the first lattice has 37 × 76 = 2,812 cells and is built in 0.44 s, almost all of it classification.

3.3.2 A* and Dijkstra

Both algorithms use a binary heap as the priority queue and a closed set to avoid re-expansion. Step lengths are pre-computed once per lattice row and per direction, since the great-circle length of a one-cell step depends on the latitude but not on the longitude; the inner loop of the search therefore contains no trigonometry.

3.3.3 An optimisation of the heuristic

The first implementation evaluated the heuristic by calling the vectorised haversine function with scalar arguments. A* then expanded about 2.5 times fewer nodes than Dijkstra and yet ran about 1.6 times slower: the heuristic is evaluated for every generated node, and the overhead of a NumPy call on scalars exceeded the cost of the expansions it saved. The heuristic was rewritten with the math module and with the per-row trigonometry pre-computed; over four

hundred random cells the two versions differ by at most $9.4 \cdot 10^{-13}$ km. After the change A* became 1.233× faster than Dijkstra on average, as theory predicts.

3.3.4 Results of the routing benchmark

Six voyages in European waters were used as the main benchmark. For each, the cost map is built once and shared by all methods, so that the run times measure the search alone. The results of the exact search are collected in table 3.5.

Table 3.5 – Routing benchmark over six voyages

Voyage	A*, km	Great circle, km	Extra	A* nodes	Dijkstra nodes	A*, s
Kiel–Tallinn	1249	1052	+18.8 %	1,856	4,171	0.012
Helsinki–Copenhagen	1152	883	+30.5 %	991	1,123	0.007
Gdansk–Stockholm	636	555	+14.6 %	210	356	0.003
Rotterdam–Lisbon	2077	1794	+15.8 %	305	779	0.004
Bergen–Reykjavik	1746	1458	+19.8 %	587	920	0.006
Brest–Gibraltar	1423	1364	+4.3 %	61	354	0.003

A* expands on average 2.5 times fewer nodes than Dijkstra and runs 1.233 times faster; the advantage ranges from 1.13× on Helsinki–Copenhagen to 5.8× on Brest–Gibraltar, where the heuristic is most informative. Both algorithms return the identical optimal cost on every voyage (maximum deviation 0 %), which confirms the admissibility proved in subsection 2.4.2. All 6 great-circle routes cross land, while every route produced by the application is navigable, at a cost of 17.29 % additional distance on average. At the resolutions of table 3.5 the longest complete request, including the construction and refinement of the cost map, took 2.9 s.

3.3.5 Dynamic programming

Value iteration (subsection 2.4.6) is implemented without an explicit loop over cells. The step costs (2.10) of the eight directions are computed once as eight arrays of the size of the lattice; one sweep of the Bellman equation (2.12) is then eight array shifts, additions and element-wise minima (listing 3.1).

Listing 3.1 – One sweep of value iteration on the Bellman equation

```
value = np.full((n_rows, n_cols), np.inf)
value[goal] = 0.0
while True:
    best = value.copy()
    for (dr, dc), step in costs.items():      # eight directions
```

```
        np.minimum(best, step + shift(value, dr, dc), out=best)
    best[goal] = 0.0
    if not np.any(best < value - 1e-9):      # nothing improved: V is exact
        break
    value = best
```

The value function of the Odesa–Novorossiysk voyage is shown in figure 3.3. The iso-cost lines run around the Crimean peninsula, and the greedy policy (2.13) follows the steepest descent of V from the departure to the destination.

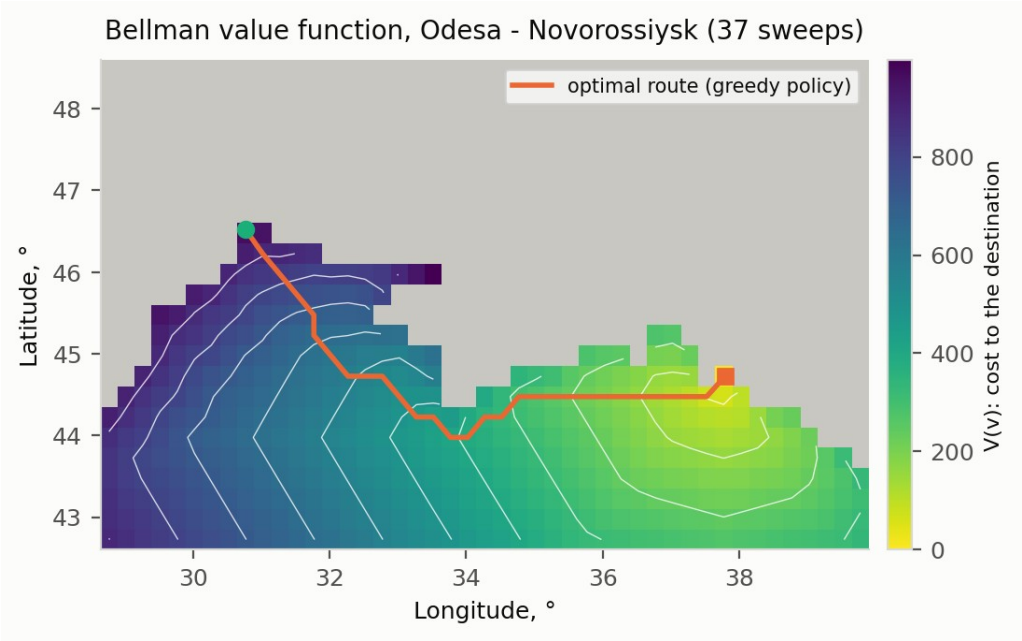


Figure 3.3 – Bellman value function V and the optimal route, Odesa → Novorossiysk

On all seventeen test voyages the cost of the dynamic-programming route equals the A* optimum (maximum deviation 0 %), as it must, since both methods are exact on the same graph. Dynamic programming performs far more elementary updates than A* expands nodes — it computes the value of every reachable cell, including those far from the route — but the updates are array operations, so its run time is of the same order as that of A* (table 3.6).

3.3.6 The genetic algorithm

The genetic algorithm of subsection 2.4.7 is implemented in genetic.py. Segment costs are cached, because the same segment occurs in many individuals and generations, and each segment is rasterised with array operations. The effort and the quality of the three search methods on the benchmark voyages are compared in table 3.6.

Table 3.6 – A*, dynamic programming and the genetic algorithm

Voyage	A*, s	DP, s	DP updates	GA, s	GA evaluations	GA above optimum
Kiel–Tallinn	0.012	0.041	1,178,491	0.41	23,281	4.5 %
Helsinki–Copenhagen	0.007	0.008	74,420	0.14	7,201	3.6 %
Gdansk–Stockholm	0.003	0.004	10,025	0.09	7,281	3.4 %
Rotterdam–Lisbon	0.004	0.006	48,345	0.11	6,721	3.7 %
Bergen–Reykjavik	0.006	0.010	57,240	0.25	11,521	7.9 %
Brest–Gibraltar	0.003	0.004	15,756	0.08	5,681	0.0 %

The genetic algorithm found a navigable route on every benchmark voyage, on average 3.8 % above the optimum, and on the eleven regional voyages of subsection 3.5.5 on average 1.07 % above it. It is 15–45 times slower than A*. Its convergence on the Kiel–Tallinn voyage is shown in figure 3.4 for three random seeds.

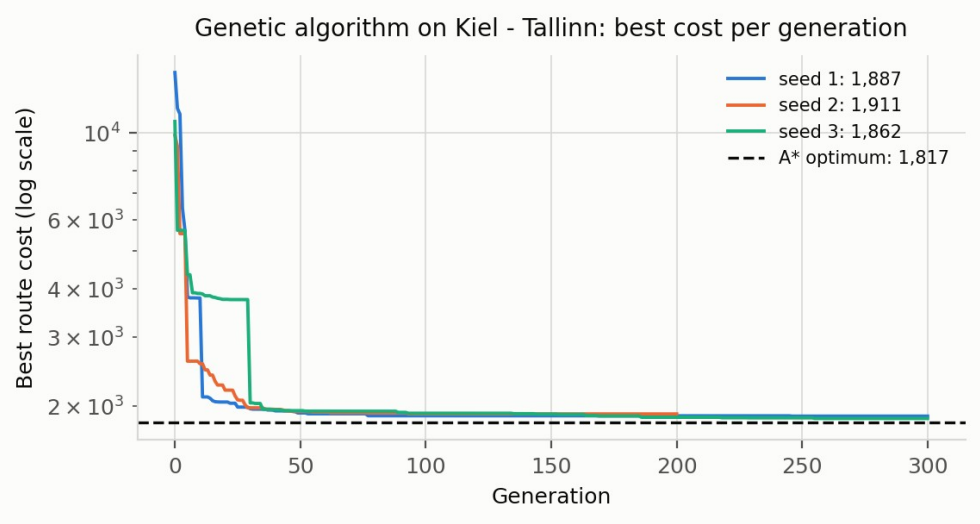


Figure 3.4 – Best route cost per generation of the genetic algorithm, Kiel → Tallinn

The figure shows the typical behaviour of the method. All three runs fall steeply during the first generations, while the population discards routes that pay the land penalty, and then improve slowly; they end 2–5 % above the optimum. The third run spends about thirty generations on a route twice as expensive as the optimum before a mutation of its waypoints opens a better passage: every small change of that route makes it worse, and the large change that is needed is unlikely under mutation. On 2 regional voyages such a trap was not escaped at all. Between Rotterdam and Gdańsk the algorithm converged on the Øresund, which the lattice closes by one cell, while the optimal route passes through the Great Belt; between Piraeus and Thessaloniki, on the 0.1° lattice of the Aegean Sea, no individual found a land-free passage among the islands within the generation limit. In a maze-like sea with narrow passages the fitness

function has local optima that the Bellman function does not have. For the static problem of this project dynamic programming and A* are therefore preferable; the genetic algorithm is retained because its fitness function can include criteria that are not additive per step, as discussed in subsection 2.4.7.

3.4 Implementation of the visualisation module

3.4.1 The client

The client is a single HTML page with three scripts: a transport layer that wraps the REST calls and the WebSocket, a small plotting module for the metric charts, and the controller. Leaflet is stored in the repository rather than loaded from a content delivery network, so that the interface starts with no internet connection. The main window after a route has been built is shown in figure 3.5.

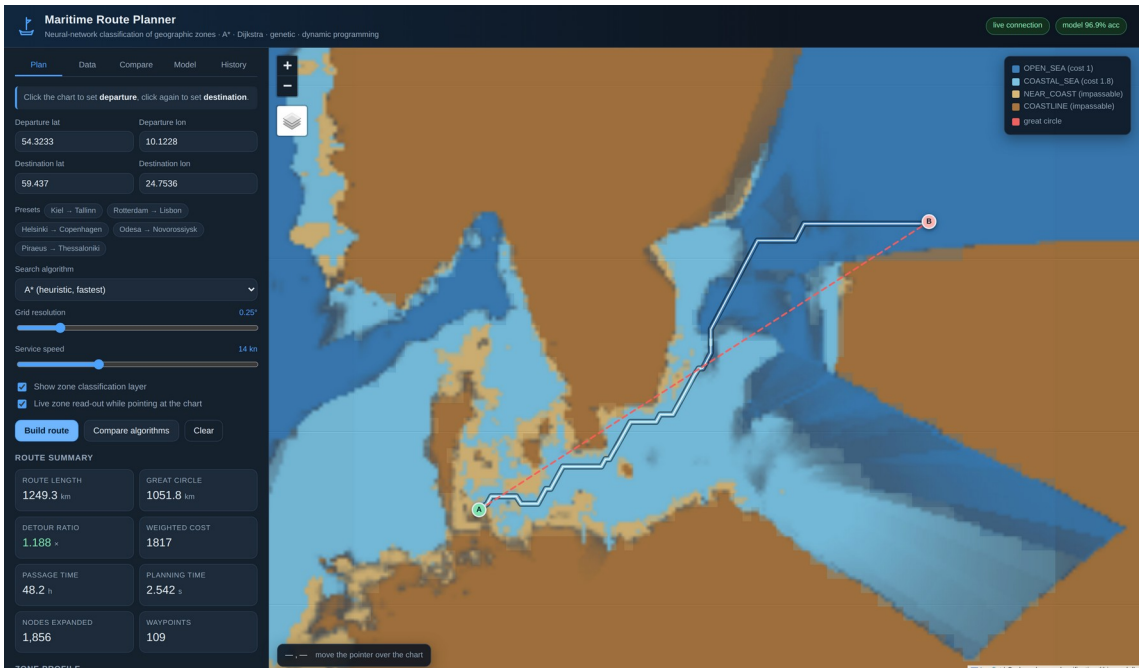


Figure 3.5 – The main window after a route has been built (Kiel → Tallinn)

The four zones are ordered around sea level, so they are encoded by a diverging scheme — water blue against land ochre — rather than by an arbitrary set of hues [27]. The route is drawn segment by segment and coloured by the zone of each segment; the great circle is drawn as a red dashed line. The classification layer is drawn underneath as translucent cells, down-sampled by the server by majority vote to at most six thousand cells.

3.4.2 Real-time interaction

Three mechanisms make the interface interactive in real time.

- Progress streaming. While a job runs, the panel names the current stage and shows a progress bar with the figures behind it — cells classified, nodes expanded by A*, sweeps of value iteration or generations of the genetic algorithm.
- Live zone read-out. As the pointer moves over the chart, its position is sent over the socket, and the zone, the confidence and the depth under the pointer are displayed at the bottom-left corner within a few milliseconds.
- Direct manipulation. The departure and the destination are set by clicking the chart and can be dragged; the coordinate fields and the markers are kept synchronised.

A progress frame captured during the classification of the lattice is shown in figure 3.6.

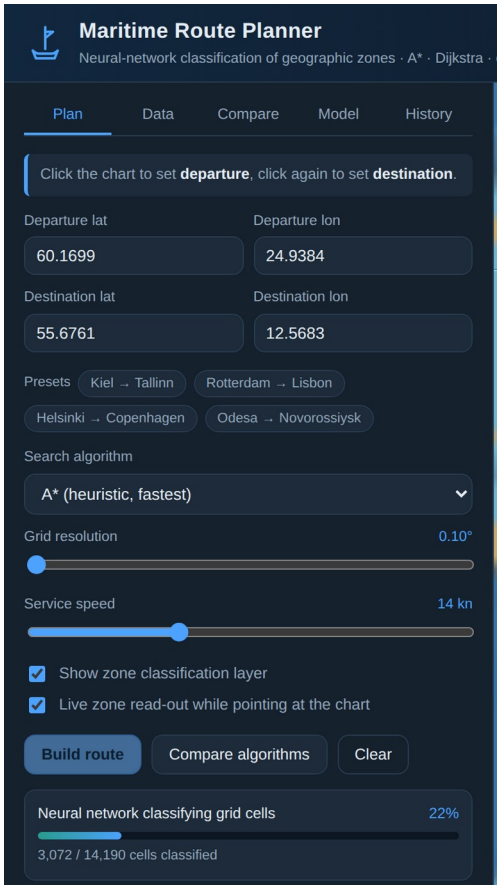


Figure 3.6 – Live progress during the classification of the lattice

3.4.3 The on-board chart

The client requests map tiles from the OpenStreetMap tile service when the machine is online. A vessel at sea frequently is not, so the server also renders its own tiles: each tile is a lattice of coordinates classified by the network and painted with the zone palette, with low-confidence cells blended towards grey. If four tile requests to the public service fail, the client switches to the on-board chart by itself. Every screenshot in this note was taken in this offline mode.

3.4.4 Export

The three export formats required by the assignment are listed in table 3.7.

Table 3.7 – Export formats

Format	**Content**	**Intended use**
GeoJSON [21]	Route line, every waypoint with its zone and cumulative distance, the great-circle reference line, styling properties	opens directly in QGIS, geojson.io and other GIS software
CSV	Waypoint table: sequence, coordinates, zone, weight, cumulative distance, bearing	further analysis in a spreadsheet
PDF	Voyage summary, zone profile, comparison with the shortest route, information on the model, the waypoint list	a document for the voyage file

Non-finite values are converted to null in GeoJSON, because JSON has no representation for infinity and a naive serialiser would emit a document that strict parsers reject.

3.5 Integration of the modules and testing

3.5.1 Integration

The modules are integrated through two façades. The classification service is a thread-safe singleton, so that the reference index — 105,848 samples, the training points that carry a depth or an elevation measurement — is held in memory once, and inference is serialised by a lock. The planner composes the classifier, the cost map and the four search methods and is the only module that knows the order of the stages. Persistence is confined to the repository, the only module that issues SQL; when an older database is opened, the repository upgrades its schema so that routes of the two new methods can be stored.

3.5.2 The test suite

One hundred and one automated tests were written with pytest. They cover every module of the application and run in about 15 seconds. The composition of the suite is given in table 3.8.

Table 3.8 – Composition of the test suite

Test module	**Tests**	**What is verified**
test_geodesy.py	7	great-circle distance against reference values, bearings, interpolation, densification, simplification
test_cleaning.py	7	removal of sea points above and land points below sea level, the tolerance, conflicts revealed by the elevation alone, missing values, the Aegean region
test_data_loading.py	12	column aliases, malformed input, validation, feature matrix, missing values, index round-trip and self-exclusion
test_routing.py	11	lattice, coarsening, agreement of A* and Dijkstra, obstacle avoidance, absence of a path, snapping, cheaper zones, overlay tiling
test_dynamic_programming.py	5	equality with the A* optimum, the Bellman equation in every cell, land excluded, unreachable goal
test_genetic.py	7	eight-connected rasterisation, gap in a wall, never below the optimum, reproducibility, monotone best fitness
test_storage_export.py	14	GeoJSON, CSV and PDF export, database round-trip, deletion, storage of the new methods, upgrade of an old database
test_model_and_api.py	23	probabilities, known locations, determinism, route safety, refinement, every HTTP endpoint, upload, tiles, the WebSocket planning session
test_coast_check.py	15	verdicts of the land mask, the unchecked port stretches, blocking of cells, automatic correction of a route over a missed island by A*, Dijkstra and dynamic programming, the warning of an uncorrectable route, the unchanged benchmark map, storage, upgrade of an old database, the PDF line

Several tests encode properties rather than particular numbers. The equality of the A*, Dijkstra and dynamic-programming costs is asserted directly, so a change that breaks the admissibility of the heuristic fails the suite; the test of dynamic programming checks the Bellman equation (2.12) in every cell; and the WebSocket test drives a complete planning session.

3.5.3 Experimental comparison with the shortest-distance route

Table 3.9 reports the comparison for the Kiel–Tallinn voyage, which is representative of a route through confined waters. The route was requested at the default resolution of 0.25° and planned on the 0.1375° lattice produced by the automatic refinement (subsection 3.5.4).

Table 3.9 – Optimised route against the great circle, Kiel → Tallinn

Criterion	**Great circle**	**Optimised route (A*)**
Length	1,052 km	1,249 km
Additional length	—	+18.8 %
Interior sample points over land	57 of 158	0 of 107
Share of the interior over land	36.1 %	0.0 %
Weighted traversal cost	undefined (crosses land)	1,817
Navigable	no	yes
Estimated passage time at 14 kn	(not sailable)	48.2 h

The corresponding view of the interface, with all five methods compared on one cost map, is shown in figure 3.7.

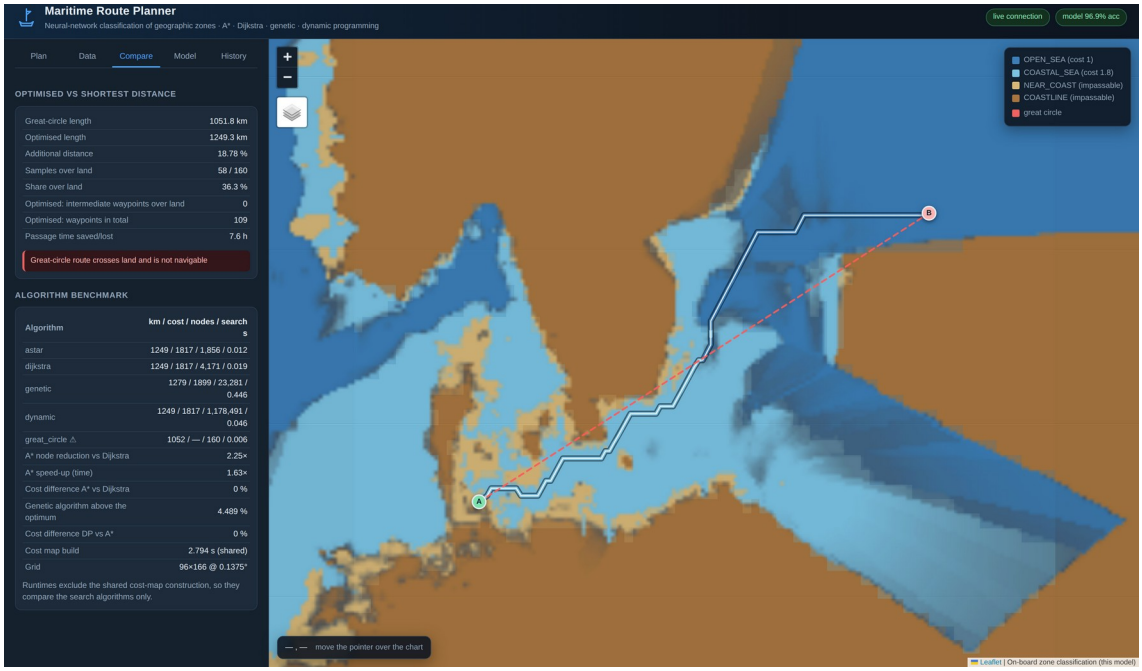


Figure 3.7 – Comparison of the methods on one shared cost map

The great-circle route is 198 km shorter, and it is worthless: 36 % of it runs over Denmark, southern Sweden and Estonia. The additional distance is not an inefficiency of the optimiser but the price of a route that exists, which is why the cost of the great circle is reported as undefined rather than as a number.

3.5.4 Sensitivity to the parameters

Lattice resolution. A coarser lattice is faster but may lose a narrow strait. Requested at 0.5° and at 0.25°, the Kiel–Tallinn voyage finds no path on the first lattice, because the Danish straits are narrower than a cell; the automatic refinement rebuilds the map at 0.275° and at 0.1375° respectively, with a wider margin, and succeeds. The routes returned are 1,226 km and 1,249 km, against 1,280 km when 0.1375° is requested directly with the default margin: a finer lattice follows the shoreline more closely and does not necessarily give a shorter route, because it also resolves more of the coastal cells that the planner avoids.

The coastal weight. Raising the weight of COASTAL_SEA pushes the track away from the shore at the price of length; lowering it gives a shorter but more coastal track. At 1.8 the Kiel–Tallinn route stands off the Swedish coast in the Baltic proper while accepting coastal passage where no alternative exists. The parameter is exposed in the configuration.

3.5.5 Example routes in the eleven sea regions

As recommended by the supervisor, one realistic port-to-port voyage was planned in each of the eleven sea regions of the corpus with all methods of the application. The routes are shown in figure 3.8 and the numbers are given in table 3.10.

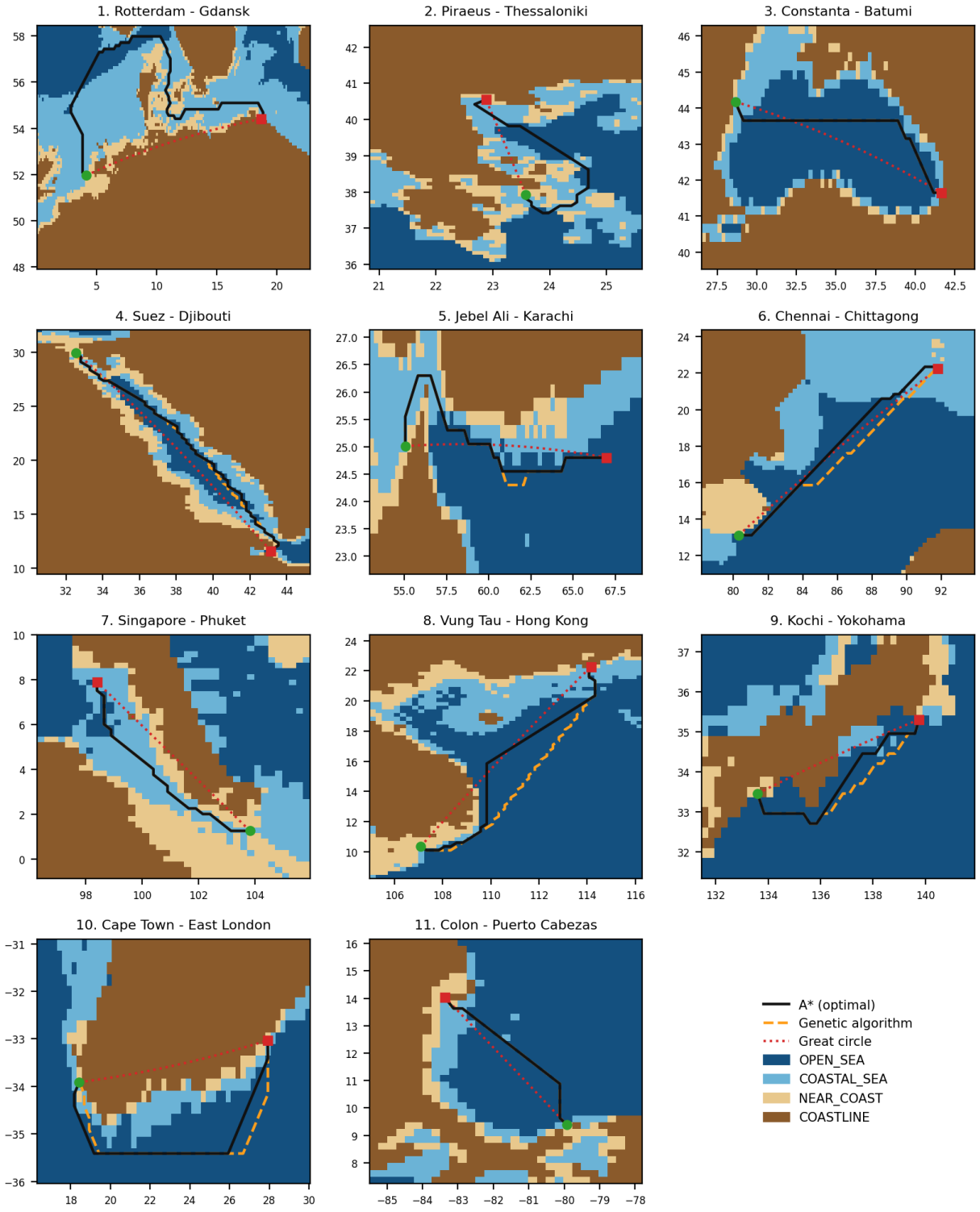


Figure 3.8 – Example routes in the eleven sea regions: A* (solid), genetic algorithm (dashed) and great circle (dotted) over the classified lattice

Table 3.10 – Example voyages in the eleven sea regions

Voyage	A*, km	Great circle, km	Great circle over land	A* = DP	GA above optimum
Rotterdam – Gdansk	2039	1003	96.9 %	yes	failed
Piraeus – Thessaloniki	532	299	33.1 %	yes	failed
Constanta – Batumi	1190	1092	1.3 %	yes	0.00 %
Suez – Djibouti	2510	2313	13.8 %	yes	1.15 %
Jebel Ali – Karachi	1396	1201	10.6 %	yes	1.43 %
Chennai – Chittagong	1659	1586	14.4 %	yes	0.19 %
Singapore – Phuket	1061	950	67.5 %	yes	0.00 %
Vung Tau – Hong Kong	1771	1526	33.1 %	yes	0.19 %
Kochi – Yokohama	763	601	46.9 %	yes	0.18 %
Cape Town – East London	1150	885	97.5 %	yes	6.46 %
Colon – Puerto Cabezas	681	639	5.6 %	yes	0.00 %

A* found a navigable route in all 11 regions, dynamic programming found a route of exactly the same cost in every region, and the genetic algorithm succeeded in 9 of them. All 11 great circles touch land — from 1.3 % of the samples between Constanta and Batumi to 97.5 % between Cape Town and East London, where the great circle runs across the southern tip of Africa. The safe routes are on average 28.28 % longer than the great circles. The choice of the voyages also exposed the limits of the data: ports outside the covered coasts, such as Durban or the Pacific side of Panama, lie in areas that the model classifies as land, so the voyages were chosen within the covered areas.

The Aegean Sea. The new region is one of the most intricate of the corpus, and it showed a weakness of the default lattice. At 0.25° the A* route from Piraeus leaves the Saronic Gulf by a diagonal step into the South Euboean Gulf: both cells are water, but the step passes over Attica, which is narrower than a cell. The waypoints alone do not reveal this; sampling the route every 2 km does — 17 of its 193 samples (8.8 %) lie on land, against at most 1.9 % on every other voyage, where such samples only touch the five-kilometre near-coast zone. The Aegean voyage is therefore planned at 0.1°, and the preset of the client selects this step. The route, 532 km long and 78 % longer than the great circle, then leaves the Saronic Gulf to the south, rounds Cape Sounion, passes the Kafireas Strait between Euboea and Andros and runs north along the outer side of Euboea into the Thermaic Gulf; none of its 246 samples lies on land according to the classifier. The great circle runs over the mainland of Greece for 33 % of its length.

The same voyage also shows a limit of the data. The corpus samples the Aegean rectangle with about one point per eleven kilometres, and on some islands — Kea, Naxos and Skiathos among them — there is no land sample at all, only sea points around them. The

classifier, which describes a point by its sampled neighbourhood, takes such an island for coastal sea, and the route above crosses Kea. Islands of this size become visible to the model only when the corpus contains land points on them.

3.5.6 Testing on 100 random voyages

The voyages above were chosen by hand. To find out whether the application ever lays a course incorrectly, it was also tested on 100 voyages drawn at random with a fixed seed: about nine in each sea region, between water points of the corpus 150–1200 km apart. Each voyage was planned as a user would plan it — A* on the default 0.25° lattice with automatic refinement. The verdict is not taken from the neural network itself: every route is sampled every 1 km and every sample is tested against the 1 km land mask that the package global-land-mask [42] derives from the GLOBE elevation model [41]. The first and the last 5 km are ignored, because a port lies on the coast. A route is correct when no sample falls on land; it grazes the coast when every land run is shorter than 3 km — at the resolution of the mask a shoreline, a narrow channel or an islet touched; it crosses land, and the course is laid incorrectly, when a continuous land run of 3 km or more occurs. The results are summarised in table 3.11; every voyage is listed in table C.1 and drawn in figures C.1–C.11 of appendix C, and the script route_audit.py reproduces them.

Table 3.11 – Results of the test on 100 random voyages

Region	Voyages	Correct	Grazing	Crossing land	No route
North Europe	10	7	1	2	0
Red Sea and Gulf of Aden	9	6	0	3	0
Aegean Sea	9	4	2	3	0
Black Sea and Eastern Mediterranean	9	7	0	0	2
Persian Gulf and Arabian Sea	9	5	0	4	0
Bay of Bengal	9	9	0	0	0
Strait of Malacca and Andaman Sea	9	3	2	4	0
South China Sea	9	8	0	1	0
Japan and North-West Pacific	9	6	0	2	1
Southern Africa	9	9	0	0	0
Central America	9	2	0	0	7
All regions	100	66	5	19	10

The course was correct on 66 voyages and grazed the coast on 5; on 19 voyages it crosses land, and on 10 no route was found. Incorrect courses therefore exist and are not rare. Their

causes were attributed automatically. For a land run, the classifier is asked at the land samples themselves: when it answers “water” there, the classifier has taken land for water; when it answers “land”, the route has stepped diagonally between two water cells across land narrower than a cell. For a voyage without a route, the land mask is searched for a sea connection between the two ports. The voyages with an error were then planned again on a 0.1° lattice (table 3.12).

Table 3.12 – Causes of the errors and the effect of a 0.1° lattice

Error	**Cause**	**Voyages**	**Corrected at 0.1°**
Crossing land	the classifier takes land for water	16	5
Crossing land	diagonal step across land narrower than a cell	3	2
No route	no open sea connection between the ports	7	—
No route	a sea connection exists but the classified lattice closes it	3	0

The classifier is the dominant cause: in 16 of the 19 crossings it takes for water a piece of land that the corpus samples too sparsely — islands such as Masirah off Oman, Singapore, the Danish islands and the islands of the Aegean Sea, the northern tip of Honshu and the Horn of Africa, where voyage 79 crosses 185 km of Somalia. Such places lie at the edges of the sampled areas or between sampled points, and the classifier, which describes a point by its sampled neighbourhood, fills them with water. The lattice is responsible for 3 crossings, among them a cut across the tip of the Musandam peninsula at the Strait of Hormuz. A 0.1° lattice corrects 7 of the 19 crossings, but it cannot correct a wrong classification, and in 3 cases it closes the passage completely. Of the 10 voyages without a route, 7 have no open sea connection: most join the Caribbean coast of Panama with its Pacific coast, which only the Panama Canal connects, so the refusal is correct; the others join the Black Sea with the Mediterranean through the Bosporus, which is narrower than both the land mask and the finest lattice of 0.04°, a limitation of a raster planner. In the remaining 3 a connection exists but the classifier closes it, in the sparsely sampled Gulf of Panama and between the Pacific coast of Honshu and the Sea of Japan. A request took 2.1 s on average and at most 53 s, when the refinement went down to the finest lattice. Examples of incorrect courses are shown in figure 3.9.

The test shows that the safety of the course depends first of all on how densely the corpus samples the coast: where land is not sampled, the classifier fills it with water, and the exact search then uses the gap. Three remedies follow. Land samples should be added exactly where the errors occurred — the test lists them. Every planned route should be checked against an independent coastline in the same way as in this test, and corrected or rejected when it crosses land, which turns the test into a safeguard of the application. In island seas a finer lattice should be used from the start. The second remedy has been built into the application; it is described in the next subsection.

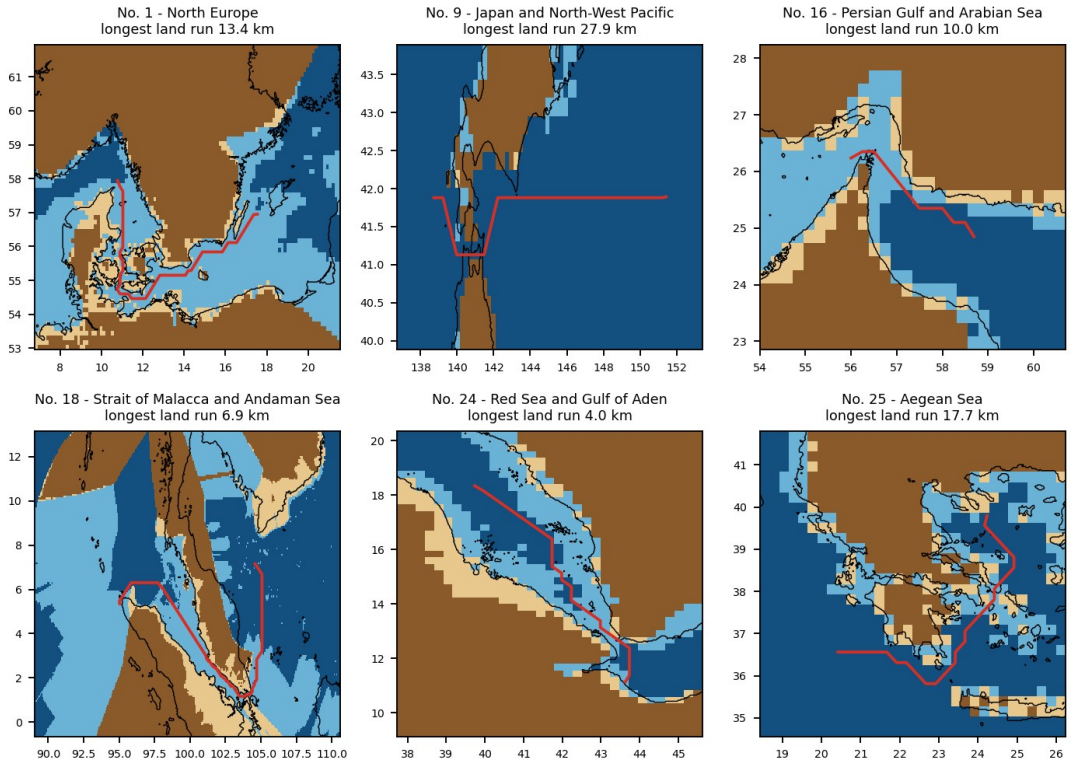


Figure 3.9 – Incorrect courses of the test: classified lattice, coastline of the independent land mask (black line) and the route (red)

3.5.7 Automatic coastline check of the route

Following the second remedy, the check of the test was built into the planner itself, as the module routing/coast_check.py of version 1.2 of the application. After every search the route is sampled every kilometre and each sample is looked up in the same independent land mask [41, 42]; the first and the last 5 km are not checked. A continuous land run of 3 km or more makes the course incorrect, and the planner then corrects it automatically:

1) the cells under the land samples, the route vertices at both ends of the offending legs and every navigable cell within two cells of the run whose centre lies on land of the mask are made impassable on a copy of the cost map, and A* searches again; the round is repeated, up to eight times, until the route no longer crosses land. Water cells are not blocked, so a real strait stays open, and the classification itself is not changed;
2) when the blocked cells close the only passage, or the rounds do not remove the land, the voyage is planned again on a finer lattice — 0.1°, then 0.55 of the step — which is checked in the same way;

3) Dijkstra, dynamic programming or the genetic algorithm, when one of them was chosen, is then run on the corrected map, so that the route shown is that of the selected method; the heuristic genetic algorithm gets up to three more rounds on its own route. A route that still crosses land is returned with a warning, and its land samples are marked on the chart.

The result is shown in a coloured banner above the route summary — passed, corrected automatically, or a warning (appendix A) — and it is written to the PDF report, to the GeoJSON export and to a new column of the route table; an older database receives the column when it is opened. A port given by a city-centre coordinate lies on land, and the straight connector from it to the first navigable cell may cross the shore; land that begins at the quay and ends within that connector is reported as the port approach and is not treated as an error. Without the package global-land-mask the check is skipped and the planner works as before. The check is covered by fifteen automated tests (table 3.8).

The test of subsection 3.5.6 was repeated with the check switched on; table 3.13 compares the two runs. All 19 courses that crossed land were corrected automatically, and none needs the warning. No voyage that was correct before became worse. The corrected routes are on average 62 km longer than the incorrect ones, the price of going round the land. The voyages without a route are unchanged: the check corrects routes that exist, it cannot open a connection that the lattice closes. Every changed voyage is listed in table C.2 of appendix C, and examples are shown in figure 3.10.

Table 3.13 – The test on 100 random voyages without and with the coastline check

Result	Without the check	With the check
Correct course	66	75
Grazing the coast (land runs under 3 km)	5	15
Crossing land	19	0
No route	10	10
Routes corrected automatically	–	19
of them on a finer lattice	–	3
Routes with a warning	–	0
Mean extra distance over the great circle, %	23.0	26.0
Mean request time in the repeated test, s	3.56	3.60

The check costs little time. A route that passes is checked in a few milliseconds; only a corrected one needs additional searches and possibly a new lattice. The repeated test was run on another computer, about 1.6 times slower than the one of subsection 3.5.6, so its times are compared only with each other: the mean request time rose from 3.56 s to 3.60 s, by about 1 %. The check also changed four of the five preset voyages of the user interface. Without it, Helsinki – Copenhagen ran 108 km over the skerries of the Finnish coast, Rotterdam – Lisbon 41 km across Cabo da Roca, Kiel – Tallinn 25 km over the peninsulas of northern Estonia and Piraeus –

Thessaloniki 12 km across the island of Kea; with the check every preset passes, with A*, Dijkstra and dynamic programming alike, in at most 4.8 s on that computer.

One reservation must be stated. After the correction the final route is judged by the same mask that guided the correction, so “correct” in the right-hand column of table 3.13 means agreement with the mask rather than an independent confirmation; the left-hand column remains the independent evaluation of the planner and the classifier. The mask itself has a resolution of about one kilometre, so a channel narrower than that appears as land, and a small islet may be missing. The check therefore does not make the application a navigation system: it removes the gross errors of the classifier, which were the dominant cause of the incorrect courses.

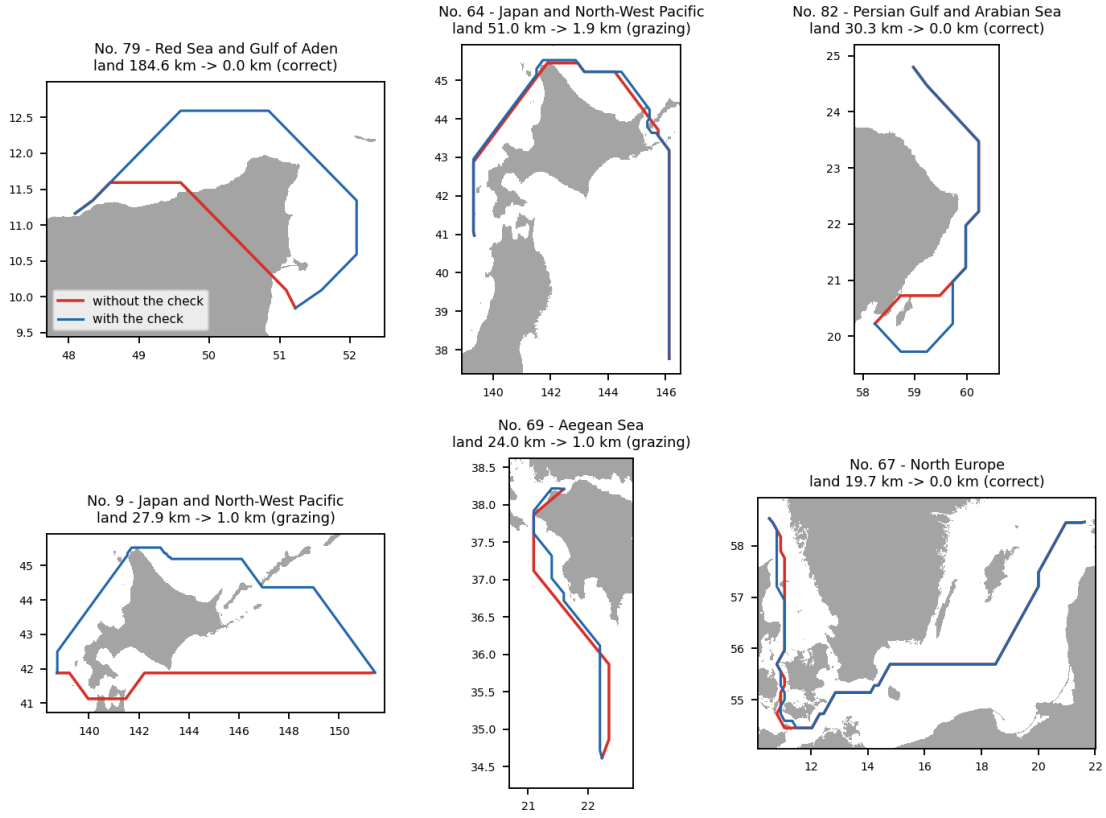


Figure 3.10 – Courses of the test that crossed land (red) and the same voyages with the automatic coastline check (blue), on the land of the independent mask (grey)

3.5.8 Limitations

The following limitations are stated explicitly, because a decision-support tool whose limits are not stated is more dangerous than one that is absent.

- The application is not a certified navigation system and does not use official electronic navigational charts; it must not be the sole basis for navigation.

- The cost model takes no account of weather, currents, tides, ice, traffic separation schemes or the draught of the particular vessel.
- The corpus covers eleven sea regions; outside them the classification is unreliable and may turn open sea into land. Inside them, land that the corpus samples too sparsely — small islands such as Kea or Naxos, narrow peninsulas — is taken for sea; on 19 of 100 random voyages the course crossed land for this reason or because of the lattice (subsection 3.5.6). The automatic coastline check corrects 19 of them (subsection 3.5.7), but it relies on a land mask of about one kilometre, which cannot resolve narrower channels.
- The resolution of the lattice limits the narrowest channel that can be found, and a diagonal step may cut across a peninsula narrower than a cell; automatic refinement and a finer step for island seas mitigate this but cannot eliminate it.
- The genetic algorithm can converge on a strait that the lattice closes and gives no guarantee of optimality.

3.5.9 Conclusions on the chapter

All modules required by the assignment have been implemented and integrated. After the removal of mislabelled points the classifier reaches 96.95 % accuracy on a held-out test set, above the target of ninety per cent, with water–land errors in 0.29 % of points. A*, Dijkstra and dynamic programming return identical optimal costs on all seventeen test voyages; the genetic algorithm finds a navigable route on fifteen of them, a few per cent above the optimum. Every route produced by the exact methods is navigable over its whole interior as judged by the classifier on the lattice, while none of the corresponding great-circle routes is. On 100 random voyages checked against an independent land mask the course was correct or only grazed the coast on 71, crossed land on 19 — mostly where the corpus samples the coast too sparsely — and was not found on 10, 7 of them without an open sea connection. With the automatic coastline check built into the planner, 19 of the 19 incorrect courses are corrected and the course is correct or grazes the coast on 90 voyages. One hundred and one automated tests cover the modules and pass.

4 TECHNICAL AND ECONOMIC JUSTIFICATION

This chapter estimates the cost of developing the application and evaluates the economic effect of putting it into service. All monetary values are given in Belarusian roubles (BYN) at the price level assumed for the 2026/27 academic year. Every rate used below is stated explicitly, because the conclusion of the chapter depends on those rates far more than on the method of calculation.

4.1 Calculation of the development costs

4.1.1 Labour intensity

The calendar plan of the project runs from 1 September 2026 to 31 January 2027, that is five months. With an average monthly working-time fund of 167.3 hours, the total labour intensity is 836.5 hours for one engineer-programmer. Table 4.1 distributes that time over the stages in the proportions given in the assignment.

Table 4.1 – Labour intensity by stage

Stage	**Share**	**Hours**	**Labour cost, BYN**
1 System analysis and statement of the problem	15 %	125.5	3 000.00
2 Design of the application	20 %	167.3	4 000.00
3 Implementation and testing	40 %	334.6	8 000.00
4 Technical and economic justification	5 %	41.8	1 000.00
5 Energy and resource saving	5 %	41.8	1 000.00
6 Preparation of the documentation	10 %	83.7	2 000.00
7 Preparation for the defence	5 %	41.8	1 000.00
Total	100 %	836.5	20 000.00

4.1.2 Cost items

The monthly salary of an engineer-programmer is taken as 4 000.00 BYN, which gives an hourly rate of 23.91 BYN. The remaining items are computed from that base at the rates in force in the Republic of Belarus: additional salary at 15 % of the basic salary, contributions to the

Social Protection Fund at 34 % and compulsory accident insurance at 0.6 % of the total payroll, and overheads at 60 % of the basic salary.

The complete estimate is given in table 4.2.

Table 4.2 – Development cost estimate

No.	Cost item	Basis of calculation	Amount, BYN
1	Basic salary	836.5 h × 23.91 BYN/h	20 000.00
2	Additional salary	15 % of item 1	3 000.00
3	Social Protection Fund contributions	34 % of items 1 + 2	7 820.00
4	Compulsory accident insurance	0.6 % of items 1 + 2	138.00
5	Amortisation of the workstation	3 500 BYN over 5 years, 5 months, 90 % use	262.50
6	Electricity	100.4 kWh × 0.2969 BYN/kWh	29.80
7	Software licences	the entire stack is open source	0.00
8	Materials and consumables	printing, storage media	180.00
9	Overheads	60 % of item 1	12 000.00
10	Other expenses	5 % of items 1–9	2 171.52
–	Total cost of development	–	45 601.82

Item 7 deserves a comment, because it is the largest single saving in the estimate rather than the smallest entry. Python, PyTorch, scikit-learn, FastAPI, SQLite, ReportLab and Leaflet are all distributed under permissive or copyleft open-source licences that permit commercial use without payment. An equivalent development on a proprietary numerical platform, or against a licensed chart-data library, would have added several thousand roubles of annually renewed licence fees to this estimate and to every year of operation thereafter.

Item 5 also reflects a design decision rather than an accident: the application trains and runs on an ordinary processor, so no graphics accelerator had to be bought or rented. This is examined further in chapter 5.

4.2 Evaluation of the economic effectiveness

4.2.1 Sources of the economic effect

The application produces an economic effect from two sources.

1) Avoided licence fees. The application performs, for the class of tasks it covers, the work of a commercial route-planning module. Where such a module would otherwise be licensed per seat, that fee is avoided.

2) Saved planning labour. Constructing and checking a course by hand on a chart takes a watch officer tens of minutes per voyage. The application produces a first approximation within a few seconds, together with a quantitative comparison against the shortest-distance route, leaving the officer to verify and refine rather than to construct.

A saving of fuel is deliberately not claimed. The routes the application produces are on average 17.29 % (benchmark voyages) to 28.28 % (regional voyages) longer than the great circle, and the application models neither weather nor currents nor the hydrodynamics of a particular hull. Claiming a fuel saving on this basis would not be defensible. The value of the application lies in producing a route that can actually be sailed, and in producing it quickly, not in shortening one that already could be.

4.2.2 Scenario analysis

The size of the effect depends almost entirely on assumptions about the scale of deployment and about the price of the software it displaces, neither of which can be known precisely in advance. A single point estimate would therefore convey false precision. Three scenarios are evaluated instead, with the annual cost of maintenance taken as a share of the development cost and the hourly rate of a watch officer taken as 32.87 BYN (5 500 BYN per month).

The assumptions of the three scenarios are collected in table 4.3, and the results they lead to in table 4.4.

Table 4.3 – Assumptions of the three scenarios

Assumption	**Pessimistic**	**Base**	**Optimistic**
Seats replaced	4	12	20
Licence fee avoided per seat, BYN/year	1 200	1 500	1 800
Vessels served	4	8	12
Voyages planned per vessel per year	30	40	45
Planning time saved per voyage, min	15	25	35
Annual maintenance, share of development cost	12 %	10 %	8 %

Table 4.4 – Results of the scenario analysis

Indicator	Pessimistic	Base	Optimistic
Saving of planning labour, BYN/year	986	4 383	10 356
Avoided licence fees, BYN/year	4 800	18 000	36 000
Gross annual effect, BYN	5 786	22 383	46 356
Annual maintenance, BYN	5 472	4 560	3 648
Net annual effect, BYN	314	17 823	42 708
Return on the development cost	0.7 %	39.1 %	93.7 %
Payback period, years	not achieved	2.56	1.07
Net present value over 5 years at 12 %, BYN	−44 470	+18 647	+108 349

4.2.3 Interpretation

The scenarios differ by two orders of magnitude in their outcome, and the reason is instructive. The development cost is fixed and is incurred once; the marginal cost of an additional installation is essentially zero, since the application is a single Python package with no per-seat licence and no server infrastructure. The economics of the project are therefore entirely a question of how many seats it is deployed to.

In the pessimistic scenario — four seats, a small operator, a cheap displaced licence — the net effect barely exceeds the cost of maintaining the application, and the development does not pay for itself within any reasonable horizon. In the base scenario, which corresponds to a training centre together with the operations office of a small fleet, the investment is recovered in about two and a half years and the five-year net present value is positive. In the optimistic scenario the payback falls to just over a year.

The practical conclusion is that the project is economically justified under the base and optimistic scenarios, and that the decisive factor is the breadth of deployment rather than any technical parameter of the application. It follows that effort spent on making the application easy to install and to operate — the single-file database, the absence of a server requirement, the offline chart, the installation guide in appendix B — is not incidental polish but the part of the work with the greatest economic leverage.

Two further sources of value are not quantified above because they cannot be expressed reliably in money, but they should be recorded. The first is safety: a route that has been checked against a depth-aware model before it is sailed reduces the probability of a grounding, whose cost is measured in hundreds of thousands of roubles and potentially in lives. The second is educational: the application, its source code and this explanatory note form a complete worked

example of applying machine learning to a classical optimisation problem, which has value in teaching independently of any operational deployment.

4.2.4 Conclusions on the chapter

The cost of developing the application has been estimated at 45 601.82 BYN, of which 20 000.00 BYN is the basic salary of one engineer-programmer over 836.5 hours. No licence fees were incurred, because the entire technology stack is open source, and no accelerator hardware was required. The economic effect has been evaluated in three scenarios; under the base scenario the net annual effect is 17 823 BYN, the payback period is 2.56 years and the five-year net present value at a discount rate of 12 % is positive.

5 ENERGY AND RESOURCE SAVING

Energy saving is treated here at two levels: the energy consumed by the application itself, which is small but was influenced by several explicit design decisions, and the energy consumed by the vessels whose routes the application plans, which is larger by many orders of magnitude and which the application can influence only indirectly.

5.1 Analysis of energy consumption

5.1.1 Energy consumed in training the model

The classifier was trained on the processor of an ordinary workstation, without a graphics accelerator. Training ran for 97 epochs in 255 s. Taking the power drawn by four loaded processor cores together with the rest of the system as 65 W, the energy consumed by a complete training run is obtained from formula (5.1):

$$E = 65\ W \times 255\ s = 16.6\ kJ \approx 4.6\ W{\cdot}h \tag{5.1}$$

At the Belarusian emission factor for electricity of about 0.40 kg of CO_2 per kilowatt-hour, a training run corresponds to roughly 1.8 g of CO_2 — comparable with boiling a tablespoon of water. The figure is reported not because it is large but because it is the correct order of magnitude to have in mind: for a model of this size, energy is not a constraint, and design decisions should be taken on other grounds.

That conclusion is itself the result of a design decision. A model of 98,364 parameters over 28 tabular features trains in 4.3 minutes on a processor. Had the same task been approached by, for example, feeding raster tiles of bathymetry to a convolutional network, training would have required a graphics accelerator drawing 250–400 W for hours, and the energy of a single run would have risen from watt-hours to kilowatt-hours — a factor of several hundred. The feature-engineering effort described in subsection 2.3.3 is, among other things, an energy saving measure.

5.1.2 Energy consumed in operation

Table 5.1 reports the energy of the operations the application performs, computed from the measured run times at the same 65 W under load and 12 W at idle.

Table 5.1 – Energy consumption of the application

Operation	**Time**	**Energy**	**Remark**
Complete training run	255 s	4.6 W·h	performed once
Genetic-algorithm search of one route	≤ 1 s	≤ 0.018 W·h	search time 15–45 times that of A*
Preparation of the dataset	16 s	0.29 W·h	performed once
Planning one route, typical / longest	0.1 s / 2.9 s	0.002 / 0.053 W·h	about 480,000 typical routes per kW·h
Classifying one point (live read-out)	< 5 ms	< 0.0001 W·h	negligible
Rendering one chart tile	0.35 s	0.006 W·h	cached after the first request
Server at idle	24 h	288 W·h	dominates if the server is left running

The table contains the most important single fact of this chapter: the energy cost of an idle server over one day exceeds the energy cost of ten thousand planned routes. For this application, and for small machine-learning services generally, the useful work is not what consumes the energy; the machine waiting to do it is.

5.1.3 Consumption of other resources

The storage and network resources the application consumes are listed in table 5.2.

Table 5.2 – Consumption of storage and network resources

Resource	**Volume**	**Remark**
Trained model weights	0.4 MB	small enough to ship inside the repository
Reference index	2.0 MB	compressed arrays of the training points
Database after 100 routes	≈ 0.5 MB	single SQLite file
Client payload on first load	≈ 0.35 MB	including the vendored map library
Network traffic per planning request	≈ 0.3 MB	mostly the classification overlay
External network traffic	0 in on-board mode	no tile service, no content delivery network

5.2 Recommendations for reducing energy consumption

5.2.1 Measures already implemented

Six measures that reduce energy or resource consumption were incorporated in the course of the work.

1) A compact model over engineered features rather than a large model over raw rasters. This removes the need for accelerator hardware entirely and reduces the energy of a training run by roughly two orders of magnitude.
2) Early stopping with restoration of the best weights. Training stops when the validation loss ceases to improve, so no energy is spent on epochs that do not improve the model.
3) Optimisation of the A* heuristic. Replacing the vectorised call with pre-computed scalar trigonometry made the search about 2.0 times faster; since the search is the only part of a planning request that scales with the size of the problem, this is a proportional energy saving on every request.
4) Caching of chart tiles. A tile is rendered once and then served from memory, so panning and zooming cost nothing after the first pass.
5) Batched inference. The lattice is classified in batches of 16 384 coordinates rather than one at a time, which amortises the per-call overhead of the framework over thousands of points.
6) An embedded database and vendored client libraries. No database server process runs alongside the application, and no request leaves the machine to fetch a script or a map tile in on-board mode, which removes both the idle power of a second process and the network traffic.

5.2.2 Further recommendations

- Do not leave the server running when it is not in use. Given the figures in table 5.1, this single measure dominates every other software optimisation: starting the application on demand saves more energy than any conceivable improvement to the search.
- Reuse the cost map across requests with the same bounding box and resolution. The construction of the cost map is about ninety-five per cent of the energy of a planning request, and an operator exploring alternative destinations in the same sea area rebuilds it needlessly each time.
- Cache the classification of the lattice on disk between sessions. The classification of a given cell does not change unless the model is retrained, so a persistent cache keyed by the model version would eliminate the dominant cost for repeated areas.
- Retrain only when the corpus has materially changed. Retraining is cheap in absolute terms but it is also almost always unnecessary; the metadata file records the training date and the corpus statistics so that this can be judged rather than guessed.

- Quantise the model to eight-bit integers if it is ever deployed to an embedded bridge computer. For a model of this size the gain on a workstation would not repay the loss of accuracy, but on a low-power device the trade-off reverses.
- Serve the application from a single low-power machine for several workstations rather than installing it on each. The application is stateless between requests apart from its database, so one instance can serve a whole operations office at essentially the idle power of one computer.

5.2.3 Resource saving in the subject area

The energy consumed by the application is negligible beside the energy consumed by the vessels it plans for. A medium-sized cargo vessel burns of the order of twenty to forty tonnes of fuel per day at sea; one day of steaming, about 250–450 MW·h of fuel energy, therefore exceeds a whole year of running this application continuously by two to three orders of magnitude.

It follows that any influence the application has on the route is far more significant, in energy terms, than its own consumption. Two honest observations must be made here. The routes produced are longer than the great circle — by 17 % on the benchmark voyages and 28 % on the regional ones — so the application does not save fuel relative to an idealised straight line. But that straight line does not exist: all seventeen great circles of the experiments cross land. Against the route that would otherwise be sailed, the application avoids corrections at sea, keeps the track out of shallow congested water, and makes the cost of every detour explicit.

The cost model itself is the natural place to extend this. The traversal weight w(z) is currently a risk weight; nothing in the design prevents it from becoming a fuel weight, combining zone risk with a weather and current field and with the speed–consumption curve of the particular vessel. The search, the interface, the export and the database would be unchanged. This is the most promising direction for further work.

5.2.4 Conclusions on the chapter

The energy consumed by the application has been measured and is small: 4.6 W·h for a complete training run and about 0.009 W·h for a typical planned route, so that a kilowatt-hour is sufficient for about 110 000 routes. Six energy-saving measures were implemented in the course of the work, of which the compact model over engineered features is by far the most significant, and six further recommendations have been formulated. The analysis also shows where attention should not be directed: the idle power of the server exceeds the energy of the useful work by a wide margin, and the energy of the vessel exceeds both by several orders of magnitude.

CONCLUSION

In the course of the diploma project an application has been developed for constructing the course of a maritime vessel on the basis of a neural-network classification of geographic zones. All tasks set in the introduction have been completed.

The analysis of the subject area showed that the difficulty of automated maritime route planning lies in the cost model rather than in the search, and that existing open solutions either ignore depth or are confined to a fixed network of legs, while certified systems verify a route rather than propose one. On that basis the problem was stated as the construction of a learned four-zone cost model together with an exact search over it, and nine functional and eight non-functional requirements were formulated.

The application was designed as a five-layer monolith with a single-page web client and a WebSocket channel for live progress. A relational schema of six relations and one view was designed and SQLite was selected and justified as the storage engine. A classification model was designed around a twenty-eight-component feature vector derived from a label-free reference index, with three explicit measures against target leakage, and its forward pass and training were described mathematically. A rule for removing training points whose label contradicts the measured height was designed; it removed 5,427 of 156,838 points. The cost model and four search methods were designed: A* with an admissible heuristic, Dijkstra, dynamic programming built on the Bellman function, and a genetic algorithm built on a fitness function; the two evaluation functions share the same step cost and were compared.

All modules were implemented and integrated, and the following results were obtained:

- the classifier reaches 96.95 % accuracy, a macro F1 of 0.9695, Cohen’s kappa of 0.9582 and a macro ROC-AUC of 0.9985 on a held-out test set of 22,712 points, against a target of ninety per cent;
- the errors of the classifier are concentrated where they are least harmful: most confuse the two navigable classes with each other, and after the cleaning of the data only 67 test points (0.29 %) are confused across the water–land boundary, against 189 before it;
- the accuracy in the eleven sea regions of the corpus ranges from 93.5 % in the South China Sea to 99.5 % in the Red Sea, and reaches 97.9 % in the Aegean Sea added in the second release of the corpus;
- A* expands 2.5 times fewer nodes than Dijkstra and runs 1.233 times faster, while both return the identical optimal cost on every test voyage (maximum deviation 0 %), which confirms the proof of admissibility experimentally; dynamic programming returns the same optimal cost on all seventeen test voyages;

- the genetic algorithm finds a navigable route on fifteen of the seventeen voyages, on average a few per cent above the optimum; on the remaining two it is trapped by a strait that the lattice closes or by the islands of the Aegean Sea, which shows the weakness of evolutionary search in a maze-like sea;
- all seventeen great-circle routes of the experiments cross land and are not navigable, whereas the interior of every route produced by the application lies entirely in navigable water as judged by the classifier on the lattice, at a cost of 17.29 % (benchmark) and 28.28 % (regional voyages) additional distance on average;
- a test on 100 random voyages checked against an independent 1 km land mask found the course correct or only grazing the coast on 71 voyages, crossing land on 19 and not found on 10 (7 of them without an open sea connection); in 16 of the crossings the classifier took for water land that the corpus samples too sparsely. A check of every route against that coastline was therefore built into the planner: it blocks the offending cells and searches again, if necessary on a finer lattice, and corrected 19 of the 19 courses;
- at the default resolution a complete planning request, including the construction and where necessary the refinement of the cost map, takes at most 2.9 s on the benchmark voyages, against a target of five seconds; on the random voyages it takes 2.1 s on average but up to 53 s when the refinement descends to the finest lattice, and the automatic coastline check adds about 1 % to the mean time;
- one hundred and one automated tests cover every module and pass; the source code, with documentation comments for every module, class and function, is published in a public repository.

Three findings of the work are worth stating separately, because they were not anticipated at the design stage. The first is the resolution failure of a raster planner in narrow straits and its remedy by automatic refinement: at a coarse lattice the Danish straits are invisible and the route is reported as impossible, while a finer and wider lattice recovers it. The second is that an asymptotically superior algorithm can be defeated by the constant factor of its own heuristic: A* expanded 2.5 times fewer nodes than Dijkstra and nevertheless ran slower until the heuristic was rewritten without vectorised scalar calls. The third, found only by end-to-end testing, is that storing the reference coordinates as 32-bit floats made a point fail to match itself by half a metre, so that data from the training corpus was classified with a neighbourhood the network had never seen; identifying the query point by distance rather than by a flag removed the defect.

The technical and economic justification estimated the cost of development at 45 601.82 BYN and evaluated the economic effect in three scenarios; under the base scenario the payback period is 2.56 years and the five-year net present value at 12 % is positive. The analysis of energy consumption showed that a training run costs 4.6 W·h and a planned route about 0.01 W·h, so

that the energy of the application is negligible beside the idle power of the machine that hosts it and beside the fuel of the vessel it plans for.

The practical value of the work consists in a complete, documented and tested application that runs offline on an ordinary computer without licence fees, and that can be used as a decision-support tool for voyage planning, as a teaching aid, and as a reference implementation for further research into learned cost models for maritime routing.

The following directions for further work are proposed:

1) attaching a real bathymetric grid (NOAA or GEBCO) [24, 25] and an OpenStreetMap extract [26] in place of the reference index, which would extend the coverage of the model beyond the regions represented in the corpus;
2) extending the traversal weight from a pure risk weight to a fuel weight that combines zone risk with a weather and current field and with the speed–consumption curve of the vessel [28] — the search, the interface, the export and the database would remain unchanged;
3) replacing the uniform lattice with an adaptive one, fine near coasts and coarse in open water, which would remove the resolution compromise entirely instead of mitigating it by refinement;
4) introducing the draught of the particular vessel as an input, so that the boundary between navigable and impassable water becomes vessel-specific rather than universal;
5) calibrating the confidence of the classifier, so that the reported probability can be used as a formal measure of the risk of a cell rather than only as an indication to the operator.

LIST OF ABBREVIATIONS

Abbreviation	Expansion	Abbreviation	Expansion
A*	A-star, an informed graph search algorithm	GIS	Geographic Information System
ACID	Atomicity, Consistency, Isolation, Durability	GPU	Graphics Processing Unit
AdamW	Adaptive Moment Estimation with decoupled Weight decay	HTTP	HyperText Transfer Protocol
AIS	Automatic Identification System	IMO	International Maritime Organization
API	Application Programming Interface	JSON	JavaScript Object Notation
BN	Batch Normalisation	MLP	Multilayer Perceptron
BYN	Belarusian rouble (ISO 4217 currency code)	NOAA	National Oceanic and Atmospheric Administration
CDN	Content Delivery Network	NPV	Net Present Value
CPU	Central Processing Unit	OSM	OpenStreetMap
CRS	Coordinate Reference System	PDF	Portable Document Format
CSV	Comma-Separated Values	PNG	Portable Network Graphics
CVD	Colour Vision Deficiency	REST	Representational State Transfer
DP	Dynamic Programming	ROC-AUC	Area Under the Receiver Operating Characteristic curve
ECDIS	Electronic Chart Display and Information System	ROI	Return On Investment
ENC	Electronic Navigational Chart	SOLAS	International Convention for the Safety of Life at Sea
ESPD	Unified System of Program Documentation	SQL	Structured Query Language
GA	Genetic Algorithm	UI	User Interface
GEBCO	General Bathymetric Chart of the Oceans	UTC	Coordinated Universal Time
GELU	Gaussian Error Linear Unit	XML	Extensible Markup Language
GeoJSON	Geographic JavaScript Object Notation	–	–

LIST OF REFERENCES

1. Guidelines for Voyage Planning : Resolution A.893(21) : adopted on 25 November 1999 / International Maritime Organization. — London : IMO, 2000. — (A 21/Res.893). — URL: https://wwwcdn.imo.org/localresources/en/KnowledgeCentre/IndexofIMOResolutions/AssemblyDocuments/A.893(21).pdf (date of access: 24.09.2026).
2. SOLAS : Consolidated Edition 2024 : International Convention for the Safety of Life at Sea, 1974, its Protocol of 1988, and amendments in force as of 1 July 2024. — London : International Maritime Organization, 2024. — 604 p. — ISBN 978-92-801-1793-6.
3. Hart, P. E. A Formal Basis for the Heuristic Determination of Minimum Cost Paths / P. E. Hart, N. J. Nilsson, B. Raphael // IEEE Transactions on Systems Science and Cybernetics. — 1968. — Vol. 4, No. 2. — P. 100–107. — DOI: 10.1109/TSSC.1968.300136.
4. Dijkstra, E. W. A note on two problems in connexion with graphs / E. W. Dijkstra // Numerische Mathematik. — 1959. — Vol. 1, No. 1. — P. 269–271. — DOI: 10.1007/BF01386390.
5. Russell, S. Artificial Intelligence: A Modern Approach / S. Russell, P. Norvig. — 4th ed. — Hoboken : Pearson, 2021. — 1136 p. — ISBN 978-0-13-461099-3.
6. Introduction to Algorithms / T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. — 4th ed. — Cambridge, MA : The MIT Press, 2022. — 1312 p. — ISBN 978-0-262-04630-5.
7. Goodfellow, I. Deep Learning / I. Goodfellow, Y. Bengio, A. Courville. — Cambridge, MA : The MIT Press, 2016. — 800 p. — ISBN 978-0-262-03561-3. — URL: https://www.deeplearningbook.org (date of access: 24.09.2026).
8. Deep Residual Learning for Image Recognition / K. He, X. Zhang, S. Ren, J. Sun // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). — Las Vegas, 2016. — P. 770–778. — DOI: 10.1109/CVPR.2016.90.
9. Ioffe, S. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift / S. Ioffe, C. Szegedy // Proceedings of the 32nd International Conference on Machine Learning. — PMLR, 2015. — (Proceedings of Machine Learning Research ; Vol. 37). — P. 448–456. — URL: https://proceedings.mlr.press/v37/ioffe15.html (date of access: 24.09.2026).
10. Hendrycks, D. Gaussian Error Linear Units (GELUs) / D. Hendrycks, K. Gimpel // arXiv. — 2016. — arXiv:1606.08415. — URL: https://arxiv.org/abs/1606.08415 (date of access: 24.09.2026).
11. Loshchilov, I. Decoupled Weight Decay Regularization / I. Loshchilov, F. Hutter // 7th International Conference on Learning Representations (ICLR 2019). — New Orleans, 2019. — URL: https://arxiv.org/abs/1711.05101 (date of access: 24.09.2026).

12. Dropout: A Simple Way to Prevent Neural Networks from Overfitting / N. Srivastava, G. Hinton, A. Krizhevsky [et al.] // Journal of Machine Learning Research. — 2014. — Vol. 15, No. 56. — P. 1929–1958. — URL: https://jmlr.org/papers/v15/srivastava14a.html (date of access: 24.09.2026).
13. Rethinking the Inception Architecture for Computer Vision / C. Szegedy, V. Vanhoucke, S. Ioffe [et al.] // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). — Las Vegas, 2016. — P. 2818–2826. — DOI: 10.1109/CVPR.2016.308.
14. PyTorch: An Imperative Style, High-Performance Deep Learning Library / A. Paszke, S. Gross, F. Massa [et al.] // Advances in Neural Information Processing Systems 32 (NeurIPS 2019). — Red Hook : Curran Associates, 2019. — P. 8024–8035. — URL: https://papers.nips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html (date of access: 24.09.2026).
15. Scikit-learn: Machine Learning in Python / F. Pedregosa, G. Varoquaux, A. Gramfort [et al.] // Journal of Machine Learning Research. — 2011. — Vol. 12, No. 85. — P. 2825–2830. — URL: https://jmlr.org/papers/v12/pedregosa11a.html (date of access: 24.09.2026).
16. Omohundro, S. M. Five Balltree Construction Algorithms : Technical Report TR-89-063 / S. M. Omohundro. — Berkeley : International Computer Science Institute, 1989. — 22 p.
17. Sinnott, R. W. Virtues of the Haversine / R. W. Sinnott // Sky and Telescope. — 1984. — Vol. 68, No. 2. — P. 158–159.
18. Douglas, D. H. Algorithms for the Reduction of the Number of Points Required to Represent a Digitized Line or its Caricature / D. H. Douglas, T. K. Peucker // The Canadian Cartographer. — 1973. — Vol. 10, No. 2. — P. 112–122. — DOI: 10.3138/FM57-6770-U75U-7727.
19. Cohen, J. A Coefficient of Agreement for Nominal Scales / J. Cohen // Educational and Psychological Measurement. — 1960. — Vol. 20, No. 1. — P. 37–46. — DOI: 10.1177/001316446002000104.
20. Fawcett, T. An Introduction to ROC Analysis / T. Fawcett // Pattern Recognition Letters. — 2006. — Vol. 27, No. 8. — P. 861–874. — DOI: 10.1016/j.patrec.2005.10.010.
21. The GeoJSON Format : RFC 7946 / H. Butler, M. Daly, A. Doyle [et al.]. — Internet Engineering Task Force, 2016. — 28 p. — DOI: 10.17487/RFC7946. — URL: https://www.rfc-editor.org/rfc/rfc7946.html (date of access: 24.09.2026).
22. Fette, I. The WebSocket Protocol : RFC 6455 / I. Fette, A. Melnikov. — Internet Engineering Task Force, 2011. — 71 p. — DOI: 10.17487/RFC6455. — URL: https://www.rfc-editor.org/rfc/rfc6455.html (date of access: 24.09.2026).
23. Fielding, R. T. Architectural Styles and the Design of Network-based Software Architectures : Doctoral dissertation / R. T. Fielding ; University of California, Irvine. —

Irvine, 2000. — URL: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm (date of access: 24.09.2026).
24. GEBCO Compilation Group. GEBCO 2024 Grid [Electronic resource]. — 2024. — DOI: 10.5285/1c44ce99-0a0d-5f4f-e063-7086abc0ea0f. — URL: https://www.gebco.net/data-products-gridded-bathymetry-data/gebco2024-grid (date of access: 24.09.2026).
25. NOAA National Centers for Environmental Information. ETOPO 2022 15 Arc-Second Global Relief Model [Electronic resource]. — 2022. — DOI: 10.25921/fd45-gt74. — URL: https://www.ncei.noaa.gov/products/etopo-global-relief-model (date of access: 24.09.2026).
26. OpenStreetMap contributors. Planet dump [Electronic resource]. — Data licensed under ODbL. — URL: https://planet.openstreetmap.org ; https://www.openstreetmap.org/copyright (date of access: 24.09.2026).
27. Harrower, M. ColorBrewer.org: An Online Tool for Selecting Colour Schemes for Maps / M. Harrower, C. A. Brewer // The Cartographic Journal. — 2003. — Vol. 40, No. 1. — P. 27–37. — DOI: 10.1179/000870403235002042.
28. Maritime routing and speed optimization with emission control areas / K. Fagerholt, N. T. Gausel, J. G. Rakke, H. N. Psaraftis // Transportation Research Part C: Emerging Technologies. — 2015. — Vol. 52. — P. 57–73. — DOI: 10.1016/j.trc.2014.12.010.
29. Kingma, D. P. Adam: A Method for Stochastic Optimization / D. P. Kingma, J. Ba // 3rd International Conference on Learning Representations (ICLR 2015). — San Diego, 2015. — arXiv:1412.6980. — URL: https://arxiv.org/abs/1412.6980 (date of access: 24.09.2026).
30. Holland, J. H. Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence / J. H. Holland. — Ann Arbor : University of Michigan Press, 1975. — viii, 183 p. — ISBN 0-472-08460-7.
31. Goldberg, D. E. Genetic Algorithms in Search, Optimization, and Machine Learning / D. E. Goldberg. — Reading, MA : Addison-Wesley, 1989. — xiii, 412 p. — ISBN 0-201-15767-5.
32. Frénay, B. Classification in the Presence of Label Noise: A Survey / B. Frénay, M. Verleysen // IEEE Transactions on Neural Networks and Learning Systems. — 2014. — Vol. 25, No. 5. — P. 845–869. — DOI: 10.1109/TNNLS.2013.2292894.
33. Rumelhart, D. E. Learning representations by back-propagating errors / D. E. Rumelhart, G. E. Hinton, R. J. Williams // Nature. — 1986. — Vol. 323, No. 6088. — P. 533–536. — DOI: 10.1038/323533a0.
34. Hornik, K. Multilayer feedforward networks are universal approximators / K. Hornik, M. Stinchcombe, H. White // Neural Networks. — 1989. — Vol. 2, No. 5. — P. 359–366. — DOI: 10.1016/0893-6080(89)90020-8.

35. Lamini, C. Genetic Algorithm Based Approach for Autonomous Mobile Robot Path Planning / C. Lamini, S. Benhlima, A. Elbekri // Procedia Computer Science. — 2018. — Vol. 127. — P. 180–189. — DOI: 10.1016/j.procs.2018.01.113.
36. A new weather-routing system that accounts for ship stability based on a real-coded genetic algorithm / A. Maki, Y. Akimoto, Y. Nagata [et al.] // Journal of Marine Science and Technology. — 2011. — Vol. 16, No. 3. — P. 311–322. — DOI: 10.1007/s00773-011-0128-z.
37. Prechelt, L. Early Stopping — But When? / L. Prechelt // Neural Networks: Tricks of the Trade / ed. G. B. Orr, K.-R. Müller. — Berlin ; Heidelberg : Springer, 1998. — (Lecture Notes in Computer Science ; Vol. 1524). — P. 55–69. — DOI: 10.1007/3-540-49430-8_3.
38. Müller, R. When does label smoothing help? / R. Müller, S. Kornblith, G. E. Hinton // Advances in Neural Information Processing Systems 32 (NeurIPS 2019). — 2019. — P. 4694–4703. — URL: https://papers.nips.cc/paper/2019/hash/f1748d6b0fd9d439f71450117eba2725-Abstract.html (date of access: 24.09.2026).
39. Bellman, R. Dynamic Programming / R. Bellman. — Mineola, NY : Dover Publications, 2003. — xxv, 340 p. — Reprint of the edition: Princeton : Princeton University Press, 1957.
40. Sutton, R. S. Reinforcement Learning: An Introduction / R. S. Sutton, A. G. Barto. — 2nd ed. — Cambridge, MA : The MIT Press, 2018. — 552 p. — ISBN 978-0-262-03924-6. — URL: http://incompleteideas.net/book/the-book-2nd.html (date of access: 25.09.2026).
41. Global Land One-kilometer Base Elevation (GLOBE) : Digital Elevation Model, Documentation, Version 1.0 / ed. by D. A. Hastings, P. K. Dunbar ; National Geophysical Data Center. — Boulder : NOAA, 1999. — (Key to Geophysical Records Documentation ; No. 34). — URL: https://repository.library.noaa.gov/view/noaa/13424 (date of access: 13.10.2026).
42. Karin, T. Global Land Mask : software, version 1.0.0 / T. Karin. — 2020. — DOI: 10.5281/zenodo.4066722. — URL: https://github.com/toddkarin/global-land-mask (date of access: 13.10.2026).

Appendix A. User guide

A.1 Starting the application

Run python scripts/run_server.py and open <http://127.0.0.1:8000>.

The header shows two indicators:

- live connection — the real-time channel to the server is open. While it reads reconnecting… the application still works through ordinary HTTP requests, only without live progress.
- model 96.9% acc — the loaded classifier and its test-set accuracy.

The window is split into the control panel (left) and the chart (right). The panel has five tabs: Plan, Data, Compare, Model, History.

A.2 Planning a route

A.2.1 Setting departure and destination

Three equivalent ways:

- Click the chart. The first click sets the departure (marker A), the second the destination (marker B). The hint above the coordinate fields says which one the next click will set.
- Drag the markers. Both markers can be dragged; the coordinate fields follow.
- Type the coordinates into the four fields, or press one of the preset chips (Kiel → Tallinn, Rotterdam → Lisbon, Helsinki → Copenhagen, Odesa → Novorossiysk, Piraeus → Thessaloniki). The Piraeus → Thessaloniki preset also sets the grid resolution to 0.1°, which the island-dense Aegean Sea needs; the other presets restore the default of 0.25°.

A.2.2 Choosing the parameters

The data are given in table A.1.

Table A.1 – Choosing the parameters

Control	**Meaning**
Search algorithm	A* — heuristic search, the default; Dijkstra — uniform cost, same result, more nodes; Genetic algorithm — evolutionary search: a population of

Control	Meaning
	candidate routes is improved by selection, crossover and mutation under a fitness function; it gives no guarantee of optimality and is shown for comparison; Dynamic programming — solves the Bellman equation for the cost to the destination of every cell (value iteration) and follows it from the departure; exact, same result as A*; Great circle — the naive shortest-distance reference
Grid resolution	lattice step in degrees. Finer resolution follows narrow straits more precisely but costs more time.
Service speed	used only to convert the route length into a passage time
Show zone classification layer	overlays the classified cells on the chart
Live zone read-out	classifies the point under the pointer continuously and shows the zone, confidence and depth at the bottom-left of the chart

A.2.3 Building the route

Press Build route. A progress block reports each stage:

1) Building geographic lattice
2) Neural network classifying grid cells — with the number of cells done
3) Cost map ready — with the share of each zone
4) A* search in progress — with nodes expanded and frontier size (for Dijkstra the same; for the genetic algorithm Genetic algorithm: evolving routes with the generation number and the best route cost so far; for dynamic programming Dynamic programming: solving the Bellman equation with the sweep number and the number of values improved in it)
5) Coastline check: correcting the route — only when the first route crosses land on the independent coastline (subsection A.2.5)
6) Route ready

If no route is found — because a strait is narrower than one lattice cell, or because the only passage lies outside the rectangle around the two ports — the message Refining the lattice appears and the application retries on a finer and wider lattice (up to three times). A departure or destination is moved onto the nearest navigable cell only if that cell is within 60 km; a point further inland is reported as lying on land.

The genetic algorithm may fail to find a navigable route where A* succeeds (it can converge on a strait that the lattice closes). In that case the lattice is not refined, because A* has proved that a route exists; choose A* instead.

A.2.4 Reading the result

The chart shows:

- the optimised route, coloured per zone along its length;
- the great-circle route as a red dashed line;
- the zone classification layer, if enabled.

The Route summary panel reports the route length, the great-circle distance, the detour ratio, the weighted traversal cost, the estimated passage time, the planning time, the number of expanded nodes and the number of waypoints. Below it, the Zone profile bar shows how much of the route lies in each zone.

A.2.5 Coastline check

Every route is checked automatically against an independent coastline — the 1 km GLOBE land mask of the package global-land-mask, which plays no part in the classification. The route is sampled every kilometre; the first and the last 5 km (the quays) are not checked. A banner above the route summary shows the outcome:

- green, Coastline check passed — no land on the route, or it touches the coast for less than 3 km;
- amber, the first route crossed land … and was corrected automatically — the neural network had taken some land for water; the cells under that land were blocked and the route searched again, if necessary on a finer lattice; the route shown passes the check (figure A.2);
- red, The route crosses land … — the automatic correction could not remove the land; the places are marked on the chart with red circles. Such a course must not be followed: try a finer grid resolution or check the passage on an official chart.

A port given by a city-centre coordinate lies on land, and the straight connector from it to the open water may cross the shore: land touching the quay is counted as the port approach and is not an error. The result of the check is also written to the PDF report (line Coastline check), to the GeoJSON export and to the database. Without the package global-land-mask, installed with requirements.txt, the banner is not shown and the application works as before.

A.3 Comparing with the shortest route

The Compare tab is filled automatically after every route. It reports the length of both routes, the additional distance accepted by the optimised route, how many sample points of the great circle fall on land, and a verdict: a great-circle route that crosses land is not navigable at all, so the extra distance is the price of a route that can actually be sailed.

Press Compare algorithms to run A*, Dijkstra, the genetic algorithm, dynamic programming and the great circle on one shared cost map. The table reports route length, cost, expanded nodes (for the genetic algorithm: the number of routes evaluated) and search time for each, the node reduction of A* against Dijkstra, and how far the genetic algorithm lies above the optimum found by A*, and the cost difference between dynamic programming and A* (zero, since both are exact). The reported times exclude the shared cost-map construction, so they compare the search methods only.

A.4 Working with AIS data

Open the Data tab and drop a .geojson, .csv or .json file onto the drop zone (or click it to browse).

- CSV files must contain latitude and longitude columns; the usual spellings (lat, lon, lng, y, x, …) are recognised automatically.
- GeoJSON files must be a FeatureCollection of points.

The application validates the file, classifies the points and stores them in the database. The report shows how many features were read, how many were valid, how many were dropped and how the points are distributed over the four zones. If the file carries class_code labels, the agreement between the model and those labels is reported as well.

The classified points are drawn on the chart and can be downloaded as Points GeoJSON or Points CSV.

The Database block at the bottom of the tab shows how many sessions, points, routes and waypoints are stored and how large the database file is.

A.5 Exporting the results

After a route has been built, three buttons appear under the route summary:

The data are given in table A.2.

Table A.2 – Exporting the results

Button	**File**	**Content**
GeoJSON	route_<id>.geojson	route line, every waypoint with its zone, and the great-circle reference line
CSV	route_<id>.csv	waypoint table: sequence, coordinates, zone, weight, cumulative distance, bearing
PDF report	route_<id>.pdf	voyage summary (with the coastline check and its warning, if any), zone profile, comparison with the shortest route, model information and the waypoint list

The GeoJSON file carries styling properties (stroke, marker-color) and opens directly in QGIS, geojson.io and similar tools.

A.6 Inspecting the model

The Model tab reports the architecture, the number of input features and parameters, the size of the reference index, and the full test-set metrics: accuracy, macro and weighted F1, Cohen's κ and macro ROC-AUC. Below them the confusion matrix is shown cell by cell, together with the training and validation curves of the run that produced the deployed weights.

A.7 Stored routes

The History tab lists the routes saved in the database, newest first, with the algorithm, length, creation time, number of waypoints, search time and detour ratio. Clicking an entry draws that route on the chart again.

A.8 Keyboard and mouse summary

The data are given in table A.3.

Table A.3 – Keyboard and mouse summary

Action	**Result**
Click on the chart	set departure, then destination
Drag marker A / B	move departure / destination

Action	Result
Scroll on the chart	zoom
Move the pointer over the chart	live zone read-out (when enabled)
Layer control, top-left	switch between the online chart and the on-board chart

A.9 Views of the interface

The state of the application after start-up is shown in figure A.1.



Figure A.1 – The application after start-up, on-board chart selected

The banner of the coastline check after a route that was corrected automatically (preset Helsinki → Copenhagen) is shown in figure A.2.

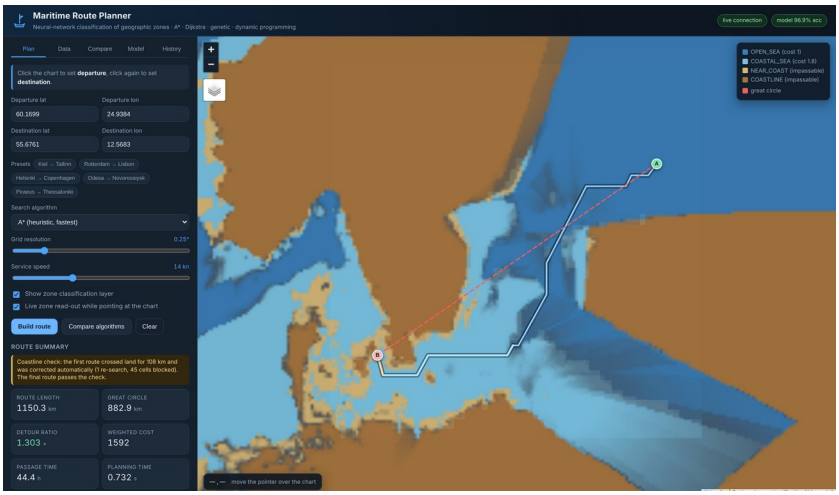


Figure A.2 – A route corrected by the coastline check: the amber banner above the route summary

Appendix B. Installation and configuration guide

B.1 System requirements

The data are given in table B.1.

Table B.1 – System requirements

Item	Minimum	Recommended
Operating system	Windows 10 / Ubuntu 20.04 / macOS 12	any 64-bit, current
Python	3.10	3.11
RAM	4 GB	8 GB
Free disk space	3 GB	5 GB
CPU	2 cores	4 cores or more
GPU	not required	not required (CPU inference is sufficient)
Browser	any with WebSocket support	Chrome / Firefox / Edge, current

Training the model from scratch takes about five minutes on four CPU cores; no GPU is needed at any stage.

B.2 Installation

B.2.1 Obtain the sources

Clone the repository from GitHub (or unpack the delivered archive):

```
git clone https://github.com/EDXiaoM/MaritimeRouteAI.git
cd MaritimeRouteAI
```

On Windows without Git, use Code → Download ZIP on the GitHub page, unpack the archive and open a command prompt in the unpacked folder.

B.2.2 Create an isolated environment

```
python -m venv .venv
```

```
source .venv/bin/activate        # Linux / macOS
.venv\Scripts\activate           # Windows
```

B.2.3 Install the dependencies

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

If pip cannot reach the default PyTorch wheels, install the CPU build explicitly:

```
pip install torch --index-url https://download.pytorch.org/whl/cpu
```

B.2.4 Verify the installation

```
pip install -r requirements-dev.txt
pytest
```

All tests must pass. Tests that need the trained model are skipped automatically when models/zone_classifier.pt is absent.

B.3 Preparing the data and the model

The archive already contains a trained model (models/), so this section is needed only to retrain or to use a different corpus.

The labelled corpus is attached to the GitHub release v1.0 as merged_geojson_v2.zip (https://github.com/EDXiaoM/MaritimeRouteAI/releases/download/v1.0/merged_geojson_v2.zip); unpack it into data/raw/.

```
# 1. put the labelled corpus here:
#    data/raw/merged.geojson
python scripts/prepare_dataset.py           # ~16 s, cleaning + features,
writes data/processed/
python scripts/train_model.py               # ~5 min on CPU, writes models/
python scripts/evaluate_regions.py          # optional: accuracy per sea
region
python scripts/benchmark_routing.py         # optional: 6 European voyages,
all methods
python scripts/regional_examples.py         # optional: 11 regional voyages +
map figure
python scripts/make_figures.py              # optional: regenerate the
figures
```

prepare_dataset.py removes the points whose class contradicts their measured height or depth. Both measurements of a point, noaa_depth and elevation, are checked; at the default tolerance of 5 m (LABEL_CHECK_TOLERANCE_M in

config.py) 5 427 of the 156 838 points are removed and 151 411 remain. Pass --no-clean to keep them.

prepare_dataset.py writes:

The data are given in table B.2.

Table B.2 – Preparing the data and the model

File	**Content**
data/processed/zone_dataset.npz	feature matrices and labels for the three splits
data/processed/dataset_statistics.json	corpus statistics, cleaning report and per-region counts
models/reference_index.npz	label-free bathymetry / OSM index

train_model.py writes:

The data are given in table B.3.

Table B.3 – Preparing the data and the model

File	**Content**
models/zone_classifier.pt	network weights and architecture
models/feature_scaler.joblib	fitted standardiser
models/model_metadata.json	hyper-parameters, metrics and training history

B.4 Running the application

```
python scripts/run_server.py
```

Then open <http://127.0.0.1:8000>.

Options:

```
python scripts/run_server.py --host 0.0.0.0 --port 8080
python scripts/run_server.py --reload          # development: auto-reload
```

B.5 Configuration

All tunable values live in src/maritime_route/config.py. The most useful ones: The data are given in table B.4.

Table B.4 – Configuration

Setting	Default	Meaning
ZONE_COST	1.0 / 1.8 / ∞ / ∞	traversal weight of each zone
ROUTING.grid_resolution_deg	0.25	default lattice step, degrees
ROUTING.max_grid_cells	400 000	lattice budget; larger areas are coarsened
ROUTING.safety_margin_cells	1	width of the penalised belt along the shore
ROUTING.proximity_penalty	1.6	extra cost of a water cell next to land
GENETIC.*	see file	population size, generations, rates and seed of the genetic algorithm
LABEL_CHECK_TOLERANCE_M	5.0	tolerance of the data-cleaning check, metres
REGIONS	eleven boxes	the sea regions used for per-region evaluation
TRAINING.*	see file	network size, epochs, learning rate, seed
SERVER.progress_interval_s	0.15	minimum interval between progress frames

Several settings can also be overridden with environment variables, which is convenient for deployment:

```
export MR_HOST=0.0.0.0
export MR_PORT=8080
export MR_DB_PATH=/var/lib/maritime/routes.db
export MR_MODEL_DIR=/opt/maritime/models
export MR_OUTPUT_DIR=/var/lib/maritime/exports
```

B.6 The database

SQLite is created automatically at data/maritime_route.db on first start; the schema is in src/maritime_route/storage/schema.sql. No database server has to be installed. To reset the stored history, stop the application and delete the

data/maritime_route.db* files.

B.7 Offline operation

The client requests OpenStreetMap tiles when the machine is online. If the tile service is unreachable, after four failed tiles the client switches by itself to the on-board chart — tiles rendered by the server from the model's own classification — and the application stays fully usable. The layer can also be selected manually in the layer control at the top-left corner of the map.

B.8 Troubleshooting

The data are given in table B.5.

Table B.5 – Troubleshooting

Symptom	**Cause and remedy**
503 Model weights not found	the model has not been trained — run scripts/train_model.py
The status pill shows reconnecting…	the server was restarted; the client reconnects by itself with exponential back-off
Address already in use	another process holds port 8000 — start with --port 8081
Planning reports no navigable path	departure or destination is landlocked; the lattice is already refined and widened three times automatically
Planning reports lies on land	the port is more than 60 km from any navigable cell, often because it lies outside the eleven sea regions covered by the training data
The genetic algorithm reports no navigable route evolved	the evolutionary search converged on a closed strait; use A*
A red banner The route crosses land …	the automatic coastline check could not correct the route (the network takes too much land for water there); set the grid resolution to 0.1° and build again, or check the passage on an official chart
No coastline-check banner appears	the package global-land-mask is not installed — pip install -r requirements.txt
The map is blank	no internet access — switch to On-board chart (offline) in the layer control
ModuleNotFoundError: maritime_route	run the scripts from the project root, or set PYTHONPATH=src

Appendix C. Results of the test on 100 random voyages

Table C.1 lists every voyage of the test described in subsection 3.5.6: the two ends (latitude, longitude in degrees), the great-circle distance, the length of the planned route, the verdict of the check against the independent land mask, the longest continuous land run and, for an error, its cause. The same data are written by the script route_audit.py to the files docs/figures/route_audit.csv and route_audit.json. Verdicts: correct — no sample on land; grazing — land runs shorter than 3 km; crosses land — a land run of 3 km or more; no route — the planner found none. Causes: classifier — land taken for water; lattice — a diagonal step across land narrower than a cell; no sea link — no open sea connection between the ports; closed on map — a connection exists but the classified lattice closes it.

Table C.1 – Voyages of the test

No.	Region	From	To	GC, km	Route, km	Verdict	Land run, km	Cause
1	North Europe	56.95, 17.54	57.95, 10.78	420	976	crosses land	13.4	classifier
2	Red Sea, Aden	11.28, 43.55	12.56, 49.17	628	669	correct	0.0	–
3	Aegean Sea	35.45, 28.09	35.42, 22.98	463	508	correct	0.0	–
4	Black Sea, E. Med.	42.66, 38.16	43.43, 33.94	353	385	correct	0.0	–
5	Persian Gulf	25.18, 57.32	16.49, 62.34	1099	1169	correct	0.0	–
6	Bay of Bengal	17.41, 83.00	16.99, 88.28	562	656	correct	0.0	–
7	Malacca, Andaman	2.35, 100.99	4.28, 98.36	363	391	correct	0.0	–
8	South China Sea	7.69, 116.15	8.99, 109.94	698	742	correct	0.0	–
9	Japan, NW Pacific	41.88, 138.72	41.89, 151.43	1051	1137	crosses land	27.9	classifier
10	Southern Africa	-38.44, 17.00	-37.98, 15.34	154	168	correct	0.0	–
11	Central America	10.70, -79.27	8.73, -79.62	222	–	no route	–	no sea link
12	North Europe	54.08, 10.95	50.64, 0.27	818	1745	grazing	2.9	–
13	Red Sea, Aden	25.47, 51.59	27.86, 50.96	273	301	correct	0.0	–
14	Aegean Sea	40.46, 22.81	35.85, 25.56	566	651	grazing	2.0	–
15	Black Sea, E. Med.	42.52, 34.10	43.89, 31.65	251	274	correct	0.0	–
16	Persian Gulf	26.24, 56.00	24.85, 58.69	311	349	crosses land	10.0	lattice
17	Bay of Bengal	17.42, 83.00	20.36, 90.09	814	889	correct	0.0	–
18	Malacca, Andaman	7.15, 104.58	5.35, 95.05	1073	2040	crosses land	6.9	classifier
19	South China Sea	11.60, 115.28	14.32, 111.06	548	587	correct	0.0	–
20	Japan, NW Pacific	41.06, 145.29	37.86, 153.33	776	857	correct	0.0	–
21	Southern Africa	-37.73, 17.84	-34.79, 22.59	537	578	correct	0.0	–

No.	Region	From	To	GC, km	Route, km	Verdict	Land run, km	Cause
22	Central America	8.25, -80.24	6.08, -83.94	474	–	no route	–	closed on map
23	North Europe	54.08, 11.38	55.94, 15.52	335	383	correct	0.0	–
24	Red Sea, Aden	18.34, 39.74	11.13, 43.63	904	978	crosses land	4.0	classifier
25	Aegean Sea	39.79, 24.23	36.56, 20.42	490	791	crosses land	17.7	classifier
26	Black Sea, E. Med.	41.30, 28.97	43.00, 38.52	809	877	correct	0.0	–
27	Persian Gulf	24.61, 58.43	26.99, 53.09	597	696	crosses land	7.0	lattice
28	Bay of Bengal	21.18, 89.99	16.17, 87.66	608	656	correct	0.0	–
29	Malacca, Andaman	6.26, 97.65	8.40, 96.55	268	290	correct	0.0	–
30	South China Sea	17.24, 117.45	15.29, 122.11	542	1138	crosses land	14.6	classifier
31	Japan, NW Pacific	36.39, 142.16	37.09, 137.83	393	–	no route	–	closed on map
32	Southern Africa	-38.50, 15.38	-35.05, 22.37	731	799	correct	0.0	–
33	Central America	8.23, -80.25	10.70, -81.95	333	–	no route	–	no sea link
34	North Europe	54.94, 18.26	54.09, 11.34	457	529	correct	0.0	–
35	Red Sea, Aden	14.77, 41.01	18.31, 39.72	417	446	correct	0.0	–
36	Aegean Sea	34.46, 28.12	35.05, 19.87	756	782	correct	0.0	–
37	Black Sea, E. Med.	32.35, 32.71	42.35, 30.04	1137	–	no route	–	no sea link
38	Persian Gulf	25.12, 58.92	23.23, 61.50	336	349	correct	0.0	–
39	Bay of Bengal	19.59, 89.89	16.66, 90.23	328	341	correct	0.0	–
40	Malacca, Andaman	5.54, 97.67	4.61, 103.96	704	1388	grazing	2.9	–
41	South China Sea	4.48, 106.63	5.11, 105.22	172	198	correct	0.0	–
42	Japan, NW Pacific	43.87, 137.38	35.66, 135.65	925	974	correct	0.0	–
43	Southern Africa	-35.97, 19.71	-36.41, 17.59	196	209	correct	0.0	–
44	Central America	9.35, -80.38	6.08, -83.69	516	–	no route	–	no sea link
45	North Europe	54.94, 18.23	56.14, 16.48	173	192	correct	0.0	–
46	Red Sea, Aden	20.08, 39.34	21.20, 38.08	181	182	correct	0.0	–
47	Aegean Sea	36.93, 23.71	38.25, 25.50	216	259	crosses land	9.7	classifier
48	Black Sea, E. Med.	44.18, 31.63	43.92, 35.94	346	359	correct	0.0	–
49	Persian Gulf	26.19, 54.16	25.02, 60.51	650	740	crosses land	4.0	classifier
50	Bay of Bengal	14.48, 84.89	21.71, 90.57	1003	1048	correct	0.0	–
51	Malacca, Andaman	1.12, 103.60	6.20, 97.55	877	936	crosses land	7.9	classifier

No.	Region	From	To	GC, km	Route, km	Verdict	Land run, km	Cause
52	South China Sea	7.95, 107.39	5.53, 111.80	556	607	correct	0.0	–
53	Japan, NW Pacific	35.93, 146.75	37.94, 144.80	282	283	correct	0.0	–
54	Southern Africa	-36.25, 18.96	-34.47, 18.63	200	211	correct	0.0	–
55	Central America	6.11, -83.66	8.24, -80.28	442	–	no route	–	closed on map
56	North Europe	54.60, 16.49	54.25, 12.11	286	332	correct	0.0	–
57	Red Sea, Aden	18.30, 39.72	12.54, 44.85	845	981	correct	0.0	–
58	Aegean Sea	35.26, 30.49	34.54, 25.37	474	505	correct	0.0	–
59	Black Sea, E. Med.	45.42, 30.50	42.06, 31.55	383	400	correct	0.0	–
60	Persian Gulf	16.76, 54.23	16.27, 58.49	458	477	correct	0.0	–
61	Bay of Bengal	11.41, 81.71	17.38, 82.98	678	718	correct	0.0	–
62	Malacca, Andaman	5.73, 99.90	4.83, 104.04	469	1246	grazing	1.0	–
63	South China Sea	6.21, 107.64	9.63, 115.46	942	1023	correct	0.0	–
64	Japan, NW Pacific	37.77, 146.12	40.97, 139.34	683	1638	crosses land	51.0	classifier
65	Southern Africa	-35.05, 19.37	-33.06, 17.88	260	298	correct	0.0	–
66	Central America	8.24, -80.21	11.88, -80.02	405	–	no route	–	no sea link
67	North Europe	58.46, 21.62	58.53, 10.53	644	1411	crosses land	19.7	classifier
68	Red Sea, Aden	26.96, 50.96	29.27, 48.15	377	387	correct	0.0	–
69	Aegean Sea	34.62, 22.23	38.21, 21.59	403	461	crosses land	24.0	lattice
70	Black Sea, E. Med.	43.04, 27.95	44.51, 33.21	453	506	correct	0.0	–
71	Persian Gulf	25.35, 58.13	15.88, 53.92	1141	1491	correct	0.0	–
72	Bay of Bengal	17.42, 83.00	19.84, 89.27	714	781	correct	0.0	–
73	Malacca, Andaman	7.57, 96.89	6.91, 98.79	223	247	correct	0.0	–
74	South China Sea	7.67, 113.59	17.27, 118.25	1181	1266	correct	0.0	–
75	Japan, NW Pacific	42.75, 151.65	39.60, 154.76	436	438	correct	0.0	–
76	Southern Africa	-34.40, 18.47	-34.51, 21.93	318	437	correct	0.0	–
77	Central America	10.69, -79.36	9.38, -80.31	179	186	correct	0.0	–
78	North Europe	61.55, -27.23	57.67, -25.32	445	460	correct	0.0	–
79	Red Sea, Aden	9.84, 51.22	11.16, 48.09	372	447	crosses land	184.6	classifier
80	Aegean Sea	40.02, 23.32	36.18, 20.31	502	946	grazing	2.0	–
81	Black Sea, E. Med.	43.79, 30.48	35.35, 35.38	1028	–	no route	–	no sea link

No.	Region	From	To	GC, km	Route, km	Verdict	Land run, km	Cause
82	Persian Gulf	24.80, 58.97	20.22, 58.23	514	686	crosses land	30.3	classifier
83	Bay of Bengal	19.98, 89.23	21.33, 90.90	230	263	correct	0.0	–
84	Malacca, Andaman	7.22, 96.76	5.21, 104.04	836	1667	crosses land	10.6	classifier
85	South China Sea	19.02, 118.42	19.18, 120.83	254	268	correct	0.0	–
86	Japan, NW Pacific	36.39, 142.16	36.29, 146.58	395	398	correct	0.0	–
87	Southern Africa	-33.71, 27.13	-35.52, 18.83	785	861	correct	0.0	–
88	Central America	9.65, -79.47	8.03, -80.29	202	–	no route	–	no sea link
89	North Europe	57.00, 17.57	54.95, 18.25	232	248	correct	0.0	–
90	Red Sea, Aden	11.22, 43.61	18.32, 39.77	891	973	crosses land	5.0	classifier
91	Aegean Sea	34.69, 27.74	36.78, 20.16	723	788	correct	0.0	–
92	Black Sea, E. Med.	42.07, 31.90	43.82, 31.73	196	202	correct	0.0	–
93	Persian Gulf	16.04, 59.12	23.84, 58.57	870	980	correct	0.0	–
94	Bay of Bengal	11.71, 83.95	11.90, 80.10	419	429	correct	0.0	–
95	Malacca, Andaman	6.18, 97.42	6.68, 104.28	760	1632	crosses land	5.0	classifier
96	South China Sea	8.02, 114.34	10.47, 117.25	420	437	correct	0.0	–
97	Japan, NW Pacific	39.48, 146.94	36.07, 150.30	481	483	correct	0.0	–
98	Southern Africa	-37.06, 15.24	-34.25, 22.85	755	837	correct	0.0	–
99	Central America	9.54, -82.47	11.54, -79.23	419	448	correct	0.0	–
100	North Europe	56.17, 17.88	54.09, 11.34	476	547	correct	0.0	–

Table C.2 lists the 19 voyages whose route was changed when the test was repeated with the automatic coastline check (subsection 3.5.7): the verdict, the longest land run and the route length without and with the check, the lattice finally used and the number of additional searches. All other voyages kept their route.

Table C.2 – Voyages changed by the coastline check

No.	Region	Without: verdict	land run, km	route, km	With: verdict	land run, km	route, km	lattice, °	extra searches
1	North Europe	crosses land	13.4	976	grazing	2.0	976	0.1375	2
9	Japan & North-West Pacific	crosses land	27.9	1137	grazing	1.0	1460	0.055	2
16	Persian Gulf & Arabian Sea	crosses land	10.0	349	correct	0.0	396	0.25	1
18	Strait of Malacca & Andaman Sea	crosses land	6.9	2040	grazing	2.9	2035	0.0756	2
24	Red Sea & Gulf of Aden	crosses land	4.0	978	correct	0.0	978	0.25	1

No.	Region	Without: verdict	land run, km	route, km	With: verdict	land run, km	route, km	lattice, °	extra searches
25	Aegean Sea	crosses land	17.7	791	grazing	3.0	956	0.25	3
27	Persian Gulf & Arabian Sea	crosses land	7.0	696	correct	0.0	754	0.25	1
30	South China Sea	crosses land	14.6	1138	grazing	2.0	1138	0.25	4
47	Aegean Sea	crosses land	9.7	259	grazing	3.0	399	0.25	2
49	Persian Gulf & Arabian Sea	crosses land	4.0	740	correct	0.0	758	0.25	1
51	Strait of Malacca & Andaman Sea	crosses land	7.9	936	correct	0.0	934	0.25	1
64	Japan & North-West Pacific	crosses land	51.0	1638	grazing	1.9	1683	0.0756	2
67	North Europe	crosses land	19.7	1411	correct	0.0	1400	0.1375	2
69	Aegean Sea	crosses land	24.0	461	grazing	1.0	462	0.1	0
79	Red Sea & Gulf of Aden	crosses land	184.6	447	correct	0.0	769	0.25	2
82	Persian Gulf & Arabian Sea	crosses land	30.3	686	correct	0.0	782	0.25	3
84	Strait of Malacca & Andaman Sea	crosses land	10.6	1667	grazing	1.9	1650	0.1	0
90	Red Sea & Gulf of Aden	crosses land	5.0	972	correct	0.0	973	0.25	1
95	Strait of Malacca & Andaman Sea	crosses land	5.0	1632	grazing	2.9	1632	0.0756	1

The maps of figures C.1–C.11 show the voyages of each region on the land of the independent mask (grey): green — correct, orange — grazing, red — crosses land; a voyage without a route is shown by its two ends in black. The number next to a route is its number in table C.1.

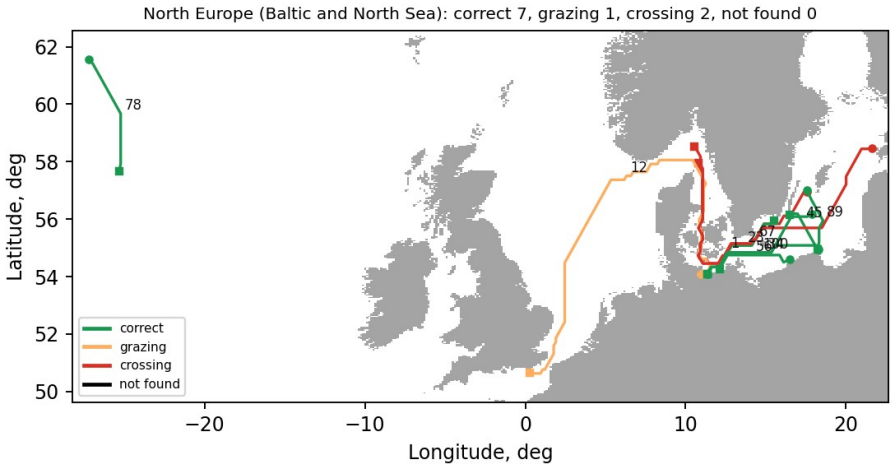


Figure C.1 – Test voyages in the region North Europe (Baltic and North Sea)

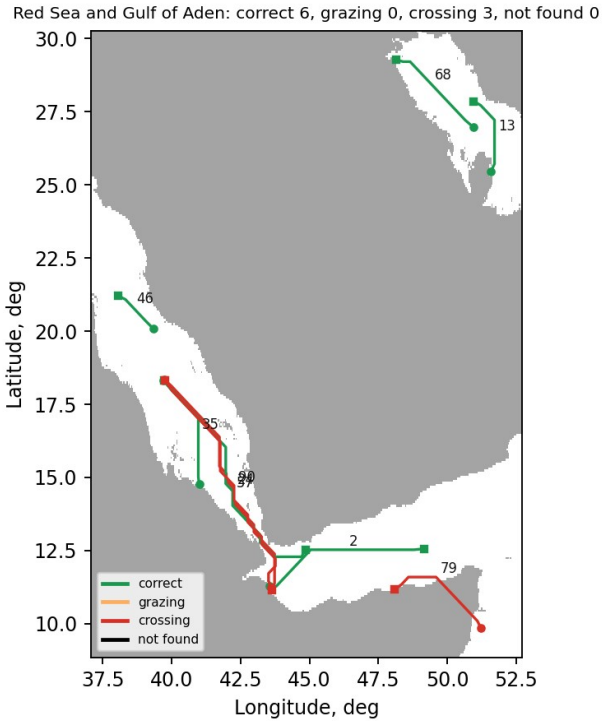


Figure C.2 – Test voyages in the region Red Sea and Gulf of Aden

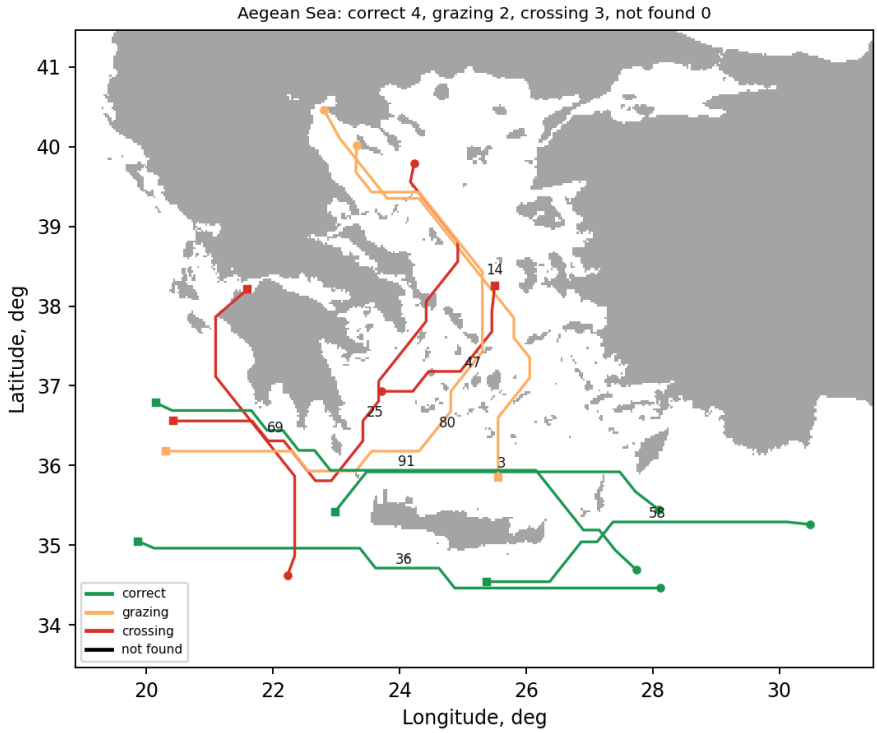


Figure C.3 – Test voyages in the region Aegean Sea

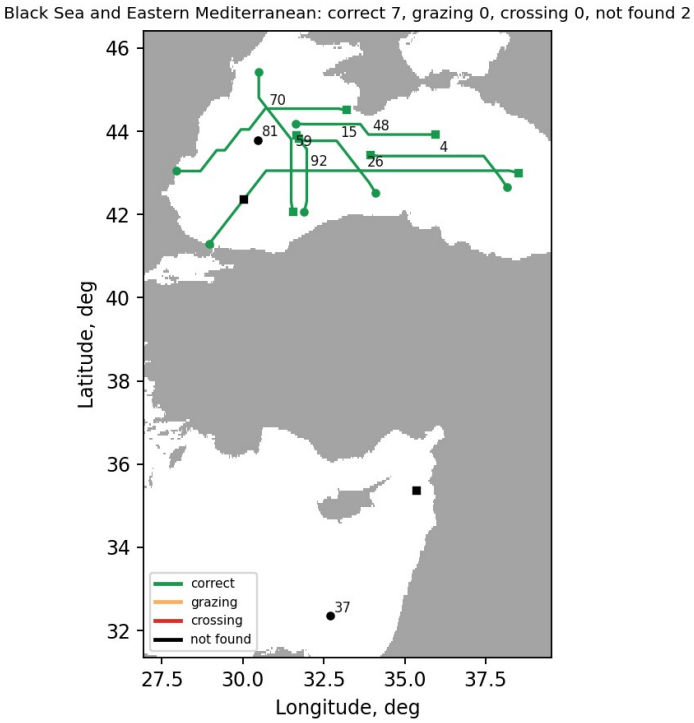


Figure C.4 – Test voyages in the region Black Sea and Eastern Mediterranean

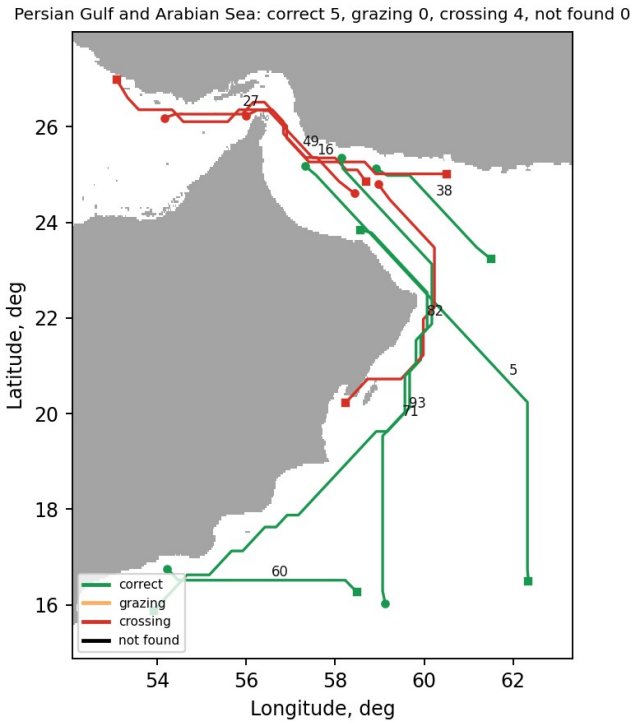


Figure C.5 – Test voyages in the region Persian Gulf and Arabian Sea

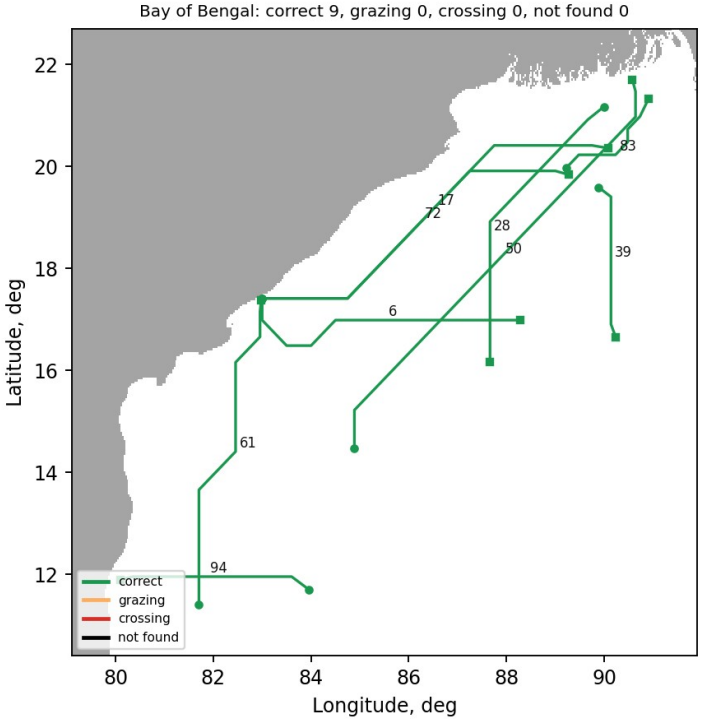


Figure C.6 – Test voyages in the region Bay of Bengal

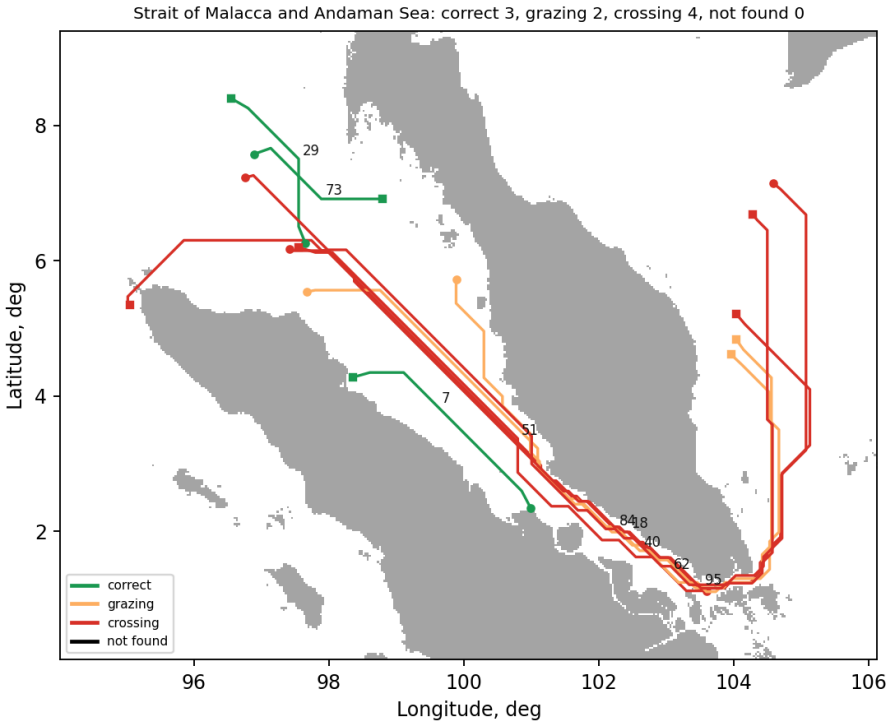


Figure C.7 – Test voyages in the region Strait of Malacca and Andaman Sea

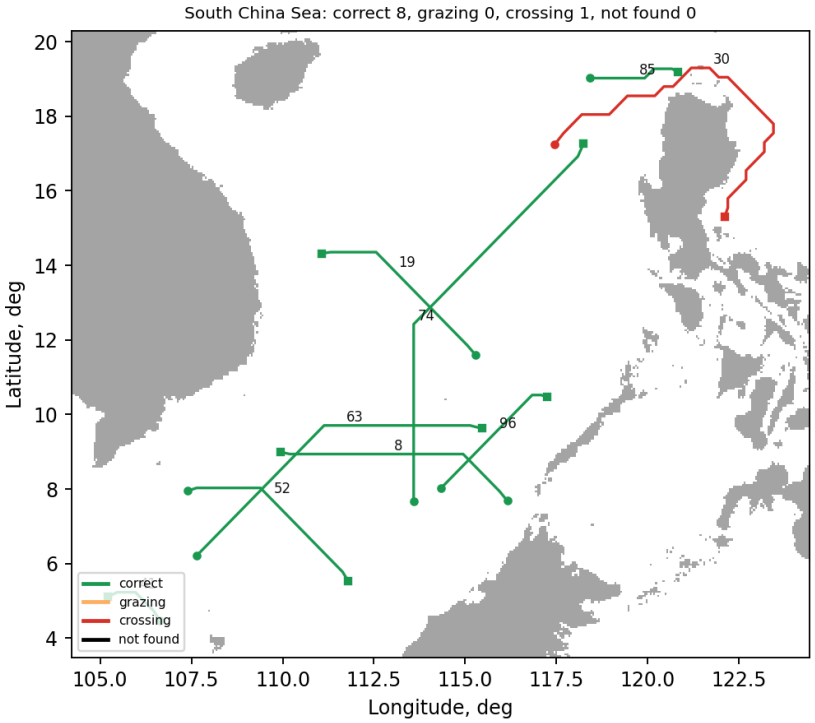


Figure C.8 – Test voyages in the region South China Sea

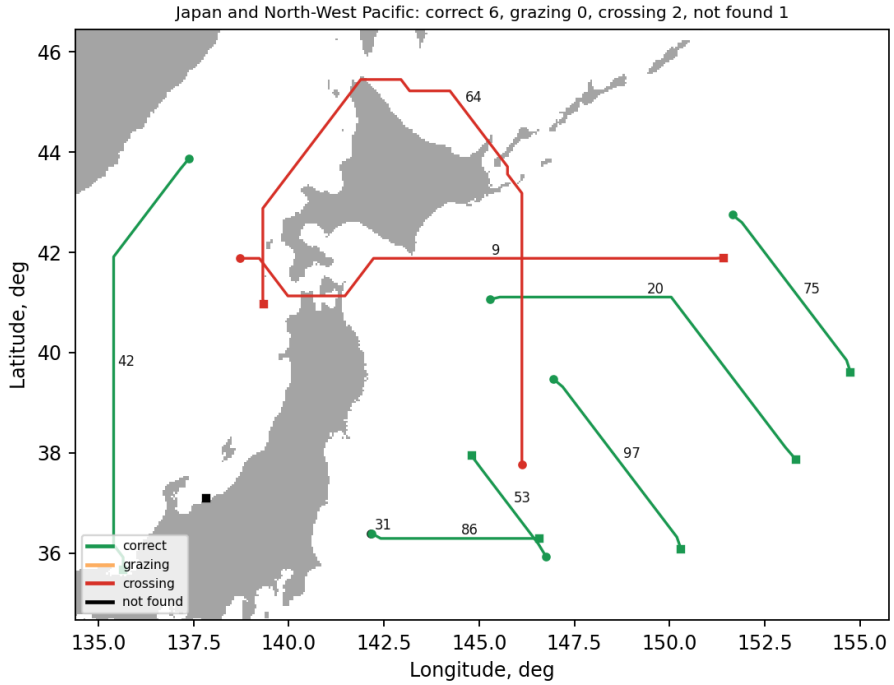


Figure C.9 – Test voyages in the region Japan and North-West Pacific

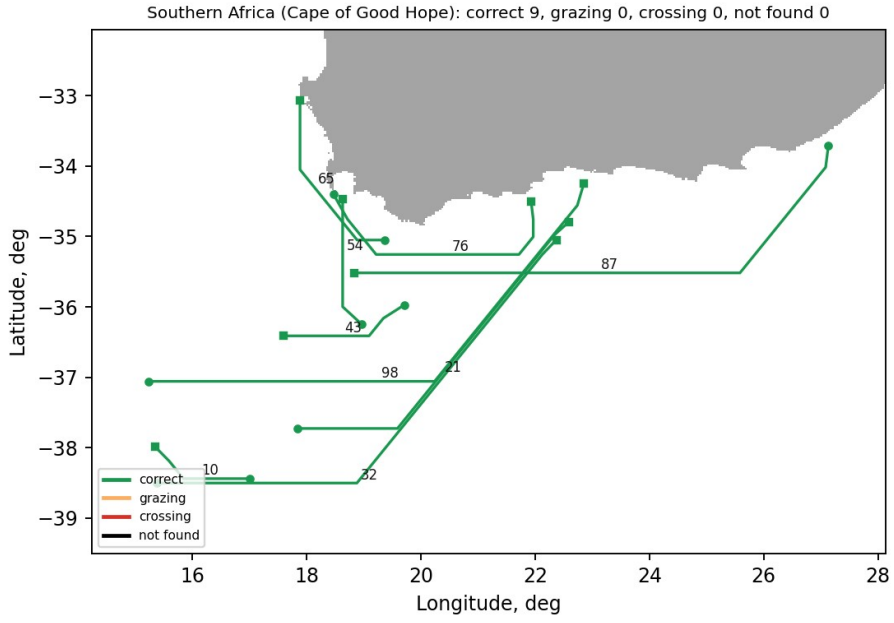


Figure C.10 – Test voyages in the region Southern Africa (Cape of Good Hope)

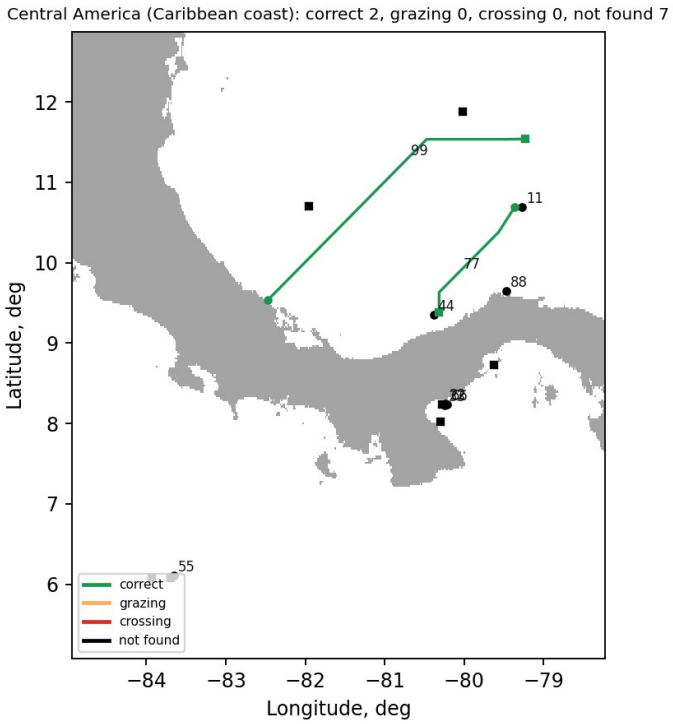


Figure C.11 – Test voyages in the region Central America (Caribbean coast)

Appendix D

MINISTRY OF EDUCATION OF THE REPUBLIC OF BELARUS
EDUCATIONAL INSTITUTION
«BREST STATE TECHNICAL UNIVERSITY»
Department of Intelligent Information Technologies

Application for constructing a maritime vessel's course based on neural-network classification of geographic zones

PROGRAM TEXT
ДР.ИИ2025.250978-11 12 01

Sheets: 210

Supervisor	А.А.Козинский
Completed	Li Haoran
Standard control	А.А.Козинский

2027

ABSTRACT

This document contains the text of the program “Maritime Route Planner”, an application that constructs the course of a maritime vessel on the basis of a neural-network classification of geographic zones.

Purpose of the program: loading of AIS data, classification of geographic zones with a pre-trained neural network, construction of a traversal cost map, search of the route with A*, Dijkstra, dynamic programming or a genetic algorithm, visualisation of the route, comparison with the shortest-distance route and export of the results to GeoJSON, CSV and PDF.

Composition of the program: an application package of 10,731 lines of Python in five layers, a single-page web client of 2,064 lines, a database schema of 213 lines of SQL, seven command-line scripts and one hundred and one automated tests. The line counts include the documentation comments.

The program is written in English; the comments in Chinese duplicate the English documentation strings immediately above them. The auxiliary scripts that generate the figures and the sheets of graphic material are supplied in the accompanying archive and are not reproduced here.

CONTENTS

The composition of the program text is given in table D.1.

Table D.1 – Composition of the program text

Section	Content	Files
D.1	Configuration	1
D.2	Data layer: loading, validation, cleaning, features	5
D.3	Neural network: architecture, training, inference	4
D.4	Routing: geometry, cost map, A*, Dijkstra, dynamic programming, genetic algorithm, coastline check, planner	9
D.5	Storage: schema and repository	3
D.6	Export: GeoJSON, CSV, PDF	2
D.7	Web service: REST, WebSocket, tiles	3
D.8	Web client: HTML, CSS, JavaScript	5
D.9	Command-line scripts	7
D.10	Automated tests	9

D.1 Configuration

config.py – paths, zone taxonomy, cost weights and hyper-parameters

```
"""Global configuration for the Maritime Route Planning application.

全局配置模块：集中管理路径、类别定义、模型超参数与航行代价权重。
All tunable constants live here so that the rest of the code base never
hard-codes a path, a class index or a cost weight.

Contents
--------
* **Project layout** - absolute paths of the data, model, output and
  documentation directories. Three of them can be overridden through
  environment variables (``MR_DATA_DIR``, ``MR_MODEL_DIR``, ``MR_OUTPUT_DIR``)
  and the SQLite database through ``MR_DB_PATH``, so that tests and
  deployments can redirect all file I/O without editing code.
* **Zone taxonomy** - the four classes predicted by the neural network, their
  descriptions, colours and traversal costs used by the route planner.
* **Regions** - bounding boxes of the eleven sea regions of the labelled corpus,
  used for per-region statistics and evaluation.
* **Feature extraction** - OSM column names and neighbourhood sizes used by
  :mod:`maritime_route.data.features`.
* **Hyper-parameters** - frozen dataclasses :class:`TrainingConfig`,
  :class:`RoutingConfig`, :class:`GeneticConfig` and :class:`ServerConfig`, each
  with one module-level default instance (``TRAINING``, ``ROUTING``,
  ``GENETIC``, ``SERVER``). The dataclasses are frozen so that a shared default
  cannot be modified by accident; a variant is created with
  :func:`dataclasses.replace`.

Side effect: importing this module creates the data/model/output/figure
directories if they do not exist yet.

内容：项目路径、四类区域定义与配色、航行代价、海域范围、特征参数，以及训练、
航线规划、遗传算法和 Web 服务的超参数（冻结数据类，修改需用 dataclasses.replace）。
导入本模块会自动创建所需目录。
"""
from __future__ import annotations

import os
from dataclasses import dataclass, field
from pathlib import Path
from typing import Dict, List, Tuple

# ---------------------------------------------------------------------------- #
# Project layout                                                               #
# ---------------------------------------------------------------------------- #
# The three roots are derived from the location of this file:
#   <PROJECT_ROOT>/src/maritime_route/config.py
# so the code works wherever the repository is checked out.
# 三个根目录由本文件位置推导，与仓库所在路径无关。

#: Directory of the ``maritime_route`` package itself. 包目录。
PACKAGE_ROOT: Path = Path(__file__).resolve().parent
#: The ``src`` directory that contains the package. src 目录。
SRC_ROOT: Path = PACKAGE_ROOT.parent
#: Repository root (contains ``data``, ``models``, ``docs``, ``scripts``). 项目根目录。
PROJECT_ROOT: Path = SRC_ROOT.parent

#: Root of all data files; override with the ``MR_DATA_DIR`` environment variable.
#: 数据根目录，可用环境变量 MR_DATA_DIR 覆盖。
DATA_DIR: Path = Path(os.environ.get("MR_DATA_DIR", PROJECT_ROOT / "data"))
#: Raw, unprocessed input corpora (GeoJSON). 原始数据。
RAW_DIR: Path = DATA_DIR / "raw"
#: Output of ``scripts/prepare_dataset.py``: feature matrices and statistics. 预处理结果。
PROCESSED_DIR: Path = DATA_DIR / "processed"
#: Small example files for demonstrations and the UI. 示例数据。
SAMPLE_DIR: Path = DATA_DIR / "samples"
#: Trained weights, scaler, reference index, metadata; override with ``MR_MODEL_DIR``.
#: 模型文件目录，可用 MR_MODEL_DIR 覆盖。
MODEL_DIR: Path = Path(os.environ.get("MR_MODEL_DIR", PROJECT_ROOT / "models"))
#: Exported routes (GeoJSON/CSV/PDF); override with ``MR_OUTPUT_DIR``. 导出文件目录。
OUTPUT_DIR: Path = Path(os.environ.get("MR_OUTPUT_DIR", PROJECT_ROOT / "outputs"))
#: Thesis documentation. 文档目录。
DOCS_DIR: Path = PROJECT_ROOT / "docs"
```

```
#: Figures generated by ``scripts/make_figures.py`` and related scripts. 图表输出目录。
FIGURE_DIR: Path = DOCS_DIR / "figures"

#: Default labelled corpus read by ``scripts/prepare_dataset.py``. 默认原始语料。
DEFAULT_RAW_DATASET: Path = RAW_DIR / "merged.geojson"
#: SQLite database of saved routes and uploads; override with ``MR_DB_PATH``.
#: SQLite 数据库路径，可用 MR_DB_PATH 覆盖。
DATABASE_PATH: Path = Path(os.environ.get("MR_DB_PATH", DATA_DIR / "maritime_route.db"))

# Create the writable directories once at import time so that no other module
# has to check for their existence before writing. exist_ok makes it idempotent.
# 导入时一次性创建可写目录，其他模块写文件前无需再检查；exist_ok 保证可重复执行。
for _d in (RAW_DIR, PROCESSED_DIR, SAMPLE_DIR, MODEL_DIR, OUTPUT_DIR, FIGURE_DIR):
    _d.mkdir(parents=True, exist_ok=True)

# ---------------------------------------------------------------------------- #
# Geographic zone taxonomy (four classes required by the assignment)          #
# ---------------------------------------------------------------------------- #
#: The four zone classes. The list order defines the integer class index used
#: by the network output layer (index 0 = OPEN_SEA ... 3 = COASTLINE), so it must
#: not be changed after a model has been trained.
#: 四个区域类别；列表顺序即网络输出的类别索引，训练后不可更改。
CLASS_NAMES: List[str] = ["OPEN_SEA", "COASTAL_SEA", "NEAR_COAST", "COASTLINE"]
#: Class name -> integer index (used to encode labels). 类别名 -> 索引。
CLASS_TO_INDEX: Dict[str, int] = {name: i for i, name in enumerate(CLASS_NAMES)}
#: Integer index -> class name (used to decode ``argmax`` of the network). 索引 -> 类别名。
INDEX_TO_CLASS: Dict[int, str] = {i: name for name, i in CLASS_TO_INDEX.items()}

#: Human readable descriptions used by the UI and the PDF report.
#: Note: the code ``COASTLINE`` historically denotes *inland* land, i.e. land
#: remote from the sea, while ``NEAR_COAST`` is land within 5 km of the shore.
#: 注意：COASTLINE 实际表示远离海岸的内陆，NEAR_COAST 表示距海岸 5 km 以内的陆地。
CLASS_DESCRIPTION: Dict[str, str] = {
    "OPEN_SEA": "Open sea - deep water far from any shoreline",
    "COASTAL_SEA": "Coastal sea - navigable water close to the shore",
    "NEAR_COAST": "Near coast - land within 5 km of the shoreline",
    "COASTLINE": "Inland - land remote from the sea",
}

# The four zones are ordinal around sea level (deep water -> shallow water ->
# shore -> inland), so they are encoded with a DIVERGING hypsometric scheme:
# two hues (water blue / land ochre) whose lightness is monotone within each
# pole and whose neutral midpoint is the coastline itself. This is the standard
# cartographic depth/height tint and, unlike a categorical palette, it also
# conveys the ordering of the classes.
# 四个区域沿海平面呈有序分布，因此采用"发散型"等深-等高配色：
# 水域蓝 / 陆地赭，海岸线为中性中点，每一极内部亮度单调变化。
#
# Separation was verified numerically (OKLab dE x100, all pairs):
#   light  - worst CVD dE 15.8 (protan), normal-vision dE 20.2
#   dark   - worst CVD dE 14.8 (protan), normal-vision dE 16.8
# Both clear the >=8 CVD and >=15 normal-vision gates. The two light tints fall
# below 3:1 contrast on white, so every figure and map carries direct labels or
# a legend - colour never encodes identity on its own.
# (CVD = colour vision deficiency; dE = perceptual colour difference.)
# （CVD 指色觉缺陷；dE 为感知色差。）

#: Print / figure / export palette (light surfaces). Hex RGB strings.
#: Used by the GeoJSON/PDF exporters and the database seed. 打印/导出用浅色底配色。
CLASS_COLOR: Dict[str, str] = {
    "OPEN_SEA": "#14507f",
    "COASTAL_SEA": "#6bb3d6",
    "NEAR_COAST": "#e8c88c",
    "COASTLINE": "#8a5a2b",
}

#: Same ramp re-stepped for the dark user interface and the on-board tiles.
#: Used by the web API legend and the raster tile renderer. 深色界面与地图瓦片配色。
CLASS_COLOR_DARK: Dict[str, str] = {
    "OPEN_SEA": "#3d7fb8",
    "COASTAL_SEA": "#7cc3e3",
    "NEAR_COAST": "#d9b877",
    "COASTLINE": "#a9763f",
}

#: Categorical slots for statistical series (training curves, algorithm bars).
#: ``grid`` is the colour of chart grid lines, ``neutral`` of reference lines.
#: 统计图表的分类配色（已通过色觉与对比度校验）。
SERIES_COLOR: Dict[str, str] = {
    "primary": "#2a78d6",
```

```
    "secondary": "#eb6834",
    "tertiary": "#1baf7a",
    "quaternary": "#4a3aa7",
    "neutral": "#52514e",
    "grid": "#d8d8d4",
}

#: Traversal cost multiplier per zone. Navigable zones are cheap, land is
#: effectively impassable. 通行代价权重：数值越大越不适合航行。
#: The cost of a grid step is (step length in km) x (multiplier of the cell), so
#: the value is dimensionless. Open sea is the reference (1.0). Coastal sea costs
#: 1.8, i.e. the planner accepts a detour of up to 80 % in open water to avoid a
#: stretch of coastal water (shallower, denser traffic). Land is infinite and is
#: stored as :data:`IMPASSABLE_COST` in numeric rasters.
#: 每步代价 = 步长(km) × 单元权重（无量纲）。近岸海 1.8 意味着为避开近岸水域可接受
#: 最多 80% 的远海绕行；陆地为无穷大，在数值栅格中以 IMPASSABLE_COST 表示。
ZONE_COST: Dict[str, float] = {
    "OPEN_SEA": 1.0,
    "COASTAL_SEA": 1.8,
    "NEAR_COAST": float("inf"),
    "COASTLINE": float("inf"),
}

#: Finite stand-in used when the planner is asked to allow land crossing
#: (debug mode) and inside numeric cost rasters that cannot store infinity.
#: Search algorithms treat any cell with cost >= this value as a wall. 1e6 is
#: several orders of magnitude above any real water cost, so a single land cell
#: always outweighs any water detour.
#: 代价 >= 该值的单元被视为不可通行；1e6 远大于任何水域代价。
IMPASSABLE_COST: float = 1.0e6

#: Classes a ship may sail through; all other classes are land.
#: 可航行的类别；其余类别均为陆地。
NAVIGABLE_CLASSES: Tuple[str, ...] = ("OPEN_SEA", "COASTAL_SEA")

#: Classes that describe water and classes that describe land. A labelled point
#: whose class disagrees with the sign of its measured height/depth is treated
#: as a labelling error (see :mod:`maritime_route.data.cleaning`).
#: 水域类与陆地类；类别与实测高程/水深符号矛盾的样本视为标注错误。
WATER_CLASSES: Tuple[str, ...] = ("OPEN_SEA", "COASTAL_SEA")
LAND_CLASSES: Tuple[str, ...] = ("NEAR_COAST", "COASTLINE")

#: Tolerance of the label/measurement consistency check, metres. A sea point
#: may lie up to this height above zero (tide, DEM error) and a land point up to
#: this depth below zero before it is considered mislabelled.
#: 一致性检查容差（米），用于吸收潮汐与高程模型误差。
LABEL_CHECK_TOLERANCE_M: float = 5.0

# ---------------------------------------------------------------------------- #
# Geographic regions of the corpus                                             #
# ---------------------------------------------------------------------------- #
#: The labelled corpus was collected in eleven separate sea regions (the
#: Aegean Sea was added by the supervisor in the second release of the corpus).
#: Each region is described by a bounding box (lat_min, lat_max, lon_min,
#: lon_max); boxes are tested in order and the first match wins, so the Aegean
#: box is placed before the larger Eastern Mediterranean box that contains it. Points
outside every box
#: (inland samples far from any of the seas) are reported as "Other".
#: Units: decimal degrees (WGS84); lower bounds inclusive, upper bounds exclusive.
#: 语料覆盖的十一个海域（爱琴海为第二版语料新增），按经纬度范围划分，依次匹配
#: （爱琴海范围排在包含它的东地中海范围之前），均不匹配者记为 Other。
#: 单位为十进制度（WGS84）；下界包含，上界不包含。
REGIONS: List[Tuple[str, Tuple[float, float, float, float]]] = [
    ("North Europe (Baltic and North Sea)",      (47.5, 68.0, -40.0, 33.0)),
    ("Red Sea and Gulf of Aden",                 (5.0, 30.0, 32.0, 52.0)),
    ("Aegean Sea",                               (34.3, 41.3, 19.0, 31.0)),
    ("Black Sea and Eastern Mediterranean",      (29.0, 47.5, 19.0, 42.5)),
    ("Persian Gulf and Arabian Sea",             (5.0, 31.5, 48.0, 70.0)),
    ("Bay of Bengal",                            (10.0, 25.0, 78.0, 93.0)),
    ("Strait of Malacca and Andaman Sea",        (-1.0, 10.0, 92.0, 105.0)),
    ("South China Sea",                          (-1.0, 26.0, 103.0, 125.0)),
    ("Japan and North-West Pacific",             (19.0, 48.0, 125.0, 165.0)),
    ("Southern Africa (Cape of Good Hope)",      (-42.0, -28.0, 13.0, 32.0)),
    ("Central America (Caribbean coast)",        (4.0, 18.0, -90.0, -76.0)),
]

# ---------------------------------------------------------------------------- #
# Feature extraction                                                           #
# ---------------------------------------------------------------------------- #
#: OpenStreetMap object counts stored with every labelled point (number of
```

```
#: objects of each type near the point). The order defines the last axis of
#: ``ReferenceIndex.osm_counts``; missing columns are filled with 0 on load.
#: 每个样本附带的 OSM 对象计数列; 顺序决定参考索引中 osm_counts 的列顺序。
OSM_COLUMNS: List[str] = [
    "osm_piers",
    "osm_lighthouses",
    "osm_breakwaters",
    "osm_harbors",
    "osm_rivers",
    "osm_lakes",
    "osm_beaches",
    "osm_cliffs",
    "osm_industrial",
]

#: Number of bathymetric reference samples queried around every point.
#: k = 8 neighbours give a stable mean/std/min/max of the local relief while
#: keeping the neighbourhood small (a few km in the dense corpus).
#: 每个点查询的邻近参考样本数 k。
BATHY_NEIGHBOURS: int = 8
#: Radius (km) inside which OSM objects are aggregated for a query point.
#: In :func:`maritime_route.data.features.build_features` it is used only as a
#: floor: 10 % of it (1.2 km) is the minimum neighbourhood span when the OSM
#: density per km is computed, which prevents division by a near-zero distance.
#: OSM 聚合半径 (km) ; 特征计算中取其 10% (1.2 km) 作为密度分母的下限。
OSM_RADIUS_KM: float = 12.0
#: Mean Earth radius, km (IUGG value). Converts haversine angles (radians) to km.
#: 地球平均半径 (km) , 用于将球面角距离 (弧度) 换算为公里。
EARTH_RADIUS_KM: float = 6371.0088

# ---------------------------------------------------------------------------- #
# Neural network hyper-parameters                                              #
# ---------------------------------------------------------------------------- #
@dataclass(frozen=True)
class TrainingConfig:
    """Hyper-parameters of the zone classifier. 神经网络训练超参数。

    Consumed by :func:`maritime_route.model.trainer.train_model` and by
    :func:`maritime_route.data.dataset.prepare` (split sizes and seed).
    The instance is immutable; ``scripts/train_model.py`` creates a modified
    copy with :func:`dataclasses.replace` when command-line options are given.

    Attributes
    ----------
    hidden_sizes:
        Width of each residual block of the MLP. A funnel 256 -> 128 -> 64 on
        a 28-dimensional input gives about 98 k trainable parameters
        (including the skip projections), enough capacity
        for the non-linear coastal boundaries without overfitting.
    dropout:
        Dropout probability applied after every hidden block.
    batch_size:
        Samples per optimiser step.
    epochs:
        Upper bound on the number of passes over the training set; early
        stopping normally ends training earlier.
    learning_rate:
        Initial learning rate of AdamW.
    weight_decay:
        Decoupled L2 regularisation coefficient of AdamW.
    early_stopping_patience:
        Epochs without validation-loss improvement before training stops.
    lr_scheduler_patience:
        Epochs without improvement before the learning rate is halved.
        Smaller than ``early_stopping_patience`` so that the rate is reduced
        (twice) before training is abandoned.
    label_smoothing:
        Probability mass moved from the true class to the other classes in
        the cross-entropy target.
    test_size, val_size:
        Shares of the corpus held out for testing and validation (70/15/15).
    random_seed:
        Seed of the data split, weight initialisation and batch shuffling.
    device:
        PyTorch device string; ``"cpu"`` because the model is small.

    各字段: 隐藏层宽度、Dropout、批大小、最大轮数、学习率、权重衰减、早停耐心、
    学习率调度耐心、标签平滑、测试/验证集比例、随机种子与计算设备。
    """
```

```
    hidden_sizes: Tuple[int, ...] = (256, 128, 64)  # neurons per residual block 各残差块
宽度
    dropout: float = 0.20                  # dropout probability 随机失活概率
    batch_size: int = 512                  # samples per mini-batch 批大小
    epochs: int = 120                      # max epochs (early stopping usually ends
earlier) 最大轮数
    learning_rate: float = 1.5e-3          # initial AdamW learning rate 初始学习率
    weight_decay: float = 1.0e-4           # AdamW decoupled weight decay 权重衰减
    early_stopping_patience: int = 15      # epochs without val-loss improvement 早停耐心
    lr_scheduler_patience: int = 6         # epochs before LR is halved 学习率减半前的等待
轮数
    label_smoothing: float = 0.02          # 2 % of target mass spread over other
classes 标签平滑
    test_size: float = 0.15                # share of the corpus for testing 测试集比例
    val_size: float = 0.15                 # share of the corpus for validation 验证集比例
    random_seed: int = 42                  # reproducibility 随机种子
    device: str = "cpu"                    # PyTorch device 计算设备


#: Default training configuration. 默认训练配置。
TRAINING = TrainingConfig()

# Files written by the trainer / dataset preparation and read by the inference
# service. They must be kept together: the scaler and the reference index are
# part of the model (features are meaningless without them).
# 训练与预处理生成、推理服务读取的文件；标准化器和参考索引与权重必须配套使用。
#: PyTorch checkpoint: ``{"state_dict": ..., "config": ...}``. 网络权重。
MODEL_WEIGHTS_PATH: Path = MODEL_DIR / "zone_classifier.pt"
#: JSON with hyper-parameters, test metrics and training history. 模型元数据。
MODEL_METADATA_PATH: Path = MODEL_DIR / "model_metadata.json"
#: Fitted :class:`sklearn.preprocessing.StandardScaler` (joblib). 特征标准化器。
SCALER_PATH: Path = MODEL_DIR / "feature_scaler.joblib"
#: Label-free :class:`~maritime_route.data.features.ReferenceIndex` (.npz). 参考索引。
BATHY_INDEX_PATH: Path = MODEL_DIR / "reference_index.npz"

# ---------------------------------------------------------------------------- #
# Routing                                                                      #
# ---------------------------------------------------------------------------- #
@dataclass(frozen=True)
class RoutingConfig:
    """Default parameters of the grid based route planner. 航线规划默认参数。

    Used by :mod:`maritime_route.routing.cost_map`, ``astar``, ``dijkstra`` and
    ``planner``.

    Attributes
    ----------
    grid_resolution_deg:
        Side of one lattice cell, degrees (0.25 deg is about 28 km of latitude).
    max_grid_cells:
        Cell budget of one cost map. If the bounding box needs more cells, the
        resolution is coarsened by a factor 1.5 until it fits (the network has
        to classify every cell, so this bounds the classification time).
    margin_deg:
        Margin added around the bounding box of the waypoints, degrees, so the
        route can bend around a headland that lies outside the direct box.
    diagonal_moves:
        ``True`` - 8-connected lattice (N, S, E, W and diagonals); ``False`` -
        4-connected.
    heuristic_weight:
        Multiplier of the A* great-circle heuristic. 1.0 keeps the heuristic
        admissible (optimal path); values > 1 give a faster, possibly
        sub-optimal weighted A*.
    safety_margin_cells:
        Width, in cells, of the band of water next to land whose cost is
        multiplied by ``proximity_penalty``.
    proximity_penalty:
        Extra multiplicative penalty applied to a cell adjacent to land.

    各字段：网格分辨率（度）、单元数上限、边界余量（度）、是否允许对角移动、
    A* 启发函数权重、近岸安全带宽度（单元数）与近岸惩罚系数。
    """

    grid_resolution_deg: float = 0.25      # cell size, degrees 网格分辨率（度）
    max_grid_cells: int = 400_000          # cell budget before coarsening 单元数上限
    margin_deg: float = 2.0                # bbox margin, degrees 边界余量（度）
    diagonal_moves: bool = True            # 8-neighbour moves 八邻域移动
    heuristic_weight: float = 1.0          # 1.0 = admissible A* 启发权重
    safety_margin_cells: int = 1           # width of the near-land band, cells 近岸安全带
宽度
```

```
    #: Extra multiplicative penalty applied to a cell adjacent to land.
    #: Water cells in the safety band cost 1.6x their zone cost (capped below
    #: IMPASSABLE_COST), which keeps routes one cell off the shore when possible.
    #: 近岸惩罚系数：安全带内的水域单元代价乘以 1.6。
    proximity_penalty: float = 1.6


#: Default routing configuration. 默认航线规划配置。
ROUTING = RoutingConfig()


@dataclass(frozen=True)
class GeneticConfig:
    """Parameters of the genetic-algorithm route search.

    A chromosome is a list of intermediate waypoints (lattice cells); the route
    is the chain of straight lattice lines start -> w1 -> ... -> wk -> goal.
    See :mod:`maritime_route.routing.genetic` for the operators.
    遗传算法参数：染色体为中间航点序列，航线由相邻航点之间的直线段连接而成。
    """

    population_size: int = 80          # individuals per generation 种群规模
    generations: int = 300             # upper limit of generations 最大代数
    stagnation_limit: int = 60         # stop after this many generations without
improvement
    tournament_size: int = 3           # tournament selection 锦标赛规模
    crossover_rate: float = 0.9        # probability of one-point crossover 交叉概率
    mutation_rate: float = 0.35        # probability that a child is mutated 变异概率
    elite: int = 2                     # best individuals copied unchanged 精英保留数
    max_waypoints: int = 14            # longest chromosome allowed 最大航点数
    initial_waypoints: Tuple[int, int] = (1, 6)   # length range of random chromosomes
    land_penalty_weight: float = 200.0  # weight of an impassable cell in the fitness
    random_seed: int = 42              # reproducible runs 随机种子
    # stagnation_limit: 连续多少代无改进即停止。
    # initial_waypoints: 随机初始染色体的航点数范围（最少，最多）。
    # land_penalty_weight: 适应度中每个陆地单元的惩罚权重（以有限值代替无穷大，
    #   使穿越陆地的个体仍可比较并逐步被修复）。


#: Default genetic-algorithm configuration. 默认遗传算法配置。
GENETIC = GeneticConfig()

#: Algorithm identifiers accepted by the web API and the planner.
#: ``great_circle`` is the direct geodesic baseline without obstacle avoidance.
#: Web 接口与规划器支持的算法；great_circle 为不避障的大圆航线基线。
SUPPORTED_ALGORITHMS: Tuple[str, ...] = ("astar", "dijkstra", "genetic", "dynamic",
"great_circle")

# ---------------------------------------------------------------------------- #
# Web server                                                                   #
# ---------------------------------------------------------------------------- #
@dataclass(frozen=True)
class ServerConfig:
    """Settings of the FastAPI / Uvicorn web service. Web 服务配置。

    Attributes
    ----------
    host:
        Interface to bind; ``MR_HOST`` environment variable, default loopback
        (127.0.0.1) so the service is not exposed on the network by default.
    port:
        TCP port; ``MR_PORT`` environment variable, default 8000.
    title, version:
        Shown in the OpenAPI documentation of the service.
    progress_interval_s:
        Minimum time between two progress messages sent over the WebSocket
        while a route is computed; throttles traffic to the browser.

    注意：host 与 port 的默认值在导入模块时从环境变量读取一次。
    """

    host: str = os.environ.get("MR_HOST", "127.0.0.1")   # bind address 监听地址
    port: int = int(os.environ.get("MR_PORT", "8000"))   # TCP port 端口
    title: str = "Maritime Route Planner"                # OpenAPI title 服务标题
    version: str = "1.2.0"                               # OpenAPI version 服务版本
    #: Seconds between progress frames pushed over the WebSocket.
    #: WebSocket 进度推送的最小间隔（秒）。
    progress_interval_s: float = 0.15
```

```
#: Default server configuration. 默认服务配置。
SERVER = ServerConfig()

#: Application metadata shown in exports, the PDF report and ``__version__``.
#: 应用元数据：版本、作者、所属大学（用于导出文件与报告）。
APP_VERSION: str = "1.2.0"
APP_AUTHOR: str = "Li Haoran"
APP_UNIVERSITY: str = "Brest State Technical University"
```

D.2 Data layer

data/__init__.py

```
"""Data access layer: loading, validation and feature engineering.

数据访问层：加载、校验与特征工程。

Modules
-------
:mod:`~maritime_route.data.ais_loader`
    Reads GeoJSON / CSV / JSON point files into a :class:`pandas.DataFrame`,
    normalises column names and drops rows with invalid coordinates.
:mod:`~maritime_route.data.cleaning`
    Removes labelled points whose class contradicts their measured
    height/depth, and assigns every point to one of the eleven sea regions.
:mod:`~maritime_route.data.features`
    Builds the label-free :class:`ReferenceIndex` and the 28-column feature
    matrix that is the input of the neural network.
:mod:`~maritime_route.data.dataset`
    Runs the whole preparation pipeline (load -> validate -> clean -> split ->
    index -> features) and saves the train/validation/test matrices.

Only the names used by other layers (web service, model, routing) are
re-exported here; ``cleaning`` and ``dataset`` are imported explicitly by the
scripts that need them.

子模块：ais_loader（读取与校验）、cleaning（清洗与海域划分）、features（参考索引
与 28 维特征）、dataset（完整预处理流程）。此处仅导出其他层需要的名称。
"""
# Re-export the public API so callers can write ``from maritime_route.data import ...``.
# 重新导出公共接口，便于上层直接从 maritime_route.data 导入。
from .ais_loader import DataLoadError, LoadReport, load_any, load_geojson, validate
from .features import FEATURE_NAMES, N_FEATURES, ReferenceIndex, build_features,
features_for_coordinates

__all__ = [
    "DataLoadError", "LoadReport", "load_any", "load_geojson", "validate",
    "FEATURE_NAMES", "N_FEATURES", "ReferenceIndex", "build_features",
    "features_for_coordinates",
]
```

data/ais_loader.py – loading and validation of AIS data

```
"""Loading of AIS / route-point data from GeoJSON, CSV and JSON sources.

AIS 数据加载模块：支持 GeoJSON(FeatureCollection)、CSV 与 JSON 数组三种输入。
The loader is streaming-capable so that files of several hundred megabytes can
be ingested without materialising the whole document in memory.

Role in the pipeline
--------------------
This is the first stage of both the training pipeline
(:func:`maritime_route.data.dataset.prepare`) and the web upload endpoint.
Every loader returns a :class:`pandas.DataFrame` with one row per point and at
least the columns ``latitude`` and ``longitude`` (decimal degrees, WGS84).
Optional columns understood by the rest of the code are:

* ``class_code`` - one of :data:`~maritime_route.config.CLASS_NAMES` (labelled
  corpus only);
* ``noaa_depth`` - NOAA/ETOPO relief value, metres, positive above sea level;
* ``noaa_type`` - ``"height"`` or ``"depth"``, which kind of relief sample it is;
```

```
* ``elevation`` - terrain elevation, metres;
* the OSM count columns of :data:`~maritime_route.config.OSM_COLUMNS`.

After loading, :func:`validate` drops rows with missing or out-of-range
coordinates and returns a :class:`LoadReport` that the UI shows to the user.

Main objects
------------
:func:`load_any`
    Entry point; dispatches on the file extension, accepts in-memory uploads.
:func:`load_geojson`, :func:`load_csv`, :func:`load_json`
    Format-specific readers.
:func:`validate`
    Coordinate range check and report.
:class:`DataLoadError`, :class:`LoadReport`
    Error type and summary object.

在流水线中的位置：训练预处理与网页上传的第一步。所有读取函数返回每行一个点、
至少含 latitude/longitude（十进制度）的 DataFrame，随后由 validate 剔除无效坐标。
"""
from __future__ import annotations

import csv
import io
import json
import logging
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any, Dict, Iterable, Iterator, List, Optional, Sequence

import numpy as np
import pandas as pd

from ..config import CLASS_NAMES, OSM_COLUMNS

LOGGER = logging.getLogger(__name__)

#: Column aliases accepted in user supplied CSV files. 兼容常见列名写法。
#: The first alias present in the file is used; matching falls back to a
#: case-insensitive comparison (see :func:`_first_present`).
LAT_ALIASES = ("latitude", "lat", "y", "LAT", "Latitude")
LON_ALIASES = ("longitude", "lon", "lng", "long", "x", "LON", "Longitude")

#: Columns without which a row cannot be used at all. 必需列。
REQUIRED_COLUMNS = ("latitude", "longitude")
#: Columns converted to numbers on load; unparsable cells become NaN.
#: 加载时转换为数值的列，无法解析的值变为 NaN。
NUMERIC_COLUMNS = ("latitude", "longitude", "elevation", "noaa_depth", *OSM_COLUMNS)


class DataLoadError(ValueError):
    """Raised when an input file cannot be interpreted as AIS point data.

    Subclasses :class:`ValueError` (the input value is wrong, not the
    program). The web upload endpoint catches it and answers HTTP 400 with the
    message, instead of a server error.
    无法将输入解析为点数据时抛出；Web 上传接口捕获后返回 HTTP 400。
    """


@dataclass
class LoadReport:
    """Summary returned to the UI after an upload. 加载结果摘要。

    Attributes
    ----------
    source:
        File name or other description of the input.
    n_features:
        Number of rows (GeoJSON features / CSV lines) read from the input.
    n_valid:
        Rows that passed validation.
    n_dropped:
        Rows removed because of missing or out-of-range coordinates
        (``n_features - n_valid``).
    labelled:
        ``True`` if the input has a non-empty ``class_code`` column, i.e. it
        can be used for training or for measuring accuracy.
    columns:
        Column names of the validated frame.
```

```
    warnings:
        Human-readable messages shown next to the upload in the UI.
    """

    source: str
    n_features: int
    n_valid: int
    n_dropped: int
    labelled: bool
    columns: List[str] = field(default_factory=list)
    warnings: List[str] = field(default_factory=list)

    def to_dict(self) -> Dict[str, Any]:
        """Return the report as a JSON-serialisable dictionary.

        Returns
        -------
        dict
            All fields of the report, keyed by field name.

        转换为可 JSON 序列化的字典（用于 API 响应与统计文件）。
        """
        return {
            "source": self.source,
            "n_features": self.n_features,
            "n_valid": self.n_valid,
            "n_dropped": self.n_dropped,
            "labelled": self.labelled,
            "columns": self.columns,
            "warnings": self.warnings,
        }


# ---------------------------------------------------------------------------- #
# GeoJSON                                                                      #
# ---------------------------------------------------------------------------- #
def iter_geojson_features(path: Path | str) -> Iterator[Dict[str, Any]]:
    """Yield ``properties`` dictionaries of a GeoJSON FeatureCollection.

    使用 ijson 流式解析，避免一次性载入 100MB+ 文件。
    Falls back to a plain ``json.load`` when ijson is unavailable.

    ijson reads the file incrementally and yields one element of the
    ``features`` array at a time (prefix ``"features.item"``), so memory use
    stays proportional to one feature instead of the whole document.
    ``use_float=True`` makes ijson return ``float`` instead of
    :class:`decimal.Decimal`, which pandas and NumPy handle directly.

    Parameters
    ----------
    path:
        Path of a GeoJSON file whose top-level object is a FeatureCollection.

    Yields
    ------
    dict
        The ``properties`` of each feature, with ``latitude`` and
        ``longitude`` taken from the point geometry (see
        :func:`_merge_geometry`).
    """
    path = Path(path)
    try:
        import ijson  # type: ignore  # optional dependency 可选依赖

        with path.open("rb") as handle:
            for feature in ijson.items(handle, "features.item", use_float=True):
                yield _merge_geometry(feature)
        # The streaming branch finished; do not run the fallback below.
        # 流式解析已完成，不再执行下面的回退分支。
        return
    except ImportError:  # pragma: no cover - exercised only without ijson
        LOGGER.warning("ijson not installed, falling back to in-memory parsing")

    # Fallback: parse the whole document in memory (fine for small files).
    # 回退方案：整体读入内存解析（适用于小文件）。
    with path.open("r", encoding="utf-8") as handle:
        document = json.load(handle)
    for feature in document.get("features", []):
        yield _merge_geometry(feature)
```

```
def _merge_geometry(feature: Dict[str, Any]) -> Dict[str, Any]:
    """Copy geometry coordinates into the properties dict when they are missing.

    In practice the geometry always overrides the properties: a GeoJSON Point
    geometry is ``{"type": "Point", "coordinates": [lon, lat]}`` and is the
    authoritative position, whereas ``latitude``/``longitude`` properties may
    be stale or rounded. Features without a usable geometry keep whatever
    coordinate properties they have.

    Parameters
    ----------
    feature:
        One GeoJSON Feature object (``dict``).

    Returns
    -------
    dict
        A new dictionary (the input is not modified) with the feature
        properties plus ``latitude`` and ``longitude``.

    将几何坐标写入属性字典；几何坐标优先（GeoJSON 顺序为 经度, 纬度）。
    """
    # ``or {}`` handles both a missing key and an explicit JSON null.
    # ``or {}`` 同时处理缺失键与 JSON null。
    props: Dict[str, Any] = dict(feature.get("properties") or {})
    geometry = feature.get("geometry") or {}
    coords = geometry.get("coordinates")
    if isinstance(coords, (list, tuple)) and len(coords) >= 2:
        props.setdefault("longitude", coords[0])
        props.setdefault("latitude", coords[1])
        # GeoJSON stores (lon, lat) - the geometry is authoritative.
        # The assignments below overwrite any property values set above.
        # GeoJSON 坐标顺序为 (经度, 纬度)，以几何为准并覆盖属性中的值。
        props["longitude"] = coords[0]
        props["latitude"] = coords[1]
    return props


def load_geojson(path: Path | str, limit: Optional[int] = None) -> pd.DataFrame:
    """Read a GeoJSON point FeatureCollection into a DataFrame.

    Parameters
    ----------
    path:
        GeoJSON file.
    limit:
        If given, only the first ``limit`` features are read (quick
        experiments on a large corpus).

    Returns
    -------
    pandas.DataFrame
        One row per feature; columns are the union of all property names plus
        ``latitude`` and ``longitude``. Column names are *not* normalised here.

    Raises
    ------
    DataLoadError
        If the collection contains no features.

    读取 GeoJSON 点集合为 DataFrame，可用 limit 限制读取数量。
    """
    rows: List[Dict[str, Any]] = []
    for i, props in enumerate(iter_geojson_features(path)):
        if limit is not None and i >= limit:
            break
        rows.append(props)
    if not rows:
        raise DataLoadError(f"No features found in {path}")
    return pd.DataFrame(rows)


# ---------------------------------------------------------------------------- #
# CSV / JSON                                                                   #
# ---------------------------------------------------------------------------- #
def load_csv(path_or_buffer: Any) -> pd.DataFrame:
    """Read a CSV file of route points, normalising the coordinate columns.

    Parameters
```

```
    ----------
    path_or_buffer:
        File path or text buffer accepted by :func:`pandas.read_csv`.

    Returns
    -------
    pandas.DataFrame
        Frame with canonical ``latitude``/``longitude`` columns (see
        :func:`_normalise_columns`).

    Raises
    ------
    DataLoadError
        If no latitude/longitude column can be identified.

    读取 CSV 并统一坐标列名。
    """
    frame = pd.read_csv(path_or_buffer)
    return _normalise_columns(frame)


def load_json(path_or_buffer: Any) -> pd.DataFrame:
    """Read a JSON document: either a FeatureCollection or a list of records.

    Parameters
    ----------
    path_or_buffer:
        File path, or any object with a ``read`` method (e.g.
        :class:`io.StringIO` holding an uploaded file).

    Returns
    -------
    pandas.DataFrame
        Normalised frame, one row per point.

    Raises
    ------
    DataLoadError
        If the document is neither a FeatureCollection nor a list, or if it
        contains no points, or if coordinate columns are missing.

    读取 JSON：支持 FeatureCollection 或记录数组两种结构（整体读入内存）。
    """
    # Duck typing: anything with ``read`` is treated as an open file object.
    # 鸭子类型：具有 read 方法的对象视为已打开的文件。
    if hasattr(path_or_buffer, "read"):
        document = json.load(path_or_buffer)
    else:
        with open(path_or_buffer, "r", encoding="utf-8") as handle:
            document = json.load(handle)
    if isinstance(document, dict) and document.get("type") == "FeatureCollection":
        rows = [_merge_geometry(f) for f in document.get("features", [])]
    elif isinstance(document, list):
        # A plain array of records: [{"latitude": .., "longitude": ..}, ...]
        # 普通记录数组。
        rows = document
    else:
        raise DataLoadError("Unsupported JSON structure for AIS data")
    if not rows:
        raise DataLoadError("JSON document contains no points")
    return _normalise_columns(pd.DataFrame(rows))


def load_any(source: Path | str, content: Optional[bytes] = None) -> pd.DataFrame:
    """Dispatch on the file extension. ``content`` allows in-memory uploads.

    Parameters
    ----------
    source:
        File path, or, when ``content`` is given, only the original file name
        (used to pick the format).
    content:
        Raw bytes of an uploaded file. When present the file system is not
        touched.

    Returns
    -------
    pandas.DataFrame
        Normalised frame with ``latitude``/``longitude`` columns. Call
        :func:`validate` afterwards to drop invalid rows.
```

```
    Raises
    ------
    DataLoadError
        Unsupported extension or content that cannot be parsed.

    按文件扩展名选择读取方式；content 参数用于处理网页上传的内存数据。
    """
    name = str(source).lower()
    if content is not None:
        # Uploaded bytes. "utf-8-sig" strips a byte-order mark that some
        # spreadsheet programs write at the start of CSV/JSON exports;
        # errors="replace" keeps a single bad byte from rejecting the file.
        # 上传的字节流：utf-8-sig 去除 BOM；errors="replace" 容忍个别非法字节。
        buffer: Any
        if name.endswith(".csv"):
            buffer = io.StringIO(content.decode("utf-8-sig", errors="replace"))
            return load_csv(buffer)
        # Every other upload (.json, .geojson) is parsed as JSON; load_json
        # recognises FeatureCollections itself.
        # 其他上传（.json/.geojson）一律按 JSON 解析，load_json 可识别 FeatureCollection。
        buffer = io.StringIO(content.decode("utf-8-sig", errors="replace"))
        return load_json(buffer)
    if name.endswith(".csv"):
        return load_csv(source)
    if name.endswith((".geojson", ".json")):
        # Try the streaming GeoJSON reader first; if the file is a plain JSON
        # array (no "features"), it raises DataLoadError and load_json is used.
        # 先尝试流式 GeoJSON 读取；若为普通 JSON 数组则回退到 load_json。
        try:
            return _normalise_columns(load_geojson(source))
        except DataLoadError:
            return load_json(source)
    raise DataLoadError(f"Unsupported file type: {source}")


# ---------------------------------------------------------------------------- #
# Normalisation & validation                                                   #
# ---------------------------------------------------------------------------- #
def _first_present(frame: pd.DataFrame, aliases: Sequence[str]) -> Optional[str]:
    """Return the first column of ``frame`` that matches one of ``aliases``.

    Exact matches are tried first (in alias order), then a case-insensitive
    match, so ``"LaT"`` is still recognised as latitude.

    Parameters
    ----------
    frame:
        Input frame.
    aliases:
        Candidate column names in order of preference.

    Returns
    -------
    str or None
        The actual column name in ``frame``, or ``None`` if nothing matches.

    按别名顺序查找列名：先精确匹配，再忽略大小写匹配。
    """
    for alias in aliases:
        if alias in frame.columns:
            return alias
    # Map lower-cased names back to the real column names. 小写名 -> 实际列名。
    lowered = {c.lower(): c for c in frame.columns}
    for alias in aliases:
        if alias.lower() in lowered:
            return lowered[alias.lower()]
    return None


def _normalise_columns(frame: pd.DataFrame) -> pd.DataFrame:
    """Rename coordinate aliases and coerce numeric dtypes.

    Steps: rename the detected coordinate columns to ``latitude`` /
    ``longitude``; convert :data:`NUMERIC_COLUMNS` to numbers (bad values ->
    NaN); add missing OSM count columns filled with 0 (no objects known); add
    an empty ``noaa_type`` column. After this every downstream function can
    rely on the same column set.

    Parameters
```

```
    ----------
    frame:
        Raw frame from any reader.

    Returns
    -------
    pandas.DataFrame
        A normalised copy; the input frame is not modified.

    Raises
    ------
    DataLoadError
        If no latitude or no longitude column can be found.

    统一列名与数据类型，并补齐缺失的 OSM 计数列（填 0）与 noaa_type 列。
    """
    frame = frame.copy()
    lat_col = _first_present(frame, LAT_ALIASES)
    lon_col = _first_present(frame, LON_ALIASES)
    if lat_col is None or lon_col is None:
        # Show at most 12 column names so the error message stays readable.
        # 最多列出 12 个列名，保持错误信息简洁。
        raise DataLoadError(
            "Input must provide latitude and longitude columns "
            f"(got: {list(frame.columns)[:12]})"
        )
    frame = frame.rename(columns={lat_col: "latitude", lon_col: "longitude"})
    for column in NUMERIC_COLUMNS:
        if column in frame.columns:
            # errors="coerce": text such as "n/a" becomes NaN instead of failing.
            # 无法解析的文本转为 NaN，而不是抛出异常。
            frame[column] = pd.to_numeric(frame[column], errors="coerce")
    for column in OSM_COLUMNS:
        if column not in frame.columns:
            frame[column] = 0.0
    if "noaa_type" not in frame.columns:
        frame["noaa_type"] = None
    return frame


def validate(frame: pd.DataFrame, source: str = "upload") -> tuple[pd.DataFrame,
LoadReport]:
    """Drop invalid rows and build a :class:`LoadReport`.

    校验经纬度范围并剔除缺失/越界的记录。

    A row is kept when both coordinates are present and inside the valid WGS84
    range: latitude in [-90, 90] degrees, longitude in [-180, 180] degrees.
    If the frame is labelled, unknown class codes are reported as a warning
    (they are not removed here; :func:`~maritime_route.data.dataset.prepare`
    filters them out).

    Parameters
    ----------
    frame:
        Normalised frame (output of one of the loaders).
    source:
        Description of the input, copied into the report.

    Returns
    -------
    (pandas.DataFrame, LoadReport)
        The valid rows (index reset to 0..n-1) and the report.

    Raises
    ------
    DataLoadError
        If a required column is missing or no valid row remains.
    """
    n_in = len(frame)
    warnings: List[str] = []
    for column in REQUIRED_COLUMNS:
        if column not in frame.columns:
            raise DataLoadError(f"Missing required column '{column}'")

    # Boolean row mask: both coordinates present and within the WGS84 range
    # (``between`` is inclusive on both ends).
    # 行掩码：坐标非空且位于合法范围内（between 两端均包含）。
    mask = frame["latitude"].notna() & frame["longitude"].notna()
    mask &= frame["latitude"].between(-90.0, 90.0)
```

```
    mask &= frame["longitude"].between(-180.0, 180.0)
    clean = frame.loc[mask].reset_index(drop=True)
    dropped = n_in - len(clean)
    if dropped:
        warnings.append(f"{dropped} row(s) dropped: missing or out-of-range
coordinates")
    if clean.empty:
        raise DataLoadError("No valid coordinates remain after validation")

    # The data counts as labelled if at least one row carries a class code.
    # 只要至少一行带有类别代码即视为带标签数据。
    labelled = "class_code" in clean.columns and clean["class_code"].notna().any()
    if labelled:
        unknown = sorted(set(clean["class_code"].dropna().unique()) - set(CLASS_NAMES))
        if unknown:
            warnings.append(f"Unknown class codes ignored: {unknown}")

    report = LoadReport(
        source=source,
        n_features=n_in,
        n_valid=len(clean),
        n_dropped=dropped,
        labelled=bool(labelled),
        columns=list(clean.columns),
        warnings=warnings,
    )
    return clean, report


def to_coordinate_array(frame: pd.DataFrame) -> np.ndarray:
    """Return an ``(n, 2)`` float array of (latitude, longitude).

    Parameters
    ----------
    frame:
        Frame with ``latitude`` and ``longitude`` columns.

    Returns
    -------
    numpy.ndarray
        Shape ``(n, 2)``, dtype float64, degrees; column 0 is latitude.

    返回 (n, 2) 的 (纬度, 经度) 数组，单位为度。
    """
    return frame[["latitude", "longitude"]].to_numpy(dtype=float)
```

data/cleaning.py – removal of mislabelled points, sea regions

```
"""Cleaning of the labelled corpus before training.

The labelled corpus contains points whose class contradicts the physical
measurements stored with them. The most frequent case, pointed out by the
project supervisor, is a ``COASTAL_SEA`` (sea) point that lies several metres
*above* sea level; the symmetric case is a land point (``NEAR_COAST`` or
``COASTLINE``) that lies below the sea surface. Such points teach the network
something that is physically impossible, so they are removed before the
dataset is split and the model is trained.

Every point carries two independent height measurements, both in metres,
positive above sea level and negative below it:

* ``noaa_depth`` - the NOAA/ETOPO relief value;
* ``elevation`` - the terrain elevation recorded by the data collector.

Each measurement is checked separately and a point is removed if *either* of
them contradicts the class. A tolerance
(:data:`~maritime_route.config.LABEL_CHECK_TOLERANCE_M`, 5 m by default)
absorbs tides and the vertical error of the relief models, so that a beach
point at +1 m labelled as coastal sea is *not* flagged.

=====================  ============================================
Rule                    Condition (for each measurement h)
=====================  ============================================
``sea_above_level``     class OPEN_SEA or COASTAL_SEA and h > +tol
``land_below_level``    class NEAR_COAST or COASTLINE and h < -tol
=====================  ============================================

With the 5 m tolerance the rule removes 5 427 of the 156 838 points of the
```

```
corpus delivered with the Aegean Sea extension and leaves 151 411 points,
exactly the number of "clean" points stated by the supervisor for that
corpus. Using ``noaa_depth`` alone would leave 332 more points.

A missing measurement is simply not checked; a point without any
measurement is kept, and the feature extractor marks the missing value
explicitly.

Main objects
------------
:func:`remove_label_conflicts`
    Entry point called by :func:`maritime_route.data.dataset.prepare`.
:func:`conflict_mask`
    The rule itself, as a boolean mask (also used for the sensitivity table).
:func:`assign_regions`
    Maps points to the eleven sea regions of
    :data:`~maritime_route.config.REGIONS`; also used for per-region
    statistics and evaluation.
:class:`CleaningReport`
    Counts written to ``dataset_statistics.json``.

数据清洗：删除类别与实测高程/水深相矛盾的样本（例如位于海平面以上数米的
"近岸海"点，或位于海平面以下的陆地点）。noaa_depth 与 elevation 两个测量值分别检查，
任意一个矛盾即删除；容差 5 米用于吸收潮汐和高程模型误差。新语料清洗后剩 151 411 点，
与指导老师给出的数字一致。
"""
from __future__ import annotations

from dataclasses import dataclass, field
from typing import Dict, Tuple

import numpy as np
import pandas as pd

from ..config import (
    CLASS_NAMES,
    LABEL_CHECK_TOLERANCE_M,
    LAND_CLASSES,
    REGIONS,
    WATER_CLASSES,
)

#: Name used for points that fall outside every region box. 不属于任何海域的点。
OTHER_REGION = "Other (inland samples)"


@dataclass
class CleaningReport:
    """Outcome of :func:`remove_label_conflicts`.

    Attributes
    ----------
    n_before, n_after:
        Number of points before and after cleaning.
    tolerance_m:
        Tolerance that was applied, metres.
    removed_by_class:
        Number of removed points per class (the rule that fired is implied by
        the class: a water class means "above sea level", a land class means
        "below sea level").
    removed_by_region:
        Number of removed points per geographic region.
    removed_by_measure:
        Which measurement revealed the conflict: ``noaa_depth_only``,
        ``elevation_only`` or ``both``.
    sensitivity:
        How many points *would* be removed at other tolerances; reported in
        the thesis to justify the chosen value.

    清洗报告：清洗前后点数、按类别与海域统计的删除数量，以及不同容差下的敏感性分析。
    """

    n_before: int
    n_after: int
    tolerance_m: float
    removed_by_class: Dict[str, int] = field(default_factory=dict)
    removed_by_region: Dict[str, int] = field(default_factory=dict)
    removed_by_measure: Dict[str, int] = field(default_factory=dict)
    sensitivity: Dict[str, int] = field(default_factory=dict)
```

```
    @property
    def n_removed(self) -> int:
        """Total number of removed points. 删除总数。"""
        return self.n_before - self.n_after

    def to_dict(self) -> Dict:
        """JSON-serialisable form, stored in ``dataset_statistics.json``.

        Returns
        -------
        dict
            All fields plus the derived ``n_removed`` and ``share_removed``
            (fraction 0..1 of the input that was removed, 6 decimals).

        转换为可 JSON 序列化的字典，附加删除总数与删除比例。
        """
        return {
            "n_before": self.n_before,
            "n_after": self.n_after,
            "n_removed": self.n_removed,
            # max(.., 1) avoids division by zero for an empty corpus. 防止空语料除零。
            "share_removed": round(self.n_removed / max(self.n_before, 1), 6),
            "tolerance_m": self.tolerance_m,
            "removed_by_class": self.removed_by_class,
            "removed_by_region": self.removed_by_region,
            "removed_by_measure": self.removed_by_measure,
            "sensitivity": self.sensitivity,
        }


def assign_regions(latitude: np.ndarray, longitude: np.ndarray) -> np.ndarray:
    """Return the region name of every point.

    The boxes of :data:`~maritime_route.config.REGIONS` are tested in order and
    the first box that contains the point wins, so overlapping boxes are
    resolved deterministically.

    Parameters
    ----------
    latitude, longitude:
        Arrays of equal length, degrees.

    Returns
    -------
    numpy.ndarray of str
        One region name per point, :data:`OTHER_REGION` where no box matches.

    为每个点分配所属海域：按顺序检查各经纬度范围，先匹配者优先。
    """
    lat = np.asarray(latitude, dtype=float)
    lon = np.asarray(longitude, dtype=float)
    # dtype=object stores Python strings of any length (a fixed-width NumPy
    # string dtype would truncate the longer region names).
    # 使用 object 类型存放任意长度的字符串（定长字符串会截断较长的海域名）。
    out = np.full(lat.shape, OTHER_REGION, dtype=object)
    # Points not yet claimed by an earlier box. 尚未被前面范围匹配的点。
    unassigned = np.ones(lat.shape, dtype=bool)
    for name, (lat_min, lat_max, lon_min, lon_max) in REGIONS:
        # Half-open box [min, max): a point on a shared border belongs to one box only.
        # 半开区间 [min, max)：位于公共边界上的点只属于一个范围。
        inside = (lat >= lat_min) & (lat < lat_max) & (lon >= lon_min) & (lon < lon_max)
        hit = inside & unassigned
        out[hit] = name
        unassigned &= ~hit
    return out


#: Height measurements checked against the class, in this order.
#: 与类别对照检查的两个高度测量值。
MEASUREMENTS: Tuple[str, ...] = ("noaa_depth", "elevation")


def measurement_conflicts(frame: pd.DataFrame, column: str,
                          tolerance_m: float = LABEL_CHECK_TOLERANCE_M) -> np.ndarray:
    """Boolean mask of points whose class contradicts one height measurement.

    Parameters
    ----------
    frame:
        Validated corpus with a ``class_code`` column.
```

```
    column:
        Name of the measurement column (``noaa_depth`` or ``elevation``);
        metres, positive above sea level. A missing column or value never
        produces a conflict.
    tolerance_m:
        Allowed deviation from sea level in metres.

    Returns
    -------
    numpy.ndarray of bool
        ``True`` where the measurement contradicts the class.

    单个测量值与类别的矛盾掩码；缺失值不参与判断。
    """
    if column in frame:
        height = pd.to_numeric(frame[column], errors="coerce").to_numpy(dtype=float)
    else:
        height = np.full(len(frame), np.nan)
    cls = frame["class_code"].astype(str).to_numpy()
    # NaN compares False with everything, so a missing value is never flagged.
    # NaN 与任何数比较均为 False，因此缺失值不会被标记。
    with np.errstate(invalid="ignore"):
        sea_above = np.isin(cls, WATER_CLASSES) & (height > tolerance_m)
        land_below = np.isin(cls, LAND_CLASSES) & (height < -tolerance_m)
    return sea_above | land_below


def conflict_mask(frame: pd.DataFrame, tolerance_m: float = LABEL_CHECK_TOLERANCE_M) ->
np.ndarray:
    """Boolean mask of points whose class contradicts their height/depth.

    A point is flagged when *either* measurement of :data:`MEASUREMENTS`
    contradicts its class (see :func:`measurement_conflicts`).

    Parameters
    ----------
    frame:
        Validated corpus with ``class_code`` and the measurement columns.
    tolerance_m:
        Allowed deviation from sea level in metres.

    Returns
    -------
    numpy.ndarray of bool
        ``True`` for every point that should be removed.

    返回类别与高程/水深矛盾的样本掩码（任一测量值矛盾即为 True，表示应删除）。
    """
    mask = np.zeros(len(frame), dtype=bool)
    for column in MEASUREMENTS:
        mask |= measurement_conflicts(frame, column, tolerance_m)
    return mask


def remove_label_conflicts(
    frame: pd.DataFrame,
    tolerance_m: float = LABEL_CHECK_TOLERANCE_M,
) -> Tuple[pd.DataFrame, CleaningReport]:
    """Drop mislabelled points and describe what was dropped.

    Besides the counts per class and per region, the report contains a
    sensitivity table: the number of points that :func:`conflict_mask` would
    flag at tolerances of 0, 2, 5, 10 and 20 m. It shows how strongly the
    result depends on the chosen tolerance.

    Parameters
    ----------
    frame:
        Validated corpus (output of :func:`~maritime_route.data.ais_loader.validate`).
    tolerance_m:
        Tolerance of the check, metres.

    Returns
    -------
    (pandas.DataFrame, CleaningReport)
        The cleaned frame (index reset) and the report.

    删除标注错误的样本并返回清洗报告。
    """
    by_depth = measurement_conflicts(frame, "noaa_depth", tolerance_m)
```

```
    by_elevation = measurement_conflicts(frame, "elevation", tolerance_m)
    mask = by_depth | by_elevation
    regions = assign_regions(frame["latitude"].to_numpy(),
frame["longitude"].to_numpy())

    # Counts of the removed points by class and by region. 按类别与海域统计删除数。
    removed_cls = frame.loc[mask, "class_code"].value_counts()
    removed_reg = pd.Series(regions[mask]).value_counts()
    # int(...) converts NumPy integers to plain int so the report is JSON-serialisable.
    # 转换为 Python int，便于 JSON 序列化。
    report = CleaningReport(
        n_before=int(len(frame)),
        n_after=int((~mask).sum()),
        tolerance_m=float(tolerance_m),
        removed_by_class={c: int(removed_cls.get(c, 0)) for c in CLASS_NAMES},
        removed_by_region={str(k): int(v) for k, v in removed_reg.items()},
        removed_by_measure={
            "noaa_depth_only": int((by_depth & ~by_elevation).sum()),
            "elevation_only": int((by_elevation & ~by_depth).sum()),
            "both": int((by_depth & by_elevation).sum()),
        },
        sensitivity={f"{t:g} m": int(conflict_mask(frame, t).sum()) for t in (0.0, 2.0,
5.0, 10.0, 20.0)},
    )
    cleaned = frame.loc[~mask].reset_index(drop=True)
    return cleaned, report
```

data/features.py – feature engineering and the reference index

```
"""Feature engineering for the geographic-zone classifier.

特征工程模块。
Two feature sources are combined for every geographic point:

1. **Geodetic encoding** - latitude/longitude expressed in a form that has no
   artificial discontinuity at the antimeridian.
2. **Environmental context** - bathymetry/orography statistics and OSM object
   counts sampled from a reference index around the query point.

The reference index (:class:`ReferenceIndex`) is built once from the training
corpus and is *label free*: it only stores depth, elevation and OSM counts, so
using it at inference time introduces no target leakage. It plays the role that
a NOAA bathymetry raster and an OpenStreetMap extract would play in a
production deployment.

Role in the pipeline
--------------------
* Training: :func:`maritime_route.data.dataset.prepare` builds the index from
  the training split, then calls :func:`build_features` for the training,
  validation and test points.
* Inference: :class:`maritime_route.model.inference.ZoneClassifierService`
  loads the saved index and calls :func:`build_features` (uploaded points
  with measurements) or :func:`features_for_coordinates` (bare grid cells of
  the route planner).

The output is always an ``(n, 28)`` float32 matrix whose columns are listed in
:data:`FEATURE_NAMES`. The network never sees raw values; the matrix is
standardised by the scaler fitted in :mod:`maritime_route.model.trainer`.

Units used throughout: coordinates in degrees (radians only inside the
BallTree), distances in km, depth/elevation in metres with the NOAA sign
convention (positive = above sea level, negative = below).

流水线位置：训练时由 dataset.prepare 调用，推理时由 ZoneClassifierService 调用。
输出恒为 (n, 28) float32 矩阵。单位：坐标为度（BallTree 内为弧度），距离为 km，
水深/高程为米（正值在海平面以上，负值在海平面以下）。
"""
from __future__ import annotations

import logging
from dataclasses import dataclass
from pathlib import Path
from typing import List, Optional, Sequence, Tuple

import numpy as np
import pandas as pd
from sklearn.neighbors import BallTree
```

```
from ..config import BATHY_NEIGHBOURS, EARTH_RADIUS_KM, OSM_COLUMNS, OSM_RADIUS_KM

#: Two neighbours closer than this are treated as the same location. The index
#: stores float32 coordinates, whose rounding error reaches about half a metre.
#: 小于该距离视为同一位置（float32 存储带来的舍入误差约 0.5 m）。
#: Value: 0.002 km = 2 m, i.e. a few times the float32 rounding error but far
#: below the spacing of distinct corpus points. (Index files written by the
#: current :meth:`ReferenceIndex.save` keep float64 coordinates; the tolerance
#: keeps older float32 files working and costs nothing for new ones.)
#: 取值 2 m：大于 float32 误差、远小于样本间距；当前 save 已保存 float64，
#: 该容差用于兼容旧的 float32 索引文件。
SELF_MATCH_EPS_KM: float = 0.002

LOGGER = logging.getLogger(__name__)

#: Names of the 28 columns of the feature matrix, in column order.
#: The order is part of the trained model (it is stored in the metadata) and
#: must not change without retraining. Formulas, with d = depth (m), e =
#: elevation (m), k = BATHY_NEIGHBOURS neighbours from the reference index:
#:
#: ======  ========================  ========================================
#: col     name                      definition
#: ======  ========================  ========================================
#: 0-3     lat_sin .. lon_cos        sin/cos of latitude and longitude (radians)
#: 4       abs_lat_norm              |lat| / 90, distance from the equator 0..1
#: 5       depth_signed_log          slog(d); d replaced by ctx mean if missing
#: 6       depth_is_missing          1 if the point had no measured depth
#: 7       is_height_sample          1 if noaa_type == "height" (land sample)
#: 8       elevation_signed_log      slog(e); e replaced by filled d if missing
#: 9       elevation_is_missing      1 if the point had no measured elevation
#: 10-13   ctx_depth_mean..max       slog of mean/min/max, log1p of std of the
#:                                   k neighbour depths
#: 14      ctx_depth_range           log1p(max - min) of neighbour depths
#: 15      ctx_land_fraction         share of neighbours with depth > 0
#: 16      ctx_sign_agreement        1 if sign(d) == sign(neighbour mean)
#: 17      ctx_distance_km_log       log1p(distance to nearest neighbour, km)
#: 18      ctx_gradient              slog((max - min) / mean distance), m/km
#: 19-24   osm_* / ctx_osm_*         log1p of grouped OSM counts, own point and
#:                                   neighbour mean (marine infra, natural
#:                                   coast, inland water)
#: 25      osm_industrial_log        log1p(own industrial count)
#: 26      ctx_osm_total             log1p(mean total OSM count of neighbours)
#: 27      ctx_osm_density           log1p(mean total count / span km)
#: ======  ========================  ========================================
#:
#: slog(x) = sign(x) * log(1 + |x|), see :func:`signed_log`.
#: 28 个特征的名称（列顺序为模型的一部分，更改需重新训练）。
FEATURE_NAMES: List[str] = [
    "lat_sin",
    "lat_cos",
    "lon_sin",
    "lon_cos",
    "abs_lat_norm",
    "depth_signed_log",
    "depth_is_missing",
    "is_height_sample",
    "elevation_signed_log",
    "elevation_is_missing",
    "ctx_depth_mean",
    "ctx_depth_std",
    "ctx_depth_min",
    "ctx_depth_max",
    "ctx_depth_range",
    "ctx_land_fraction",
    "ctx_sign_agreement",
    "ctx_distance_km_log",
    "ctx_gradient",
    "osm_marine_infra",
    "ctx_osm_marine_infra",
    "osm_natural_coastal",
    "ctx_osm_natural_coastal",
    "osm_inland_water",
    "ctx_osm_inland_water",
    "osm_industrial_log",
    "ctx_osm_total",
    "ctx_osm_density",
]
#: Width of the feature matrix (28). 特征维数。
N_FEATURES: int = len(FEATURE_NAMES)
```

```
#: OSM groups. 海事基础设施 / 自然海岸 / 内陆水体。
#: The nine raw OSM columns are summed into three semantically meaningful
#: groups; single object types are too sparse to be informative on their own.
#: 九个原始 OSM 列合并为三组，单一类型过于稀疏。
MARINE_INFRA = ["osm_piers", "osm_lighthouses", "osm_breakwaters", "osm_harbors"]
NATURAL_COASTAL = ["osm_beaches", "osm_cliffs"]
INLAND_WATER = ["osm_rivers", "osm_lakes"]


def signed_log(values: np.ndarray) -> np.ndarray:
    """``sign(x) * log1p(|x|)`` - compresses the huge bathymetric range.

    Relief values span roughly -11 000 m (ocean trenches) to +8 800 m, while
    the class boundaries lie within a few metres of zero. The signed logarithm
    keeps the sign (land vs water), is continuous and monotone through zero
    (``log1p(0) = 0``), and maps the full range to about [-9.3, +9.1], so
    that small depths near the coast stay distinguishable after
    standardisation. Example: -4000 m -> -8.29, -10 m -> -2.40, +3 m -> +1.39.

    Parameters
    ----------
    values:
        Array of any shape, metres (or any signed quantity).

    Returns
    -------
    numpy.ndarray
        Same shape, float64.

    带符号对数变换：保留正负号（陆地/水域），压缩数千米的水深范围，
    同时保持近海平面处的分辨率。
    """
    values = np.asarray(values, dtype=float)
    return np.sign(values) * np.log1p(np.abs(values))


# ---------------------------------------------------------------------------- #
# Reference index                                                              #
# ---------------------------------------------------------------------------- #
@dataclass
class ReferenceIndex:
    """Label-free spatial index over environmental measurements.

    参考索引：仅保存水深/高程/OSM 计数，不含类别标签，因此不会造成标签泄漏。
    Queried with a haversine :class:`~sklearn.neighbors.BallTree`.

    A BallTree partitions the points into nested hyperspheres, so a
    k-nearest-neighbour query costs about O(log n) instead of O(n). With
    ``metric="haversine"`` it works on the sphere: inputs are
    ``[latitude, longitude]`` in **radians** and returned distances are
    central angles in radians, converted to km by multiplying with
    :data:`~maritime_route.config.EARTH_RADIUS_KM`. This avoids the distortion
    of a Euclidean distance on raw degrees (one degree of longitude shrinks
    with cos(latitude)).

    Attributes
    ----------
    coordinates_rad:
        ``(n, 2)`` float64, radians, columns (lat, lon).
    depth:
        ``(n,)`` relief value, metres, positive above sea level.
    elevation:
        ``(n,)`` terrain elevation, metres.
    osm_counts:
        ``(n, 9)`` OSM object counts in the order of
        :data:`~maritime_route.config.OSM_COLUMNS`.
    tree:
        The BallTree; built automatically in :meth:`__post_init__` when not
        supplied.

    BallTree 采用 haversine 距离：输入为弧度制 (纬度, 经度)，返回球面角距离（弧度），
    乘以地球半径得到公里。
    """

    coordinates_rad: np.ndarray  # (n, 2) radians, (lat, lon)
    depth: np.ndarray            # (n,) metres, positive = above sea level
    elevation: np.ndarray        # (n,) metres
    osm_counts: np.ndarray       # (n, len(OSM_COLUMNS))
    tree: Optional[BallTree] = None
```

```
    def __post_init__(self) -> None:
        """Build the haversine BallTree if none was passed in.

        Building takes O(n log n) and is done once per index (at training time
        and when the inference service starts). The tree is not saved to disk;
        it is rebuilt by :meth:`load`.
        若未提供则构建 BallTree（O(n log n)，每个索引只构建一次，不写入磁盘）。
        """
        if self.tree is None:
            self.tree = BallTree(self.coordinates_rad, metric="haversine")

    # -- construction ------------------------------------------------------ #
    @classmethod
    def from_frame(cls, frame: pd.DataFrame) -> "ReferenceIndex":
        """Build an index from a frame of measured points.

        The ``class_code`` column, if present, is ignored, which is what makes
        the index label-free. ``noaa_depth`` and ``elevation`` describe the
        same relief surface, so a gap in one is filled from the other; points
        with neither value are dropped because they add no information.

        Parameters
        ----------
        frame:
            Points with ``latitude``, ``longitude`` (degrees) and optionally
            ``noaa_depth``, ``elevation`` and the OSM count columns. At
            training time this is the training split only.

        Returns
        -------
        ReferenceIndex
            Index over the points that have a depth or elevation value.

        从 DataFrame 构建索引：忽略类别标签；水深与高程互相补缺；两者皆缺的点被丢弃。
        """
        # (n, 2) degrees -> radians, as required by the haversine BallTree.
        # 度 -> 弧度（haversine BallTree 的要求）。
        coords = np.radians(frame[["latitude", "longitude"]].to_numpy(dtype=float))
        depth = pd.to_numeric(frame.get("noaa_depth"),
errors="coerce").to_numpy(dtype=float)
        elevation = pd.to_numeric(frame.get("elevation"),
errors="coerce").to_numpy(dtype=float)
        # Fill gaps in one source from the other; both encode the same surface.
        # 两个数据源描述同一地表，互相填补缺失值。
        depth = np.where(np.isnan(depth), elevation, depth)
        elevation = np.where(np.isnan(elevation), depth, elevation)
        # (n, 9) matrix of OSM counts; a missing column becomes all zeros.
        # (n, 9) OSM 计数矩阵；缺失列视为 0。
        osm = np.stack(
            [pd.to_numeric(frame.get(c, 0.0),
errors="coerce").fillna(0.0).to_numpy(float)
             for c in OSM_COLUMNS],
            axis=1,
        )
        # Keep only points that have some relief value. 仅保留有高程/水深的点。
        keep = ~np.isnan(depth)
        return cls(coords[keep], depth[keep], np.nan_to_num(elevation[keep]), osm[keep])

    # -- persistence ------------------------------------------------------- #
    def save(self, path: Path | str) -> None:
        """Write the index arrays to a compressed ``.npz`` file.

        Measurements are stored as float32 (half the size; centimetre
        precision is irrelevant for relief values in metres). The BallTree is
        not stored and is rebuilt on :meth:`load`.

        Parameters
        ----------
        path:
            Target file, normally :data:`~maritime_route.config.BATHY_INDEX_PATH`.

        保存为压缩 npz：测量值用 float32，坐标保持 float64；BallTree 不保存。
        """
        np.savez_compressed(
            path,
            # Coordinates keep full precision: rounding them to float32
            # would displace a point from itself by up to ~1.5 m.
            # 坐标保持双精度：float32 会使点与自身产生最多约 1.5 m 的偏差。
            coordinates_rad=self.coordinates_rad.astype(np.float64),
            depth=self.depth.astype(np.float32),
```

```
            elevation=self.elevation.astype(np.float32),
            osm_counts=self.osm_counts.astype(np.float32),
        )

    @classmethod
    def load(cls, path: Path | str) -> "ReferenceIndex":
        """Read an index written by :meth:`save` and rebuild its BallTree.

        All arrays are converted to float64 so that the arithmetic in
        :func:`build_features` is identical to the one done at training time.

        Parameters
        ----------
        path:
            ``.npz`` file written by :meth:`save`.

        Returns
        -------
        ReferenceIndex
            The loaded index.

        读取索引并重建 BallTree；统一转换为 float64 以保证与训练时计算一致。
        """
        blob = np.load(path)
        return cls(
            coordinates_rad=blob["coordinates_rad"].astype(np.float64),
            depth=blob["depth"].astype(np.float64),
            elevation=blob["elevation"].astype(np.float64),
            osm_counts=blob["osm_counts"].astype(np.float64),
        )

    # -- queries ------------------------------------------------------------ #
    def query(
        self,
        lat: np.ndarray,
        lon: np.ndarray,
        k: int = BATHY_NEIGHBOURS,
        exclude_self: bool = False,
    ) -> Tuple[np.ndarray, np.ndarray]:
        """Return ``(distance_km, neighbour_index)`` arrays of shape (n, k).

        The query point itself is always excluded, and the decision is taken by
        distance rather than by a flag. This matters because the index stores
        coordinates as float32: a point that *is* in the index matches itself at
        a distance of roughly half a metre rather than exactly zero, so an
        equality test would miss it. Dropping by distance makes the features of
        a point identical whether or not that point happens to be part of the
        index, which is what keeps training and inference consistent.

        查询点自身总是被剔除，且依据距离而非标志位判断：索引以 float32 存储
        坐标，点与自身的距离约为 0.5 m 而非严格为 0，用相等判断会漏掉。
        按距离剔除可保证"点是否在索引中"不影响其特征，从而使训练与推理一致。

        ``exclude_self`` is retained for backward compatibility and no longer
        changes the result.

        Algorithm: ask the BallTree for ``k + 1`` neighbours. If the nearest
        one is within :data:`SELF_MATCH_EPS_KM` it is the query point itself
        and is dropped; otherwise the farthest of the ``k + 1`` is dropped.
        Either way exactly ``k`` neighbours remain, sorted by distance.

        Parameters
        ----------
        lat, lon:
            Query coordinates, degrees, arrays of length n.
        k:
            Number of neighbours wanted. Clipped to [1, number of indexed
            points].
        exclude_self:
            Ignored (see above).

        Returns
        -------
        (numpy.ndarray, numpy.ndarray)
            ``distance_km`` - shape (n, k), float64, great-circle distance in
            km, ascending along axis 1; ``neighbour_index`` - shape (n, k),
            int, row indices into ``depth``/``elevation``/``osm_counts``.
        """
        # (n, 2) radians, columns (lat, lon) - the BallTree's input convention.
        # (n, 2) 弧度 (纬度, 经度)，BallTree 的输入格式。
```

```
        query_rad = np.radians(np.stack([np.asarray(lat, float), np.asarray(lon,
float)], axis=1))
        n_samples = len(self.depth)
        # Cannot ask for more neighbours than there are points in the index.
        # 请求的邻居数不能超过索引中的点数。
        k = max(1, min(k, n_samples))
        # One spare neighbour, so that the point itself can be discarded.
        # 多取一个邻居，以便剔除查询点自身。
        k_eff = min(k + 1, n_samples)
        dist, idx = self.tree.query(query_rad, k=k_eff)
        # Haversine distances are central angles (radians); arc length = R * angle.
        # haversine 距离为圆心角（弧度），弧长 = 地球半径 × 角度。
        dist = dist * EARTH_RADIUS_KM

        if k_eff > k:
            # Drop the first neighbour when it is the query point itself,
            # otherwise drop the farthest one; either way exactly k remain.
            # 若最近邻即自身则去掉第一个，否则去掉最远的一个，始终保留 k 个。
            # is_self has shape (n, 1) so it broadcasts over the k columns.
            is_self = (dist[:, 0] <= SELF_MATCH_EPS_KM)[:, None]
            dist = np.where(is_self, dist[:, 1:], dist[:, :-1])
            idx = np.where(is_self, idx[:, 1:], idx[:, :-1])
        return dist, idx

    def sample_depth(self, lat: np.ndarray, lon: np.ndarray, k: int = 4) -> np.ndarray:
        """Inverse-distance weighted depth at arbitrary coordinates.

        反距离加权插值，用于为航线网格的任意单元估计水深。

        Inverse-distance weighting (IDW) with power 2::

            w_i = 1 / max(d_i, 0.001 km)^2
            depth = sum(w_i * depth_i) / sum(w_i)

        The 1 m floor on the distance prevents division by zero when a query
        coincides with a reference point. Power 2 makes the nearest
        neighbours dominate, which suits a surface that changes quickly at the
        coast.

        Parameters
        ----------
        lat, lon:
            Query coordinates, degrees, length n.
        k:
            Number of neighbours used for the interpolation.

        Returns
        -------
        numpy.ndarray
            ``(n,)`` interpolated relief, metres (positive above sea level).
        """
        dist_km, idx = self.query(lat, lon, k=k)
        weights = 1.0 / np.maximum(dist_km, 1e-3) ** 2   # (n, k), 1/km^2
        values = self.depth[idx]                          # (n, k), metres
        return np.sum(values * weights, axis=1) / np.sum(weights, axis=1)


# ---------------------------------------------------------------------------- #
# Feature matrix                                                               #
# ---------------------------------------------------------------------------- #
def build_features(
    frame: pd.DataFrame,
    index: ReferenceIndex,
    exclude_self: bool = False,
) -> np.ndarray:
    """Compute the ``(n, N_FEATURES)`` design matrix for a frame of points.

    构建特征矩阵。``exclude_self=True`` 用于训练集，避免点查询到自身。
    (Since :meth:`ReferenceIndex.query` now always removes a self-match by
    distance, the flag is passed through but no longer changes the result.)
    （目前 query 总是按距离剔除自身，该参数已不影响结果。）

    Feature groups (see :data:`FEATURE_NAMES` for the full table):

    * **Geodetic** - latitude and longitude are encoded as sin/cos pairs. A raw
      longitude jumps from +180 to -180 at the antimeridian although the two
      points are neighbours; (sin, cos) places the angle on the unit circle,
      so neighbours stay close. ``|lat| / 90`` adds a climate-zone proxy.
    * **Point measurements** - the point's own depth and elevation (signed
      log) plus 0/1 flags that say whether each value was measured or
```

```
      imputed, so the network can learn how much to trust it.
    * **Neighbourhood relief** - statistics of the depths of the
      ``BATHY_NEIGHBOURS`` nearest reference points: mean/min/max (signed
      log), std and range (log1p, both >= 0), the share of land neighbours,
      whether the point's sign agrees with the neighbourhood, the log
      distance to the nearest neighbour and a slope proxy (relief range
      divided by mean neighbour distance, metres per km).
    * **OSM context** - grouped OSM counts of the point and the mean over its
      neighbours, compressed with log1p because counts are heavy-tailed.

    Missing values: a missing depth is replaced by the neighbourhood mean, a
    missing elevation by the (filled) depth; any remaining NaN/inf in the
    final matrix is set to 0.

    Parameters
    ----------
    frame:
        Points with ``latitude``/``longitude`` (degrees) and optionally
        ``noaa_depth``, ``elevation``, ``noaa_type`` and OSM count columns.
    index:
        Reference index built from the training split.
    exclude_self:
        Passed to :meth:`ReferenceIndex.query`; has no effect any more.

    Returns
    -------
    numpy.ndarray
        Shape ``(n, 28)``, dtype float32, columns in :data:`FEATURE_NAMES`
        order. Not standardised (the scaler is applied by the model code).
    """
    lat = frame["latitude"].to_numpy(dtype=float)    # (n,) degrees
    lon = frame["longitude"].to_numpy(dtype=float)   # (n,) degrees
    n = len(lat)

    lat_rad, lon_rad = np.radians(lat), np.radians(lon)

    # Point measurements; a missing column is replaced by an all-NaN series.
    # 点自身的测量值；缺失列以全 NaN 序列代替。
    depth_raw = pd.to_numeric(frame.get("noaa_depth", pd.Series(np.nan,
index=frame.index)),
                              errors="coerce").to_numpy(dtype=float)
    elev_raw = pd.to_numeric(frame.get("elevation", pd.Series(np.nan,
index=frame.index)),
                             errors="coerce").to_numpy(dtype=float)

    # Missing-value indicators (0.0 / 1.0), computed before imputation.
    # 缺失指示变量（0/1），在填补之前计算。
    depth_missing = np.isnan(depth_raw).astype(float)
    elev_missing = np.isnan(elev_raw).astype(float)

    # 1.0 when the NOAA sample is a land height rather than a sea depth.
    # NOAA 样本为陆地高程时取 1。
    noaa_type = frame.get("noaa_type")
    if noaa_type is None:
        is_height = np.zeros(n)
    else:
        # astype("string") gives a nullable string column so None/NaN survive
        # .str.lower(); fillna(False) turns those into "not a height sample".
        # 转为可空字符串类型，None/NaN 最终视为非高程样本。
        is_height = (noaa_type.astype("string").str.lower() ==
"height").fillna(False).to_numpy(float)

    # Contextual statistics from the reference index.
    # 来自参考索引的邻域统计量。
    dist_km, idx = index.query(lat, lon, k=BATHY_NEIGHBOURS, exclude_self=exclude_self)
    ctx_depth = index.depth[idx]                      # (n, k)
    ctx_osm = index.osm_counts[idx]                   # (n, k, 9)

    # Relief statistics over the k neighbours (axis 1), metres. 邻域高程统计（米）。
    ctx_mean = ctx_depth.mean(axis=1)
    ctx_std = ctx_depth.std(axis=1)
    ctx_min = ctx_depth.min(axis=1)
    ctx_max = ctx_depth.max(axis=1)
    # Share of neighbours above sea level, 0..1. 邻居中陆地点的比例。
    ctx_land_fraction = (ctx_depth > 0.0).mean(axis=1)
    nearest_km = dist_km[:, 0]
    # Mean neighbour distance (km), floored at 1 m to avoid division by zero.
    # 邻居平均距离（km），下限 1 m 以避免除零。
    span_km = np.maximum(dist_km.mean(axis=1), 1e-3)
    # Slope proxy: relief range over the neighbourhood size, metres per km.
```

```
    # Steep values indicate a shelf edge or a cliff coast.
    # 坡度近似：邻域高差 / 邻域尺度（米/公里），大值对应陆架边缘或陡峭海岸。
    ctx_gradient = (ctx_max - ctx_min) / span_km

    # Missing point measurements fall back to the interpolated context value.
    # 缺失的点测量值用邻域均值代替；缺失高程再用已填补的水深代替。
    depth_filled = np.where(np.isnan(depth_raw), ctx_mean, depth_raw)
    elev_filled = np.where(np.isnan(elev_raw), depth_filled, elev_raw)
    # 1.0 when the point and its neighbourhood are on the same side of sea
    # level; 0.0 marks points at a land/water transition (np.sign(0) = 0).
    # 点与邻域位于海平面同侧时为 1；为 0 表示处于水陆交界。
    sign_agreement = (np.sign(depth_filled) == np.sign(ctx_mean)).astype(float)

    # OSM aggregation: the point's own counts plus the neighbourhood.
    # OSM 聚合：点自身计数 + 邻域均值。
    own_osm = np.stack(
        [pd.to_numeric(frame.get(c, 0.0), errors="coerce").fillna(0.0).to_numpy(float)
         for c in OSM_COLUMNS],
        axis=1,
    )                                                    # (n, 9)
    col = {name: i for i, name in enumerate(OSM_COLUMNS)}  # OSM column name -> position

    def group(mat: np.ndarray, names: Sequence[str], axis_sum: bool = False) ->
np.ndarray:
        """Sum the OSM columns listed in ``names`` along the last axis.

        Works for both the per-point matrix ``(n, 9) -> (n,)`` and the
        neighbourhood tensor ``(n, k, 9) -> (n, k)`` because the columns are
        selected with ``...`` (ellipsis) indexing.

        Parameters
        ----------
        mat:
            Array whose last axis follows :data:`OSM_COLUMNS`.
        names:
            OSM column names to add up.
        axis_sum:
            Unused; kept for signature compatibility.

        Returns
        -------
        numpy.ndarray
            ``mat`` with the last axis replaced by the group total.

        按组对 OSM 列求和（最后一个轴），同时适用于 (n, 9) 与 (n, k, 9)。
        """
        cols = [col[nm] for nm in names]
        sub = mat[..., cols]
        return sub.sum(axis=-1)

    # log1p compresses the heavy-tailed counts (0 stays 0; 100 -> 4.6).
    # log1p 压缩长尾计数分布（0 仍为 0）。
    own_marine = np.log1p(group(own_osm, MARINE_INFRA))
    own_natural = np.log1p(group(own_osm, NATURAL_COASTAL))
    own_inland = np.log1p(group(own_osm, INLAND_WATER))
    own_industrial = np.log1p(own_osm[:, col["osm_industrial"]])

    # Group totals per neighbour (n, k), averaged over the k neighbours -> (n,).
    # 每个邻居的组合计 (n, k)，再对 k 个邻居取平均 -> (n,)。
    ctx_marine = np.log1p(group(ctx_osm, MARINE_INFRA).mean(axis=1))
    ctx_natural = np.log1p(group(ctx_osm, NATURAL_COASTAL).mean(axis=1))
    ctx_inland = np.log1p(group(ctx_osm, INLAND_WATER).mean(axis=1))
    # All nine OSM types summed per neighbour, then averaged. 全部 OSM 类型的邻域平均总数。
    ctx_osm_total = np.log1p(ctx_osm.sum(axis=2).mean(axis=1))
    # Objects per km of neighbourhood size; the denominator is at least
    # 0.1 * OSM_RADIUS_KM = 1.2 km so that very dense sampling does not
    # produce extreme densities.
    # 每公里对象数；分母至少为 1.2 km，避免密集采样导致极端值。
    ctx_osm_density = ctx_osm.sum(axis=2).mean(axis=1) / np.maximum(span_km,
OSM_RADIUS_KM * 0.1)

    # Assemble the 28 columns in FEATURE_NAMES order -> (n, 28).
    # 按 FEATURE_NAMES 顺序拼接 28 列。
    matrix = np.stack(
        [
            np.sin(lat_rad),                # 0  lat_sin
            np.cos(lat_rad),                # 1  lat_cos
            np.sin(lon_rad),                # 2  lon_sin
            np.cos(lon_rad),                # 3  lon_cos
```

```
            np.abs(lat) / 90.0,              # 4  abs_lat_norm, 0 at equator .. 1 at
pole
            signed_log(depth_filled),        # 5  depth_signed_log
            depth_missing,                   # 6  depth_is_missing
            is_height,                       # 7  is_height_sample
            signed_log(elev_filled),         # 8  elevation_signed_log
            elev_missing,                    # 9  elevation_is_missing
            signed_log(ctx_mean),            # 10 ctx_depth_mean
            np.log1p(ctx_std),               # 11 ctx_depth_std (std >= 0, plain log1p)
            signed_log(ctx_min),             # 12 ctx_depth_min
            signed_log(ctx_max),             # 13 ctx_depth_max
            np.log1p(ctx_max - ctx_min),     # 14 ctx_depth_range (>= 0)
            ctx_land_fraction,               # 15 ctx_land_fraction
            sign_agreement,                  # 16 ctx_sign_agreement
            np.log1p(nearest_km),            # 17 ctx_distance_km_log
            np.log1p(np.abs(ctx_gradient)) * np.sign(ctx_gradient),  # 18 ctx_gradient
(signed log)
            own_marine,                      # 19 osm_marine_infra
            ctx_marine,                      # 20 ctx_osm_marine_infra
            own_natural,                     # 21 osm_natural_coastal
            ctx_natural,                     # 22 ctx_osm_natural_coastal
            own_inland,                      # 23 osm_inland_water
            ctx_inland,                      # 24 ctx_osm_inland_water
            own_industrial,                  # 25 osm_industrial_log
            ctx_osm_total,                   # 26 ctx_osm_total
            np.log1p(ctx_osm_density),       # 27 ctx_osm_density
        ],
        axis=1,
    )
    # Safety net: any NaN/inf left (e.g. from an empty or corrupt input value)
    # becomes 0; float32 halves memory and matches the PyTorch model dtype.
    # 兜底：残留的 NaN/inf 置 0；float32 节省内存并与 PyTorch 模型精度一致。
    return np.nan_to_num(matrix, nan=0.0, posinf=0.0, neginf=0.0).astype(np.float32)


def features_for_coordinates(
    lat: Sequence[float],
    lon: Sequence[float],
    index: ReferenceIndex,
) -> np.ndarray:
    """Build features for bare coordinates (no measured attributes available).

    仅有经纬度时的特征构建：全部环境属性来自参考索引。

    Used for the cells of the routing grid, which have only a position. The
    missing point attributes are synthesised from the 4 nearest reference
    points before calling :func:`build_features`:

    * ``noaa_depth`` and ``elevation`` - inverse-distance-weighted (power 2)
      average of the neighbours, as in :meth:`ReferenceIndex.sample_depth`;
    * ``noaa_type`` - ``"height"`` if the interpolated value is above sea
      level, else ``"depth"``;
    * OSM counts - copied from the single nearest neighbour (counts cannot be
      meaningfully interpolated).

    Because the imputed values are marked as measured (the ``*_is_missing``
    flags are 0), a grid cell looks to the network like an ordinary sample at
    that location.

    Parameters
    ----------
    lat, lon:
        Coordinates, degrees, length n.
    index:
        Reference index.

    Returns
    -------
    numpy.ndarray
        ``(n, 28)`` float32 feature matrix.
    """
    frame = pd.DataFrame({"latitude": np.asarray(lat, float), "longitude":
np.asarray(lon, float)})
    dist_km, idx = index.query(frame["latitude"].to_numpy(),
frame["longitude"].to_numpy(), k=4)
    # IDW weights 1/d^2 with a 1 m distance floor, shape (n, 4).
    # 反距离平方权重，距离下限 1 m，形状 (n, 4)。
    weights = 1.0 / np.maximum(dist_km, 1e-3) ** 2
    frame["noaa_depth"] = np.sum(index.depth[idx] * weights, axis=1) / np.sum(weights,
axis=1)
```

```
    frame["elevation"] = np.sum(index.elevation[idx] * weights, axis=1) /
np.sum(weights, axis=1)
    frame["noaa_type"] = np.where(frame["noaa_depth"] > 0, "height", "depth")
    # OSM counts of the nearest neighbour (column 0 of idx), shape (n, 9).
    # 取最近邻（idx 第 0 列）的 OSM 计数。
    nearest_osm = index.osm_counts[idx[:, 0]]
    for i, column in enumerate(OSM_COLUMNS):
        frame[column] = nearest_osm[:, i]
    return build_features(frame, index, exclude_self=False)
```

data/dataset.py – dataset preparation and splitting

```
"""Dataset preparation: raw GeoJSON -> stratified train/val/test tensors.

数据集准备：原始 GeoJSON -> 分层划分的训练/验证/测试集。

This module runs the offline part of the machine-learning pipeline, invoked by
``scripts/prepare_dataset.py``:

1. load the labelled corpus (:func:`~maritime_route.data.ais_loader.load_geojson`);
2. validate coordinates (:func:`~maritime_route.data.ais_loader.validate`);
3. keep only rows with one of the four known class codes;
4. remove mislabelled points
   (:func:`~maritime_route.data.cleaning.remove_label_conflicts`);
5. split 70 / 15 / 15 into train / validation / test, stratified by class;
6. build the label-free :class:`~maritime_route.data.features.ReferenceIndex`
   from the *training* split only and save it;
7. compute the 28 features of every point and save the matrices, labels,
   test-point regions and corpus statistics.

Outputs
-------
:data:`PROCESSED_NPZ`
    ``x_train``/``x_val``/``x_test`` (float32, shape (n, 28)),
    ``y_train``/``y_val``/``y_test`` (int64 class indices), ``region_test``
    and ``feature_names``. Read back with :func:`load_prepared`.
:data:`STATS_JSON`
    Descriptive statistics of the corpus before and after cleaning, per
    region, the split sizes and the cleaning report (used in the thesis).
:data:`~maritime_route.config.BATHY_INDEX_PATH`
    The reference index needed at inference time.

流程：加载 -> 校验 -> 过滤未知类别 -> 清洗 -> 70/15/15 分层划分 -> 仅用训练集建立
参考索引 -> 提取特征并保存矩阵、标签与统计信息。
"""
from __future__ import annotations

import json
import logging
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, Optional, Tuple

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

from ..config import (
    BATHY_INDEX_PATH,
    CLASS_NAMES,
    CLASS_TO_INDEX,
    PROCESSED_DIR,
    TRAINING,
)
from .ais_loader import load_geojson, validate
from .cleaning import assign_regions, remove_label_conflicts
from .features import FEATURE_NAMES, ReferenceIndex, build_features

LOGGER = logging.getLogger(__name__)

#: Feature matrices and labels of the three splits (compressed NumPy archive).
#: 三个数据子集的特征矩阵与标签。
PROCESSED_NPZ = PROCESSED_DIR / "zone_dataset.npz"
#: Corpus statistics written by :func:`prepare`. 语料统计信息。
STATS_JSON = PROCESSED_DIR / "dataset_statistics.json"


@dataclass
```

```
class SplitData:
    """Container for one train/val/test split. 数据划分容器。

    Attributes
    ----------
    x_train, x_val, x_test:
        Feature matrices, shape ``(n_split, 28)``, float32, *not*
        standardised.
    y_train, y_val, y_test:
        Class indices 0..3 (see :data:`~maritime_route.config.CLASS_NAMES`),
        shape ``(n_split,)``, int64.
    feature_names:
        Column names of the feature matrices, in order.
    """

    x_train: np.ndarray
    y_train: np.ndarray
    x_val: np.ndarray
    y_val: np.ndarray
    x_test: np.ndarray
    y_test: np.ndarray
    feature_names: list

    @property
    def n_features(self) -> int:
        """Number of feature columns (28). 特征维数。"""
        return self.x_train.shape[1]

    def describe(self) -> Dict[str, int]:
        """Return the size of each split and the feature count.

        Returns
        -------
        dict
            ``{"train": .., "validation": .., "test": .., "features": ..}``;
            stored in the statistics file and the model metadata.

        返回各子集样本数与特征维数。
        """
        return {
            "train": int(len(self.y_train)),
            "validation": int(len(self.y_val)),
            "test": int(len(self.y_test)),
            "features": self.n_features,
        }


def compute_statistics(frame: pd.DataFrame) -> Dict:
    """Descriptive statistics reported in chapter 3 of the thesis.

    Parameters
    ----------
    frame:
        Validated corpus; ``class_code``, ``noaa_depth``, ``elevation`` and
        ``source_session`` columns are used when present.

    Returns
    -------
    dict
        JSON-serialisable dictionary with keys:

        * ``n_points`` - number of rows;
        * ``class_distribution`` - count and share per class;
        * ``bbox`` - bounding box of the points, degrees;
        * ``missing`` - number of missing elevation / depth values;
        * ``depth_by_class`` - mean, std and 5 / 50 / 95 % quantiles of
          ``noaa_depth`` per class, metres (shows how well depth alone
          separates the classes);
        * ``n_sessions`` - number of distinct collection sessions.

    描述性统计：样本数、类别分布、范围、缺失值、各类别水深分位数、采集会话数。
    """
    stats: Dict = {
        "n_points": int(len(frame)),
        "class_distribution": {},
        "bbox": {
            "lat_min": float(frame["latitude"].min()),
            "lat_max": float(frame["latitude"].max()),
            "lon_min": float(frame["longitude"].min()),
            "lon_max": float(frame["longitude"].max()),
```

```
        },
        "missing": {
            "elevation": int(frame["elevation"].isna().sum()) if "elevation" in frame
else 0,
            "noaa_depth": int(frame["noaa_depth"].isna().sum()) if "noaa_depth" in frame
else 0,
        },
    }
    if "class_code" in frame.columns:
        counts = frame["class_code"].value_counts()
        for name in CLASS_NAMES:
            n = int(counts.get(name, 0))
            stats["class_distribution"][name] = {
                "count": n,
                # max(.., 1) guards against division by zero on an empty frame.
                # 防止空表时除零。
                "share": round(n / max(len(frame), 1), 6),
            }
    if "noaa_depth" in frame.columns and "class_code" in frame.columns:
        grouped = frame.groupby("class_code")["noaa_depth"]
        stats["depth_by_class"] = {
            str(k): {
                "mean": float(v.mean()),
                "std": float(v.std()),
                "p05": float(v.quantile(0.05)),
                "p50": float(v.quantile(0.50)),
                "p95": float(v.quantile(0.95)),
            }
            for k, v in grouped
        }
    if "source_session" in frame.columns:
        stats["n_sessions"] = int(frame["source_session"].nunique())
    return stats


def prepare(
    raw_path: Path | str,
    output_npz: Path | str = PROCESSED_NPZ,
    index_path: Path | str = BATHY_INDEX_PATH,
    limit: Optional[int] = None,
    seed: int = TRAINING.random_seed,
    clean: bool = True,
) -> Tuple[SplitData, Dict]:
    """Full preparation pipeline. Returns the split and the statistics dict.

    Steps: load the GeoJSON corpus, validate coordinates, remove mislabelled
    points (``clean=True``), split 70/15/15 with stratification by class, build
    the label-free reference index from the training part only, extract the
    28 features of every point and save everything to ``output_npz``.

    Stratification keeps the class proportions equal in the three splits,
    which matters because the classes are imbalanced. Building the reference
    index only from training points ensures that no validation or test point
    contributes its own measurements to its neighbourhood features, so the
    test accuracy is an honest estimate for unseen locations.

    Parameters
    ----------
    raw_path:
        Labelled corpus (GeoJSON FeatureCollection of points).
    output_npz:
        Where the feature matrices are written.
    index_path:
        Where the reference index is written.
    limit:
        Read only the first ``limit`` features (for quick experiments).
    seed:
        Random seed of the split.
    clean:
        Remove points whose class contradicts their height/depth.

    Returns
    -------
    (SplitData, dict)
        The feature matrices / labels and the statistics dictionary (also
        written to :data:`STATS_JSON`).

    Raises
    ------
    ValueError
```

```
        If no row carries a known class code.
    DataLoadError
        If the input file contains no valid points.

    完整预处理流程：加载 -> 校验 -> 清洗 -> 分层划分 -> 建立参考索引 -> 特征提取。
    """
    LOGGER.info("Loading raw dataset from %s", raw_path)
    frame = load_geojson(raw_path, limit=limit)
    frame, report = validate(frame, source=str(raw_path))
    LOGGER.info("Validated %d points (%d dropped)", report.n_valid, report.n_dropped)

    # Training needs a label: drop unlabelled rows and unknown class codes.
    # 训练需要标签：删除无标签或类别未知的行。
    frame = frame[frame["class_code"].isin(CLASS_NAMES)].reset_index(drop=True)
    if frame.empty:
        raise ValueError("Dataset contains no rows with a known class_code")

    # Statistics of the corpus as delivered, then removal of the points whose
    # class contradicts their measured height/depth (labelling errors).
    # 先统计原始语料，再删除类别与高程/水深矛盾的标注错误样本。
    raw_statistics = compute_statistics(frame)
    if clean:
        frame, cleaning = remove_label_conflicts(frame)
        LOGGER.info("Removed %d mislabelled points (%.2f %%)",
                    cleaning.n_removed, 100.0 * cleaning.n_removed /
max(cleaning.n_before, 1))
    else:
        cleaning = None

    # Encode class names as integers 0..3, shape (n,), int64 (PyTorch's
    # CrossEntropyLoss expects int64 targets). 类别名编码为整数 0..3。
    labels = frame["class_code"].map(CLASS_TO_INDEX).to_numpy(dtype=np.int64)
    statistics = compute_statistics(frame)
    statistics["raw"] = raw_statistics
    statistics["cleaning"] = cleaning.to_dict() if cleaning else None
    # Region name of every point, and the class counts per region.
    # 每个点所属海域及各海域的类别计数。
    regions = assign_regions(frame["latitude"].to_numpy(),
frame["longitude"].to_numpy())
    statistics["regions"] = {
        str(name): {
            "count": int((regions == name).sum()),
            "classes": {c: int(((regions == name) & (frame["class_code"].to_numpy() ==
c)).sum())
                        for c in CLASS_NAMES},
        }
        for name in pd.unique(regions)
    }

    # Split *before* the index is built so that the reference index is derived
    # from training data only. 先划分再建索引，保证参考索引只来自训练集。
    # Two-stage split on row indices: first 70 % train vs 30 % hold-out, then
    # the hold-out is halved into validation and test (rel = 0.15 / 0.30 = 0.5).
    # Both stages are stratified by class and seeded for reproducibility.
    # 两阶段划分：先 70% 训练 / 30% 保留，再将保留部分对半分为验证与测试；均按类别分层。
    idx_all = np.arange(len(frame))
    idx_train, idx_hold = train_test_split(
        idx_all, test_size=TRAINING.test_size + TRAINING.val_size,
        random_state=seed, stratify=labels,
    )
    rel = TRAINING.test_size / (TRAINING.test_size + TRAINING.val_size)
    idx_val, idx_test = train_test_split(
        idx_hold, test_size=rel, random_state=seed, stratify=labels[idx_hold],
    )

    LOGGER.info("Building reference index from %d training points", len(idx_train))
    index = ReferenceIndex.from_frame(frame.iloc[idx_train])
    Path(index_path).parent.mkdir(parents=True, exist_ok=True)
    index.save(index_path)

    # Feature matrices, each (n_split, 28) float32. reset_index gives every
    # sub-frame a clean 0..n-1 index, so row i of the matrix is row i of labels.
    # 各子集特征矩阵 (n, 28) float32；reset_index 使子表索引为 0..n-1，与标签逐行对应。
    LOGGER.info("Extracting features")
    x_train = build_features(frame.iloc[idx_train].reset_index(drop=True), index,
exclude_self=True)
    x_val = build_features(frame.iloc[idx_val].reset_index(drop=True), index,
exclude_self=False)
    x_test = build_features(frame.iloc[idx_test].reset_index(drop=True), index,
exclude_self=False)
```

```
    split = SplitData(
        x_train=x_train, y_train=labels[idx_train],
        x_val=x_val, y_val=labels[idx_val],
        x_test=x_test, y_test=labels[idx_test],
        feature_names=list(FEATURE_NAMES),
    )

    Path(output_npz).parent.mkdir(parents=True, exist_ok=True)
    # The region of every test point is stored so that the accuracy can be
    # reported per sea region. 保存测试点所属海域，便于按海域统计准确率。
    np.savez_compressed(
        output_npz,
        x_train=split.x_train, y_train=split.y_train,
        x_val=split.x_val, y_val=split.y_val,
        x_test=split.x_test, y_test=split.y_test,
        region_test=np.array(regions[idx_test], dtype=str),
        feature_names=np.array(FEATURE_NAMES),
    )
    statistics["split"] = split.describe()
    statistics["load_report"] = report.to_dict()
    # ensure_ascii=False keeps non-ASCII text readable in the JSON file.
    # ensure_ascii=False 使 JSON 中的非 ASCII 文本保持可读。
    STATS_JSON.write_text(json.dumps(statistics, indent=2, ensure_ascii=False),
encoding="utf-8")
    LOGGER.info("Saved processed dataset to %s", output_npz)
    return split, statistics


def load_prepared(path: Path | str = PROCESSED_NPZ) -> SplitData:
    """Load a previously prepared split from disk.

    Parameters
    ----------
    path:
        ``.npz`` archive written by :func:`prepare`.

    Returns
    -------
    SplitData
        The six arrays and the feature names. ``region_test`` is not part of
        :class:`SplitData`; ``scripts/evaluate_regions.py`` reads it directly.

    读取已预处理的数据集。``allow_pickle=True`` 用于读取字符串数组。
    """
    blob = np.load(path, allow_pickle=True)
    return SplitData(
        x_train=blob["x_train"], y_train=blob["y_train"],
        x_val=blob["x_val"], y_val=blob["y_val"],
        x_test=blob["x_test"], y_test=blob["y_test"],
        feature_names=list(blob["feature_names"]),
    )
```

D.3 Neural network

model/__init__.py

```
"""Neural network model package.

神经网络模型层：结构、训练与推理。

Modules
-------
:mod:`~maritime_route.model.network`
    PyTorch definition of :class:`ZoneClassifier`, a residual multilayer
    perceptron mapping the 28 engineered features of a point to logits of the
    four zone classes.
:mod:`~maritime_route.model.trainer`
    Training loop (AdamW, class-weighted cross-entropy with label smoothing,
    learning-rate scheduling, early stopping), test-set metrics and saving of
    weights, feature scaler and metadata.
:mod:`~maritime_route.model.inference`
    :class:`ZoneClassifierService`, a thread-safe singleton that loads the
```

```
    saved artefacts once and classifies arbitrary coordinates for the route
    planner and the web API.

子模块：network（网络结构）、trainer（训练与评估）、inference（推理服务）。
"""
# Re-export the two classes used by the rest of the application. The trainer is
# not re-exported: it pulls in scikit-learn metrics that inference does not need.
# 只导出应用其余部分需要的两个类；trainer 依赖较多，不在此处导入。
from .network import ZoneClassifier
from .inference import ZoneClassifierService

__all__ = ["ZoneClassifier", "ZoneClassifierService"]
```

model/network.py – the residual multilayer perceptron

```
"""PyTorch definition of the geographic-zone classifier.

地理区域分类神经网络定义。
A residual multilayer perceptron is used: the tabular feature vector is small
(28 dimensions) but the decision boundaries between COASTAL_SEA / NEAR_COAST
are strongly non-linear, so depth plus batch normalisation and skip
connections give a markedly better convergence than a plain MLP.

Architecture (default sizes)
----------------------------
::

    x (batch, 28)
      -> BatchNorm1d(28)                        input normalisation
      -> ResidualBlock(28  -> 256)              Linear, BN, GELU, Dropout + skip
      -> ResidualBlock(256 -> 128)
      -> ResidualBlock(128 -> 64)
      -> Linear(64 -> 4)                        logits, one per zone class

The network outputs raw logits; the softmax is applied by the loss function
during training (:class:`torch.nn.CrossEntropyLoss`) and explicitly at
inference time (:meth:`ZoneClassifier.predict_proba`,
:mod:`maritime_route.model.inference`). With the default sizes the model has
98 364 trainable parameters.

Main objects: :class:`ResidualBlock`, :class:`ZoneClassifier`.

结构：输入 BatchNorm -> 三个残差块 (256/128/64) -> 线性输出层（4 类 logits）。
网络输出未归一化的 logits，softmax 在损失函数或推理阶段计算。
"""
from __future__ import annotations

from typing import Sequence

import torch
import torch.nn as nn


class ResidualBlock(nn.Module):
    """Fully connected block with batch-norm, GELU, dropout and a skip path.

    残差块：BatchNorm + GELU + Dropout，并带有线性投影的跳跃连接。

    Computes ``y = Dropout(GELU(BN(W x + b))) + P(x)`` where ``P`` is the
    identity if the input and output widths are equal and a learned linear
    projection otherwise (the widths must match for the addition).

    * **Linear** ``W x + b`` - affine map from ``in_features`` to
      ``out_features``.
    * **BatchNorm1d** - normalises every output unit to zero mean and unit
      variance over the mini-batch, then applies a learned scale and shift.
      It keeps the activations in a well-conditioned range, which allows a
      higher learning rate and faster convergence. In ``eval()`` mode running
      averages collected during training are used instead of batch
      statistics.
    * **GELU** - Gaussian Error Linear Unit, ``x * Phi(x)`` with ``Phi`` the
      standard normal CDF. A smooth alternative to ReLU that lets small
      negative values through, giving non-zero gradients for them.
    * **Dropout** - during training zeroes each activation with probability
      ``p`` and scales the survivors by ``1 / (1 - p)``; a regulariser against
      co-adaptation of neurons. Disabled in ``eval()`` mode.
    * **Skip connection** ``+ P(x)`` - the block only has to learn a
      correction to its input; gradients flow directly through the addition,
```

```
      which eases the training of deeper stacks.

    Parameters
    ----------
    in_features:
        Width of the input vector.
    out_features:
        Width of the output vector.
    dropout:
        Dropout probability ``p`` in [0, 1).
    """

    def __init__(self, in_features: int, out_features: int, dropout: float) -> None:
        """Create the layers of the block (see the class docstring). 构建残差块各层。"""
        super().__init__()
        self.linear = nn.Linear(in_features, out_features)
        self.norm = nn.BatchNorm1d(out_features)
        self.activation = nn.GELU()
        self.dropout = nn.Dropout(dropout)
        # Skip path: identity when widths match, otherwise a learned projection
        # so that ``h + projection(x)`` has matching shapes.
        # 跳跃连接：宽度相同用恒等映射，否则用线性投影以对齐维度。
        self.projection = (
            nn.Identity() if in_features == out_features else nn.Linear(in_features,
out_features)
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:  # noqa: D102
        """Apply the block.

        Parameters
        ----------
        x:
            Tensor of shape ``(batch, in_features)``.

        Returns
        -------
        torch.Tensor
            Tensor of shape ``(batch, out_features)``.

        前向计算：主路径 Linear->BN->GELU->Dropout，加上跳跃连接。
        """
        # Main path: (batch, in) -> (batch, out). 主路径。
        h = self.dropout(self.activation(self.norm(self.linear(x))))
        # Residual addition. 残差相加。
        return h + self.projection(x)


class ZoneClassifier(nn.Module):
    """Four-way classifier over the engineered geographic features.

    输入: (batch, n_features) -> 输出: (batch, n_classes) 的 logits。

    The input features arrive already standardised by the scikit-learn
    scaler; the extra ``input_norm`` BatchNorm layer makes the network robust
    to small differences in scale that remain (e.g. between the binary flag
    features and the log-transformed ones).

    Parameters
    ----------
    n_features:
        Width of the input vector (28 for the current feature set).
    n_classes:
        Number of output classes (4 zone classes).
    hidden_sizes:
        Width of each residual block, from input to output.
    dropout:
        Dropout probability used in every block.

    Attributes
    ----------
    n_features, n_classes, hidden_sizes, dropout_p:
        Copies of the constructor arguments, stored so that
        :meth:`config_dict` can save them with the weights and the inference
        service can rebuild an identical network.
    input_norm:
        BatchNorm1d over the input features.
    blocks:
        :class:`torch.nn.Sequential` of :class:`ResidualBlock`.
    head:
```

```
    Final linear layer producing the logits.
"""

def __init__(
    self,
    n_features: int,
    n_classes: int = 4,
    hidden_sizes: Sequence[int] = (256, 128, 64),
    dropout: float = 0.2,
) -> None:
    """Build the layers and initialise the weights. 构建网络层并初始化权重。"""
    super().__init__()
    self.n_features = int(n_features)
    self.n_classes = int(n_classes)
    self.hidden_sizes = tuple(int(h) for h in hidden_sizes)
    self.dropout_p = float(dropout)

    self.input_norm = nn.BatchNorm1d(self.n_features)
    # Chain the blocks: the output width of one is the input width of the next.
    # 依次连接残差块：上一块的输出宽度即下一块的输入宽度。
    blocks = []
    prev = self.n_features
    for size in self.hidden_sizes:
        blocks.append(ResidualBlock(prev, size, dropout))
        prev = size
    self.blocks = nn.Sequential(*blocks)
    self.head = nn.Linear(prev, self.n_classes)
    self._init_weights()

def _init_weights(self) -> None:
    """Initialise every linear layer with Kaiming-uniform weights and zero bias.

    Kaiming (He) initialisation draws weights from
    ``U(-b, b)`` with ``b = sqrt(6 / fan_in)`` (gain for ReLU), which keeps
    the variance of the activations roughly constant from layer to layer
    for ReLU-like non-linearities such as GELU. Zero biases start every
    unit centred. BatchNorm layers keep PyTorch's default (scale 1,
    shift 0).

    Kaiming 均匀初始化线性层权重（适用于 ReLU/GELU 类激活），偏置置零。
    """
    for module in self.modules():
        if isinstance(module, nn.Linear):
            nn.init.kaiming_uniform_(module.weight, nonlinearity="relu")
            if module.bias is not None:
                nn.init.zeros_(module.bias)

def forward(self, x: torch.Tensor) -> torch.Tensor:  # noqa: D102
    """Compute class logits.

    Parameters
    ----------
    x:
        Standardised features, shape ``(batch, n_features)``, float32.
        In training mode the batch must contain at least 2 samples
        (BatchNorm needs a variance).

    Returns
    -------
    torch.Tensor
        Logits, shape ``(batch, n_classes)``; not normalised.

    前向计算，返回未归一化的 logits。
    """
    return self.head(self.blocks(self.input_norm(x)))

@torch.no_grad()
def predict_proba(self, x: torch.Tensor) -> torch.Tensor:
    """Softmax probabilities. 推理时返回各类别概率。

    Switches the model to evaluation mode (BatchNorm uses its running
    statistics, Dropout is disabled) and applies the softmax
    ``p_c = exp(z_c) / sum_j exp(z_j)`` over the class axis, so each row
    sums to 1. Gradient tracking is disabled by the decorator.

    Parameters
    ----------
    x:
        Standardised features, shape ``(batch, n_features)``.
```

```
        Returns
        -------
        torch.Tensor
            Probabilities, shape ``(batch, n_classes)``.
        """
        self.eval()
        return torch.softmax(self.forward(x), dim=1)

    def config_dict(self) -> dict:
        """Return the constructor arguments needed to rebuild the network.

        Saved inside the checkpoint and the metadata JSON; the inference
        service reads it to create a network of identical shape before loading
        the weights.

        Returns
        -------
        dict
            ``n_features``, ``n_classes``, ``hidden_sizes``, ``dropout`` and an
            ``architecture`` label.

        返回重建网络所需的结构参数（随权重一同保存）。
        """
        return {
            "n_features": self.n_features,
            "n_classes": self.n_classes,
            "hidden_sizes": list(self.hidden_sizes),
            "dropout": self.dropout_p,
            "architecture": "ResidualMLP",
        }

    def count_parameters(self) -> int:
        """Return the number of trainable parameters (weights and biases).

        BatchNorm running statistics are buffers, not parameters, and are not
        counted.

        Returns
        -------
        int
            Sum of the element counts of all parameters with
            ``requires_grad=True``.

        返回可训练参数总数（不含 BatchNorm 的滑动统计量）。
        """
        return sum(p.numel() for p in self.parameters() if p.requires_grad)
```

model/trainer.py – training loop and evaluation

```
"""Training loop, evaluation metrics and model persistence.

训练与评估模块：包含早停、学习率调度、类别加权与完整评估指标。

Role in the pipeline
--------------------
Called by ``scripts/train_model.py`` with the :class:`SplitData` produced by
:mod:`maritime_route.data.dataset`. :func:`train_model`:

1. fits a :class:`~sklearn.preprocessing.StandardScaler` on the training
   features (per-column ``z = (x - mean) / std``, statistics from the training
   split only, so no information leaks from validation/test);
2. trains a :class:`~maritime_route.model.network.ZoneClassifier` with AdamW
   on a class-weighted, label-smoothed cross-entropy loss;
3. halves the learning rate when the validation loss stops improving
   (ReduceLROnPlateau) and stops early when it has not improved for
   ``early_stopping_patience`` epochs, restoring the best weights;
4. evaluates the best model on the test split (:func:`evaluate`);
5. saves the weights, the scaler and a metadata JSON (hyper-parameters,
   metrics, per-epoch history) to :data:`~maritime_route.config.MODEL_DIR`.

The three saved files are what
:class:`~maritime_route.model.inference.ZoneClassifierService` loads.

Main objects: :func:`train_model`, :func:`evaluate`, :class:`EpochRecord`,
:class:`TrainingHistory`.

流程：拟合标准化器 -> AdamW 训练（类别加权 + 标签平滑的交叉熵）-> 学习率调度与早停
-> 在测试集上评估 -> 保存权重、标准化器与元数据。
```

```
"""
from __future__ import annotations

import json
import logging
import time
from dataclasses import dataclass, field
from pathlib import Path
from typing import Dict, List, Optional, Tuple

import joblib
import numpy as np
import torch
import torch.nn as nn
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    cohen_kappa_score,
    confusion_matrix,
    f1_score,
    precision_recall_fscore_support,
    roc_auc_score,
)
from sklearn.preprocessing import StandardScaler
from torch.utils.data import DataLoader, TensorDataset

from ..config import (
    CLASS_NAMES,
    MODEL_METADATA_PATH,
    MODEL_WEIGHTS_PATH,
    SCALER_PATH,
    TRAINING,
    TrainingConfig,
)
from ..data.dataset import SplitData
from ..data.features import FEATURE_NAMES
from .network import ZoneClassifier

LOGGER = logging.getLogger(__name__)


@dataclass
class EpochRecord:
    """Metrics of one training epoch. 单轮训练记录。

    Attributes
    ----------
    epoch:
        Epoch number, starting at 1.
    train_loss:
        Mean loss over the training samples of this epoch (computed while the
        weights were still changing, with dropout active).
    val_loss:
        Mean loss on the validation split after the epoch (eval mode).
    train_accuracy, val_accuracy:
        Share of correctly classified samples, 0..1.
    val_macro_f1:
        Unweighted mean of the per-class F1 scores on validation data; unlike
        accuracy it is not dominated by the largest class.
    learning_rate:
        Learning rate after the scheduler step of this epoch.
    seconds:
        Wall-clock duration of the epoch.
    """

    epoch: int
    train_loss: float
    val_loss: float
    train_accuracy: float
    val_accuracy: float
    val_macro_f1: float
    learning_rate: float
    seconds: float


@dataclass
class TrainingHistory:
    """Ordered list of :class:`EpochRecord`; source of the training-curve figures.

    训练历史：按轮次保存的记录，用于绘制训练曲线。
```

```
    """

    records: List[EpochRecord] = field(default_factory=list)

    def append(self, record: EpochRecord) -> None:
        """Add the record of the epoch that has just finished. 追加一轮记录。"""
        self.records.append(record)

    def to_dict(self) -> Dict[str, List[float]]:
        """Return the history as parallel lists (one list per metric).

        This column layout is stored in the metadata JSON and can be plotted
        directly (x = ``epoch``). ``seconds`` is not included.

        Returns
        -------
        dict
            Metric name -> list of values, one per epoch.

        转换为"每个指标一个列表"的字典，便于保存与绘图。
        """
        return {
            "epoch": [r.epoch for r in self.records],
            "train_loss": [r.train_loss for r in self.records],
            "val_loss": [r.val_loss for r in self.records],
            "train_accuracy": [r.train_accuracy for r in self.records],
            "val_accuracy": [r.val_accuracy for r in self.records],
            "val_macro_f1": [r.val_macro_f1 for r in self.records],
            "learning_rate": [r.learning_rate for r in self.records],
        }


def _loaders(split: SplitData, scaler: StandardScaler, cfg: TrainingConfig):
    """Build PyTorch data loaders for the three splits.

    Every split is standardised with the *same* scaler (fitted on the
    training data) and wrapped in a :class:`~torch.utils.data.TensorDataset`.
    Only the training loader shuffles, so that every epoch sees the samples
    in a new order (better stochastic gradients); validation and test order
    is irrelevant and kept fixed.

    Parameters
    ----------
    split:
        Feature matrices and labels.
    scaler:
        Fitted scaler.
    cfg:
        Training configuration (batch size).

    Returns
    -------
    tuple of DataLoader
        ``(train_loader, val_loader, test_loader)``; each batch is
        ``(x, y)`` with ``x`` of shape ``(batch, 28)`` float32 and ``y`` of
        shape ``(batch,)`` int64.

    构建三个数据加载器：统一用训练集拟合的标准化器变换，仅训练集打乱顺序。
    """
    def make(x: np.ndarray, y: np.ndarray, shuffle: bool) -> DataLoader:
        """Standardise ``x`` and wrap ``(x, y)`` in a DataLoader. 标准化并封装。"""
        tensors = TensorDataset(
            # Standardised features as float32 (the dtype of the model weights).
            torch.tensor(scaler.transform(x), dtype=torch.float32),
            # int64 ("long") class indices, required by CrossEntropyLoss.
            torch.tensor(y, dtype=torch.long),
        )
        # drop_last=False keeps the final, smaller batch so no sample is skipped.
        # 保留最后一个不完整批次，不丢弃样本。
        return DataLoader(tensors, batch_size=cfg.batch_size, shuffle=shuffle,
drop_last=False)

    return (make(split.x_train, split.y_train, True),
            make(split.x_val, split.y_val, False),
            make(split.x_test, split.y_test, False))


def _class_weights(y: np.ndarray, n_classes: int) -> torch.Tensor:
    """Inverse-frequency weights, normalised to mean 1. 类别不平衡加权。
```

```
    ``w_c = N / (C * n_c)`` where ``N`` is the number of samples, ``C`` the
    number of classes and ``n_c`` the count of class ``c``. A class that is
    twice as rare gets twice the weight in the loss, so rare classes are not
    ignored by the optimiser. Dividing by the mean keeps the overall loss
    scale (and therefore the effective learning rate) unchanged.

    Parameters
    ----------
    y:
        Training labels, int, shape ``(n,)``.
    n_classes:
        Number of classes.

    Returns
    -------
    torch.Tensor
        Shape ``(n_classes,)``, float32, mean 1.
    """
    counts = np.bincount(y, minlength=n_classes).astype(float)
    # An absent class would give a division by zero; treat it as count 1.
    # 缺失的类别计数按 1 处理，避免除零。
    counts[counts == 0] = 1.0
    weights = counts.sum() / (n_classes * counts)
    return torch.tensor(weights / weights.mean(), dtype=torch.float32)


@torch.no_grad()
def _evaluate_loader(model: nn.Module, loader: DataLoader, criterion, device: str):
    """Run the model over a loader and collect loss, labels and probabilities.

    The model is put into evaluation mode (BatchNorm uses running statistics,
    Dropout is off); the decorator disables gradient tracking.

    Parameters
    ----------
    model:
        The network.
    loader:
        Validation or test loader.
    criterion:
        Loss function (the same weighted, smoothed loss as in training, so
        train and validation losses are comparable).
    device:
        PyTorch device string.

    Returns
    -------
    tuple
        ``(mean_loss, y_true, y_pred, probs)`` - mean loss per sample
        (float), true labels ``(n,)``, predicted labels ``(n,)`` and softmax
        probabilities ``(n, n_classes)``.

    在评估模式下遍历数据，返回平均损失、真实标签、预测标签与概率。
    """
    model.eval()
    total_loss, n = 0.0, 0
    probs_all, y_all = [], []
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        logits = model(xb)                          # (batch, 4)
        loss = criterion(logits, yb)                # mean over the batch
        # Multiply by the batch size to get a sum, so the final division gives
        # the exact per-sample mean even when the last batch is smaller.
        # 乘以批大小得到总和，使最终均值在最后一批较小时仍然精确。
        total_loss += float(loss.item()) * len(yb)
        n += len(yb)
        # Softmax turns logits into probabilities that sum to 1 per row.
        # softmax 将 logits 转换为每行和为 1 的概率。
        probs_all.append(torch.softmax(logits, dim=1).cpu().numpy())
        y_all.append(yb.cpu().numpy())
    probs = np.concatenate(probs_all)
    y_true = np.concatenate(y_all)
    y_pred = probs.argmax(axis=1)                   # most probable class
    return total_loss / max(n, 1), y_true, y_pred, probs


def train_model(
    split: SplitData,
    cfg: TrainingConfig = TRAINING,
    weights_path: Path | str = MODEL_WEIGHTS_PATH,
```

```
    scaler_path: Path | str = SCALER_PATH,
    metadata_path: Path | str = MODEL_METADATA_PATH,
    progress_callback=None,
) -> Tuple[ZoneClassifier, TrainingHistory, Dict]:
    """Train the classifier and persist weights, scaler and metadata.

    训练主函数，返回 (模型, 训练历史, 评估指标)。

    Training components:

    * **Loss** - cross-entropy ``-sum_c q_c log p_c`` over the softmax
      probabilities ``p``, with

      - *class weights* from :func:`_class_weights` (compensate for class
        imbalance), and
      - *label smoothing* ``eps = cfg.label_smoothing``: the target ``q`` is
        ``1 - eps + eps/C`` for the true class and ``eps/C`` for the others
        (C = 4). The model is discouraged from producing extreme
        probabilities, which reduces overconfidence on label noise.

    * **Optimiser** - AdamW: Adam (per-parameter step sizes from running
      means of the gradient and its square) with *decoupled* weight decay,
      i.e. the weights are shrunk by ``lr * weight_decay`` directly instead
      of adding an L2 term to the gradient; this regularises more
      consistently than L2 in plain Adam.
    * **Gradient clipping** - the global gradient norm is limited to 5.0 to
      prevent a single bad batch from causing a very large update.
    * **Scheduler** - ReduceLROnPlateau halves the learning rate
      (``factor=0.5``) when the validation loss has not improved for
      ``cfg.lr_scheduler_patience`` epochs, down to ``min_lr=1e-6``.
    * **Early stopping** - training stops after
      ``cfg.early_stopping_patience`` epochs without a validation-loss
      improvement larger than 1e-5; the weights of the best epoch are then
      restored.

    Parameters
    ----------
    split:
        Prepared feature matrices and labels.
    cfg:
        Hyper-parameters.
    weights_path, scaler_path, metadata_path:
        Output files.
    progress_callback:
        Optional callable receiving each :class:`EpochRecord` after every
        epoch (a hook for progress display; the command-line script does not
        use it).

    Returns
    -------
    (ZoneClassifier, TrainingHistory, dict)
        The model with the best validation weights, the per-epoch history and
        the test metrics (see :func:`evaluate`) plus a ``training`` summary.
    """
    # Seed PyTorch (weight init, dropout masks, shuffling) and NumPy for
    # reproducible runs. 固定随机种子以保证结果可复现。
    torch.manual_seed(cfg.random_seed)
    np.random.seed(cfg.random_seed)
    device = cfg.device

    # Standardisation z = (x - mean) / std per feature column, fitted on the
    # training split only. 按列标准化，仅用训练集拟合均值与标准差。
    scaler = StandardScaler().fit(split.x_train)
    # test_loader is not used below: evaluate() processes the test split in one pass.
    # test_loader 未在下文使用：evaluate() 一次性处理测试集。
    train_loader, val_loader, test_loader = _loaders(split, scaler, cfg)

    model = ZoneClassifier(
        n_features=split.n_features,
        n_classes=len(CLASS_NAMES),
        hidden_sizes=cfg.hidden_sizes,
        dropout=cfg.dropout,
    ).to(device)
    LOGGER.info("Model has %d trainable parameters", model.count_parameters())

    # Weighted, label-smoothed cross-entropy (applies log-softmax internally,
    # so the model outputs raw logits). 类别加权 + 标签平滑的交叉熵（内部含 log-softmax）。
    criterion = nn.CrossEntropyLoss(
        weight=_class_weights(split.y_train, len(CLASS_NAMES)).to(device),
        label_smoothing=cfg.label_smoothing,
```

```
    )
    # AdamW: Adam with decoupled weight decay. AdamW 优化器（解耦权重衰减）。
    optimiser = torch.optim.AdamW(model.parameters(), lr=cfg.learning_rate,
                                  weight_decay=cfg.weight_decay)
    # Halve the LR when val loss plateaus ("min": lower is better).
    # 验证损失停滞时学习率减半。
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimiser, mode="min", factor=0.5, patience=cfg.lr_scheduler_patience,
min_lr=1e-6,
    )

    history = TrainingHistory()
    # Early-stopping state: best validation loss, a copy of the weights that
    # achieved it, and the number of epochs since the last improvement.
    # 早停状态：最佳验证损失、对应权重副本、自上次改进以来的轮数。
    best_val = float("inf")
    best_state: Optional[Dict] = None
    patience = 0
    t_start = time.perf_counter()

    for epoch in range(1, cfg.epochs + 1):
        t0 = time.perf_counter()
        # Training mode: Dropout active, BatchNorm uses batch statistics and
        # updates its running averages. 训练模式：启用 Dropout，BN 使用批统计量。
        model.train()
        running, seen, correct = 0.0, 0, 0
        for xb, yb in train_loader:
            xb, yb = xb.to(device), yb.to(device)
            # Clear the gradients of the previous step (set_to_none saves a
            # memset). 清除上一步梯度。
            optimiser.zero_grad(set_to_none=True)
            logits = model(xb)                          # (batch, 4)
            loss = criterion(logits, yb)
            loss.backward()                             # back-propagation 反向传播
            # Rescale gradients if their global L2 norm exceeds 5.0.
            # 梯度裁剪：全局 L2 范数超过 5.0 时按比例缩小。
            nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
            optimiser.step()                            # parameter update 参数更新
            running += float(loss.item()) * len(yb)
            seen += len(yb)
            correct += int((logits.argmax(1) == yb).sum().item())

        train_loss = running / max(seen, 1)
        train_acc = correct / max(seen, 1)
        val_loss, y_val, y_val_pred, _ = _evaluate_loader(model, val_loader, criterion,
device)
        val_acc = float(accuracy_score(y_val, y_val_pred))
        val_f1 = float(f1_score(y_val, y_val_pred, average="macro"))
        # The scheduler watches the validation loss. 调度器依据验证损失调整学习率。
        scheduler.step(val_loss)

        record = EpochRecord(
            epoch=epoch, train_loss=train_loss, val_loss=val_loss,
            train_accuracy=train_acc, val_accuracy=val_acc, val_macro_f1=val_f1,
            learning_rate=float(optimiser.param_groups[0]["lr"]),
            seconds=time.perf_counter() - t0,
        )
        history.append(record)
        if progress_callback is not None:
            progress_callback(record)
        # Log the first epoch and every 5th to keep the console readable.
        # 仅记录第 1 轮和每 5 轮，保持日志简洁。
        if epoch % 5 == 0 or epoch == 1:
            LOGGER.info(
                "epoch %3d | train loss %.4f acc %.4f | val loss %.4f acc %.4f f1 %.4f",
                epoch, train_loss, train_acc, val_loss, val_acc, val_f1,
            )

        # Early stopping: an improvement must exceed 1e-5 to count, so that
        # numerical noise does not reset the counter.
        # 早停：改进幅度需大于 1e-5 才计入，避免数值噪声重置计数。
        if val_loss < best_val - 1e-5:
            best_val, patience = val_loss, 0
            # Deep copy of the weights (state_dict returns references that the
            # next optimiser step would overwrite). 深拷贝当前最佳权重。
            best_state = {k: v.detach().clone() for k, v in model.state_dict().items()}
        else:
            patience += 1
            if patience >= cfg.early_stopping_patience:
                LOGGER.info("Early stopping at epoch %d", epoch)
```

```
                break

    # Restore the best epoch, not the last one. 恢复最佳轮次的权重（而非最后一轮）。
    if best_state is not None:
        model.load_state_dict(best_state)
    train_seconds = time.perf_counter() - t_start

    metrics = evaluate(model, split, scaler, cfg)
    metrics["training"] = {
        "epochs_run": len(history.records),
        "seconds": round(train_seconds, 2),
        "best_val_loss": round(best_val, 6),
        "parameters": model.count_parameters(),
        "device": device,
    }

    # Persist the three artefacts used by the inference service.
    # The checkpoint stores the architecture next to the weights so that the
    # network can be rebuilt without knowing the training configuration.
    # 保存推理所需的三个文件；检查点中同时保存网络结构参数。
    Path(weights_path).parent.mkdir(parents=True, exist_ok=True)
    torch.save({"state_dict": model.state_dict(), "config": model.config_dict()},
weights_path)
    joblib.dump(scaler, scaler_path)
    metadata = {
        "version": "1.0.0",
        "created_utc": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
        "classes": CLASS_NAMES,
        "feature_names": list(FEATURE_NAMES),
        "model": model.config_dict(),
        "hyperparameters": {
            "hidden_sizes": list(cfg.hidden_sizes), "dropout": cfg.dropout,
            "batch_size": cfg.batch_size, "epochs": cfg.epochs,
            "learning_rate": cfg.learning_rate, "weight_decay": cfg.weight_decay,
            "label_smoothing": cfg.label_smoothing, "seed": cfg.random_seed,
        },
        "dataset": split.describe(),
        "metrics": metrics,
        "history": history.to_dict(),
    }
    Path(metadata_path).write_text(json.dumps(metadata, indent=2), encoding="utf-8")
    LOGGER.info("Saved model to %s", weights_path)
    return model, history, metrics


def evaluate(model: ZoneClassifier, split: SplitData, scaler: StandardScaler,
             cfg: TrainingConfig = TRAINING) -> Dict:
    """Compute the full metric set on the held-out test split.

    在测试集上计算准确率、宏平均 F1、Kappa、每类指标、混淆矩阵与 ROC-AUC。

    Metrics:

    * ``accuracy`` - share of correct predictions;
    * ``macro_f1`` - unweighted mean of per-class F1 (F1 = harmonic mean of
      precision and recall); every class counts equally;
    * ``weighted_f1`` - per-class F1 weighted by class support;
    * ``cohen_kappa`` - agreement corrected for the agreement expected by
      chance, ``(p_o - p_e) / (1 - p_e)``;
    * ``macro_roc_auc`` - one-vs-rest ROC AUC averaged over classes; measures
      ranking quality of the probabilities independently of the argmax
      threshold (NaN if a class is absent from the test set);
    * ``per_class`` - precision, recall, F1 and support of each class;
    * ``confusion_matrix`` - 4 x 4 counts, rows = true class, columns =
      predicted class, in :data:`~maritime_route.config.CLASS_NAMES` order;
    * ``report_text`` - scikit-learn's text report;
    * ``mean_confidence`` - mean of the highest class probability.

    Parameters
    ----------
    model:
        Trained network.
    split:
        Data; only ``x_test`` / ``y_test`` are used.
    scaler:
        The scaler fitted on the training split.
    cfg:
        Training configuration (device).

    Returns
```

```
    -------
    dict
        JSON-serialisable metrics as listed above.
    """
    device = cfg.device
    model.eval()
    # The whole test split in one tensor (n_test, 28); it is small enough.
    # 整个测试集一次性前向计算。
    x = torch.tensor(scaler.transform(split.x_test), dtype=torch.float32).to(device)
    with torch.no_grad():
        probs = torch.softmax(model(x), dim=1).cpu().numpy()   # (n_test, 4)
    y_true = split.y_test
    y_pred = probs.argmax(axis=1)

    # labels=[0..3] fixes the class order and keeps absent classes as zeros;
    # zero_division=0 avoids warnings for classes that are never predicted.
    # 固定类别顺序；从未被预测的类别指标记为 0。
    precision, recall, f1, support = precision_recall_fscore_support(
        y_true, y_pred, labels=list(range(len(CLASS_NAMES))), zero_division=0
    )
    try:
        auc = float(roc_auc_score(y_true, probs, multi_class="ovr", average="macro"))
    except ValueError:
        # Raised when a class is missing from y_true (e.g. on a tiny subset).
        # 当测试集中缺少某一类别时无法计算 AUC。
        auc = float("nan")

    return {
        "accuracy": float(accuracy_score(y_true, y_pred)),
        "macro_f1": float(f1_score(y_true, y_pred, average="macro")),
        "weighted_f1": float(f1_score(y_true, y_pred, average="weighted")),
        "cohen_kappa": float(cohen_kappa_score(y_true, y_pred)),
        "macro_roc_auc": auc,
        "per_class": {
            CLASS_NAMES[i]: {
                "precision": float(precision[i]),
                "recall": float(recall[i]),
                "f1": float(f1[i]),
                "support": int(support[i]),
            }
            for i in range(len(CLASS_NAMES))
        },
        "confusion_matrix": confusion_matrix(
            y_true, y_pred, labels=list(range(len(CLASS_NAMES)))
        ).tolist(),
        "report_text": classification_report(
            y_true, y_pred, labels=list(range(len(CLASS_NAMES))),
            target_names=CLASS_NAMES, digits=4, zero_division=0,
        ),
        "mean_confidence": float(probs.max(axis=1).mean()),
    }
```

model/inference.py – the inference service

```
"""Inference service wrapping the trained classifier.

推理服务：加载已训练权重 + 标准化器 + 参考索引，对任意坐标进行区域分类。
The service is a process-wide singleton so that the 100k-point reference index
is memory-resident only once.

Role in the pipeline
--------------------
This is the bridge between the neural network and the rest of the
application:

* the route planner (:mod:`maritime_route.routing.cost_map`) calls
  :meth:`ZoneClassifierService.predict_coordinates` for every cell of the
  routing grid and turns the most probable class into a traversal cost;
* the web API calls :meth:`ZoneClassifierService.classify_frame` for
  uploaded AIS files and :meth:`ZoneClassifierService.classify_coordinates`
  for single clicked points.

Every prediction runs the same chain as training: features
(:mod:`maritime_route.data.features`) -> standardisation with the saved
scaler -> network -> softmax.

Main objects: :class:`ZoneClassifierService`, :class:`ClassificationResult`,
:class:`ModelNotTrainedError`.
```

```
用途：路线规划器对网格单元分类、Web 接口对上传文件与单点分类。
推理链与训练一致：特征提取 -> 标准化 -> 网络 -> softmax。
"""
from __future__ import annotations

import json
import logging
import threading
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Optional, Sequence

import joblib
import numpy as np
import pandas as pd
import torch

from ..config import (
    BATHY_INDEX_PATH,
    CLASS_NAMES,
    INDEX_TO_CLASS,
    MODEL_METADATA_PATH,
    MODEL_WEIGHTS_PATH,
    SCALER_PATH,
)
from ..data.features import ReferenceIndex, build_features, features_for_coordinates
from .network import ZoneClassifier

LOGGER = logging.getLogger(__name__)


class ModelNotTrainedError(RuntimeError):
    """Raised when the weights file is missing. 模型尚未训练。

    The web application catches it when the model is first needed and answers
    HTTP 503 with the message, which tells the user to run
    ``scripts/train_model.py``. Tests that need a model are skipped when the
    weights file is absent (see ``tests/conftest.py``).
    Web 应用在首次需要模型时捕获并返回 HTTP 503。
    """


@dataclass
class ClassificationResult:
    """Per-point classification output. 单点分类结果。

    Attributes
    ----------
    latitude, longitude:
        Position, degrees.
    class_index:
        Index of the most probable class (0..3).
    class_code:
        Name of that class, e.g. ``"COASTAL_SEA"``.
    confidence:
        Probability of the predicted class, 0..1.
    probabilities:
        Probability of every class, keyed by class name; sums to 1.
    depth_m:
        Relief at the point, metres, positive above sea level; the measured
        value if the input had one, otherwise interpolated from the reference
        index. ``None`` if unknown.
    """

    latitude: float
    longitude: float
    class_index: int
    class_code: str
    confidence: float
    probabilities: Dict[str, float]
    depth_m: Optional[float] = None

    def to_dict(self) -> Dict:
        """Return a JSON-ready dictionary with rounded values.

        Rounding: coordinates to 6 decimals (about 0.1 m), probabilities to 4
        decimals, depth to 0.1 m; this keeps API responses compact without
        losing meaningful precision.
```

```
        Returns
        -------
        dict
            All fields of the result.

        转换为 JSON 字典并做适当舍入（坐标 6 位小数约 0.1 m）。
        """
        return {
            "latitude": round(self.latitude, 6),
            "longitude": round(self.longitude, 6),
            "class_index": self.class_index,
            "class_code": self.class_code,
            "confidence": round(self.confidence, 4),
            "probabilities": {k: round(v, 4) for k, v in self.probabilities.items()},
            "depth_m": None if self.depth_m is None else round(float(self.depth_m), 1),
        }


class ZoneClassifierService:
    """Thread-safe façade over the trained model.

    Loads the four saved artefacts once (network weights, feature scaler,
    reference index, metadata) and offers prediction methods for data frames
    and bare coordinates.

    Concurrency: the web server runs classification in worker threads
    (``asyncio.to_thread``), so several requests can arrive at once. The
    class-level ``_lock`` guards creation of the singleton, and the
    per-instance ``_infer_lock`` serialises forward passes through the network,
    so that concurrent requests never run the same PyTorch module at the same
    time.

    Use :meth:`instance` to obtain the shared object and :meth:`reset` to
    force a reload after retraining.

    线程安全的推理门面：类锁保护单例创建，实例锁串行化网络前向计算。
    """

    #: The shared instance created by :meth:`instance`. 共享单例。
    _instance: Optional["ZoneClassifierService"] = None
    #: Guards creation / reset of the singleton. 保护单例创建与重置。
    _lock = threading.Lock()

    def __init__(
        self,
        weights_path: Path | str = MODEL_WEIGHTS_PATH,
        scaler_path: Path | str = SCALER_PATH,
        index_path: Path | str = BATHY_INDEX_PATH,
        metadata_path: Path | str = MODEL_METADATA_PATH,
    ) -> None:
        """Load the model artefacts from disk.

        Parameters
        ----------
        weights_path:
            PyTorch checkpoint ``{"state_dict", "config"}`` written by the
            trainer.
        scaler_path:
            Fitted StandardScaler (joblib).
        index_path:
            Reference index (.npz).
        metadata_path:
            Metadata JSON; optional - missing metadata only affects
            :meth:`summary`.

        Raises
        ------
        ModelNotTrainedError
            If the weights file does not exist.

        从磁盘加载权重、标准化器、参考索引与元数据。
        """
        weights_path, scaler_path, index_path = Path(weights_path), Path(scaler_path),
Path(index_path)
        if not weights_path.exists():
            raise ModelNotTrainedError(
                f"Model weights not found at {weights_path}. Run scripts/train_model.py
first."
            )
        # map_location="cpu" allows loading a checkpoint saved on a GPU machine.
```

```
        # weights_only=False because the checkpoint also holds the config dict
        # (the file is produced by this project, not an untrusted source).
        # 映射到 CPU 以兼容 GPU 上保存的检查点；检查点包含配置字典，故 weights_only=False。
        checkpoint = torch.load(weights_path, map_location="cpu", weights_only=False)
        config = checkpoint["config"]
        # Rebuild a network of identical shape, then load the trained weights.
        # 先按保存的结构参数重建网络，再加载权重。
        self.model = ZoneClassifier(
            n_features=config["n_features"],
            n_classes=config["n_classes"],
            hidden_sizes=config["hidden_sizes"],
            dropout=config["dropout"],
        )
        self.model.load_state_dict(checkpoint["state_dict"])
        # Evaluation mode: BatchNorm uses running statistics, Dropout is off.
        # 评估模式：BN 使用训练时的滑动统计量，关闭 Dropout。
        self.model.eval()
        self.scaler = joblib.load(scaler_path)
        self.index = ReferenceIndex.load(index_path)
        self.metadata: Dict = {}
        if Path(metadata_path).exists():
            self.metadata = json.loads(Path(metadata_path).read_text(encoding="utf-8"))
        self._infer_lock = threading.Lock()
        LOGGER.info("Zone classifier ready (%d reference samples)",
len(self.index.depth))

    # -- singleton ------------------------------------------------------------ #
    @classmethod
    def instance(cls) -> "ZoneClassifierService":
        """Return the process-wide service, creating it on first use.

        The check and the creation happen under ``_lock``, so two threads that
        call this at the same time cannot both load the model.

        Returns
        -------
        ZoneClassifierService
            The shared instance (default file paths).

        Raises
        ------
        ModelNotTrainedError
            If no trained model exists.

        返回进程级单例，首次调用时加载模型（加锁防止重复加载）。
        """
        with cls._lock:
            if cls._instance is None:
                cls._instance = cls()
            return cls._instance

    @classmethod
    def reset(cls) -> None:
        """Forget the shared instance; the next :meth:`instance` call reloads.

        Intended for use after retraining, so that the new weights are picked
        up without restarting the process. 重置单例，下次调用时重新加载模型。
        """
        with cls._lock:
            cls._instance = None

    # -- prediction ----------------------------------------------------------- #
    def _forward(self, matrix: np.ndarray, batch_size: int = 8192) -> np.ndarray:
        """Standardise a raw feature matrix and return class probabilities.

        The matrix is processed in chunks of ``batch_size`` rows to bound
        memory use for large grids (hundreds of thousands of cells).

        Parameters
        ----------
        matrix:
            Unscaled features, shape ``(n, 28)``.
        batch_size:
            Rows per forward pass.

        Returns
        -------
        numpy.ndarray
            Softmax probabilities, shape ``(n, 4)``; each row sums to 1.
            Shape ``(0, 4)`` for an empty input.
```

```
            标准化后分批前向计算，返回 softmax 概率。
            """
            # Same standardisation as in training: z = (x - mean_train) / std_train.
            # 与训练时相同的标准化。
            scaled = self.scaler.transform(matrix).astype(np.float32)
            outputs: List[np.ndarray] = []
            # _infer_lock: one forward pass at a time; no_grad: no autograd graph.
            # 加锁串行化前向计算；no_grad 不构建计算图，节省内存与时间。
            with self._infer_lock, torch.no_grad():
                for start in range(0, len(scaled), batch_size):
                    # from_numpy shares memory with the array (no copy).
                    chunk = torch.from_numpy(scaled[start:start + batch_size])
                    outputs.append(torch.softmax(self.model(chunk), dim=1).numpy())
            return np.concatenate(outputs) if outputs else np.empty((0, len(CLASS_NAMES)))

        def predict_frame(self, frame: pd.DataFrame) -> np.ndarray:
            """Probabilities for a DataFrame that already carries measured attributes.

            Parameters
            ----------
            frame:
                Points with ``latitude``/``longitude`` and optionally the measured
                columns (``noaa_depth``, ``elevation``, OSM counts); missing values
                are imputed by :func:`~maritime_route.data.features.build_features`.

            Returns
            -------
            numpy.ndarray
                Shape ``(n, 4)`` probabilities.

            对带测量属性的数据表进行预测。
            """
            matrix = build_features(frame, self.index, exclude_self=False)
            return self._forward(matrix)

        def predict_coordinates(self, lat: Sequence[float], lon: Sequence[float]) ->
    np.ndarray:
            """Probabilities for bare coordinates. 仅凭经纬度预测。

            All environmental attributes are derived from the reference index
            (see :func:`~maritime_route.data.features.features_for_coordinates`).
            This is the method the route planner uses for grid cells.

            Parameters
            ----------
            lat, lon:
                Coordinates, degrees, length n.

            Returns
            -------
            numpy.ndarray
                Shape ``(n, 4)`` probabilities.
            """
            matrix = features_for_coordinates(lat, lon, self.index)
            return self._forward(matrix)

        def classify_frame(self, frame: pd.DataFrame) -> List[ClassificationResult]:
            """Classify a frame of AIS points into :class:`ClassificationResult` objects.

            The reported depth is the measured ``noaa_depth`` when the frame has
            that column (NaN entries become ``None``), otherwise it is
            interpolated from the reference index.

            Parameters
            ----------
            frame:
                Validated points.

            Returns
            -------
            list of ClassificationResult
                One result per row, in row order.

            对 AIS 数据表逐点分类并封装结果。
            """
            probs = self.predict_frame(frame)
            lat = frame["latitude"].to_numpy(float)
            lon = frame["longitude"].to_numpy(float)
            # Measured depth if available, otherwise IDW interpolation from the index.
```

```
        # 有实测水深则使用实测值，否则由参考索引插值。
        depth = (pd.to_numeric(frame["noaa_depth"], errors="coerce").to_numpy(float)
                 if "noaa_depth" in frame.columns else self.index.sample_depth(lat,
lon))
        return self._package(lat, lon, probs, depth)

    def classify_coordinates(self, lat: Sequence[float], lon: Sequence[float]
                             ) -> List[ClassificationResult]:
        """Classify bare coordinates.

        Parameters
        ----------
        lat, lon:
            Coordinates, degrees, length n.

        Returns
        -------
        list of ClassificationResult
            One result per coordinate; depth interpolated from the index.

        仅凭经纬度分类，水深由参考索引插值。
        """
        lat = np.asarray(lat, float)
        lon = np.asarray(lon, float)
        probs = self.predict_coordinates(lat, lon)
        return self._package(lat, lon, probs, self.index.sample_depth(lat, lon))

    @staticmethod
    def _package(lat, lon, probs, depth) -> List[ClassificationResult]:
        """Turn probability rows into :class:`ClassificationResult` objects.

        Parameters
        ----------
        lat, lon:
            Arrays of length n, degrees.
        probs:
            ``(n, 4)`` class probabilities.
        depth:
            ``(n,)`` depth in metres (NaN allowed) or ``None``.

        Returns
        -------
        list of ClassificationResult
            The predicted class is the argmax of each row; its probability is
            the confidence.

        将概率矩阵封装为结果对象：取每行最大概率的类别作为预测，其概率为置信度。
        """
        best = probs.argmax(axis=1)   # (n,) index of the most probable class
        results: List[ClassificationResult] = []
        for i in range(len(lat)):
            results.append(
                ClassificationResult(
                    latitude=float(lat[i]),
                    longitude=float(lon[i]),
                    class_index=int(best[i]),
                    class_code=INDEX_TO_CLASS[int(best[i])],
                    confidence=float(probs[i, best[i]]),
                    probabilities={CLASS_NAMES[c]: float(probs[i, c]) for c in
range(len(CLASS_NAMES))},
                    # NaN (no measurement) is reported as None (JSON null).
                    # NaN 以 None（JSON null）表示。
                    depth_m=None if depth is None or np.isnan(depth[i]) else
float(depth[i]),
                )
            )
        return results

    # -- introspection ------------------------------------------------------ #
    def summary(self) -> Dict:
        """Describe the loaded model; returned by ``GET /api/model`` in the web API.

        Returns
        -------
        dict
            Classes, input width, parameter count, architecture label, size of
            the reference index, test accuracy and macro F1 from the metadata
            (``None`` if the metadata file was missing) and the training time
            stamp (UTC).
```

```
        返回模型摘要：类别、特征维数、参数量、结构、参考样本数、测试集指标与训练时间。
        """
        metrics = self.metadata.get("metrics", {})
        return {
            "classes": CLASS_NAMES,
            "n_features": self.model.n_features,
            "parameters": self.model.count_parameters(),
            "architecture": self.model.config_dict()["architecture"],
            "reference_samples": int(len(self.index.depth)),
            "accuracy": metrics.get("accuracy"),
            "macro_f1": metrics.get("macro_f1"),
            "trained_utc": self.metadata.get("created_utc"),
        }
```

D.4 Routing

routing/__init__.py

```
"""Routing package: cost map construction and graph search algorithms.

航线规划层：代价地图与图搜索算法。

This package turns two coordinates (departure, destination) into a navigable
route. It sits after the neural classifier in the pipeline::

    classifier (model.inference) -> cost_map -> astar | dijkstra | genetic | dynamic ->
coast_check -> planner -> storage / export / web

Modules
-------
geodesy
    Spherical geometry: haversine distance, bearing, great-circle
    interpolation, Douglas-Peucker simplification.
cost_map
    :class:`GridSpec` (regular latitude/longitude lattice) and
    :class:`CostMap` (zone class, confidence and traversal cost of every cell),
    built by :func:`build_cost_map` from the classifier output.
astar
    :func:`astar` - optimal search with an admissible great-circle heuristic;
    also defines :class:`SearchResult` and the shared helpers.
dijkstra
    :func:`dijkstra` - uniform-cost search, the reference exact method.
genetic
    :func:`~maritime_route.routing.genetic.genetic` - evolutionary search on
    the same cost model (not re-exported here; imported by the planner).
coast_check
    Independent coastline check of a route against the 1 km GLOBE land mask
    (package ``global-land-mask``) and the choice of the lattice cells to
    block when a route crosses land (imported by the planner).
planner
    :class:`RoutePlanner` - the service used by the web application: builds
    the lattice, snaps the ports onto water, runs the chosen algorithm,
    refines the lattice on failure, corrects a route that crosses land on the
    coastline check and compares the result with the great circle. :func:`benchmark`
runs all methods on one shared cost map.

The names below are the public API of the package.
以下名称为本包对外公开的接口。
"""
from .astar import SearchResult, astar
from .cost_map import CostMap, GridSpec, build_cost_map, make_grid_spec
from .dijkstra import dijkstra
from .planner import RouteLeg, RoutePlan, RoutePlanner, benchmark

# Public names exported by ``from maritime_route.routing import *``.
__all__ = [
    "SearchResult", "astar", "dijkstra", "CostMap", "GridSpec", "build_cost_map",
    "make_grid_spec", "RouteLeg", "RoutePlan", "RoutePlanner", "benchmark",
]
```

routing/geodesy.py – spherical geometry

```
"""Spherical geodesy helpers used by the feature extractor and the planner.

球面几何工具：大圆距离、方位角、球面插值。
All functions are vectorised with NumPy where it matters for performance.

Role in the pipeline
--------------------
Every distance in the application is a great-circle distance on a sphere of
radius :data:`~maritime_route.config.EARTH_RADIUS_KM` (6371.0088 km, the IUGG
mean Earth radius). The ellipsoidal error of this model is below 0.5 %, which
is far smaller than the size of one lattice cell. The module is used by

* :mod:`~maritime_route.routing.astar` / ``dijkstra`` / ``genetic`` - edge
  lengths of the lattice graph (through :func:`haversine_km`);
* :mod:`~maritime_route.routing.planner` - route length, the great-circle
  reference route, bearings of the waypoint table, and optional route
  simplification (:func:`douglas_peucker`);
* :mod:`~maritime_route.export.exporters` - the great-circle line written to
  GeoJSON.

Conventions
-----------
* A coordinate is a ``(latitude, longitude)`` tuple in decimal degrees
  (WGS 84 values treated as spherical). Latitude comes first everywhere in
  this module; only the GeoJSON exporter swaps the order to lon/lat.
* Distances are in kilometres, bearings in degrees clockwise from true north.

约定：坐标为 (纬度, 经度)，单位为度；距离单位为千米；方位角以正北为 0 度顺时针计。
"""
from __future__ import annotations

import math
from typing import Iterable, List, Sequence, Tuple

import numpy as np

from ..config import EARTH_RADIUS_KM

Coordinate = Tuple[float, float]  # (latitude, longitude) in degrees


def haversine_km(lat1, lon1, lat2, lon2):
    """Great-circle distance in kilometres. Accepts scalars or arrays.

    Uses the haversine formula, which is numerically stable for the short
    distances (a few kilometres between neighbouring lattice cells) that
    dominate the search::

        a = sin^2(dlat / 2) + cos(lat1) * cos(lat2) * sin^2(dlon / 2)
        d = 2 * R * asin(sqrt(a))

    The law-of-cosines form ``acos(...)`` loses precision for such small
    angles, which is why it is not used here.
    半正矢公式：对小距离数值稳定，适合相邻网格单元之间的距离计算。

    Parameters
    ----------
    lat1, lon1 : float or array_like
        Latitude and longitude of the first point(s), degrees.
    lat2, lon2 : float or array_like
        Latitude and longitude of the second point(s), degrees. Arrays are
        broadcast against the first point(s) by NumPy rules.

    Returns
    -------
    float or numpy.ndarray
        Distance in kilometres; a NumPy scalar for scalar input, an array of
        the broadcast shape otherwise.
    """
    lat1, lon1, lat2, lon2 = map(np.radians, (lat1, lon1, lat2, lon2))
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat / 2.0) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2.0) ** 2
    # Rounding can push ``a`` marginally outside [0, 1] for identical or
    # antipodal points; clipping keeps sqrt/arcsin defined.
    # 浮点误差可能使 a 略超出 [0, 1]，裁剪后再开方与求反正弦。
    return 2.0 * EARTH_RADIUS_KM * np.arcsin(np.sqrt(np.clip(a, 0.0, 1.0)))
```

```
def initial_bearing_deg(lat1: float, lon1: float, lat2: float, lon2: float) -> float:
    """Initial great-circle bearing from point 1 to point 2, degrees [0, 360).

    The initial bearing (forward azimuth) is the course a ship must steer at
    point 1 to follow the great circle to point 2. On a great circle the
    course changes continuously, so this is the heading only at the start of
    the segment. Formula::

        theta = atan2(sin(dlon) * cos(lat2),
                      cos(lat1) * sin(lat2) - sin(lat1) * cos(lat2) * cos(dlon))

    初始方位角：从点 1 沿大圆驶向点 2 时在点 1 处的航向。

    Parameters
    ----------
    lat1, lon1 : float
        Departure point, degrees.
    lat2, lon2 : float
        Target point, degrees.

    Returns
    -------
    float
        Bearing in degrees clockwise from true north, normalised to [0, 360).
    """
    p1, p2 = math.radians(lat1), math.radians(lat2)
    dl = math.radians(lon2 - lon1)
    y = math.sin(dl) * math.cos(p2)
    x = math.cos(p1) * math.sin(p2) - math.sin(p1) * math.cos(p2) * math.cos(dl)
    # atan2 returns (-180, 180]; "+360 then mod 360" maps it to [0, 360).
    return (math.degrees(math.atan2(y, x)) + 360.0) % 360.0


def to_unit_vector(lat, lon):
    """Convert degrees to 3-D unit vectors on the sphere (n x 3).

    Earth-centred Cartesian coordinates of a unit sphere:
    ``x = cos(lat) cos(lon)``, ``y = cos(lat) sin(lon)``, ``z = sin(lat)``.
    Working with vectors avoids the singularities of latitude/longitude at
    the poles and at the antimeridian during interpolation.
    经纬度转换为单位球面上的三维向量（地心直角坐标）。

    Parameters
    ----------
    lat, lon : float or array_like
        Latitude and longitude in degrees.

    Returns
    -------
    numpy.ndarray
        Array of shape ``(..., 3)``; shape ``(3,)`` for scalar input.
    """
    p, l = np.radians(lat), np.radians(lon)
    return np.stack([np.cos(p) * np.cos(l), np.cos(p) * np.sin(l), np.sin(p)], axis=-1)


def great_circle_points(start: Coordinate, end: Coordinate, n: int = 64) ->
List[Coordinate]:
    """Sample ``n`` points along the great circle using spherical interpolation.

    球面线性插值(slerp)，用于生成"最短距离"参考航线。

    Algorithm (slerp): with unit vectors ``v1``, ``v2`` of the end points and
    the central angle ``omega = acos(v1 . v2)``, the point at fraction ``t``
    of the arc is::

        v(t) = sin((1 - t) * omega) / sin(omega) * v1 + sin(t * omega) / sin(omega) * v2

    The ``n`` samples are equally spaced in arc length (``t = i / (n - 1)``)
    and include both end points. The result is converted back to
    latitude/longitude with ``lat = asin(z)``, ``lon = atan2(y, x)``.

    Parameters
    ----------
    start, end : Coordinate
        End points ``(lat, lon)`` in degrees.
    n : int, default 64
        Number of samples including both ends; values below 2 are raised to 2.
```

```
    Returns
    -------
    list of Coordinate
        ``n`` points from ``start`` to ``end``. If the two points coincide
        (``omega`` ~ 0) only ``[start, end]`` is returned. Longitudes are in
        (-180, 180], so a line crossing the antimeridian jumps from +180 to
        -180 between two samples.

    Notes
    -----
    For exactly antipodal points the great circle is not unique
    (``sin(omega) = 0``); this does not occur for the regional voyages the
    application plans.
    """
    n = max(2, int(n))
    v1 = to_unit_vector(start[0], start[1])
    v2 = to_unit_vector(end[0], end[1])
    # Clip the dot product so that rounding cannot make acos undefined.
    dot = float(np.clip(np.dot(v1, v2), -1.0, 1.0))
    omega = math.acos(dot)                  # central angle between the points, radians
    if omega < 1e-12:
        # Coincident points: interpolation would divide by sin(0) = 0.
        # 起终点重合：避免除以零。
        return [start, end]
    sin_omega = math.sin(omega)
    out: List[Coordinate] = []
    for i in range(n):
        t = i / (n - 1)                     # fraction of the arc, 0 .. 1
        a = math.sin((1.0 - t) * omega) / sin_omega
        b = math.sin(t * omega) / sin_omega
        v = a * v1 + b * v2
        # Slerp keeps |v| = 1 analytically; normalising removes rounding drift.
        v /= np.linalg.norm(v)
        lat = math.degrees(math.asin(float(v[2])))
        lon = math.degrees(math.atan2(float(v[1]), float(v[0])))
        out.append((lat, lon))
    return out


def path_length_km(path: Sequence[Coordinate]) -> float:
    """Total great-circle length of a poly-line given as (lat, lon) pairs.

    Each segment between consecutive vertices is treated as a great-circle
    arc; the lengths are summed in one vectorised :func:`haversine_km` call.

    Parameters
    ----------
    path : sequence of Coordinate
        Vertices ``(lat, lon)`` in degrees.

    Returns
    -------
    float
        Length in kilometres; ``0.0`` for fewer than two vertices.
    """
    if len(path) < 2:
        return 0.0
    arr = np.asarray(path, dtype=float)
    # arr[:-1] are the segment starts and arr[1:] the segment ends.
    return float(np.sum(haversine_km(arr[:-1, 0], arr[:-1, 1], arr[1:, 0], arr[1:, 1])))


def densify(path: Sequence[Coordinate], step_km: float = 25.0) -> List[Coordinate]:
    """Insert intermediate points so that no segment is longer than ``step_km``.

    Every segment is replaced by points sampled on its great circle
    (:func:`great_circle_points`), so the densified line follows the sphere
    rather than a straight line in the latitude/longitude plane. This matters
    when a route is drawn on a Mercator map or sampled against the lattice.
    航线加密：按大圆插值插入中间点，使每段长度不超过 step_km。

    Parameters
    ----------
    path : sequence of Coordinate
        Poly-line vertices ``(lat, lon)`` in degrees.
    step_km : float, default 25.0
        Maximum segment length after densification, kilometres.

    Returns
```

```
    -------
    list of Coordinate
        Densified poly-line; the original vertices are kept, and a path with
        fewer than two vertices is returned unchanged (as a list).
    """
    if len(path) < 2:
        return list(path)
    dense: List[Coordinate] = [tuple(path[0])]
    for a, b in zip(path[:-1], path[1:]):
        seg = float(haversine_km(a[0], a[1], b[0], b[1]))
        # Number of samples including both ends; the max(..., 1e-6) guards
        # against division by zero for a non-positive step.
        n = max(2, int(seg / max(step_km, 1e-6)) + 1)
        # [1:] skips the first sample, which equals the previous segment's end.
        for p in great_circle_points(tuple(a), tuple(b), n)[1:]:
            dense.append(p)
    return dense


def douglas_peucker(path: Sequence[Coordinate], tolerance_km: float = 3.0) ->
List[Coordinate]:
    """Simplify a poly-line, keeping its shape within ``tolerance_km``.

    道格拉斯-普克算法，用于压缩网格搜索产生的锯齿状路径。

    Algorithm (Ramer-Douglas-Peucker): keep the two end points; find the
    interior vertex farthest from the chord between them; if that distance
    exceeds the tolerance, keep the vertex and repeat on the two halves,
    otherwise drop every interior vertex of the span. The recursion is
    implemented with an explicit stack so that long lattice paths (thousands
    of cells) cannot hit Python's recursion limit. Complexity is O(n log n)
    on average and O(n^2) in the worst case.

    Parameters
    ----------
    path : sequence of Coordinate
        Poly-line ``(lat, lon)`` in degrees, typically the staircase path of
        the lattice search.
    tolerance_km : float, default 3.0
        Maximum allowed perpendicular deviation of a removed vertex from the
        simplified line, kilometres.

    Returns
    -------
    list of Coordinate
        Subset of the input vertices in their original order; the first and
        the last vertex are always kept.

    Notes
    -----
    The simplification checks only geometry, not the cost map: a shortcut
    within ``tolerance_km`` may cut a corner of a land cell. The planner
    therefore applies it only when the user asks for it (``simplify_km > 0``).
    """
    pts = [tuple(p) for p in path]
    if len(pts) < 3:
        return pts

    def perpendicular_km(pt: Coordinate, a: Coordinate, b: Coordinate) -> float:
        """Distance from ``pt`` to the segment ``a``-``b`` in kilometres.

        The three points are projected onto a local plane (equirectangular
        projection around the mean latitude of the segment): one degree of
        longitude is ``111.320 * cos(lat0)`` km and one degree of latitude
        110.574 km (WGS 84 values at the equator). The point is then projected
        onto the segment with the parameter ``t`` clamped to [0, 1], so the
        distance is to the *segment*, not to the infinite line.
        局部平面近似下点到线段的距离（千米）。

        Parameters
        ----------
        pt : Coordinate
            The vertex being tested.
        a, b : Coordinate
            End points of the chord.

        Returns
        -------
        float
            Distance in kilometres.
```

```
        """
        # Local planar approximation is accurate enough at the tolerances used.
        lat0 = math.radians((a[0] + b[0]) / 2.0)
        kx = 111.320 * math.cos(lat0)       # km per degree of longitude at lat0
        ky = 110.574                        # km per degree of latitude
        # Vectors a->pt and a->b in kilometres (x = east, y = north).
        px, py = (pt[1] - a[1]) * kx, (pt[0] - a[0]) * ky
        bx, by = (b[1] - a[1]) * kx, (b[0] - a[0]) * ky
        den = bx * bx + by * by
        if den < 1e-12:
            # Degenerate chord (a == b): distance to the point a.
            return math.hypot(px, py)
        # Projection parameter of pt on a->b, clamped to the segment.
        t = max(0.0, min(1.0, (px * bx + py * by) / den))
        return math.hypot(px - t * bx, py - t * by)

    keep = [False] * len(pts)
    keep[0] = keep[-1] = True
    # Stack of index spans (i, j) still to be examined. 待处理的区间栈。
    stack = [(0, len(pts) - 1)]
    while stack:
        i, j = stack.pop()
        if j <= i + 1:
            continue                        # no interior vertex in this span
        best, best_d = -1, 0.0
        for k in range(i + 1, j):
            d = perpendicular_km(pts[k], pts[i], pts[j])
            if d > best_d:
                best, best_d = k, d
        if best_d > tolerance_km and best > 0:
            # The farthest vertex is significant: keep it and split the span.
            keep[best] = True
            stack.append((i, best))
            stack.append((best, j))
        # Otherwise every interior vertex of (i, j) stays keep=False (dropped).
    return [p for p, k in zip(pts, keep) if k]


def bounding_box(points: Iterable[Coordinate], margin_deg: float = 0.0):
    """Axis aligned bounding box (lat_min, lat_max, lon_min, lon_max).

    The box is enlarged by ``margin_deg`` on every side and clamped to
    latitude [-89.5, 89.5] (away from the poles, where a longitude lattice
    degenerates) and longitude [-180, 180]. Boxes crossing the antimeridian
    are not supported: the clamp simply cuts them at +/-180.
    外包矩形：四周扩展 margin_deg 度，并裁剪到合法经纬度范围。

    Parameters
    ----------
    points : iterable of Coordinate
        Points ``(lat, lon)`` in degrees; must contain at least one point.
    margin_deg : float, default 0.0
        Margin added on each side, degrees.

    Returns
    -------
    tuple of float
        ``(lat_min, lat_max, lon_min, lon_max)`` in degrees.

    Raises
    ------
    IndexError, ValueError
        If ``points`` is empty (NumPy cannot index or reduce an empty array).
    """
    arr = np.asarray(list(points), dtype=float)
    lat_min = float(arr[:, 0].min()) - margin_deg
    lat_max = float(arr[:, 0].max()) + margin_deg
    lon_min = float(arr[:, 1].min()) - margin_deg
    lon_max = float(arr[:, 1].max()) + margin_deg
    return (max(-89.5, lat_min), min(89.5, lat_max), max(-180.0, lon_min), min(180.0,
lon_max))
```

routing/cost_map.py – lattice and traversal cost map

```
"""Construction of the navigation cost map from neural-network classification.

通行代价地图构建模块。
The planner works on a regular geographic lattice. Every cell centre is
```

```
classified by the neural network; the predicted zone is converted into a
traversal cost, and cells adjacent to land receive an additional safety
penalty so that the optimiser keeps a clearance from the shoreline.

Role in the pipeline
--------------------
This module is the bridge between the classifier and the graph searches::

    GridSpec (make_grid_spec)            - where the lattice lies and how fine it is
      -> classifier.predict_coordinates  - zone probabilities per lattice node
      -> CostMap (build_cost_map)        - class, confidence, traversal cost per node
      -> astar / dijkstra / genetic      - search on the 8-connected lattice graph

Lattice
-------
The lattice is a regular grid in latitude/longitude with the same step
``resolution_deg`` in both directions. Node ``(row, col)`` lies at::

    lat = lat_min + row * resolution_deg
    lon = lon_min + col * resolution_deg

so ``row`` grows northwards and ``col`` eastwards, and the first and the last
row/column lie on the box boundary. A node is called a "cell" throughout the
code; it stands for the area of one resolution step around it. In kilometres a
cell is ``111.2 * res`` tall and ``111.2 * res * cos(lat)`` wide, so cells
become narrower towards the poles; the searches account for this through
latitude-dependent step lengths (see ``astar._cell_distance_table``).

Cost model
----------
Each zone class has a weight ``w`` from :data:`~maritime_route.config.ZONE_COST`
(open sea 1.0, coastal sea 1.8, near-coast and coastline infinite). Infinite
weights are stored as the finite sentinel
:data:`~maritime_route.config.IMPASSABLE_COST` (1e6) so that the array stays a
plain ``float32`` array; every cell with ``cost >= IMPASSABLE_COST`` is land.
Water cells close to land are multiplied by ``proximity_penalty`` (the safety
margin). The searches then charge ``d(u, v) * (w(u) + w(v)) / 2`` for a move
between neighbouring cells ``u`` and ``v``.

网格节点 (row, col) 对应纬度 lat_min + row*res、经度 lon_min + col*res;
代价 >= IMPASSABLE_COST 的单元为陆地（不可通行）。
"""
from __future__ import annotations

import math

import logging
import time
from dataclasses import dataclass
from typing import Callable, Dict, List, Optional, Tuple

import numpy as np

from ..config import (
    CLASS_NAMES,
    IMPASSABLE_COST,
    INDEX_TO_CLASS,
    NAVIGABLE_CLASSES,
    ROUTING,
    ZONE_COST,
    RoutingConfig,
)

LOGGER = logging.getLogger(__name__)

#: Optional callback ``progress(stage, fraction, details)`` used to report the
#: build progress to the web client (``fraction`` in [0, 1]).
#: 进度回调：progress(阶段名, 完成比例, 详情字典)。
ProgressCallback = Optional[Callable[[str, float, Dict], None]]


@dataclass
class GridSpec:
    """Geographic lattice definition. 规则经纬网格定义。

    A regular latitude/longitude lattice covering the box
    ``[lat_min, lat_max] x [lon_min, lon_max]`` with step ``resolution_deg``.
    The class only stores the five numbers; the shape and all coordinate
    conversions are derived from them.
```

```
    Attributes
    ----------
    lat_min, lat_max : float
        Southern and northern edge of the box, degrees.
    lon_min, lon_max : float
        Western and eastern edge of the box, degrees. The box must not cross
        the antimeridian (``lon_min < lon_max``).
    resolution_deg : float
        Distance between neighbouring nodes in both latitude and longitude,
        degrees (0.25 deg ~ 27.8 km in latitude).
    """

    lat_min: float
    lat_max: float
    lon_min: float
    lon_max: float
    resolution_deg: float

    @property
    def n_rows(self) -> int:
        """Number of lattice rows (latitude direction).

        ``round(span / resolution) + 1`` because both boundaries are nodes
        (fence-post count); at least 2 so that the lattice is never degenerate.

        Returns
        -------
        int
            Row count, >= 2.
        """
        return max(2, int(round((self.lat_max - self.lat_min) / self.resolution_deg)) +
1)

    @property
    def n_cols(self) -> int:
        """Number of lattice columns (longitude direction), >= 2.

        Computed like :attr:`n_rows` from the longitude span.

        Returns
        -------
        int
            Column count, >= 2.
        """
        return max(2, int(round((self.lon_max - self.lon_min) / self.resolution_deg)) +
1)

    @property
    def n_cells(self) -> int:
        """Total number of lattice nodes, ``n_rows * n_cols``.

        This is the quantity limited by ``RoutingConfig.max_grid_cells``,
        because it determines both classifier time and search memory.

        Returns
        -------
        int
            Number of cells.
        """
        return self.n_rows * self.n_cols

    def cell_to_coord(self, row: int, col: int) -> Tuple[float, float]:
        """Geographic coordinate of a lattice node.

        Parameters
        ----------
        row, col : int
            Lattice indices; not range-checked.

        Returns
        -------
        tuple of float
            ``(lat, lon)`` in degrees: ``lat_min + row * res``,
            ``lon_min + col * res``.
        """
        return (self.lat_min + row * self.resolution_deg,
                self.lon_min + col * self.resolution_deg)

    def coord_to_cell(self, lat: float, lon: float) -> Tuple[int, int]:
        """Nearest lattice node of a geographic coordinate.
```

```
        Inverse of :meth:`cell_to_coord`: the offset from the south-west
        corner is divided by the step and rounded to the nearest integer, so
        a point is assigned to the node within half a step of it. Points
        outside the box are clamped onto the border row/column instead of
        raising an error, which lets callers sample lines that leave the box.
        坐标转网格：四舍五入到最近节点，并裁剪到网格范围内。

        Parameters
        ----------
        lat, lon : float
            Coordinate in degrees.

        Returns
        -------
        tuple of int
            ``(row, col)`` with ``0 <= row < n_rows`` and ``0 <= col < n_cols``.
        """
        row = int(round((lat - self.lat_min) / self.resolution_deg))
        col = int(round((lon - self.lon_min) / self.resolution_deg))
        return (min(max(row, 0), self.n_rows - 1), min(max(col, 0), self.n_cols - 1))

    def mesh(self) -> Tuple[np.ndarray, np.ndarray]:
        """Latitude and longitude of every node as two 2-D arrays.

        ``indexing="ij"`` makes the first axis the row (latitude) and the
        second the column (longitude), matching ``class_index[row, col]``.

        Returns
        -------
        tuple of numpy.ndarray
            ``(grid_lat, grid_lon)``, each of shape ``(n_rows, n_cols)``,
            degrees.
        """
        lats = self.lat_min + np.arange(self.n_rows) * self.resolution_deg
        lons = self.lon_min + np.arange(self.n_cols) * self.resolution_deg
        grid_lat, grid_lon = np.meshgrid(lats, lons, indexing="ij")
        return grid_lat, grid_lon

    def to_dict(self) -> Dict:
        """JSON-serialisable description of the lattice.

        Used in API responses, in :class:`~maritime_route.routing.planner.RoutePlan`
        and in the benchmark files. Box edges are rounded to 5 decimals
        (about 1 m).

        Returns
        -------
        dict
            Box edges, resolution and the derived row, column and cell counts.
        """
        return {
            "lat_min": round(self.lat_min, 5), "lat_max": round(self.lat_max, 5),
            "lon_min": round(self.lon_min, 5), "lon_max": round(self.lon_max, 5),
            "resolution_deg": self.resolution_deg,
            "n_rows": self.n_rows, "n_cols": self.n_cols, "n_cells": self.n_cells,
        }


@dataclass
class CostMap:
    """Classified lattice plus the derived traversal cost. 代价地图。

    All three arrays have shape ``(spec.n_rows, spec.n_cols)`` and are indexed
    ``[row, col]`` exactly like the lattice.

    Attributes
    ----------
    spec : GridSpec
        Lattice geometry.
    class_index : numpy.ndarray of int8
        Predicted zone of every node as an index into
        :data:`~maritime_route.config.CLASS_NAMES`
        (0 OPEN_SEA, 1 COASTAL_SEA, 2 NEAR_COAST, 3 COASTLINE).
    confidence : numpy.ndarray of float32
        Probability of the predicted class (max of the softmax output), 0..1.
    cost : numpy.ndarray of float32
        Traversal weight ``w`` of every node after the safety margin;
        ``IMPASSABLE_COST`` marks land.
    build_seconds : float
```

```
            Wall-clock time spent building the map (classification included).
        """

        spec: GridSpec
        class_index: np.ndarray   # (rows, cols) int8
        confidence: np.ndarray    # (rows, cols) float32
        cost: np.ndarray          # (rows, cols) float32, IMPASSABLE_COST on land
        build_seconds: float = 0.0

        @property
        def passable(self) -> np.ndarray:
            """Boolean mask of navigable cells.

            A cell is navigable when its cost is below the land sentinel. Water
            cells penalised by the safety margin stay navigable because their
            cost is capped at ``IMPASSABLE_COST - 1``.

            Returns
            -------
            numpy.ndarray of bool
                Shape ``(n_rows, n_cols)``; recomputed on every access.
            """
            return self.cost < IMPASSABLE_COST

        def zone_of(self, row: int, col: int) -> str:
            """Zone class name of one cell.

            Parameters
            ----------
            row, col : int
                Lattice indices.

            Returns
            -------
            str
                One of :data:`~maritime_route.config.CLASS_NAMES`.
            """
            return INDEX_TO_CLASS[int(self.class_index[row, col])]

        def statistics(self) -> Dict:
            """Summary of the map for the UI, the reports and the benchmarks.

            Returns
            -------
            dict
                ``grid`` (see :meth:`GridSpec.to_dict`), total and navigable cell
                counts, the navigable share, the count and share of every zone,
                the mean classifier confidence and the build time in seconds.
            """
            total = int(self.class_index.size)
            counts = {
                name: int((self.class_index == i).sum()) for i, name in
    enumerate(CLASS_NAMES)
            }
            return {
                "grid": self.spec.to_dict(),
                "cells_total": total,
                "cells_navigable": int(self.passable.sum()),
                "navigable_share": round(float(self.passable.mean()), 4),
                "zone_counts": counts,
                # max(total, 1) avoids a division by zero for an empty map.
                "zone_shares": {k: round(v / max(total, 1), 4) for k, v in counts.items()},
                "mean_confidence": round(float(self.confidence.mean()), 4),
                "build_seconds": round(self.build_seconds, 3),
            }

        def to_overlay(self, max_cells: int = 20000) -> Dict:
            """Down-sampled cell list for rendering the classification on the map.

            为前端地图生成降采样的分类图层。返回的 ``cell_size_deg`` 已按降采样
            步长放大，使前端绘制的矩形彼此相接而不留缝隙。
            Each returned cell represents a ``step x step`` block, whose class is the
            majority vote of the block so that thin land features are not lost.

            The step is ``ceil(sqrt(n_cells / max_cells))``: taking every
            ``step``-th row and column divides the cell count by about
            ``step^2``, which brings it down to ``max_cells`` or fewer. A browser
            can draw about 20 000 rectangles without noticeable delay.

            Parameters
```

```
        ----------
        max_cells : int, default 20000
            Upper bound on the number of cells sent to the client.

        Returns
        -------
        dict
            ``cells`` - list of ``{"lat", "lon", "c", "p"}`` where ``lat``/``lon``
            is the centre of the block (degrees, 4 decimals), ``c`` the class
            index and ``p`` the mean confidence of the block;
            ``step`` - the down-sampling factor;
            ``cell_size_deg`` - side of one drawn block in degrees.
        """
        step = max(1, int(np.ceil(np.sqrt(self.spec.n_cells / max(max_cells, 1)))))
        cells: List[Dict] = []
        for row in range(0, self.spec.n_rows, step):
            for col in range(0, self.spec.n_cols, step):
                # Blocks on the northern/eastern border may be smaller than
                # step x step; NumPy slicing truncates them automatically.
                block = self.class_index[row:row + step, col:col + step]
                counts = np.bincount(block.ravel(), minlength=len(CLASS_NAMES))
                # Majority vote of the block. On a tie argmax returns the lowest
                # class index, i.e. the water class.
                # (An earlier note said "land dominates visually: keep a block
                # as land if any cell is land"; the code uses the majority.)
                # 块内多数投票；平票时取索引较小的类别。
                winner = int(counts.argmax())
                lat, lon = self.spec.cell_to_coord(row, col)
                cells.append({
                    # Shift from the block's first node to the block centre:
                    # the block spans (step - 1) steps, half of it is added.
                    "lat": round(lat + (step - 1) * self.spec.resolution_deg / 2.0, 4),
                    "lon": round(lon + (step - 1) * self.spec.resolution_deg / 2.0, 4),
                    "c": winner,
                    "p": round(float(self.confidence[row:row + step, col:col +
step].mean()), 3),
                 })
        return {
            "cells": cells,
            "step": step,
            "cell_size_deg": round(self.spec.resolution_deg * step, 5),
        }


def make_grid_spec(
    waypoints: List[Tuple[float, float]],
    resolution_deg: float = ROUTING.grid_resolution_deg,
    margin_deg: float = ROUTING.margin_deg,
    max_cells: int = ROUTING.max_grid_cells,
) -> GridSpec:
    """Bounding box around the waypoints, coarsened until it fits the budget.

    自动确定网格范围与分辨率：单元数超过上限时自动降低分辨率。

    The box is the bounding box of the waypoints (normally departure and
    destination) enlarged by ``margin_deg`` on each side, so that the route
    can leave the straight corridor between the ports to go round a
    peninsula. Latitude is clamped to +/-85 deg (the limit of the Web
    Mercator map in the client) and longitude to +/-180 deg.

    If the lattice would exceed ``max_cells`` nodes, the step is multiplied by
    1.5 repeatedly (each step reduces the cell count by about 2.25x) until
    it fits or the step reaches 4 deg. This bounds both classifier time and
    search memory for ocean-scale voyages.

    Parameters
    ----------
    waypoints : list of (lat, lon)
        Points that must lie inside the lattice, degrees.
    resolution_deg : float
        Requested lattice step, degrees.
    margin_deg : float
        Margin around the bounding box, degrees.
    max_cells : int
        Upper limit for ``n_rows * n_cols``.

    Returns
    -------
    GridSpec
        Lattice whose resolution is ``resolution_deg`` or a coarser value.
```

```
    """
    arr = np.asarray(waypoints, dtype=float)
    lat_min = max(-85.0, float(arr[:, 0].min()) - margin_deg)
    lat_max = min(85.0, float(arr[:, 0].max()) + margin_deg)
    lon_min = max(-180.0, float(arr[:, 1].min()) - margin_deg)
    lon_max = min(180.0, float(arr[:, 1].max()) + margin_deg)

    spec = GridSpec(lat_min, lat_max, lon_min, lon_max, resolution_deg)
    # Coarsen by a factor of 1.5 until the cell budget is met (upper limit 4 deg).
    # 每次将分辨率放大 1.5 倍，直到单元数不超过上限（分辨率上限 4 度）。
    while spec.n_cells > max_cells and spec.resolution_deg < 4.0:
        spec = GridSpec(lat_min, lat_max, lon_min, lon_max, round(spec.resolution_deg *
1.5, 4))
        LOGGER.info("Grid too large, coarsening to %.3f deg", spec.resolution_deg)
    return spec


def build_cost_map(
    spec: GridSpec,
    classifier,
    config: RoutingConfig = ROUTING,
    progress: ProgressCallback = None,
    batch_size: int = 16384,
) -> CostMap:
    """Classify every lattice cell and convert the zones into traversal costs.

    对网格每个单元进行神经网络分类并转换为通行代价。

    Steps:

    1. Flatten the node coordinates and classify them in batches of
       ``batch_size`` (bounds the memory of the feature matrix and lets the
       client see progress). The predicted class is the argmax of the
       probabilities; the confidence is the maximum probability.
    2. Map each class to its weight from ``ZONE_COST``; classes with an
       infinite weight keep the ``IMPASSABLE_COST`` sentinel.
    3. Apply the safety margin (:func:`_apply_safety_margin`) to water cells
       next to land.

    Parameters
    ----------
    spec : GridSpec
        Lattice to classify.
    classifier
        Object with ``predict_coordinates(lat, lon) -> ndarray (n, 4)`` of
        class probabilities, normally
        :class:`~maritime_route.model.inference.ZoneClassifierService`.
    config : RoutingConfig
        Supplies ``safety_margin_cells`` and ``proximity_penalty``.
    progress : callable, optional
        Receives ``("classify", fraction, {...})`` after every batch and
        ``("cost_map", 1.0, statistics)`` at the end.
    batch_size : int, default 16384
        Number of nodes classified per call.

    Returns
    -------
    CostMap
        The classified lattice with traversal costs and its build time.
    """
    t0 = time.perf_counter()
    grid_lat, grid_lon = spec.mesh()
    # Row-major flattening: flat index = row * n_cols + col; reshaped back below.
    flat_lat = grid_lat.ravel()
    flat_lon = grid_lon.ravel()
    n = flat_lat.size

    class_flat = np.zeros(n, dtype=np.int8)
    conf_flat = np.zeros(n, dtype=np.float32)

    # At least ~20 batches, so that the web client receives regular progress
    # frames even for a small lattice; never fewer than 1024 cells per batch.
    # 至少分成约 20 批，使小网格也能向前端连续推送进度；每批不少于 1024 个格子。
    batch_size = min(batch_size, max(1024, math.ceil(n / 20)))
    for start in range(0, n, batch_size):
        stop = min(start + batch_size, n)
        probs = classifier.predict_coordinates(flat_lat[start:stop],
flat_lon[start:stop])
        # Predicted class = most probable zone; confidence = its probability.
        class_flat[start:stop] = probs.argmax(axis=1).astype(np.int8)
```

```
        conf_flat[start:stop] = probs.max(axis=1).astype(np.float32)
        if progress is not None:
            progress("classify", stop / n, {"cells_done": int(stop), "cells_total":
int(n)})

    class_index = class_flat.reshape(spec.n_rows, spec.n_cols)
    confidence = conf_flat.reshape(spec.n_rows, spec.n_cols)

    # Start with every cell impassable, then write the finite zone weights.
    # 先全部设为不可通行，再为有限权重的类别写入代价。
    cost = np.full(class_index.shape, IMPASSABLE_COST, dtype=np.float32)
    for i, name in enumerate(CLASS_NAMES):
        weight = ZONE_COST[name]
        if np.isfinite(weight):
            cost[class_index == i] = float(weight)

    # Land mask = every class that is not navigable (NEAR_COAST, COASTLINE).
    land = ~np.isin(class_index, [CLASS_NAMES.index(c) for c in NAVIGABLE_CLASSES])
    cost = _apply_safety_margin(cost, land, config)

    cost_map = CostMap(spec=spec, class_index=class_index, confidence=confidence,
                       cost=cost, build_seconds=time.perf_counter() - t0)
    if progress is not None:
        progress("cost_map", 1.0, cost_map.statistics())
    LOGGER.info("Cost map built: %s", cost_map.statistics()["zone_shares"])
    return cost_map


def _apply_safety_margin(cost: np.ndarray, land: np.ndarray, config: RoutingConfig) ->
np.ndarray:
    """Penalise water cells within ``safety_margin_cells`` of land.

    安全余量：靠近陆地的水域单元代价乘以惩罚系数，使航线远离海岸。

    The land mask is dilated ``safety_margin_cells`` times with a
    4-neighbourhood (cross-shaped) structuring element: each pass marks every
    cell whose north, south, west or east neighbour is already marked. After
    ``m`` passes the mask covers every cell within Manhattan distance ``m`` of
    land. The water cells in that band have their weight multiplied by
    ``proximity_penalty`` (default 1.6, so open sea 1.0 -> 1.6 and coastal sea
    1.8 -> 2.88). The result is capped at ``IMPASSABLE_COST - 1`` so that a
    penalised cell never turns into land: narrow straits stay passable, only
    more expensive. The searches therefore prefer a route one or more cells
    away from the shore whenever such a route exists.

    Parameters
    ----------
    cost : numpy.ndarray
        Cell weights before the margin; not modified.
    land : numpy.ndarray of bool
        True for non-navigable cells.
    config : RoutingConfig
        ``safety_margin_cells`` (band width in cells) and
        ``proximity_penalty`` (multiplier).

    Returns
    -------
    numpy.ndarray
        A new cost array, or the input array itself when the margin is
        disabled (width 0 or penalty <= 1).
    """
    margin = max(0, int(config.safety_margin_cells))
    if margin == 0 or config.proximity_penalty <= 1.0:
        return cost
    influence = land.astype(np.float32)
    padded = influence
    for _ in range(margin):
        # One dilation pass: copy the mask shifted by one cell in each of the
        # four axis directions, without wrapping around the array edges.
        # 一次膨胀：将掩膜沿上下左右各平移一格后取最大值。
        shifted = np.zeros_like(padded)
        shifted[1:, :] = np.maximum(shifted[1:, :], padded[:-1, :])    # from the south
neighbour
        shifted[:-1, :] = np.maximum(shifted[:-1, :], padded[1:, :])   # from the north
neighbour
        shifted[:, 1:] = np.maximum(shifted[:, 1:], padded[:, :-1])    # from the west
neighbour
        shifted[:, :-1] = np.maximum(shifted[:, :-1], padded[:, 1:])   # from the east
neighbour
        padded = np.maximum(padded, shifted)
```

```
    # Water cells inside the dilated band (land itself is excluded).
    near_land = (padded > 0) & (~land)
    result = cost.copy()
    result[near_land] = np.minimum(
        result[near_land] * float(config.proximity_penalty), IMPASSABLE_COST - 1.0
    )
    return result


def nearest_navigable(cost_map: CostMap, row: int, col: int, max_radius: int = 40
                      ) -> Optional[Tuple[int, int]]:
    """Snap a cell onto the closest navigable cell (ring search).

    将起点/终点吸附到最近的可航行单元。

    Ports lie on the shore, so their lattice cell is usually classified as
    land. The search examines square rings of growing Chebyshev radius
    ``r = 1, 2, ...`` around the cell; only the perimeter of each ring is
    visited (the full top and bottom rows, and the two side cells of every
    other row). The first ring that contains a navigable cell ends the search,
    and within that ring the cell with the smallest Euclidean distance
    ``dr^2 + dc^2`` (in cell units) is returned.

    The result is the nearest cell up to the ring granularity: a corner cell
    of ring ``r`` (distance ``r * sqrt(2)``) can be chosen although an axis
    cell of ring ``r + 1`` would be slightly closer. Distances are counted in
    cells, not kilometres. Both effects are below one cell and irrelevant for
    snapping a port.
    按方环逐圈向外搜索，返回第一个含可航行单元的环中欧氏距离最小的单元。

    Parameters
    ----------
    cost_map : CostMap
        Map providing the ``passable`` mask.
    row, col : int
        Cell to snap.
    max_radius : int, default 40
        Largest ring radius examined, in cells. The planner derives it from
        :attr:`RoutePlanner.SNAP_RADIUS_KM` so that a point far inland is not
        moved into open sea.

    Returns
    -------
    tuple of int or None
        ``(row, col)`` of a navigable cell (the input cell itself if it is
        navigable), or ``None`` if none lies within ``max_radius``.
    """
    if cost_map.passable[row, col]:
        return (row, col)
    rows, cols = cost_map.class_index.shape
    for radius in range(1, max_radius + 1):
        best: Optional[Tuple[int, int]] = None
        best_d = float("inf")
        for dr in range(-radius, radius + 1):
            # Top/bottom row of the ring (|dr| == radius): every column.
            # Other rows: only the left and right edge (dc = -radius, +radius).
            for dc in (-radius, radius) if abs(dr) != radius else range(-radius, radius
+ 1):
                r, c = row + dr, col + dc
                if 0 <= r < rows and 0 <= c < cols and cost_map.passable[r, c]:
                    d = dr * dr + dc * dc       # squared Euclidean distance, cells^2
                    if d < best_d:
                        best, best_d = (r, c), d
        if best is not None:
            return best
    return None
```

routing/astar.py – the A* algorithm

```
"""A* search over the navigation cost map.

A* 寻路算法实现。
Cost model: moving from cell *u* to neighbour *v* costs

    d(u, v) * (w(u) + w(v)) / 2

where ``d`` is the great-circle distance in kilometres and ``w`` the zone
weight produced by the neural classifier. The heuristic is the great-circle
```

```
distance to the goal multiplied by the minimum zone weight, which keeps it
admissible (never over-estimates) and therefore keeps A* optimal.

Role in the pipeline
--------------------
A* is the default search of :class:`~maritime_route.routing.planner.RoutePlanner`.
It receives a :class:`~maritime_route.routing.cost_map.CostMap` and two
navigable lattice cells and returns a :class:`SearchResult`. The module also
holds the pieces shared with :mod:`.dijkstra` and :mod:`.genetic`: the
neighbourhood offsets, the step-length table, path reconstruction and the
path-length helper.

Graph
-----
Nodes are the lattice cells; each cell is connected to its 8 neighbours
(4 when ``RoutingConfig.diagonal_moves`` is off). Land cells
(``cost >= IMPASSABLE_COST``) are not entered. The edge cost is the length of
the move multiplied by the mean weight of its two end cells - the trapezoidal
rule for the integral of the weight along the segment. The unit of cost is
therefore "weighted kilometres": a route entirely in open sea (w = 1) costs
exactly its length in km.

Admissibility of the heuristic
------------------------------
Let ``w_min`` be the smallest weight of any navigable cell. Every edge costs
at least ``d(u, v) * w_min``, and the sum of the edge lengths of any path from
``n`` to the goal is at least the great-circle distance ``d_gc(n, goal)``
(triangle inequality on the sphere). Hence

    h(n) = w_min * d_gc(n, goal)

never exceeds the true remaining cost, i.e. it is admissible. It is also
consistent (``h(u) <= c(u, v) + h(v)``), so a cell never has to be re-opened
after it has been closed, and the first time the goal is popped its cost is
optimal. ``RoutingConfig.heuristic_weight`` > 1 would turn this into weighted
A*, which is faster but no longer guaranteed optimal; the default is 1.

启发函数 h(n) = 最小权重 × 到终点的大圆距离，既可采纳又一致，保证 A* 得到最优解。
"""
from __future__ import annotations

import heapq
import math
import time
from dataclasses import dataclass, field
from typing import Callable, Dict, List, Optional, Tuple

import numpy as np

from ..config import EARTH_RADIUS_KM, IMPASSABLE_COST, ROUTING, RoutingConfig
from .cost_map import CostMap
from .geodesy import haversine_km

#: A lattice cell as ``(row, col)``. 网格单元 (行, 列)。
Cell = Tuple[int, int]
#: Optional callback ``progress(stage, fraction, details)`` for the web client.
ProgressCallback = Optional[Callable[[str, float, Dict], None]]

#: 8-connected neighbourhood offsets. 八邻域偏移量。
#: ``(drow, dcol)``: the four axis moves (N, S, W, E) first, then the four
#: diagonals. A diagonal move is about sqrt(2) times longer than an axis move
#: (exactly so only where one degree of longitude equals one of latitude).
OFFSETS_8: Tuple[Cell, ...] = ((-1, 0), (1, 0), (0, -1), (0, 1),
                               (-1, -1), (-1, 1), (1, -1), (1, 1))
#: 4-connected neighbourhood (axis moves only), used when diagonal moves are off.
#: 四邻域偏移量（不允许斜向移动时使用）。
OFFSETS_4: Tuple[Cell, ...] = ((-1, 0), (1, 0), (0, -1), (0, 1))


@dataclass
class SearchResult:
    """Outcome of a graph search. 搜索结果。

    Returned by :func:`astar`, :func:`~maritime_route.routing.dijkstra.dijkstra`,
    :func:`~maritime_route.routing.genetic.genetic` and the planner's
    great-circle wrapper, so that the planner treats every method alike.

    Attributes
    ----------
    found : bool
```

```
        True if a navigable path was found.
    path_cells : list of Cell
        Lattice cells from start to goal (empty if not found; empty for the
        great-circle reference, which is not a lattice path).
    path_coords : list of (lat, lon)
        Coordinates of the path vertices, degrees.
    total_cost : float
        Sum of edge costs, weighted km; ``inf`` when no path was found.
    distance_km : float
        Geometric length of ``path_coords``, km.
    nodes_expanded : int
        Cells removed from the priority queue and expanded (A*, Dijkstra) or
        number of fitness evaluations (genetic algorithm).
    nodes_generated : int
        Queue insertions (A*, Dijkstra) or generations run (genetic algorithm).
    runtime_s : float
        Wall-clock time of the search, seconds.
    algorithm : str
        ``"astar"``, ``"dijkstra"``, ``"genetic"`` or ``"great_circle"``.
    message : str
        Human-readable outcome shown in the UI.
    history : list of float
        Best fitness after each generation (genetic algorithm only).
    """

    found: bool
    path_cells: List[Cell] = field(default_factory=list)
    path_coords: List[Tuple[float, float]] = field(default_factory=list)
    total_cost: float = float("inf")
    distance_km: float = 0.0
    nodes_expanded: int = 0
    nodes_generated: int = 0
    runtime_s: float = 0.0
    algorithm: str = "astar"
    message: str = ""
    #: Best fitness per generation (genetic algorithm only). 每代最优适应度（仅遗传算法）。
    history: List[float] = field(default_factory=list)
    #: Value function V (dynamic programming only), shape (n_rows, n_cols).
    #: 价值函数（仅动态规划）。
    value: Optional[np.ndarray] = None

    def to_dict(self) -> Dict:
        """JSON-serialisable summary without the path itself.

        An infinite ``total_cost`` is written as ``None`` because JSON has no
        representation for infinity.

        Returns
        -------
        dict
            Scalar fields rounded for display plus ``n_waypoints``.
        """
        return {
            "found": self.found,
            "algorithm": self.algorithm,
            "total_cost": None if not math.isfinite(self.total_cost) else
round(self.total_cost, 3),
            "distance_km": round(self.distance_km, 3),
            "nodes_expanded": self.nodes_expanded,
            "nodes_generated": self.nodes_generated,
            "runtime_s": round(self.runtime_s, 4),
            "n_waypoints": len(self.path_coords),
            "message": self.message,
        }


def _cell_distance_table(cost_map: CostMap) -> Dict[Cell, np.ndarray]:
    """Pre-compute per-row step lengths for each neighbour offset.

    预计算每一行、每个方向的步长(km)，避免在搜索内部反复做三角函数运算。

    On a regular latitude/longitude lattice the length of a move depends only
    on the latitude of the starting row and on the direction, not on the
    column: an east-west move shrinks with ``cos(lat)``, a north-south move is
    constant. The table therefore stores, for every offset ``(dr, dc)``, a
    vector of ``n_rows`` lengths computed once with :func:`haversine_km`
    (longitude difference ``dc * res`` measured from longitude 0, which is
    equivalent because only the difference matters).

    Parameters
```

```
    ----------
    cost_map : CostMap
        Supplies the lattice geometry.

    Returns
    -------
    dict
        ``{(dr, dc): ndarray of shape (n_rows,)}`` - length in km of the move
        ``(dr, dc)`` starting from each row. All 8 offsets are included even
        when only 4 are used.
    """
    spec = cost_map.spec
    table: Dict[Cell, np.ndarray] = {}
    rows = np.arange(spec.n_rows)
    lat = spec.lat_min + rows * spec.resolution_deg
    for dr, dc in OFFSETS_8:
        # Latitude of the target row; the clip keeps it valid for moves that
        # would leave the lattice (they are rejected by the bounds check anyway).
        lat2 = np.clip(lat + dr * spec.resolution_deg, -90.0, 90.0)
        table[(dr, dc)] = haversine_km(lat, 0.0, lat2, dc *
spec.resolution_deg).astype(np.float64)
    return table


def _reconstruct(came_from: Dict[Cell, Cell], goal: Cell) -> List[Cell]:
    """Rebuild the path by following predecessor links back from the goal.

    ``came_from`` maps each reached cell to the cell it was reached from on
    the best known path. The start cell has no entry, which ends the walk.
    沿前驱指针从终点回溯到起点，再反转得到路径。

    Parameters
    ----------
    came_from : dict
        Predecessor of every reached cell.
    goal : Cell
        Last cell of the path.

    Returns
    -------
    list of Cell
        Cells in order from start to goal.
    """
    path = [goal]
    while path[-1] in came_from:
        path.append(came_from[path[-1]])
    path.reverse()
    return path


def astar(
    cost_map: CostMap,
    start: Cell,
    goal: Cell,
    config: RoutingConfig = ROUTING,
    progress: ProgressCallback = None,
    progress_every: int = 20000,
) -> SearchResult:
    """A* shortest-cost path between two lattice cells.

    Classical A* with a binary heap and lazy deletion:

    * The open list is a heap of ``(f, g, cell)`` with ``f = g + h``. Python's
      ``heapq`` has no decrease-key operation, so when a cheaper path to a
      cell is found a *new* entry is pushed and the old one stays in the heap.
    * When an entry is popped for a cell that is already in the closed set
      it is a stale duplicate and is skipped ("lazy deletion").
    * Because the heuristic is consistent, the first pop of a cell is its
      final, optimal cost; the cell is then closed.
    * The search stops when the goal is popped.

    With ``h = 0`` this would be Dijkstra's algorithm; the heuristic only
    changes the order of expansion, steering the search towards the goal and
    expanding far fewer cells (compared in the benchmark).

    Parameters
    ----------
    cost_map : CostMap
        Lattice with traversal weights.
    start, goal : Cell
```

```
        ``(row, col)`` of departure and destination; both must be navigable
        (the planner snaps them beforehand).
    config : RoutingConfig
        Uses ``diagonal_moves`` (8- or 4-neighbourhood) and
        ``heuristic_weight`` (1.0 = admissible A*).
    progress : callable, optional
        Called every ``progress_every`` expansions with
        ``("astar", fraction, {"expanded", "frontier"})``.
    progress_every : int, default 20000
        Expansion interval between progress reports.

    Returns
    -------
    SearchResult
        ``found=True`` with the optimal path, cost and counters, or
        ``found=False`` with a message when start/goal is on land or the goal
        is unreachable.
    """
    t0 = time.perf_counter()
    spec = cost_map.spec
    weights = cost_map.cost
    offsets = OFFSETS_8 if config.diagonal_moves else OFFSETS_4
    steps = _cell_distance_table(cost_map)
    n_rows, n_cols = spec.n_rows, spec.n_cols

    if not cost_map.passable[start] or not cost_map.passable[goal]:
        return SearchResult(found=False, algorithm="astar",
                            runtime_s=time.perf_counter() - t0,
                            message="Start or goal cell is not navigable")

    # Smallest weight of any navigable cell: the factor that keeps h admissible.
    # 可航行单元的最小权重，用于保证启发函数可采纳。
    min_weight = float(np.min(weights[cost_map.passable])) if cost_map.passable.any()
else 1.0
    goal_lat, goal_lon = spec.cell_to_coord(*goal)
    h_weight = float(config.heuristic_weight)

    # The heuristic is evaluated once per generated node, so it is written with
    # the math module and pre-computed trigonometry: a NumPy call per node costs
    # roughly an order of magnitude more and dominates the whole search.
    # 启发函数每生成一个节点都会调用一次，这里改用 math 并预计算三角函数，
    # 否则 NumPy 的标量调用开销会成为搜索的瓶颈。
    # h = h_weight * w_min * 2R * asin(sqrt(a)) - the haversine formula with
    # the constant factor folded into `scale`.
    scale = h_weight * min_weight * 2.0 * EARTH_RADIUS_KM
    goal_lat_rad = math.radians(goal_lat)
    goal_lon_rad = math.radians(goal_lon)
    cos_goal_lat = math.cos(goal_lat_rad)
    lat0, res_rad = math.radians(spec.lat_min), math.radians(spec.resolution_deg)
    lon0 = math.radians(spec.lon_min)
    # Per-row latitude trigonometry is identical for every column.
    # row_sin[r] = sin^2(dlat / 2) and row_cos[r] = cos(lat_r) * cos(lat_goal):
    # the two latitude-only terms of the haversine formula.
    row_lat_rad = [lat0 + r * res_rad for r in range(n_rows)]
    row_sin = [math.sin((lat - goal_lat_rad) * 0.5) ** 2 for lat in row_lat_rad]
    row_cos = [math.cos(lat) * cos_goal_lat for lat in row_lat_rad]

    def heuristic(cell: Cell) -> float:
        """Admissible estimate of the remaining cost from ``cell`` to the goal.

        ``w_min * great-circle distance(cell, goal)`` (times
        ``heuristic_weight``), evaluated with the pre-computed per-row terms
        so that only one ``sin`` depends on the column.

        Parameters
        ----------
        cell : Cell
            ``(row, col)`` of the cell.

        Returns
        -------
        float
            Lower bound of the remaining cost, weighted km (0 at the goal).
        """
        row, col = cell
        dlon_half = (lon0 + col * res_rad - goal_lon_rad) * 0.5
        a = row_sin[row] + row_cos[row] * math.sin(dlon_half) ** 2
        if a <= 0.0:
            return 0.0
        # Clamp a <= 1 against rounding before asin(sqrt(a)).
```

```
        return scale * math.asin(math.sqrt(a) if a < 1.0 else 1.0)

    # g_score: best known cost from the start; came_from: predecessor links;
    # closed: cells whose optimal cost is final. 已知最优代价 / 前驱 / 关闭集。
    g_score: Dict[Cell, float] = {start: 0.0}
    came_from: Dict[Cell, Cell] = {}
    closed: set = set()
    # Heap entries are (f, g, cell); ties on f are broken by the smaller g.
    heap: List[Tuple[float, float, Cell]] = [(heuristic(start), 0.0, start)]
    expanded = 0
    generated = 1
    # Admissible heuristic value at the start - used for the progress estimate.
    h0 = max(heuristic(start), 1e-6)

    while heap:
        f_score, g_current, current = heapq.heappop(heap)
        if current in closed:
            # Stale duplicate left by a later, cheaper push (lazy deletion).
            # 过期的重复条目（惰性删除），直接跳过。
            continue
        closed.add(current)
        expanded += 1

        if current == goal:
            cells = _reconstruct(came_from, goal)
            coords = [spec.cell_to_coord(r, c) for r, c in cells]
            return SearchResult(
                found=True, path_cells=cells, path_coords=coords,
                total_cost=g_current,
                distance_km=_path_km(coords),
                nodes_expanded=expanded, nodes_generated=generated,
                runtime_s=time.perf_counter() - t0, algorithm="astar",
                message="Optimal path found",
            )

        if progress is not None and expanded % progress_every == 0:
            # Progress estimate: share of the initial heuristic distance
            # already covered; capped below 1 until the goal is reached.
            remaining = heuristic(current)
            done = float(np.clip(1.0 - remaining / h0, 0.0, 0.995))
            progress("astar", done, {"expanded": expanded, "frontier": len(heap)})

        row, col = current
        w_current = float(weights[row, col])
        for dr, dc in offsets:
            r, c = row + dr, col + dc
            if not (0 <= r < n_rows and 0 <= c < n_cols):
                continue                    # outside the lattice
            neighbour = (r, c)
            if neighbour in closed:
                continue                    # already final
            w_next = float(weights[r, c])
            if w_next >= IMPASSABLE_COST:
                continue                    # land cell - never entered
            # Edge cost d(u, v) * (w(u) + w(v)) / 2, d looked up by source row.
            # 边代价 = 步长 × 两端权重的平均值。
            step_km = float(steps[(dr, dc)][row])
            tentative = g_current + step_km * 0.5 * (w_current + w_next)
            if tentative < g_score.get(neighbour, float("inf")):
                # Cheaper path to the neighbour: record it and push a new entry
                # (the old entry, if any, becomes stale).
                g_score[neighbour] = tentative
                came_from[neighbour] = current
                heapq.heappush(heap, (tentative + heuristic(neighbour), tentative,
neighbour))
                generated += 1

    # Heap exhausted without reaching the goal: the two cells lie in
    # different connected water bodies of this lattice.
    return SearchResult(found=False, algorithm="astar", nodes_expanded=expanded,
                        nodes_generated=generated, runtime_s=time.perf_counter() - t0,
                        message="No navigable path exists between the given points")


def _path_km(coords: List[Tuple[float, float]]) -> float:
    """Great-circle length of a coordinate path in kilometres.

    Same computation as :func:`~maritime_route.routing.geodesy.path_length_km`;
    kept here as the shared helper imported by the search modules.
```

```
    Parameters
    ----------
    coords : list of (lat, lon)
        Path vertices, degrees.

    Returns
    -------
    float
        Length in km; 0.0 for fewer than two vertices.
    """
    if len(coords) < 2:
        return 0.0
    arr = np.asarray(coords, dtype=float)
    return float(np.sum(haversine_km(arr[:-1, 0], arr[:-1, 1], arr[1:, 0], arr[1:, 1])))
```

routing/dijkstra.py – the Dijkstra algorithm

```
"""Dijkstra search over the navigation cost map.

迪杰斯特拉算法实现（即启发函数恒为 0 的 A*），用于与 A* 做性能对比。
Implemented separately rather than as ``astar(h=0)`` so that the thesis can
present two genuinely independent implementations and compare their node
expansion counts on identical cost maps.

Role in the pipeline
--------------------
Selected in :class:`~maritime_route.routing.planner.RoutePlanner` with
``algorithm="dijkstra"`` and run next to A* by
:func:`~maritime_route.routing.planner.benchmark`. It uses exactly the same
graph (8- or 4-neighbourhood on the lattice), the same edge cost
``d(u, v) * (w(u) + w(v)) / 2`` and the same step-length table as
:mod:`.astar`, so both searches must return the same optimal cost; the
benchmark reports their difference as ``optimality_gap_pct`` (expected 0).

Algorithm
---------
Uniform-cost search: cells are expanded in order of increasing distance from
the start, using a binary heap with lazy deletion (see :func:`dijkstra`).
Without a heuristic the search grows as a roughly circular "wave front"
around the start and typically expands several times more cells than A*.
Time complexity is O((V + E) log V) for V cells and E = 8V edges.
"""
from __future__ import annotations

import heapq
import time
from typing import Dict, List, Tuple

import numpy as np

from ..config import IMPASSABLE_COST, ROUTING, RoutingConfig
from .astar import OFFSETS_4, OFFSETS_8, Cell, ProgressCallback, SearchResult,
_cell_distance_table, _path_km, _reconstruct
from .cost_map import CostMap


def dijkstra(
    cost_map: CostMap,
    start: Cell,
    goal: Cell,
    config: RoutingConfig = ROUTING,
    progress: ProgressCallback = None,
    progress_every: int = 20000,
) -> SearchResult:
    """Uniform-cost search - guarantees the optimum without a heuristic.

    The priority queue holds ``(distance, cell)``. As in :func:`astar`,
    ``heapq`` has no decrease-key, so an improved distance is pushed as a new
    entry and outdated entries are skipped when popped for an already visited
    cell (lazy deletion). Because all edge costs are positive, the first time
    a cell is popped its distance is final; the search ends when the goal is
    popped.
    一致代价搜索：按到起点的距离从小到大扩展节点，采用惰性删除的二叉堆。

    Parameters
    ----------
    cost_map : CostMap
        Lattice with traversal weights.
```

```
start, goal : Cell
    ``(row, col)`` of departure and destination; both must be navigable.
config : RoutingConfig
    Uses ``diagonal_moves`` to choose the 8- or 4-neighbourhood.
progress : callable, optional
    Called every ``progress_every`` expansions with
    ``("dijkstra", fraction, {"expanded", "frontier"})``.
progress_every : int, default 20000
    Expansion interval between progress reports.

Returns
-------
SearchResult
    ``found=True`` with the optimal path, or ``found=False`` with a
    message when start/goal is on land or no path exists.
"""
t0 = time.perf_counter()
spec = cost_map.spec
weights = cost_map.cost
offsets = OFFSETS_8 if config.diagonal_moves else OFFSETS_4
# Step length (km) of every move, per source row - shared with A*.
steps = _cell_distance_table(cost_map)
n_rows, n_cols = spec.n_rows, spec.n_cols

if not cost_map.passable[start] or not cost_map.passable[goal]:
    return SearchResult(found=False, algorithm="dijkstra",
                        runtime_s=time.perf_counter() - t0,
                        message="Start or goal cell is not navigable")

# distance: best known cost from the start; visited: cells with final cost.
# 距离表与已确定集合。
distance: Dict[Cell, float] = {start: 0.0}
came_from: Dict[Cell, Cell] = {}
visited: set = set()
heap: List[Tuple[float, Cell]] = [(0.0, start)]
expanded, generated = 0, 1
# Without a heuristic there is no distance-to-goal estimate, so progress is
# reported as the share of all lattice cells expanded so far (an upper bound
# of the work). 无启发信息，进度按已扩展单元占总单元数的比例估计。
budget = float(n_rows * n_cols)

while heap:
    d_current, current = heapq.heappop(heap)
    if current in visited:
        continue                        # stale heap entry (lazy deletion)
    visited.add(current)
    expanded += 1

    if current == goal:
        cells = _reconstruct(came_from, goal)
        coords = [spec.cell_to_coord(r, c) for r, c in cells]
        return SearchResult(
            found=True, path_cells=cells, path_coords=coords,
            total_cost=d_current, distance_km=_path_km(coords),
            nodes_expanded=expanded, nodes_generated=generated,
            runtime_s=time.perf_counter() - t0, algorithm="dijkstra",
            message="Optimal path found",
        )

    if progress is not None and expanded % progress_every == 0:
        progress("dijkstra", min(expanded / budget, 0.995),
                 {"expanded": expanded, "frontier": len(heap)})

    row, col = current
    w_current = float(weights[row, col])
    for dr, dc in offsets:
        r, c = row + dr, col + dc
        if not (0 <= r < n_rows and 0 <= c < n_cols):
            continue                    # outside the lattice
        neighbour = (r, c)
        if neighbour in visited:
            continue
        w_next = float(weights[r, c])
        if w_next >= IMPASSABLE_COST:
            continue                    # land cell
        # Edge relaxation with cost d(u, v) * (w(u) + w(v)) / 2.
        # 松弛操作：边代价 = 步长 × 两端权重平均值。
        step_km = float(steps[(dr, dc)][row])
        tentative = d_current + step_km * 0.5 * (w_current + w_next)
        if tentative < distance.get(neighbour, float("inf")):
```

```
                distance[neighbour] = tentative
                came_from[neighbour] = current
                heapq.heappush(heap, (tentative, neighbour))
                generated += 1

    # Every reachable cell was visited and the goal was not among them.
    return SearchResult(found=False, algorithm="dijkstra", nodes_expanded=expanded,
                        nodes_generated=generated, runtime_s=time.perf_counter() - t0,
                        message="No navigable path exists between the given points")
```

routing/dynamic_programming.py – value iteration on the Bellman equation

```
"""Dynamic-programming route search: value iteration on the Bellman equation.

This module is the fourth search method of the application. It follows the
suggestion of the project supervisor to solve the routing problem as a problem
of optimal control by dynamic programming (R. Bellman), and to compare the
evaluation function it is built on - the *Bellman function*, also called the
*value function* - with the *fitness function* of the genetic algorithm.

The discrete step and its cost
------------------------------
The state of the vessel is a lattice cell ``v``; a control is one of the eight
moves to a neighbouring cell ``u``. A move costs exactly what it costs in A*,
Dijkstra and the genetic algorithm::

    c(v, u) = d_gc(v, u) * (w(v) + w(u)) / 2

where ``d_gc`` is the great-circle length of the step (km) and ``w`` the zone
weight predicted by the neural network. Moves into land are not allowed.

The Bellman function
--------------------
The value ``V(v)`` of a cell is the minimum cost of reaching the goal from it.
It satisfies the Bellman equation::

    V(goal) = 0
    V(v)    = min over neighbours u of  [ c(v, u) + V(u) ]

The equation is solved by *value iteration*: ``V`` starts at 0 in the goal and
at +infinity everywhere else, and every sweep replaces each cell's value by the
right-hand side of the equation, computed for all cells at once with NumPy
array shifts. After ``k`` sweeps ``V(v)`` is the cheapest cost over routes of at
most ``k`` moves, so the iteration stops as soon as a sweep changes nothing;
the values are then exact.

The optimal route is read off the value function by the *greedy policy*: from
the departure, repeatedly move to the neighbour ``u`` that minimises
``c(v, u) + V(u)``, until the goal is reached. Its cost equals ``V(start)`` and
is the same optimum that A* and Dijkstra find - which the tests assert.

Unlike A*, the method computes the cost-to-go of *every* cell, not only of the
cells near the optimal route. That costs more work for one query, but once
``V`` is known the optimal route to the same destination from any other
departure is obtained immediately by the greedy policy.

动态规划寻路：在栅格上用价值迭代求解贝尔曼方程 V(v) = min_u [c(v,u) + V(u)]，
V(goal) = 0；再从起点按贪心策略沿价值函数下降即得最优航线。离散步代价与 A*、
Dijkstra、遗传算法完全相同，因此最优代价应与 A* 一致。
"""
from __future__ import annotations

import time
from typing import Dict, List, Optional, Tuple

import numpy as np

from ..config import IMPASSABLE_COST
from .astar import OFFSETS_8, Cell, ProgressCallback, SearchResult,
_cell_distance_table, _path_km
from .cost_map import CostMap


def _shift(values: np.ndarray, dr: int, dc: int, fill: float) -> np.ndarray:
    """Array whose element ``[r, c]`` is ``values[r + dr, c + dc]``.
```

```
    Positions whose neighbour lies outside the lattice receive ``fill``.
    Used to read, for every cell at once, the value of its neighbour in the
    direction ``(dr, dc)``.

    Parameters
    ----------
    values : numpy.ndarray
        2-D array of shape ``(n_rows, n_cols)``.
    dr, dc : int
        Row and column offset of the neighbour, each in ``{-1, 0, 1}``.
    fill : float
        Value used where the neighbour does not exist.

    Returns
    -------
    numpy.ndarray
        Array of the same shape as ``values``.

    数组平移：结果的 [r, c] 元素为 values[r+dr, c+dc]，越界处填 fill。
    """
    out = np.full_like(values, fill)
    n_rows, n_cols = values.shape
    r0, r1 = max(0, -dr), n_rows - max(0, dr)
    c0, c1 = max(0, -dc), n_cols - max(0, dc)
    out[r0:r1, c0:c1] = values[r0 + dr:r1 + dr, c0 + dc:c1 + dc]
    return out


def step_costs(cost_map: CostMap) -> Dict[Cell, np.ndarray]:
    """Cost of every discrete step, for all cells and all eight directions.

    For each offset ``(dr, dc)`` the returned array holds, in ``[r, c]``, the
    cost ``c(v, u)`` of the move from cell ``v = (r, c)`` to ``u = (r + dr,
    c + dc)``: the great-circle length of the step multiplied by the mean of
    the two zone weights. The cost is ``+inf`` where the move leaves the
    lattice or where either cell is impassable. This is the "evaluation
    function of the discrete step" on which the Bellman equation is built.

    Parameters
    ----------
    cost_map : CostMap
        Classified lattice with zone weights.

    Returns
    -------
    dict
        ``{(dr, dc): ndarray (n_rows, n_cols) of float64}``.

    离散步代价：对每个方向给出从每个格子出发走一步的代价，越界或陆地为 +inf。
    """
    weights = cost_map.cost.astype(np.float64)
    blocked = weights >= IMPASSABLE_COST
    weights = np.where(blocked, np.inf, weights)
    lengths = _cell_distance_table(cost_map)          # km per row and direction
    n_rows, n_cols = weights.shape
    costs: Dict[Cell, np.ndarray] = {}
    for dr, dc in OFFSETS_8:
        w_next = _shift(weights, dr, dc, np.inf)       # weight of the target cell
        step_km = lengths[(dr, dc)][:, None]           # (n_rows, 1), broadcast over
columns
        costs[(dr, dc)] = step_km * 0.5 * (weights + w_next)
    return costs


def value_iteration(
    cost_map: CostMap,
    goal: Cell,
    progress: ProgressCallback = None,
    max_sweeps: Optional[int] = None,
) -> Tuple[np.ndarray, int, Dict[Cell, np.ndarray]]:
    """Solve the Bellman equation for the cost-to-go of every cell.

    Parameters
    ----------
    cost_map : CostMap
        Classified lattice with zone weights.
    goal : tuple of int
        Destination cell (row, col); its value is fixed at zero.
    progress : callable, optional
        Receives ``("dynamic", fraction, {"sweep": k, "updated": m})`` after
```

```
        every tenth sweep; ``m`` is the number of cells whose value changed.
    max_sweeps : int, optional
        Safety limit; by default the number of cells, which is never reached
        because a shortest path cannot have more moves than there are cells.

    Returns
    -------
    (numpy.ndarray, int, dict)
        The value function ``V`` (``+inf`` for cells from which the goal cannot
        be reached), the number of sweeps performed, and the step costs used.

    价值迭代：V 初值在终点为 0、其余为 +inf；每次扫描用贝尔曼方程右端更新所有格子，
    直到一次扫描不再改变任何值为止，此时 V 即精确的剩余代价。
    """
    costs = step_costs(cost_map)
    n_rows, n_cols = cost_map.cost.shape
    value = np.full((n_rows, n_cols), np.inf)
    value[goal] = 0.0
    limit = max_sweeps or n_rows * n_cols
    sweeps = 0
    while sweeps < limit:
        sweeps += 1
        # Right-hand side of the Bellman equation for all cells at once:
        # min over the eight moves of (cost of the step + value of the target).
        # 贝尔曼方程右端：对八个方向取 (步代价 + 目标格价值) 的最小值。
        best = value.copy()
        for (dr, dc), step in costs.items():
            np.minimum(best, step + _shift(value, dr, dc, np.inf), out=best)
        best[goal] = 0.0
        updated = int(np.count_nonzero(best < value - 1e-9))
        value = best
        if progress is not None and sweeps % 10 == 0:
            # The number of sweeps is not known in advance; the fraction of the
            # lattice already reached is used as the progress estimate.
            reached = float(np.isfinite(value).mean())
            progress("dynamic", min(reached, 0.99), {"sweep": sweeps, "updated":
updated})
        if updated == 0:
            break
    return value, sweeps, costs


def greedy_policy(value: np.ndarray, costs: Dict[Cell, np.ndarray],
                  start: Cell, goal: Cell) -> List[Cell]:
    """Follow the value function from ``start`` down to ``goal``.

    At every cell the move that minimises ``c(v, u) + V(u)`` is taken. With an
    exact value function this move always leads to a cell of strictly lower
    value, so the walk ends in the goal after as many moves as the optimal
    route has.

    Parameters
    ----------
    value : numpy.ndarray
        Converged value function from :func:`value_iteration`.
    costs : dict
        Step costs from :func:`step_costs`.
    start, goal : tuple of int
        Departure and destination cells.

    Returns
    -------
    list of tuple
        The cells of the optimal route, from ``start`` to ``goal`` inclusive;
        empty if the goal cannot be reached from ``start``.

    贪心策略：每一步选择使 c(v,u) + V(u) 最小的相邻格子，直到到达终点。
    """
    if not np.isfinite(value[start]):
        return []
    n_rows, n_cols = value.shape
    path = [start]
    current = start
    for _ in range(n_rows * n_cols):                    # guard against a cycle
        if current == goal:
            return path
        r, c = current
        best_cell, best_total = None, np.inf
        for (dr, dc), step in costs.items():
            nr, nc = r + dr, c + dc
```

```
            if 0 <= nr < n_rows and 0 <= nc < n_cols:
                total = step[r, c] + value[nr, nc]
                if total < best_total:
                    best_cell, best_total = (nr, nc), total
        if best_cell is None or not np.isfinite(best_total):
            return []
        path.append(best_cell)
        current = best_cell
    return []


def dynamic_programming(
    cost_map: CostMap,
    start: Cell,
    goal: Cell,
    progress: ProgressCallback = None,
) -> SearchResult:
    """Optimal route by dynamic programming (value iteration + greedy policy).

    Parameters
    ----------
    cost_map : CostMap
        Classified lattice with traversal costs (shared with the other methods).
    start, goal : tuple of int
        Navigable departure and destination cells (row, col).
    progress : callable, optional
        Receives progress frames during value iteration, see
        :func:`value_iteration`.

    Returns
    -------
    SearchResult
        ``total_cost`` is ``V(start)``; ``nodes_expanded`` is the number of
        Bellman updates performed (sweeps × navigable cells), so that the work
        can be compared with the node counts of A* and Dijkstra;
        ``nodes_generated`` is the number of sweeps; ``value`` holds the value
        function for plotting.

    动态规划求最优航线：先价值迭代求 V，再按贪心策略提取路径。
    """
    t0 = time.perf_counter()
    spec = cost_map.spec
    passable = cost_map.passable
    if not passable[start] or not passable[goal]:
        return SearchResult(found=False, algorithm="dynamic",
                            runtime_s=time.perf_counter() - t0,
                            message="Start or goal cell is not navigable")

    value, sweeps, costs = value_iteration(cost_map, goal, progress=progress)
    cells = greedy_policy(value, costs, start, goal)
    updates = sweeps * int(passable.sum())
    if not cells:
        return SearchResult(found=False, algorithm="dynamic", nodes_expanded=updates,
                            nodes_generated=sweeps, runtime_s=time.perf_counter() - t0,
                            value=value,
                            message="No navigable path exists between the given points")
    coords = [spec.cell_to_coord(r, c) for r, c in cells]
    return SearchResult(
        found=True, path_cells=cells, path_coords=coords,
        total_cost=float(value[start]), distance_km=_path_km(coords),
        nodes_expanded=updates, nodes_generated=sweeps,
        runtime_s=time.perf_counter() - t0, algorithm="dynamic", value=value,
        message=f"Bellman equation solved in {sweeps} sweeps",
    )
```

routing/genetic.py – the genetic algorithm

```
"""Genetic-algorithm route search over the navigation cost map.

This module is the third search method of the application, added so that a
population-based metaheuristic can be compared with the two exact graph
searches (A* and Dijkstra). The idea follows the classical use of genetic
algorithms for path planning on an occupancy grid - "finding a way through a
maze": a candidate route is encoded as a short list of intermediate waypoints,
a population of such routes is evolved by selection, crossover and mutation,
and the fittest route of the last generation is returned.

Encoding
```

```
--------
A chromosome is a list of lattice cells ``[w1, w2, ..., wk]`` (0 <= k <=
``max_waypoints``). The route it represents is the poly-line

    start -> w1 -> w2 -> ... -> wk -> goal

in which every segment is rasterised into an 8-connected chain of cells, so the
decoded route is a path on exactly the same graph that A* and Dijkstra search.

Fitness
-------
The fitness is the cost of the decoded path under the *same* edge cost that A*
uses::

    c(u, v) = d_gc(u, v) * (w(u) + w(v)) / 2

except that an impassable (land) cell is given the finite weight
``land_penalty_weight`` instead of infinity. A route that touches land is
therefore not rejected outright - evolution needs a gradient - but it is so
expensive that any navigable route beats it. Lower fitness is better. Because
the cost model is identical, the GA result can be compared with the optimum
found by A*: the relative difference is the *optimality gap* of the GA.

Operators
---------
* **Initialisation** - random chromosomes of 1..6 waypoints drawn from the
  navigable cells inside an ellipse around the start and the goal, sorted by
  their projection on the start-goal axis (this prevents loops).
* **Selection** - tournament selection of size 3.
* **Crossover** - one-point crossover: the head of one parent is joined to the
  tail of the other at a random cut point.
* **Mutation** - one of five moves, chosen at random: *segment repair*
  (insert a waypoint that detours around the first stretch of land a segment
  crosses), shift a waypoint by a few cells, insert a waypoint in the middle of
  a segment, delete a waypoint, or *waypoint repair* (move a waypoint that sits
  on land to the nearest navigable cell).
* **Elitism** - the two best individuals are copied unchanged into the next
  generation, so the best fitness never gets worse.
* **Termination** - after ``generations`` generations, or earlier when the best
  fitness has not improved for ``stagnation_limit`` generations.

Unlike A*, the genetic algorithm gives no guarantee of optimality and its
result depends on the random seed; its value lies in its generality (the
fitness can be any function of the whole route, e.g. one including weather or
fuel), not in speed on a static grid. The experimental comparison is given in
section 3.3 of the thesis.

Main objects
------------
:class:`_Evaluator`
    Decodes chromosomes into lattice paths and computes their fitness with a
    per-segment cache.
:func:`_projection`
    Position of a cell along the start-goal axis, used to keep waypoints in
    travel order.
:func:`genetic`
    The search itself; the genetic operators are nested functions that share
    the random generator, the candidate pool and the evaluator.

Reproducibility: all randomness comes from one ``random.Random`` instance
seeded with ``seed`` or ``GeneticConfig.random_seed`` (42), so a run with the
same inputs and seed returns the same route.

遗传算法寻路：染色体为中间航点序列，航线由相邻航点之间的栅格直线段组成；
适应度与 A* 使用同一代价模型（陆地格子给一个很大的有限惩罚权重），
通过锦标赛选择、单点交叉、四种变异（平移/插入/删除/修复）和精英保留进化种群。
遗传算法不保证最优，结果与 A* 的最优代价之差即为其"最优性差距"。
（注：当前实现另含"线段修复"变异，共五种变异操作。）
"""
from __future__ import annotations

import math
import random
import time
from typing import Dict, List, Optional, Sequence, Tuple

import numpy as np

from ..config import GENETIC, IMPASSABLE_COST, GeneticConfig
```

```
from .astar import OFFSETS_8, Cell, ProgressCallback, SearchResult,
_cell_distance_table, _path_km
from .cost_map import CostMap, nearest_navigable

#: A chromosome: ordered list of intermediate waypoints (lattice cells), not
#: including the start and the goal. 染色体：中间航点列表（不含起终点）。
Chromosome = List[Cell]


class _Evaluator:
    """Decodes chromosomes into lattice paths and computes their cost.

    Segment costs are cached, because the same segment (pair of consecutive
    waypoints) occurs in many individuals and in many generations.

    解码染色体并计算代价；相同线段的代价会被缓存，避免重复计算。

    Attributes
    ----------
    cost_map : CostMap
        The shared cost map.
    impassable : numpy.ndarray of bool
        True for land cells.
    weights : numpy.ndarray of float64
        Cell weights with land replaced by ``land_weight``.
    step_km : numpy.ndarray
        Shape ``(3, 3, n_rows)``; ``step_km[dr + 1, dc + 1, row]`` is the
        length in km of the move ``(dr, dc)`` from ``row``.
    evaluations : int
        Number of :meth:`fitness` calls - the effort measure reported as
        ``nodes_expanded`` of the GA.
    """

    def __init__(self, cost_map: CostMap, land_weight: float) -> None:
        """Prepare the weight array, the step-length lookup and the cache.

        Parameters
        ----------
        cost_map : CostMap
            Cost map shared with A* and Dijkstra.
        land_weight : float
            Finite weight given to land cells in the fitness
            (``GeneticConfig.land_penalty_weight``, default 200, i.e. about
            100 times the coastal-sea weight).
        """
        self.cost_map = cost_map
        # Cell weights with land replaced by a large finite penalty.
        # 陆地格子用一个很大的有限权重代替无穷大，使适应度可比较。
        weights = cost_map.cost.astype(np.float64).copy()
        self.impassable = weights >= IMPASSABLE_COST
        weights[self.impassable] = land_weight
        self.weights = weights
        # step_km[dr + 1, dc + 1, row] = length of the move (dr, dc) from `row`
        # 步长表：从第 row 行沿 (dr, dc) 方向走一步的长度（千米）。
        # The dict from A* is copied into a dense array so that a whole
        # segment can be priced with one fancy-indexing operation.
        table = _cell_distance_table(cost_map)
        n_rows = cost_map.spec.n_rows
        self.step_km = np.zeros((3, 3, n_rows))
        for (dr, dc), lengths in table.items():
            self.step_km[dr + 1, dc + 1] = lengths
        # Cache: (a, b) -> (cost, land cells) of the directed segment a -> b.
        self._cache: Dict[Tuple[Cell, Cell], Tuple[float, int]] = {}
        self.evaluations = 0

    @staticmethod
    def rasterise(a: Cell, b: Cell) -> Tuple[np.ndarray, np.ndarray]:
        """Cells of the straight segment a -> b as an 8-connected chain.

        With ``n = max(|drow|, |dcol|)`` steps both coordinates advance by at
        most one cell per step, which is exactly one 8-connected move.
        线段栅格化：步数取行差与列差的较大者，每步行列各至多变化 1，即一次八邻域移动。

        The segment is sampled at ``n + 1`` equally spaced parameters
        ``t = 0, 1/n, ..., 1`` and each sample is rounded to the nearest cell
        (a DDA-style line). The dominant coordinate advances by exactly one
        cell per step, so there are no repeated cells and no gaps.

        Parameters
        ----------
```

```
            a, b : Cell
                End cells ``(row, col)``.

            Returns
            -------
            tuple of numpy.ndarray
                ``(rows, cols)``, int64 arrays of length ``n + 1`` including both
                ends; a single cell when ``a == b``.
            """
            n = max(abs(b[0] - a[0]), abs(b[1] - a[1]))
            if n == 0:
                return np.array([a[0]]), np.array([a[1]])
            t = np.linspace(0.0, 1.0, n + 1)
            rows = np.rint(a[0] + (b[0] - a[0]) * t).astype(np.int64)
            cols = np.rint(a[1] + (b[1] - a[1]) * t).astype(np.int64)
            return rows, cols

        def segment(self, a: Cell, b: Cell) -> Tuple[float, int]:
            """Cost of the segment a -> b and the number of land cells on it.

            The segment is rasterised and priced move by move with the A* edge
            cost ``step_km * (w_i + w_{i+1}) / 2`` (land at ``land_weight``). The
            result is cached per ordered pair ``(a, b)``.
            计算线段代价与其经过的陆地格子数（结果缓存）。

            Parameters
            ----------
            a, b : Cell
                Segment end cells.

            Returns
            -------
            tuple
                ``(cost, land)`` - cost in weighted km and the number of land
                cells on the segment, both end cells included.
            """
            key = (a, b)
            cached = self._cache.get(key)
            if cached is not None:
                return cached
            rows, cols = self.rasterise(a, b)
            if len(rows) < 2:
                # Zero-length segment: no cost, but the cell itself may be land.
                result = (0.0, int(self.impassable[rows[0], cols[0]]))
            else:
                w = self.weights[rows, cols]
                # Move directions along the chain, each component in {-1, 0, 1}.
                dr = np.diff(rows)
                dc = np.diff(cols)
                # Step length of every move, looked up by direction and source row.
                steps = self.step_km[dr + 1, dc + 1, rows[:-1]]
                # Sum of d * (w_i + w_{i+1}) / 2 over all moves (trapezoidal rule).
                cost = float(np.sum(steps * 0.5 * (w[:-1] + w[1:])))
                land = int(self.impassable[rows, cols].sum())
                result = (cost, land)
            self._cache[key] = result
            return result

        def fitness(self, start: Cell, goal: Cell, chromosome: Chromosome) -> Tuple[float,
    int]:
            """Total cost and total number of land cells of the decoded route.

            Sums :meth:`segment` over ``start -> w1 -> ... -> wk -> goal`` and
            increments :attr:`evaluations`. A waypoint on land is counted in both
            adjacent segments; the land count is only compared with zero, so the
            double count is harmless.
            适应度：整条航线的总代价与陆地格子数。

            Parameters
            ----------
            start, goal : Cell
                Fixed end cells.
            chromosome : Chromosome
                Intermediate waypoints.

            Returns
            -------
            tuple
                ``(cost, land)``; lower cost is better, ``land == 0`` means the
                route is navigable.
```

```
            """
            self.evaluations += 1
            points = [start, *chromosome, goal]
            total, land = 0.0, 0
            for a, b in zip(points[:-1], points[1:]):
                c, l = self.segment(a, b)
                total += c
                land += l
            return total, land

        def decode(self, start: Cell, goal: Cell, chromosome: Chromosome) -> List[Cell]:
            """Full lattice path of a chromosome, without repeated cells.

            Concatenates the rasterised segments. The first cell of each segment
            equals the last cell of the previous one and is skipped; a cell equal
            to its predecessor (zero-length segment) is dropped as well.
            将染色体解码为完整的网格路径（去除相邻重复格子）。

            Parameters
            ----------
            start, goal : Cell
                Fixed end cells.
            chromosome : Chromosome
                Intermediate waypoints.

            Returns
            -------
            list of Cell
                8-connected path from ``start`` to ``goal``.
            """
            points = [start, *chromosome, goal]
            path: List[Cell] = [start]
            for a, b in zip(points[:-1], points[1:]):
                rows, cols = self.rasterise(a, b)
                for r, c in zip(rows[1:].tolist(), cols[1:].tolist()):
                    if (r, c) != path[-1]:
                        path.append((r, c))
            return path


    def _projection(start: Cell, goal: Cell):
        """Return a function giving the position of a cell along start -> goal (0..1).

        The position is the scalar projection of ``cell - start`` on the vector
        ``goal - start`` divided by its squared length, so the start maps to 0,
        the goal to 1, and cells beside the axis to their foot point. Values
        outside [0, 1] occur for cells behind the start or beyond the goal.
        Sorting waypoints by this value keeps a route moving forward and prevents
        loops. Computed in cell units (row/col), which is sufficient for ordering.
        返回一个函数：计算格子在起点->终点方向上的投影位置（起点 0，终点 1）。

        Parameters
        ----------
        start, goal : Cell
            End cells of the voyage.

        Returns
        -------
        callable
            ``along(cell) -> float``.
        """
        ax, ay = goal[0] - start[0], goal[1] - start[1]
        # "or 1.0" guards against start == goal (zero-length axis).
        norm = float(ax * ax + ay * ay) or 1.0

        def along(cell: Cell) -> float:
            """Normalised projection of ``cell`` on the start-goal axis.

            Parameters
            ----------
            cell : Cell
                ``(row, col)``.

            Returns
            -------
            float
                0 at the start, 1 at the goal.
            """
            return ((cell[0] - start[0]) * ax + (cell[1] - start[1]) * ay) / norm
```

```
    return along


def genetic(
    cost_map: CostMap,
    start: Cell,
    goal: Cell,
    config: GeneticConfig = GENETIC,
    progress: ProgressCallback = None,
    seed: Optional[int] = None,
) -> SearchResult:
    """Search a low-cost navigable route with a genetic algorithm.

    Parameters
    ----------
    cost_map:
        Classified lattice with traversal costs (shared with A* and Dijkstra).
    start, goal:
        Navigable lattice cells (row, col).
    config:
        Population size, rates, limits -
see :class:`~maritime_route.config.GeneticConfig`.
    progress:
        Optional callback ``progress(stage, fraction, details)`` used by the web
        client to show the generation counter in real time.
    seed:
        Random seed; ``None`` takes ``config.random_seed``.

    Returns
    -------
    SearchResult
        ``found`` is ``True`` only when the best route does not touch land.
        ``nodes_expanded`` holds the number of fitness evaluations, so that the
        effort of the GA can be put next to the node counts of A* and Dijkstra.

    用遗传算法搜索低代价的可航行路线。返回结果中 nodes_expanded 记录适应度评估次数。

    Notes
    -----
    ``nodes_generated`` holds the number of generations actually run and
    ``history`` the best fitness after each generation (index 0 = initial
    population), which ``scripts/make_figures.py`` plots as the convergence
    curve. ``total_cost`` is ``inf`` when the best route still touches land.
    """
    t0 = time.perf_counter()
    rng = random.Random(config.random_seed if seed is None else seed)
    spec = cost_map.spec
    n_rows, n_cols = spec.n_rows, spec.n_cols
    passable = cost_map.passable

    if not passable[start] or not passable[goal]:
        return SearchResult(found=False, algorithm="genetic",
                            runtime_s=time.perf_counter() - t0,
                            message="Start or goal cell is not navigable")

    evaluator = _Evaluator(cost_map, config.land_penalty_weight)
    along = _projection(start, goal)

    # ---- candidate cells for random waypoints ---------------------------- #
    # Navigable cells inside an ellipse whose foci are the start and the goal;
    # the ellipse is widened until it holds a reasonable number of cells.
    # 随机航点候选：以起点和终点为焦点的椭圆内的可航行格子。
    # A cell P lies inside the ellipse when |P - start| + |P - goal| <=
    # factor * |goal - start| (+ 2 cells so that very short voyages still get
    # candidates). factor 1.3 allows detours of up to 30 % of the direct
    # distance; the last factor (1e9) accepts every navigable cell.
    rr, cc = np.nonzero(passable)
    d_direct = math.hypot(goal[0] - start[0], goal[1] - start[1])
    d_sum = np.hypot(rr - start[0], cc - start[1]) + np.hypot(rr - goal[0], cc -
goal[1])
    candidates = np.empty(0, dtype=np.int64)
    for factor in (1.3, 1.6, 2.0, 3.0, 1e9):
        candidates = np.nonzero(d_sum <= factor * d_direct + 2.0)[0]
        if len(candidates) >= 50:
            break
    pool: List[Cell] = [(int(rr[i]), int(cc[i])) for i in candidates]

    def random_cell() -> Cell:
        """Uniformly random navigable cell from the ellipse pool.
```

```
    Returns
    -------
    Cell
        A candidate waypoint.
    """
    return pool[rng.randrange(len(pool))]

def ordered(cells: Sequence[Cell]) -> Chromosome:
    """Sort waypoints by their position along the start-goal axis.

    Keeps every chromosome in travel order so that operators cannot
    create routes that double back on themselves.

    Parameters
    ----------
    cells : sequence of Cell
        Waypoints in any order.

    Returns
    -------
    Chromosome
        The same cells sorted by :func:`_projection`.
    """
    return sorted(cells, key=along)

def random_chromosome() -> Chromosome:
    """New random individual with ``initial_waypoints`` (1..6) waypoints.

    Returns
    -------
    Chromosome
        Ordered list of random pool cells.
    """
    low, high = config.initial_waypoints
    return ordered(random_cell() for _ in range(rng.randint(low, high)))

def score(ch: Chromosome) -> float:
    """Fitness value used for selection (lower is better).

    Parameters
    ----------
    ch : Chromosome
        Individual to evaluate.

    Returns
    -------
    float
        Route cost in weighted km, land priced at ``land_penalty_weight``.
    """
    # Land is already priced in through land_penalty_weight. 陆地已通过惩罚权重计入。
    return evaluator.fitness(start, goal, ch)[0]

# ---- genetic operators ------------------------------------------ #
def tournament(population: List[Chromosome], fitness: List[float]) -> Chromosome:
    """Tournament selection: best of ``tournament_size`` random individuals.

    Selection pressure grows with the tournament size; size 3 keeps
    weaker individuals in play and so preserves diversity.
    锦标赛选择：随机抽取若干个体，取其中适应度最好者。

    Parameters
    ----------
    population : list of Chromosome
        Current generation.
    fitness : list of float
        Fitness of each individual, same order.

    Returns
    -------
    Chromosome
        The winner (the list object itself, not a copy).
    """
    best = min(rng.sample(range(len(population)), config.tournament_size),
               key=lambda i: fitness[i])
    return population[best]

def crossover(a: Chromosome, b: Chromosome) -> Chromosome:
    """One-point crossover along the start-goal axis.

    A random cut position in [0, 1) along the axis is drawn; the child
```

```
        takes the waypoints of ``a`` before the cut and those of ``b`` after
        it, so the child stays ordered. It is truncated to ``max_waypoints``.

        Parameters
        ----------
        a, b : Chromosome
            Parents.

        Returns
        -------
        Chromosome
            New child list.
        """
        # One-point crossover on the start-goal axis: head of `a`, tail of `b`.
        # 单点交叉：沿起终点方向取 a 的前半段与 b 的后半段。
        cut = rng.random()
        head = [w for w in a if along(w) <= cut]
        tail = [w for w in b if along(w) > cut]
        return (head + tail)[: config.max_waypoints]

    def clamp(cell: Cell) -> Cell:
        """Clip a cell index into the lattice bounds.

        Parameters
        ----------
        cell : Cell
            Possibly out-of-range ``(row, col)``.

        Returns
        -------
        Cell
            ``(row, col)`` with ``0 <= row < n_rows``, ``0 <= col < n_cols``.
        """
        return (min(max(cell[0], 0), n_rows - 1), min(max(cell[1], 0), n_cols - 1))

    def repair_segment(ch: Chromosome) -> Chromosome:
        """Insert a waypoint that detours around the first land run of a segment.

        The land cells of every segment are known from rasterisation. For a
        segment that crosses land, the middle cell of its first land run is
        snapped to the nearest navigable cell and inserted as a new waypoint,
        which splits the blocked segment in two. Repeated over generations this
        lets the route wrap around headlands and thread narrow straits.
        线段修复：对穿越陆地的线段，取第一段陆地的中点，吸附到最近的可航行格子并作为新航点插入。

        Parameters
        ----------
        ch : Chromosome
            Individual to repair (not modified).

        Returns
        -------
        Chromosome
            Repaired, re-ordered copy; ``ch`` itself when no segment is
            blocked, the chromosome is already at ``max_waypoints``, or no
            navigable cell is found near the land run.
        """
        points = [start, *ch, goal]
        # Indices j of blocked segments points[j] -> points[j + 1].
        blocked = [j for j in range(len(points) - 1)
                   if evaluator.segment(points[j], points[j + 1])[1] > 0]
        if not blocked or len(ch) >= config.max_waypoints:
            return ch
        j = rng.choice(blocked)
        rows, cols = evaluator.rasterise(points[j], points[j + 1])
        land = np.nonzero(evaluator.impassable[rows, cols])[0]
        # First contiguous run of land cells on the segment. 线段上第一段连续陆地。
        run_end = land[0]
        while run_end + 1 < len(rows) and evaluator.impassable[rows[run_end + 1],
cols[run_end + 1]]:
            run_end += 1
        mid = (land[0] + run_end) // 2
        snapped = nearest_navigable(cost_map, int(rows[mid]), int(cols[mid]))
        if snapped is None:
            return ch
        out = list(ch)
        # Segment j runs from points[j] (= ch[j - 1]) to points[j + 1] (= ch[j]),
        # so inserting at index j places the new waypoint between them;
        # ordered() then restores the axis order.
        out.insert(j, snapped)
```

```
        return ordered(out)

    def mutate(ch: Chromosome) -> Chromosome:
        """Apply one randomly chosen mutation. 随机施加一种变异。

        One uniform draw ``move`` in [0, 1) selects the operator. Nominal
        probabilities: segment repair 25 %, shift 25 %, insert 18 %,
        delete 16 %, waypoint repair 16 %. A branch whose precondition fails
        (e.g. shift on an empty chromosome) falls through to the next
        ``elif``, so the actual shares vary slightly with the chromosome.

        Parameters
        ----------
        ch : Chromosome
            Individual to mutate (not modified).

        Returns
        -------
        Chromosome
            Mutated, re-ordered copy.
        """
        ch = list(ch)
        move = rng.random()
        if move < 0.25:
            return repair_segment(ch)
        if ch and move < 0.50:
            # Shift a waypoint by a few cells. 平移一个航点。
            # Radius: 5 % of the direct start-goal distance, at least 2 cells.
            i = rng.randrange(len(ch))
            radius = max(2, int(0.05 * d_direct))
            ch[i] = clamp((ch[i][0] + rng.randint(-radius, radius),
                           ch[i][1] + rng.randint(-radius, radius)))
        elif move < 0.68 and len(ch) < config.max_waypoints:
            # Insert a waypoint near the middle of a random segment. 插入航点。
            # The new point is scattered around the midpoint by up to half the
            # segment length, so it can pull the route sideways off the line.
            points = [start, *ch, goal]
            j = rng.randrange(len(points) - 1)
            a, b = points[j], points[j + 1]
            span = max(2, int(0.5 * math.hypot(b[0] - a[0], b[1] - a[1])))
            mid = clamp(((a[0] + b[0]) // 2 + rng.randint(-span, span),
                         (a[1] + b[1]) // 2 + rng.randint(-span, span)))
            ch.insert(j, mid)
        elif ch and move < 0.84:
            # Delete a waypoint - shortens routes once the detours are no longer needed.
            # 删除航点：绕行不再需要时缩短航线。
            del ch[rng.randrange(len(ch))]
        elif ch:
            # Repair: snap a waypoint on land to the nearest navigable cell.
            # 修复：把落在陆地上的航点吸附到最近的可航行格子。
            on_land = [i for i, w in enumerate(ch) if not passable[w]]
            if on_land:
                i = rng.choice(on_land)
                snapped = nearest_navigable(cost_map, *ch[i])
                if snapped is not None:
                    ch[i] = snapped
        return ordered(ch)

    # ---- evolution ------------------------------------------------------ #
    population: List[Chromosome] = [[]]                # the direct line is always
tried
    population += [random_chromosome() for _ in range(config.population_size - 1)]
    fitness = [score(ch) for ch in population]
    best_i = min(range(len(population)), key=lambda i: fitness[i])
    # Best individual seen so far (a copy) and its fitness. 迄今最优个体。
    best, best_fit = list(population[best_i]), fitness[best_i]
    history: List[float] = [best_fit]
    stagnant = 0            # generations since the last improvement 未改进的代数
    generation = 0

    for generation in range(1, config.generations + 1):
        # Elitism: the `elite` best individuals pass unchanged (as copies).
        # 精英保留：最优的若干个体原样进入下一代。
        order = sorted(range(len(population)), key=lambda i: fitness[i])
        next_population = [list(population[i]) for i in order[: config.elite]]
        # Fill the rest by selection -> (crossover) -> (mutation).
        while len(next_population) < config.population_size:
            parent_a = tournament(population, fitness)
            if rng.random() < config.crossover_rate:
                child = crossover(parent_a, tournament(population, fitness))
```

```
            else:
                child = list(parent_a)
            if rng.random() < config.mutation_rate:
                child = mutate(child)
            next_population.append(child)
        population = next_population
        fitness = [score(ch) for ch in population]

        gen_best = min(range(len(population)), key=lambda i: fitness[i])
        # 1e-9 tolerance: a change caused only by rounding is not an improvement.
        if fitness[gen_best] < best_fit - 1e-9:
            best, best_fit = list(population[gen_best]), fitness[gen_best]
            stagnant = 0
        else:
            stagnant += 1
        history.append(best_fit)

        if progress is not None and generation % 10 == 0:
            progress("genetic", generation / config.generations,
                     {"generation": generation, "best_cost": round(best_fit, 1)})
        # Early stop after `stagnation_limit` generations without improvement.
        # 连续若干代无改进则提前终止。
        if stagnant >= config.stagnation_limit:
            break

    # ---- result ------------------------------------------------------- #
    cells = evaluator.decode(start, goal, best)
    # Re-evaluate only for the land count (this also adds one evaluation).
    _, land = evaluator.fitness(start, goal, best)
    coords = [spec.cell_to_coord(r, c) for r, c in cells]
    found = land == 0
    return SearchResult(
        found=found,
        path_cells=cells,
        path_coords=coords,
        total_cost=best_fit if found else float("inf"),
        distance_km=_path_km(coords),
        nodes_expanded=evaluator.evaluations,
        nodes_generated=generation,
        runtime_s=time.perf_counter() - t0,
        algorithm="genetic",
        history=history,
        message=(f"Best route after {generation} generations "
                 f"({len(best)} waypoints)" if found else
                 f"No navigable route evolved in {generation} generations"),
    )
```

routing/coast_check.py – independent coastline check of the route

```
"""Independent coastline check of a planned route.

航线的独立海岸线检查：规划完成后，用与神经网络无关的 1 km 陆地掩膜逐公里检查航线。

Why
---
The planner only knows what the neural classifier tells it. The test on
100 random voyages (``scripts/route_audit.py``, section 3.5.6 of the note)
showed that the classifier sometimes takes sparsely sampled land - small
islands, narrow peninsulas - for water, and that on a coarse lattice a
diagonal step can cut a corner of land between two water cells. The route
is then optimal *on the classified lattice* but crosses land in reality.

This module is the safeguard against such courses. After every search the
route is sampled every :data:`STEP_KM` kilometres and each sample is looked
up in an independent land mask: the 1 km GLOBE-based raster of the package
``global-land-mask`` (Karin, 2020; Hastings and Dunbar, 1999). The mask
plays no part in training or classification, so it is an outside witness.

The planner (:meth:`~.planner.RoutePlanner.plan`) uses the verdict to
correct the route automatically:

1. the lattice cells under the land samples, and the route vertices of the
   offending legs, are made impassable and the same method searches again;
2. if that does not help, the lattice is made finer and the route planned
   again;
3. if the route still crosses land, it is returned with a warning that the
   UI, the PDF report and the database show to the navigator.
```

```
Verdicts
--------
``clear``     no sample on land;
``grazing``   land runs shorter than :data:`CROSSING_KM` - the route touches
              a coast, a channel narrower than the mask resolves, or an islet;
              accepted;
``crossing``  at least one continuous land run of :data:`CROSSING_KM` or more
              - the course is incorrect;
``unchecked`` the land mask is not installed (``pip install global-land-mask``);
              the planner then works exactly as before.

The first and the last :data:`PORT_KM` kilometres are not checked, because
a voyage begins and ends at a quay, which lies on the coast by definition.

Port approaches. The planner joins the exact quay to the centre of the
nearest navigable lattice cell by a straight connector (up to 60 km, see
``RoutePlanner.SNAP_RADIUS_KM``). When the quay lies inside a harbour basin,
a canal or a city centre, that connector necessarily runs over land on a
1 km mask. A land stretch that begins at the quay (the mask shows land
right up to the unchecked :data:`PORT_KM`) and ends within the connector is
therefore reported separately as a *port approach* (``approach_land_km``):
it is shown to the navigator, but it is not counted as an incorrect course,
because no lattice search can change it - the way into the harbour is
found by pilotage along the harbour fairway.

Units: kilometres and degrees. 单位：千米与度。
"""
from __future__ import annotations

import logging
import math
from dataclasses import dataclass, field
from typing import Dict, List, Optional, Sequence, Set, Tuple

import numpy as np

from .geodesy import haversine_km

LOGGER = logging.getLogger(__name__)

try:                                    # optional at import time, see is_available()
    from global_land_mask import globe as _GLOBE
except ImportError:                     # pragma: no cover - depends on the installation
    _GLOBE = None

#: Sampling step along the route, km (the mask resolution is about 1 km).
#: 沿航线的采样步长，千米。
STEP_KM: float = 1.0
#: Length at each end of the voyage that is not checked, km. 首尾不检查的长度。
PORT_KM: float = 5.0
#: A continuous land run of this length or more makes the course incorrect, km.
#: 连续穿陆长度达到该值即判为航线错误。
CROSSING_KM: float = 3.0
#: Largest number of land points returned for drawing on the map.
MAX_REPORTED_POINTS: int = 60

Coordinate = Tuple[float, float]


def is_available() -> bool:
    """True when the independent land mask is installed. 陆地掩膜是否可用。"""
    return _GLOBE is not None


def is_land(lat, lon) -> np.ndarray:
    """Look points up in the 1 km land mask.

    Parameters
    ----------
    lat, lon : array_like
        Coordinates, degrees; longitudes outside -180..180 are wrapped.

    Returns
    -------
    numpy.ndarray of bool
        True where the mask shows land.
    """
    lat = np.clip(np.asarray(lat, dtype=float), -90.0, 90.0)
    lon = (np.asarray(lon, dtype=float) + 180.0) % 360.0 - 180.0
    return np.asarray(_GLOBE.is_land(lat, lon), dtype=bool)
```

```
@dataclass
class LandRun:
    """One continuous stretch of the route over land. 一段连续穿陆的航段。

    Attributes
    ----------
    length_km : float
        Length of the stretch, km.
    start_km : float
        Distance from the departure to the first land sample, km.
    legs : list of int
        Indices ``i`` of the route legs ``waypoints[i] -> waypoints[i+1]`` that
        contain samples of this stretch.
    points : list of Coordinate
        The land samples, ``(lat, lon)``.
    """

    length_km: float
    start_km: float
    legs: List[int] = field(default_factory=list)
    points: List[Coordinate] = field(default_factory=list)


#: Longest connector between a quay and the lattice that may count as a
#: port approach, km (snap radius 60 km plus the unchecked 5 km).
APPROACH_MAX_KM: float = 65.0


@dataclass
class CoastCheck:
    """Outcome of the coastline check of one route. 一条航线的检查结果。

    Attributes
    ----------
    verdict : str
        ``clear``, ``grazing``, ``crossing`` or ``unchecked``.
    max_land_run_km : float
        Longest continuous land stretch, km (0 when clear).
    land_km : float
        Total length over land, km.
    samples, samples_on_land : int
        Checked samples and those on land.
    runs : list of LandRun
        All land stretches, longest first (port approaches excluded).
    approach_runs : list of LandRun
        Land stretches of the port approaches (see the module notes).
    """

    verdict: str
    max_land_run_km: float = 0.0
    land_km: float = 0.0
    samples: int = 0
    samples_on_land: int = 0
    runs: List[LandRun] = field(default_factory=list)
    approach_runs: List[LandRun] = field(default_factory=list)

    @property
    def crossing_runs(self) -> List[LandRun]:
        """Stretches long enough to make the course incorrect."""
        return [r for r in self.runs if r.length_km >= CROSSING_KM]

    def to_dict(self, corrections: int = 0, refined: bool = False,
                cells_blocked: int = 0, initial: Optional[str] = None) -> Dict:
        """JSON-ready summary stored with the route and sent to the client.

        Parameters
        ----------
        corrections : int
            Number of automatic re-searches that were made.
        refined : bool
            Whether a finer lattice was used because of the check.
        cells_blocked : int
            Lattice cells made impassable by the corrections.
        initial : str, optional
            Verdict of the first route, before any correction.

        Returns
        -------
```

```
            dict
                ``verdict``, ``initial_verdict``, ``max_land_run_km``, ``land_km``,
                ``samples``, ``samples_on_land``, ``corrections``, ``refined``,
                ``cells_blocked``, ``threshold_km`` and up to
                :data:`MAX_REPORTED_POINTS` ``land_points`` ``[lat, lon]``.
            """
            points = [p for r in self.crossing_runs for p in r.points] or \
                     [p for r in self.runs for p in r.points]
            step = max(1, math.ceil(len(points) / MAX_REPORTED_POINTS))
            return {
                "verdict": self.verdict,
                "initial_verdict": initial or self.verdict,
                "max_land_run_km": round(self.max_land_run_km, 1),
                "land_km": round(self.land_km, 1),
                "samples": self.samples,
                "samples_on_land": self.samples_on_land,
                "corrections": int(corrections),
                "refined": bool(refined),
                "cells_blocked": int(cells_blocked),
                "threshold_km": CROSSING_KM,
                "approach_land_km": round(sum(r.length_km for r in self.approach_runs), 1),
                "land_points": [[round(a, 4), round(b, 4)] for a, b in points[::step]],
            }


def sample_route(waypoints: Sequence[Coordinate], step_km: float = STEP_KM
                 ) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
    """Points every ``step_km`` along the poly-line.

    Each leg is divided into ``ceil(length / step_km)`` equal parts by linear
    interpolation in degrees, which for legs of a lattice (at most a few tens
    of kilometres) differs from the great circle by metres.

    Returns
    -------
    tuple
        ``(points, leg, dist)`` - an ``(n, 2)`` array of ``(lat, lon)``, the
        index of the leg of every point and the distance of every point from
        the departure along the route, km.
    """
    pts, legs = [], []
    for i, ((a1, b1), (a2, b2)) in enumerate(zip(waypoints[:-1], waypoints[1:])):
        n = max(1, int(math.ceil(float(haversine_km(a1, b1, a2, b2)) / step_km)))
        t = np.arange(n) / n
        pts.append(np.column_stack([a1 + (a2 - a1) * t, b1 + (b2 - b1) * t]))
        legs.append(np.full(n, i))
    pts.append(np.array([waypoints[-1]], dtype=float))
    legs.append(np.array([max(len(waypoints) - 2, 0)]))
    pts = np.vstack(pts).astype(float)
    leg = np.concatenate(legs)
    seg = haversine_km(pts[:-1, 0], pts[:-1, 1], pts[1:, 0], pts[1:, 1])
    dist = np.r_[0.0, np.cumsum(seg)]
    return pts, leg, dist


def check_route(waypoints: Sequence[Coordinate], approach: bool = False) -> CoastCheck:
    """Check a route against the independent land mask. 检查航线是否穿越陆地。

    Parameters
    ----------
    waypoints : sequence of Coordinate
        Route poly-line from the departure to the destination, ``(lat, lon)``.
    approach : bool, default False
        Treat the first and the last leg as the connectors between the quays
        and the lattice: land that begins at a quay and ends within its
        connector is a port approach, not an incorrect course (see the module
        notes). The planner sets it; the 100-voyage test, whose ends are
        clear water, does not need it.

    Returns
    -------
    CoastCheck
        Verdict and the land stretches; ``unchecked`` when the mask is not
        installed or the route has fewer than two points.
    """
    if not is_available() or len(waypoints) < 2:
        return CoastCheck(verdict="unchecked")
    pts, leg, dist = sample_route(waypoints)
    keep = (dist >= PORT_KM) & (dist <= dist[-1] - PORT_KM)
    on_land = is_land(pts[:, 0], pts[:, 1])
```

```
    land = on_land & keep

    runs: List[LandRun] = []
    current: Optional[LandRun] = None
    for i in range(len(pts)):
        if land[i]:
            if current is None:
                # A run starts: one sample stands for STEP_KM of route.
                current = LandRun(length_km=STEP_KM, start_km=float(dist[i]))
            else:
                current.length_km += float(dist[i] - dist[i - 1])
            current.points.append((float(pts[i, 0]), float(pts[i, 1])))
            if not current.legs or current.legs[-1] != int(leg[i]):
                current.legs.append(int(leg[i]))
        elif current is not None:
            runs.append(current)
            current = None
    if current is not None:
        runs.append(current)

    approach_runs: List[LandRun] = []
    if approach and runs and keep.any():
        total = float(dist[-1])
        first_leg = min(float(haversine_km(*waypoints[0], *waypoints[1])),
APPROACH_MAX_KM)
        last_leg = min(float(haversine_km(*waypoints[-2], *waypoints[-1])),
APPROACH_MAX_KM)
        first_kept, last_kept = np.flatnonzero(keep)[[0, -1]]
        # Land at the boundary of the unchecked stretch = the quay is on land.
        quay_start_on_land = bool(first_kept > 0 and on_land[first_kept - 1])
        quay_end_on_land = bool(last_kept + 1 < len(pts) and on_land[last_kept + 1])
        rest = []
        for run in runs:
            end_km = run.start_km + run.length_km
            at_start = (quay_start_on_land and run.start_km <= dist[first_kept] + 1e-6
                        and end_km <= max(first_leg, PORT_KM) + STEP_KM)
            at_end = (quay_end_on_land and end_km >= dist[last_kept] - STEP_KM - 1e-6
                      and run.start_km >= total - max(last_leg, PORT_KM) - STEP_KM)
            (approach_runs if at_start or at_end else rest).append(run)
        runs = rest

    runs.sort(key=lambda r: r.length_km, reverse=True)
    longest = runs[0].length_km if runs else 0.0
    verdict = "clear" if not runs else ("crossing" if longest >= CROSSING_KM else
"grazing")
    return CoastCheck(
        verdict=verdict,
        max_land_run_km=longest,
        land_km=float(sum(r.length_km for r in runs)),
        samples=int(keep.sum()),
        samples_on_land=int(land.sum()),
        runs=runs,
        approach_runs=approach_runs,
    )


#: Half-width of the window around a land stretch in which navigable cells
#: centred on land of the mask are blocked as well, in cells.
BLOCK_WINDOW_CELLS: int = 2


def cells_to_block(check: CoastCheck, cost_map, path_cells: Sequence[Tuple[int, int]],
                   protected: Set[Tuple[int, int]],
                   window: int = BLOCK_WINDOW_CELLS) -> Set[Tuple[int, int]]:
    """Lattice cells to make impassable so that the next search avoids the land.

    Three kinds of cells are taken for every stretch
of :attr:`CoastCheck.crossing_runs`:

    * the cell nearest to each land sample - this removes land that the
      classifier took for water;
    * the route vertices at both ends of the legs that contain the stretch -
      when the leg is a diagonal step between two water cells across a corner
      of land, blocking its ends forces the search to go round;
    * every navigable cell whose centre lies on land of the mask within
      ``window`` cells of the stretch - where the classifier is wrong about
      one cell it is usually wrong about its neighbours too (a whole island
      chain), and without this the route would move one cell per round.
      Cells whose centre is water are not touched, so a strait stays open.
```

```
    The snapped departure and destination cells (``protected``) are never
    blocked.
    为每段穿陆航段选出需要封锁的网格：陆地采样点所在格子、该航段两端的航路点格子，
    以及附近中心点位于陆地上的可航行格子。

    Parameters
    ----------
    check : CoastCheck
        Result of :func:`check_route` for the route.
    cost_map : CostMap
        Map the route was searched on.
    path_cells : sequence of (row, col)
        Lattice path of the search; ``path_cells[i]`` corresponds to the
        route vertex ``i`` (the planner replaces only the coordinates of the
        first and the last vertex by the exact ports).
    protected : set of (row, col)
        Cells that must stay navigable.

    Returns
    -------
    set of (row, col)
        Cells that are navigable now and should be blocked.
    """
    spec = cost_map.spec
    passable = cost_map.passable
    cells: Set[Tuple[int, int]] = set()
    for run in check.crossing_runs:
        run_cells = [spec.coord_to_cell(lat, lon) for lat, lon in run.points]
        cells.update(run_cells)
        if window > 0 and run_cells:
            r0 = max(min(r for r, _ in run_cells) - window, 0)
            r1 = min(max(r for r, _ in run_cells) + window, spec.n_rows - 1)
            c0 = max(min(c for _, c in run_cells) - window, 0)
            c1 = min(max(c for _, c in run_cells) + window, spec.n_cols - 1)
            rows, cols = np.mgrid[r0:r1 + 1, c0:c1 + 1]
            lat, lon = spec.cell_to_coord(rows, cols)
            on_land = is_land(lat, lon) & passable[rows, cols]
            cells.update(zip(rows[on_land].tolist(), cols[on_land].tolist()))
        if path_cells:
            for i in run.legs:
                for j in (i, i + 1):
                    if 0 <= j < len(path_cells):
                        cells.add(tuple(path_cells[j]))
    return {c for c in cells if c not in protected and passable[c]}


def block_cells(cost_map, cells: Set[Tuple[int, int]]):
    """Copy of the cost map with ``cells`` made impassable (the original is kept).

    Only the cost array changes; the classification (``class_index``) is
    left as the classifier predicted it, so the map overlay in the UI still
    shows the classifier's opinion.

    Parameters
    ----------
    cost_map : CostMap
        Map to copy.
    cells : set of (row, col)
        Cells to block.

    Returns
    -------
    CostMap
        New map sharing ``spec``, ``class_index`` and ``confidence``.
    """
    from dataclasses import replace

    from ..config import IMPASSABLE_COST

    cost = cost_map.cost.copy()
    for row, col in cells:
        cost[row, col] = IMPASSABLE_COST
    return replace(cost_map, cost=cost)
```

routing/planner.py – planning orchestration and comparison

```
"""High level route planning service.
```

```
航线规划服务：串联"分类 -> 代价地图 -> 路径搜索 -> 与大圆航线对比"全过程。

Role in the pipeline
--------------------
:class:`RoutePlanner` is the single entry point that the web application
(:mod:`maritime_route.web.app`) and the scripts call to obtain a route. For one
request it performs:

1. **Lattice** - :func:`~.cost_map.make_grid_spec` around the two ports.
2. **Classification** - :func:`~.cost_map.build_cost_map` asks the neural
   classifier for the zone of every lattice cell and derives the costs
   (skipped when a cost map is passed in for reuse).
3. **Snapping** - the ports, which normally lie on a shore cell, are moved to
   the nearest navigable cell within :attr:`RoutePlanner.SNAP_RADIUS_KM`.
4. **Search** - A*, Dijkstra, the genetic algorithm, or the great-circle
   reference, all on the same cost map.
5. **Refinement** - if no route exists on the lattice (a strait narrower than
   a cell, or a passage outside the box), the lattice is made finer and wider
   and the whole procedure is repeated (up to :attr:`RoutePlanner.MAX_REFINEMENTS`
   times).
6. **Coastline check** - the route is sampled every kilometre against an
   independent 1 km land mask (:mod:`.coast_check`). If it crosses land, the
   offending cells are made impassable and the search is repeated; if that
   does not help, the lattice is refined; a route that still crosses land is
   returned with a warning.
7. **Post-processing** - the path is anchored on the exact port coordinates,
   optionally simplified (Douglas-Peucker), turned into a waypoint table
   (:class:`RouteLeg`) and compared with the great-circle route.

The outcome is a :class:`RoutePlan`, which :mod:`maritime_route.storage`
persists and :mod:`maritime_route.export` writes to GeoJSON, CSV and PDF.
:func:`benchmark` runs every method on one shared cost map for the
comparison in the thesis.

Units: distances in km, coordinates in degrees, speed in knots
(1 kn = 1.852 km/h), passage time in hours, cost in weighted km
(see :mod:`.astar`).
单位：距离千米，坐标度，航速节（1 节 = 1.852 千米/小时），航行时间小时。
"""
from __future__ import annotations

import logging
import math
import time
import uuid
from dataclasses import asdict, dataclass, field
from typing import Callable, Dict, List, Optional, Sequence, Tuple

import numpy as np

from ..config import (
    CLASS_NAMES,
    IMPASSABLE_COST,
    INDEX_TO_CLASS,
    NAVIGABLE_CLASSES,
    ROUTING,
    ZONE_COST,
    RoutingConfig,
)
from . import coast_check as coastline
from .astar import SearchResult, astar
from .cost_map import CostMap, GridSpec, build_cost_map, make_grid_spec,
nearest_navigable
from .dijkstra import dijkstra
from .dynamic_programming import dynamic_programming
from .genetic import genetic
from .geodesy import douglas_peucker, great_circle_points, haversine_km, path_length_km

LOGGER = logging.getLogger(__name__)

#: ``(latitude, longitude)`` in degrees. 坐标 (纬度, 经度)。
Coordinate = Tuple[float, float]
#: Optional callback ``progress(stage, fraction, details)`` for the web client.
ProgressCallback = Optional[Callable[[str, float, Dict], None]]


@dataclass
class RouteLeg:
    """One sampled point of the final route. 航线上的一个航路点。
```

```
    One row of the waypoint table shown in the UI, stored in the ``waypoint``
    database table and written to the CSV/PDF exports.

    Attributes
    ----------
    latitude, longitude : float
        Position, degrees.
    class_code : str
        Zone of the lattice cell nearest to the point.
    zone_cost : float
        Nominal weight of that zone from ``ZONE_COST`` (without the safety
        margin); ``IMPASSABLE_COST`` for a land zone.
    cumulative_km : float
        Distance along the route from the departure to this point, km.
    bearing_deg : float
        Initial great-circle course to the next point, degrees from true
        north; the last point repeats the previous course.
    """

    latitude: float
    longitude: float
    class_code: str
    zone_cost: float
    cumulative_km: float
    bearing_deg: float

    def to_dict(self) -> Dict:
        """JSON-ready dictionary with display rounding.

        Coordinates keep 6 decimals (about 0.1 m), distances 2 decimals,
        bearings 1 decimal.

        Returns
        -------
        dict
            The six fields, rounded.
        """
        return {
            "latitude": round(self.latitude, 6),
            "longitude": round(self.longitude, 6),
            "class_code": self.class_code,
            "zone_cost": round(self.zone_cost, 3),
            "cumulative_km": round(self.cumulative_km, 2),
            "bearing_deg": round(self.bearing_deg, 1),
        }


@dataclass
class RoutePlan:
    """Complete planning outcome, ready for storage, export and rendering.

    完整规划结果，可直接用于数据库存储、文件导出与前端渲染。

    Attributes
    ----------
    route_id : str
        16 hexadecimal characters taken from a random UUID4 (64 bits);
        stored as ``route.route_uid`` and used in URLs and file names.
    algorithm : str
        Method actually used: ``astar``, ``dijkstra``, ``genetic`` or
        ``great_circle``.
    start, end : Coordinate
        Departure and destination exactly as requested, degrees.
    waypoints : list of Coordinate
        Route poly-line from ``start`` to ``end``; empty if infeasible.
    legs : list of RouteLeg
        Waypoint table, one entry per vertex of ``waypoints``.
    total_cost : float
        Search cost in weighted km (``inf`` if infeasible).
    distance_km : float
        Geometric route length, km.
    great_circle_km : float
        Shortest possible distance between the two ports, km.
    detour_ratio : float
        ``distance_km / great_circle_km`` (>= 1; ``nan`` if infeasible).
    estimated_hours : float
        Passage time at ``speed_knots``, hours.
    nodes_expanded : int
        Search effort (see :class:`~.astar.SearchResult`).
    runtime_s : float
```

```
    Wall-clock planning time, seconds (includes the cost-map build unless
    a cost map was reused).
grid : dict
    Lattice description (:meth:`~.cost_map.GridSpec.to_dict`) of the
    lattice finally used.
zone_profile : dict
    Number of route waypoints in each zone.
baseline : dict
    Great-circle comparison (:meth:`RoutePlanner.compare_with_great_circle`).
cost_map_stats : dict
    :meth:`~.cost_map.CostMap.statistics` of the map used.
created_utc : str
    ISO 8601 UTC timestamp.
speed_knots : float
    Planned speed, knots.
interior_land_waypoints : int
    See the field comment below; a validity check that must be 0.
feasible : bool
    False when no navigable route was found.
message : str
    Outcome text for the UI.
coast_check : dict
    Result of the independent coastline check
    (:meth:`~.coast_check.CoastCheck.to_dict`): verdict, longest land
    stretch, number of automatic corrections and the land points.
warning : str
    Non-empty when the route still crosses land after the automatic
    corrections; shown in the UI and in the PDF report.
"""

route_id: str
algorithm: str
start: Coordinate
end: Coordinate
waypoints: List[Coordinate]
legs: List[RouteLeg]
total_cost: float
distance_km: float
great_circle_km: float
detour_ratio: float
estimated_hours: float
nodes_expanded: int
runtime_s: float
grid: Dict
zone_profile: Dict[str, int]
baseline: Dict
cost_map_stats: Dict
created_utc: str
speed_knots: float
#: Non-navigable waypoints strictly between departure and destination.
#: A voyage begins and ends at a quay, so the two end points legitimately
#: fall into a shore zone; every *intermediate* waypoint must be navigable.
#: 航程始于泊位、终于泊位，故首尾点位于岸线区属正常；
#: 中间航路点必须全部可航行，该计数应恒为 0。
interior_land_waypoints: int = 0
feasible: bool = True
message: str = ""
#: Independent coastline check (see :mod:`.coast_check`). 独立海岸线检查结果。
coast_check: Dict = field(default_factory=dict)
#: Warning for the navigator; empty when the route passed the check.
warning: str = ""

def to_dict(self) -> Dict:
    """JSON-ready dictionary of the whole plan.

    Nested dataclasses are converted by :func:`dataclasses.asdict`; the
    legs are then replaced by their rounded form, tuples become lists
    (``[lat, lon]``), and non-finite numbers (``inf``/``nan`` of an
    infeasible plan) become ``None`` because JSON cannot represent them.

    Returns
    -------
    dict
        Payload sent to the web client and stored in the job results.
    """
    payload = asdict(self)
    payload["legs"] = [leg.to_dict() for leg in self.legs]
    payload["waypoints"] = [[round(a, 6), round(b, 6)] for a, b in self.waypoints]
    payload["start"] = [round(self.start[0], 6), round(self.start[1], 6)]
    payload["end"] = [round(self.end[0], 6), round(self.end[1], 6)]
```

```
        for key in ("total_cost", "distance_km", "great_circle_km", "detour_ratio",
                    "estimated_hours", "runtime_s"):
            value = payload[key]
            payload[key] = None if value is None or not math.isfinite(value) else
round(value, 4)
        return payload


class RoutePlanner:
    """Builds cost maps and searches optimal maritime routes.

    One instance is created per classifier (the web app keeps one for the
    process lifetime). The planner itself holds no per-request state, so
    :meth:`plan` can be called repeatedly.
    航线规划器：负责构建代价地图并调用搜索算法。

    Parameters
    ----------
    classifier
        Zone classifier with ``predict_coordinates(lat, lon)``, normally
        :class:`~maritime_route.model.inference.ZoneClassifierService`.
    config : RoutingConfig
        Default lattice and search parameters.
    """

    #: Largest distance over which a departure or destination is moved onto water.
    SNAP_RADIUS_KM: float = 60.0

    def __init__(self, classifier, config: RoutingConfig = ROUTING) -> None:
        """Store the classifier and the default routing configuration.

        Parameters
        ----------
        classifier
            Object providing ``predict_coordinates``.
        config : RoutingConfig
            Defaults used when :meth:`plan` is not given overrides.
        """
        self.classifier = classifier
        self.config = config

    # ---------------------------------------------------------------------- #
    def plan(
        self,
        start: Coordinate,
        end: Coordinate,
        algorithm: str = "astar",
        resolution_deg: Optional[float] = None,
        speed_knots: float = 14.0,
        simplify_km: float = 0.0,
        progress: ProgressCallback = None,
        reuse_cost_map: Optional[CostMap] = None,
        auto_refine: bool = True,
        coast_guard: bool = True,
        _attempt: int = 0,
        _margin_deg: Optional[float] = None,
        _coast_initial: Optional[Dict] = None,
    ) -> Tuple[RoutePlan, CostMap]:
        """Plan a route between two coordinates. 规划从起点到终点的航线。

        When the search fails - because a strait is narrower than one grid cell,
        or because the only passage lies outside the rectangle around the two
        ports - the method automatically retries on a finer *and* wider lattice
        (``auto_refine``).
        当海峡窄于一个网格单元、或唯一通道位于起终点外包矩形之外而导致搜索失败时，
        自动加密并扩大网格重试。

        Parameters
        ----------
        start, end : Coordinate
            Departure and destination ``(lat, lon)``, degrees.
        algorithm : str, default "astar"
            ``"astar"``, ``"dijkstra"``, ``"genetic"``, ``"dynamic"`` or
``"great_circle"``
            (case-insensitive); any other value falls back to A*.
        resolution_deg : float, optional
            Lattice step; ``None`` uses ``config.grid_resolution_deg``.
            May be coarsened by :func:`~.cost_map.make_grid_spec` to respect
            ``max_grid_cells``.
        speed_knots : float, default 14.0
```

```
            Planned speed for the passage-time estimate.
        simplify_km : float, default 0.0
            Douglas-Peucker tolerance in km; 0 keeps every lattice vertex.
        progress : callable, optional
            Progress callback forwarded to every stage.
        reuse_cost_map : CostMap, optional
            Existing cost map to search on (no classification, no
            refinement). Used by :func:`benchmark` so that all methods share
            one map.
        auto_refine : bool, default True
            Allow the finer/wider retry on failure.
        coast_guard : bool, default True
            Check the route against the independent land mask and correct it
            automatically when it crosses land (:meth:`_coast_guard`). With
            ``reuse_cost_map`` the route is only checked, not corrected, so
            that the benchmark compares all methods on one unchanged map.
        _attempt : int
            Internal: refinement depth of this call.
        _margin_deg : float, optional
            Internal: margin override set by :meth:`_refine`.
        _coast_initial : dict, optional
            Internal: coastline check of the first route when this call is a
            refinement made because of the coastline check.

        Returns
        -------
        tuple
            ``(RoutePlan, CostMap)`` - the plan (``feasible=False`` with a
            message if no route was found) and the cost map it was computed on.
        """
        t0 = time.perf_counter()
        algorithm = algorithm.lower()
        config = self.config
        # Per-call overrides (resolution from the user, margin from _refine)
        # produce a new config; the other parameters are copied unchanged.
        # 若指定了分辨率或边距，则基于默认配置生成一份新的配置。
        if resolution_deg is not None or _margin_deg is not None:
            config = RoutingConfig(
                grid_resolution_deg=float(resolution_deg if resolution_deg is not None
                                          else self.config.grid_resolution_deg),
                max_grid_cells=self.config.max_grid_cells,
                margin_deg=float(_margin_deg if _margin_deg is not None
                                 else self.config.margin_deg),
                diagonal_moves=self.config.diagonal_moves,
                heuristic_weight=self.config.heuristic_weight,
                safety_margin_cells=self.config.safety_margin_cells,
                proximity_penalty=self.config.proximity_penalty,
            )

        # ---- 1-2. lattice and cost map ---------------------------------- #
        if progress:
            progress("grid", 0.0, {"stage": "Building geographic lattice"})
        cost_map = reuse_cost_map
        if cost_map is None:
            spec = make_grid_spec([start, end], config.grid_resolution_deg,
                                  config.margin_deg, config.max_grid_cells)
            LOGGER.info("Grid %dx%d (%d cells) at %.3f deg",
                        spec.n_rows, spec.n_cols, spec.n_cells, spec.resolution_deg)
            cost_map = build_cost_map(spec, self.classifier, config, progress=progress)
        spec = cost_map.spec

        # ---- 3. snapping the ports onto water --------------------------- #
        # A port usually lies in a shore cell, so the ends are snapped to the
        # nearest navigable cell - but never further than SNAP_RADIUS_KM, so
        # that a point far inland is reported instead of being moved to open sea.
        # 起终点吸附到最近的可航行格子，但距离不超过 SNAP_RADIUS_KM，避免把内陆点移到远海。
        # Radius in cells = km / (degrees per cell * 111.2 km per degree),
        # rounded up, at least 3 cells. 111.2 km is one degree of latitude; a
        # degree of longitude is shorter, so the east-west reach is smaller.
        snap_cells = max(3, math.ceil(self.SNAP_RADIUS_KM / (spec.resolution_deg *
111.2)))
        start_cell = nearest_navigable(cost_map, *spec.coord_to_cell(*start),
max_radius=snap_cells)
        goal_cell = nearest_navigable(cost_map, *spec.coord_to_cell(*end),
max_radius=snap_cells)
        if start_cell is not None and start_cell == goal_cell:
            # Both ends snapped onto the same cell: the lattice is too coarse.
            # 起终点吸附到同一格子，说明网格过粗。
            start_cell = goal_cell = None
        great_circle_km = float(haversine_km(start[0], start[1], end[0], end[1]))
```

```
        if start_cell is None or goal_cell is None:
            # No water near a port: try a finer lattice, else report infeasible.
            retry = self._refine(start, end, algorithm, config, speed_knots,
simplify_km,
                                 progress, auto_refine, _attempt, reuse_cost_map,
                                 coast_guard, _coast_initial)
            if retry is not None:
                return retry
            return (self._infeasible(start, end, algorithm, great_circle_km, cost_map,
                                     speed_knots, time.perf_counter() - t0,
                                     "Start or destination lies on land and no navigable
"
                                     "cell was found nearby"), cost_map)

        # ---- 4. search ------------------------------------------------ #
        if progress:
            progress("search", 0.0, {"stage": f"Running {algorithm.upper()} search"})

        if algorithm not in ("dijkstra", "genetic", "dynamic", "great_circle"):
            algorithm = "astar"
        if algorithm == "great_circle":
            result = self._great_circle_result(cost_map, start, end)
        else:
            result = self._search(algorithm, cost_map, start_cell, goal_cell, config,
progress)

        # ---- 5. refinement on failure --------------------------------- #
        if not result.found:
            # A failed genetic run does not prove that no path exists: the lattice
            # is refined only when the exact search confirms that it is blocked.
            # 遗传算法失败不代表无路可走：仅当 A* 也确认不连通时才细化网格。
            lattice_blocked = True
            if algorithm == "genetic":
                lattice_blocked = not astar(cost_map, start_cell, goal_cell,
config).found
            retry = None
            if lattice_blocked:
                retry = self._refine(start, end, algorithm, config, speed_knots,
simplify_km,
                                     progress, auto_refine, _attempt, reuse_cost_map,
                                 coast_guard, _coast_initial)
            if retry is not None:
                return retry
            return (self._infeasible(start, end, algorithm, great_circle_km, cost_map,
                                     speed_knots, time.perf_counter() - t0,
                                     result.message + " at the finest lattice tried "
                                     f"({spec.resolution_deg}°)"), cost_map)

        # ---- 6. independent coastline check --------------------------- #
        # The route is optimal on the classified lattice; the land mask, which
        # plays no part in the classification, checks it against reality.
        # 用与分类器无关的陆地掩膜检查航线，穿陆时自动修正。
        search_map = cost_map
        coast = coastline.check_route(self._anchored(result, start, end), approach=True)
        corrections, blocked = 0, 0
        initial = _coast_initial or coast.to_dict()
        if (coast.verdict == "crossing" and coast_guard and reuse_cost_map is None
                and algorithm != "great_circle"):
            result, search_map, coast, corrections, blocked = self._coast_guard(
                algorithm, result, cost_map, start_cell, goal_cell, config, start, end,
                coast, progress)
            if coast.verdict == "crossing" and auto_refine:
                # Blocking cells did not help (or closed the passage): plan the
                # voyage again on a finer lattice, which checks itself again.
                # 封锁网格无效时，在更细的网格上重新规划（新规划同样会自检）。
                retry = self._refine_for_coast(start, end, algorithm, config,
speed_knots,
                                               simplify_km, progress, _attempt, initial)
                if retry is not None and retry[0].feasible and (
                        retry[0].coast_check.get("verdict") != "crossing"
                        or retry[0].coast_check.get("max_land_run_km", 1e9)
                        < coast.max_land_run_km):
                    return retry

        # ---- 7. post-processing --------------------------------------- #
        coords = self._anchored(result, start, end)
        if simplify_km > 0:
            coords = douglas_peucker(coords, simplify_km)
            # The simplified line may cut a corner the lattice path went round.
```

```
            coast = coastline.check_route(coords, approach=True)

        legs = self._build_legs(coords, cost_map)
        distance_km = path_length_km(coords)
        zone_profile = {name: 0 for name in CLASS_NAMES}
        for leg in legs:
            zone_profile[leg.class_code] = zone_profile.get(leg.class_code, 0) + 1
        # Land waypoints other than the two ports - a self-check, expected 0.
        interior_land = sum(1 for leg in legs[1:-1]
                            if leg.class_code not in NAVIGABLE_CLASSES)

        baseline = self.compare_with_great_circle(start, end, cost_map, distance_km,
                                                  result.total_cost)
        # Knots to km/h: 1 nautical mile = 1.852 km. The floor of 0.1 kn avoids
        # a division by zero for a zero speed. 节换算为千米/小时。
        speed_kmh = max(speed_knots, 0.1) * 1.852

        if progress:
            progress("done", 1.0, {"stage": "Route ready"})

        plan = RoutePlan(
            route_id=uuid.uuid4().hex[:16],
            algorithm=algorithm,
            start=start,
            end=end,
            waypoints=coords,
            legs=legs,
            total_cost=float(result.total_cost),
            distance_km=distance_km,
            great_circle_km=great_circle_km,
            detour_ratio=distance_km / great_circle_km if great_circle_km > 0 else 1.0,
            estimated_hours=distance_km / speed_kmh,
            nodes_expanded=result.nodes_expanded,
            runtime_s=time.perf_counter() - t0,
            grid=spec.to_dict(),
            zone_profile=zone_profile,
            interior_land_waypoints=interior_land,
            baseline=baseline,
            cost_map_stats=cost_map.statistics(),
            created_utc=time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
            speed_knots=float(speed_knots),
            feasible=True,
            message=result.message,
        )
        refined = _coast_initial is not None
        plan.coast_check = coast.to_dict(corrections=corrections, refined=refined,
                                         cells_blocked=blocked,
                                         initial=initial.get("verdict"))
        plan.coast_check["initial_max_land_run_km"] = initial.get("max_land_run_km",
0.0)
        plan.message, plan.warning = self._coast_messages(plan, algorithm)
        # The map returned is the one the route was found on (with the cells
        # blocked by the coastline check), so that the overlay matches the route.
        return plan, search_map

    # ------------------------------------------------------------------ #
    @staticmethod
    def _anchored(result: SearchResult, start: Coordinate, end: Coordinate) ->
List[Coordinate]:
        """Route poly-line of a search result drawn from/to the exact ports.

        The search starts and ends at the snapped cell centres; replacing the
        first and last vertex draws the route from/to the actual quay.
        用用户输入的精确坐标替换首尾网格点。
        """
        coords = list(result.path_coords)
        coords[0], coords[-1] = start, end
        return coords

    @staticmethod
    def _search(algorithm: str, cost_map: CostMap, start_cell, goal_cell,
                config: RoutingConfig, progress: ProgressCallback) -> SearchResult:
        """Run one lattice search method on ``cost_map``.

        Parameters
        ----------
        algorithm : str
            ``astar``, ``dijkstra``, ``genetic`` or ``dynamic``.
        cost_map : CostMap
            Map to search on.
```

```
        start_cell, goal_cell : (row, col)
            Snapped ends.
        config : RoutingConfig
            Lattice and search parameters (A* and Dijkstra).
        progress : callable, optional
            Progress callback.

        Returns
        -------
        SearchResult
            Result of the method.
        """
        if algorithm == "dijkstra":
            return dijkstra(cost_map, start_cell, goal_cell, config, progress=progress)
        if algorithm == "genetic":
            # The GA uses its own GeneticConfig defaults (GENETIC); the routing
            # config does not apply to it.
            return genetic(cost_map, start_cell, goal_cell, progress=progress)
        if algorithm == "dynamic":
            # Dynamic programming: value iteration on the Bellman equation.
            # 动态规划：在贝尔曼方程上做价值迭代。
            return dynamic_programming(cost_map, start_cell, goal_cell,
progress=progress)
        return astar(cost_map, start_cell, goal_cell, config, progress=progress)

    #: Largest number of A* re-searches with blocked cells per lattice (phase 1
    #: of :meth:`_coast_guard`); A* takes a fraction of a second per round.
    #: 每个网格上"封锁后用 A* 重新搜索"的最大次数。
    MAX_COAST_CORRECTIONS: int = 8
    #: Extra rounds for the genetic algorithm on its own route (phase 2).
    MAX_METHOD_CORRECTIONS: int = 3
    #: Coarsest lattice used when the coastline check asks for refinement, deg.
    COAST_REFINE_DEG: float = 0.1

    def _coast_guard(self, algorithm, result, cost_map, start_cell, goal_cell, config,
                     start, end, coast, progress):
        """Correct a route that crosses land by blocking cells and searching again.

        封锁穿陆格子并重新搜索，直到航线不再穿越陆地。

        Phase 1 - finding the land. Every round makes impassable, on a copy of
        the map, the cells chosen by :func:`~.coast_check.cells_to_block` (the
        cells under the land samples and the vertices of the offending legs)
        and searches again with A*, which is exact and fast; the new route is
        checked, and its land is blocked in the next round. The rounds stop
        when the route no longer crosses land, when there is nothing left to
        block, when the blocked map has no route (the blocked cells closed the
        passage) or after :attr:`MAX_COAST_CORRECTIONS` rounds.

        Phase 2 - the chosen method. Dijkstra, dynamic programming and the
        genetic algorithm are then run once on the corrected map, so that the
        route shown is still the one of the method the user chose. The genetic
        algorithm is heuristic: if its route still crosses land, its own land
        is blocked for up to :attr:`MAX_METHOD_CORRECTIONS` more rounds.
        For A* the route of phase 1 is already the result.

        Parameters
        ----------
        algorithm : str
            Method chosen by the user.
        result : SearchResult
            Route that failed the check.
        cost_map : CostMap
            Map it was found on.
        start_cell, goal_cell : (row, col)
            Snapped ends (never blocked).
        config : RoutingConfig
            Search parameters.
        start, end : Coordinate
            Exact ports.
        coast : CoastCheck
            Check of ``result``.
        progress : callable, optional
            Progress callback.

        Returns
        -------
        tuple
            ``(result, cost_map, coast, corrections, cells_blocked)`` - the best
            route of the chosen method, the map it was found on, its check,
```

```
            the number of re-searches and the number of blocked cells.
        """
        protected = {tuple(start_cell), tuple(goal_cell)}
        state = {"map": cost_map, "corrections": 0, "blocked": 0}
        best = (result, coast, cost_map)

        def better(check):
            return (check.verdict != "crossing"
                    or check.max_land_run_km < best[1].max_land_run_km)

        def block_and_search(route, check, method):
            """Block the land of ``route`` and search again; None if impossible."""
            cells = coastline.cells_to_block(check, state["map"], route.path_cells,
protected)
            if not cells:
                return None
            state["corrections"] += 1
            if progress:
                progress("coast", 0.0, {"stage": (
                    f"Route crosses land for {check.max_land_run_km:.0f} km on the
coastline "
                    f"check - blocking {len(cells)} cells and searching again")})
            LOGGER.info("Coastline check: land run %.1f km, blocking %d cells (round %d,
%s)",
                        check.max_land_run_km, len(cells), state["corrections"], method)
            candidate_map = coastline.block_cells(state["map"], cells)
            if method == "astar":
                candidate = astar(candidate_map, start_cell, goal_cell, config)
            else:
                candidate = self._search(method, candidate_map, start_cell, goal_cell,
config,
                                         progress)
            if not candidate.found:
                # The blocked cells closed the only passage on this lattice.
                return None
            state["map"], state["blocked"] = candidate_map, state["blocked"] +
len(cells)
            return candidate, coastline.check_route(self._anchored(candidate, start,
end),
                                                    approach=True)

        # ---- phase 1: A* probes find the land to block ----------------- #
        probe, probe_check = result, coast
        for _ in range(self.MAX_COAST_CORRECTIONS):
            step = block_and_search(probe, probe_check, "astar")
            if step is None:
                break
            probe, probe_check = step
            if algorithm == "astar" and better(probe_check):
                best = (probe, probe_check, state["map"])
            if probe_check.verdict != "crossing":
                break

        # ---- phase 2: the chosen method on the corrected map ----------- #
        if algorithm != "astar" and state["map"] is not cost_map:
            state["corrections"] += 1
            final = self._search(algorithm, state["map"], start_cell, goal_cell, config,
progress)
            if final.found:
                check = coastline.check_route(self._anchored(final, start, end),
approach=True)
                if better(check):
                    best = (final, check, state["map"])
                for _ in range(self.MAX_METHOD_CORRECTIONS):
                    if check.verdict != "crossing":
                        break
                    step = block_and_search(final, check, algorithm)
                    if step is None:
                        break
                    final, check = step
                    if better(check):
                        best = (final, check, state["map"])
        result, coast, best_map = best
        return result, best_map, coast, state["corrections"], state["blocked"]

    def _refine_for_coast(self, start, end, algorithm, config, speed_knots, simplify_km,
                          progress, attempt, initial):
        """Plan again on a finer lattice because the route crosses land.

        The new step is :attr:`COAST_REFINE_DEG` (0.1 deg) or, when the lattice
```

```
        is already that fine, 0.55 of the current step, but not finer than
        :attr:`MIN_RESOLUTION_DEG`. The margin is kept. Returns ``None`` when
        no further refinement is allowed.
        因海岸线检查不通过而加密网格重新规划。
        """
        if attempt >= self.MAX_REFINEMENTS:
            return None
        current = config.grid_resolution_deg
        finer = self.COAST_REFINE_DEG if current > self.COAST_REFINE_DEG + 1e-9 else
round(current * 0.55, 4)
        if finer < self.MIN_RESOLUTION_DEG:
            return None
        if progress:
            progress("coast", 0.0, {"stage": (
                f"Route still crosses land at {current}° - planning again on a {finer}°
lattice")})
        LOGGER.info("Coastline check: refining %.3f -> %.3f deg", current, finer)
        return self.plan(start, end, algorithm, finer, speed_knots, simplify_km,
                         progress=progress, auto_refine=True, _attempt=attempt + 1,
                         _margin_deg=config.margin_deg, _coast_initial=initial)

    @staticmethod
    def _coast_messages(plan: RoutePlan, algorithm: str) -> Tuple[str, str]:
        """Outcome message and warning text derived from the coastline check."""
        c = plan.coast_check
        message = plan.message
        verdict = c.get("verdict")
        if verdict == "unchecked":
            return message, ""
        changed = c.get("initial_verdict") == "crossing" and verdict != "crossing"
        if changed:
            steps = []
            if c.get("corrections"):
                n = c["corrections"]
                steps.append(f"{n} re-search{'es' if n > 1 else ''} with "
                             f"{c['cells_blocked']} cells blocked")
            if c.get("refined"):
                steps.append(f"a finer lattice ({plan.grid.get('resolution_deg')}°)")
            message += (f"; coastline check: the first route crossed land for "
                        f"{c.get('initial_max_land_run_km', 0):.0f} km and was corrected
"
                        f"automatically ({' and '.join(steps)})")
        elif verdict in ("clear", "grazing"):
            message += "; coastline check passed"
        if verdict != "crossing" or algorithm == "great_circle":
            return message, ""
        warning = (f"The route crosses land for {c['max_land_run_km']:.0f} km according
to the "
                   f"independent 1 km coastline (GLOBE land mask), and the automatic
correction "
                   f"could not remove it. Do not follow this course without checking the
"
                   f"passage on an official nautical chart.")
        return message, warning

    # ------------------------------------------------------------------ #
    def compare_with_great_circle(
        self,
        start: Coordinate,
        end: Coordinate,
        cost_map: CostMap,
        optimal_km: float,
        optimal_cost: float,
        samples: int = 160,
    ) -> Dict:
        """Evaluate the naive shortest-distance route on the same cost map.

        在同一代价地图上评估"最短距离"大圆航线，用于安全性对比。
        Returns the zone profile of the great-circle line, the number of samples
        that fall on land, and the relative gain of the optimised route.

        The great circle is sampled at ``samples`` equally spaced points; each
        sample is assigned the zone of its nearest lattice cell. The weighted
        cost is approximated as ``sum(segment_km * w(end of segment))`` (a
        right-endpoint rule, coarser than the trapezoidal edge cost of the
        searches, which is adequate for a reference value).

        Parameters
        ----------
        start, end : Coordinate
```

```
            Ports, degrees.
        cost_map : CostMap
            Map on which the optimised route was computed.
        optimal_km : float
            Length of the optimised route, km.
        optimal_cost : float
            Search cost of the optimised route, weighted km.
        samples : int, default 160
            Number of points on the great circle.

        Returns
        -------
        dict
            ``distance_km``, ``weighted_cost`` (None if the line crosses
            land), ``zone_profile``, ``samples``, ``land_samples``,
            ``interior_land_samples`` (ports excluded), ``land_share``,
            ``is_navigable``, ``extra_distance_km`` and ``extra_distance_pct``
            of the optimised route, ``cost_saving_pct`` (None if not
            comparable) and a textual ``verdict``.
        """
        line = great_circle_points(start, end, samples)
        spec = cost_map.spec
        profile = {name: 0 for name in CLASS_NAMES}
        unsafe = 0
        interior_unsafe = 0
        total_cost = 0.0
        previous: Optional[Coordinate] = None
        last_index = len(line) - 1
        for i, (lat, lon) in enumerate(line):
            row, col = spec.coord_to_cell(lat, lon)
            code = INDEX_TO_CLASS[int(cost_map.class_index[row, col])]
            profile[code] += 1
            if code not in NAVIGABLE_CLASSES:
                unsafe += 1
                # Exclude the two berths so that this figure is directly
                # comparable with interior_land_waypoints of the optimised route.
                # 排除首尾泊位，使该指标与优化航线的中间航路点统计口径一致。
                if 0 < i < last_index:
                    interior_unsafe += 1
            weight = float(cost_map.cost[row, col])
            if previous is not None:
                seg = float(haversine_km(previous[0], previous[1], lat, lon))
                total_cost += seg * min(weight, IMPASSABLE_COST)
            previous = (lat, lon)

        gc_km = path_length_km(line)
        navigable = unsafe == 0
        # A route that crosses land has no meaningful finite cost; report None
        # so that the UI and the PDF show "infeasible" instead of a huge number.
        # 穿越陆地的航线不存在有限代价，此处返回 None，前端显示"不可行"。
        return {
            "distance_km": round(gc_km, 3),
            "weighted_cost": round(total_cost, 3) if navigable else None,
            "zone_profile": profile,
            "samples": samples,
            "land_samples": unsafe,
            "interior_land_samples": interior_unsafe,
            "land_share": round(unsafe / max(samples, 1), 4),
            "is_navigable": navigable,
            # Extra length the optimised route accepts for safety, km and %.
            "extra_distance_km": round(optimal_km - gc_km, 3),
            "extra_distance_pct": round(100.0 * (optimal_km - gc_km) / gc_km, 2) if
gc_km else 0.0,
            # Relative cost reduction versus the great circle (only when the
            # great circle is navigable and therefore has a finite cost).
            "cost_saving_pct": (round(100.0 * (total_cost - optimal_cost) / total_cost,
2)
                                if navigable and total_cost > 0 else None),
            "verdict": ("Great-circle route crosses land and is not navigable"
                        if unsafe else "Great-circle route is navigable but ignores zone
risk"),
        }

    # ------------------------------------------------------------------ #
    @staticmethod
    def _build_legs(coords: Sequence[Coordinate], cost_map: CostMap) -> List[RouteLeg]:
        """Turn the route poly-line into the waypoint table.

        For every vertex: the cumulative great-circle distance from the
        departure, the zone of its nearest lattice cell with the nominal zone
```

```
        weight, and the initial bearing towards the next vertex. The last
        vertex has no successor and repeats the previous bearing.
        生成航路点表：累计距离、所在区域及其权重、驶向下一点的方位角。

        Parameters
        ----------
        coords : sequence of Coordinate
            Route vertices, degrees.
        cost_map : CostMap
            Map used to look up the zone of each vertex.

        Returns
        -------
        list of RouteLeg
            One entry per vertex, in route order.
        """
        from .geodesy import initial_bearing_deg

        spec = cost_map.spec
        legs: List[RouteLeg] = []
        cumulative = 0.0
        for i, (lat, lon) in enumerate(coords):
            if i > 0:
                cumulative += float(haversine_km(coords[i - 1][0], coords[i - 1][1],
lat, lon))
            row, col = spec.coord_to_cell(lat, lon)
            code = INDEX_TO_CLASS[int(cost_map.class_index[row, col])]
            # Nominal zone weight (the safety-margin penalty is not included).
            weight = ZONE_COST[code]
            bearing = (initial_bearing_deg(lat, lon, coords[i + 1][0], coords[i + 1][1])
                       if i + 1 < len(coords) else (legs[-1].bearing_deg if legs else
0.0))
            legs.append(RouteLeg(
                latitude=float(lat), longitude=float(lon), class_code=code,
                # inf (land) is stored as the finite sentinel for numeric columns.
                zone_cost=float(weight) if math.isfinite(weight) else IMPASSABLE_COST,
                cumulative_km=cumulative, bearing_deg=bearing,
            ))
        return legs

    def _great_circle_result(self, cost_map: CostMap, start: Coordinate,
                             end: Coordinate) -> SearchResult:
        """Wrap the naive baseline in a :class:`SearchResult` for uniformity.

        The great circle is sampled at 160 points and priced like in
        :meth:`compare_with_great_circle` (segment length times the weight of
        the cell at the segment end; land at ``IMPASSABLE_COST``). ``found``
        is always True because the line always exists; whether it crosses
        land is reported through ``RoutePlan.baseline["is_navigable"]``.
        ``nodes_expanded`` is set to the number of samples.

        Parameters
        ----------
        cost_map : CostMap
            Map used for pricing.
        start, end : Coordinate
            Ports, degrees.

        Returns
        -------
        SearchResult
            ``algorithm="great_circle"`` with an empty ``path_cells``.
        """
        t0 = time.perf_counter()
        coords = great_circle_points(start, end, 160)
        spec = cost_map.spec
        total = 0.0
        for a, b in zip(coords[:-1], coords[1:]):
            row, col = spec.coord_to_cell(*b)
            seg = float(haversine_km(a[0], a[1], b[0], b[1]))
            total += seg * min(float(cost_map.cost[row, col]), IMPASSABLE_COST)
        return SearchResult(
            found=True, path_cells=[], path_coords=coords, total_cost=total,
            distance_km=path_length_km(coords), nodes_expanded=len(coords),
            nodes_generated=len(coords), runtime_s=time.perf_counter() - t0,
            algorithm="great_circle", message="Great-circle reference route",
        )

    #: Maximum number of automatic finer/wider retries. 最大重试次数。
    MAX_REFINEMENTS: int = 3
```

```
#: Finest lattice step the refinement may reach, degrees (~4.4 km).
MIN_RESOLUTION_DEG: float = 0.04

def _refine(self, start, end, algorithm, config, speed_knots, simplify_km,
            progress, auto_refine, attempt, reuse_cost_map, coast_guard=True,
            coast_initial=None):
    """Retry on a finer and wider lattice; ``None`` when refinement is over.

    Every retry multiplies the cell size by 0.55 (to open straits narrower
    than a cell) and widens the margin around the two ports by 2 degrees
    (so that a passage lying outside the first rectangle - for example the
    Skagerrak on a voyage from the North Sea into the Baltic - can be found).

    网格重试：每次分辨率乘以 0.55、外扩边距增加 2 度，最多重试 MAX_REFINEMENTS 次。

    With the default 0.25 deg the sequence is 0.1375 -> 0.0756 -> 0.0416
    deg. Refinement is not done when a cost map was supplied for reuse
    (the benchmark must stay on one map) or when ``auto_refine`` is off.

    Parameters
    ----------
    start, end : Coordinate
        Ports, degrees.
    algorithm : str
        Method to rerun.
    config : RoutingConfig
        Configuration of the failed attempt (its resolution and margin
        are the basis of the next one).
    speed_knots, simplify_km, progress
        Passed through to :meth:`plan`.
    auto_refine : bool
        Whether refinement is allowed.
    attempt : int
        Number of refinements already made.
    reuse_cost_map : CostMap or None
        Cost map supplied by the caller, if any.
    coast_guard : bool
        Passed through to :meth:`plan`.
    coast_initial : dict, optional
        Passed through to :meth:`plan` (refinement inside a refinement
        started by the coastline check).

    Returns
    -------
    tuple or None
        The ``(RoutePlan, CostMap)`` of the retry, or ``None`` if no
        further refinement is allowed.
    """
    if (not auto_refine or reuse_cost_map is not None
            or attempt >= self.MAX_REFINEMENTS):
        return None
    finer = round(config.grid_resolution_deg * 0.55, 4)
    if finer < self.MIN_RESOLUTION_DEG:
        return None
    LOGGER.info("No path at %.3f deg - refining to %.3f deg (attempt %d)",
                config.grid_resolution_deg, finer, attempt + 1)
    if progress:
        progress("refine", 0.0,
                 {"stage": f"No passage at {config.grid_resolution_deg}° - "
                           f"refining the lattice to {finer}°"})
    # Recursive call: builds a new cost map on the finer, wider lattice.
    return self.plan(start, end, algorithm, finer, speed_knots, simplify_km,
                     progress=progress, auto_refine=True, coast_guard=coast_guard,
                     _attempt=attempt + 1, _margin_deg=config.margin_deg + 2.0,
                     _coast_initial=coast_initial)

def _infeasible(self, start, end, algorithm, gc_km, cost_map, speed_knots,
                runtime, message) -> RoutePlan:
    """Build a :class:`RoutePlan` that reports failure.

    The plan has no waypoints, ``feasible=False``, infinite cost and
    ``nan`` ratios (converted to ``None`` by :meth:`RoutePlan.to_dict`),
    so the client can show the reason instead of a route.
    构造"不可行"结果：无航路点，代价为无穷大。

    Parameters
    ----------
    start, end : Coordinate
        Requested ports.
    algorithm : str
```

```
            Method that was attempted.
        gc_km : float
            Great-circle distance between the ports, km.
        cost_map : CostMap
            Last cost map tried (its grid and statistics are reported).
        speed_knots : float
            Requested speed.
        runtime : float
            Elapsed time, seconds.
        message : str
            Reason shown to the user.

        Returns
        -------
        RoutePlan
            Infeasible plan.
        """
        return RoutePlan(
            route_id=uuid.uuid4().hex[:16], algorithm=algorithm, start=start, end=end,
            waypoints=[], legs=[], total_cost=float("inf"), distance_km=0.0,
            great_circle_km=gc_km, detour_ratio=float("nan"),
estimated_hours=float("nan"),
            nodes_expanded=0, runtime_s=runtime, grid=cost_map.spec.to_dict(),
            zone_profile={name: 0 for name in CLASS_NAMES},
            baseline={}, cost_map_stats=cost_map.statistics(),
            created_utc=time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
            speed_knots=float(speed_knots), feasible=False, message=message,
        )


def benchmark(planner: RoutePlanner, start: Coordinate, end: Coordinate,
              resolution_deg: Optional[float] = None) -> Dict:
    """Run all algorithms on one shared cost map and compare them.

    在同一代价地图上运行三种算法并对比耗时、扩展节点数与总代价。
    The cost map is built once *before* timing so that the reported runtimes
    measure the search only and are therefore directly comparable.
    先构建一次代价地图再计时，使三种算法的耗时可直接比较。

    Timing: each ``runtime_s`` is the ``time.perf_counter`` interval of one
    :meth:`RoutePlanner.plan` call with ``reuse_cost_map``, so it excludes
    classification and cost-map construction; it includes the search plus
    the same small post-processing for every method (snapping, waypoint
    table, great-circle comparison). Four methods are run: A*, Dijkstra, the
    genetic algorithm and the great-circle reference.

    Parameters
    ----------
    planner : RoutePlanner
        Planner with a loaded classifier.
    start, end : Coordinate
        Ports, degrees.
    resolution_deg : float, optional
        Initial lattice step; the warm-up run may refine it.

    Returns
    -------
    dict
        ``grid``, ``cost_map_build_s``, a ``results`` row per method
        (feasibility, distance, cost, nodes, runtime, detour ratio, note),
        ``astar_node_reduction`` (Dijkstra nodes / A* nodes),
        ``astar_time_ratio`` (Dijkstra time / A* time),
        ``optimality_gap_pct`` (|A* - Dijkstra| cost, expected 0),
        ``genetic_gap_pct`` (GA cost above the A* optimum) and the
        cost-map statistics.
    """
    # Build (and, if a strait is too narrow, refine) the shared cost map once.
    warmup, cost_map = planner.plan(start, end, "astar", resolution_deg)
    resolved = cost_map.spec.resolution_deg

    rows = []
    plans = {}
    for algorithm in ("astar", "dijkstra", "genetic", "dynamic", "great_circle"):
        plan, _ = planner.plan(start, end, algorithm, resolved,
                               reuse_cost_map=cost_map, auto_refine=False)
        plans[algorithm] = plan
        # The great-circle "route" always exists; it counts as infeasible
        # when any of its samples lies on land.
        crosses_land = (algorithm == "great_circle"
                        and not (plan.baseline or {}).get("is_navigable", True))
```

```
        rows.append({
            "algorithm": algorithm,
            "feasible": bool(plan.feasible and not crosses_land),
            "distance_km": round(plan.distance_km, 2),
            "total_cost": (None if crosses_land or not math.isfinite(plan.total_cost)
                           else round(plan.total_cost, 2)),
            "nodes_expanded": plan.nodes_expanded,
            "runtime_s": round(plan.runtime_s, 4),
            "detour_ratio": (None if not math.isfinite(plan.detour_ratio)
                             else round(plan.detour_ratio, 4)),
            "note": ("crosses land - not navigable" if crosses_land else plan.message),
        })

    a, d = plans["astar"], plans["dijkstra"]
    # How many times fewer nodes / less time A* needs than Dijkstra.
    speedup = round(d.nodes_expanded / a.nodes_expanded, 2) if a.nodes_expanded else
None
    time_ratio = round(d.runtime_s / a.runtime_s, 2) if a.runtime_s > 0 else None
    # Both searches are optimal, so their costs must agree within rounding.
    optimality_gap = None
    if math.isfinite(a.total_cost) and math.isfinite(d.total_cost) and d.total_cost > 0:
        optimality_gap = round(100.0 * abs(a.total_cost - d.total_cost) / d.total_cost,
6)

    # The genetic algorithm is not exact: its distance from the optimum found by
    # A* is reported as a percentage. 遗传算法不保证最优，报告其与 A* 最优代价的差距。
    g = plans["genetic"]
    genetic_gap = None
    if g.feasible and math.isfinite(g.total_cost) and math.isfinite(a.total_cost) and
a.total_cost > 0:
        # max(0, ...) removes a "-0.0" left by floating-point summation order.
        genetic_gap = max(0.0, round(100.0 * (g.total_cost - a.total_cost) /
a.total_cost, 3))

    # Dynamic programming is exact as well: its cost must equal the A* optimum.
    # 动态规划同样是精确算法，其代价应与 A* 最优值相同。
    dp = plans["dynamic"]
    dynamic_gap = None
    if dp.feasible and math.isfinite(dp.total_cost) and math.isfinite(a.total_cost) and
a.total_cost > 0:
        dynamic_gap = round(100.0 * abs(dp.total_cost - a.total_cost) / a.total_cost, 6)

    return {
        "start": list(start), "end": list(end),
        "grid": cost_map.spec.to_dict(),
        "cost_map_build_s": round(cost_map.build_seconds, 4),
        "results": rows,
        "genetic_gap_pct": genetic_gap,
        "dynamic_gap_pct": dynamic_gap,
        "astar_node_reduction": speedup,
        "astar_time_ratio": time_ratio,
        "optimality_gap_pct": optimality_gap,
        "cost_map": cost_map.statistics(),
    }
```

D.5 Storage

storage/schema.sql – the database schema

```
-- ---------------------------------------------------------------------------
-- Maritime Route Planner - relational schema (SQLite)
-- 数据库模式：存储会话、点位、分类结果、航线及航路点。
-- ---------------------------------------------------------------------------
-- This script is executed by RouteRepository.initialise() on every start-up.
-- Every statement uses IF NOT EXISTS, so running it against an existing
-- database is harmless and never drops data (idempotent).
-- 本脚本在每次启动时执行，所有语句均带 IF NOT EXISTS，可重复执行而不丢失数据。
--
-- Entity overview
--   session 1 --- n point        classified AIS / user points of one upload
--   session 1 --- n route        routes planned within a session (optional)
--   route   1 --- n waypoint     ordered vertices of a route
--   zone    1 --- n point / waypoint   reference table of the four zone classes
```

```
--   model_run                   independent log of classifier training runs
--
-- Conventions
--   * Coordinates are WGS 84 decimal degrees (latitude, longitude).
--   * Distances in km, speed in knots, durations in hours or seconds as named.
--   * Timestamps are ISO 8601 UTC text "YYYY-MM-DDTHH:MM:SSZ" (SQLite has no
--     native date type, and this format sorts correctly as text).
--   * Booleans are INTEGER 0/1 guarded by CHECK constraints.
--   * Free-form structured data is stored as JSON text in *_json columns.
--
-- NOTE for maintainers: RouteRepository._migrate_algorithm_check() copies the
-- text of the route table below, from its CREATE line up to the first
-- closing bracket followed by a semicolon. Comments inside that table must
-- therefore not contain that two-character sequence, and must not quote the
-- name of any search method in single quotes (the migration looks for the
-- quoted names to detect an up-to-date table).
-- ---------------------------------------------------------------------------

-- Enforce REFERENCES clauses. SQLite ignores foreign keys unless this pragma
-- is on for the connection. The repository also sets it on every connection
-- it opens, because the pragma is per connection, not stored in the file.
-- 开启外键约束（SQLite 默认关闭，且为连接级设置）。
PRAGMA foreign_keys = ON;

-- Ingestion sessions: one row per uploaded AIS file or interactive session.
-- 会话表：每次上传的 AIS 文件或一次交互会话对应一行。
CREATE TABLE IF NOT EXISTS session (
    -- Surrogate key. AUTOINCREMENT guarantees ids are never reused after a delete.
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    -- Descriptive data: display name and origin (file name or "interactive").
    name            TEXT    NOT NULL,
    source          TEXT    NOT NULL,
    -- Denormalised counter of point rows, incremented by save_points() so the
    -- session list does not need a COUNT(*) join. 冗余计数，避免联表统计。
    n_points        INTEGER NOT NULL DEFAULT 0,
    -- Creation time, filled by SQLite when the insert does not supply it.
    created_utc     TEXT    NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%SZ','now')),
    notes           TEXT
);

-- Individual geographic points with their classification result.
-- 点位表：单个地理点及其神经网络分类结果。
CREATE TABLE IF NOT EXISTS point (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    -- Owning session. ON DELETE CASCADE: deleting a session deletes its points.
    session_id      INTEGER NOT NULL REFERENCES session(id) ON DELETE CASCADE,
    -- Position in degrees, range-checked so that swapped lat/lon values are rejected
    -- whenever the "latitude" would exceed +/-90.
    latitude        REAL    NOT NULL CHECK (latitude  BETWEEN -90  AND 90),
    longitude       REAL    NOT NULL CHECK (longitude BETWEEN -180 AND 180),
    -- Physical measurements in metres (either may be unknown = NULL).
    -- elevation_m: height above sea level, depth_m: NOAA relief value
    -- (positive above sea level, negative below).
    elevation_m     REAL,
    depth_m         REAL,
    -- Classification result: zone code (foreign key to the zone reference
    -- table) and the softmax probability of that class, 0..1.
    class_code      TEXT    REFERENCES zone(code),
    confidence      REAL    CHECK (confidence IS NULL OR (confidence BETWEEN 0 AND 1)),
    -- Original AIS timestamp of the observation, if the source had one.
    observed_utc    TEXT
);
-- Indexes for the three access paths: by session (listing, cascade delete),
-- by position (spatial range queries) and by class (histogram GROUP BY).
-- 索引：按会话、按坐标、按类别查询。
CREATE INDEX IF NOT EXISTS ix_point_session ON point(session_id);
CREATE INDEX IF NOT EXISTS ix_point_coords  ON point(latitude, longitude);
CREATE INDEX IF NOT EXISTS ix_point_class   ON point(class_code);

-- Reference table of the four geographic zones and their traversal weights.
-- 区域参照表：四个地理区域类别及其通行权重（启动时由程序根据 config 同步）。
-- The rows are (re)written from config.ZONE_COST by RouteRepository.initialise()
-- with an UPSERT, so the table always matches the weights the planner uses.
CREATE TABLE IF NOT EXISTS zone (
    -- Class name, e.g. OPEN_SEA. Natural primary key referenced by point and waypoint.
    code            TEXT PRIMARY KEY,
    -- Presentation: title for the UI and a longer description.
    title           TEXT NOT NULL,
    description     TEXT,
    -- Weight w(z) of the routing cost model. An impassable zone (infinite
```

```
    -- weight in the code) is stored as -1 because SQL REAL has no infinity.
    traversal_cost  REAL NOT NULL,
    -- 1 = ships may enter the zone (OPEN_SEA, COASTAL_SEA), 0 = land/shore.
    navigable       INTEGER NOT NULL CHECK (navigable IN (0, 1)),
    -- Map colour of the zone, hex "#rrggbb".
    colour          TEXT
);

-- Planned routes with their aggregated parameters.
-- 航线表：每条规划航线的汇总参数（航路点明细见 waypoint 表）。
CREATE TABLE IF NOT EXISTS route (
    -- Internal integer key (used by waypoint.route_id) and the public
    -- 16-hex-digit identifier from RoutePlan.route_id (used in URLs and file names).
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    route_uid         TEXT    NOT NULL UNIQUE,
    -- Optional owning session. ON DELETE SET NULL: a route outlives its session.
    session_id        INTEGER REFERENCES session(id) ON DELETE SET NULL,
    -- Search method. The CHECK list must include every method the planner
    -- offers - older databases lacked the genetic and the dynamic-programming
    -- methods and are upgraded by the repository migration.
    algorithm         TEXT    NOT NULL CHECK (algorithm IN
('astar','dijkstra','genetic','dynamic','great_circle')),
    -- Requested departure and destination, degrees.
    start_lat         REAL    NOT NULL,
    start_lon         REAL    NOT NULL,
    end_lat           REAL    NOT NULL,
    end_lon           REAL    NOT NULL,
    -- Route quality: search cost (weighted km, NULL if infinite), length and
    -- great-circle distance in km, and their ratio (>= 1, NULL if undefined).
    total_cost        REAL,
    distance_km       REAL    NOT NULL,
    great_circle_km   REAL    NOT NULL,
    detour_ratio      REAL,
    -- Voyage estimate: passage time in hours at the planned speed in knots.
    estimated_hours   REAL,
    speed_knots       REAL,
    -- Search effort and wall-clock planning time in seconds.
    nodes_expanded    INTEGER,
    runtime_s         REAL    NOT NULL,
    -- Lattice used: step in degrees and total number of cells.
    grid_resolution   REAL,
    grid_cells        INTEGER,
    -- 1 = a navigable route was found, 0 = the planner reported failure.
    feasible          INTEGER NOT NULL DEFAULT 1 CHECK (feasible IN (0, 1)),
    -- JSON documents: great-circle comparison and waypoint count per zone.
    baseline_json     TEXT,
    zone_profile_json TEXT,
    created_utc       TEXT    NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%SZ','now')),
    -- Independent coastline check of the route (JSON: verdict, longest land
    -- stretch, automatic corrections). Added in version 1.2 as the last column,
    -- so that older databases are upgraded with ALTER TABLE ... ADD COLUMN.
    coast_check_json  TEXT
);
-- Indexes for per-method statistics (GROUP BY algorithm) and for listing
-- routes newest first (ORDER BY created_utc).
CREATE INDEX IF NOT EXISTS ix_route_algorithm ON route(algorithm);
CREATE INDEX IF NOT EXISTS ix_route_created   ON route(created_utc);

-- Ordered waypoints belonging to a route.
-- 航路点表：属于某条航线的有序航路点。
CREATE TABLE IF NOT EXISTS waypoint (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    -- Owning route. ON DELETE CASCADE: deleting a route removes its waypoints,
    -- which is how RouteRepository.delete_route() cleans up.
    route_id        INTEGER NOT NULL REFERENCES route(id) ON DELETE CASCADE,
    -- Position of the waypoint along the route, 0 = departure.
    seq             INTEGER NOT NULL,
    -- Waypoint position, degrees.
    latitude        REAL    NOT NULL,
    longitude       REAL    NOT NULL,
    -- Zone of the waypoint and its nominal weight (1e6 marks an impassable zone).
    class_code      TEXT    REFERENCES zone(code),
    zone_cost       REAL,
    -- Distance from the departure in km and course to the next waypoint in
    -- degrees from true north.
    cumulative_km   REAL,
    bearing_deg     REAL,
    -- A route cannot have two waypoints with the same sequence number.
    UNIQUE (route_id, seq)
);
```

```
CREATE INDEX IF NOT EXISTS ix_waypoint_route ON waypoint(route_id);

-- Model training runs, kept for traceability of the deployed classifier.
-- 模型训练记录：用于追溯当前部署的分类模型。
CREATE TABLE IF NOT EXISTS model_run (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    -- Model version string, e.g. "1.0.0".
    version         TEXT NOT NULL,
    -- Test-set quality metrics (fractions 0..1, Cohen kappa -1..1).
    accuracy        REAL,
    macro_f1        REAL,
    cohen_kappa     REAL,
    -- Size of the network (trainable parameters) and epochs actually trained.
    parameters      INTEGER,
    epochs          INTEGER,
    trained_utc     TEXT,
    -- Full training metadata as JSON (the per-epoch history is omitted).
    metadata_json   TEXT
);

-- Convenience view used by the statistics endpoint.
-- 汇总视图：每条航线一行，并附带航路点数量。
-- One row per route with its waypoint count. LEFT JOIN keeps routes without
-- waypoints (infeasible plans) with n_waypoints = 0, and GROUP BY r.id
-- collapses the joined waypoint rows back to one row per route.
CREATE VIEW IF NOT EXISTS v_route_summary AS
SELECT r.route_uid,
       r.algorithm,
       r.distance_km,
       r.great_circle_km,
       r.detour_ratio,
       r.total_cost,
       r.runtime_s,
       r.nodes_expanded,
       r.feasible,
       r.created_utc,
       COUNT(w.id) AS n_waypoints
FROM route r
LEFT JOIN waypoint w ON w.route_id = r.id
GROUP BY r.id;
```

storage/__init__.py

```
"""Persistence package.

持久化层：SQLite 仓储。

Stores classification sessions, classified points, planned routes with their
waypoints, and classifier training runs in a single SQLite file
(``config.DATABASE_PATH``).

Contents
--------
repository.RouteRepository
    All database access of the application (create/list/get/delete routes,
    save points, statistics, schema creation and migration).
schema.sql
    The relational schema (tables ``session``, ``point``, ``zone``,
    ``route``, ``waypoint``, ``model_run`` and the view ``v_route_summary``),
    executed by the repository at start-up.
"""
from .repository import RouteRepository

# Public name exported by ``from maritime_route.storage import *``.
__all__ = ["RouteRepository"]
```

storage/repository.py – the repository

```
"""SQLite persistence layer.

数据持久化层：负责会话、点位、航线与航路点的存取。
SQLite was selected because the application is a single-user desktop/web tool:
it needs zero administration, ships as one file, and still provides full
transactional SQL with foreign keys and views.

Role in the pipeline
```

```
--------------------
The web application stores every classification upload (``session`` +
``point`` rows) and every planned route (``route`` + ``waypoint`` rows) through
:class:`RouteRepository`, and reads them back for the history list, the route
detail view, re-export and the statistics page. The table definitions live in
``schema.sql`` next to this module; see the comments there for every column.

Design
------
* Repository pattern: SQL is confined to this class; callers exchange plain
  dictionaries and the planner's dataclasses.
* One short-lived connection per operation (:meth:`RouteRepository.connect`):
  SQLite connections must not be shared between threads, and the web
  application calls the repository both from the event loop and from worker
  threads (``asyncio.to_thread``).
* A lock owned by the repository instance serialises *writes*, so that
  concurrent requests do not fail with "database is locked"; reads run
  without it.
* WAL journal mode lets readers proceed while a write is in progress.

设计：仓储模式；每次操作使用独立连接；写操作加锁串行化；WAL 模式允许读写并发。
"""
from __future__ import annotations

import json
import logging
import sqlite3
import threading
from contextlib import contextmanager
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, Iterable, List, Optional, Sequence

from ..config import (
    CLASS_COLOR,
    CLASS_DESCRIPTION,
    CLASS_NAMES,
    DATABASE_PATH,
    NAVIGABLE_CLASSES,
    SUPPORTED_ALGORITHMS,
    ZONE_COST,
)

LOGGER = logging.getLogger(__name__)
#: DDL script executed at start-up. 建表脚本路径。
SCHEMA_PATH = Path(__file__).with_name("schema.sql")


class RouteRepository:
    """Thin repository over the SQLite database. 仓储模式封装。

    Parameters
    ----------
    path : Path or str, default ``config.DATABASE_PATH``
        Database file; its parent directory is created if missing, and the
        schema is created or upgraded immediately.

    Attributes
    ----------
    path : Path
        Location of the database file.
    """

    def __init__(self, path: Path | str = DATABASE_PATH) -> None:
        """Open (or create) the database file and bring the schema up to date.

        Parameters
        ----------
        path : Path or str
            Database file location.
        """
        self.path = Path(path)
        self.path.parent.mkdir(parents=True, exist_ok=True)
        # Serialises write transactions within this process. 写操作互斥锁。
        self._lock = threading.Lock()
        self.initialise()

    # ------------------------------------------------------------------ #
    @contextmanager
    def connect(self):
```

```
        """Yield a connection with foreign keys and row factory configured.

        The connection is committed when the ``with`` block finishes normally,
        rolled back if it raises, and always closed. Every block is therefore
        one transaction.
        上下文管理器：正常结束则提交，异常则回滚，最后关闭连接。

        Yields
        ------
        sqlite3.Connection
            Connection whose rows are :class:`sqlite3.Row` (accessible by
            column name and convertible with ``dict(row)``).

        Raises
        ------
        sqlite3.Error
            Any database error is re-raised after the rollback.
        """
        # timeout=30 s: wait for a lock held by another process instead of
        # failing immediately.
        connection = sqlite3.connect(self.path, timeout=30.0)
        connection.row_factory = sqlite3.Row
        # Foreign keys are off by default in SQLite and the setting is per
        # connection, so it is switched on for every new connection.
        connection.execute("PRAGMA foreign_keys = ON")
        # Write-ahead log: readers are not blocked by a writer. The mode is
        # stored in the file, so repeating the pragma is cheap.
        connection.execute("PRAGMA journal_mode = WAL")
        try:
            yield connection
            connection.commit()
        except Exception:
            connection.rollback()
            raise
        finally:
            connection.close()

    def initialise(self) -> None:
        """Create the schema and seed the zone reference table.

        Runs ``schema.sql`` (idempotent), upgrades an old ``route`` table if
        necessary, and upserts the four zone rows from ``config`` so that the
        stored weights always match the ones used by the planner.
        建表、必要时迁移旧表，并按配置写入/更新四个区域的参照数据。
        """
        with self._lock, self.connect() as conn:
            conn.executescript(SCHEMA_PATH.read_text(encoding="utf-8"))
            # The column must exist before the table rebuild below copies rows
            # with SELECT * (same column order as schema.sql).
            self._migrate_coast_column(conn)
            self._migrate_algorithm_check(conn)
            for name in CLASS_NAMES:
                cost = ZONE_COST[name]
                # UPSERT: insert the zone, or refresh cost and navigability if
                # it already exists (title, description and colour are kept).
                conn.execute(
                    "INSERT INTO zone(code, title, description, traversal_cost,
navigable, colour) "
                    "VALUES (?,?,?,?,?,?) ON CONFLICT(code) DO UPDATE SET "
                    "traversal_cost=excluded.traversal_cost,
navigable=excluded.navigable",
                    (
                        name,
                        name.replace("_", " ").title(),     # OPEN_SEA -> "Open Sea"
                        CLASS_DESCRIPTION[name],
                        # SQL REAL has no infinity: -1 marks an impassable zone.
                        float(cost) if cost != float("inf") else -1.0,
                        1 if name in NAVIGABLE_CLASSES else 0,
                        CLASS_COLOR[name],
                    ),
                )

    @staticmethod
    def _migrate_coast_column(conn) -> None:
        """Add ``route.coast_check_json`` to a database created before version 1.2.

        The column is the last one of the table in ``schema.sql``, so
        ``ALTER TABLE ... ADD COLUMN`` (which appends) gives exactly the
        current column order. Old routes keep NULL there (not checked).
        为旧数据库添加海岸线检查结果列。
```

```
        Parameters
        ----------
        conn : sqlite3.Connection
            Open connection inside the initialisation transaction.
        """
        columns = [row[1] for row in conn.execute("PRAGMA table_info(route)")]
        if columns and "coast_check_json" not in columns:
            conn.execute("ALTER TABLE route ADD COLUMN coast_check_json TEXT")
            LOGGER.info("Database upgraded: route.coast_check_json added")

    @staticmethod
    def _migrate_algorithm_check(conn) -> None:
        """Upgrade a database created before the genetic algorithm existed.

        Older databases constrain ``route.algorithm`` to three values, so a
        genetic-algorithm route could not be saved. SQLite cannot alter a CHECK
        constraint in place; the table is therefore rebuilt with the current
        definition from ``schema.sql`` and the stored rows are copied across.
        升级旧数据库：旧表的 CHECK 约束不含 genetic，SQLite 无法直接修改约束，
        因此按 schema.sql 的新定义重建 route 表并复制原有数据。

        Procedure (the standard SQLite "12-step" table rebuild, reduced):

        1. Detect: read the stored ``CREATE TABLE route`` text from
           ``sqlite_master``; nothing to do if it already names every method
           of ``SUPPORTED_ALGORITHMS`` or the table does not exist.
        2. Cut the current ``route`` definition out of ``schema.sql`` and
           rename it to ``route_new``.
        3. Switch foreign keys off, so that dropping ``route`` does not cascade
           into ``waypoint``, and drop the views that reference ``route``.
        4. Create ``route_new``, copy all rows, drop ``route``, rename.
           ``SELECT *`` relies on the column order being unchanged between the
           old and the new definition (only the CHECK differs). The row ids
           are preserved, so ``waypoint.route_id`` still points at the right
           routes.
        5. Recreate the indexes and the views (by re-running the idempotent
           schema) and switch foreign keys back on.

        Parameters
        ----------
        conn : sqlite3.Connection
            Open connection inside the initialisation transaction.
        """
        row = conn.execute("SELECT sql FROM sqlite_master WHERE type='table' AND
name='route'").fetchone()
        # Up to date when every method the planner offers appears, quoted, in the
        # stored CHECK constraint. 存储的 CHECK 约束已包含全部算法名时无需升级。
        if row is None or all(f"'{name}'" in row[0] for name in SUPPORTED_ALGORITHMS):
            return
        schema = SCHEMA_PATH.read_text(encoding="utf-8")
        # Text of the route table definition: from its CREATE line to the
        # closing ");". 从 schema.sql 中截取 route 表的定义。
        start = schema.index("CREATE TABLE IF NOT EXISTS route (")
        definition = schema[start:schema.index(");", start) + 2]
        definition = definition.replace("CREATE TABLE IF NOT EXISTS route (", "CREATE
TABLE route_new (")
        # Prevent ON DELETE CASCADE into waypoint when the old table is dropped.
        conn.execute("PRAGMA foreign_keys = OFF")
        # Views that read the table are dropped first and recreated from the
        # schema afterwards. 先删除依赖该表的视图，重建后再按 schema 恢复。
        views = [v for (v,) in conn.execute("SELECT name FROM sqlite_master WHERE
type='view'")]
        for view in views:
            conn.execute(f'DROP VIEW IF EXISTS "{view}"')
        conn.execute(definition)
        conn.execute("INSERT INTO route_new SELECT * FROM route")
        conn.execute("DROP TABLE route")
        conn.execute("ALTER TABLE route_new RENAME TO route")
        # Indexes were dropped together with the old table.
        conn.execute("CREATE INDEX IF NOT EXISTS ix_route_algorithm ON
route(algorithm)")
        conn.execute("CREATE INDEX IF NOT EXISTS ix_route_created   ON
route(created_utc)")
        conn.executescript(schema)                # recreates the dropped views
        conn.execute("PRAGMA foreign_keys = ON")
        LOGGER.info("Database upgraded: route.algorithm now accepts %s",
SUPPORTED_ALGORITHMS)

    # -- sessions ------------------------------------------------------- #
```

```
def create_session(self, name: str, source: str, n_points: int = 0,
                   notes: str = "") -> int:
    """Insert a new ingestion session. 新建会话。

    Parameters
    ----------
    name : str
        Display name.
    source : str
        Origin of the data (file name or ``"interactive"``).
    n_points : int, default 0
        Initial point counter; normally 0 and increased by
        :meth:`save_points`.
    notes : str, default ""
        Free text.

    Returns
    -------
    int
        The new ``session.id``.
    """
    with self._lock, self.connect() as conn:
        cur = conn.execute(
            "INSERT INTO session(name, source, n_points, notes) VALUES (?,?,?,?)",
            (name, source, int(n_points), notes),
        )
        return int(cur.lastrowid)

def list_sessions(self, limit: int = 50) -> List[Dict]:
    """Most recent sessions first. 按时间倒序列出会话。

    Parameters
    ----------
    limit : int, default 50
        Maximum number of rows.

    Returns
    -------
    list of dict
        One dictionary per ``session`` row (all columns).
    """
    with self.connect() as conn:
        # id DESC = newest first (ids grow monotonically with AUTOINCREMENT).
        rows = conn.execute(
            "SELECT * FROM session ORDER BY id DESC LIMIT ?", (limit,)
        ).fetchall()
        return [dict(r) for r in rows]

# -- points ------------------------------------------------------------ #
def save_points(self, session_id: int, results: Sequence) -> int:
    """Bulk-insert classification results. 批量写入分类结果。

    All rows and the session counter update are written in one
    transaction with ``executemany``, which is much faster than one
    insert per point.

    Parameters
    ----------
    session_id : int
        Owning session (must exist - enforced by the foreign key).
    results : sequence
        :class:`~maritime_route.model.inference.ClassificationResult`
        objects with ``latitude``, ``longitude``, ``depth_m``,
        ``class_code`` and ``confidence``.

    Returns
    -------
    int
        Number of points written.

    Raises
    ------
    sqlite3.IntegrityError
        If ``session_id`` does not exist or a value violates a CHECK.
    """
    # Column order: session_id, latitude, longitude, elevation_m (not
    # available -> NULL), depth_m, class_code, confidence, observed_utc (NULL).
    payload = [
        (session_id, r.latitude, r.longitude, None, r.depth_m,
         r.class_code, r.confidence, None)
```

```
            for r in results
        ]
        with self._lock, self.connect() as conn:
            conn.executemany(
                "INSERT INTO point(session_id, latitude, longitude, elevation_m,
depth_m, "
                "class_code, confidence, observed_utc) VALUES (?,?,?,?,?,?,?,?)",
                payload,
            )
            # Keep the denormalised counter in step with the inserted rows.
            conn.execute("UPDATE session SET n_points = n_points + ? WHERE id = ?",
                         (len(payload), session_id))
        return len(payload)

    def class_histogram(self, session_id: Optional[int] = None) -> Dict[str, int]:
        """Number of stored points per zone class. 各类别点数统计。

        Parameters
        ----------
        session_id : int, optional
            Restrict to one session; ``None`` counts all points.

        Returns
        -------
        dict
            ``{class_code: count}``; classes without points are absent.
        """
        query = "SELECT class_code, COUNT(*) AS n FROM point"
        params: tuple = ()
        if session_id is not None:
            # Parameter placeholder, never string formatting (SQL injection safe).
            query += " WHERE session_id = ?"
            params = (session_id,)
        query += " GROUP BY class_code"
        with self.connect() as conn:
            return {row["class_code"]: row["n"] for row in conn.execute(query, params)}

    # -- routes ------------------------------------------------------------ #
    def save_route(self, plan, session_id: Optional[int] = None) -> int:
        """Persist a :class:`RoutePlan` together with its waypoints.

        The route row and all waypoint rows are written in one transaction,
        so a route is never stored without its waypoints. Non-finite numbers
        (cost, ratio and time of an infeasible plan) are stored as NULL.
        The nested ``baseline``, ``zone_profile`` and ``coast_check``
        dictionaries are stored as JSON text.
        在同一事务中写入航线及其全部航路点。

        Parameters
        ----------
        plan : RoutePlan
            Result of :meth:`~maritime_route.routing.planner.RoutePlanner.plan`.
        session_id : int, optional
            Session the route belongs to.

        Returns
        -------
        int
            Internal ``route.id`` of the new row.

        Raises
        ------
        sqlite3.IntegrityError
            If ``plan.route_id`` already exists (UNIQUE) or the algorithm is
            not allowed by the CHECK constraint.
        """
        with self._lock, self.connect() as conn:
            cur = conn.execute(
                """INSERT INTO route(route_uid, session_id, algorithm, start_lat,
start_lon,
                                     end_lat, end_lon, total_cost, distance_km,
great_circle_km,
                                     detour_ratio, estimated_hours, speed_knots,
nodes_expanded,
                                     runtime_s, grid_resolution, grid_cells, feasible,
                                     baseline_json, zone_profile_json, created_utc,
                                     coast_check_json)
                   VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)""",
                (
                    plan.route_id, session_id, plan.algorithm,
```

```
                    plan.start[0], plan.start[1], plan.end[0], plan.end[1],
                    _finite(plan.total_cost), plan.distance_km, plan.great_circle_km,
                    _finite(plan.detour_ratio), _finite(plan.estimated_hours),
                    plan.speed_knots, plan.nodes_expanded, plan.runtime_s,
                    plan.grid.get("resolution_deg"), plan.grid.get("n_cells"),
                    1 if plan.feasible else 0,
                    json.dumps(plan.baseline), json.dumps(plan.zone_profile),
                    plan.created_utc,
                    json.dumps(getattr(plan, "coast_check", {}) or {}),
                ),
            )
            route_id = int(cur.lastrowid)
            # seq = position of the leg in the route (0 = departure).
            conn.executemany(
                "INSERT INTO waypoint(route_id, seq, latitude, longitude, class_code, "
                "zone_cost, cumulative_km, bearing_deg) VALUES (?,?,?,?,?,?,?,?)",
                [
                    (route_id, i, leg.latitude, leg.longitude, leg.class_code,
                     leg.zone_cost, leg.cumulative_km, leg.bearing_deg)
                    for i, leg in enumerate(plan.legs)
                ],
            )
            return route_id

    def list_routes(self, limit: int = 50) -> List[Dict]:
        """Newest routes first, from the ``v_route_summary`` view.

        ``route_uid`` is the tie-breaker for routes created in the same second
        (timestamps have one-second resolution), which keeps the order stable.

        Parameters
        ----------
        limit : int, default 50
            Maximum number of rows.

        Returns
        -------
        list of dict
            Summary columns of each route plus ``n_waypoints``.
        """
        with self.connect() as conn:
            rows = conn.execute(
                "SELECT * FROM v_route_summary ORDER BY created_utc DESC, route_uid DESC
LIMIT ?",
                (limit,),
            ).fetchall()
            return [dict(r) for r in rows]

    def get_route(self, route_uid: str) -> Optional[Dict]:
        """Load one route with its decoded JSON fields and its waypoints.

        Parameters
        ----------
        route_uid : str
            Public route identifier.

        Returns
        -------
        dict or None
            All ``route`` columns, with ``baseline_json``,
            ``zone_profile_json`` and ``coast_check_json`` replaced by the
            parsed ``baseline``, ``zone_profile`` and ``coast_check`` dictionaries, plus
``waypoints`` - a list of
            dictionaries ordered by ``seq``. ``None`` if the route is unknown.
        """
        with self.connect() as conn:
            row = conn.execute("SELECT * FROM route WHERE route_uid = ?",
(route_uid,)).fetchone()
            if row is None:
                return None
            route = dict(row)
            # Replace the JSON text columns by parsed objects ("{}" for NULL).
            route["baseline"] = json.loads(route.pop("baseline_json") or "{}")
            route["zone_profile"] = json.loads(route.pop("zone_profile_json") or "{}")
            route["coast_check"] = json.loads(route.pop("coast_check_json", None) or
"{}")
            route["waypoints"] = [
                dict(w) for w in conn.execute(
                    "SELECT seq, latitude, longitude, class_code, zone_cost,
cumulative_km, "
```

```
                    "bearing_deg FROM waypoint WHERE route_id = ? ORDER BY seq",
(route["id"],)
                )
            ]
            return route

    def delete_route(self, route_uid: str) -> bool:
        """Delete a route; its waypoints go with it (ON DELETE CASCADE).

        Parameters
        ----------
        route_uid : str
            Public route identifier.

        Returns
        -------
        bool
            True if a route was deleted, False if it did not exist.
        """
        with self._lock, self.connect() as conn:
            cur = conn.execute("DELETE FROM route WHERE route_uid = ?", (route_uid,))
            return cur.rowcount > 0

    # -- model runs ------------------------------------------------------- #
    def record_model_run(self, metadata: Dict) -> int:
        """Log a classifier training run. 记录一次模型训练。

        Parameters
        ----------
        metadata : dict
            Contents of the model metadata JSON written by the trainer:
            ``version``, ``created_utc`` and ``metrics`` with ``accuracy``,
            ``macro_f1``, ``cohen_kappa`` and ``training`` (``parameters``,
            ``epochs_run``). Missing keys are stored as NULL.

        Returns
        -------
        int
            The new ``model_run.id``.
        """
        metrics = metadata.get("metrics", {})
        with self._lock, self.connect() as conn:
            cur = conn.execute(
                "INSERT INTO model_run(version, accuracy, macro_f1, cohen_kappa,
parameters, "
                "epochs, trained_utc, metadata_json) VALUES (?,?,?,?,?,?,?,?)",
                (
                    metadata.get("version", "1.0.0"),
                    metrics.get("accuracy"), metrics.get("macro_f1"),
                    metrics.get("cohen_kappa"),
                    (metrics.get("training") or {}).get("parameters"),
                    (metrics.get("training") or {}).get("epochs_run"),
                    metadata.get("created_utc"),
                    # The per-epoch history is large and already in the model
                    # metadata file, so it is left out of the database copy.
                    json.dumps({k: v for k, v in metadata.items() if k != "history"}),
                ),
            )
            return int(cur.lastrowid)

    # -- statistics ------------------------------------------------------- #
    def statistics(self) -> Dict:
        """Aggregate figures for the statistics page. 数据库统计信息。

        Returns
        -------
        dict
            Row counts of the four main tables, the class histogram of all
            points, per-algorithm route count and mean runtime (s), distance
            (km) and nodes expanded, the database path and its size in KiB.
        """
        with self.connect() as conn:
            def scalar(sql: str, default=0):
                """First column of the first row, or ``default`` for NULL.

                Parameters
                ----------
                sql : str
                    Query returning one value.
                default
```

```
                    Value returned when the query yields NULL.

                Returns
                -------
                object
                    The scalar result.
                """
                value = conn.execute(sql).fetchone()[0]
                return default if value is None else value

            # AVG() over an empty group is NULL, hence "or 0.0" before rounding.
            algorithms = {
                row["algorithm"]: {
                    "n": row["n"],
                    "avg_runtime_s": round(row["avg_runtime"] or 0.0, 4),
                    "avg_distance_km": round(row["avg_distance"] or 0.0, 2),
                    "avg_nodes": round(row["avg_nodes"] or 0.0, 1),
                }
                for row in conn.execute(
                    "SELECT algorithm, COUNT(*) n, AVG(runtime_s) avg_runtime, "
                    "AVG(distance_km) avg_distance, AVG(nodes_expanded) avg_nodes "
                    "FROM route GROUP BY algorithm"
                )
            }
            return {
                "sessions": scalar("SELECT COUNT(*) FROM session"),
                "points": scalar("SELECT COUNT(*) FROM point"),
                "routes": scalar("SELECT COUNT(*) FROM route"),
                "waypoints": scalar("SELECT COUNT(*) FROM waypoint"),
                "class_histogram": self.class_histogram(),
                "by_algorithm": algorithms,
                "database_file": str(self.path),
                "database_size_kb": round(self.path.stat().st_size / 1024, 1)
                if self.path.exists() else 0.0,
            }


def _finite(value) -> Optional[float]:
    """Convert inf/nan to NULL so that SQL aggregates stay meaningful.

    AVG() over a column containing a huge sentinel or NaN would be
    meaningless; NULL values are simply skipped by SQL aggregates.
    将无穷大/非数值转换为 NULL。

    Parameters
    ----------
    value
        Any value convertible with ``float()``.

    Returns
    -------
    float or None
        The finite float, or ``None`` for inf, nan and non-numeric input.
    """
    try:
        value = float(value)
    except (TypeError, ValueError):
        return None
    # value == value is False only for NaN (IEEE 754).
    return value if value == value and abs(value) != float("inf") else None
```

D.6 Export

export/__init__.py

```
"""Export package: GeoJSON, CSV and PDF writers.

导出层：GeoJSON / CSV / PDF 写出。

Converts a :class:`~maritime_route.routing.planner.RoutePlan` (and optionally
the classified AIS points of a session) into files a user can open in other
tools:
```

```
* **GeoJSON** (RFC 7946) - for GIS software and web maps; coordinates are
  written in ``[longitude, latitude]`` order as the standard requires.
* **CSV** - the waypoint table for spreadsheets and pandas.
* **PDF** - a printable voyage report built with ReportLab.

All functions live in :mod:`.exporters`; the names below are its public API,
used by the web application's download endpoints.
"""
from .exporters import (
    classified_points_to_geojson, export_all, points_to_csv_string, route_to_csv_string,
    route_to_geojson, route_to_pdf, write_csv, write_geojson,
)

# Public names exported by ``from maritime_route.export import *``.
__all__ = [
    "classified_points_to_geojson", "export_all", "points_to_csv_string",
    "route_to_csv_string", "route_to_geojson", "route_to_pdf", "write_csv",
"write_geojson",
]
```

export/exporters.py – GeoJSON, CSV and PDF writers

```
"""Export of planning results to GeoJSON, CSV and PDF.

结果导出模块：GeoJSON / CSV / PDF 三种格式，满足任务书的导出要求。

Role in the pipeline
--------------------
The last stage of the application: a
:class:`~maritime_route.routing.planner.RoutePlan` produced by the planner (or
re-loaded from the database) and, optionally, the classified points of an AIS
upload are converted into files. The web application calls these functions
from its download endpoints; :func:`export_all` writes every format at once.

Formats
-------
GeoJSON (RFC 7946)
    A ``FeatureCollection`` with the optimised route (``LineString``), one
    ``Point`` per waypoint and the great-circle reference line. RFC 7946
    requires positions in ``[longitude, latitude]`` order (x, y), the reverse
    of the ``(lat, lon)`` tuples used everywhere else in the code, so every
    coordinate is swapped when written. Styling properties (``stroke``,
    ``marker-color`` ...) follow the informal "simplestyle" convention
    understood by geojson.io and GitHub.
CSV
    One row per waypoint (see :data:`ROUTE_CSV_HEADER`), or one row per
    classified point with the probability of every class.
PDF
    An A4 voyage report rendered with ReportLab (summary table, optional map,
    zone profile, comparison with the great circle, optional model summary
    and a waypoint appendix).

Numeric conventions: coordinates are rounded to 6 decimals (about 0.1 m),
distances are in km (the PDF also gives nautical miles, 1 NM = 1.852 km),
speed in knots, time in hours. Values that cannot be represented (infinite
cost of an impassable zone, NaN of an infeasible plan) are written as JSON
``null``, CSV ``inf`` or "-" in the PDF.
数值约定：坐标保留 6 位小数；距离单位千米；不可表示的数值写为 null / inf / "-"。
"""
from __future__ import annotations

import csv
import io
import json
import logging
import math
from datetime import datetime, timezone
from pathlib import Path
from typing import Dict, Iterable, List, Optional, Sequence

from ..config import (
    APP_AUTHOR,
    IMPASSABLE_COST,
    APP_UNIVERSITY,
    APP_VERSION,
    CLASS_COLOR,
    CLASS_DESCRIPTION,
    CLASS_NAMES,
```

```
    OUTPUT_DIR,
    ZONE_COST,
)

LOGGER = logging.getLogger(__name__)


# ---------------------------------------------------------------------------- #
# GeoJSON                                                                      #
# ---------------------------------------------------------------------------- #
def route_to_geojson(plan, include_points: bool = True,
                     include_baseline: bool = True) -> Dict:
    """Build a GeoJSON FeatureCollection describing the planned route.

    生成 GeoJSON：包含航线 LineString、各航路点 Point 以及大圆参考航线。

    Feature order: the optimised route (only if the plan has waypoints), the
    waypoints, then the great-circle reference (only if the plan has a
    baseline comparison). Each feature carries ``feature_type`` so a reader
    can tell them apart.

    Parameters
    ----------
    plan : RoutePlan
        Planning result.
    include_points : bool, default True
        Add a ``Point`` feature for every waypoint with its zone, cost,
        cumulative distance (km) and bearing (degrees).
    include_baseline : bool, default True
        Add the great-circle line (128 samples) with its navigability verdict.

    Returns
    -------
    dict
        GeoJSON document, ready for :func:`json.dumps`.
    """
    features: List[Dict] = []

    if plan.waypoints:
        features.append({
            "type": "Feature",
            "properties": {
                "feature_type": "optimal_route",
                "route_id": plan.route_id,
                "algorithm": plan.algorithm,
                "distance_km": round(plan.distance_km, 3),
                "great_circle_km": round(plan.great_circle_km, 3),
                "detour_ratio": _clean(plan.detour_ratio),
                "total_cost": _clean(plan.total_cost),
                "estimated_hours": _clean(plan.estimated_hours),
                "speed_knots": plan.speed_knots,
                "nodes_expanded": plan.nodes_expanded,
                "runtime_s": round(plan.runtime_s, 4),
                "created_utc": plan.created_utc,
                "coast_check": (getattr(plan, "coast_check", {}) or {}).get("verdict"),
                "coast_max_land_run_km": (getattr(plan, "coast_check", {}) or
{}).get("max_land_run_km"),
                "warning": getattr(plan, "warning", ""),
                "stroke": "#0b6efd", "stroke-width": 4, "stroke-opacity": 0.9,
            },
            "geometry": {
                "type": "LineString",
                # RFC 7946 order is [lon, lat]; the plan stores (lat, lon).
                # GeoJSON 坐标顺序为 [经度, 纬度]，与内部 (纬度, 经度) 相反。
                "coordinates": [[round(lon, 6), round(lat, 6)] for lat, lon in
plan.waypoints],
            },
        })

    if include_points:
        for i, leg in enumerate(plan.legs):
            features.append({
                "type": "Feature",
                "properties": {
                    "feature_type": "waypoint",
                    "seq": i,
                    "class_code": leg.class_code,
                    "class_name": CLASS_DESCRIPTION.get(leg.class_code, ""),
                    # JSON has no representation for infinity - use null.
                    # JSON 无法表示无穷大，此处写为 null。
```

```
                    "zone_cost": (None if leg.zone_cost >= IMPASSABLE_COST
                                  else round(leg.zone_cost, 3)),
                    "navigable": leg.zone_cost < IMPASSABLE_COST,
                    "cumulative_km": round(leg.cumulative_km, 2),
                    "bearing_deg": round(leg.bearing_deg, 1),
                    "marker-color": CLASS_COLOR.get(leg.class_code, "#666666"),
                },
                "geometry": {"type": "Point",
                             "coordinates": [round(leg.longitude, 6),
round(leg.latitude, 6)]},
            })

    if include_baseline and plan.baseline:
        # Imported here to keep the export package free of a module-level
        # dependency on the routing package.
        from ..routing.geodesy import great_circle_points
        line = great_circle_points(plan.start, plan.end, 128)
        features.append({
            "type": "Feature",
            "properties": {
                "feature_type": "great_circle_reference",
                "distance_km": plan.baseline.get("distance_km"),
                "is_navigable": plan.baseline.get("is_navigable"),
                "land_share": plan.baseline.get("land_share"),
                "verdict": plan.baseline.get("verdict"),
                "stroke": "#d62828", "stroke-width": 2, "stroke-dasharray": "6,6",
            },
            "geometry": {"type": "LineString",
                         "coordinates": [[round(lon, 6), round(lat, 6)] for lat, lon in
line]},
        })

    # "name", "crs" and "metadata" are foreign members (RFC 7946, 6.1): readers
    # that do not know them ignore them. "crs" is the pre-RFC (2008) way of
    # declaring the reference system; CRS84 = WGS 84 in lon/lat order, which
    # is also the RFC 7946 default, so it only helps older tools such as QGIS 2.
    return {
        "type": "FeatureCollection",
        "name": f"maritime_route_{plan.route_id}",
        "crs": {"type": "name", "properties": {"name":
"urn:ogc:def:crs:OGC:1.3:CRS84"}},
        "metadata": {
            "generator": f"Maritime Route Planner {APP_VERSION}",
            "author": APP_AUTHOR,
            "exported_utc": datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ"),
            # Cost model used for the route; infinite weights become null.
            "zone_weights": {k: (None if math.isinf(v) else v) for k, v in
ZONE_COST.items()},
        },
        "features": features,
    }


def classified_points_to_geojson(results: Sequence) -> Dict:
    """GeoJSON of classified AIS points. 分类后的 AIS 点导出。

    One ``Point`` feature per point with the predicted class, its
    confidence, the NOAA depth in metres and the probability of every class
    (``p_OPEN_SEA`` ...). Coordinates are in RFC 7946 ``[lon, lat]`` order.

    Parameters
    ----------
    results : sequence of ClassificationResult
        Output of the classifier service.

    Returns
    -------
    dict
        GeoJSON ``FeatureCollection``.
    """
    return {
        "type": "FeatureCollection",
        "name": "classified_points",
        "crs": {"type": "name", "properties": {"name":
"urn:ogc:def:crs:OGC:1.3:CRS84"}},
        "features": [
            {
                "type": "Feature",
                "properties": {
                    "class_code": r.class_code,
```

```
                    "class_name": CLASS_DESCRIPTION.get(r.class_code, ""),
                    "confidence": round(r.confidence, 4),
                    "depth_m": None if r.depth_m is None else round(r.depth_m, 1),
                    "marker-color": CLASS_COLOR.get(r.class_code, "#666666"),
                    **{f"p_{k}": round(v, 4) for k, v in r.probabilities.items()},
                },
                "geometry": {"type": "Point",
                             "coordinates": [round(r.longitude, 6), round(r.latitude,
6)]},
            }
            for r in results
        ],
    }


def write_geojson(document: Dict, path: Path | str) -> Path:
    """Write a GeoJSON document to disk as UTF-8 text.

    ``ensure_ascii=False`` keeps non-ASCII characters (e.g. Chinese or
    Russian zone descriptions) readable instead of ``\\uXXXX`` escapes;
    ``indent=2`` makes the file human-readable.

    Parameters
    ----------
    document : dict
        GeoJSON object.
    path : Path or str
        Target file; missing parent directories are created.

    Returns
    -------
    Path
        The written file.
    """
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(document, indent=2, ensure_ascii=False), encoding="utf-
8")
    return path


# ---------------------------------------------------------------------------- #
# CSV                                                                          #
# ---------------------------------------------------------------------------- #
#: Columns of the route CSV, one row per waypoint:
#: ``seq`` (0 = departure), ``latitude``/``longitude`` (degrees, 6 decimals),
#: ``class_code`` (zone), ``zone_cost`` (nominal weight or ``inf``),
#: ``cumulative_km`` (distance from departure, km), ``bearing_deg`` (course to
#: the next waypoint, degrees from north), and the ``route_id`` and
#: ``algorithm`` of the plan repeated on every row so that files can be
#: concatenated. 航线 CSV 的列定义。
ROUTE_CSV_HEADER = [
    "seq", "latitude", "longitude", "class_code", "zone_cost",
    "cumulative_km", "bearing_deg", "route_id", "algorithm",
]


def route_to_csv_string(plan) -> str:
    """Serialise the waypoint table to CSV text. 航路点 CSV。

    Parameters
    ----------
    plan : RoutePlan
        Planning result.

    Returns
    -------
    str
        CSV text with a header row (:data:`ROUTE_CSV_HEADER`) and ``\\n``
        line endings; comma separator, dot as decimal mark.
    """
    buffer = io.StringIO()
    writer = csv.writer(buffer, lineterminator="\n")
    writer.writerow(ROUTE_CSV_HEADER)
    for i, leg in enumerate(plan.legs):
        # The planner stores impassability as a large finite constant so that the
        # weights fit in a numeric array; the export reports it as "inf", which
        # pandas and NumPy both read back as infinity.
        # 内部用大常数表示不可通行，导出时写作 "inf"，便于下游正确解析。
        cost = ("inf" if leg.zone_cost >= IMPASSABLE_COST else f"{leg.zone_cost:.3f}")
```

```
        writer.writerow([
            i, f"{leg.latitude:.6f}", f"{leg.longitude:.6f}", leg.class_code,
            cost, f"{leg.cumulative_km:.2f}", f"{leg.bearing_deg:.1f}",
            plan.route_id, plan.algorithm,
        ])
    return buffer.getvalue()


def points_to_csv_string(results: Sequence) -> str:
    """Serialise classification results to CSV text. 分类结果 CSV。

    Columns: ``latitude``, ``longitude`` (degrees), ``class_code``,
    ``confidence`` (0..1), ``depth_m`` (empty when unknown) and one
    ``p_<CLASS>`` probability column per class in ``CLASS_NAMES`` order.

    Parameters
    ----------
    results : sequence of ClassificationResult
        Output of the classifier service.

    Returns
    -------
    str
        CSV text with a header row.
    """
    buffer = io.StringIO()
    writer = csv.writer(buffer, lineterminator="\n")
    writer.writerow(["latitude", "longitude", "class_code", "confidence", "depth_m",
                     *[f"p_{c}" for c in CLASS_NAMES]])
    for r in results:
        writer.writerow([
            f"{r.latitude:.6f}", f"{r.longitude:.6f}", r.class_code,
f"{r.confidence:.4f}",
            "" if r.depth_m is None else f"{r.depth_m:.1f}",
            *[f"{r.probabilities.get(c, 0.0):.4f}" for c in CLASS_NAMES],
        ])
    return buffer.getvalue()


def write_csv(text: str, path: Path | str) -> Path:
    """Write CSV text to disk.

    Parameters
    ----------
    text : str
        CSV content.
    path : Path or str
        Target file; missing parent directories are created.

    Returns
    -------
    Path
        The written file.
    """
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    # "utf-8-sig" prepends a byte-order mark so that Microsoft Excel detects
    # UTF-8 instead of the local ANSI code page. 带 BOM，便于 Excel 正确识别编码。
    path.write_text(text, encoding="utf-8-sig")
    return path


# ---------------------------------------------------------------------------- #
# PDF                                                                          #
# ---------------------------------------------------------------------------- #
def route_to_pdf(plan, path: Path | str, map_png: Optional[bytes] = None,
                 model_summary: Optional[Dict] = None) -> Path:
    """Render a one-file PDF voyage report using ReportLab.

    生成 PDF 航行报告：包含摘要表、区域分布、对比结论与航路点清单。

    Layout (A4 portrait, margins 18 mm left/right and 16 mm top/bottom, so
    the text column is 174 mm wide; every table is at most 168 mm):

    1. Title, generator line and route identifier.
    2. *Voyage summary* - two-column parameter table.
    3. *Route chart* - only when ``map_png`` is given (168 x 100 mm image).
    4. *Geographic zone profile* - waypoints per zone with weight and share.
    5. *Comparison with the shortest-distance route* - only when the plan has
       a baseline.
```

```
    6. *Classification model* - only when ``model_summary`` is given
       (unnumbered).
    7. *Appendix* on a new page - the waypoint list, thinned to about 120 rows.

    Section numbers shift by one when the map is absent. ReportLab
    ("platypus") lays out the flowables in ``story`` and breaks pages
    automatically.

    Parameters
    ----------
    plan : RoutePlan
        Planning result.
    path : Path or str
        Output PDF file; missing parent directories are created.
    map_png : bytes, optional
        PNG image of the route map, supplied by the caller (the web
        application currently passes ``None``, so the section is omitted).
    model_summary : dict, optional
        ``architecture``, ``n_features``, ``parameters``, ``accuracy``,
        ``macro_f1`` and ``trained_utc`` of the classifier.

    Returns
    -------
    Path
        The written PDF.

    Raises
    ------
    ImportError
        If ReportLab is not installed (imported lazily so that the other
        formats work without it).
    """
    from reportlab.lib import colors
    from reportlab.lib.pagesizes import A4
    from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
    from reportlab.lib.units import mm
    from reportlab.platypus import (
        Image, PageBreak, Paragraph, SimpleDocTemplate, Spacer, Table, TableStyle,
    )

    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)

    # Paragraph styles: dark-blue headings, 9.5 pt body text, 8 pt grey notes.
    styles = getSampleStyleSheet()
    h1 = ParagraphStyle("H1", parent=styles["Heading1"], fontSize=17, spaceAfter=8,
                        textColor=colors.HexColor("#0b3c68"))
    h2 = ParagraphStyle("H2", parent=styles["Heading2"], fontSize=12.5, spaceBefore=10,
                        spaceAfter=5, textColor=colors.HexColor("#0b3c68"))
    body = ParagraphStyle("Body", parent=styles["BodyText"], fontSize=9.5, leading=13)
    small = ParagraphStyle("Small", parent=body, fontSize=8,
textColor=colors.HexColor("#555555"))

    document = SimpleDocTemplate(
        str(path), pagesize=A4,
        leftMargin=18 * mm, rightMargin=18 * mm, topMargin=16 * mm, bottomMargin=16 *
mm,
        title=f"Voyage report {plan.route_id}", author=APP_AUTHOR,
    )
    story: List = []

    story.append(Paragraph("Maritime Route Planning Report", h1))
    story.append(Paragraph(
        f"Generated by Maritime Route Planner v{APP_VERSION} &mdash;
{APP_UNIVERSITY}<br/>"
        f"Route identifier <b>{plan.route_id}</b>, created {plan.created_utc}", small))
    story.append(Spacer(1, 6))

    def table(rows: List[List[str]], widths: List[float], header: bool = True) -> Table:
        """Build a report table in the common style.

        Thin grid, 8.5 pt text; with ``header`` the first row is white bold
        text on dark blue and the body rows alternate white / light blue.
        Cell coordinates in ``TableStyle`` are ``(column, row)``, and
        ``-1`` means the last column/row.

        Parameters
        ----------
        rows : list of list of str
            Table content, header row first.
```

```
        widths : list of float
            Column widths in points (``mm`` multiples).
        header : bool, default True
            Style the first row as a header.

        Returns
        -------
        reportlab.platypus.Table
            Styled table flowable.
        """
        t = Table(rows, colWidths=widths, hAlign="LEFT")
        style = [
            ("FONTSIZE", (0, 0), (-1, -1), 8.5),
            ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
            ("GRID", (0, 0), (-1, -1), 0.4, colors.HexColor("#b9c6d4")),
            ("TOPPADDING", (0, 0), (-1, -1), 3),
            ("BOTTOMPADDING", (0, 0), (-1, -1), 3),
        ]
        if header:
            style += [
                ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#0b3c68")),
                ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
                ("FONTNAME", (0, 0), (-1, 0), "Helvetica-Bold"),
                ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white,
colors.HexColor("#eef3f8")]),
            ]
        t.setStyle(TableStyle(style))
        return t

    # --- 1. voyage summary ---------------------------------------------------
    story.append(Paragraph("1. Voyage summary", h2))
    story.append(table([
        ["Parameter", "Value"],
        ["Departure (lat, lon)", f"{plan.start[0]:.4f}, {plan.start[1]:.4f}"],
        ["Destination (lat, lon)", f"{plan.end[0]:.4f}, {plan.end[1]:.4f}"],
        ["Search algorithm", plan.algorithm.upper()],
        # 1 nautical mile = 1.852 km. 1 海里 = 1.852 千米。
        ["Route length", f"{plan.distance_km:,.1f} km ({plan.distance_km / 1.852:,.1f}
NM)"],
        ["Great-circle distance", f"{plan.great_circle_km:,.1f} km"],
        ["Detour ratio", _fmt(plan.detour_ratio, "{:.3f}")],
        ["Weighted traversal cost", _fmt(plan.total_cost, "{:,.1f}")],
        ["Planned speed", f"{plan.speed_knots:.1f} kn"],
        ["Estimated passage time", _fmt(plan.estimated_hours, "{:,.1f} h")],
        ["Nodes expanded", f"{plan.nodes_expanded:,}"],
        ["Planning runtime", f"{plan.runtime_s:.3f} s"],
        ["Grid", f"{plan.grid.get('n_rows')} x {plan.grid.get('n_cols')} cells at "
                 f"{plan.grid.get('resolution_deg')}°"],
        ["Feasible", "Yes" if plan.feasible else "No"],
        ["Coastline check", _coast_text(getattr(plan, "coast_check", {}) or {})],
    ], [58 * mm, 110 * mm]))
    if getattr(plan, "warning", ""):
        # The route still crosses land on the independent coastline.
        # 航线经独立海岸线检查仍穿越陆地：在报告中醒目提示。
        story.append(Spacer(1, 4))
        story.append(Paragraph(f"<font color='#b00020'><b>Warning.</b>
{plan.warning}</font>", body))

    # --- 2. map --------------------------------------------------------------
    if map_png:
        story.append(Paragraph("2. Route chart", h2))
        story.append(Image(io.BytesIO(map_png), width=168 * mm, height=100 * mm))
        story.append(Paragraph("Blue: optimised route. Red dashed: great-circle
reference.",
                               small))

    # --- 3. zone profile -----------------------------------------------------
    story.append(Paragraph(f"{'3' if map_png else '2'}. Geographic zone profile", h2))
    # max(..., 1) avoids a division by zero for an infeasible plan without legs.
    total_legs = max(sum(plan.zone_profile.values()), 1)
    rows = [["Zone", "Description", "Weight", "Waypoints", "Share"]]
    for name in CLASS_NAMES:
        count = plan.zone_profile.get(name, 0)
        weight = ZONE_COST[name]
        rows.append([
            name, CLASS_DESCRIPTION[name],
            "impassable" if math.isinf(weight) else f"{weight:.1f}",
            str(count), f"{100.0 * count / total_legs:.1f}%",
        ])
    story.append(table(rows, [28 * mm, 72 * mm, 22 * mm, 22 * mm, 22 * mm]))
```

```
    # --- 4. comparison ------------------------------------------------------
    if plan.baseline:
        story.append(Paragraph(f"{'4' if map_png else '3'}. Comparison with the "
                               "shortest-distance route", h2))
        b = plan.baseline
        # The optimised column shows 0 land samples by construction: the
        # search never enters land cells (interior waypoints are navigable).
        story.append(table([
            ["Criterion", "Great-circle route", "Optimised route"],
            ["Length, km", f"{b.get('distance_km', 0):,.1f}",
f"{plan.distance_km:,.1f}"],
            ["Extra length", "-", f"+{b.get('extra_distance_pct', 0):.2f} %"],
            ["Samples over land", f"{b.get('land_samples', 0)} of {b.get('samples',
0)}", "0"],
            ["Share over land", f"{100.0 * b.get('land_share', 0):.1f} %", "0.0 %"],
            ["Navigable", "Yes" if b.get("is_navigable") else "No",
             "No - see the warning" if getattr(plan, "warning", "") else "Yes"],
        ], [45 * mm, 60 * mm, 60 * mm]))
        story.append(Spacer(1, 4))
        story.append(Paragraph(f"<b>Conclusion.</b> {b.get('verdict', '')} "
                               f"The optimised route accepts "
                               f"{b.get('extra_distance_pct', 0):.2f}% additional
distance in "
                               f"exchange for a passage that remains inside navigable
zones "
                               f"over its whole length.", body))

    # --- 5. model -----------------------------------------------------------
    if model_summary:
        story.append(Paragraph("Classification model", h2))
        story.append(table([
            ["Property", "Value"],
            ["Architecture", str(model_summary.get("architecture"))],
            ["Input features", str(model_summary.get("n_features"))],
            ["Trainable parameters", f"{model_summary.get('parameters', 0):,}"],
            ["Test accuracy", _fmt(model_summary.get("accuracy"), "{:.4f}")],
            ["Macro F1", _fmt(model_summary.get("macro_f1"), "{:.4f}")],
            ["Trained", str(model_summary.get("trained_utc"))],
        ], [58 * mm, 110 * mm]))

    # --- 6. waypoint table --------------------------------------------------
    story.append(PageBreak())
    story.append(Paragraph("Appendix. Waypoint list", h2))
    rows = [["#", "Latitude", "Longitude", "Zone", "Cum. km", "Bearing"]]
    # Thin long routes to every step-th waypoint (about 120 rows, 3-4 pages).
    # 航路点过多时按步长抽样，约 120 行。
    step = max(1, len(plan.legs) // 120)
    for i in range(0, len(plan.legs), step):
        leg = plan.legs[i]
        rows.append([str(i), f"{leg.latitude:.4f}", f"{leg.longitude:.4f}",
leg.class_code,
                     f"{leg.cumulative_km:.1f}", f"{leg.bearing_deg:.0f}°"])
    # Always list the destination, even if thinning skipped the last index.
    if len(plan.legs) and (len(plan.legs) - 1) % step:
        leg = plan.legs[-1]
        rows.append([str(len(plan.legs) - 1), f"{leg.latitude:.4f}",
f"{leg.longitude:.4f}",
                     leg.class_code, f"{leg.cumulative_km:.1f}",
f"{leg.bearing_deg:.0f}°"])
    story.append(table(rows, [12 * mm, 30 * mm, 30 * mm, 36 * mm, 26 * mm, 24 * mm]))
    if step > 1:
        story.append(Spacer(1, 3))
        story.append(Paragraph(f"Table shows every {step}-th waypoint of "
                               f"{len(plan.legs)} in total.", small))

    # Lay out all flowables and write the file.
    document.build(story)
    LOGGER.info("PDF report written to %s", path)
    return path


# ---------------------------------------------------------------------------- #
def export_all(plan, results: Optional[Sequence] = None,
               directory: Path | str = OUTPUT_DIR,
               map_png: Optional[bytes] = None,
               model_summary: Optional[Dict] = None) -> Dict[str, str]:
    """Write GeoJSON + CSV + PDF for one plan and return the file paths.

    Files are named ``route_<route_id>.<ext>``; with ``results`` two more
```

```
    files ``route_<route_id>_points.geojson`` and ``..._points.csv`` hold the
    classified points.
    一次性导出全部格式并返回文件路径。

    Parameters
    ----------
    plan : RoutePlan
        Planning result.
    results : sequence of ClassificationResult, optional
        Classified points to export alongside the route.
    directory : Path or str, default ``config.OUTPUT_DIR``
        Output directory.
    map_png : bytes, optional
        Map image for the PDF.
    model_summary : dict, optional
        Model section of the PDF.

    Returns
    -------
    dict
        Format key (``geojson``, ``csv``, ``pdf``, ``points_geojson``,
        ``points_csv``) -> file path as string.
    """
    directory = Path(directory)
    stem = f"route_{plan.route_id}"
    paths = {
        "geojson": str(write_geojson(route_to_geojson(plan), directory /
f"{stem}.geojson")),
        "csv": str(write_csv(route_to_csv_string(plan), directory / f"{stem}.csv")),
        "pdf": str(route_to_pdf(plan, directory / f"{stem}.pdf", map_png,
model_summary)),
    }
    if results:
        paths["points_geojson"] = str(write_geojson(
            classified_points_to_geojson(results), directory /
f"{stem}_points.geojson"))
        paths["points_csv"] = str(write_csv(
            points_to_csv_string(results), directory / f"{stem}_points.csv"))
    return paths


def _coast_text(check: Dict) -> str:
    """One-line description of the coastline check for the PDF report.

    Parameters
    ----------
    check : dict
        ``RoutePlan.coast_check``.

    Returns
    -------
    str
        For example ``"passed (corrected automatically: 2 re-searches)"``.
    """
    verdict = check.get("verdict")
    if not verdict or verdict == "unchecked":
        return "not performed (land mask not installed)"
    text = {"clear": "passed - no land on the route",
            "grazing": f"passed - touches the coast for at most
{check.get('max_land_run_km', 0):.0f} km",
            "crossing": f"FAILED - crosses land for {check.get('max_land_run_km',
0):.0f} km"}[verdict]
    if check.get("initial_verdict") == "crossing" and verdict != "crossing":
        parts = []
        if check.get("corrections"):
            n = check["corrections"]
            parts.append(f"{n} re-search{'es' if n > 1 else ''}")
        if check.get("refined"):
            parts.append("finer lattice")
        text += f" (corrected automatically: {', '.join(parts)})"
    return text


def _clean(value) -> Optional[float]:
    """Round a number for JSON, mapping inf/nan/non-numbers to ``None``.

    JSON (RFC 8259) has no literal for infinity or NaN; Python would write
    the non-standard tokens ``Infinity``/``NaN``, which strict parsers reject.

    Parameters
```

```
    ----------
    value
        Any value.

    Returns
    -------
    float or None
        ``round(value, 4)`` for a finite number, else ``None``.
    """
    try:
        value = float(value)
    except (TypeError, ValueError):
        return None
    return round(value, 4) if math.isfinite(value) else None


def _fmt(value, spec: str) -> str:
    """Format a number for the PDF, or "-" when it is missing or not finite.

    Parameters
    ----------
    value
        Any value.
    spec : str
        :meth:`str.format` pattern such as ``"{:.3f}"``.

    Returns
    -------
    str
        Formatted text or ``"-"``.
    """
    try:
        value = float(value)
    except (TypeError, ValueError):
        return "-"
    return spec.format(value) if math.isfinite(value) else "-"
```

D.7 Web service

web/__init__.py

```
"""Web layer: FastAPI application and static single-page client.

Web 层：FastAPI 服务与单页前端。

Contents of the package
-----------------------
``app.py``
    FastAPI application: REST endpoints (classification, upload, routing,
    benchmark, stored routes, export, offline tiles) and the ``/ws/plan``
    WebSocket that streams planning progress.
``tiles.py``
    Offline map tiles rendered by the zone classifier (used when no internet
    tile server is reachable).
``static/``
    The browser client: ``index.html``, ``css/app.css``, ``js/api.js`` (HTTP and
    WebSocket client), ``js/app.js`` (Leaflet map and UI logic),
    ``js/charts.js`` (canvas charts) and ``vendor/`` (bundled Leaflet so the
    page also works offline).

Exports
-------
``app``
    Ready-made application instance, e.g. ``uvicorn maritime_route.web:app``.
``create_app``
    Factory that builds a fresh application instance.
"""
from .app import app, create_app

__all__ = ["app", "create_app"]
```

web/app.py – REST endpoints and the WebSocket channel

```
"""FastAPI application: REST endpoints + WebSocket for real-time interaction.

Web 服务层：REST 接口 + WebSocket 实时交互。
The WebSocket channel streams planning progress (grid construction, neural
network classification, graph search) to the browser so that the user sees the
route being built instead of waiting for a single blocking response.

Place in the application
------------------------
This module is the outermost layer of the system. It does not implement any
navigation logic itself; it validates HTTP/WebSocket input, forwards the work
to the lower layers and serialises their results to JSON:

* ``maritime_route.model.inference.ZoneClassifierService`` - the trained
  PyTorch zone classifier (coordinates -> navigation zone probabilities);
* ``maritime_route.routing.planner.RoutePlanner`` - cost-map construction and
  A* / Dijkstra / genetic / great-circle route search;
* ``maritime_route.storage.repository.RouteRepository`` - SQLite persistence of
  upload sessions, classified points and planned routes;
* ``maritime_route.export.exporters`` - GeoJSON / CSV / PDF export;
* ``maritime_route.web.tiles`` - offline map tiles rendered from the model.

Main objects
------------
``PlanRequest`` / ``ClassifyRequest``
    Pydantic models that validate request bodies (FastAPI answers 422 when
    validation fails).
``AppState`` / ``STATE``
    Process-wide singleton with the lazily loaded model, the planner, the
    repository and two small in-memory caches (recent plans, recent uploads).
``create_app()``
    Application factory that registers every route; ``app`` at the bottom of
    the module is the instance served by Uvicorn (``maritime_route.web:app``).
``_ws_plan`` / ``_ws_classify``
    Helpers that implement the WebSocket actions.

Concurrency model
-----------------
All endpoints are ``async``. CPU-bound work (neural-network inference, graph
search, file parsing, SQLite writes, PDF rendering) is moved to the default
thread pool with :func:`asyncio.to_thread` so that the event loop keeps serving
other requests and, for the WebSocket, keeps forwarding progress frames while
a route is being computed.
所有耗时计算都通过 asyncio.to_thread 放到线程池执行，避免阻塞事件循环。
"""
from __future__ import annotations

import asyncio
import json
import logging
import time
from pathlib import Path
from typing import Any, Dict, List, Optional

import numpy as np
import pandas as pd
from fastapi import FastAPI, File, HTTPException, Query, UploadFile, WebSocket,
WebSocketDisconnect
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import FileResponse, HTMLResponse, JSONResponse, Response
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel, Field, field_validator

from ..config import (
    APP_AUTHOR,
    APP_VERSION,
    CLASS_COLOR,
    CLASS_COLOR_DARK,
    CLASS_DESCRIPTION,
    CLASS_NAMES,
    OUTPUT_DIR,
    ROUTING,
    SUPPORTED_ALGORITHMS,
    SERVER,
    ZONE_COST,
)
from ..data.ais_loader import DataLoadError, load_any, validate
```

```
from ..export.exporters import (
    classified_points_to_geojson, points_to_csv_string, route_to_csv_string,
    route_to_geojson, route_to_pdf,
)
from ..model.inference import ModelNotTrainedError, ZoneClassifierService
from ..routing.planner import RoutePlanner, benchmark
from ..storage.repository import RouteRepository
from .tiles import cache_info as tile_cache_info, render_tile

LOGGER = logging.getLogger(__name__)
#: Directory with index.html, css/, js/ and vendor/ (served under /static).
#: 前端静态文件目录。
STATIC_DIR = Path(__file__).parent / "static"


# ---------------------------------------------------------------------------- #
# Request models                                                               #
# ---------------------------------------------------------------------------- #
class PlanRequest(BaseModel):
    """Body of ``POST /api/route``. 航线规划请求体。

    Also used by ``POST /api/benchmark`` and, field by field, by the
    ``"plan"`` action of the ``/ws/plan`` WebSocket. Range checks are declared
    with ``Field`` constraints, so an out-of-range value is rejected by
    FastAPI with HTTP 422 before the endpoint body runs.

    Attributes
    ----------
    start_lat, start_lon : float
        Departure point in decimal degrees (WGS-84), latitude in [-90, 90],
        longitude in [-180, 180].
    end_lat, end_lon : float
        Destination point, same ranges.
    algorithm : str
        One of ``SUPPORTED_ALGORITHMS`` (``"astar"``, ``"dijkstra"``,
        ``"genetic"``, ``"great_circle"``); case-insensitive, normalised to
        lower case by the validator.
    resolution_deg : float
        Cell size of the routing lattice in degrees, 0.01 < value <= 3.0.
        The planner may refine it automatically when a strait is too narrow.
    speed_knots : float
        Vessel speed used only for the travel-time estimate, 0.1 < value <= 60.
    simplify_km : float
        Douglas-Peucker tolerance for the final poly-line; 0 disables
        simplification.
    persist : bool
        If true the finished plan is also written to the SQLite database.
    """

    start_lat: float = Field(..., ge=-90, le=90)
    start_lon: float = Field(..., ge=-180, le=180)
    end_lat: float = Field(..., ge=-90, le=90)
    end_lon: float = Field(..., ge=-180, le=180)
    algorithm: str = Field("astar")
    resolution_deg: float = Field(ROUTING.grid_resolution_deg, gt=0.01, le=3.0)
    speed_knots: float = Field(14.0, gt=0.1, le=60.0)
    simplify_km: float = Field(0.0, ge=0.0, le=200.0)
    persist: bool = True

    @field_validator("algorithm")
    @classmethod
    def _check_algorithm(cls, value: str) -> str:
        """Normalise the algorithm name and reject unknown ones.

        Parameters
        ----------
        value : str
            Algorithm name as sent by the client.

        Returns
        -------
        str
            The lower-case name, guaranteed to be in ``SUPPORTED_ALGORITHMS``.

        Raises
        ------
        ValueError
            If the name is not supported (FastAPI turns it into HTTP 422).
        """
        value = value.lower()
```

```
        if value not in SUPPORTED_ALGORITHMS:
            raise ValueError("algorithm must be one of: " + ",
".join(SUPPORTED_ALGORITHMS))
        return value


class ClassifyRequest(BaseModel):
    """Body of ``POST /api/classify``. 坐标分类请求体。

    Attributes
    ----------
    points : list of [float, float]
        Between 1 and 50 000 ``[latitude, longitude]`` pairs. The upper bound
        keeps a single request from occupying the model for too long.
    """

    points: List[List[float]] = Field(..., min_length=1, max_length=50000)

    @field_validator("points")
    @classmethod
    def _check_points(cls, value):
        """Check that every item is a valid ``[lat, lon]`` pair.

        Parameters
        ----------
        value : list of list of float
            Raw point list from the request body.

        Returns
        -------
        list of list of float
            The same list, unchanged, if all points are valid.

        Raises
        ------
        ValueError
            If a point does not have exactly two numbers or lies outside the
            valid latitude/longitude range (reported to the client as 422).
        """
        for p in value:
            if len(p) != 2:
                raise ValueError("each point must be [latitude, longitude]")
            if not (-90 <= p[0] <= 90 and -180 <= p[1] <= 180):
                raise ValueError(f"coordinate out of range: {p}")
        return value


# ---------------------------------------------------------------------------- #
# Application state                                                            #
# ---------------------------------------------------------------------------- #
class AppState:
    """Process-wide singletons. 应用级共享状态。

    One instance (``STATE``) is created at import time and shared by all
    requests of the process. The model is *not* loaded here: loading is
    deferred to :meth:`ensure_model` so that the server can start (and serve
    the UI and ``/api/health``) even before a model has been trained.
    模型延迟加载，未训练模型时服务器依然可以启动。

    Attributes
    ----------
    classifier : ZoneClassifierService or None
        Trained zone classifier, ``None`` until the first successful
        :meth:`ensure_model` call.
    planner : RoutePlanner or None
        Route planner bound to ``classifier``.
    repository : RouteRepository
        SQLite repository (sessions, points, routes, waypoints).
    plans : dict of str to RoutePlan
        The most recent plans (at most 64) keyed by ``route_id``; needed by
        ``/api/export/{route_id}.{fmt}`` because exporters work on the full
        in-memory ``RoutePlan`` object, not on the database row.
    uploads : dict of str to pandas.DataFrame
        Uploaded AIS frames keyed by ``"upload-<session_id>"``, used by
        ``/api/export/points/{token}.{fmt}``. This cache is not trimmed; it
        lives as long as the process.
    started : float
        UNIX time when the state was created, used for ``uptime_s``.
    """
```

```
    def __init__(self) -> None:
        """Create empty state; the model is loaded lazily later."""
        self.classifier: Optional[ZoneClassifierService] = None
        self.planner: Optional[RoutePlanner] = None
        self.repository = RouteRepository()
        self.plans: Dict[str, Any] = {}          # route_id -> RoutePlan
        self.uploads: Dict[str, pd.DataFrame] = {}
        self.started = time.time()

    def ensure_model(self) -> RoutePlanner:
        """Load the classifier and planner on first use and return the planner.

        ``ZoneClassifierService.instance()`` is itself a thread-safe
        singleton, so concurrent first calls end up sharing one model object.

        Returns
        -------
        RoutePlanner
            The shared planner instance.

        Raises
        ------
        HTTPException
            Status 503 (Service Unavailable) if no trained model files exist;
            the detail text tells the user to run the training script.
        """
        if self.planner is None:
            try:
                self.classifier = ZoneClassifierService.instance()
            except ModelNotTrainedError as exc:
                # 503 = the service is up but its model is missing. 模型缺失返回 503。
                raise HTTPException(status_code=503, detail=str(exc)) from exc
            self.planner = RoutePlanner(self.classifier)
        return self.planner

    def remember(self, plan) -> None:
        """Keep a plan in memory so that it can be exported later.

        Parameters
        ----------
        plan : RoutePlan
            Finished plan; stored under ``plan.route_id``.

        Notes
        -----
        Python dictionaries keep insertion order, so the first keys are the
        oldest plans. When more than 64 plans are stored, everything except
        the last 64 keys is dropped (a first-in-first-out trim).
        字典保持插入顺序，超过 64 条时删除最早的记录。
        """
        self.plans[plan.route_id] = plan
        if len(self.plans) > 64:                        # simple LRU trim
            for key in list(self.plans)[:-64]:
                self.plans.pop(key, None)


#: The single shared state object used by every endpoint. 全局唯一状态对象。
STATE = AppState()


def create_app() -> FastAPI:
    """Application factory. 创建 FastAPI 应用。

    Builds a :class:`fastapi.FastAPI` instance, enables CORS, mounts the static
    single-page client and registers all REST and WebSocket endpoints as
    closures. All endpoints share the module-level ``STATE`` object.

    Endpoint overview
    -----------------
    ======  ==================================  ================================
    Method  Path                                Purpose
    ======  ==================================  ================================
    GET     /                                   single-page client (index.html)
    GET     /static/...                         CSS, JS, vendor libraries
    GET     /favicon.ico                        inline SVG icon
    GET     /api/health                         liveness + model availability
    GET     /api/zones                          zone classes, colours, costs
    GET     /api/model                          model summary and metrics
    GET     /api/statistics                     database statistics
    GET     /api/tiles/zones/{z}/{x}/{y}.png    offline chart tile
```

```
GET     /api/tiles/info                     tile cache size
POST    /api/classify                       classify coordinates
POST    /api/upload                         upload + classify AIS file
POST    /api/route                          plan one route
POST    /api/benchmark                      compare all algorithms
GET     /api/routes                         list stored routes
GET     /api/routes/{route_uid}             one stored route
DELETE  /api/routes/{route_uid}             delete stored route
GET     /api/export/{route_id}.{fmt}        export route (geojson/csv/pdf)
GET     /api/export/points/{token}.{fmt}    export upload (geojson/csv)
WS      /ws/plan                            live planning with progress
======  ==================================  ==============================

Returns
-------
FastAPI
    Fully configured application.
"""
app = FastAPI(title=SERVER.title, version=SERVER.version,
              description="Neural-network based maritime route planning service")
# CORS is fully open: the API is meant to be called from the bundled page
# or from local tools/notebooks, and it carries no user credentials.
# 允许任意来源跨域访问（接口不涉及用户凭据）。
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"],
                   allow_headers=["*"])

# ---------------------------------------------------------------------- #
# Static UI                                                              #
# ---------------------------------------------------------------------- #
app.mount("/static", StaticFiles(directory=str(STATIC_DIR)), name="static")

@app.get("/", response_class=HTMLResponse, include_in_schema=False)
async def index() -> HTMLResponse:
    """Serve the single-page client.

    ``GET /`` - returns ``static/index.html``. The file is read on every
    request (it is small), so edits to the page are visible without a
    server restart. Hidden from the OpenAPI schema.

    Returns
    -------
    HTMLResponse
        The HTML page, status 200.
    """
    return HTMLResponse((STATIC_DIR / "index.html").read_text(encoding="utf-8"))

# ---------------------------------------------------------------------- #
# Metadata                                                               #
# ---------------------------------------------------------------------- #
@app.get("/favicon.ico", include_in_schema=False)
async def favicon() -> Response:
    """Inline SVG favicon so that no external asset is required.

    ``GET /favicon.ico`` - returns a 24x24 SVG (anchor symbol on a dark
    square) with a one-week ``Cache-Control`` header. Browsers request this
    path automatically; answering it avoids 404 lines in the log.

    Returns
    -------
    Response
        ``image/svg+xml`` body.
    """
    svg = (
        '<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 24 24">'
        '<rect width="24" height="24" rx="5" fill="#0e1621"/>'
        '<path d="M12 4v11M12 6h4l-4 3M5 17c2 2 4 2 4 2h6s2 0 4-2l-2 4H7z" '
        'fill="none" stroke="#4da3ff" stroke-width="1.5" '
        'stroke-linecap="round" stroke-linejoin="round"/></svg>'
    )
    return Response(svg, media_type="image/svg+xml",
                    headers={"Cache-Control": "public, max-age=604800"})

@app.get("/api/health")
async def health() -> Dict:
    """Report whether the service and its model are available.

    ``GET /api/health``. The call also triggers lazy model loading, so the
    first health check after start-up may take a moment. It never fails
    with an error status: a missing model is reported in the body.
```

```
        Returns
        -------
        dict
            ``{"status": "ok" | "model_missing", "detail": str,
            "version": str, "uptime_s": float}``; ``detail`` is ``"ready"``
            or the reason why the model could not be loaded.
        """
        ready = True
        detail = "ready"
        try:
            STATE.ensure_model()
        except HTTPException as exc:
            ready, detail = False, str(exc.detail)
        return {"status": "ok" if ready else "model_missing", "detail": detail,
                "version": APP_VERSION, "uptime_s": round(time.time() - STATE.started,
1)}

    @app.get("/api/zones")
    async def zones() -> Dict:
        """Describe the navigation-zone classes predicted by the model.

        ``GET /api/zones``. Does not need the model; the data comes from
        ``config``. The client uses it to build the map legend.

        Returns
        -------
        dict
            ``{"classes": [{"code", "index", "description", "colour", "cost",
            "navigable"}, ...]}`` in model output order. ``cost`` is ``None``
            (JSON ``null``) for impassable zones because JSON has no infinity;
            ``navigable`` is ``False`` for the same zones.
        """
        return {
            "classes": [
                {
                    "code": name,
                    "index": i,
                    "description": CLASS_DESCRIPTION[name],
                    "colour": CLASS_COLOR_DARK[name],
                    # float("inf") is not valid JSON, so impassable zones get null.
                    # JSON 不支持无穷大，不可航行区的代价输出为 null。
                    "cost": None if ZONE_COST[name] == float("inf") else
ZONE_COST[name],
                    "navigable": ZONE_COST[name] != float("inf"),
                }
                for i, name in enumerate(CLASS_NAMES)
            ]
        }

    @app.get("/api/model")
    async def model_info() -> Dict:
        """Return the model summary and the metadata saved at training time.

        ``GET /api/model``.

        Returns
        -------
        dict
            ``{"summary", "metrics", "hyperparameters", "dataset",
            "history"}``. ``summary`` comes from
            ``ZoneClassifierService.summary()`` (architecture, parameter
            count); the other keys are copied from the metadata JSON written by
            the trainer (``history`` holds per-epoch loss/accuracy curves used
            by the charts on the "Model" tab). Missing keys become ``{}``.

        Raises
        ------
        HTTPException
            503 if the model has not been trained.
        """
        STATE.ensure_model()
        metadata = STATE.classifier.metadata
        return {
            "summary": STATE.classifier.summary(),
            "metrics": metadata.get("metrics", {}),
            "hyperparameters": metadata.get("hyperparameters", {}),
            "dataset": metadata.get("dataset", {}),
            "history": metadata.get("history", {}),
        }
```

```
@app.get("/api/statistics")
async def statistics() -> Dict:
    """Return aggregate statistics of the SQLite database.

    ``GET /api/statistics``. Does not need the model.

    Returns
    -------
    dict
        Output of ``RouteRepository.statistics()``: counts of sessions,
        points, routes and waypoints, the class histogram of stored
        points, per-algorithm averages (``by_algorithm``) and the database
        file name and size.
    """
    return STATE.repository.statistics()

@app.get("/api/tiles/zones/{z}/{x}/{y}.png", include_in_schema=False)
async def zone_tile(z: int, x: int, y: int):
    """On-board chart tile: the network's classification rendered as PNG.

    离线海图瓦片接口，使应用在无网络环境下仍可显示底图。

    ``GET /api/tiles/zones/{z}/{x}/{y}.png`` - standard slippy-map
    (XYZ, Web Mercator) addressing, as used by Leaflet ``L.tileLayer``.

    Parameters
    ----------
    z : int
        Zoom level, 0..10. Higher levels are refused because the model
        works on a coarse lattice and deeper tiles would add no detail.
    x, y : int
        Tile column and row, each in ``[0, 2**z)``; ``y = 0`` is the
        northern edge.

    Returns
    -------
    Response
        ``image/png`` 256x256 RGBA tile with a one-day ``Cache-Control``.

    Raises
    ------
    HTTPException
        503 if the model is missing, 400 for a zoom outside 0..10, 404 for
        ``x``/``y`` outside the tile matrix of that zoom level.
    """
    STATE.ensure_model()
    if not (0 <= z <= 10):
        raise HTTPException(400, "Zoom level out of range for the on-board chart")
    # At zoom z the world is a 2**z x 2**z matrix of tiles. 第 z 级共有 2^z × 2^z 张瓦
片。
    limit = 2 ** z
    if not (0 <= x < limit and 0 <= y < limit):
        raise HTTPException(404, "Tile out of range")
    # Rendering runs inference + PNG encoding: done in a worker thread so the
    # event loop can serve the other tiles Leaflet requests in parallel.
    # 渲染在线程池中执行，不阻塞其它并发的瓦片请求。
    payload = await asyncio.to_thread(render_tile, STATE.classifier, z, x, y)
    return Response(payload, media_type="image/png",
                    headers={"Cache-Control": "public, max-age=86400"})

@app.get("/api/tiles/info")
async def tile_info() -> Dict:
    """Report the state of the in-memory tile cache.

    ``GET /api/tiles/info``.

    Returns
    -------
    dict
        ``{"tiles_cached": int, "limit": int}``.
    """
    return tile_cache_info()

# ---------------------------------------------------------------- #
# Classification                                                   #
# ---------------------------------------------------------------- #
@app.post("/api/classify")
async def classify(request: ClassifyRequest) -> Dict:
    """Classify arbitrary coordinates with the neural network.
```

```
        ``POST /api/classify``

        Parameters
        ----------
        request : ClassifyRequest
            JSON body ``{"points": [[lat, lon], ...]}`` (1..50 000 points).

        Returns
        -------
        dict
            ``{"n": int, "histogram": {class_code: count},
            "results": [ClassificationResult.to_dict(), ...]}``; each result
            carries ``latitude``, ``longitude``, ``class_index``,
            ``class_code``, ``confidence``, ``probabilities`` and ``depth_m``.

        Raises
        ------
        HTTPException
            503 if the model is missing; 422 for an invalid body.
        """
        STATE.ensure_model()
        array = np.asarray(request.points, dtype=float)
        results = await asyncio.to_thread(
            STATE.classifier.classify_coordinates, array[:, 0], array[:, 1]
        )
        # Start from zero for every class so that absent classes still appear.
        # 所有类别初始化为 0，保证未出现的类别也在直方图中。
        histogram: Dict[str, int] = {c: 0 for c in CLASS_NAMES}
        for r in results:
            histogram[r.class_code] += 1
        return {"n": len(results), "histogram": histogram,
                "results": [r.to_dict() for r in results]}

    @app.post("/api/upload")
    async def upload(file: UploadFile = File(...)) -> Dict:
        """Upload an AIS file (GeoJSON/CSV/JSON) and classify its points.

        上传 AIS 数据文件并即时分类。

        ``POST /api/upload`` - ``multipart/form-data`` with one field ``file``.

        Processing steps: size check -> parse by file type (``load_any``) ->
        clean and validate (``validate``) -> keep the first 20 000 rows ->
        classify -> store a new session and its points in SQLite -> cache the
        frame in ``STATE.uploads`` for later export.

        Parameters
        ----------
        file : UploadFile
            The uploaded file; the extension of ``file.filename`` selects the
            parser.

        Returns
        -------
        dict
            ``token`` (``"upload-<session_id>"``, used by the points export
            endpoint), ``session_id``, ``report`` (``LoadReport.to_dict()``:
            ``source``, ``n_features``, ``n_valid``, ``n_dropped``,
            ``labelled``, ``columns``, ``warnings``), ``classified`` (number of
classified
            points), ``histogram`` (count per class), ``accuracy_vs_labels``
            (``{"n_labelled", "agreement"}`` or ``None`` when the file has no
            ``class_code`` column) and ``points`` (at most about 6 000 evenly
            thinned classified points for drawing on the map).

        Raises
        ------
        HTTPException
            503 if the model is missing, 413 if the file exceeds 80 MB, 400 if
            the file cannot be parsed or contains no valid AIS rows.
        """
        STATE.ensure_model()
        # The whole body is read into memory, hence the explicit size limit.
        # 文件整体读入内存，因此设置 80 MB 上限。
        content = await file.read()
        if len(content) > 80 * 1024 * 1024:
            raise HTTPException(413, "File too large (limit 80 MB)")
        try:
            frame = await asyncio.to_thread(load_any, file.filename or "upload.json",
content)
```

```
            frame, report = validate(frame, source=file.filename or "upload")
        except DataLoadError as exc:
            raise HTTPException(400, str(exc)) from exc

        # Only the first 20 000 rows are classified and stored interactively;
        # larger data sets belong to the offline preparation scripts.
        # 交互模式只处理前 20000 行，大数据集应使用离线脚本。
        preview = frame.head(20000).reset_index(drop=True)
        results = await asyncio.to_thread(STATE.classifier.classify_frame, preview)
        session_id = STATE.repository.create_session(
            name=file.filename or "upload", source="web-upload", notes="interactive
upload")
        await asyncio.to_thread(STATE.repository.save_points, session_id, results)
        token = f"upload-{session_id}"
        STATE.uploads[token] = preview
        # Keep only the 16 most recent uploads in memory (dicts preserve insertion
        # order), so a long-running server does not grow without bound.
        # 内存中只保留最近 16 次上传，避免长时间运行时内存无限增长。
        while len(STATE.uploads) > 16:
            STATE.uploads.pop(next(iter(STATE.uploads)))

        histogram: Dict[str, int] = {c: 0 for c in CLASS_NAMES}
        for r in results:
            histogram[r.class_code] += 1
        # Thin the returned points to roughly 6 000 so the browser stays fluid;
        # the histogram above is still computed on all points.
        # 返回给前端的点抽稀到约 6000 个，直方图仍基于全部点。
        step = max(1, len(results) // 6000)
        return {
            "token": token,
            "session_id": session_id,
            "report": report.to_dict(),
            "classified": len(results),
            "histogram": histogram,
            "accuracy_vs_labels": _label_agreement(preview, results),
            "points": [r.to_dict() for r in results[::step]],
        }

    # ------------------------------------------------------------------ #
    # Routing                                                            #
    # ------------------------------------------------------------------ #
    @app.post("/api/route")
    async def plan_route(request: PlanRequest) -> Dict:
        """Plan one route in a single blocking request (no progress frames).

        ``POST /api/route`` - the non-streaming alternative to ``/ws/plan``;
        useful for scripts and tests.

        Parameters
        ----------
        request : PlanRequest
            JSON body with start/end coordinates and planning options.

        Returns
        -------
        dict
            ``RoutePlan.to_dict()``: ``route_id``, ``algorithm``, ``start``,
            ``end``, ``waypoints`` ([[lat, lon], ...]), ``legs`` (per-waypoint
            zone, cost, cumulative distance, bearing), ``total_cost``,
            ``distance_km``, ``great_circle_km``, ``detour_ratio``,
            ``estimated_hours``, ``nodes_expanded``, ``runtime_s``, ``grid``,
            ``zone_profile``, ``baseline``, ``cost_map_stats``,
            ``created_utc``, ``speed_knots``, ``interior_land_waypoints``,
            ``feasible``, ``message``, ``coast_check`` (independent coastline
            check: verdict, longest land stretch, automatic corrections, land
            points) and ``warning`` (non-empty if the route still crosses land). An
impossible voyage is *not* an
            HTTP error: it is returned with ``feasible = false`` and a
            ``message`` explaining why.

        Raises
        ------
        HTTPException
            503 if the model is missing; 422 for an invalid body.
        """
        planner = STATE.ensure_model()
        plan, _ = await asyncio.to_thread(
            planner.plan,
            (request.start_lat, request.start_lon),
            (request.end_lat, request.end_lon),
```

```
        request.algorithm, request.resolution_deg, request.speed_knots,
        request.simplify_km,
    )
    STATE.remember(plan)
    if request.persist:
        await asyncio.to_thread(STATE.repository.save_route, plan)
    return plan.to_dict()

@app.post("/api/benchmark")
async def run_benchmark(request: PlanRequest) -> Dict:
    """Run every algorithm on the same cost map and compare them.

    ``POST /api/benchmark`` - uses only the coordinates and
    ``resolution_deg`` of the body; ``algorithm``, ``speed_knots``,
    ``simplify_km`` and ``persist`` are ignored and nothing is stored.

    Parameters
    ----------
    request : PlanRequest
        Same body as ``/api/route``.

    Returns
    -------
    dict
        Output of :func:`maritime_route.routing.planner.benchmark`: one
        row per algorithm (``feasible``, ``distance_km``, ``total_cost``,
        ``nodes_expanded``, ``runtime_s``, ``detour_ratio``, ``note``)
        plus derived comparisons such as the A*/Dijkstra node-expansion
        speed-up and the genetic algorithm's gap to the optimum.

    Raises
    ------
    HTTPException
        503 if the model is missing; 422 for an invalid body.
    """
    planner = STATE.ensure_model()
    return await asyncio.to_thread(
        benchmark, planner, (request.start_lat, request.start_lon),
        (request.end_lat, request.end_lon), request.resolution_deg,
    )

@app.get("/api/routes")
async def list_routes(limit: int = Query(30, ge=1, le=200)) -> Dict:
    """List the most recent stored routes.

    ``GET /api/routes?limit=30``

    Parameters
    ----------
    limit : int
        Maximum number of routes, 1..200 (default 30); other values give
        HTTP 422.

    Returns
    -------
    dict
        ``{"routes": [...]}`` - summary rows from
        ``RouteRepository.list_routes`` (newest first).
    """
    return {"routes": STATE.repository.list_routes(limit)}

@app.get("/api/routes/{route_uid}")
async def get_route(route_uid: str) -> Dict:
    """Fetch one stored route with its waypoints.

    ``GET /api/routes/{route_uid}``

    Parameters
    ----------
    route_uid : str
        The ``route_id`` assigned by the planner.

    Returns
    -------
    dict
        The stored route as returned by ``RouteRepository.get_route``.

    Raises
    ------
    HTTPException
```

```
            404 if no route with this id is stored.
        """
        route = STATE.repository.get_route(route_uid)
        if route is None:
            raise HTTPException(404, "Route not found")
        return route

    @app.delete("/api/routes/{route_uid}")
    async def delete_route(route_uid: str) -> Dict:
        """Delete a stored route and forget its in-memory copy.

        ``DELETE /api/routes/{route_uid}``

        Parameters
        ----------
        route_uid : str
            The ``route_id`` to delete.

        Returns
        -------
        dict
            ``{"deleted": route_uid}``.

        Raises
        ------
        HTTPException
            404 if the route does not exist in the database.
        """
        if not STATE.repository.delete_route(route_uid):
            raise HTTPException(404, "Route not found")
        # Also drop it from the export cache so it cannot be exported any more.
        # 同时从内存缓存移除，之后不能再导出。
        STATE.plans.pop(route_uid, None)
        return {"deleted": route_uid}

    # ---------------------------------------------------------------------- #
    # Export                                                                 #
    # ---------------------------------------------------------------------- #
    @app.get("/api/export/{route_id}.{fmt}")
    async def export_route(route_id: str, fmt: str):
        """Download a planned route as GeoJSON, CSV or PDF. 导出航线。

        ``GET /api/export/{route_id}.{fmt}`` - all formats are sent with a
        ``Content-Disposition: attachment`` header so the browser saves them.

        Parameters
        ----------
        route_id : str
            Id of a plan still held in ``STATE.plans`` (the last 64 plans of
            this server process). Plans only present in the database cannot be
            exported, because the exporters need the full ``RoutePlan``.
        fmt : str
            ``geojson``, ``csv`` or ``pdf`` (case-insensitive).

        Returns
        -------
        Response or FileResponse
            ``application/geo+json``, ``text/csv`` or ``application/pdf``.
            The PDF is first written to ``OUTPUT_DIR/route_<id>.pdf`` and then
            streamed from disk.

        Raises
        ------
        HTTPException
            404 if the plan is not in memory, 400 for an unsupported format.
        """
        plan = STATE.plans.get(route_id)
        if plan is None:
            raise HTTPException(404, "Route not in memory; re-plan it before exporting")
        fmt = fmt.lower()
        if fmt == "geojson":
            payload = json.dumps(route_to_geojson(plan), indent=2, ensure_ascii=False)
            return Response(payload, media_type="application/geo+json", headers={
                "Content-Disposition": f'attachment;
filename="route_{route_id}.geojson"'})
        if fmt == "csv":
            return Response(route_to_csv_string(plan), media_type="text/csv", headers={
                "Content-Disposition": f'attachment; filename="route_{route_id}.csv"'})
        if fmt == "pdf":
            path = OUTPUT_DIR / f"route_{route_id}.pdf"
```

```
            # The model summary is printed in the PDF report when available.
            # PDF 报告中附带模型摘要（若模型已加载）。
            summary = STATE.classifier.summary() if STATE.classifier else None
            await asyncio.to_thread(route_to_pdf, plan, path, None, summary)
            return FileResponse(path, media_type="application/pdf",
                                filename=f"route_{route_id}.pdf")
        raise HTTPException(400, "Supported formats: geojson, csv, pdf")

    @app.get("/api/export/points/{token}.{fmt}")
    async def export_points(token: str, fmt: str):
        """Download the classified points of an earlier upload.

        ``GET /api/export/points/{token}.{fmt}``

        Parameters
        ----------
        token : str
            ``token`` returned by ``POST /api/upload`` (``"upload-<id>"``).
        fmt : str
            ``csv`` or ``geojson`` (case-insensitive).

        Returns
        -------
        Response
            ``text/csv`` or ``application/geo+json`` attachment with every
            stored point (not the thinned map sample) and its predicted class.

        Raises
        ------
        HTTPException
            404 if the token is unknown to this process, 503 if the model is
            missing, 400 for an unsupported format.

        Notes
        -----
        The points are classified again rather than read back from SQLite;
        the model is deterministic, so the output matches the upload result.
        重新分类而非读取数据库，模型是确定性的，结果一致。
        """
        frame = STATE.uploads.get(token)
        if frame is None:
            raise HTTPException(404, "Upload not found")
        STATE.ensure_model()
        results = await asyncio.to_thread(STATE.classifier.classify_frame, frame)
        if fmt.lower() == "csv":
            return Response(points_to_csv_string(results), media_type="text/csv",
headers={
                "Content-Disposition": f'attachment; filename="{token}_points.csv"'})
        if fmt.lower() == "geojson":
            payload = json.dumps(classified_points_to_geojson(results), indent=2)
            return Response(payload, media_type="application/geo+json", headers={
                "Content-Disposition": f'attachment;
filename="{token}_points.geojson"'})
        raise HTTPException(400, "Supported formats: geojson, csv")

    # ------------------------------------------------------------------ #
    # WebSocket - real-time planning                                     #
    # ------------------------------------------------------------------ #
    @app.websocket("/ws/plan")
    async def ws_plan(websocket: WebSocket) -> None:
        """Bidirectional channel streaming live planning progress.

        实时规划通道：客户端发送请求，服务端持续推送进度与结果。

        ``WS /ws/plan`` - all messages are JSON text frames. One connection
        can serve any number of requests; requests are processed one after
        another (a new message is read only after the previous job finished).

        Protocol
        --------
        Server -> client, immediately after the connection is accepted::

            {"type": "hello", "version": str,
             "zones": [{"code": str, "colour": "#rrggbb"}, ...]}

        Client -> server messages are objects with an ``action`` key
        (``"plan"`` when the key is missing):

        ``{"action": "ping"}``
            Answered with ``{"type": "pong", "t": <server UNIX time>}``.
```

```
            Used by the client as a keep-alive.
        ``{"action": "classify", "points": [[lat, lon], ...]}``
            Answered with ``{"type": "classified", "results": [...]}`` (items
            as in ``POST /api/classify``) or an ``error`` frame.
        ``{"action": "plan", <PlanRequest fields>, "include_overlay": bool}``
            Starts a planning job; see :func:`_ws_plan` for the frames
            ``started`` -> ``progress``* -> ``overlay``? -> ``result``.
        any other ``action``
            Answered with ``{"type": "error", "detail": "unknown action '...'"}``;
            the connection stays open.

        Errors inside one request are reported as ``{"type": "error",
        "detail": str}`` and the loop continues. The loop ends when the client
        disconnects (``WebSocketDisconnect``, logged at INFO level). Any other
        unexpected exception is logged, reported as an ``error`` frame if the
        socket is still writable, and ends the handler.

        Parameters
        ----------
        websocket : WebSocket
            The accepted connection supplied by FastAPI.
        """
        await websocket.accept()
        # The running loop is captured here because the planner's progress
        # callback runs in a worker thread and must post frames back to it.
        # 记录事件循环，供工作线程中的进度回调跨线程投递消息。
        loop = asyncio.get_running_loop()
        try:
            await websocket.send_json({"type": "hello", "version": APP_VERSION,
                                       "zones": [{"code": c, "colour":
CLASS_COLOR_DARK[c]}
                                                 for c in CLASS_NAMES]})
            while True:
                message = await websocket.receive_json()
                action = message.get("action", "plan")

                if action == "ping":
                    await websocket.send_json({"type": "pong", "t": time.time()})
                    continue

                if action == "classify":
                    await _ws_classify(websocket, message)
                    continue

                if action != "plan":
                    await websocket.send_json({"type": "error",
                                               "detail": f"unknown action '{action}'"})
                    continue

                await _ws_plan(websocket, loop, message)

        except WebSocketDisconnect:
            LOGGER.info("WebSocket client disconnected")
        except Exception as exc:  # pragma: no cover - defensive
            LOGGER.exception("WebSocket failure")
            # The socket may already be closed; a second failure is ignored.
            # 连接可能已关闭，此时发送失败直接忽略。
            try:
                await websocket.send_json({"type": "error", "detail": str(exc)})
            except Exception:
                pass

    return app


# ---------------------------------------------------------------------------- #
# WebSocket helpers                                                            #
# ---------------------------------------------------------------------------- #
async def _ws_classify(websocket: WebSocket, message: Dict) -> None:
    """Classify a batch of coordinates pushed over the socket.

    Implements the ``"classify"`` WebSocket action.

    Parameters
    ----------
    websocket : WebSocket
        Open connection used for the reply.
    message : dict
        Client message; ``message["points"]`` must be a non-empty list of
        ``[lat, lon]`` pairs.
```

```
    Notes
    -----
    Sends exactly one frame: ``{"type": "classified", "results": [...]}`` on
    success, or ``{"type": "error", "detail": str}`` when the model is missing,
    the points have the wrong shape, lie outside the valid coordinate range, or
    there are more than 50 000 of them (the same limits as ``POST /api/classify``).
    """
    try:
        STATE.ensure_model()
    except HTTPException as exc:
        await websocket.send_json({"type": "error", "detail": str(exc.detail)})
        return
    points = np.asarray(message.get("points", []), dtype=float)
    # Accept only an (N, 2) array with N > 0. 仅接受形状为 (N, 2) 的数组。
    if points.ndim != 2 or points.shape[1] != 2 or len(points) == 0:
        await websocket.send_json({"type": "error", "detail": "points must be [[lat,
lon], ...]"})
        return
    # Same limits as the REST endpoint: at most 50 000 points, valid lat/lon range.
    # 与 REST 接口相同的限制：最多 50000 个点，经纬度在有效范围内。
    if len(points) > 50000 or not (np.all(np.abs(points[:, 0]) <= 90)
                                   and np.all(np.abs(points[:, 1]) <= 180)):
        await websocket.send_json({"type": "error",
                                   "detail": "at most 50000 points with |lat| <= 90 and
|lon| <= 180"})
        return
    results = await asyncio.to_thread(
        STATE.classifier.classify_coordinates, points[:, 0], points[:, 1])
    await websocket.send_json({"type": "classified",
                               "results": [r.to_dict() for r in results]})


async def _ws_plan(websocket: WebSocket, loop: asyncio.AbstractEventLoop, message: Dict)
-> None:
    """Run a planning job in a worker thread, streaming progress frames.

    Implements the ``"plan"`` WebSocket action.

    Parameters
    ----------
    websocket : WebSocket
        Open connection used for all frames of this job.
    loop : asyncio.AbstractEventLoop
        The event loop that owns ``websocket``; the progress callback, which
        runs in a worker thread, uses it to hand frames over safely.
    message : dict
        Client message. Keys that are fields of :class:`PlanRequest` are
        validated by it; unknown keys (``action``, ``include_overlay``) are
        filtered out first. ``include_overlay`` (default ``True``) selects
        whether the classified lattice is sent back.

    Notes
    -----
    Frames sent to the client, in order:

    1. ``{"type": "started", "algorithm": str, "resolution_deg": float}``
    2. zero or more ``{"type": "progress", "stage": str, "progress": float,
       "detail": dict}`` frames. ``stage`` is one of ``"grid"``,
       ``"classify"`` (``detail`` has ``cells_done`` / ``cells_total``),
       ``"cost_map"`` (``detail`` = cost-map statistics), ``"search"``,
       ``"astar"`` / ``"dijkstra"`` / ``"genetic"`` (search progress),
       ``"refine"`` (the lattice is refined and planning restarts) and
       ``"done"``. ``progress`` is a fraction in [0, 1] for the current stage.
    3. if ``include_overlay``: ``{"type": "overlay", "cells": [{"lat", "lon",
       "c", "p"}, ...], "step": int, "cell_size_deg": float, "grid": {...}}``
       where ``c`` is the class index and ``p`` the mean confidence of a
       down-sampled block (at most about 6 000 cells).
    4. ``{"type": "result", "plan": RoutePlan.to_dict()}``.

    Instead of this sequence a single ``{"type": "error", "detail": str}`` is
    sent when the request is invalid or the model is missing; if the planner
    raises, ``error`` is sent after the progress frames already delivered.

    Threading
    ---------
    ``planner.plan`` is synchronous and CPU-bound, so it runs in a thread via
    :func:`asyncio.to_thread`. Its progress callback executes in that thread,
    where calling ``websocket.send_json`` (a coroutine bound to the event loop)
    is not allowed. The callback therefore only puts the frame into an
```

```
    :class:`asyncio.Queue` using ``loop.call_soon_threadsafe`` (asyncio queues
    are not thread-safe themselves), and this coroutine drains the queue and
    sends the frames.
    工作线程不能直接调用 send_json，只能通过 call_soon_threadsafe 把消息放入队列，
    再由协程从队列取出并发送。
    """
    try:
        # Keep only PlanRequest fields; extra keys would otherwise be ignored
        # silently by Pydantic anyway, but filtering keeps the intent explicit.
        # 只保留 PlanRequest 字段，再交给 Pydantic 校验。
        request = PlanRequest(**{k: v for k, v in message.items()
                                 if k in PlanRequest.model_fields})
    except Exception as exc:
        await websocket.send_json({"type": "error", "detail": f"invalid request:
{exc}"})
        return

    try:
        planner = STATE.ensure_model()
    except HTTPException as exc:
        await websocket.send_json({"type": "error", "detail": str(exc.detail)})
        return

    queue: asyncio.Queue = asyncio.Queue()
    # Mutable holder (dict) so the nested callback can update the timestamp
    # without a ``nonlocal`` declaration. 用字典保存时间戳，便于闭包内修改。
    last_sent = {"t": 0.0}

    def progress(stage: str, fraction: float, detail: Dict) -> None:
        """Called from the worker thread; throttled to the configured interval.

        Parameters
        ----------
        stage : str
            Planning stage name (see :func:`_ws_plan` notes).
        fraction : float
            Progress of the stage in [0, 1].
        detail : dict
            Stage-specific data; may contain NumPy values, which are converted
            by :func:`_jsonable`.

        Notes
        -----
        At most one frame per ``SERVER.progress_interval_s`` (0.15 s) is
        forwarded, so a search that reports thousands of expansions does not
        flood the socket. ``"done"`` and ``"cost_map"`` frames are always
        forwarded because the client needs them (final state, statistics).
        按时间间隔节流，但 done 与 cost_map 两个阶段总是发送。
        """
        now = time.monotonic()
        if stage not in ("done", "cost_map") and now - last_sent["t"] <
SERVER.progress_interval_s:
            return
        last_sent["t"] = now
        # Thread-safe hand-over: put_nowait itself runs later on the loop thread.
        # 线程安全地把消息交给事件循环线程执行 put_nowait。
        loop.call_soon_threadsafe(
            queue.put_nowait,
            {"type": "progress", "stage": stage,
             "progress": round(float(fraction), 4), "detail": _jsonable(detail)},
        )

    await websocket.send_json({"type": "started", "algorithm": request.algorithm,
                               "resolution_deg": request.resolution_deg})

    def work():
        """Blocking planning call executed in the worker thread."""
        return planner.plan(
            (request.start_lat, request.start_lon), (request.end_lat, request.end_lon),
            request.algorithm, request.resolution_deg, request.speed_knots,
            request.simplify_km, progress=progress,
        )

    task = asyncio.create_task(asyncio.to_thread(work))
    # Forward progress frames while the job runs. The 0.1 s timeout lets the
    # loop re-check ``task.done()``; the ``queue.empty()`` condition makes sure
    # frames queued just before the job finished are still delivered.
    # 任务运行期间循环转发进度；超时 0.1 秒用于重新检查任务状态，
    # 任务结束后继续发送队列中剩余的消息。
    while not task.done() or not queue.empty():
```

```
            try:
                frame = await asyncio.wait_for(queue.get(), timeout=0.1)
                await websocket.send_json(frame)
            except asyncio.TimeoutError:
                continue

        try:
            # Awaiting the finished task returns its value or re-raises its exception.
            # 等待已完成的任务：返回结果或重新抛出线程中的异常。
            plan, cost_map = await task
        except Exception as exc:
            LOGGER.exception("Planning failed")
            await websocket.send_json({"type": "error", "detail": str(exc)})
            return

        STATE.remember(plan)
        if request.persist:
            await asyncio.to_thread(STATE.repository.save_route, plan)

        if message.get("include_overlay", True):
            # Down-sample the lattice to <= ~6000 cells so the browser can draw it.
            # 将分类网格降采样至约 6000 个单元供前端绘制。
            overlay = cost_map.to_overlay(max_cells=6000)
            await websocket.send_json({"type": "overlay", **overlay,
                                       "grid": cost_map.spec.to_dict()})
        await websocket.send_json({"type": "result", "plan": plan.to_dict()})


    def _jsonable(value: Any) -> Any:
        """Make NumPy scalars/arrays JSON serialisable.

        Converts recursively: dicts and lists/tuples are walked, NumPy integers
        and floats become Python ``int``/``float``, arrays become nested lists and
        positive infinity becomes ``None`` (JSON has no infinity). Other values
        are returned unchanged.

        Parameters
        ----------
        value : Any
            Progress detail or any nested structure produced by the planner.

        Returns
        -------
        Any
            A structure that ``json.dumps`` accepts.
        """
        if isinstance(value, dict):
            return {k: _jsonable(v) for k, v in value.items()}
        if isinstance(value, (list, tuple)):
            return [_jsonable(v) for v in value]
        if isinstance(value, (np.integer,)):
            return int(value)
        if isinstance(value, (np.floating,)):
            return float(value)
        if isinstance(value, np.ndarray):
            return value.tolist()
        if isinstance(value, float) and value == float("inf"):
            return None
        return value


    def _label_agreement(frame: pd.DataFrame, results: List) -> Optional[Dict]:
        """If the upload carries ground-truth labels, report the agreement rate.

        若上传数据带有真实标签，则统计模型与标签的一致率。

        Parameters
        ----------
        frame : pandas.DataFrame
            The validated upload; labels are read from an optional
            ``class_code`` column.
        results : list of ClassificationResult
            Predictions for the same rows, in the same order.

        Returns
        -------
        dict or None
            ``{"n_labelled": int, "agreement": float}`` where ``agreement`` is
            the share (0..1) of labelled rows whose predicted class equals the
            label. ``None`` if there is no ``class_code`` column or no row holds a
```

```
        known class name (unknown labels are ignored).
    """
    if "class_code" not in frame.columns:
        return None
    truth = frame["class_code"].to_numpy()
    predicted = np.array([r.class_code for r in results])
    # Rows with empty or unknown labels are excluded from the comparison.
    # 排除空标签或未知标签的行。
    mask = np.isin(truth, CLASS_NAMES)
    if not mask.any():
        return None
    return {
        "n_labelled": int(mask.sum()),
        "agreement": round(float((truth[mask] == predicted[mask]).mean()), 4),
    }


#: Module-level application instance served by Uvicorn (``maritime_route.web.app:app``).
#: 供 Uvicorn 加载的应用实例。
app = create_app()
```

web/tiles.py – rendering of the on-board chart tiles

```
"""On-board chart tiles rendered from the model itself (offline mode).

离线海图瓦片：直接用神经网络对瓦片像素分类并渲染 PNG。
A shipboard planner cannot rely on an internet tile service, so the server can
render its own slippy-map tiles: every tile is a small lattice of coordinates
classified by the network and painted with the zone palette. Tiles are cached
in an LRU so panning stays responsive.

Place in the application
------------------------
Used only by the endpoint ``GET /api/tiles/zones/{z}/{x}/{y}.png`` in
:mod:`maritime_route.web.app`; the browser shows these tiles through a Leaflet
``L.tileLayer`` ("On-board chart" base layer) when no internet map is
available.

Main objects
------------
``tile_bounds(z, x, y)``
    Geographic extent of a Web-Mercator tile.
``render_tile(classifier, z, x, y)``
    Classify a ``SAMPLE x SAMPLE`` lattice inside the tile and return PNG bytes.
``cache_info()``
    Size of the in-memory tile cache.

Tile addressing (Web Mercator / XYZ)
------------------------------------
At zoom ``z`` the world between about 85.05 deg S and 85.05 deg N is split into
``n = 2**z`` columns and ``n`` rows of 256x256-pixel tiles; ``x`` grows
eastwards from 180 deg W and ``y`` grows southwards from the northern edge.
Longitude is linear in ``x``::

    lon = x / n * 360 - 180

Latitude is not linear in ``y``; the inverse Mercator projection gives::

    lat = degrees(atan(sinh(pi * (1 - 2 * y / n))))

For fractional ``y`` the same formula gives the latitude of any pixel row.
瓦片编号规则：经度与 x 成线性关系，纬度需通过反墨卡托公式由 y 计算。
"""
from __future__ import annotations

import io
import logging
import math
import threading
from collections import OrderedDict
from typing import Dict, Tuple

import numpy as np
from PIL import Image

from ..config import CLASS_COLOR_DARK, CLASS_NAMES

LOGGER = logging.getLogger(__name__)
```

```
#: Output tile edge in pixels (the Leaflet default). 瓦片边长（像素）。
TILE_SIZE = 256
#: Coordinates are classified on a coarse lattice and then upsampled.
#: 48 x 48 = 2304 model evaluations per tile instead of 65 536 per-pixel ones;
#: zone boundaries are smooth enough that bilinear upsampling hides the difference.
#: 每张瓦片仅分类 48×48 个点，再双线性放大到 256×256。
SAMPLE = 48
#: Maximum number of PNG tiles kept in memory (each is a few kilobytes).
#: 缓存瓦片数量上限。
CACHE_LIMIT = 900

#: Class index -> RGB colour. Built from the ``#rrggbb`` strings of the dark
#: theme: characters 1-2, 3-4 and 5-6 are parsed as hexadecimal R, G, B.
#: Shape (n_classes, 3), so ``_PALETTE[classes]`` maps a class grid to an image.
#: 调色板：把 "#rrggbb" 解析为 RGB，按类别索引直接查表上色。
_PALETTE = np.array(
    [[int(CLASS_COLOR_DARK[name][i:i + 2], 16) for i in (1, 3, 5)] for name in
CLASS_NAMES],
    dtype=np.uint8,
)
#: Slightly darker shade used for the 1-pixel graticule inside each tile.
#: (Currently not used by ``render_tile``; kept for an optional grid overlay.)
#: （当前未被 render_tile 使用。）
_GRID_RGB = np.array([12, 20, 30], dtype=np.uint8)

#: LRU cache ``(z, x, y) -> PNG bytes``. An ``OrderedDict`` keeps usage order:
#: a hit is moved to the end, eviction removes from the front (least recent).
#: The lock is needed because tiles are rendered in several worker threads at
#: once (``asyncio.to_thread`` in the endpoint).
#: LRU 缓存；多个线程同时渲染瓦片，因此需要加锁。
_cache: "OrderedDict[Tuple[int, int, int], bytes]" = OrderedDict()
_cache_lock = threading.Lock()


def tile_bounds(z: int, x: int, y: int) -> Tuple[float, float, float, float]:
    """Web-Mercator tile bounds as (lat_north, lat_south, lon_west, lon_east).

    Parameters
    ----------
    z : int
        Zoom level; the tile matrix has ``2**z`` columns and rows.
    x : int
        Tile column, 0 at 180 deg W, increasing eastwards.
    y : int
        Tile row, 0 at the northern edge (about 85.05 deg N), increasing
        southwards.

    Returns
    -------
    tuple of float
        ``(lat_north, lat_south, lon_west, lon_east)`` in decimal degrees.
        The north edge is row ``y``, the south edge is row ``y + 1``.
    """
    n = 2.0 ** z
    # Longitude is a linear function of the column. 经度与列号线性相关。
    lon_w = x / n * 360.0 - 180.0
    lon_e = (x + 1) / n * 360.0 - 180.0
    # Inverse Mercator: y/n in [0, 1] maps to Mercator ordinate pi*(1 - 2y/n)
    # in [pi, -pi]; latitude = atan(sinh(ordinate)) (the Gudermannian function).
    # 反墨卡托投影：纬度 = atan(sinh(π(1 − 2y/n)))。
    lat_n = math.degrees(math.atan(math.sinh(math.pi * (1 - 2 * y / n))))
    lat_s = math.degrees(math.atan(math.sinh(math.pi * (1 - 2 * (y + 1) / n))))
    return lat_n, lat_s, lon_w, lon_e


def render_tile(classifier, z: int, x: int, y: int, opacity: int = 235) -> bytes:
    """Render one PNG tile of the zone classification. 渲染单张分类瓦片。

    The tile is covered by a ``SAMPLE x SAMPLE`` lattice of coordinates that
    are evenly spaced in *projected* (Mercator) space, so they line up with the
    pixels Leaflet will display. Each coordinate is classified by the neural
    network; the most probable class selects the colour and the probability of
    that class (confidence) controls how much the colour is faded to grey.
    The small image is then resized to ``TILE_SIZE`` with bilinear filtering
    and PNG-encoded. Results are cached per ``(z, x, y)``.

    Parameters
    ----------
    classifier : ZoneClassifierService
```

```
        Object with ``predict_coordinates(lat, lon) -> (N, n_classes)``
        probability array.
    z, x, y : int
        Tile address (validated by the calling endpoint).
    opacity : int, optional
        Alpha channel value 0..255 for every pixel (default 235), so the tile
        is slightly transparent over the page background.

    Returns
    -------
    bytes
        PNG image, ``TILE_SIZE x TILE_SIZE`` RGBA.

    Notes
    -----
    The cache key does not include ``opacity``; all callers use the default.
    Two threads that miss the cache for the same tile at the same moment both
    render it; the second result simply overwrites the first (the images are
    identical), which is cheaper than holding the lock during rendering.
    缓存未命中时不持锁渲染，两个线程可能重复渲染同一瓦片，但结果相同。
    """
    key = (z, x, y)
    # Fast path: return a cached tile and mark it as most recently used.
    # 命中缓存：直接返回并移到末尾（最近使用）。
    with _cache_lock:
        cached = _cache.get(key)
        if cached is not None:
            _cache.move_to_end(key)
            return cached

    lat_n, lat_s, lon_w, lon_e = tile_bounds(z, x, y)
    # Mercator is non-linear in latitude: sample in projected space, then map back.
    # ``rows`` are fractional tile-row offsets 0..1, i.e. global rows y..y+1;
    # the inverse-Mercator formula of tile_bounds turns each into a latitude.
    # 在投影空间中等间距取样（y 到 y+1），再用反墨卡托公式换算成纬度。
    rows = np.linspace(0.0, 1.0, SAMPLE)
    n = 2.0 ** z
    lat_grid = np.degrees(np.arctan(np.sinh(math.pi * (1 - 2 * (y + rows) / n))))
    # Longitude is linear, so a plain linspace between the edges is exact.
    lon_grid = np.linspace(lon_w, lon_e, SAMPLE)
    # indexing="ij": first axis = image row (north -> south), second = column.
    # 第一维为图像行（北到南），第二维为列（西到东）。
    mesh_lat, mesh_lon = np.meshgrid(lat_grid, lon_grid, indexing="ij")

    probs = classifier.predict_coordinates(mesh_lat.ravel(), mesh_lon.ravel())
    classes = probs.argmax(axis=1).reshape(SAMPLE, SAMPLE).astype(np.uint8)
    confidence = probs.max(axis=1).reshape(SAMPLE, SAMPLE)

    # Palette lookup: (SAMPLE, SAMPLE) class indices -> (SAMPLE, SAMPLE, 3) RGB.
    rgb = _PALETTE[classes]
    # Low-confidence cells are blended towards grey so uncertainty is visible.
    # blend = 1 (pure class colour) for confidence >= 0.90, falls linearly to
    # 0.25 at confidence <= 0.5625; grey RGB(70, 84, 98) fills the remainder.
    # 置信度 ≥ 0.90 时为纯类别色，越低越接近灰色（最低保留 25% 类别色）。
    blend = np.clip((confidence - 0.45) / 0.45, 0.25, 1.0)[..., None]
    rgb = (rgb * blend + np.array([70, 84, 98]) * (1 - blend)).astype(np.uint8)

    # Append a constant alpha channel -> RGBA, then upsample to the tile size.
    # 添加统一的透明度通道，然后双线性放大到 256×256。
    alpha = np.full((SAMPLE, SAMPLE, 1), opacity, dtype=np.uint8)
    image = Image.fromarray(np.concatenate([rgb, alpha], axis=2), mode="RGBA")
    image = image.resize((TILE_SIZE, TILE_SIZE), Image.BILINEAR)

    buffer = io.BytesIO()
    image.save(buffer, format="PNG", optimize=True)
    payload = buffer.getvalue()

    # Store and evict the least recently used tiles beyond CACHE_LIMIT.
    # 写入缓存；超过上限时从头部（最久未用）淘汰。
    with _cache_lock:
        _cache[key] = payload
        while len(_cache) > CACHE_LIMIT:
            _cache.popitem(last=False)
    return payload


def cache_info() -> Dict:
    """Return the current tile-cache occupancy. 返回瓦片缓存状态。

    Returns
```

```
    -------
    dict
        ``{"tiles_cached": int, "limit": int}``; served by
        ``GET /api/tiles/info``.
    """
    with _cache_lock:
        return {"tiles_cached": len(_cache), "limit": CACHE_LIMIT}
```

D.8 Web client

static/index.html – the single-page client

```
<!DOCTYPE html>
<html lang="en">
<head>
<!-- Maritime Route Planner - single-page client.
     单页前端：左侧为控制面板（规划/数据/对比/模型/历史五个标签页），
     右侧为 Leaflet 海图。所有交互通过 api.js 的 REST 与 WebSocket 完成。
     Served by GET / (maritime_route/web/app.py). Layout: a top bar with status
     pills, a left control panel with five tabs (Plan, Data, Compare, Model,
     History) and a Leaflet chart on the right. The page has no inline script;
     behaviour lives in js/api.js (server access), js/charts.js (charts) and
     js/app.js (UI logic), which look elements up by the id attributes below.
     Leaflet is bundled under /static/vendor so the page works without
     internet access. -->
<!-- Document metadata: encoding, responsive viewport, favicon (served inline by GET
/favicon.ico). -->
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="icon" href="/favicon.ico" type="image/svg+xml">
<title>Maritime Route Planner</title>
<!-- Stylesheets: Leaflet first, then the application sheet, which overrides some
Leaflet rules. -->
<link rel="stylesheet" href="/static/vendor/leaflet/leaflet.css">
<link rel="stylesheet" href="/static/css/app.css">
</head>
<body>
<!-- Top bar: application name and two status pills. 顶栏：标题与状态标签。 -->
<header class="topbar">
  <!-- Brand: inline SVG anchor icon and title/subtitle. -->
  <div class="brand">
    <svg viewBox="0 0 24 24" class="brand-icon" aria-hidden="true">
      <path d="M12 2v14M12 5h5l-5 4M4 17c2 2 4 2 4 2h8s2 0 4-2l-2 5H6z"/>
    </svg>
    <div>
      <h1>Maritime Route Planner</h1>
      <p>Neural-network classification of geographic zones &middot; A* · Dijkstra ·
genetic · dynamic programming</p>
    </div>
  </div>
  <!-- Status pills updated by app.js: #ws-status = WebSocket state
       (live connection / reconnecting / connection error), #model-pill = model
       test accuracy or "model unavailable". 状态标签：WebSocket 连接状态与模型状态。 -->
  <div class="status-cluster">
    <span id="ws-status" class="pill pill-wait">connecting…</span>
    <span id="model-pill" class="pill pill-muted">model —</span>
  </div>
</header>

<!-- Two-column layout: control panel and chart. 两栏布局：控制面板 + 海图。 -->
<main class="layout">
  <!-- ---------------------------------------- control panel  控制面板 -->
  <aside class="panel" id="panel">
    <!-- Tab bar: a button with data-tab="X" shows the section with id="tab-X"
         (tab handler in app.js). 选项卡：data-tab 对应 id 为 tab-X 的页面。 -->
    <nav class="tabs" role="tablist">
      <button class="tab active" data-tab="plan" role="tab">Plan</button>
      <button class="tab" data-tab="data" role="tab">Data</button>
      <button class="tab" data-tab="compare" role="tab">Compare</button>
      <button class="tab" data-tab="model" role="tab">Model</button>
      <button class="tab" data-tab="history" role="tab">History</button>
    </nav>
```

```
    <!-- ---------------------------------------------------- PLAN -->
    <section class="tab-page active" id="tab-plan">
      <!-- Click hint; app.js rewrites it to say whether the next map click sets
           the departure or the destination. -->
      <div class="hint" id="pick-hint">
        Click the chart to set <b>departure</b>, click again to set <b>destination</b>.
      </div>

      <!-- Departure coordinates in decimal degrees. Filled by map clicks,
           marker drags and preset chips, and editable by hand; read by app.js
           before every request. The default voyage is Kiel -> Tallinn.
           起点坐标（十进制度）。 -->
      <div class="field-row">
        <label class="field">
          <span>Departure lat</span>
          <input type="number" id="start-lat" step="0.0001" value="54.3233">
        </label>
        <label class="field">
          <span>Departure lon</span>
          <input type="number" id="start-lon" step="0.0001" value="10.1228">
        </label>
      </div>
      <!-- Destination coordinates in decimal degrees. 终点坐标。 -->
      <div class="field-row">
        <label class="field">
          <span>Destination lat</span>
          <input type="number" id="end-lat" step="0.0001" value="59.4370">
        </label>
        <label class="field">
          <span>Destination lon</span>
          <input type="number" id="end-lon" step="0.0001" value="24.7536">
        </label>
      </div>

      <!-- Preset voyages: data-preset="startLat,startLon,endLat,endLon". A click
           fills the four inputs and zooms the map; planning is not started.
           data-resolution: recommended lattice step (default 0.25°).
           预设航线：点击后填入坐标并缩放地图。 -->
      <div class="preset-row">
        <span class="preset-label">Presets</span>
        <button class="chip" data-preset="54.3233,10.1228,59.4370,24.7536">Kiel →
Tallinn</button>
        <button class="chip" data-preset="51.9500,4.1400,38.6900,-9.4000">Rotterdam →
Lisbon</button>
        <button class="chip" data-preset="60.1699,24.9384,55.6761,12.5683">Helsinki →
Copenhagen</button>
        <button class="chip" data-preset="46.5209,30.7727,44.7252,37.7930">Odesa →
Novorossiysk</button>
        <button class="chip" data-preset="37.9200,23.5800,40.5500,22.8800" data-
resolution="0.1">Piraeus → Thessaloniki</button>
      </div>

      <!-- Search algorithm; option values are the names accepted by the server
           (config.SUPPORTED_ALGORITHMS). 搜索算法选择。 -->
      <label class="field">
        <span>Search algorithm</span>
        <select id="algorithm">
          <option value="astar">A* (heuristic, fastest)</option>
          <option value="dijkstra">Dijkstra (uniform cost)</option>
          <option value="genetic">Genetic algorithm (evolutionary)</option>
          <option value="dynamic">Dynamic programming (Bellman equation)</option>
          <option value="great_circle">Great circle (shortest distance)</option>
        </select>
      </label>

      <!-- Grid resolution slider (cell size of the routing lattice, 0.1..1.0 deg);
           the <output> shows the current value. Finer grids pass narrower straits
           but take longer; the server refines automatically if no passage is found.
           网格分辨率：越细越能通过狭窄海峡，但计算更慢。 -->
      <label class="field">
        <span>Grid resolution <output id="res-out">0.25°</output></span>
        <input type="range" id="resolution" min="0.1" max="1.0" step="0.05"
value="0.25">
      </label>

      <!-- Service speed in knots; used only for the passage-time estimate. 航速（节）。
-->
      <label class="field">
        <span>Service speed <output id="speed-out">14 kn</output></span>
        <input type="range" id="speed" min="6" max="30" step="0.5" value="14">
```

```
        </label>

        <!-- Display options: the classified lattice sent with the result
             ("overlay" WebSocket frame) and the live zone read-out under the pointer
             ("classify" WebSocket action). 显示选项：分类网格图层与指针处实时分类。 -->
        <label class="checkbox">
          <input type="checkbox" id="show-overlay" checked>
          <span>Show zone classification layer</span>
        </label>
        <label class="checkbox">
          <input type="checkbox" id="live-classify" checked>
          <span>Live zone read-out while pointing at the chart</span>
        </label>

        <!-- Actions: plan one route (WebSocket, REST fallback), benchmark all
             algorithms (POST /api/benchmark, result on the Compare tab), clear the
             map. 操作按钮：规划、算法对比、清除。 -->
        <div class="actions">
          <button class="btn btn-primary" id="btn-plan">Build route</button>
          <button class="btn" id="btn-benchmark">Compare algorithms</button>
          <button class="btn btn-ghost" id="btn-clear">Clear</button>
        </div>

        <!-- Live progress (hidden until a "started" frame arrives): stage name,
             percentage of the current stage, bar and a detail line (cells classified,
             nodes expanded, GA generation...). 实时进度块。 -->
        <div class="progress-block" id="progress-block" hidden>
          <div class="progress-head">
            <span id="progress-stage">working…</span>
            <span id="progress-pct">0%</span>
          </div>
          <div class="progress-rail"><div class="progress-fill"
id="progress-fill"></div></div>
          <div class="progress-detail" id="progress-detail"></div>
        </div>

        <!-- Route summary (hidden until a result is rendered): metric tiles, zone
             profile bar with legend, and export links whose href is set to
             /api/export/{route_id}.{geojson|csv|pdf} by app.js.
             结果摘要：指标、区域占比与导出链接。 -->
        <div id="result-block" hidden>
          <h2 class="section-title">Route summary</h2>
          <!-- Result of the independent coastline check (GLOBE 1 km land mask):
               green when passed, amber when corrected automatically, red when the
               route still crosses land. 独立海岸线检查结果。 -->
          <div id="coast-check" class="verdict" hidden></div>
          <div class="metric-grid" id="metric-grid"></div>
          <h3 class="section-title">Zone profile</h3>
          <div id="zone-bar" class="zone-bar"></div>
          <div id="zone-legend" class="zone-legend"></div>
          <div class="export-row">
            <a class="btn btn-small" id="dl-geojson" download>GeoJSON</a>
            <a class="btn btn-small" id="dl-csv" download>CSV</a>
            <a class="btn btn-small" id="dl-pdf" download>PDF report</a>
          </div>
        </div>
      </section>

      <!-- ------------------------------------------------- DATA -->
      <section class="tab-page" id="tab-data">
        <!-- Upload of an AIS file (POST /api/upload). The <label> wraps a hidden
             file input, so a click opens the file dialog; drag-and-drop is handled
             in app.js. 上传 AIS 文件：点击或拖放。 -->
        <h2 class="section-title">Load AIS data</h2>
        <p class="muted">Upload a GeoJSON, CSV or JSON file of route points. The
          points are classified by the network and stored in the database.</p>
        <label class="dropzone" id="dropzone">
          <input type="file" id="file-input" accept=".geojson,.json,.csv" hidden>
          <span id="dropzone-text">Drop a file here or click to browse</span>
        </label>
        <!-- Load/validation report and class histogram of the upload, followed by
             export links for the classified points (/api/export/points/{token}.*). -->
        <div id="upload-report" class="report" hidden></div>
        <div class="export-row" id="points-export" hidden>
          <a class="btn btn-small" id="dl-points-geojson" download>Points GeoJSON</a>
          <a class="btn btn-small" id="dl-points-csv" download>Points CSV</a>
        </div>
        <!-- Database counters from GET /api/statistics. 数据库统计。 -->
        <h3 class="section-title">Database</h3>
        <div id="db-stats" class="report"></div>
```

```
    </section>

    <!-- ---------------------------------------------------- COMPARE -->
    <section class="tab-page" id="tab-compare">
      <!-- Comparison of the planned route with the great-circle baseline
           (filled by renderComparison in app.js). 优化航线与大圆航线对比。 -->
      <h2 class="section-title">Optimised vs shortest distance</h2>
      <div id="compare-block" class="report">Build a route first.</div>
      <!-- Benchmark table filled by the "Compare algorithms" button. 算法基准测试结果。 --
>
      <h3 class="section-title">Algorithm benchmark</h3>
      <div id="benchmark-block" class="report">Run “Compare algorithms”.</div>
    </section>

    <!-- ------------------------------------------------------ MODEL -->
    <section class="tab-page" id="tab-model">
      <!-- Model summary and test metrics from GET /api/model. 模型摘要与测试指标。 -->
      <h2 class="section-title">Classification model</h2>
      <div id="model-block" class="report">loading…</div>
      <!-- Confusion matrix (Charts.confusionMatrix) and training curves
           (Charts.trainingCurves). The canvas width/height attributes give its
           CSS size; charts.js enlarges the pixel buffer for high-DPI screens. -->
      <h3 class="section-title">Confusion matrix (test set)</h3>
      <div id="confusion" class="confusion"></div>
      <h3 class="section-title">Training curves</h3>
      <canvas id="history-chart" width="360" height="170"></canvas>
    </section>

    <!-- ---------------------------------------------------- HISTORY -->
    <section class="tab-page" id="tab-history">
      <!-- Stored routes from GET /api/routes?limit=25; clicking an item draws
           that route on the map. 已存储航线列表，点击可在地图上显示。 -->
      <h2 class="section-title">Stored routes</h2>
      <button class="btn btn-small" id="btn-refresh-history">Refresh</button>
      <div id="history-list" class="history-list"></div>
    </section>
  </aside>

  <!-- ------------------------------------------------------ map  海图区域 -->
  <section class="map-wrap">
    <!-- Leaflet map container, sized by CSS to fill .map-wrap. 地图容器。 -->
    <div id="map"></div>
    <!-- Floating zone legend, filled from GET /api/zones. 图例。 -->
    <div class="map-legend" id="map-legend"></div>
    <!-- Cursor read-out: coordinates under the pointer and, if the live
         read-out is enabled, the zone class, confidence and depth/elevation
         there. 指针位置坐标与实时分类结果。 -->
    <div class="cursor-readout" id="cursor-readout">
      <span class="coord">— , —</span>
      <span class="zone">move the pointer over the chart</span>
    </div>
  </section>
</main>

<!-- Scripts, in dependency order: Leaflet (global L), api.js (Api,
     LiveChannel), charts.js (Charts), then app.js, which uses all of them
     and starts the application. Placed at the end of <body> so that every
     element referenced by id already exists when app.js runs.
     脚本按依赖顺序加载，放在 body 末尾以保证元素已存在。 -->
<script src="/static/vendor/leaflet/leaflet.js"></script>
<script src="/static/js/api.js"></script>
<script src="/static/js/charts.js"></script>
<script src="/static/js/app.js"></script>
</body>
</html>
```

static/css/app.css – the stylesheet

```
/* ============================================================================
   Maritime Route Planner - user interface stylesheet
   界面样式表：深色海图风格，响应式两栏布局。

   Place in the application: the only project stylesheet of index.html; it is
   loaded after vendor/leaflet/leaflet.css so that the rules at the end can
   restyle Leaflet controls for the dark theme.
   Organisation: design tokens (:root custom properties) first, then one
   section per UI area in the order they appear on the page (top bar, layout,
   tabs, fields, buttons, progress, metrics, zone bar, report, dropzone,
```

```
   confusion matrix, history, map overlays, Leaflet overrides) and finally
   the responsive breakpoint.
   All colours are referenced through the variables below, except a few
   one-off tints of status colours.
   样式按页面区域分节，颜色统一通过 :root 变量引用。
   ========================================================================== */
/* ---------------------------------------------------- design tokens
   CSS custom properties used throughout the sheet.
   --bg / --bg-2 / --bg-3 : page, panel and card backgrounds (dark to lighter).
   --line                 : borders and grid lines.
   --text / --muted       : primary and secondary text.
   --accent / --accent-2  : interactive blue and success teal.
   --danger / --warn      : error red and warning amber.
   --open-sea ... --coastline : zone colours; must stay identical to
       ZONE_COLOURS in js/app.js and CLASS_COLOR_DARK in config.py.
   设计变量：背景、文字、强调色以及四个区域颜色（需与 app.js、config.py 一致）。 */
:root {
  --bg:        #0e1621;
  --bg-2:      #16212e;
  --bg-3:      #1d2b3a;
  --line:      #2a3c4f;
  --text:      #e6edf5;
  --muted:     #94a7bb;
  --accent:    #4da3ff;
  --accent-2:  #2a9d8f;
  --danger:    #ef6461;
  --warn:      #e9c46a;
  --open-sea:     #3d7fb8;
  --coastal-sea:  #7cc3e3;
  --near-coast:   #d9b877;
  --coastline:    #a9763f;
  --radius: 9px;
  --shadow: 0 6px 22px rgba(0, 0, 0, .35);
}

/* Global reset: padding and border are included in element widths. */
* { box-sizing: border-box; }

/* Full-height page so the map can fill the viewport; base font and colours. */
html, body {
  margin: 0; height: 100%;
  background: var(--bg); color: var(--text);
  font: 14px/1.45 "Segoe UI", "Helvetica Neue", Arial, system-ui, sans-serif;
}

/* ------------------------------------------------------------ top bar */
.topbar {
  display: flex; align-items: center; justify-content: space-between;
  gap: 16px; padding: 10px 18px;
  background: linear-gradient(90deg, #12283f, #0e1621 70%);
  border-bottom: 1px solid var(--line);
}
/* Logo (inline SVG anchor) and application title. 标志与标题。 */
.brand { display: flex; align-items: center; gap: 12px; }
.brand-icon {
  width: 30px; height: 30px; fill: none;
  stroke: var(--accent); stroke-width: 1.6; stroke-linecap: round; stroke-linejoin:
round;
}
.brand h1 { margin: 0; font-size: 17px; letter-spacing: .2px; }
.brand p  { margin: 1px 0 0; font-size: 11.5px; color: var(--muted); }

/* Status pills in the top bar (#ws-status, #model-pill). The modifier classes
   are set by setStatus() in app.js: ok = green, wait = amber, bad = red.
   状态标签：ok 绿色、wait 黄色、bad 红色。 */
.status-cluster { display: flex; gap: 8px; align-items: center; }
.pill {
  font-size: 11px; padding: 4px 10px; border-radius: 20px;
  border: 1px solid var(--line); background: var(--bg-2); color: var(--muted);
  white-space: nowrap;
}
.pill-ok   { color: #8ff0c0; border-color: #2f6b52; background: #14301f; }
.pill-wait { color: var(--warn); border-color: #6b5a2f; background: #302a14; }
.pill-bad  { color: #ffb4b2; border-color: #6b2f2f; background: #301414; }

/* ------------------------------------------------------------ layout */
/* Two-column grid: fixed 392 px control panel + map taking the rest.
   The height subtracts the ~53 px top bar so the page itself does not scroll;
   the panel scrolls internally.
   两栏网格布局；高度减去顶栏高度，面板内部滚动。 */
```

```
.layout {
  display: grid; grid-template-columns: 392px 1fr;
  height: calc(100vh - 53px);
}
.panel {
  background: var(--bg-2); border-right: 1px solid var(--line);
  overflow-y: auto; padding: 0 16px 28px;
}
.map-wrap { position: relative; }
/* Leaflet needs an element with an explicit size: #map fills .map-wrap. */
#map { position: absolute; inset: 0; background: #0a1420; }

/* ---------------------------------------------------------------- tabs */
/* Tab bar of the side panel. Sticky so it stays visible while the panel
   scrolls. Only the .tab-page with class .active is displayed; the class is
   switched by the tab handler in app.js.
   选项卡栏吸顶；只有带 active 类的页面可见。 */
.tabs {
  position: sticky; top: 0; z-index: 5;
  display: flex; gap: 2px; padding: 10px 0 0;
  background: var(--bg-2); border-bottom: 1px solid var(--line);
}
.tab {
  flex: 1; padding: 8px 4px; font-size: 12px; cursor: pointer;
  background: transparent; color: var(--muted);
  border: none; border-bottom: 2px solid transparent;
}
.tab:hover { color: var(--text); }
.tab.active { color: var(--accent); border-bottom-color: var(--accent); }
.tab-page { display: none; padding-top: 14px; }
.tab-page.active { display: block; }

/* ---------------------------------------------------------------- fields */
/* Small upper-case headings inside the panel. */
.section-title {
  font-size: 12px; text-transform: uppercase; letter-spacing: .7px;
  color: var(--muted); margin: 16px 0 8px; font-weight: 600;
}
/* Labelled form controls. The label <span> is a flex row so that a live
   <output> value (resolution, speed) is right-aligned next to the caption. */
.field { display: block; margin-bottom: 10px; }
.field > span {
  display: flex; justify-content: space-between;
  font-size: 11.5px; color: var(--muted); margin-bottom: 4px;
}
.field-row { display: grid; grid-template-columns: 1fr 1fr; gap: 8px; }
/* Dark-themed number inputs and drop-down; a visible outline on focus for
   keyboard users. */
input[type=number], select {
  width: 100%; padding: 7px 9px; border-radius: var(--radius);
  background: var(--bg-3); border: 1px solid var(--line); color: var(--text);
  font-size: 13px;
}
input[type=number]:focus, select:focus { outline: 2px solid var(--accent); border-color:
transparent; }
input[type=range] { width: 100%; accent-color: var(--accent); }
output { color: var(--accent); font-variant-numeric: tabular-nums; }

/* Checkbox rows (overlay layer, live read-out). */
.checkbox {
  display: flex; align-items: center; gap: 8px;
  font-size: 12.5px; color: var(--muted); margin: 6px 0;
}
.checkbox input { accent-color: var(--accent); }

/* Hint box above the coordinate inputs (#pick-hint). */
.hint {
  font-size: 12px; color: var(--muted); background: var(--bg-3);
  border-left: 3px solid var(--accent); border-radius: 5px;
  padding: 8px 10px; margin-bottom: 12px;
}
.hint b { color: var(--text); }
.muted { color: var(--muted); font-size: 12px; }

/* Preset voyage chips (Kiel -> Tallinn etc.). 预设航线按钮。 */
.preset-row { display: flex; flex-wrap: wrap; gap: 6px; align-items: center; margin: 4px
0 12px; }
.preset-label { font-size: 11px; color: var(--muted); }
.chip {
  font-size: 11px; padding: 4px 9px; border-radius: 14px; cursor: pointer;
```

```
  background: var(--bg-3); border: 1px solid var(--line); color: var(--muted);
}
.chip:hover { color: var(--text); border-color: var(--accent); }

/* ------------------------------------------------------------ buttons */
/* Buttons. .btn is also used on <a> export links, hence text-decoration
   none and inline-block. :disabled is applied while a job is running.
   Variants: primary (filled accent), ghost (transparent), small. */
.actions { display: flex; gap: 8px; margin: 14px 0 6px; flex-wrap: wrap; }
.btn {
  padding: 8px 13px; border-radius: var(--radius); cursor: pointer;
  background: var(--bg-3); border: 1px solid var(--line); color: var(--text);
  font-size: 12.5px; text-decoration: none; display: inline-block;
}
.btn:hover { border-color: var(--accent); }
.btn:disabled { opacity: .5; cursor: not-allowed; }
.btn-primary { background: var(--accent); border-color: var(--accent); color: #062036;
font-weight: 600; }
.btn-primary:hover { background: #6fb6ff; }
.btn-ghost { background: transparent; }
.btn-small { padding: 5px 10px; font-size: 11.5px; }

/* ------------------------------------------------------------ progress */
/* Planning progress block (#progress-block), fed by WebSocket 'progress'
   frames. The fill width is set from JavaScript; the short width transition
   smooths the jumps between throttled frames (one per ~0.15 s).
   进度条：宽度由 JS 设置，过渡动画平滑节流后的跳变。 */
.progress-block {
  margin: 12px 0; padding: 10px; border-radius: var(--radius);
  background: var(--bg-3); border: 1px solid var(--line);
}
.progress-head { display: flex; justify-content: space-between; font-size: 12px; margin-
bottom: 6px; }
#progress-pct { color: var(--accent); font-variant-numeric: tabular-nums; }
.progress-rail { height: 6px; border-radius: 3px; background: #0e1621; overflow: hidden;
}
.progress-fill {
  height: 100%; width: 0%; border-radius: 3px;
  background: linear-gradient(90deg, var(--accent-2), var(--accent));
  transition: width .18s ease-out;
}
.progress-detail { font-size: 11px; color: var(--muted); margin-top: 6px; min-height:
14px; }

/* ------------------------------------------------------------ metrics */
/* Route summary tiles built by metric() in app.js; two per row.
   .good / .warn / .bad colour the value (e.g. detour ratio > 1.5 = warn). */
.metric-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 8px; }
.metric {
  background: var(--bg-3); border: 1px solid var(--line);
  border-radius: var(--radius); padding: 9px 10px;
}
.metric .label { font-size: 10.5px; color: var(--muted); text-transform: uppercase;
letter-spacing: .5px; }
.metric .value { font-size: 17px; font-variant-numeric: tabular-nums; margin-top: 2px; }
.metric .value small { font-size: 11px; color: var(--muted); }
.metric.good .value { color: #7fe3b0; }
.metric.warn .value { color: var(--warn); }
.metric.bad  .value { color: var(--danger); }

/* ------------------------------------------------------------ zone bar */
/* Stacked zone-profile bar (Charts.zoneBar): a flex row whose child <div>s
   get percentage widths inline; overflow hidden clips the rounded corners. */
.zone-bar { display: flex; height: 18px; border-radius: 5px; overflow: hidden; border:
1px solid var(--line); }
.zone-bar div { height: 100%; }
.zone-legend { display: flex; flex-wrap: wrap; gap: 10px; margin-top: 7px; font-size:
11px; color: var(--muted); }
.zone-legend i { display: inline-block; width: 10px; height: 10px; border-radius: 2px;
margin-right: 4px; }

/* Row of export links (GeoJSON / CSV / PDF, points export). */
.export-row { display: flex; gap: 7px; margin-top: 13px; flex-wrap: wrap; }

/* ------------------------------------------------------------ report */
/* Generic card with a two-column key/value table, used for the upload
   report, database statistics, comparison, benchmark and model panels.
   Values (last column) are right-aligned with tabular digits.
   .verdict is the coloured conclusion line of the great-circle comparison.
   通用信息卡片：键值表格，数值右对齐。 */
```

```
.report {
  background: var(--bg-3); border: 1px solid var(--line); border-radius: var(--radius);
  padding: 10px; font-size: 12px; color: var(--muted); margin-top: 6px;
}
.report table { width: 100%; border-collapse: collapse; }
.report td { padding: 3px 4px; border-bottom: 1px solid var(--line); }
.report td:first-child { color: var(--muted); }
.report td:last-child { color: var(--text); text-align: right; font-variant-numeric:
tabular-nums; }
.report tr:last-child td { border-bottom: none; }
.report h4 { margin: 8px 0 4px; color: var(--text); font-size: 12px; }
.verdict { padding: 7px 9px; border-radius: 6px; margin-top: 8px; font-size: 11.5px; }
.verdict.bad  { background: #33161a; color: #ffb9b7; border-left: 3px solid var(--
danger); }
.verdict.good { background: #143026; color: #93e8c1; border-left: 3px solid var(--
accent-2); }
/* Coastline check banner: .fixed = the first route crossed land and was
   corrected automatically. 海岸线检查横幅：fixed 表示已自动修正。 */
.verdict.fixed { background: #33280f; color: #ffd98a; border-left: 3px solid var(--
warn); }
#coast-check { margin: 0 0 8px; line-height: 1.35; }

/* ------------------------------------------------------------ dropzone */
/* File drop area of the Data tab; .drag is added by app.js while a file is
   dragged over it. 文件拖放区域。 */
.dropzone {
  display: flex; align-items: center; justify-content: center;
  height: 82px; border: 1.5px dashed var(--line); border-radius: var(--radius);
  color: var(--muted); font-size: 12.5px; cursor: pointer; text-align: center; padding:
8px;
}
.dropzone:hover, .dropzone.drag { border-color: var(--accent); color: var(--text);
background: var(--bg-3); }

/* ------------------------------------------------------------ confusion */
/* Confusion matrix grid (Charts.confusionMatrix): each row is a grid with a
   74 px label column and 4 equal columns, one per zone class. Cell
   backgrounds are set inline from the row-normalised share.
   混淆矩阵：标签列 74 px，其余 4 列对应四个区域类别。 */
.confusion { display: grid; gap: 2px; font-size: 10px; }
.confusion .cm-row { display: grid; grid-template-columns: 74px repeat(4, 1fr); gap:
2px; }
.confusion .cm-cell {
  padding: 5px 2px; text-align: center; border-radius: 3px;
  background: var(--bg-3); color: var(--text); font-variant-numeric: tabular-nums;
}
.confusion .cm-head { background: transparent; color: var(--muted); font-size: 9px; }

/* ------------------------------------------------------------ history */
/* Stored-route list of the History tab; each item is clickable. */
.history-list { margin-top: 10px; display: flex; flex-direction: column; gap: 6px; }
.history-item {
  background: var(--bg-3); border: 1px solid var(--line); border-radius: var(--radius);
  padding: 8px 10px; font-size: 11.5px; cursor: pointer;
}
.history-item:hover { border-color: var(--accent); }
.history-item .row1 { display: flex; justify-content: space-between; color: var(--text);
}
.history-item .row2 { color: var(--muted); margin-top: 2px; }

/* ------------------------------------------------------------ map extra */
/* Floating boxes over the map: the zone legend (top right) and the cursor
   read-out (bottom left). They are siblings of #map, so z-index 500 puts
   them above Leaflet's map pane (z-index 400, which holds tiles, vectors,
   markers and popups) but below Leaflet's control corners (z-index 1000).
   地图浮层：z-index 500 高于 Leaflet 地图窗格(400)，低于控件(1000)。 */
.map-legend, .cursor-readout {
  position: absolute; z-index: 500;
  background: rgba(14, 22, 33, .9); border: 1px solid var(--line);
  border-radius: var(--radius); padding: 8px 11px; font-size: 11.5px;
  box-shadow: var(--shadow); backdrop-filter: blur(3px);
}
.map-legend { top: 12px; right: 12px; }
.map-legend div { display: flex; align-items: center; gap: 6px; margin: 2px 0; color:
var(--muted); }
.map-legend i { width: 11px; height: 11px; border-radius: 2px; display: inline-block; }
.cursor-readout { bottom: 12px; left: 12px; display: flex; gap: 12px; align-items:
center; }
.cursor-readout .coord { color: var(--text); font-variant-numeric: tabular-nums; }
.cursor-readout .zone { color: var(--muted); }
```

```
/* Leaflet overrides for the dark theme. The container background is the
   "sea" shown while tiles load or when none are available; it must be set
   here and not drawn as a map layer (see the note in app.js).
   Leaflet 深色主题覆盖；底色必须在此设置，而不是绘制为图层。 */
.leaflet-container { background: #0a1420; }
.leaflet-popup-content-wrapper, .leaflet-popup-tip {
  background: var(--bg-2); color: var(--text); border: 1px solid var(--line);
}
.leaflet-popup-content { font-size: 12px; margin: 9px 11px; }
.leaflet-bar a { background: var(--bg-2); color: var(--text); border-color: var(--line);
}
.leaflet-bar a:hover { background: var(--bg-3); }

/* Narrow screens (tablets, phones): stack the panel above the map instead of
   side by side. 窄屏时面板与地图上下排列。 */
@media (max-width: 980px) {
  .layout { grid-template-columns: 1fr; grid-template-rows: 45vh 55vh; height: auto; }
  .panel { border-right: none; border-bottom: 1px solid var(--line); }
  .map-wrap { min-height: 55vh; }
}
```

static/js/api.js – REST and WebSocket transport

```
/* ============================================================================
 * api.js - REST helpers and the auto-reconnecting WebSocket client.
 * 接口层：封装 REST 请求与带自动重连的 WebSocket 客户端。
 *
 * Place in the application: the only file of the browser client that talks
 * to the server. It is loaded before app.js (see index.html) and defines two
 * globals used there:
 *   - Api         : object with one promise-returning method per REST endpoint
 *                   of maritime_route/web/app.py;
 *   - LiveChannel : class wrapping the /ws/plan WebSocket (progress frames,
 *                   automatic reconnection, heartbeat, send queue).
 * No build step or module system is used; both globals are plain scripts.
 * 本文件定义全局对象 Api 与类 LiveChannel，供 app.js 使用。
 * ============================================================================ */
'use strict';

/**
 * REST client for the FastAPI back end.
 *
 * Built with an immediately invoked function expression (IIFE) so that the
 * private helper `request` is not visible outside; only the returned object
 * is exposed as the global `Api`. Every method returns a Promise that
 * resolves with the parsed JSON body or rejects with an Error whose message
 * is the server's `detail` text (FastAPI's error field) or the HTTP status text.
 * 使用立即执行函数隐藏内部辅助函数，仅暴露 Api 对象。
 *
 * @namespace Api
 */
const Api = (() => {

  /**
   * Perform a JSON request and unwrap the response.
   *
   * @param {string} path - URL path, e.g. '/api/zones'.
   * @param {RequestInit} [options={}] - fetch options; they are spread after
   *   the default headers, so a caller-supplied `headers` replaces them.
   * @returns {Promise<any>} Parsed JSON body of a 2xx response.
   * @throws {Error} For non-2xx responses; message = server `detail` if the
   *   error body is JSON, otherwise the HTTP status text.
   */
  async function request(path, options = {}) {
    const response = await fetch(path, {
      headers: { 'Content-Type': 'application/json' },
      ...options,
    });
    if (!response.ok) {
      let detail = response.statusText;
      // Error bodies are normally {"detail": "..."}; a non-JSON body is ignored.
      // 错误响应通常为 {"detail": ...}，非 JSON 时保留状态文本。
      try { detail = (await response.json()).detail || detail; } catch (_) { /* ignore
*/ }
      throw new Error(detail);
    }
    return response.json();
```

```
  }

  // One entry per endpoint. The method name mirrors the endpoint purpose:
  //   health     GET  /api/health            model availability + uptime
  //   zones      GET  /api/zones             zone classes (legend)
  //   model      GET  /api/model             model summary, metrics, history
  //   statistics GET  /api/statistics        database statistics
  //   routes     GET  /api/routes?limit=N    stored routes (newest first)
  //   route      GET  /api/routes/{uid}      one stored route
  //   plan       POST /api/route             blocking route planning
  //   benchmark  POST /api/benchmark         compare all algorithms
  //   classify   POST /api/classify          classify [[lat, lon], ...]
  //   upload     POST /api/upload            multipart AIS file upload
  // 每个方法对应一个后端接口。
  return {
    health:      ()          => request('/api/health'),
    zones:       ()          => request('/api/zones'),
    model:       ()          => request('/api/model'),
    statistics:  ()          => request('/api/statistics'),
    routes:      (limit = 30)=> request(`/api/routes?limit=${limit}`),
    route:       (uid)       => request(`/api/routes/${uid}`),
    plan:        (body)      => request('/api/route',     { method: 'POST', body:
JSON.stringify(body) }),
    benchmark:   (body)      => request('/api/benchmark', { method: 'POST', body:
JSON.stringify(body) }),
    classify:    (points)    => request('/api/classify',  { method: 'POST', body:
JSON.stringify({ points }) }),

    /**
     * Upload an AIS file for classification (POST /api/upload).
     *
     * Does not use `request` because the body is multipart FormData: the
     * browser must set the Content-Type header itself (it adds the multipart
     * boundary), so no JSON header may be sent.
     * 上传使用 FormData，浏览器会自动设置带 boundary 的 Content-Type。
     *
     * @param {File} file - File chosen in the upload input or dropped on the page.
     * @returns {Promise<Object>} Upload summary: token, session_id, report,
     *   classified, histogram, accuracy_vs_labels, points.
     * @throws {Error} With the server `detail` (e.g. 400 parse error, 413 too large).
     */
    async upload(file) {
      const form = new FormData();
      form.append('file', file);
      const response = await fetch('/api/upload', { method: 'POST', body: form });
      if (!response.ok) {
        let detail = response.statusText;
        try { detail = (await response.json()).detail || detail; } catch (_) { /* ignore
*/ }
        throw new Error(detail);
      }
      return response.json();
    },
  };
})();


/**
 * Real-time channel to the planning service.
 * 实时通道：断线后以指数退避自动重连，并保证消息按类型分发。
 *
 * Wraps one browser WebSocket to /ws/plan and adds:
 *   - dispatch of incoming frames to handlers registered per frame `type`
 *     ('hello', 'started', 'progress', 'overlay', 'result', 'classified',
 *     'pong', 'error'), plus the wildcard type '*' that receives every frame;
 *   - a pseudo-type 'status' emitted locally with {state: 'open'|'closed'|'error'}
 *     so the UI can show the connection state;
 *   - an outgoing queue: messages sent while the socket is not open are kept
 *     and flushed in order as soon as the connection opens;
 *   - automatic reconnection with exponential back-off;
 *   - a heartbeat ('ping' every 25 s) so proxies do not close an idle socket.
 * 功能：按类型分发消息、离线消息队列、指数退避重连、心跳保活。
 */
class LiveChannel {
  /**
   * @param {string} [path='/ws/plan'] - WebSocket path on the same host as the page.
   */
  constructor(path = '/ws/plan') {
    // Use wss:// when the page itself was loaded over HTTPS (browsers block
    // insecure sockets from secure pages). 页面为 HTTPS 时使用 wss。
```

```
    const scheme = location.protocol === 'https:' ? 'wss' : 'ws';
    /** @type {string} Absolute socket URL. */
    this.url = `${scheme}://${location.host}${path}`;
    /** @type {Map<string, Function[]>} Frame type -> list of handlers. */
    this.handlers = new Map();
    /** @type {WebSocket|null} Current socket; replaced on every reconnect. */
    this.socket = null;
    /** @type {number} Consecutive failed attempts; drives the back-off delay. */
    this.retry = 0;
    /** @type {string[]} Serialised messages waiting for an open socket. */
    this.queue = [];
    /** @type {number|null} setInterval id of the ping heartbeat. */
    this.heartbeat = null;
  }

  /**
   * Register a handler for one frame type.
   *
   * @param {string} type - Server frame `type`, 'status' or '*'.
   * @param {function(Object): void} handler - Called with the parsed frame.
   * @returns {LiveChannel} `this`, so calls can be chained.
   */
  on(type, handler) {
    if (!this.handlers.has(type)) this.handlers.set(type, []);
    this.handlers.get(type).push(handler);
    return this;
  }

  /**
   * Call every handler registered for `type`.
   *
   * Each handler runs in its own try/catch so that an exception in one UI
   * handler neither stops the others nor breaks the socket's onmessage.
   * 单个处理函数出错不影响其它处理函数。
   *
   * @param {string} type - Frame type.
   * @param {Object} payload - Frame object passed to the handlers.
   */
  emit(type, payload) {
    (this.handlers.get(type) || []).forEach((fn) => {
      try { fn(payload); } catch (err) { console.error('handler failed', type, err); }
    });
  }

  /**
   * Open the socket and install its event handlers.
   *
   * Called once by app.js at start-up and then again by the onclose handler
   * after each disconnect, so it must be safe to call repeatedly: every call
   * creates a new WebSocket object and the old one is simply dropped.
   *
   * @returns {LiveChannel} `this`.
   */
  connect() {
    this.socket = new WebSocket(this.url);

    /** On open: reset back-off, report status, flush queued messages, start heartbeat.
*/
    this.socket.onopen = () => {
      this.retry = 0;
      this.emit('status', { state: 'open' });
      // Deliver messages sent while disconnected, in their original order.
      // 按原顺序发送断线期间排队的消息。
      while (this.queue.length) this.socket.send(this.queue.shift());
      // Keep intermediaries from dropping an idle socket. 心跳保活。
      // The previous interval (from an earlier connection) is cleared first so
      // reconnects never stack several heartbeats.
      clearInterval(this.heartbeat);
      this.heartbeat = setInterval(() => this.send({ action: 'ping' }), 25000);
    };

    /** On message: parse JSON and dispatch to type-specific and '*' handlers. */
    this.socket.onmessage = (event) => {
      let frame;
      try { frame = JSON.parse(event.data); }
      catch (err) { return console.error('bad frame', event.data); }
      this.emit(frame.type, frame);
      this.emit('*', frame);
    };
```

```
    /** On close: stop heartbeat, report status, schedule a reconnect with back-off. */
    this.socket.onclose = () => {
      clearInterval(this.heartbeat);
      this.emit('status', { state: 'closed' });
      // Exponential back-off: 1 s, 1.7 s, 2.9 s, 4.9 s, 8.4 s, 14.2 s, then
      // capped at 15 s. `retry` is reset to 0 by the next successful onopen.
      // 指数退避：每次延迟乘以 1.7，上限 15 秒；连接成功后清零。
      const delay = Math.min(1000 * Math.pow(1.7, this.retry++), 15000);
      setTimeout(() => this.connect(), delay);
    };

    // A failed connection fires 'error' followed by 'close', so reconnection
    // is handled in onclose; here the UI is only informed.
    // 连接失败时先触发 error 再触发 close，重连逻辑在 onclose 中处理。
    this.socket.onerror = () => this.emit('status', { state: 'error' });
    return this;
  }

  /**
   * Send a message, or queue it until the socket is open.
   *
   * @param {Object} message - Client message, e.g. {action: 'plan', ...}
   *   or {action: 'ping'}; serialised with JSON.stringify.
   */
  send(message) {
    const payload = JSON.stringify(message);
    if (this.socket && this.socket.readyState === WebSocket.OPEN)
this.socket.send(payload);
    else this.queue.push(payload);
  }

  /**
   * Whether the socket is currently open.
   * @type {boolean}
   */
  get isOpen() {
    return this.socket && this.socket.readyState === WebSocket.OPEN;
  }
}
```

static/js/charts.js – metric charts

```
/* ============================================================================
 * charts.js - dependency-free canvas/SVG drawing helpers.
 * 轻量绘图工具：训练曲线与混淆矩阵，无第三方依赖。
 *
 * Place in the application: loaded by index.html before app.js; defines the
 * global `Charts` used by app.js for the "Model" tab (training curves,
 * confusion matrix) and the route summary (zone profile bar). No chart
 * library is bundled, so the page stays small and works offline.
 * Drawing uses a <canvas> 2D context (curves) and plain HTML <div> grids
 * styled by app.css (confusion matrix, zone bar).
 * 本文件定义全局对象 Charts，供 app.js 调用。
 * ============================================================================ */
'use strict';

/**
 * Small drawing helpers.
 * Built as an IIFE so that only the three public functions are exported.
 * @namespace Charts
 */
const Charts = (() => {

  /**
   * Draw the training/validation curves on a canvas. 绘制训练曲线。
   *
   * Loss (blue) and accuracy (teal) share the plot area but use separate
   * vertical scales: loss is scaled between its own min and max (labels on
   * the left axis) and accuracy between its own min and max (percent labels
   * on the right axis). Solid lines = training set, dashed = validation set.
   * 损失与准确率共用绘图区，但各自按最小/最大值缩放（左轴为损失，右轴为准确率）。
   *
   * @param {HTMLCanvasElement} canvas - Target canvas; its width/height
   *   attributes give the CSS size in pixels.
   * @param {Object} history - Per-epoch arrays saved by the trainer:
   *   epoch, train_loss, val_loss, train_accuracy, val_accuracy (equal length).
   *   Nothing is drawn if the history is missing or empty.
   */
```

```
function trainingCurves(canvas, history) {
  if (!canvas || !history || !history.epoch || !history.epoch.length) return;
  const ctx = canvas.getContext('2d');
  // High-DPI support: enlarge the backing store by devicePixelRatio, keep the
  // CSS size, and scale the context so drawing code can use CSS pixels.
  // The canvas attributes are overwritten, so this function is meant to be
  // called once per canvas (app.js calls it once, at page load).
  // 高分屏适配：按像素比放大画布缓冲区，保持 CSS 尺寸不变。
  const dpr = window.devicePixelRatio || 1;
  const W = canvas.width, H = canvas.height;
  canvas.width = W * dpr; canvas.height = H * dpr;
  canvas.style.width = W + 'px'; canvas.style.height = H + 'px';
  ctx.scale(dpr, dpr);
  ctx.clearRect(0, 0, W, H);

  // Padding around the plot area (left/right leave room for axis labels);
  // iw/ih = inner width/height of the plot area.
  const pad = { l: 34, r: 34, t: 10, b: 20 };
  const iw = W - pad.l - pad.r, ih = H - pad.t - pad.b;
  const n = history.epoch.length;

  // Common range of both loss curves, and of both accuracy curves.
  const losses = history.train_loss.concat(history.val_loss);
  const lMin = Math.min(...losses), lMax = Math.max(...losses);
  const accs = history.train_accuracy.concat(history.val_accuracy);
  const aMin = Math.min(...accs), aMax = Math.max(...accs);

  // grid
  // Five horizontal grid lines (0 %, 25 %, 50 %, 75 %, 100 % of the height).
  ctx.strokeStyle = '#2a3c4f'; ctx.lineWidth = 1;
  ctx.beginPath();
  for (let i = 0; i <= 4; i++) {
    const y = pad.t + (ih * i) / 4;
    ctx.moveTo(pad.l, y); ctx.lineTo(pad.l + iw, y);
  }
  ctx.stroke();

  /**
   * Draw one series as a poly-line.
   * x: epochs evenly spread over the plot width (max(n-1, 1) avoids a
   * division by zero for a single epoch).
   * y: value mapped linearly from [min, max] to [bottom, top]; canvas y
   * grows downwards, hence `pad.t + ih - ...`. 1e-9 guards a flat series.
   * @param {number[]} values - Series values.
   * @param {number} min - Value drawn at the bottom edge.
   * @param {number} max - Value drawn at the top edge.
   * @param {string} colour - Stroke colour.
   * @param {number[]} [dash] - Canvas dash pattern; omitted = solid line.
   */
  const line = (values, min, max, colour, dash) => {
    ctx.strokeStyle = colour; ctx.lineWidth = 1.6;
    ctx.setLineDash(dash || []);
    ctx.beginPath();
    values.forEach((v, i) => {
      const x = pad.l + (iw * i) / Math.max(n - 1, 1);
      const y = pad.t + ih - (ih * (v - min)) / Math.max(max - min, 1e-9);
      // First point starts the path, the rest extend it.
      i ? ctx.lineTo(x, y) : ctx.moveTo(x, y);
    });
    ctx.stroke();
    ctx.setLineDash([]);
  };

  line(history.train_loss,     lMin, lMax, '#4da3ff');
  line(history.val_loss,       lMin, lMax, '#4da3ff', [4, 3]);
  line(history.train_accuracy, aMin, aMax, '#2a9d8f');
  line(history.val_accuracy,   aMin, aMax, '#2a9d8f', [4, 3]);

  // Axis labels: loss range on the left, accuracy range (in %) on the right,
  // first/last epoch below, and a two-entry colour key at the top.
  // 坐标标注：左侧损失范围，右侧准确率范围，下方为首末轮次。
  ctx.fillStyle = '#94a7bb'; ctx.font = '9px sans-serif';
  ctx.textAlign = 'right';
  ctx.fillText(lMax.toFixed(2), pad.l - 4, pad.t + 7);
  ctx.fillText(lMin.toFixed(2), pad.l - 4, pad.t + ih);
  ctx.textAlign = 'left';
  ctx.fillText((aMax * 100).toFixed(0) + '%', pad.l + iw + 4, pad.t + 7);
  ctx.fillText((aMin * 100).toFixed(0) + '%', pad.l + iw + 4, pad.t + ih);
  ctx.textAlign = 'center';
  ctx.fillText('epoch 1', pad.l + 16, H - 6);
```

```
    ctx.fillText('epoch ' + n, pad.l + iw - 16, H - 6);
    ctx.fillStyle = '#4da3ff'; ctx.textAlign = 'left';
    ctx.fillText('— loss', pad.l + 4, pad.t + 9);
    ctx.fillStyle = '#2a9d8f';
    ctx.fillText('— accuracy', pad.l + 48, pad.t + 9);
  }

  /**
   * Render the confusion matrix as a coloured grid. 绘制混淆矩阵。
   *
   * Rows are true classes, columns predicted classes; each cell shows the raw
   * count. Colour intensity is the row-normalised share (count / row total,
   * i.e. the recall contribution): diagonal cells (correct) are teal,
   * off-diagonal cells (errors) red, both more opaque for larger shares.
   * The share in percent is shown as the cell tooltip. Layout uses the
   * .cm-row / .cm-cell / .cm-head classes of app.css.
   * 行为真实类别，列为预测类别；颜色深浅按行归一化比例，对角线为绿色，其余为红色。
   *
   * @param {HTMLElement} container - Element that receives the grid.
   * @param {number[][]} matrix - Square count matrix from the model metrics.
   * @param {string[]} labels - Class codes in matrix order; shortened for
   *   display ('OPEN_SEA' -> 'OPEN SEA', 'NEAR_COAST' -> 'NEAR CST').
   */
  function confusionMatrix(container, matrix, labels) {
    if (!container || !matrix) return;
    const short = labels.map((l) => l.replace('_SEA', ' SEA').replace('_COAST', '
CST'));
    const rowTotals = matrix.map((r) => r.reduce((a, b) => a + b, 0));
    // Header row: corner cell + one column header per predicted class.
    let html = '<div class="cm-row"><div class="cm-cell cm-head">true \\ pred</div>' +
      short.map((s) => `<div class="cm-cell cm-head">${s}</div>`).join('') + '</div>';

    matrix.forEach((row, i) => {
      html += `<div class="cm-row"><div class="cm-cell cm-head">${short[i]}</div>`;
      row.forEach((value, j) => {
        const share = rowTotals[i] ? value / rowTotals[i] : 0;
        // Alpha ranges: diagonal 0.18..0.90, off-diagonal 0.10..0.90.
        const colour = i === j
          ? `rgba(42,157,143,${0.18 + share * 0.72})`
          : `rgba(239,100,97,${0.10 + share * 0.80})`;
        html += `<div class="cm-cell" style="background:${colour}" ` +
                `title="${(share * 100).toFixed(1)}%">${value}</div>`;
      });
      html += '</div>';
    });
    container.innerHTML = html;
  }

  /**
   * Stacked horizontal bar of the zone profile. 区域占比条。
   *
   * One <div> per zone with a non-zero count, its width set to the zone's
   * percentage of all waypoints, so the segments fill the bar (a flex row in
   * app.css). An optional legend lists the non-empty zones with percentages.
   *
   * @param {HTMLElement} container - The bar element (#zone-bar).
   * @param {?HTMLElement} legend - Legend element (#zone-legend) or null.
   * @param {Object<string, number>} profile - Waypoint count per zone code
   *   (RoutePlan.zone_profile).
   * @param {Object<string, string>} colours - Zone code -> CSS colour.
   */
  function zoneBar(container, legend, profile, colours) {
    if (!container) return;
    // `|| 1` avoids a division by zero for an empty profile.
    const total = Object.values(profile).reduce((a, b) => a + b, 0) || 1;
    container.innerHTML = Object.entries(profile)
      .map(([code, n]) => n
        ? `<div style="width:${(100 * n / total).toFixed(2)}%;background:$
{colours[code]}"
            title="${code}: ${n} (${(100 * n / total).toFixed(1)}%)"></div>`
        : '')
      .join('');
    if (legend) {
      legend.innerHTML = Object.entries(profile)
        .filter(([, n]) => n > 0)
        .map(([code, n]) =>
          `<span><i style="background:${colours[code]}"></i>${code} ${(100 * n /
total).toFixed(1)}%</span>`)
        .join('');
    }
```

```
  }

  // Public interface of the Charts namespace.
  return { trainingCurves, confusionMatrix, zoneBar };
})();
```

static/js/app.js – the application controller

```
/* ============================================================================
 * app.js - application controller: map interaction, live planning, exports.
 * 主控脚本：地图交互、实时规划、结果展示与导出。
 *
 * Place in the application: the main script of the single-page client
 * (index.html). It is loaded last, after vendor/leaflet/leaflet.js (global L),
 * js/api.js (globals Api, LiveChannel) and js/charts.js (global Charts), and
 * runs once when the page loads. It has no exports; everything it defines is
 * used inside this file.
 *
 * Sections (marked by the dashed banners below):
 *   map             Leaflet map, base layers, offline tile fallback, overlay groups
 *   elements        cached DOM references
 *   helpers         number formatting, status pills, marker icons
 *   map picking     departure/destination selection by click or drag
 *   cursor          live zone read-out under the mouse pointer (WebSocket)
 *   live channel    WebSocket frame handlers (progress, overlay, result, error)
 *   render result   drawing the route and the summary tables
 *   actions         buttons, sliders, preset chips
 *   tabs, upload, model panel, legend, bootstrap
 * 本文件是前端主控脚本，按上述分区组织。
 * ============================================================================ */
'use strict';

/* Diverging hypsometric ramp: water blue -> land ochre, lightness monotone
   within each pole (see config.py for the validated separation figures).
   发散型等深-等高配色，与服务端 CLASS_COLOR_DARK 保持一致。 */
/**
 * Zone class code -> CSS colour. Keys are the class codes returned by the
 * server (ClassificationResult.class_code, RouteLeg.class_code).
 * @type {Object<string, string>}
 */
const ZONE_COLOURS = {
  OPEN_SEA:    '#3d7fb8',
  COASTAL_SEA: '#7cc3e3',
  NEAR_COAST:  '#d9b877',
  COASTLINE:   '#a9763f',
};

/**
 * Mutable UI state shared by all handlers of this file.
 * 前端全局状态。
 *
 * @property {?number[]} start - Departure [lat, lon] or null.
 * @property {?number[]} end - Destination [lat, lon] or null.
 * @property {string} pickNext - Which endpoint the next map click sets: 'start' |
'end'.
 * @property {?Object} plan - Last rendered RoutePlan (as received from the server).
 * @property {?string} uploadToken - Token of the last upload ('upload-<session_id>').
 * @property {Object<string, L.LayerGroup>} layers - Overlay groups: overlay, points,
route, markers.
 * @property {boolean} liveBusy - True while a cursor 'classify' request is in flight;
 *   prevents sending a new one before the answer arrives. Cleared only by a
 *   'classified' frame.
 */
const State = {
  start: null,
  end: null,
  pickNext: 'start',
  plan: null,
  uploadToken: null,
  layers: {},
  liveBusy: false,
};

/* ---------------------------------------------------------------- map */
/**
 * The Leaflet map, centred on the Baltic Sea at zoom 5.
 * worldCopyJump keeps markers on the visible world copy when the user pans
 * across the antimeridian. 地图对象，初始视图为波罗的海。
```

```
 * @type {L.Map}
 */
const map = L.map('map', { zoomControl: true, worldCopyJump: true })
  .setView([56.5, 17.0], 5);

/* Two base layers: the public OSM chart and an on-board layer rendered by the
   server from the neural network itself, so the tool also works offline.
   两种底图：在线 OSM 海图，以及由服务端神经网络渲染的离线随船底图。 */
/** Online OpenStreetMap tiles (require internet access). @type {L.TileLayer} */
const osmLayer = L.tileLayer('https://tile.openstreetmap.org/{z}/{x}/{y}.png', {
  maxZoom: 18, attribution: '&copy; OpenStreetMap contributors',
});
/**
 * Offline tiles from GET /api/tiles/zones/{z}/{x}/{y}.png (web/tiles.py).
 * maxZoom 10 matches the server-side zoom limit; minZoom 2 avoids rendering
 * the few whole-world tiles, which carry little information.
 * @type {L.TileLayer}
 */
const onboardLayer = L.tileLayer('/api/tiles/zones/{z}/{x}/{y}.png', {
  maxZoom: 10, minZoom: 2, attribution: 'On-board zone classification (this model)',
});
/* The dark backdrop behind the tiles is set in CSS on .leaflet-container.
   It must NOT be drawn as an L.rectangle: vector overlays live in the
   overlayPane (z-index 400), above the tilePane (z-index 200), so an opaque
   world-covering rectangle hides every basemap tile, online and offline alike.
   底图背景色由 CSS 的 .leaflet-container 提供，不能用矩形绘制：
   矢量图层位于 overlayPane(z-index 400)，会完全遮挡 tilePane(z-index 200) 的瓦片。 */
osmLayer.addTo(map);
// Base-layer switcher (radio buttons); no optional overlays are listed ({}).
// 底图切换控件（仅底图，无叠加层）。
L.control.layers(
  { 'OpenStreetMap (online)': osmLayer, 'On-board chart (offline)': onboardLayer },
  {}, { position: 'topleft' }
).addTo(map);

/* If the public tile service is unreachable, switch to the on-board chart.
   在线瓦片不可用时自动切换到离线底图。 */
/** Number of OSM tiles that failed to load since page start. @type {number} */
let tileErrors = 0;
/**
 * Offline fallback: on the 4th failed OSM tile (a single failure may be a
 * transient network error) the online layer is replaced by the on-board
 * chart. The strict `=== 4` makes the switch happen once; later errors do
 * nothing. The hasLayer check leaves the choice alone if the user has already
 * switched layers manually.
 * 第 4 次瓦片加载失败时切换一次；若用户已手动切换则不再处理。
 */
osmLayer.on('tileerror', () => {
  if (++tileErrors === 4 && map.hasLayer(osmLayer)) {
    map.removeLayer(osmLayer);
    onboardLayer.addTo(map);
    console.info('Tile service unreachable - switched to the on-board chart.');
  }
});

// Overlay groups, added in drawing order (later groups are drawn on top):
//   overlay - classified lattice from the 'overlay' WebSocket frame
//   points  - classified points of an uploaded AIS file
//   route   - the planned route, its casing and the great-circle reference
//   markers - draggable departure (A) and destination (B) markers
// Each group can be cleared independently with clearLayers().
// 图层组按绘制顺序添加，后添加的位于上方，可分别清空。
State.layers.overlay  = L.layerGroup().addTo(map);
State.layers.points   = L.layerGroup().addTo(map);
State.layers.route    = L.layerGroup().addTo(map);
State.layers.markers  = L.layerGroup().addTo(map);

/* ------------------------------------------------------------ elements */
/**
 * Shorthand for document.getElementById.
 * @param {string} id - Element id.
 * @returns {?HTMLElement}
 */
const $ = (id) => document.getElementById(id);
/**
 * DOM elements used by the handlers, looked up once at start-up.
 * Keys are camelCase versions of the element ids in index.html.
 * 启动时一次性缓存页面元素引用。
 * @type {Object<string, HTMLElement>}
 */
```

```
const els = {
  wsStatus: $('ws-status'), modelPill: $('model-pill'),
  startLat: $('start-lat'), startLon: $('start-lon'),
  endLat: $('end-lat'), endLon: $('end-lon'),
  algorithm: $('algorithm'), resolution: $('resolution'), resOut: $('res-out'),
  speed: $('speed'), speedOut: $('speed-out'),
  showOverlay: $('show-overlay'), liveClassify: $('live-classify'),
  btnPlan: $('btn-plan'), btnBenchmark: $('btn-benchmark'), btnClear: $('btn-clear'),
  progressBlock: $('progress-block'), progressStage: $('progress-stage'),
  progressPct: $('progress-pct'), progressFill: $('progress-fill'),
  progressDetail: $('progress-detail'),
  resultBlock: $('result-block'), metricGrid: $('metric-grid'), coastCheck: $('coast-
check'),
  zoneBar: $('zone-bar'), zoneLegend: $('zone-legend'),
  compareBlock: $('compare-block'), benchmarkBlock: $('benchmark-block'),
  modelBlock: $('model-block'), confusion: $('confusion'), historyChart: $('history-
chart'),
  historyList: $('history-list'), dbStats: $('db-stats'),
  uploadReport: $('upload-report'), pointsExport: $('points-export'),
  dropzone: $('dropzone'), fileInput: $('file-input'), dropzoneText: $('dropzone-text'),
  cursorReadout: $('cursor-readout'), mapLegend: $('map-legend'), pickHint: $('pick-
hint'),
};

/* ------------------------------------------------------------ helpers */
/**
 * Format a number with fixed decimals; missing values become an em dash.
 * The server sends null for infinite/undefined metrics (e.g. an infeasible
 * route), which is why null/undefined/NaN are handled explicitly.
 * @param {?number} v - Value.
 * @param {number} [digits=1] - Decimal places.
 * @returns {string}
 */
const fmt = (v, digits = 1) =>
  (v === null || v === undefined || Number.isNaN(v)) ? '—' : Number(v).toFixed(digits);

/**
 * Format an integer with thousands separators ('12,345'); missing -> em dash.
 * @param {?number} v - Value.
 * @returns {string}
 */
const fmtInt = (v) =>
  (v === null || v === undefined) ? '—' : Number(v).toLocaleString('en-US');

/**
 * Update a status pill in the header (text and colour class).
 * @param {HTMLElement} element - The pill element.
 * @param {string} text - Text to show.
 * @param {string} cls - One of 'pill-ok', 'pill-wait', 'pill-bad' (see app.css).
 */
function setStatus(element, text, cls) {
  element.textContent = text;
  element.className = 'pill ' + cls;
}

/**
 * Build a round lettered marker icon (A = departure, B = destination).
 * An L.divIcon is used instead of the default image marker so no icon image
 * files are needed. iconAnchor [11, 11] puts the centre of the 22 px circle
 * on the coordinate.
 * @param {string} colour - Fill colour.
 * @param {string} label - Letter drawn in the circle.
 * @returns {L.DivIcon}
 */
function markerIcon(colour, label) {
  return L.divIcon({
    className: '',
    html: `<div style="background:${colour};color:#06202f;font:700 11px sans-serif;
           width:22px;height:22px;border-radius:50%;display:flex;align-items:center;
           justify-content:center;border:2px solid #e6edf5;box-shadow:0 2px 6px #0008">$
{label}</div>`,
    iconSize: [22, 22], iconAnchor: [11, 11],
  });
}

/* ------------------------------------------------------------ map picking */
/**
 * Set the departure or destination, copy it into the form inputs (4 decimals,
 * about 11 m) and redraw the markers.
 * @param {'start'|'end'} which - Endpoint to set.
```

```
 * @param {number} lat - Latitude in degrees.
 * @param {number} lon - Longitude in degrees.
 */
function setEndpoint(which, lat, lon) {
  State[which] = [lat, lon];
  if (which === 'start') {
    els.startLat.value = lat.toFixed(4);
    els.startLon.value = lon.toFixed(4);
  } else {
    els.endLat.value = lat.toFixed(4);
    els.endLon.value = lon.toFixed(4);
  }
  redrawEndpoints();
}

/**
 * Redraw the A/B markers from State.start / State.end.
 * The markers are draggable; on 'dragend' the new position is written back
 * through setEndpoint, which redraws both markers again.
 */
function redrawEndpoints() {
  State.layers.markers.clearLayers();
  if (State.start) {
    L.marker(State.start, { icon: markerIcon('#7fe3b0', 'A'), draggable: true })
      .on('dragend', (e) => { const p = e.target.getLatLng(); setEndpoint('start',
p.lat, p.lng); })
      .bindTooltip('Departure').addTo(State.layers.markers);
  }
  if (State.end) {
    L.marker(State.end, { icon: markerIcon('#ffb4b2', 'B'), draggable: true })
      .on('dragend', (e) => { const p = e.target.getLatLng(); setEndpoint('end', p.lat,
p.lng); })
      .bindTooltip('Destination').addTo(State.layers.markers);
  }
}

/**
 * Take the endpoints from the four coordinate inputs (the user may have typed
 * them) and redraw the markers. Empty inputs give NaN; this is checked by
 * the Plan button handler before a request is sent.
 */
function readEndpointsFromInputs() {
  State.start = [parseFloat(els.startLat.value), parseFloat(els.startLon.value)];
  State.end   = [parseFloat(els.endLat.value),   parseFloat(els.endLon.value)];
  redrawEndpoints();
}

/**
 * Map click: set the endpoint named by State.pickNext, then alternate between
 * departure and destination and update the hint text below the inputs.
 * 点击地图交替设置起点与终点。
 */
map.on('click', (event) => {
  const { lat, lng } = event.latlng;
  setEndpoint(State.pickNext, lat, lng);
  State.pickNext = State.pickNext === 'start' ? 'end' : 'start';
  els.pickHint.innerHTML = State.pickNext === 'start'
    ? 'Next click sets the <b>departure</b>. Markers can also be dragged.'
    : 'Next click sets the <b>destination</b>. Markers can also be dragged.';
});

/* ---------------------------------------------- live cursor read-out */
/** Debounce timer id for the cursor classification. @type {?number} */
let liveTimer = null;
/**
 * Mouse move: always show the pointer coordinates; if the "Live zone read-out" checkbox
is on,
 * ask the server for the zone under the pointer.
 *
 * Two throttles keep the load low: the request is debounced (sent only after
 * the pointer has rested for 160 ms) and State.liveBusy allows at most one
 * request in flight. The answer arrives as a 'classified' frame (handler
 * below). `channel` is declared further down; that is fine because this
 * handler only runs after the whole script has executed.
 * 鼠标停留 160 ms 后才发送分类请求，且同一时间只允许一个请求。
 */
map.on('mousemove', (event) => {
  const { lat, lng } = event.latlng;
  els.cursorReadout.querySelector('.coord').textContent =
    `${lat.toFixed(3)}, ${lng.toFixed(3)}`;
```

```
  if (!els.liveClassify.checked) return;
  clearTimeout(liveTimer);
  liveTimer = setTimeout(() => {
    if (State.liveBusy || !channel.isOpen) return;
    State.liveBusy = true;
    channel.send({ action: 'classify', points: [[lat, lng]] });
  }, 160);
});

/* ------------------------------------------------------ live channel */
/**
 * The single WebSocket connection to /ws/plan (see LiveChannel in api.js),
 * used for route planning with progress and for the cursor read-out.
 * @type {LiveChannel}
 */
const channel = new LiveChannel('/ws/plan');

/**
 * Connection state -> header pill. 'closed' is shown as "reconnecting"
 * because LiveChannel always schedules a reconnect after a close.
 */
channel.on('status', ({ state }) => {
  if (state === 'open')  setStatus(els.wsStatus, 'live connection', 'pill-ok');
  if (state === 'closed') setStatus(els.wsStatus, 'reconnecting…', 'pill-wait');
  if (state === 'error')  setStatus(els.wsStatus, 'connection error', 'pill-bad');
});

/**
 * 'classified' frame: answer to a cursor classification request.
 * Shows the zone colour swatch, class code, confidence and, if known, the
 * reference depth/elevation (depth_m > 0 means land elevation, < 0 water depth).
 * 显示指针处的区域类别、置信度与水深/高程。
 */
channel.on('classified', (frame) => {
  State.liveBusy = false;
  const r = frame.results && frame.results[0];
  if (!r) return;
  const depth = r.depth_m === null ? '' : ` · ${r.depth_m > 0 ? 'elev' : 'depth'} $
{Math.abs(r.depth_m).toFixed(0)} m`;
  els.cursorReadout.querySelector('.zone').innerHTML =
    `<i style="display:inline-block;width:9px;height:9px;border-radius:2px;margin-
right:5px;
        background:${ZONE_COLOURS[r.class_code]}"></i>${r.class_code}
     <span style="opacity:.7">${(r.confidence * 100).toFixed(0)}%${depth}</span>`;
});

/**
 * 'started' frame: the server accepted a planning request. Shows and resets
 * the progress block and disables the Plan button until 'result' or 'error'.
 * The bar starts at 2 % so that it is visibly present.
 */
channel.on('started', (frame) => {
  els.progressBlock.hidden = false;
  els.btnPlan.disabled = true;
  els.progressStage.textContent = `starting ${frame.algorithm.toUpperCase()}…`;
  els.progressFill.style.width = '2%';
  els.progressPct.textContent = '0%';
  els.progressDetail.textContent = '';
});

/**
 * Human-readable labels for the `stage` field of 'progress' frames
 * (stage names are defined by the planner, cost-map builder and search modules).
 * Unknown stages are shown with their raw name.
 * @type {Object<string, string>}
 */
const STAGE_TEXT = {
  grid:     'Building geographic lattice',
  classify: 'Neural network classifying grid cells',
  cost_map: 'Cost map ready',
  search:   'Searching optimal path',
  astar:    'A* search in progress',
  dijkstra: 'Dijkstra search in progress',
  genetic:  'Genetic algorithm: evolving routes',
  dynamic:  'Dynamic programming: solving the Bellman equation',
  refine:   'Refining the lattice (narrow strait)',
  coast:    'Coastline check: correcting the route',
  done:     'Route ready',
};
```

```
/**
 * 'progress' frame: {stage, progress (0..1 within the stage), detail}.
 * The progress fraction restarts for every stage, so the bar shows the
 * progress of the current stage, not of the whole job.
 * The detail line depends on which keys the stage supplies:
 *   cells_done/cells_total -> neural-network classification of the lattice;
 *   expanded/frontier      -> A* or Dijkstra search;
 *   generation/best_cost   -> genetic algorithm;
 *   zone_shares            -> cost-map statistics (share of each zone);
 *   stage                  -> plain text message (grid, search, refine, done).
 * 根据 detail 中出现的字段选择不同的进度说明文字。
 */
channel.on('progress', (frame) => {
  const pct = Math.round(frame.progress * 100);
  els.progressStage.textContent = STAGE_TEXT[frame.stage] || frame.stage;
  els.progressFill.style.width = Math.max(pct, 2) + '%';
  els.progressPct.textContent = pct + '%';
  const d = frame.detail || {};
  if (d.cells_done !== undefined) {
    els.progressDetail.textContent =
      `${fmtInt(d.cells_done)} / ${fmtInt(d.cells_total)} cells classified`;
  } else if (d.expanded !== undefined) {
    els.progressDetail.textContent =
      `${fmtInt(d.expanded)} nodes expanded · frontier ${fmtInt(d.frontier)}`;
  } else if (d.sweep !== undefined) {
    // Dynamic programming: value-iteration sweep and cells whose value changed.
    els.progressDetail.textContent =
      `sweep ${fmtInt(d.sweep)} · ${fmtInt(d.updated)} values improved`;
  } else if (d.generation !== undefined) {
    // Genetic algorithm: generation counter and best route cost so far.
    els.progressDetail.textContent =
      `generation ${fmtInt(d.generation)} · best cost ${fmt(d.best_cost, 0)}`;
  } else if (d.zone_shares) {
    els.progressDetail.textContent = Object.entries(d.zone_shares)
      .map(([k, v]) => `${k} ${(v * 100).toFixed(0)}%`).join(' · ');
  } else if (d.stage) {
    els.progressDetail.textContent = d.stage;
  }
});

/**
 * 'overlay' frame: the classified lattice used for planning, down-sampled by
 * the server to about 6 000 cells {lat, lon, c (class index), p (confidence)}.
 * Each cell is drawn as a borderless, non-interactive rectangle centred on
 * (lat, lon) at 22 % opacity so the base map stays visible. The previous
 * overlay is always removed, even if drawing is switched off.
 * 绘制降采样后的分类网格，每个单元为以中心点为中心的半透明矩形。
 */
channel.on('overlay', (frame) => {
  State.layers.overlay.clearLayers();
  if (!els.showOverlay.checked) return;
  // cell_size_deg already accounts for the server-side down-sampling step, so
  // the rectangles tile the area seamlessly. 单元尺寸已含降采样步长，绘制无缝。
  const size = frame.cell_size_deg || frame.grid.resolution_deg;
  // Class index -> code, in the model's output order (config.CLASS_NAMES).
  const codes = ['OPEN_SEA', 'COASTAL_SEA', 'NEAR_COAST', 'COASTLINE'];
  const half = size / 2;
  frame.cells.forEach((cell) => {
    L.rectangle(
      [[cell.lat - half, cell.lon - half], [cell.lat + half, cell.lon + half]],
      { stroke: false, fillColor: ZONE_COLOURS[codes[cell.c]],
        fillOpacity: 0.22, interactive: false }
    ).addTo(State.layers.overlay);
  });
});

/**
 * 'result' frame: the finished RoutePlan. Completes the progress bar, hides
 * it after 0.9 s, renders the plan and refreshes the history list and the
 * database statistics (the route has just been stored).
 */
channel.on('result', (frame) => {
  els.btnPlan.disabled = false;
  els.progressFill.style.width = '100%';
  els.progressPct.textContent = '100%';
  els.progressStage.textContent = 'Route ready';
  setTimeout(() => { els.progressBlock.hidden = true; }, 900);
  renderPlan(frame.plan);
  loadHistory();
  loadDbStats();
```

```
});

/**
 * 'error' frame: show the server's message in the progress block and
 * re-enable the Plan button. The block stays visible so the text can be read.
 */
channel.on('error', (frame) => {
  // A failed cursor read-out must not block the next one. 光标实时分类失败时解除占用标志。
  State.liveBusy = false;
  els.btnPlan.disabled = false;
  els.progressStage.textContent = 'Error';
  els.progressDetail.textContent = frame.detail;
  els.progressFill.style.width = '100%';
});

// Open the socket; LiveChannel reconnects by itself after any disconnect.
// 建立 WebSocket 连接，断线后自动重连。
channel.connect();

/* ------------------------------------------------------- render result */
/**
 * Draw a planned route on the map and fill the summary panels.
 *
 * Layers drawn into State.layers.route, bottom to top:
 *   1. dark casing polyline (9 px) for contrast against any base map;
 *   2. one 5 px segment per leg, coloured by the zone of the leg's end point;
 *   3. a thin white centre line along the waypoints;
 *   4. the dashed red great-circle line from start to end (the naive
 *      reference route, which may cross land).
 * An infeasible plan only shows its message.
 * Also sets the export links (GeoJSON, CSV, PDF) and fills the comparison tab.
 * 在地图上绘制航线并填充结果摘要。
 *
 * @param {Object} plan - RoutePlan.to_dict() from the server (see app.py).
 */
function renderPlan(plan) {
  State.plan = plan;
  State.layers.route.clearLayers();

  renderCoastCheck(plan);
  if (!plan.feasible || !plan.waypoints.length) {
    els.resultBlock.hidden = false;
    els.metricGrid.innerHTML =
      `<div class="metric bad" style="grid-column:1/-1"><div
class="label">Infeasible</div>
       <div class="value" style="font-size:13px">${plan.message || 'No navigable
route'}</div></div>`;
    return;
  }

  // A dark casing is drawn first so that the route reads against any basemap.
  // The route is coloured by zone, and the on-board chart uses the same palette,
  // so without a casing a coastal leg would vanish into coastal water.
  // 先绘制深色描边：航线按区域着色，而离线底图用的是同一套配色，
  // 没有描边时沿岸航段会完全融进浅蓝色海面。
  L.polyline(plan.waypoints, {
    color: '#06121f', weight: 9, opacity: 0.55, lineJoin: 'round', lineCap: 'round',
  }).addTo(State.layers.route);

  // Optimised route, coloured per zone segment. 按区域分段着色。
  for (let i = 1; i < plan.legs.length; i++) {
    const a = plan.legs[i - 1], b = plan.legs[i];
    L.polyline([[a.latitude, a.longitude], [b.latitude, b.longitude]], {
      color: ZONE_COLOURS[b.class_code] || '#4da3ff', weight: 5, opacity: 1,
      lineJoin: 'round', lineCap: 'round',
    }).addTo(State.layers.route);
  }
  // A bright hairline down the middle keeps the track legible when zoomed out.
  L.polyline(plan.waypoints, { color: '#ffffff', weight: 1.6, opacity: 0.9 })
    .addTo(State.layers.route);

  // Great-circle reference. 大圆参考航线。
  L.polyline([plan.start, plan.end], {
    color: '#ff5d5a', weight: 2.6, opacity: 0.95, dashArray: '8,6',
  }).bindTooltip('Great-circle (shortest distance)').addTo(State.layers.route);

  // Points where the route still crosses land on the independent coastline
  // (only when the automatic correction could not remove them).
  // 经自动修正后仍位于陆地上的航线采样点，以红色圆点标出。
  const cc = plan.coast_check || {};
```

```
  if (cc.verdict === 'crossing' && plan.algorithm !== 'great_circle') {
    (cc.land_points || []).forEach((p) => {
      L.circleMarker(p, { radius: 4, color: '#ff2d55', weight: 2, fillOpacity: 0.85 })
        .bindTooltip('Over land on the independent
coastline').addTo(State.layers.route);
    });
  }

  // Zoom to the route with 18 % padding on each side. 缩放至航线范围。
  map.fitBounds(L.polyline(plan.waypoints).getBounds().pad(0.18));

  // Summary metric tiles. A detour ratio (route length / great-circle length)
  // above 1.5 is flagged with the 'warn' colour.
  // 结果指标；绕航系数大于 1.5 时以警示色显示。
  const detour = plan.detour_ratio;
  els.metricGrid.innerHTML = `
    ${metric('Route length', fmt(plan.distance_km, 1), 'km')}
    ${metric('Great circle', fmt(plan.great_circle_km, 1), 'km')}
    ${metric('Detour ratio', fmt(detour, 3), '×', detour > 1.5 ? 'warn' : 'good')}
    ${metric('Weighted cost', fmt(plan.total_cost, 0), '')}
    ${metric('Passage time', fmt(plan.estimated_hours, 1), 'h')}
    ${metric('Planning time', fmt(plan.runtime_s, 3), 's')}
    ${metric({ genetic: 'Routes evaluated', dynamic: 'Bellman updates' }[plan.algorithm]
|| 'Nodes expanded',
             fmtInt(plan.nodes_expanded), '')}
    ${metric('Waypoints', fmtInt(plan.waypoints.length), '')}`;

  Charts.zoneBar(els.zoneBar, els.zoneLegend, plan.zone_profile, ZONE_COLOURS);

  // Export links point to GET /api/export/{route_id}.{fmt}; the server keeps
  // the plan in memory (last 64 plans) for this purpose.
  // 导出链接；服务端在内存中保留最近 64 条航线用于导出。
  $('dl-geojson').href = `/api/export/${plan.route_id}.geojson`;
  $('dl-csv').href     = `/api/export/${plan.route_id}.csv`;
  $('dl-pdf').href     = `/api/export/${plan.route_id}.pdf`;
  els.resultBlock.hidden = false;

  renderComparison(plan);
}

/**
 * Banner with the independent coastline check of the route (plan.coast_check):
 *   clear / grazing        -> green, "passed";
 *   corrected              -> amber, how the first route was corrected;
 *   crossing               -> red, the server's warning text;
 *   unchecked / absent     -> hidden (land mask not installed).
 * 显示独立海岸线检查结果的横幅。
 *
 * @param {Object} plan - RoutePlan.to_dict().
 */
function renderCoastCheck(plan) {
  const c = plan.coast_check || {};
  const box = els.coastCheck;
  if (!c.verdict || c.verdict === 'unchecked' || !plan.feasible) { box.hidden = true;
return; }
  box.hidden = false;
  if (c.verdict === 'crossing') {
    box.className = 'verdict bad';
    box.textContent = plan.algorithm === 'great_circle'
      ? `Coastline check: the great circle crosses land for ${fmt(c.max_land_run_km, 0)}
km.`
      : `⚠️ ${plan.warning}`;
    return;
  }
  const corrected = c.initial_verdict === 'crossing';
  box.className = corrected ? 'verdict fixed' : 'verdict good';
  const how = [];
  if (c.corrections) {
    how.push(`${c.corrections} re-search${c.corrections > 1 ? 'es' : ''}, ` +
             `${c.cells_blocked} cell${c.cells_blocked === 1 ? '' : 's'} blocked`);
  }
  if (c.refined) how.push(`finer lattice ${plan.grid.resolution_deg}°`);
  box.textContent = corrected
    ? `Coastline check: the first route crossed land for $
{fmt(c.initial_max_land_run_km, 0)} km ` +
      `and was corrected automatically (${how.join('; ')}). The final route passes the
check.`
    : `Coastline check passed: ${c.verdict === 'clear' ? 'no land on the route'
        : `touches the coast for at most ${fmt(c.max_land_run_km, 0)} km`} ` +
      `(independent 1 km land mask, ${fmtInt(c.samples)} samples).`;
```

```
}

/**
 * HTML for one metric tile of the result grid.
 * @param {string} label - Caption above the value.
 * @param {string} value - Already formatted value.
 * @param {string} unit - Unit shown after the value (may be '').
 * @param {string} [cls=''] - Extra class: 'good', 'warn' or 'bad' (colour).
 * @returns {string} HTML string.
 */
function metric(label, value, unit, cls = '') {
  return `<div class="metric ${cls}"><div class="label">${label}</div>
          <div class="value">${value} <small>${unit}</small></div></div>`;
}

/**
 * Fill the "Compare" tab: the optimised route versus the great-circle
 * baseline computed by the server (plan.baseline).
 *
 * The baseline samples points along the great circle and classifies them;
 * `land_samples / samples` and `land_share` show how much of the direct line
 * would cross land, and `verdict` is the server's one-line conclusion.
 * "Passage time saved/lost" is the optimised passage time minus the
 * great-circle time at the same speed (knots * 1.852 = km/h); a positive
 * value is the extra time the safe route costs.
 * 对比页：优化航线与大圆航线的距离、陆地占比和航行时间差。
 *
 * @param {Object} plan - RoutePlan.to_dict().
 */
function renderComparison(plan) {
  const b = plan.baseline || {};
  const navigable = b.is_navigable;
  els.compareBlock.innerHTML = `
    <table>
      <tr><td>Great-circle length</td><td>${fmt(b.distance_km, 1)} km</td></tr>
      <tr><td>Optimised length</td><td>${fmt(plan.distance_km, 1)} km</td></tr>
      <tr><td>Additional distance</td><td>${fmt(b.extra_distance_pct, 2)} %</td></tr>
      <tr><td>Samples over land</td><td>${b.land_samples} / ${b.samples}</td></tr>
      <tr><td>Share over land</td><td>${fmt((b.land_share || 0) * 100, 1)} %</td></tr>
      <tr><td>Optimised: intermediate waypoints over land</td>
          <td>${plan.interior_land_waypoints}</td></tr>
      <tr><td>Optimised: coastline check (1 km land mask)</td>
          <td>${(plan.coast_check || {}).verdict || '-'}${(plan.coast_check ||
{}).max_land_run_km
              ? `, longest land stretch ${fmt(plan.coast_check.max_land_run_km, 1)} km`
: ''}</td></tr>
      <tr><td>Optimised: waypoints in
total</td><td>${fmtInt(plan.waypoints.length)}</td></tr>
      <tr><td>Passage time saved/lost</td><td>${fmt(plan.estimated_hours -
(b.distance_km / (plan.speed_knots * 1.852)), 1)} h</td></tr>
    </table>
    <div class="verdict ${navigable ? 'good' : 'bad'}">${b.verdict || ''}</div>`;
}

/* ------------------------------------------------------------ actions */
/**
 * Build a planning request from the form.
 *
 * The same object is used for the WebSocket ('plan' action) and for the REST
 * fallback / benchmark; the server ignores the keys it does not need
 * (action, include_overlay for REST). simplify_km is not sent, so the server
 * default (0 = no simplification) applies.
 * 构造规划请求，WebSocket 与 REST 共用同一对象。
 *
 * @returns {Object} Request body with action, coordinates, algorithm,
 *   resolution_deg, speed_knots, include_overlay and persist.
 */
function planRequest() {
  readEndpointsFromInputs();
  return {
    action: 'plan',
    start_lat: State.start[0], start_lon: State.start[1],
    end_lat: State.end[0],     end_lon: State.end[1],
    algorithm: els.algorithm.value,
    resolution_deg: parseFloat(els.resolution.value),
    speed_knots: parseFloat(els.speed.value),
    include_overlay: els.showOverlay.checked,
    persist: true,
  };
}
```

```
/**
 * "Build route" button (#btn-plan): validate that all four coordinates are numbers,
then
 * send the request over the WebSocket (progress is shown by the frame
 * handlers above) or, if the socket is down, use the blocking REST endpoint
 * POST /api/route without progress.
 */
els.btnPlan.addEventListener('click', () => {
  const request = planRequest();
  if ([request.start_lat, request.start_lon, request.end_lat,
request.end_lon].some(Number.isNaN)) {
    alert('Please set both the departure and the destination.');
    return;
  }
  if (channel.isOpen) {
    channel.send(request);
  } else {
    // Fall back to REST when the socket is down. WebSocket 不可用时退回 REST。
    els.btnPlan.disabled = true;
    Api.plan(request)
      .then(renderPlan)
      .catch((err) => alert('Planning failed: ' + err.message))
      .finally(() => { els.btnPlan.disabled = false; });
  }
});

/**
 * "Compare algorithms" button (#btn-benchmark): run all four methods on one shared cost
map
 * (POST /api/benchmark), switch to the Compare tab and show a table with
 * distance, cost, expanded nodes and search time per algorithm, followed by
 * the derived ratios computed by the server. '⚠️' marks an infeasible row
 * (e.g. a great circle crossing land). `??` shows '—' for null ratios.
 * 基准测试：同一代价地图上比较四种方法。
 */
els.btnBenchmark.addEventListener('click', async () => {
  const request = planRequest();
  els.btnBenchmark.disabled = true;
  els.benchmarkBlock.textContent = 'running five methods on one shared cost map…';
  // Programmatic click on the tab button reuses the tab-switch handler.
  document.querySelector('[data-tab="compare"]').click();
  try {
    const data = await Api.benchmark(request);
    els.benchmarkBlock.innerHTML = `
      <table>
        <tr><td><b>Algorithm</b></td><td><b>km / cost / nodes / search s</b></td></tr>
        ${data.results.map((r) => `<tr><td>${r.algorithm}${r.feasible ? '' : '
⚠️'}</td><td>
          ${fmt(r.distance_km, 0)} / ${r.total_cost === null ? '—' : fmt(r.total_cost,
0)} /
          ${fmtInt(r.nodes_expanded)} / ${fmt(r.runtime_s, 3)}</td></tr>`).join('')}
        <tr><td>A* node reduction vs Dijkstra</td><td>${data.astar_node_reduction ??
'—'}×</td></tr>
        <tr><td>A* speed-up (time)</td><td>${data.astar_time_ratio ?? '—'}×</td></tr>
        <tr><td>Cost difference A* vs Dijkstra</td><td>${data.optimality_gap_pct ?? '—'}
%</td></tr>
        <tr><td>Genetic algorithm above the optimum</td><td>${data.genetic_gap_pct ??
'—'} %</td></tr>
        <tr><td>Cost difference DP vs A*</td><td>${data.dynamic_gap_pct ?? '—'}
%</td></tr>
        <tr><td>Cost map build</td><td>${fmt(data.cost_map_build_s, 3)} s
(shared)</td></tr>
        <tr><td>Grid</td><td>${data.grid.n_rows}×${data.grid.n_cols} @ $
{data.grid.resolution_deg}°</td></tr>
      </table>
      <div class="muted" style="margin-top:6px">Runtimes exclude the shared cost-map
      construction, so they compare the search algorithms only.</div>`;
  } catch (err) {
    els.benchmarkBlock.textContent = 'Benchmark failed: ' + err.message;
  } finally {
    els.btnBenchmark.disabled = false;
  }
});

/**
 * "Clear" button (#btn-clear): remove every overlay (lattice, points, route, markers),
 * forget the endpoints and the plan and reset the result panels. The
 * coordinate inputs keep their values.
 */
```

```
els.btnClear.addEventListener('click', () => {
  Object.values(State.layers).forEach((layer) => layer.clearLayers());
  State.start = State.end = State.plan = null;
  State.pickNext = 'start';
  els.resultBlock.hidden = true;
  els.compareBlock.textContent = 'Build a route first.';
  els.cursorReadout.querySelector('.zone').textContent = 'move the pointer over the
chart';
});

/**
 * "Show zone classification layer" checkbox (#show-overlay): unchecking removes the
current overlay;
 * checking it has effect from the next planning run (the overlay is not
 * kept on the client).
 */
els.showOverlay.addEventListener('change', () => {
  if (!els.showOverlay.checked) State.layers.overlay.clearLayers();
});

/** Resolution slider: mirror the value next to the label (degrees). */
els.resolution.addEventListener('input', () => {
  els.resOut.textContent = parseFloat(els.resolution.value).toFixed(2) + '°';
});
/** Speed slider: mirror the value next to the label (knots). */
els.speed.addEventListener('input', () => {
  els.speedOut.textContent = parseFloat(els.speed.value).toFixed(1) + ' kn';
});

/**
 * Preset voyage chips: data-preset="startLat,startLon,endLat,endLon".
 * Fills the inputs, redraws the markers and zooms to both ports with 40 %
 * padding. Planning is not started automatically. An optional
 * data-resolution sets the lattice step recommended for that voyage (the
 * island-dense Aegean Sea needs 0.1°, see subsection 3.5.5 of the thesis);
 * the other presets restore the default of 0.25°.
 * 预设航线按钮：填入坐标并缩放地图，不自动规划；data-resolution 给出该航线推荐的
 * 网格分辨率（岛屿密集的爱琴海需要 0.1°），其余预设恢复默认 0.25°。
 */
document.querySelectorAll('.chip[data-preset]').forEach((chip) => {
  chip.addEventListener('click', () => {
    const [a, b, c, d] = chip.dataset.preset.split(',').map(Number);
    els.startLat.value = a; els.startLon.value = b;
    els.endLat.value = c;   els.endLon.value = d;
    els.resolution.value = chip.dataset.resolution || '0.25';
    els.resolution.dispatchEvent(new Event('input'));     // refresh the label
    readEndpointsFromInputs();
    map.fitBounds(L.latLngBounds([[a, b], [c, d]]).pad(0.4));
  });
});

/* ---------------------------------------------------------------- tabs */
/**
 * Tab bar of the side panel: a button with data-tab="X" activates the page
 * with id "tab-X"; exactly one button and one page carry the 'active' class.
 * 选项卡切换：data-tab="X" 对应 id 为 "tab-X" 的页面。
 */
document.querySelectorAll('.tab').forEach((tab) => {
  tab.addEventListener('click', () => {
    document.querySelectorAll('.tab').forEach((t) => t.classList.remove('active'));
    document.querySelectorAll('.tab-page').forEach((p) => p.classList.remove('active'));
    tab.classList.add('active');
    $('tab-' + tab.dataset.tab).classList.add('active');
  });
});

/* ------------------------------------------------------------- upload */
// Clicking the drop zone opens the hidden <input type="file">.
els.dropzone.addEventListener('click', () => els.fileInput.click());
// preventDefault on dragenter/dragover is required, otherwise the browser
// does not allow a drop and would open the file instead. The 'drag' class
// highlights the zone while a file is held over it.
// 必须阻止 dragenter/dragover 的默认行为，否则浏览器会直接打开文件。
['dragenter', 'dragover'].forEach((type) =>
  els.dropzone.addEventListener(type, (e) => {
    e.preventDefault(); els.dropzone.classList.add('drag');
  }));
['dragleave', 'drop'].forEach((type) =>
  els.dropzone.addEventListener(type, (e) => {
    e.preventDefault(); els.dropzone.classList.remove('drag');
```

```
  }));
// Second 'drop' listener: hand the first dropped file to the upload routine.
els.dropzone.addEventListener('drop', (e) => {
  if (e.dataTransfer.files.length) handleUpload(e.dataTransfer.files[0]);
});
// File chosen through the browse dialog.
els.fileInput.addEventListener('change', () => {
  if (els.fileInput.files.length) handleUpload(els.fileInput.files[0]);
});

/**
 * Upload an AIS file, show the loading/validation report and draw the
 * classified points.
 *
 * The server returns at most about 6 000 thinned points; each is drawn as a
 * small borderless circle in its zone colour with a tooltip showing class
 * and confidence. The map is zoomed to the points, and the points export
 * links (GeoJSON/CSV of all stored points) are enabled.
 * 上传 AIS 文件，显示校验报告并在地图上绘制分类后的点。
 *
 * @param {File} file - The file to upload.
 * @returns {Promise<void>} Errors are shown in the report panel, not thrown.
 */
async function handleUpload(file) {
  els.dropzoneText.textContent = `uploading ${file.name}…`;
  els.uploadReport.hidden = false;
  els.uploadReport.textContent = 'classifying points with the neural network…';
  try {
    const data = await Api.upload(file);
    State.uploadToken = data.token;
    // `|| 1` avoids division by zero in the percentages of an empty histogram.
    const total = Object.values(data.histogram).reduce((a, b) => a + b, 0) || 1;
    // Report table; the label agreement row appears only if the file had labels.
    // 报告表；仅当文件带标签时才显示一致率。
    els.uploadReport.innerHTML = `
      <table>
        <tr><td>File</td><td>${data.report.source}</td></tr>
        <tr><td>Features read</td><td>${fmtInt(data.report.n_features)}</td></tr>
        <tr><td>Valid points</td><td>${fmtInt(data.report.n_valid)}</td></tr>
        <tr><td>Dropped</td><td>${fmtInt(data.report.n_dropped)}</td></tr>
        <tr><td>Classified</td><td>${fmtInt(data.classified)}</td></tr>
        ${data.accuracy_vs_labels ? `<tr><td>Agreement with labels</td>
          <td>${(data.accuracy_vs_labels.agreement * 100).toFixed(2)} %
          (n=${fmtInt(data.accuracy_vs_labels.n_labelled)})</td></tr>` : ''}
        ${Object.entries(data.histogram).map(([k, v]) =>
          `<tr><td>${k}</td><td>${fmtInt(v)} (${(100 * v /
total).toFixed(1)}%)</td></tr>`).join('')}
      </table>`;
    els.dropzoneText.textContent = `${file.name} — ${fmtInt(data.classified)} points
classified`;

    State.layers.points.clearLayers();
    data.points.forEach((p) => {
      L.circleMarker([p.latitude, p.longitude], {
        radius: 2.6, stroke: false,
        fillColor: ZONE_COLOURS[p.class_code], fillOpacity: 0.75,
      }).bindTooltip(`${p.class_code} · ${(p.confidence * 100).toFixed(0)}%`)
        .addTo(State.layers.points);
    });
    if (data.points.length) {
      map.fitBounds(L.latLngBounds(data.points.map((p) => [p.latitude,
p.longitude])).pad(0.15));
    }
    $('dl-points-geojson').href = `/api/export/points/${data.token}.geojson`;
    $('dl-points-csv').href     = `/api/export/points/${data.token}.csv`;
    els.pointsExport.hidden = false;
    loadDbStats();
  } catch (err) {
    els.uploadReport.textContent = 'Upload failed: ' + err.message;
    els.dropzoneText.textContent = 'Drop a file here or click to browse';
  }
}

/* ------------------------------------------------------------ model panel */
/**
 * Load GET /api/model and fill the "Model" tab: the header pill with the test
 * accuracy, a table of architecture and test metrics (accuracy, macro F1,
 * Cohen's kappa, macro ROC-AUC), per-class F1, the confusion matrix and the
 * training curves. If the model is missing (HTTP 503) the pill turns red and
 * the error text is shown instead.
```

```
 * 加载模型信息并填充“模型”页。
 * @returns {Promise<void>}
 */
async function loadModel() {
  try {
    const data = await Api.model();
    const s = data.summary, m = data.metrics || {};
    setStatus(els.modelPill,
      `model ${(s.accuracy * 100).toFixed(1)}% acc`, 'pill-ok');
    els.modelBlock.innerHTML = `
      <table>
        <tr><td>Architecture</td><td>${s.architecture}</td></tr>
        <tr><td>Input features</td><td>${s.n_features}</td></tr>
        <tr><td>Parameters</td><td>${fmtInt(s.parameters)}</td></tr>
        <tr><td>Reference samples</td><td>${fmtInt(s.reference_samples)}</td></tr>
        <tr><td>Test accuracy</td><td>${fmt(m.accuracy * 100, 2)} %</td></tr>
        <tr><td>Macro F1</td><td>${fmt(m.macro_f1, 4)}</td></tr>
        <tr><td>Cohen κ</td><td>${fmt(m.cohen_kappa, 4)}</td></tr>
        <tr><td>Macro ROC-AUC</td><td>${fmt(m.macro_roc_auc, 4)}</td></tr>
        <tr><td>Trained</td><td>${s.trained_utc || '—'}</td></tr>
      </table>
      ${m.per_class ? '<h4>Per class F1</h4><table>' + Object.entries(m.per_class)
        .map(([k, v]) => `<tr><td>${k}</td><td>${fmt(v.f1, 4)}</td></tr>`).join('') +
'</table>' : ''}`;
    Charts.confusionMatrix(els.confusion, m.confusion_matrix,
      ['OPEN_SEA', 'COASTAL_SEA', 'NEAR_COAST', 'COASTLINE']);
    Charts.trainingCurves(els.historyChart, data.history);
  } catch (err) {
    setStatus(els.modelPill, 'model unavailable', 'pill-bad');
    els.modelBlock.textContent = 'Model not available: ' + err.message;
  }
}

/**
 * Load GET /api/statistics and show the database counters and file size in
 * the "Data" tab. Called at start-up, after each route and each upload.
 * @returns {Promise<void>}
 */
async function loadDbStats() {
  try {
    const s = await Api.statistics();
    els.dbStats.innerHTML = `<table>
      <tr><td>Sessions</td><td>${fmtInt(s.sessions)}</td></tr>
      <tr><td>Stored points</td><td>${fmtInt(s.points)}</td></tr>
      <tr><td>Stored routes</td><td>${fmtInt(s.routes)}</td></tr>
      <tr><td>Waypoints</td><td>${fmtInt(s.waypoints)}</td></tr>
      <tr><td>Database size</td><td>${fmt(s.database_size_kb, 1)} kB</td></tr>
    </table>`;
  } catch (err) { els.dbStats.textContent = 'unavailable'; }
}

/**
 * Load the 25 most recent stored routes (GET /api/routes?limit=25) into the
 * history list. Clicking an item fetches that route (GET /api/routes/{uid})
 * and draws its waypoints as a plain blue line; stored routes are shown
 * without zone colouring and without updating the result panels.
 * 加载历史航线列表，点击条目在地图上显示该航线。
 * @returns {Promise<void>}
 */
async function loadHistory() {
  try {
    const data = await Api.routes(25);
    els.historyList.innerHTML = data.routes.map((r) => `
      <div class="history-item" data-uid="${r.route_uid}">
        <div class="row1"><span>${r.algorithm.toUpperCase()}</span>
          <span>${fmt(r.distance_km, 0)} km</span></div>
        <div class="row2">${r.created_utc} · ${fmtInt(r.n_waypoints)} wp ·
          ${fmt(r.runtime_s, 3)} s · detour ${fmt(r.detour_ratio, 3)}×</div>
      </div>`).join('') || '<div class="muted">No routes stored yet.</div>';
    // Handlers are attached after every refresh because innerHTML replaced the items.
    // 每次刷新后重新绑定点击事件（innerHTML 已替换元素）。
    els.historyList.querySelectorAll('.history-item').forEach((item) => {
      item.addEventListener('click', async () => {
        const route = await Api.route(item.dataset.uid);
        State.layers.route.clearLayers();
        const coords = route.waypoints.map((w) => [w.latitude, w.longitude]);
        if (coords.length) {
          L.polyline(coords, { color: '#4da3ff', weight: 4 }).addTo(State.layers.route);
          map.fitBounds(L.polyline(coords).getBounds().pad(0.2));
        }
```

```
      });
    });
  } catch (err) { els.historyList.textContent = 'unavailable'; }
}

// Manual refresh button of the history list.
$('btn-refresh-history').addEventListener('click', loadHistory);

/* ------------------------------------------------------------ legend */
// Map legend built from GET /api/zones: colour swatch, class code and the
// traversal cost (or "impassable"), plus the great-circle line colour.
// 由 /api/zones 生成地图图例。
Api.zones().then((data) => {
  els.mapLegend.innerHTML = data.classes.map((z) =>
    `<div><i style="background:${z.colour}"></i>${z.code}
      ${z.navigable ? `(cost ${z.cost})` : '(impassable)'}</div>`).join('') +
    `<div style="margin-top:5px"><i style="background:#ef6461"></i>great circle</div>`;
}).catch(() => { els.mapLegend.textContent = 'legend unavailable'; });

/* ------------------------------------------------------------ bootstrap */
// Initial load: draw markers for the default coordinates in the form and
// fetch the model panel, database statistics and route history in parallel.
// 页面启动：绘制默认起终点并加载模型、统计与历史数据。
readEndpointsFromInputs();
loadModel();
loadDbStats();
loadHistory();
```

D.9 Command-line scripts

scripts/prepare_dataset.py

```
#!/usr/bin/env python3
"""Command-line entry point for dataset preparation.

用法：python scripts/prepare_dataset.py --input data/raw/merged.geojson

First step of the offline pipeline (prepare -> train -> evaluate). It runs
:func:`maritime_route.data.dataset.prepare`, which loads and validates the
labelled GeoJSON corpus, removes mislabelled points, splits it 70/15/15,
builds the reference index and writes the feature matrices. Afterwards run
``scripts/train_model.py``.

Options
-------
``--input``     labelled GeoJSON corpus (default ``data/raw/merged.geojson``)
``--output``    processed ``.npz`` (default ``data/processed/zone_dataset.npz``)
``--index``     reference index ``.npz`` (default ``models/reference_index.npz``)
``--limit N``   read only the first N features, for quick experiments
``--no-clean``  keep points whose class contradicts their height/depth

Output on stdout: JSON with the split sizes, the class distribution and the
cleaning report.

离线流程第一步：加载、校验、清洗、划分数据并提取特征；随后运行 train_model.py。
"""
from __future__ import annotations

import argparse
import json
import logging
import sys
from pathlib import Path

# Make ``src`` importable when the script is run directly from a checkout
# (the package does not need to be installed). parents[1] = project root.
# 直接运行脚本时将 src 加入导入路径（无需安装包）。
sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))

from maritime_route.config import BATHY_INDEX_PATH, DEFAULT_RAW_DATASET
from maritime_route.data.dataset import PROCESSED_NPZ, prepare
```

```
def main() -> int:
    """Parse the command line, run the preparation and print a summary.

    Returns
    -------
    int
        Process exit code, 0 on success (errors propagate as exceptions).

    解析命令行参数，执行预处理并输出摘要。
    """
    parser = argparse.ArgumentParser(description="Prepare the zone classification
dataset")
    parser.add_argument("--input", default=str(DEFAULT_RAW_DATASET), help="raw GeoJSON
file")
    parser.add_argument("--output", default=str(PROCESSED_NPZ), help="processed .npz
file")
    parser.add_argument("--index", default=str(BATHY_INDEX_PATH), help="reference
index .npz")
    parser.add_argument("--limit", type=int, default=None, help="only read N features")
    parser.add_argument("--no-clean", action="store_true",
                        help="keep the points whose class contradicts their
height/depth")
    args = parser.parse_args()

    # INFO level so that the progress messages of prepare() are visible.
    # 使用 INFO 级别以显示 prepare() 的进度信息。
    logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %
(message)s")
    split, stats = prepare(args.input, args.output, args.index, limit=args.limit,
                           clean=not args.no_clean)
    print(json.dumps({"split": split.describe(),
                      "classes": stats.get("class_distribution", {}),
                      "cleaning": stats.get("cleaning")}, indent=2))
    return 0


if __name__ == "__main__":
    # SystemExit passes main()'s return value to the shell as the exit code.
    # 将 main() 的返回值作为进程退出码。
    raise SystemExit(main())
```

scripts/train_model.py

```
#!/usr/bin/env python3
"""Train the geographic-zone classifier.

用法: python scripts/train_model.py [--epochs 120]

Second step of the offline pipeline. Loads the matrices written by
``scripts/prepare_dataset.py``, trains the network with
:func:`maritime_route.model.trainer.train_model` and saves the weights, the
feature scaler and the metadata to ``models/``. Prints the per-class report
and the headline test metrics.

Options
-------
``--dataset``     processed ``.npz`` (default ``data/processed/zone_dataset.npz``)
``--epochs``      maximum number of epochs (default from ``TrainingConfig``)
``--batch-size``  mini-batch size
``--lr``          initial learning rate of AdamW

All other hyper-parameters come from :data:`maritime_route.config.TRAINING`.

离线流程第二步：读取预处理结果，训练网络并保存权重、标准化器与元数据。
"""
from __future__ import annotations

import argparse
import json
import logging
import sys
from dataclasses import replace
from pathlib import Path

# Make ``src`` importable without installing the package. 将 src 加入导入路径。
sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))

from maritime_route.config import TRAINING
```

```
from maritime_route.data.dataset import PROCESSED_NPZ, load_prepared
from maritime_route.model.trainer import train_model


def main() -> int:
    """Parse options, train the model and print the test metrics.

    Returns
    -------
    int
        Process exit code, 0 on success.

    解析参数、训练模型并输出测试集指标。
    """
    parser = argparse.ArgumentParser(description="Train the zone classifier")
    parser.add_argument("--dataset", default=str(PROCESSED_NPZ))
    parser.add_argument("--epochs", type=int, default=TRAINING.epochs)
    parser.add_argument("--batch-size", type=int, default=TRAINING.batch_size)
    parser.add_argument("--lr", type=float, default=TRAINING.learning_rate)
    args = parser.parse_args()

    logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %
(message)s")
    split = load_prepared(args.dataset)
    # TrainingConfig is frozen: replace() returns a copy with the overridden
    # fields and leaves the shared default untouched.
    # TrainingConfig 为冻结数据类：replace() 返回修改后的副本，不影响默认配置。
    cfg = replace(TRAINING, epochs=args.epochs, batch_size=args.batch_size,
                  learning_rate=args.lr)
    _, _, metrics = train_model(split, cfg)
    print(metrics["report_text"])
    # Headline metrics only; the full set is in models/model_metadata.json.
    # 仅输出主要指标，完整指标见 models/model_metadata.json。
    print(json.dumps({k: metrics[k] for k in
                      ("accuracy", "macro_f1", "weighted_f1", "cohen_kappa",
"macro_roc_auc")},
                     indent=2))
    return 0


if __name__ == "__main__":
    # Exit code of the process = return value of main(). 进程退出码。
    raise SystemExit(main())
```

scripts/evaluate_regions.py

```
#!/usr/bin/env python3
"""Accuracy of the trained classifier in each of the eleven sea regions.

The corpus was collected in eleven separate sea regions (see
``maritime_route.config.REGIONS``). A single overall accuracy can hide a
region where the model is weak, so this script evaluates the held-out test
split region by region and reports, for every region:

* the number of test points,
* the accuracy,
* the number of *water <-> land* confusions - the only errors that matter for
  safety, because they can turn land into navigable water or the reverse.

The result is written to ``models/region_metrics.json`` and printed as a table.

Usage::

    python scripts/evaluate_regions.py

Prerequisites: ``scripts/prepare_dataset.py`` (which stores the region of
every test point as ``region_test``) and ``scripts/train_model.py``. The test
features are already computed, so the script only standardises them and runs
the network; it does not rebuild features.

按十一个海域分别评估测试集：样本数、准确率，以及对航行安全最关键的"水域↔陆地"误判数。
"""
from __future__ import annotations

import json
import sys
from pathlib import Path
```

```
# Make ``src`` importable without installing the package. 将 src 加入导入路径。
sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))

import numpy as np

from maritime_route.config import CLASS_NAMES, LAND_CLASSES, MODEL_DIR, WATER_CLASSES
from maritime_route.data.dataset import PROCESSED_NPZ
from maritime_route.model.inference import ZoneClassifierService

#: Result file (JSON). 结果文件。
OUTPUT = MODEL_DIR / "region_metrics.json"


def main() -> int:
    """Evaluate the test split per region and save the table.

    A *water <-> land* error is a prediction on the other side of the
    shoreline from the true class (e.g. COASTAL_SEA predicted as NEAR_COAST).
    Confusions inside one side (OPEN_SEA vs COASTAL_SEA, NEAR_COAST vs
    COASTLINE) only change the traversal cost and are not counted.

    Returns
    -------
    int
        Exit code: 0 on success, 1 if the prepared dataset lacks the
        ``region_test`` array (dataset prepared by an older version).

    按海域评估测试集并保存结果；水陆两侧之间的误判才计为"水域↔陆地"错误。
    """
    # allow_pickle=True is needed to read the string arrays of the archive.
    # 读取字符串数组需要 allow_pickle=True。
    blob = np.load(PROCESSED_NPZ, allow_pickle=True)
    if "region_test" not in blob:
        print("The prepared dataset has no region column - rerun
scripts/prepare_dataset.py")
        return 1
    x_test, y_test, regions = blob["x_test"], blob["y_test"], blob["region_test"]

    service = ZoneClassifierService()
    # _forward applies the saved scaler and the network to the unscaled
    # (n_test, 28) matrix and returns (n_test, 4) probabilities.
    # _forward 对未标准化的测试特征做标准化并前向计算，返回概率。
    y_pred = service._forward(x_test).argmax(axis=1)   # probabilities -> class index

    # Map every class index to "water" (True) or "land" (False).
    # 将类别索引映射为水域(True)或陆地(False)。
    is_water = np.array([name in WATER_CLASSES for name in CLASS_NAMES])
    # Fancy indexing: is_water[y] gives the water flag of every sample, shape
    # (n_test,). True where truth and prediction lie on different sides.
    # 按索引取出每个样本的水域标志；真实与预测位于水陆两侧时为 True。
    crossing = is_water[y_test] != is_water[y_pred]

    table = {}
    # Regions ordered by number of test points, largest first. 按样本数降序排列海域。
    for name in sorted(set(regions.tolist()), key=lambda r: -int((regions == r).sum())):
        m = regions == name   # boolean mask of the test points in this region 该海域的掩
码
        table[name] = {
            "n_test": int(m.sum()),
            "accuracy": float((y_pred[m] == y_test[m]).mean()),
            "water_land_errors": int(crossing[m].sum()),
        }
    summary = {
        "overall_accuracy": float((y_pred == y_test).mean()),
        "overall_water_land_errors": int(crossing.sum()),
        "n_test": int(len(y_test)),
        "regions": table,
    }
    OUTPUT.write_text(json.dumps(summary, indent=2, ensure_ascii=False), encoding="utf-
8")

    # Fixed-width console table: region name 42 chars, then right-aligned numbers.
    # 固定宽度的控制台表格。
    print(f"{'Region':42s} {'n':>6s} {'accuracy':>9s} {'water<->land':>13s}")
    for name, row in table.items():
        print(f"{name:42s} {row['n_test']:6d} {100 * row['accuracy']:8.2f}%
{row['water_land_errors']:13d}")
    print(f"{'All regions':42s} {summary['n_test']:6d} {100 *
summary['overall_accuracy']:8.2f}% "
          f"{summary['overall_water_land_errors']:13d}")
```

```
    print(f"\nwritten {OUTPUT}")
    return 0


if __name__ == "__main__":
    # Exit code of the process = return value of main(). 进程退出码。
    raise SystemExit(main())
```

scripts/benchmark_routing.py

```
#!/usr/bin/env python3
"""Routing benchmark used for chapter 3 of the explanatory note.

Six voyages in European waters are planned with every method of the
application - A*, Dijkstra, the genetic algorithm and the great circle - on
one shared cost map per voyage, and the node counts, run times and costs are
compared.

路径规划基准测试：在多条真实航线上比较 A*、Dijkstra、遗传算法与大圆航线。
Results are written to docs/figures/routing_benchmark.json

Method
------
For every voyage :func:`maritime_route.routing.planner.benchmark` first builds
(and if necessary refines) one cost map, then plans the voyage with each
method on that same map. The reported run times are ``time.perf_counter``
intervals of the planning call *excluding* classification and cost-map
construction, so they compare the searches themselves. Each case is run once;
run times therefore vary by a few per cent between executions, while node
counts and costs are deterministic (the GA uses a fixed seed).

Output
------
``routing_benchmark.json`` with one record per case (grid, per-method
distance in km, cost in weighted km, nodes expanded, run time in s,
``node_reduction`` = Dijkstra nodes / A* nodes, ``time_ratio`` = Dijkstra time
/ A* time, ``optimality_gap_pct`` between A* and Dijkstra, ``genetic_gap_pct``
of the GA above the A* optimum) and an ``aggregate`` block. The file is read
by ``scripts/make_figures.py`` to draw ``fig_routing_benchmark.png``.

Usage::

    python scripts/benchmark_routing.py [--output PATH]

The trained model must exist (``scripts/train_model.py``).
"""
from __future__ import annotations

import argparse
import json
import logging
import statistics
import sys
from pathlib import Path

# Make the package importable without installation (adds <project>/src).
sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))

from maritime_route.config import FIGURE_DIR
from maritime_route.model.inference import ZoneClassifierService
from maritime_route.routing.planner import RoutePlanner, benchmark

#: Representative voyages of increasing length. 代表性航线测试用例。
#: Each entry: (name, departure (lat, lon), destination (lat, lon), lattice
#: step in degrees). Longer ocean voyages use a coarser step to stay within
#: the cell budget of ``RoutingConfig.max_grid_cells``.
CASES = [
    ("Kiel - Tallinn (Baltic)",        (54.3233, 10.1228), (59.4370, 24.7536), 0.25),
    ("Helsinki - Copenhagen",          (60.1699, 24.9384), (55.6761, 12.5683), 0.25),
    ("Gdansk - Stockholm",             (54.3520, 18.6466), (59.3293, 18.0686), 0.25),
    ("Rotterdam - Lisbon (North Sea)", (51.9500,  4.1400), (38.7223, -9.1393), 0.40),
    ("Bergen - Reykjavik (Atlantic)",  (60.3913,  5.3221), (64.1466, -21.9426), 0.50),
    ("Brest - Gibraltar",              (48.3904, -4.4861), (36.1408, -5.3536), 0.40),
]


def main() -> int:
    """Run all benchmark cases, print a line per case and save the JSON.
```

```
    Returns
    -------
    int
        Process exit code (0).
    """
    parser = argparse.ArgumentParser(description="Benchmark the routing algorithms")
    parser.add_argument("--output", default=str(FIGURE_DIR / "routing_benchmark.json"))
    args = parser.parse_args()
    # WARNING level hides the per-grid INFO messages of the planner.
    logging.basicConfig(level=logging.WARNING)

    # One classifier instance (singleton) is shared by all cases.
    planner = RoutePlanner(ZoneClassifierService.instance())
    records = []
    for name, start, end, resolution in CASES:
        print(f"-> {name}", flush=True)
        data = benchmark(planner, start, end, resolution)
        # Index the per-method result rows by algorithm name.
        by_alg = {r["algorithm"]: r for r in data["results"]}
        records.append({
            "case": name,
            "start": list(start), "end": list(end),
            "grid": data["grid"],
            "cost_map_build_s": data["cost_map_build_s"],
            "astar": by_alg["astar"],
            "dijkstra": by_alg["dijkstra"],
            "genetic": by_alg["genetic"],
            "dynamic": by_alg["dynamic"],
            "dynamic_gap_pct": data["dynamic_gap_pct"],
            "genetic_gap_pct": data["genetic_gap_pct"],
            "great_circle": by_alg["great_circle"],
            "node_reduction": data["astar_node_reduction"],
            "time_ratio": data["astar_time_ratio"],
            "optimality_gap_pct": data["optimality_gap_pct"],
            "navigable_share": data["cost_map"]["navigable_share"],
        })
        a, d, g = by_alg["astar"], by_alg["dijkstra"], by_alg["great_circle"]
        print(f"   A*: {a['distance_km']:.0f} km, {a['nodes_expanded']:,} nodes, "
              f"{a['runtime_s']:.3f} s | Dijkstra: {d['nodes_expanded']:,} nodes, "
              f"{d['runtime_s']:.3f} s | GA gap: {data['genetic_gap_pct']} % | "
              f"GC: {g['distance_km']:.0f} km "
              f"({'navigable' if g['feasible'] else 'crosses land'})")

    # Aggregates over all cases. Ratios that are None (e.g. A* expanded no
    # node) are filtered out before averaging. mean_detour_pct is the extra
    # length of the A* route relative to the great-circle distance.
    # 汇总统计：对各测试用例求平均/最大值。
    summary = {
        "cases": records,
        "aggregate": {
            "mean_node_reduction": round(statistics.mean(
                [r["node_reduction"] for r in records if r["node_reduction"]]), 3),
            "mean_time_ratio": round(statistics.mean(
                [r["time_ratio"] for r in records if r["time_ratio"]]), 3),
            "max_optimality_gap_pct": max(
                [r["optimality_gap_pct"] or 0.0 for r in records]),
            "mean_detour_pct": round(statistics.mean(
                [100.0 * (r["astar"]["distance_km"] / r["great_circle"]["distance_km"] -
1.0)
                 for r in records]), 2),
            "genetic_feasible_cases": sum(1 for r in records if r["genetic"]
["feasible"]),
            "max_dynamic_gap_pct": max(r["dynamic_gap_pct"] or 0.0 for r in records),
            "mean_genetic_gap_pct": round(statistics.mean(
                [r["genetic_gap_pct"] for r in records if r["genetic_gap_pct"] is not
None]), 3),
            "great_circle_infeasible_cases": sum(
                1 for r in records if not r["great_circle"]["feasible"]),
            "n_cases": len(records),
        },
    }
    Path(args.output).write_text(json.dumps(summary, indent=2), encoding="utf-8")
    print("\nAggregate:", json.dumps(summary["aggregate"], indent=2))
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

scripts/regional_examples.py

```
#!/usr/bin/env python3
"""Example voyages in each of the eleven sea regions of the corpus.

The labelled corpus covers eleven separate sea regions (``config.REGIONS``). This
script plans one realistic port-to-port voyage inside every region and runs all
five methods of the application on the same cost map:

* A* - the default exact search,
* Dijkstra - the exact search without a heuristic,
* the genetic algorithm - the evolutionary search (fitness function),
* dynamic programming - value iteration on the Bellman equation (value function),
* the great circle - the naive shortest-distance reference.

It writes

* ``docs/figures/regional_routes.json`` - the numbers used in the thesis
  (distance, cost, nodes or fitness evaluations, run time, gap of the GA to the
  optimum, whether the great circle crosses land);
* ``docs/figures/fig_regional_routes.png`` - eleven map panels, each showing the
  classified lattice with the A*, genetic and great-circle routes.

Usage::

    python scripts/regional_examples.py

在十一个海域各规划一条真实港口到港口的航线，在同一张代价图上运行 A*、Dijkstra、
遗传算法与大圆航线四种方法，输出数值结果（JSON）与十幅地图（PNG）。

Before planning, the script asserts that both ports of every voyage fall into
the declared region (using the same region boxes as the data-cleaning step),
so the table in the thesis cannot silently mix regions.
"""
from __future__ import annotations

import json
import logging
import math
import sys
from pathlib import Path

# Make the package importable without installation (adds <project>/src).
sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))

import matplotlib

# Non-interactive backend: renders to files, works without a display.
# 使用无界面后端，仅输出图片文件。
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.colors import ListedColormap

from maritime_route.config import CLASS_COLOR, CLASS_NAMES, FIGURE_DIR
from maritime_route.data.cleaning import assign_regions
from maritime_route.model.inference import ZoneClassifierService
from maritime_route.routing.geodesy import great_circle_points
from maritime_route.routing.planner import RoutePlanner

#: One voyage per region: (region, voyage name, departure, destination[, lattice
#: step in degrees; the planner default of 0.25 when omitted]).
#: Coordinates are the approaches of real ports. 每个海域一条航线（真实港口外锚地坐标）。
VOYAGES = [
    ("North Europe (Baltic and North Sea)", "Rotterdam - Gdansk",
     (51.9500, 4.1400), (54.4000, 18.6600)),
    # The Aegean is planned at 0.1 deg: at the default 0.25 deg a diagonal step
    # between two water cells cuts across Attica, which is narrower than a cell.
    # 爱琴海用 0.1°：默认 0.25° 时对角步会穿过比网格还窄的阿提卡半岛。
    ("Aegean Sea", "Piraeus - Thessaloniki",
     (37.9200, 23.5800), (40.5500, 22.8800), 0.1),
    ("Black Sea and Eastern Mediterranean", "Constanta - Batumi",
     (44.1700, 28.6600), (41.6500, 41.6300)),
    ("Red Sea and Gulf of Aden", "Suez - Djibouti",
     (29.9300, 32.5500), (11.6000, 43.1400)),
    ("Persian Gulf and Arabian Sea", "Jebel Ali - Karachi",
     (25.0100, 55.0600), (24.8000, 66.9700)),
```

```
    ("Bay of Bengal", "Chennai - Chittagong",
     (13.1000, 80.3000), (22.2500, 91.8000)),
    ("Strait of Malacca and Andaman Sea", "Singapore - Phuket",
     (1.2600, 103.8200), (7.8800, 98.4000)),
    ("South China Sea", "Vung Tau - Hong Kong",
     (10.3500, 107.0800), (22.2800, 114.1700)),
    ("Japan and North-West Pacific", "Kochi - Yokohama",
     (33.4500, 133.6000), (35.3000, 139.7500)),
    ("Southern Africa (Cape of Good Hope)", "Cape Town - East London",
     (-33.9100, 18.4300), (-33.0300, 27.9200)),
    ("Central America (Caribbean coast)", "Colon - Puerto Cabezas",
     (9.3800, -79.9200), (14.0300, -83.3700)),
]

#: Methods run on every voyage, in this order (A* first builds the cost map).
METHODS = ("astar", "dijkstra", "genetic", "dynamic", "great_circle")


def run_voyage(planner: RoutePlanner, start, end, resolution=None):
    """Plan one voyage with all five methods on one shared cost map.

    The lattice is built (and refined if a strait is too narrow) once by A*;
    the other methods then reuse exactly the same cost map, so their costs and
    run times are directly comparable.
    先用 A* 建立（必要时细化）网格，其余方法复用同一张代价图，保证结果可比。

    Parameters
    ----------
    planner : RoutePlanner
        Planner with the trained classifier.
    start, end : tuple of float
        Departure and destination ``(lat, lon)``, degrees.
    resolution : float, optional
        Requested lattice step, degrees; the planner default when omitted.

    Returns
    -------
    tuple
        ``(plans, cost_map)`` - a dict method name -> RoutePlan, and the
        shared CostMap.
    """
    _, cost_map = planner.plan(start, end, "astar", resolution)
    # Resolution actually used after any refinement; passed on so that the
    # per-method plans report the same lattice.
    resolution = cost_map.spec.resolution_deg
    plans = {}
    for method in METHODS:
        plan, _ = planner.plan(start, end, method, resolution,
                               reuse_cost_map=cost_map, auto_refine=False)
        plans[method] = plan
    return plans, cost_map


def summarise(plans) -> dict:
    """Numbers reported in the thesis for one voyage. 论文中报告的单条航线数据。

    Parameters
    ----------
    plans : dict
        Method name -> RoutePlan, as returned by :func:`run_voyage`.

    Returns
    -------
    dict
        Per method: ``feasible``, ``distance_km``, ``total_cost`` (weighted
        km, None when infeasible or crossing land), ``nodes_expanded`` (for
        the GA: fitness evaluations) and ``runtime_s``; plus
        ``genetic_gap_pct`` (GA cost above the A* optimum, %),
        ``great_circle_land_share`` (share of great-circle samples on land)
        and ``detour_pct`` (A* length above the great-circle distance, %).
    """
    out = {}
    for method, plan in plans.items():
        # The great circle always "exists"; it is infeasible when it crosses land.
        crosses = method == "great_circle" and not (plan.baseline or
{}).get("is_navigable", True)
        out[method] = {
            "feasible": bool(plan.feasible and not crosses),
            "distance_km": round(plan.distance_km, 1),
            "total_cost": (round(plan.total_cost, 1)
```

```
                            if plan.feasible and not crosses and
math.isfinite(plan.total_cost)
                            else None),
            "nodes_expanded": int(plan.nodes_expanded),
            "runtime_s": round(plan.runtime_s, 3),
        }
    a, g = plans["astar"], plans["genetic"]
    # max(0, ...) removes a -0.00 caused by floating-point summation order.
    out["genetic_gap_pct"] = (max(0.0, round(100.0 * (g.total_cost - a.total_cost) /
a.total_cost, 2))
                              if g.feasible and a.feasible and a.total_cost > 0 else
None)
    gc = plans["great_circle"].baseline or {}
    out["great_circle_land_share"] = gc.get("land_share")
    out["detour_pct"] = round(100.0 * (a.distance_km / a.great_circle_km - 1.0), 2)
    return out


def draw(ax, cost_map, plans, title: str) -> None:
    """One map panel: classified lattice plus the three routes.

    The zone classes are drawn as an image in plain longitude/latitude axes
    (equirectangular, not a map projection). Dijkstra is not drawn: it has
    the same optimal cost as A* and would overlap its line.
    单幅子图：分类网格 + A*、遗传算法与大圆三条航线。

    Parameters
    ----------
    ax : matplotlib.axes.Axes
        Target panel.
    cost_map : CostMap
        Shared cost map of the voyage.
    plans : dict
        Method name -> RoutePlan.
    title : str
        Panel title.
    """
    spec = cost_map.spec
    # One colour per class index 0..3; vmin/vmax = -0.5/3.5 centre each
    # integer class on its colour.
    cmap = ListedColormap([CLASS_COLOR[c] for c in CLASS_NAMES])
    # Lattice nodes are cell centres, so the image extends half a step beyond
    # the first and last node on every side.
    extent = (spec.lon_min - spec.resolution_deg / 2, spec.lon_max + spec.resolution_deg
/ 2,
              spec.lat_min - spec.resolution_deg / 2, spec.lat_max + spec.resolution_deg
/ 2)
    # origin="lower": row 0 (southernmost latitude) at the bottom.
    ax.imshow(cost_map.class_index, origin="lower", cmap=cmap, vmin=-0.5, vmax=3.5,
              extent=extent, interpolation="nearest", aspect="auto")
    a, g = plans["astar"], plans["genetic"]
    gc = np.array(great_circle_points(a.start, a.end, 120))
    if not a.feasible:                      # nothing to draw but the lattice
        ax.set_title(title + " (no route)", fontsize=8)
        return
    # Plot x = longitude (column 1), y = latitude (column 0).
    ax.plot(gc[:, 1], gc[:, 0], ":", color="#d62728", lw=1.4, label="Great circle")
    if g.feasible:
        pts = np.array(g.waypoints)
        ax.plot(pts[:, 1], pts[:, 0], "--", color="#ff9f1c", lw=1.6, label="Genetic
algorithm")
    pts = np.array(a.waypoints)
    ax.plot(pts[:, 1], pts[:, 0], "-", color="#111111", lw=1.6, label="A* (optimal)")
    # [::-1] turns (lat, lon) into (lon, lat) = (x, y). Green circle =
    # departure, red square = destination.
    ax.plot(*a.start[::-1], "o", color="#2ca02c", ms=5)
    ax.plot(*a.end[::-1], "s", color="#d62728", ms=5)
    ax.set_title(title, fontsize=8)
    ax.tick_params(labelsize=6)
    ax.set_xlim(extent[0], extent[1])
    ax.set_ylim(extent[2], extent[3])


def main() -> int:
    """Run all voyages, save the table and the figure.

    Returns
    -------
    int
        Process exit code (0).
```

```
    Raises
    ------
    AssertionError
        If a port does not lie in the region its voyage is declared for.
    """
    logging.basicConfig(level=logging.WARNING)
    planner = RoutePlanner(ZoneClassifierService.instance())
    records = []
    # 4 x 3 panels on an A4-sized figure (inches): eleven voyages, the twelfth
    # panel holds the legend. 4 行 3 列子图：11 条航线，第 12 格放图例。
    fig, axes = plt.subplots(4, 3, figsize=(8.3, 10.2))
    for i, (region, name, start, end, *rest) in enumerate(VOYAGES):
        resolution = rest[0] if rest else None
        # Sanity check: both ends of the voyage lie inside the declared region.
        # 检查起终点确实位于所声明的海域内。
        assigned = assign_regions(np.array([start[0], end[0]]), np.array([start[1],
end[1]]))
        assert all(r == region for r in assigned), (name, assigned)
        print(f"-> {region}: {name}", flush=True)
        plans, cost_map = run_voyage(planner, start, end, resolution)
        row = {"region": region, "voyage": name, "start": list(start), "end": list(end),
               "requested_resolution_deg": resolution,
               "grid": cost_map.spec.to_dict(), **summarise(plans)}
        records.append(row)
        a, g = row["astar"], row["genetic"]
        print(f"   A* {a['distance_km']:.0f} km cost {a['total_cost']} | GA gap "
              f"{row['genetic_gap_pct']} % | GC land {row['great_circle_land_share']}")
        draw(axes.flat[i], cost_map, plans, f"{i + 1}. {name}")
    # The legend is built explicitly, because a panel may lack one of the lines.
    # 图例单独构建，因为某些子图可能缺少某条线（例如遗传算法未找到路线）。
    from matplotlib.lines import Line2D
    from matplotlib.patches import Patch
    handles = [Line2D([], [], color="#111111", lw=1.6, label="A* (optimal)"),
               Line2D([], [], color="#ff9f1c", lw=1.6, ls="--", label="Genetic
algorithm"),
               Line2D([], [], color="#d62728", lw=1.4, ls=":", label="Great circle")]
    handles += [Patch(color=CLASS_COLOR[c], label=c) for c in CLASS_NAMES]
    legend_ax = axes.flat[len(VOYAGES)]
    legend_ax.axis("off")
    legend_ax.legend(handles=handles, loc="center", ncol=1, fontsize=7.5, frameon=False)
    fig.tight_layout()
    FIGURE_DIR.mkdir(parents=True, exist_ok=True)
    fig.savefig(FIGURE_DIR / "fig_regional_routes.png", dpi=200)

    gaps = [r["genetic_gap_pct"] for r in records if r["genetic_gap_pct"] is not None]
    # Aggregates over the eleven voyages. astar_dijkstra_equal_cost checks that
    # the two exact methods agree on every voyage (they must: both are optimal).
    summary = {
        "voyages": records,
        "aggregate": {
            "n_voyages": len(records),
            "astar_feasible": sum(r["astar"]["feasible"] for r in records),
            "genetic_feasible": sum(r["genetic"]["feasible"] for r in records),
            "great_circle_crossing_land": sum(not r["great_circle"]["feasible"] for r in
records),
            "genetic_gap_mean_pct": round(float(np.mean(gaps)), 2) if gaps else None,
            "genetic_gap_max_pct": round(float(np.max(gaps)), 2) if gaps else None,
            "dynamic_equal_cost": all(
                r["astar"]["total_cost"] == r["dynamic"]["total_cost"] for r in
records),
            "astar_dijkstra_equal_cost": all(
                r["astar"]["total_cost"] == r["dijkstra"]["total_cost"] for r in
records),
            "mean_detour_pct": round(float(np.mean([r["detour_pct"] for r in records])),
2),
        },
    }
    (FIGURE_DIR / "regional_routes.json").write_text(json.dumps(summary, indent=2),
                                                     encoding="utf-8")
    print("\n", json.dumps(summary["aggregate"], indent=2))
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

scripts/route_audit.py

```
"""Audit of the route planner on 100 random voyages in the eleven sea regions.

Purpose
-------
The supervisor asked whether the application ever lays a course incorrectly.
This script answers the question with an experiment that does not rely on the
neural network for the verdict:

1. 100 voyages are drawn at random (fixed seed) between water points of the
   labelled corpus, distributed over the eleven sea regions; both ends must
   also be water according to an independent land mask and lie 150-1200 km
   apart.
2. Every voyage is planned exactly as a user would plan it: A* on the default
   lattice of 0.25 deg with automatic refinement.
3. Every route is sampled every kilometre and each sample is tested against
   the GLOBE-based 1 km land mask of the package ``global-land-mask``. The
   mask is independent of the classifier, so it plays the role of the ground
   truth. Its resolution is about 1 km: a channel narrower than that (the
   Bosporus, parts of the Danish straits) appears as land.
4. Each voyage receives one of four verdicts:

   ``correct``   no sample on land;
   ``grazing``   land runs shorter than ``CROSSING_KM`` - the route touches a
                 coast, a narrow channel or a small islet at the resolution of
                 the mask;
   ``crossing``  at least one continuous land run of ``CROSSING_KM`` or more -
                 the course is laid incorrectly;
   ``not_found`` the planner returned no route.

5. Voyages with the verdict ``crossing`` or ``not_found`` are planned again on
   a 0.1 deg lattice, to see whether a finer lattice removes the error.

   Steps 2-5 are run with the planner's own coastline check switched off
   (``coast_guard=False``), i.e. as the program worked before version 1.2.
   Step 7 repeats the test with the check switched on.
7. Every voyage is planned once more with the automatic coastline check of
   version 1.2 (:mod:`maritime_route.routing.coast_check`): a route that
   crosses land is corrected by blocking cells and searching again, then by a
   finer lattice; a route that still crosses land carries a warning. The
   result is judged by the same land-mask test as in step 3.
6. The cause of every error is attributed automatically:

   * a land run is put down to the **lattice** when the classifier, asked at
     the sample point itself, also answers "land": the route steps diagonally
     between two water cells across land narrower than a cell; it is put
     down to the **classifier** when the classifier answers "water" there;
   * a voyage without a route is checked for a sea connection on the land
     mask (2 km raster, the bounding box of the two ports widened by 8 deg).
     ``no_open_connection``: the ports are not connected by water wider than
     the raster resolves - for example a Caribbean and a Pacific port (the
     Panama Canal is not modelled) or the Black Sea and the Mediterranean
     (the Bosporus is under 1 km wide); ``classifier``: the classifier labels
     the area of a port as land; ``closed_on_map``: a connection exists, but
     the classified lattice closes it (A* is complete, so a passage open on
     the cost map would have been found).

Outputs
-------
``docs/figures/route_audit.json``  every voyage with its numbers and verdict;
``docs/figures/route_audit.csv``   the same as a table (one row per voyage);
``docs/figures/fig_audit_<region>.png``  maps of the voyages of each region;
``docs/figures/fig_audit_errors.png``    the incorrect courses on the lattice;
``docs/figures/fig_audit_guard.png``     incorrect courses and their automatic
                                         correction by the coastline check.

Usage::

    pip install global-land-mask
    python scripts/route_audit.py            # about 20 minutes on a CPU

对 11 个海域中随机抽取的 100 条航线做自动测试：用默认参数规划，再用独立的
1 km 陆地掩膜（global-land-mask，基于 GLOBE 高程模型）逐公里检查航线是否穿过陆地，
并记录每条航线的结果。
"""
from __future__ import annotations

import csv
```

```
import json
import logging
import math
import sys
import time
from pathlib import Path

import numpy as np

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))

import matplotlib  # noqa: E402

matplotlib.use("Agg")
import matplotlib.pyplot as plt  # noqa: E402

from maritime_route.config import FIGURE_DIR, RAW_DIR, REGIONS, WATER_CLASSES  # noqa:
E402
from maritime_route.data.ais_loader import load_geojson, validate  # noqa: E402
from maritime_route.data.cleaning import assign_regions, conflict_mask  # noqa: E402
from maritime_route.model.inference import ZoneClassifierService  # noqa: E402
from maritime_route.routing.geodesy import haversine_km  # noqa: E402
from maritime_route.routing.planner import RoutePlanner  # noqa: E402

try:
    from global_land_mask import globe
except ImportError:                                     # pragma: no cover
    sys.exit("This script needs the land mask: pip install global-land-mask")

#: Number of voyages and the random seed. 航线数量与随机种子。
N_VOYAGES = 100
SEED = 2026
#: Great-circle distance between the two ends, km.
MIN_KM, MAX_KM = 150.0, 1200.0
#: Sampling step along a route and the length ignored at each port, km.
STEP_KM, PORT_KM = 1.0, 5.0
#: A continuous land run of this length or more makes the course incorrect, km.
CROSSING_KM = 3.0
#: Lattice used to re-plan the incorrect voyages, degrees.
FINE_DEG = 0.1


def mask_is_water(lat, lon):
    """True where the independent 1 km land mask shows water."""
    return ~globe.is_land(np.asarray(lat, dtype=float), np.asarray(lon, dtype=float))


def clear_water(lat, lon, radius_deg=0.03):
    """Water at the point and at four points around it (about 3 km)."""
    ok = mask_is_water(lat, lon)
    for dlat, dlon in ((radius_deg, 0), (-radius_deg, 0), (0, radius_deg), (0, -
radius_deg)):
        ok &= mask_is_water(lat + dlat, lon + dlon)
    return ok


def draw_voyages(rng):
    """Draw the voyages: water points of the corpus, both ends clear water."""
    frame, _ = validate(load_geojson(RAW_DIR / "merged.geojson"), source="raw")
    frame = frame[~conflict_mask(frame)].reset_index(drop=True)
    frame = frame[frame["class_code"].isin(WATER_CLASSES)].reset_index(drop=True)
    lat, lon = frame["latitude"].to_numpy(), frame["longitude"].to_numpy()
    region = assign_regions(lat, lon)
    good = clear_water(lat, lon)
    names = [name for name, _ in REGIONS]
    voyages = []
    k = 0
    while len(voyages) < N_VOYAGES:
        name = names[k % len(names)]
        k += 1
        idx = np.flatnonzero((region == name) & good)
        for _ in range(10000):
            a, b = rng.choice(idx, 2, replace=False)
            d = float(haversine_km(lat[a], lon[a], lat[b], lon[b]))
            if MIN_KM <= d <= MAX_KM:
                voyages.append({"id": len(voyages) + 1, "region": name,
                                "start": [round(float(lat[a]), 4), round(float(lon[a]),
4)],
```

```
                                    "end": [round(float(lat[b]), 4), round(float(lon[b]),
4)],
                                    "great_circle_km": round(d, 1)})
                break
    return voyages


def land_runs(waypoints):
    """Lengths (km) of the continuous land runs along a route.

    The poly-line is sampled every ``STEP_KM``; ``PORT_KM`` at each end is
    ignored, because a port lies on the coast by definition.
    """
    pts = []
    for (a1, b1), (a2, b2) in zip(waypoints[:-1], waypoints[1:]):
        n = max(1, int(math.ceil(float(haversine_km(a1, b1, a2, b2)) / STEP_KM)))
        t = np.arange(n) / n
        pts.append(np.column_stack([a1 + (a2 - a1) * t, b1 + (b2 - b1) * t]))
    pts.append(np.array([waypoints[-1]]))
    pts = np.vstack(pts)
    seg = haversine_km(pts[:-1, 0], pts[:-1, 1], pts[1:, 0], pts[1:, 1])
    dist = np.r_[0.0, np.cumsum(seg)]
    keep = (dist >= PORT_KM) & (dist <= dist[-1] - PORT_KM)
    land = ~mask_is_water(pts[:, 0], pts[:, 1]) & keep
    runs, current = [], 0.0
    for i in range(1, len(pts)):
        if land[i] and land[i - 1]:
            current += seg[i - 1]
        elif land[i]:
            current = STEP_KM
        if not land[i] and current > 0:
            runs.append(current)
            current = 0.0
    if current > 0:
        runs.append(current)
    return runs, int(land.sum()), int(keep.sum())


def sea_connected(start, end, step=0.02, margin=8.0):
    """True if the land mask connects the two ports by water (2 km raster)."""
    from scipy import ndimage
    lat0, lat1 = min(start[0], end[0]) - margin, max(start[0], end[0]) + margin
    lon0, lon1 = min(start[1], end[1]) - margin, max(start[1], end[1]) + margin
    lats = np.arange(lat0, lat1 + step, step)
    lons = np.arange(lon0, lon1 + step, step)
    gl, ga = np.meshgrid(lons, lats)
    labels, _ = ndimage.label(mask_is_water(ga, gl))
    def cell(p):
        return int(round((p[0] - lat0) / step)), int(round((p[1] - lon0) / step))
    a, b = labels[cell(start)], labels[cell(end)]
    return bool(a and a == b)


def land_samples(waypoints):
    """Coordinates of the samples on land (the same sampling as land_runs)."""
    pts = []
    for (a1, b1), (a2, b2) in zip(waypoints[:-1], waypoints[1:]):
        n = max(1, int(math.ceil(float(haversine_km(a1, b1, a2, b2)) / STEP_KM)))
        t = np.arange(n) / n
        pts.append(np.column_stack([a1 + (a2 - a1) * t, b1 + (b2 - b1) * t]))
    pts = np.vstack(pts + [np.array([waypoints[-1]])])
    seg = haversine_km(pts[:-1, 0], pts[:-1, 1], pts[1:, 0], pts[1:, 1])
    dist = np.r_[0.0, np.cumsum(seg)]
    keep = (dist >= PORT_KM) & (dist <= dist[-1] - PORT_KM)
    return pts[keep & ~mask_is_water(pts[:, 0], pts[:, 1])]


def crossing_cause(classifier, waypoints):
    """'lattice' or 'classifier', by the classifier's answer at the land samples."""
    from maritime_route.config import CLASS_NAMES
    pts = land_samples(waypoints)
    water = np.array([c in WATER_CLASSES for c in CLASS_NAMES])
    pred = classifier.predict_coordinates(pts[:, 0], pts[:, 1]).argmax(axis=1)
    taken_for_water = float(water[pred].mean())
    return ("classifier" if taken_for_water >= 0.5 else "lattice"),
round(taken_for_water, 2)


def not_found_cause(classifier, voyage):
```

```
    """'no_open_connection', 'classifier' (a port area labelled land) or
'closed_on_map'."""
    from maritime_route.config import CLASS_NAMES
    if not sea_connected(voyage["start"], voyage["end"]):
        return "no_open_connection"
    water = np.array([c in WATER_CLASSES for c in CLASS_NAMES])
    # Classifier's answer in a 30 km neighbourhood of each port. 港口附近 30 km 内的分类结
果。
    for p in (voyage["start"], voyage["end"]):
        offs = np.linspace(-0.27, 0.27, 7)
        la, lo = np.meshgrid(p[0] + offs, p[1] + offs /
max(math.cos(math.radians(p[0])), 0.2))
        pred = classifier.predict_coordinates(la.ravel(), lo.ravel()).argmax(axis=1)
        if water[pred].mean() < 0.2:
            return "classifier"
    return "closed_on_map"


def verdict_of(runs):
    if not runs:
        return "correct"
    return "crossing" if max(runs) >= CROSSING_KM else "grazing"


def plan_and_check(planner, voyage, resolution=None, guard=False):
    """Plan one voyage and test the route against the land mask.

    ``guard`` switches the planner's own coastline check on (version 1.2).
    """
    t0 = time.perf_counter()
    try:
        plan, cost_map = planner.plan(tuple(voyage["start"]), tuple(voyage["end"]),
"astar",
                                      resolution, coast_guard=guard)
    except Exception as exc:                          # a port the planner cannot use
        return {"verdict": "not_found", "message": str(exc)[:160],
                "seconds": round(time.perf_counter() - t0, 2)}, None, None
    seconds = round(time.perf_counter() - t0, 2)
    if not plan.feasible or len(plan.waypoints) < 2:
        return {"verdict": "not_found", "message": plan.message[:160], "seconds":
seconds}, plan, cost_map
    runs, on_land, samples = land_runs(plan.waypoints)
    return {
        "verdict": verdict_of(runs),
        "distance_km": round(plan.distance_km, 1),
        "detour_pct": round(100 * (plan.distance_km / plan.great_circle_km - 1), 1),
        "resolution_deg": round(plan.grid["resolution_deg"], 4),
        "samples": samples,
        "samples_on_land": on_land,
        "land_runs_km": [round(r, 1) for r in sorted(runs, reverse=True)[:5]],
        "max_land_run_km": round(max(runs), 1) if runs else 0.0,
        "classifier_interior_land": plan.interior_land_waypoints,
        "seconds": seconds,
        "coast_check": {k: plan.coast_check.get(k) for k in
                        ("verdict", "initial_verdict", "corrections", "refined",
"cells_blocked")},
        "warning": bool(plan.warning),
    }, plan, cost_map


# ------------------------------------------------------------------ figures
COLOR = {"correct": "#1a9850", "grazing": "#fdae61", "crossing": "#d73027", "not_found":
"#000000"}


def land_background(ax, box):
    """Land of the independent mask as a grey raster."""
    lat0, lat1, lon0, lon1 = box
    lats = np.linspace(lat0, lat1, 300)
    lons = np.linspace(lon0, lon1, 400)
    grid_lon, grid_lat = np.meshgrid(lons, lats)
    land = globe.is_land(grid_lat, grid_lon)
    ax.imshow(land, origin="lower", extent=(lon0, lon1, lat0, lat1), cmap="Greys",
              vmin=0, vmax=2.2, aspect="auto", interpolation="nearest")


def region_figure(region, rows, plans, path):
    lats = [p for r in rows for p in (r["start"][0], r["end"][0])]
    lons = [p for r in rows for p in (r["start"][1], r["end"][1])]
    for r in rows:
```

```
        plan = plans.get(r["id"])
        if plan is not None and plan.waypoints:
            lats += [w[0] for w in plan.waypoints]
            lons += [w[1] for w in plan.waypoints]
    pad = 1.0
    box = (min(lats) - pad, max(lats) + pad, min(lons) - pad, max(lons) + pad)
    fig, ax = plt.subplots(figsize=(6.6, 5.4))
    land_background(ax, box)
    for r in rows:
        plan = plans.get(r["id"])
        colour = COLOR[r["verdict"]]
        if plan is not None and plan.waypoints:
            w = np.array(plan.waypoints)
            ax.plot(w[:, 1], w[:, 0], color=colour, lw=1.4)
        ax.plot(*r["start"][::-1], "o", ms=3.5, color=colour)
        ax.plot(*r["end"][::-1], "s", ms=3.5, color=colour)
        mid = plan.waypoints[len(plan.waypoints) // 2] if plan is not None and
plan.waypoints else r["start"]
        ax.annotate(str(r["id"]), (mid[1], mid[0]), fontsize=7, color="#111",
                    xytext=(2, 2), textcoords="offset points")
    ax.set_xlim(box[2], box[3])
    ax.set_ylim(box[0], box[1])
    # Equal distances in both directions at the middle latitude. 按中纬度保持横纵比例。
    ax.set_aspect(1.0 / max(math.cos(math.radians(0.5 * (box[0] + box[1]))), 0.2))
    ax.set_xlabel("Longitude, deg")
    ax.set_ylabel("Latitude, deg")
    counts = {v: sum(r["verdict"] == v for r in rows) for v in COLOR}
    ax.set_title(f"{region}: correct {counts['correct']}, grazing {counts['grazing']}, "
                 f"crossing {counts['crossing']}, not found {counts['not_found']}",
fontsize=8.5)
    from matplotlib.lines import Line2D
    ax.legend(handles=[Line2D([], [], color=c, lw=2, label=v.replace("_", " ")) for v, c
in COLOR.items()],
              fontsize=6.5, loc="lower left", frameon=True)
    fig.tight_layout()
    fig.savefig(path, dpi=170)
    plt.close(fig)


def error_figure(rows, plans, maps, path):
    """The incorrect courses over the classified lattice of their cost map."""
    bad = [r for r in rows if r["verdict"] == "crossing"][:6]
    if not bad:
        return False
    n = len(bad)
    cols = 3 if n > 2 else n
    nrows = math.ceil(n / cols)
    fig, axes = plt.subplots(nrows, cols, figsize=(8.3, 3.0 * nrows), squeeze=False)
    from maritime_route.config import CLASS_COLOR, CLASS_NAMES
    from matplotlib.colors import ListedColormap
    cmap = ListedColormap([CLASS_COLOR[c] for c in CLASS_NAMES])
    for ax, r in zip(axes.flat, bad):
        cm = maps[r["id"]]
        spec = cm.spec
        ax.imshow(cm.class_index, origin="lower", cmap=cmap, vmin=-0.5, vmax=3.5,
aspect="auto",
                  extent=(spec.lon_min, spec.lon_max, spec.lat_min, spec.lat_max),
interpolation="nearest")
        lats = np.linspace(spec.lat_min, spec.lat_max, 300)
        lons = np.linspace(spec.lon_min, spec.lon_max, 400)
        gl, ga = np.meshgrid(lons, lats)
        ax.contour(gl, ga, globe.is_land(ga, gl).astype(float), levels=[0.5],
colors="black", linewidths=0.6)
        w = np.array(plans[r["id"]].waypoints)
        ax.plot(w[:, 1], w[:, 0], color="#d73027", lw=1.6)
        ax.set_title(f"No. {r['id']} - {r['region'].split(' (')[0]}\nlongest land run "
                     f"{r['max_land_run_km']} km", fontsize=7.5)
        ax.tick_params(labelsize=6)
    for ax in list(axes.flat)[n:]:
        ax.axis("off")
    fig.tight_layout()
    fig.savefig(path, dpi=170)
    plt.close(fig)
    return True


def guard_figure(rows, plans, guarded, path):
    """Incorrect courses (red) and the routes after the coastline check (blue)."""
    # The six longest land runs of the test (the most instructive cases).
    bad = sorted([r for r in rows if r["verdict"] == "crossing" and r["id"] in guarded],
```

```
                  key=lambda r: -r["max_land_run_km"])[:6]
    if not bad:
        return False
    n = len(bad)
    cols = 3 if n > 2 else n
    nrows = math.ceil(n / cols)
    fig, axes = plt.subplots(nrows, cols, figsize=(8.3, 3.1 * nrows), squeeze=False)
    for ax, r in zip(axes.flat, bad):
        old_w = np.array(plans[r["id"]].waypoints)
        new_w = np.array(guarded[r["id"]].waypoints)
        both = np.vstack([old_w, new_w])
        pad = 0.4
        box = (both[:, 0].min() - pad, both[:, 0].max() + pad,
               both[:, 1].min() - pad, both[:, 1].max() + pad)
        land_background(ax, box)
        ax.plot(old_w[:, 1], old_w[:, 0], color="#d73027", lw=1.5, label="without the
check")
        ax.plot(new_w[:, 1], new_w[:, 0], color="#2166ac", lw=1.3, label="with the
check")
        ax.set_xlim(box[2], box[3])
        ax.set_ylim(box[0], box[1])
        ax.set_aspect(1.0 / max(math.cos(math.radians(0.5 * (box[0] + box[1]))), 0.2))
        g = r["guarded"]
        ax.set_title(f"No. {r['id']} - {r['region'].split(' (')[0]}\n"
                     f"land {r['max_land_run_km']} km -> {g.get('max_land_run_km', 0)}
km "
                     f"({g['verdict']})", fontsize=7.5)
        ax.tick_params(labelsize=6)
    axes.flat[0].legend(fontsize=6.5, loc="lower left")
    for ax in list(axes.flat)[n:]:
        ax.axis("off")
    fig.tight_layout()
    fig.savefig(path, dpi=170)
    plt.close(fig)
    return True


def main() -> int:
    logging.basicConfig(level=logging.WARNING)
    rng = np.random.default_rng(SEED)
    voyages = draw_voyages(rng)
    planner = RoutePlanner(ZoneClassifierService.instance())
    rows, plans, maps, guarded = [], {}, {}, {}
    for v in voyages:
        result, plan, cost_map = plan_and_check(planner, v)
        row = {**v, **result}
        if row["verdict"] == "crossing":
            row["cause"], row["share_taken_for_water"] =
crossing_cause(planner.classifier, plan.waypoints)
        elif row["verdict"] == "not_found":
            row["cause"] = not_found_cause(planner.classifier, v)
        if row["verdict"] in ("crossing", "not_found"):
            fine, _, _ = plan_and_check(planner, v, FINE_DEG)
            row["fine"] = {k: fine.get(k) for k in ("verdict", "distance_km",
"max_land_run_km", "resolution_deg")}
        # Step 7: the same voyage with the coastline check of version 1.2.
        g, gplan, _ = plan_and_check(planner, v, guard=True)
        row["guarded"] = {k: g.get(k) for k in ("verdict", "distance_km", "detour_pct",
                                                "max_land_run_km", "resolution_deg",
                                                "seconds", "coast_check", "warning")}
        if gplan is not None and gplan.waypoints:
            guarded[v["id"]] = gplan
        rows.append(row)
        if plan is not None:
            plans[v["id"]] = plan
            maps[v["id"]] = cost_map
        print(f"{v['id']:3d} {v['region'][:28]:28s} {row['verdict']:9s} "
              f"{row.get('distance_km', '-')!s:>7} km  max land
{row.get('max_land_run_km', '-')}"
              + (f"  cause {row['cause']}" if "cause" in row else "")
              + (f"  -> 0.1 deg: {row['fine']['verdict']}" if "fine" in row else "")
              + f"  | with check: {g['verdict']}", flush=True)

    verdicts = ("correct", "grazing", "crossing", "not_found")
    by_region = {}
    for name, _ in REGIONS:
        sub = [r for r in rows if r["region"] == name]
        by_region[name] = {"n": len(sub), **{k: sum(r["verdict"] == k for r in sub) for
k in verdicts}}
```

```
    fixed = [r for r in rows if "fine" in r and r["fine"]["verdict"] in ("correct",
"grazing")]
    summary = {
        "n_voyages": len(rows), "seed": SEED, "min_km": MIN_KM, "max_km": MAX_KM,
        "crossing_threshold_km": CROSSING_KM, "port_margin_km": PORT_KM, "step_km":
STEP_KM,
        "fine_resolution_deg": FINE_DEG,
        "verdicts": {k: sum(r["verdict"] == k for r in rows) for k in verdicts},
        "by_region": by_region,
        "fixed_by_fine_lattice": len(fixed),
        "causes": {
            "crossing": {c: sum(r.get("cause") == c for r in rows if r["verdict"] ==
"crossing")
                         for c in ("lattice", "classifier")},
            "not_found": {c: sum(r.get("cause") == c for r in rows if r["verdict"] ==
"not_found")
                          for c in ("no_open_connection", "classifier",
"closed_on_map")},
        },
        "fixed_by_fine_lattice_by_cause": {
            c: sum(1 for r in fixed if r.get("cause") == c) for c in ("lattice",
"classifier", "closed_on_map")},
        "mean_detour_pct": round(float(np.mean([r["detour_pct"] for r in rows if
"detour_pct" in r])), 1),
        "mean_seconds": round(float(np.mean([r["seconds"] for r in rows])), 2),
        "max_seconds": round(float(max(r["seconds"] for r in rows)), 2),
        "refined": sum(1 for r in rows if r.get("resolution_deg", 0.25) < 0.25),
    }
    # ---- step 7: the same voyages with the coastline check ---------------- #
    g_rows = [r["guarded"] for r in rows]
    summary["with_coast_check"] = {
        "verdicts": {k: sum(g["verdict"] == k for g in g_rows) for k in verdicts},
        "by_region": {name: {k: sum(r["guarded"]["verdict"] == k for r in rows if
r["region"] == name)
                             for k in verdicts} for name, _ in REGIONS},
        # Routes the check found crossing land and changed (initial verdict 'crossing').
        "corrected": sum(1 for g in g_rows if (g.get("coast_check") or
{}).get("initial_verdict") == "crossing"
                         and g["verdict"] in ("correct", "grazing")),
        "corrected_by_refinement": sum(1 for g in g_rows if (g.get("coast_check") or
{}).get("refined")
                                       and g["verdict"] in ("correct", "grazing")),
        "warnings": sum(1 for g in g_rows if g.get("warning")),
        # Crossings of the first test that the check removed / left.
        "crossings_removed": sum(1 for r in rows if r["verdict"] == "crossing"
                                 and r["guarded"]["verdict"] in ("correct", "grazing")),
        "crossings_left": sum(1 for r in rows if r["verdict"] == "crossing"
                              and r["guarded"]["verdict"] == "crossing"),
        "crossings_now_not_found": sum(1 for r in rows if r["verdict"] == "crossing"
                                       and r["guarded"]["verdict"] == "not_found"),
        "unchanged_correct": sum(1 for r in rows if r["verdict"] in ("correct",
"grazing")
                                 and r["guarded"]["verdict"] == r["verdict"]),
        "mean_detour_pct": round(float(np.mean([g["detour_pct"] for g in g_rows
                                                if g.get("detour_pct") is not None])),
1),
        "mean_seconds": round(float(np.mean([g["seconds"] for g in g_rows])), 2),
        "max_seconds": round(float(max(g["seconds"] for g in g_rows)), 2),
        "extra_km_of_corrected": round(float(np.mean(
            [r["guarded"]["distance_km"] - r["distance_km"] for r in rows
             if r["verdict"] == "crossing" and r["guarded"].get("distance_km")])), 1)
        if any(r["verdict"] == "crossing" and r["guarded"].get("distance_km") for r in
rows) else None,
    }
    FIGURE_DIR.mkdir(parents=True, exist_ok=True)
    (FIGURE_DIR / "route_audit.json").write_text(json.dumps({"summary": summary,
"voyages": rows}, indent=1),
                                                 encoding="utf-8")
    with open(FIGURE_DIR / "route_audit.csv", "w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["id", "region", "start_lat", "start_lon", "end_lat", "end_lon",
"great_circle_km",
                    "route_km", "detour_pct", "resolution_deg", "verdict",
"max_land_run_km",
                    "samples_on_land", "cause", "seconds", "verdict_at_0.1deg",
                    "verdict_with_check", "route_km_with_check",
"max_land_run_km_with_check",
                    "corrections", "warning"])
        for r in rows:
```

```
            w.writerow([r["id"], r["region"], *r["start"], *r["end"],
r["great_circle_km"],
                        r.get("distance_km", ""), r.get("detour_pct", ""),
r.get("resolution_deg", ""),
                        r["verdict"], r.get("max_land_run_km", ""),
r.get("samples_on_land", ""),
                        r.get("cause", ""),
                        r["seconds"], r.get("fine", {}).get("verdict", ""),
                        r["guarded"]["verdict"], r["guarded"].get("distance_km", ""),
                        r["guarded"].get("max_land_run_km", ""),
                        (r["guarded"].get("coast_check") or {}).get("corrections", ""),
                        int(bool(r["guarded"].get("warning")))])
    for name, _ in REGIONS:
        sub = [r for r in rows if r["region"] == name]
        slug = name.split(" (")[0].lower().replace(" and ", "_").replace(" ", "_")
        region_figure(name, sub, plans, FIGURE_DIR / f"fig_audit_{slug}.png")
    error_figure(rows, plans, maps, FIGURE_DIR / "fig_audit_errors.png")
    guard_figure(rows, plans, guarded, FIGURE_DIR / "fig_audit_guard.png")
    print("\n", json.dumps(summary, indent=1))
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

scripts/run_server.py

```
#!/usr/bin/env python3
"""Start the web application.

用法: python scripts/run_server.py --host 127.0.0.1 --port 8000

Launches the FastAPI application ``maritime_route.web.app:app`` under the
Uvicorn ASGI server. Host and port default to ``config.SERVER``; ``--reload``
restarts the server when a source file changes (development only).
The page is then available at ``http://<host>:<port>/``.

启动 Web 服务：使用 Uvicorn 运行 FastAPI 应用；--reload 仅用于开发调试。
"""
from __future__ import annotations

import argparse
import logging
import sys
from pathlib import Path

# Make the package importable without installation: add <project>/src to the
# module search path (parents[1] of scripts/run_server.py is the project root).
# 将 src 目录加入模块搜索路径，无需安装即可导入 maritime_route。
sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))

import uvicorn

from maritime_route.config import SERVER


def main() -> int:
    """Parse the command line and run the server until it is stopped.

    Returns
    -------
    int
        Process exit code, 0 on a normal shutdown (Ctrl+C).
    """
    parser = argparse.ArgumentParser(description="Run the Maritime Route Planner
server")
    parser.add_argument("--host", default=SERVER.host)
    parser.add_argument("--port", type=int, default=SERVER.port)
    parser.add_argument("--reload", action="store_true", help="auto-reload on code
changes")
    args = parser.parse_args()

    logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %
(message)s")
    print(f"\n  Maritime Route Planner -> http://{args.host}:{args.port}\n")
    # The app is given as an import string (not an object) because Uvicorn's
    # reload mode must re-import it in a fresh worker process.
    uvicorn.run("maritime_route.web.app:app", host=args.host, port=args.port,
```

```
                reload=args.reload, log_level="info")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

D.10 Automated tests

tests/conftest.py

```
"""Shared pytest fixtures. 测试夹具。

Place in the test suite
-----------------------
pytest loads this file automatically before the test modules in ``tests/``.
It makes the in-repository package importable, defines the ``requires_model``
skip marker and provides the fixtures shared by several test modules.

Main objects
------------
``requires_model``
    Marker that skips a test when the trained model weights or the reference
    bathymetry index are missing (for example on a fresh checkout before
    ``scripts/train_model.py`` has been run). Tests that only exercise pure
    functions (geodesy, loaders, storage) do not use it.
``classifier`` / ``planner``
    Session-scoped: the model is loaded once for the whole test run because
    loading the network and the reference index is the slowest step.
``repository``
    Function-scoped: every test gets a new empty SQLite file in its own
    temporary directory, so tests cannot influence each other or the real
    database.
模型相关夹具在整个会话中只加载一次；数据库夹具每个测试使用独立的临时文件。
"""
from __future__ import annotations

import sys
from pathlib import Path

import pytest

# Import ``maritime_route`` from src/ without installing the package.
# 无需安装即可从 src 目录导入。
sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))

from maritime_route.config import BATHY_INDEX_PATH, MODEL_WEIGHTS_PATH

#: Skip marker for tests that need the trained model files on disk.
#: 缺少已训练模型时跳过相关测试。
requires_model = pytest.mark.skipif(
    not (Path(MODEL_WEIGHTS_PATH).exists() and Path(BATHY_INDEX_PATH).exists()),
    reason="trained model not available - run scripts/train_model.py",
)


@pytest.fixture(scope="session")
def classifier():
    """Shared zone classifier (the process-wide singleton).

    Returns
    -------
    ZoneClassifierService
        Loaded model with its scaler and reference index. The import is done
        inside the fixture so that test modules which do not need the model
        do not import PyTorch.
    """
    from maritime_route.model.inference import ZoneClassifierService
    return ZoneClassifierService.instance()


@pytest.fixture(scope="session")
def planner(classifier):
```

```
    """Route planner bound to the shared classifier.

    Parameters
    ----------
    classifier : ZoneClassifierService
        Injected by the ``classifier`` fixture.

    Returns
    -------
    RoutePlanner
        Planner with the default routing configuration.
    """
    from maritime_route.routing.planner import RoutePlanner
    return RoutePlanner(classifier)


@pytest.fixture()
def repository(tmp_path):
    """Fresh SQLite repository in a temporary directory.

    Parameters
    ----------
    tmp_path : pathlib.Path
        Built-in pytest fixture: a unique empty directory per test.

    Returns
    -------
    RouteRepository
        Repository whose schema has been created in ``tmp_path/test.db``.
    """
    from maritime_route.storage.repository import RouteRepository
    return RouteRepository(tmp_path / "test.db")
```

tests/test_geodesy.py

```
"""Tests of the spherical geometry helpers. 球面几何测试。

Covers ``maritime_route.routing.geodesy``: great-circle (haversine) distance,
initial bearing, great-circle sampling, densification, Douglas-Peucker
simplification and bounding boxes. These functions are used by the cost map,
the planner (leg lengths, bearings, baseline comparison) and the exporters,
so their correctness is checked against known values and geometric
invariants. No trained model is needed.
测试球面距离、方位角、大圆采样、加密、简化与外包框；不依赖模型。
"""
from __future__ import annotations

import math

import pytest

from maritime_route.routing.geodesy import (
    bounding_box, densify, douglas_peucker, great_circle_points,
    haversine_km, initial_bearing_deg, path_length_km,
)


def test_haversine_known_distance():
    """Haversine distance Kiel -> Tallinn matches an external reference (1045-1060
km)."""
    # Kiel -> Tallinn, reference value from an independent great-circle calculator.
    d = float(haversine_km(54.3233, 10.1228, 59.4370, 24.7536))
    assert 1045 < d < 1060


def test_haversine_zero_and_antipodal():
    """Distance is 0 for identical points and half the Earth circumference for antipodes
on the equator."""
    assert float(haversine_km(10, 20, 10, 20)) == pytest.approx(0.0, abs=1e-9)
    half_circumference = math.pi * 6371.0088
    assert float(haversine_km(0, 0, 0, 180)) == pytest.approx(half_circumference,
rel=1e-6)


def test_bearing_cardinal_directions():
    """Initial bearing is 0, 90 and 180 degrees for due north, east and south."""
    assert initial_bearing_deg(0, 0, 10, 0) == pytest.approx(0.0, abs=1e-6)
    assert initial_bearing_deg(0, 0, 0, 10) == pytest.approx(90.0, abs=1e-6)
```

```
    assert initial_bearing_deg(0, 0, -10, 0) == pytest.approx(180.0, abs=1e-6)


def test_great_circle_points_endpoints_and_length():
    """Sampled great circle has the requested count, exact end points and the direct
length."""
    start, end = (54.0, 10.0), (59.0, 25.0)
    line = great_circle_points(start, end, 50)
    assert len(line) == 50
    assert line[0] == pytest.approx(start, abs=1e-6)
    assert line[-1] == pytest.approx(end, abs=1e-6)
    # A sampled great circle is the shortest path: its length equals the direct one.
    assert path_length_km(line) == pytest.approx(
        float(haversine_km(*start, *end)), rel=1e-3)


def test_densify_limits_segment_length():
    """Densified poly-line has no segment much longer than the requested step (50 km, 10
% tolerance)."""
    dense = densify([(54.0, 10.0), (59.0, 25.0)], step_km=50.0)
    segments = [float(haversine_km(a[0], a[1], b[0], b[1]))
                for a, b in zip(dense[:-1], dense[1:])]
    assert max(segments) <= 55.0


def test_douglas_peucker_preserves_endpoints_and_reduces():
    """Douglas-Peucker keeps both end points and removes vertices."""
    line = great_circle_points((50.0, 0.0), (55.0, 20.0), 200)
    simple = douglas_peucker(line, tolerance_km=5.0)
    assert simple[0] == line[0] and simple[-1] == line[-1]
    assert len(simple) < len(line)


def test_bounding_box_with_margin():
    """Bounding box of two points is widened by the margin on every side."""
    lat_min, lat_max, lon_min, lon_max = bounding_box([(10, 20), (30, 40)],
margin_deg=2)
    assert (lat_min, lat_max, lon_min, lon_max) == (8, 32, 18, 42)
```

tests/test_data_loading.py

```
"""Tests of AIS loading, validation and feature extraction. 数据加载与特征测试。

Covers ``maritime_route.data.ais_loader`` (CSV/JSON/GeoJSON parsing, column
aliases, validation of coordinates) and ``maritime_route.data.features``
(signed-log transform, the 28-component feature matrix, handling of missing
measurements and the ``ReferenceIndex`` of reference bathymetry samples).
The last three tests guard against label leakage: a point that is stored in
the reference index must never be used as its own neighbour, so its features
do not depend on whether it was part of the index.
All data are generated synthetically by :func:`_frame`; no model is needed.
测试数据加载、校验与特征提取；最后三个测试防止参考索引中的点检索到自身（数据泄漏）。
"""
from __future__ import annotations

import io
import json

import numpy as np
import pandas as pd
import pytest

from maritime_route.data.ais_loader import (
    DataLoadError, load_csv, load_json, validate,
)
from maritime_route.data.features import (
    FEATURE_NAMES, N_FEATURES, ReferenceIndex, build_features, signed_log,
)


def _frame(n=40, seed=0):
    """Build a random AIS-like frame with the columns expected by the feature code.

    Parameters
    ----------
    n : int, optional
        Number of rows.
    seed : int, optional
```

```
        Seed of the NumPy random generator, so every call with the same seed
        returns the same frame.

    Returns
    -------
    pandas.DataFrame
        Coordinates in the Baltic/North Sea box, depth/elevation values,
        a depth/height type and nine OSM object counters. No class labels.
    """
    rng = np.random.default_rng(seed)
    return pd.DataFrame({
        "latitude": rng.uniform(50, 60, n),
        "longitude": rng.uniform(5, 25, n),
        "noaa_depth": rng.uniform(-3000, 500, n),
        "elevation": rng.uniform(-3000, 500, n),
        "noaa_type": rng.choice(["depth", "height"], n),
        **{c: rng.integers(0, 3, n) for c in
           ["osm_piers", "osm_lighthouses", "osm_breakwaters", "osm_harbors",
            "osm_rivers", "osm_lakes", "osm_beaches", "osm_cliffs", "osm_industrial"]},
    })


def test_csv_column_aliases_are_normalised():
    """CSV headers "lat"/"lng" are renamed to "latitude"/"longitude"."""
    frame = load_csv(io.StringIO("lat,lng\n54.5,10.2\n55.0,11.0\n"))
    assert {"latitude", "longitude"} <= set(frame.columns)
    assert len(frame) == 2


def test_csv_without_coordinates_is_rejected():
    """A CSV without any coordinate column raises DataLoadError."""
    with pytest.raises(DataLoadError):
        load_csv(io.StringIO("a,b\n1,2\n"))


def test_geojson_geometry_wins_over_properties():
    """For GeoJSON features the Point geometry overrides latitude/longitude
properties."""
    document = {
        "type": "FeatureCollection",
        "features": [{
            "type": "Feature",
            "properties": {"latitude": 0.0, "longitude": 0.0},
            "geometry": {"type": "Point", "coordinates": [10.5, 54.5]},
        }],
    }
    frame = load_json(io.StringIO(json.dumps(document)))
    assert frame.loc[0, "latitude"] == 54.5
    assert frame.loc[0, "longitude"] == 10.5


def test_validate_drops_out_of_range_rows():
    """validate() drops rows with invalid or missing coordinates and reports them."""
    frame = pd.DataFrame({"latitude": [54.0, 999.0, None], "longitude": [10.0, 10.0,
10.0]})
    clean, report = validate(frame)
    assert len(clean) == 1
    assert report.n_dropped == 2
    assert report.warnings


def test_validate_rejects_all_invalid():
    """validate() raises DataLoadError when no valid row remains."""
    with pytest.raises(DataLoadError):
        validate(pd.DataFrame({"latitude": [999.0], "longitude": [10.0]}))


def test_signed_log_is_odd_and_monotone():
    """signed_log is an odd function (f(-x) = -f(x)) and strictly increasing."""
    values = np.array([-1000.0, -1.0, 0.0, 1.0, 1000.0])
    out = signed_log(values)
    assert out[0] == pytest.approx(-out[-1])
    assert np.all(np.diff(out) > 0)


def test_feature_matrix_shape_and_finiteness():
    """Feature matrix has N_FEATURES columns matching FEATURE_NAMES and only finite
values."""
    frame = _frame()
    index = ReferenceIndex.from_frame(frame)
```

```
    matrix = build_features(frame, index, exclude_self=True)
    assert matrix.shape == (len(frame), N_FEATURES)
    assert len(FEATURE_NAMES) == N_FEATURES
    assert np.isfinite(matrix).all()


def test_features_handle_missing_measurements():
    """Missing depth/elevation yield finite features and raise the "missing" indicator
columns."""
    frame = _frame()
    frame.loc[0, "noaa_depth"] = np.nan
    frame.loc[0, "elevation"] = np.nan
    index = ReferenceIndex.from_frame(_frame(seed=1))
    matrix = build_features(frame, index)
    assert np.isfinite(matrix).all()
    # The 'missing' indicator columns must be raised for that row.
    assert matrix[0, FEATURE_NAMES.index("depth_is_missing")] == 1.0
    assert matrix[0, FEATURE_NAMES.index("elevation_is_missing")] == 1.0


def test_reference_index_round_trip(tmp_path):
    """ReferenceIndex saved to .npz and loaded back keeps its depth samples."""
    index = ReferenceIndex.from_frame(_frame())
    path = tmp_path / "index.npz"
    index.save(path)
    restored = ReferenceIndex.load(path)
    assert restored.depth.shape == index.depth.shape
    np.testing.assert_allclose(restored.depth, index.depth, rtol=1e-5)


def test_reference_index_always_excludes_the_query_point():
    """A point in the index must never retrieve itself, flag or no flag.

    索引中的点在任何情况下都不应检索到自身。
    """
    frame = _frame(n=20)
    index = ReferenceIndex.from_frame(frame)
    lat = frame["latitude"].to_numpy()
    lon = frame["longitude"].to_numpy()
    for flag in (False, True):
        dist, idx = index.query(lat, lon, k=3, exclude_self=flag)
        assert dist.shape == (len(frame), 3)
        # No returned neighbour is the query point itself.
        assert (dist[:, 0] > 1e-6).all()
        assert not (idx == np.arange(len(frame))[:, None]).any()


def test_reference_index_query_is_flag_independent():
    """The legacy exclude_self flag must no longer change the result.

    历史遗留的 exclude_self 参数不再影响结果，保证训练与推理特征一致。
    """
    frame = _frame(n=30)
    index = ReferenceIndex.from_frame(frame)
    # Query points that are NOT in the index.
    other = _frame(n=15, seed=5)
    lat = other["latitude"].to_numpy()
    lon = other["longitude"].to_numpy()
    a, ia = index.query(lat, lon, k=4, exclude_self=False)
    b, ib = index.query(lat, lon, k=4, exclude_self=True)
    np.testing.assert_allclose(a, b)
    np.testing.assert_array_equal(ia, ib)


def test_features_are_identical_whether_or_not_the_point_is_indexed():
    """The decisive property: provenance must not change a point's features.

    关键性质：同一地理点的特征不应因其是否在索引中而不同。
    """
    frame = _frame(n=60)
    index_with = ReferenceIndex.from_frame(frame)          # contains the points
    index_without = ReferenceIndex.from_frame(_frame(n=60, seed=3))
    # A point present in the index must not be able to use itself as context,
    # so its features must match those computed from an index built without it.
    probe = frame.iloc[:5].reset_index(drop=True)
    inside = build_features(probe, index_with)
    # Rebuild an index from the same data minus the probe rows.
    rest = ReferenceIndex.from_frame(frame.iloc[5:].reset_index(drop=True))
    outside = build_features(probe, rest)
    # The neighbour sets differ (different candidate pools), but neither may
```

```
    # contain a zero-distance self match, which is what the assertion checks.
    for matrix in (inside, outside):
        assert np.isfinite(matrix).all()
    d_inside, _ = index_with.query(probe["latitude"].to_numpy(),
                                   probe["longitude"].to_numpy(), k=4)
    assert (d_inside[:, 0] > 1e-6).all()
```

tests/test_cleaning.py

```
"""Tests of the data-cleaning rule and of the sea-region assignment.

清洗规则与海域划分的测试。
"""
import numpy as np
import pandas as pd

from maritime_route.data.cleaning import (
    OTHER_REGION,
    assign_regions,
    conflict_mask,
    measurement_conflicts,
    remove_label_conflicts,
)


def _frame(rows):
    """Small corpus from (class_code, noaa_depth, elevation) tuples."""
    return pd.DataFrame(
        [{"latitude": 55.0, "longitude": 15.0, "class_code": c,
          "noaa_depth": d, "elevation": e} for c, d, e in rows]
    )


def test_sea_point_above_sea_level_is_removed():
    frame = _frame([("COASTAL_SEA", 12.0, np.nan), ("OPEN_SEA", -300.0, np.nan)])
    assert conflict_mask(frame).tolist() == [True, False]


def test_land_point_below_sea_level_is_removed():
    frame = _frame([("COASTLINE", -40.0, np.nan), ("NEAR_COAST", 3.0, np.nan)])
    assert conflict_mask(frame).tolist() == [True, False]


def test_tolerance_keeps_small_deviations():
    # +4 m on a sea point and -4 m on a land point are within the 5 m tolerance.
    frame = _frame([("COASTAL_SEA", 4.0, 4.0), ("NEAR_COAST", -4.0, -4.0)])
    assert not conflict_mask(frame).any()
    assert conflict_mask(frame, tolerance_m=0.0).all()


def test_elevation_alone_can_reveal_a_conflict():
    # The relief model agrees with the class, the collector's elevation does not.
    frame = _frame([("OPEN_SEA", -20.0, 35.0), ("COASTLINE", 80.0, -60.0)])
    assert not measurement_conflicts(frame, "noaa_depth").any()
    assert measurement_conflicts(frame, "elevation").all()
    assert conflict_mask(frame).all()


def test_missing_measurements_are_never_flagged():
    frame = _frame([("COASTAL_SEA", np.nan, np.nan), ("COASTLINE", np.nan, np.nan)])
    assert not conflict_mask(frame).any()
    # A frame without an elevation column is handled as "not measured".
    assert not measurement_conflicts(frame.drop(columns="elevation"), "elevation").any()


def test_report_counts_by_measurement():
    frame = _frame([
        ("OPEN_SEA", 10.0, np.nan),     # depth only
        ("OPEN_SEA", -10.0, 10.0),      # elevation only
        ("COASTLINE", -10.0, -10.0),    # both
        ("COASTLINE", 10.0, 10.0),      # consistent
    ])
    cleaned, report = remove_label_conflicts(frame)
    assert len(cleaned) == 1 and report.n_removed == 3
    assert report.removed_by_measure == {"noaa_depth_only": 1, "elevation_only": 1,
"both": 1}
    assert report.removed_by_class["OPEN_SEA"] == 2
    assert report.to_dict()["n_after"] == 1
```

```
def test_aegean_points_get_their_own_region():
    lat = np.array([37.9, 40.5, 43.0, 54.0, 0.0])
    lon = np.array([23.6, 22.9, 34.0, 18.0, 0.0])
    regions = assign_regions(lat, lon).tolist()
    assert regions[0] == "Aegean Sea"            # Piraeus
    assert regions[1] == "Aegean Sea"            # Thessaloniki
    assert regions[2] == "Black Sea and Eastern Mediterranean"
    assert regions[3] == "North Europe (Baltic and North Sea)"
    assert regions[4] == OTHER_REGION
```

tests/test_routing.py

```
"""Tests of the cost map and the two search algorithms. 代价地图与寻路算法测试。

Covers ``GridSpec`` / ``make_grid_spec`` (lattice geometry and cell budget),
``CostMap`` (statistics, map overlay), ``nearest_navigable`` (snapping ports
onto water) and the exact searches ``astar`` and ``dijkstra``. The searches
are run on small synthetic maps built by :func:`synthetic_map`, so the
expected optimum is known and no trained model is required. The same helper
is imported by ``test_genetic``.
在合成地图上测试网格、代价地图与 A*/Dijkstra；不依赖模型。
"""
from __future__ import annotations

import numpy as np
import pytest

from maritime_route.config import IMPASSABLE_COST, ROUTING
from maritime_route.routing.astar import astar
from maritime_route.routing.cost_map import CostMap, GridSpec, make_grid_spec,
nearest_navigable
from maritime_route.routing.dijkstra import dijkstra


def synthetic_map(barrier: bool = True) -> CostMap:
    """20x20 open sea with an optional land wall leaving one gap. 合成测试地图。"""
    spec = GridSpec(50.0, 54.75, 0.0, 4.75, 0.25)
    rows, cols = spec.n_rows, spec.n_cols
    class_index = np.zeros((rows, cols), dtype=np.int8)       # OPEN_SEA
    cost = np.ones((rows, cols), dtype=np.float32)
    if barrier:
        mid = cols // 2
        class_index[:, mid] = 3                                # COASTLINE
        cost[:, mid] = IMPASSABLE_COST
        class_index[rows // 2, mid] = 0                        # the gap
        cost[rows // 2, mid] = 1.0
    return CostMap(spec=spec, class_index=class_index,
                   confidence=np.full((rows, cols), 0.9, dtype=np.float32), cost=cost)


def test_grid_spec_round_trip():
    """cell -> coordinate -> cell conversion returns the original cell."""
    spec = GridSpec(50.0, 55.0, 0.0, 5.0, 0.25)
    for row in (0, 5, spec.n_rows - 1):
        for col in (0, 7, spec.n_cols - 1):
            assert spec.coord_to_cell(*spec.cell_to_coord(row, col)) == (row, col)


def test_grid_spec_coarsens_when_over_budget():
    """A grid that would exceed max_cells is coarsened to fit the budget."""
    spec = make_grid_spec([(0.0, 0.0), (60.0, 60.0)], 0.1, 2.0, max_cells=50_000)
    assert spec.n_cells <= 50_000
    assert spec.resolution_deg > 0.1


def test_astar_and_dijkstra_agree_on_open_water():
    """A* and Dijkstra find paths of the same optimal cost on open water."""
    cost_map = synthetic_map(barrier=False)
    start, goal = (2, 2), (17, 17)
    a = astar(cost_map, start, goal)
    d = dijkstra(cost_map, start, goal)
    assert a.found and d.found
    assert a.total_cost == pytest.approx(d.total_cost, rel=1e-6)


def test_astar_expands_no_more_nodes_than_dijkstra():
```

```
    """The A* heuristic never makes it expand more nodes than Dijkstra."""
    cost_map = synthetic_map(barrier=False)
    a = astar(cost_map, (0, 0), (19, 19))
    d = dijkstra(cost_map, (0, 0), (19, 19))
    assert a.nodes_expanded <= d.nodes_expanded


def test_route_goes_through_the_gap_and_never_crosses_land():
    """The A* path uses the gap in the wall and contains no impassable cell."""
    cost_map = synthetic_map(barrier=True)
    result = astar(cost_map, (2, 2), (17, 17))
    assert result.found
    for row, col in result.path_cells:
        assert cost_map.cost[row, col] < IMPASSABLE_COST
    gap_row = cost_map.spec.n_rows // 2
    wall_col = cost_map.spec.n_cols // 2
    assert (gap_row, wall_col) in result.path_cells


def test_no_path_when_the_barrier_is_closed():
    """A* reports "No navigable path" when the land wall is closed."""
    cost_map = synthetic_map(barrier=True)
    wall_col = cost_map.spec.n_cols // 2
    cost_map.cost[:, wall_col] = IMPASSABLE_COST
    cost_map.class_index[:, wall_col] = 3
    result = astar(cost_map, (2, 2), (17, 17))
    assert not result.found
    assert "No navigable path" in result.message


def test_search_refuses_a_start_on_land():
    """A* reports no route when the start cell lies on land."""
    cost_map = synthetic_map(barrier=True)
    wall_col = cost_map.spec.n_cols // 2
    result = astar(cost_map, (0, wall_col), (17, 17))
    assert not result.found


def test_nearest_navigable_snaps_off_land():
    """nearest_navigable moves a land cell onto a navigable cell."""
    cost_map = synthetic_map(barrier=True)
    wall_col = cost_map.spec.n_cols // 2
    snapped = nearest_navigable(cost_map, 0, wall_col)
    assert snapped is not None
    assert cost_map.passable[snapped]


def test_cheaper_zone_is_preferred_over_a_shorter_crossing():
    """A detour through cheap water must beat a short crossing of very costly water.

    The band weight is chosen so that crossing it once (extra cost of roughly
    half the band weight) is dearer than the ~2*cols extra cheap steps of the
    detour - otherwise going straight really is optimal.
    绕行代价约 2*cols，故障碍带权重需远大于该值，绕行才应当胜出。
    """
    spec = GridSpec(50.0, 52.25, 0.0, 2.25, 0.25)
    rows, cols = spec.n_rows, spec.n_cols
    cost = np.ones((rows, cols), dtype=np.float32)
    cost[rows // 2, :] = 400.0                    # a very expensive band
    cost[rows // 2, 0] = 1.0                      # one cheap doorway on the left
    cost_map = CostMap(spec=spec, class_index=np.zeros((rows, cols), np.int8),
                       confidence=np.ones((rows, cols), np.float32), cost=cost)
    result = astar(cost_map, (0, cols - 1), (rows - 1, cols - 1))
    assert result.found
    assert (rows // 2, 0) in result.path_cells     # it used the cheap doorway
    # and the same optimum is reached without a heuristic
    assert dijkstra(cost_map, (0, cols - 1), (rows - 1, cols - 1)).total_cost == \
        pytest.approx(result.total_cost, rel=1e-6)


def test_statistics_report_navigable_share():
    """Cost-map statistics give a navigable share in (0, 1) and a consistent cell
total."""
    stats = synthetic_map(barrier=True).statistics()
    assert 0.0 < stats["navigable_share"] < 1.0
    assert stats["cells_total"] == stats["grid"]["n_cells"]


def test_overlay_cells_tile_without_gaps():
```

```
    """Down-sampled overlay cell size equals step x resolution and respects the cell
budget."""
    overlay = synthetic_map().to_overlay(max_cells=50)
    assert overlay["step"] >= 1
    assert overlay["cell_size_deg"] == pytest.approx(
        overlay["step"] * ROUTING.grid_resolution_deg, rel=1e-6)
    assert len(overlay["cells"]) <= 80
```

tests/test_dynamic_programming.py

```
"""Tests of the dynamic-programming route search (value iteration).

Dynamic programming is exact, so its route must cost exactly what the A*
optimum costs; the value function must satisfy the Bellman equation in every
reachable cell; and cells cut off from the goal must keep an infinite value.

动态规划测试：代价与 A* 最优值一致；价值函数在每个可达格子满足贝尔曼方程；
与终点不连通的格子价值保持为无穷大。
"""
from __future__ import annotations

import numpy as np
import pytest

from maritime_route.config import IMPASSABLE_COST
from maritime_route.routing.astar import astar
from maritime_route.routing.dynamic_programming import (
    dynamic_programming, step_costs, value_iteration,
)

from test_routing import synthetic_map


def test_dynamic_programming_matches_astar_on_open_water():
    """The DP route costs exactly the A* optimum on an open lattice."""
    cost_map = synthetic_map(barrier=False)
    exact = astar(cost_map, (2, 2), (17, 17))
    dp = dynamic_programming(cost_map, (2, 2), (17, 17))
    assert dp.found
    assert dp.total_cost == pytest.approx(exact.total_cost, rel=1e-9)


def test_dynamic_programming_goes_through_the_gap():
    """With a land wall the DP route matches A* and never touches land."""
    cost_map = synthetic_map(barrier=True)
    exact = astar(cost_map, (2, 2), (17, 17))
    dp = dynamic_programming(cost_map, (2, 2), (17, 17))
    assert dp.total_cost == pytest.approx(exact.total_cost, rel=1e-9)
    assert all(cost_map.passable[cell] for cell in dp.path_cells)
    assert dp.path_cells[0] == (2, 2) and dp.path_cells[-1] == (17, 17)


def test_value_function_satisfies_the_bellman_equation():
    """V(v) = min_u [c(v, u) + V(u)] holds in every reachable cell but the goal."""
    cost_map = synthetic_map(barrier=True)
    goal = (17, 17)
    value, sweeps, costs = value_iteration(cost_map, goal)
    assert value[goal] == 0.0 and sweeps > 1
    n_rows, n_cols = value.shape
    for r in range(n_rows):
        for c in range(n_cols):
            if (r, c) == goal or not np.isfinite(value[r, c]):
                continue
            rhs = min(costs[(dr, dc)][r, c] + value[r + dr, c + dc]
                      for (dr, dc) in costs
                      if 0 <= r + dr < n_rows and 0 <= c + dc < n_cols)
            assert value[r, c] == pytest.approx(rhs, rel=1e-12)


def test_land_cells_have_infinite_step_cost():
    """No step may enter or leave a land cell."""
    cost_map = synthetic_map(barrier=True)
    mid = cost_map.spec.n_cols // 2
    costs = step_costs(cost_map)
    assert np.isinf(costs[(0, 1)][0, mid - 1])      # into the wall
    assert np.isinf(costs[(0, 1)][0, mid])          # out of the wall
```

```
def test_unreachable_goal_is_reported():
    """A closed wall leaves the far side with an infinite value and no route."""
    cost_map = synthetic_map(barrier=True)
    mid = cost_map.spec.n_cols // 2
    cost_map.cost[:, mid] = IMPASSABLE_COST          # close the gap
    dp = dynamic_programming(cost_map, (2, 2), (17, 17))
    assert not dp.found
    assert np.isinf(dp.value[2, 2])
```

tests/test_genetic.py

```
"""Tests of the genetic-algorithm route search.

The genetic algorithm is not exact, so the tests check the properties that it
must always have: the route it returns never touches land, it is never cheaper
than the proven optimum of A*, it is reproducible for a fixed seed, and it finds
the single gap in a wall of land ("the maze").

遗传算法测试：不穿越陆地、代价不低于 A* 最优解、固定种子可复现、能找到陆地墙上的唯一缺口。
"""
from __future__ import annotations

import pytest

from maritime_route.config import GeneticConfig
from maritime_route.routing.astar import astar
from maritime_route.routing.genetic import _Evaluator, genetic

# Reuse the 20x20 synthetic map (open sea with an optional land wall) from
# test_routing; pytest puts tests/ on sys.path, so a plain import works.
# 复用 test_routing 中的合成地图。
from test_routing import synthetic_map

#: Small population and generation budget so the tests run quickly.
#: 较小的种群与代数，保证测试快速完成。
FAST = GeneticConfig(population_size=40, generations=120, stagnation_limit=40)


def test_rasterised_segment_is_eight_connected():
    """Rasterised segment starts/ends at the given cells and every step moves to an 8-
neighbour."""
    rows, cols = _Evaluator.rasterise((0, 0), (7, 3))
    assert (rows[0], cols[0]) == (0, 0) and (rows[-1], cols[-1]) == (7, 3)
    for r0, c0, r1, c1 in zip(rows[:-1], cols[:-1], rows[1:], cols[1:]):
        assert max(abs(r1 - r0), abs(c1 - c0)) == 1


def test_genetic_finds_the_gap_in_the_wall():
    """GA finds a route through the single gap in a land wall and every cell of it is
navigable."""
    cost_map = synthetic_map(barrier=True)
    result = genetic(cost_map, (2, 2), (17, 17), config=FAST)
    assert result.found
    assert all(cost_map.passable[cell] for cell in result.path_cells)


def test_genetic_is_never_better_than_the_astar_optimum():
    """GA route cost is never below the exact A* optimum on the same map."""
    cost_map = synthetic_map(barrier=True)
    exact = astar(cost_map, (2, 2), (17, 17))
    evolved = genetic(cost_map, (2, 2), (17, 17), config=FAST)
    assert evolved.total_cost >= exact.total_cost - 1e-6


def test_genetic_reaches_the_optimum_on_open_water():
    """On open water the GA cost is within 2 % of the A* optimum."""
    cost_map = synthetic_map(barrier=False)
    exact = astar(cost_map, (2, 2), (17, 17))
    evolved = genetic(cost_map, (2, 2), (17, 17), config=FAST)
    assert evolved.total_cost == pytest.approx(exact.total_cost, rel=0.02)


def test_genetic_is_reproducible_with_a_fixed_seed():
    """Two GA runs with the same seed return identical paths and costs."""
    cost_map = synthetic_map(barrier=True)
    a = genetic(cost_map, (2, 2), (17, 17), config=FAST, seed=7)
    b = genetic(cost_map, (2, 2), (17, 17), config=FAST, seed=7)
    assert a.path_cells == b.path_cells and a.total_cost == b.total_cost
```

```
def test_genetic_records_a_non_increasing_best_fitness():
    """Best cost recorded per generation never increases (elitism)."""
    cost_map = synthetic_map(barrier=True)
    result = genetic(cost_map, (2, 2), (17, 17), config=FAST)
    history = result.history
    assert all(later <= earlier + 1e-9 for earlier, later in zip(history, history[1:]))


def test_genetic_refuses_a_start_on_land():
    """GA reports no route when the start cell lies on land."""
    cost_map = synthetic_map(barrier=True)
    mid = cost_map.spec.n_cols // 2
    result = genetic(cost_map, (0, mid), (17, 17), config=FAST)
    assert not result.found
```

tests/test_coast_check.py

```
"""Tests of the independent coastline check and the automatic correction.

独立海岸线检查与自动修正的测试。

The planner is driven by small stand-in classifiers built on the land mask
itself, so that the tests do not depend on the trained network:

* ``TruthClassifier`` answers "land" exactly where the mask shows land;
* ``IslandBlindClassifier`` is the truth except that it takes Bornholm for
  water - the kind of mistake the 100-voyage test found (sparsely sampled
  islands taken for water);
* ``BlindClassifier`` takes everything for water.
"""
from __future__ import annotations

import sqlite3

import numpy as np
import pytest

pytest.importorskip("global_land_mask")

from maritime_route.config import CLASS_NAMES, IMPASSABLE_COST  # noqa: E402
from maritime_route.routing import coast_check  # noqa: E402
from maritime_route.routing.cost_map import CostMap, GridSpec  # noqa: E402
from maritime_route.routing.planner import RoutePlanner  # noqa: E402

OPEN, LAND = CLASS_NAMES.index("OPEN_SEA"), CLASS_NAMES.index("COASTLINE")


def _one_hot(index: np.ndarray) -> np.ndarray:
    probs = np.zeros((index.size, len(CLASS_NAMES)), dtype=np.float32)
    probs[np.arange(index.size), index] = 1.0
    return probs


class TruthClassifier:
    """Land exactly where the independent mask shows land."""

    def predict_coordinates(self, lat, lon):
        land = coast_check.is_land(lat, lon)
        return _one_hot(np.where(land, LAND, OPEN))


class IslandBlindClassifier(TruthClassifier):
    """The truth, except that Bornholm (55.0-55.3 N, 14.7-15.2 E) is 'water'."""

    def predict_coordinates(self, lat, lon):
        lat, lon = np.asarray(lat), np.asarray(lon)
        land = coast_check.is_land(lat, lon)
        island = (lat > 54.95) & (lat < 55.35) & (lon > 14.6) & (lon < 15.25)
        return _one_hot(np.where(land & ~island, LAND, OPEN))


class BlindClassifier:
    """Everything is open sea."""

    def predict_coordinates(self, lat, lon):
        return _one_hot(np.full(np.asarray(lat).size, OPEN))
```

```
# ------------------------------------------------------------- the check
def test_open_sea_route_is_clear():
    check = coast_check.check_route([(40.0, -40.0), (40.0, -35.0)])
    assert check.verdict == "clear"
    assert check.max_land_run_km == 0.0
    assert check.samples > 400


def test_route_across_jutland_is_a_crossing():
    # North Sea -> Kattegat straight across Denmark (about 130 km of land).
    check = coast_check.check_route([(56.2, 7.5), (56.2, 11.0)])
    assert check.verdict == "crossing"
    assert check.max_land_run_km > 100
    assert check.crossing_runs and check.crossing_runs[0].points


def test_the_quays_are_not_checked():
    # Departure 2 km inland of the Danish west coast, then out to sea: the
    # first PORT_KM are ignored, so the voyage is clear.
    check = coast_check.check_route([(56.0, 8.17), (56.0, 7.0)])
    assert check.verdict in ("clear", "grazing")


def test_summary_is_json_ready():
    check = coast_check.check_route([(56.2, 7.5), (56.2, 11.0)])
    summary = check.to_dict(corrections=2, refined=True, cells_blocked=7,
initial="crossing")
    assert summary["verdict"] == "crossing" and summary["initial_verdict"] == "crossing"
    assert 0 < len(summary["land_points"]) <= coast_check.MAX_REPORTED_POINTS
    assert summary["corrections"] == 2 and summary["refined"] is True


def test_blocking_copies_the_map():
    spec = GridSpec(lat_min=55.0, lat_max=56.0, lon_min=10.0, lon_max=11.0,
resolution_deg=0.25)
    shape = (spec.n_rows, spec.n_cols)
    cost_map = CostMap(spec=spec, class_index=np.zeros(shape, np.int8),
                       confidence=np.ones(shape, np.float32), cost=np.ones(shape,
np.float32))
    blocked = coast_check.block_cells(cost_map, {(1, 1), (2, 3)})
    assert not blocked.passable[1, 1] and not blocked.passable[2, 3]
    assert cost_map.passable.all()                      # the original is unchanged
    assert blocked.cost[1, 1] == IMPASSABLE_COST


# ---------------------------------------------------------- the safeguard
def test_route_missed_island_is_corrected_automatically():
    planner = RoutePlanner(IslandBlindClassifier())
    plan, _ = planner.plan((55.12, 13.9), (55.12, 16.0), "astar", 0.1)
    assert plan.feasible
    check = plan.coast_check
    assert check["initial_verdict"] == "crossing"   # the first route ran over Bornholm
    assert check["verdict"] != "crossing"           # ... the final one does not
    assert check["corrections"] >= 1
    assert plan.warning == ""
    assert "corrected automatically" in plan.message
    # Independent confirmation of the final route.
    assert coast_check.check_route(plan.waypoints).verdict != "crossing"


def test_correct_route_is_left_unchanged():
    planner = RoutePlanner(TruthClassifier())
    plan, _ = planner.plan((55.12, 13.9), (55.12, 16.0), "astar", 0.1)
    assert plan.coast_check["initial_verdict"] != "crossing"
    assert plan.coast_check["corrections"] == 0
    assert "coastline check passed" in plan.message


@pytest.mark.parametrize("algorithm", ["dijkstra", "dynamic"])
def test_safeguard_works_for_the_other_exact_methods(algorithm):
    planner = RoutePlanner(IslandBlindClassifier())
    plan, _ = planner.plan((55.12, 13.9), (55.12, 16.0), algorithm, 0.1)
    assert plan.feasible and plan.coast_check["verdict"] != "crossing"


def test_uncorrectable_route_carries_a_warning():
    class NoCorrection(RoutePlanner):
        MAX_COAST_CORRECTIONS = 0
```

```
    plan, _ = NoCorrection(BlindClassifier()).plan((56.2, 7.5), (56.2, 11.0), "astar",
0.25,
                                                   auto_refine=False)
    assert plan.feasible
    assert plan.coast_check["verdict"] == "crossing"
    assert "crosses land" in plan.warning
    assert plan.to_dict()["warning"] == plan.warning


def test_benchmark_map_is_checked_but_not_changed():
    planner = RoutePlanner(IslandBlindClassifier())
    _, cost_map = planner.plan((55.12, 13.9), (55.12, 16.0), "astar", 0.1,
coast_guard=False)
    plan, used = planner.plan((55.12, 13.9), (55.12, 16.0), "astar", 0.1,
                              reuse_cost_map=cost_map, auto_refine=False)
    assert plan.coast_check["verdict"] == "crossing"
    assert plan.coast_check["corrections"] == 0
    assert used is cost_map


def test_switching_the_guard_off_keeps_the_old_behaviour():
    planner = RoutePlanner(IslandBlindClassifier())
    plan, _ = planner.plan((55.12, 13.9), (55.12, 16.0), "astar", 0.1,
coast_guard=False)
    assert plan.coast_check["verdict"] == "crossing" and plan.coast_check["corrections"]
== 0


# ---------------------------------------------------------- persistence
def test_check_is_stored_with_the_route(repository):
    planner = RoutePlanner(IslandBlindClassifier())
    plan, _ = planner.plan((55.12, 13.9), (55.12, 16.0), "astar", 0.1)
    repository.save_route(plan)
    stored = repository.get_route(plan.route_id)
    assert stored["coast_check"]["initial_verdict"] == "crossing"
    assert stored["coast_check"]["verdict"] == plan.coast_check["verdict"]


def test_old_database_gets_the_new_column(tmp_path):
    from maritime_route.storage.repository import RouteRepository

    path = tmp_path / "old.db"
    with sqlite3.connect(path) as conn:
        # Route table of version 1.0: no coast_check_json column.
        conn.execute("""CREATE TABLE route (id INTEGER PRIMARY KEY AUTOINCREMENT,
            route_uid TEXT NOT NULL UNIQUE, session_id INTEGER, algorithm TEXT NOT NULL
            CHECK (algorithm IN ('astar','dijkstra','great_circle')),
            start_lat REAL NOT NULL, start_lon REAL NOT NULL, end_lat REAL NOT NULL,
            end_lon REAL NOT NULL, total_cost REAL, distance_km REAL NOT NULL,
            great_circle_km REAL NOT NULL, detour_ratio REAL, estimated_hours REAL,
            speed_knots REAL, nodes_expanded INTEGER, runtime_s REAL NOT NULL,
            grid_resolution REAL, grid_cells INTEGER, feasible INTEGER NOT NULL DEFAULT
1,
            baseline_json TEXT, zone_profile_json TEXT, created_utc TEXT NOT NULL)""")
        conn.execute("""INSERT INTO route(route_uid, algorithm, start_lat, start_lon,
end_lat,
            end_lon, distance_km, great_circle_km, runtime_s, created_utc)
            VALUES ('old0000000000001','astar',54,10,55,11,100,90,0.1,'2026-01-
01T00:00:00Z')""")
    repo = RouteRepository(path)        # the constructor runs the migrations
    with sqlite3.connect(path) as conn:
        columns = [row[1] for row in conn.execute("PRAGMA table_info(route)")]
    assert columns[-1] == "coast_check_json"
    assert repo.get_route("old0000000000001")["coast_check"] == {}


def test_pdf_line_describes_the_check():
    from maritime_route.export.exporters import _coast_text

    assert _coast_text({}).startswith("not performed")
    assert _coast_text({"verdict": "clear"}).startswith("passed")
    text = _coast_text({"verdict": "clear", "initial_verdict": "crossing",
"corrections": 2})
    assert "corrected automatically" in text
    assert _coast_text({"verdict": "crossing", "max_land_run_km":
12}).startswith("FAILED")
```

tests/test_storage_export.py

```
"""Tests of persistence and of the three export formats. 存储与导出测试。

Covers ``maritime_route.export.exporters`` (route GeoJSON, route CSV, PDF
report, classified-point GeoJSON/CSV) and
``maritime_route.storage.repository.RouteRepository`` (SQLite schema, route
and point round trips, statistics, cascading deletes, handling of non-finite
numbers and migration of an old schema). The routes and points are built by
hand with :func:`make_plan` and :func:`make_results`, and every storage test
uses a fresh temporary database from the ``repository`` fixture
(``conftest.py``), so no model is needed.
测试导出格式与 SQLite 存储；数据为手工构造，每个测试使用独立的临时数据库。
"""
from __future__ import annotations

import csv
import io
import json

import pytest

from maritime_route.config import CLASS_NAMES
from maritime_route.export.exporters import (
    classified_points_to_geojson, points_to_csv_string, route_to_csv_string,
    route_to_geojson, route_to_pdf,
)
from maritime_route.model.inference import ClassificationResult
from maritime_route.routing.planner import RouteLeg, RoutePlan


def make_plan() -> RoutePlan:
    """Build a small hand-written feasible RoutePlan (3 waypoints) for the tests.

    Returns
    -------
    RoutePlan
        Plan with fixed values (distance 320 km, great circle 300 km, a
        baseline that crosses land) so that exported and stored values can be
        compared with exact numbers. No planner or model is involved.
    """
    legs = [
        RouteLeg(54.0, 10.0, "COASTAL_SEA", 1.8, 0.0, 45.0),
        RouteLeg(55.0, 12.0, "OPEN_SEA", 1.0, 160.0, 44.0),
        RouteLeg(56.0, 14.0, "COASTAL_SEA", 1.8, 320.0, 43.0),
    ]
    return RoutePlan(
        route_id="testroute000001", algorithm="astar", start=(54.0, 10.0), end=(56.0,
14.0),
        waypoints=[(l.latitude, l.longitude) for l in legs], legs=legs,
        total_cost=430.0, distance_km=320.0, great_circle_km=300.0, detour_ratio=1.0667,
        estimated_hours=12.3, nodes_expanded=512, runtime_s=0.42,
        grid={"resolution_deg": 0.25, "n_cells": 4000, "n_rows": 50, "n_cols": 80},
        zone_profile={"OPEN_SEA": 1, "COASTAL_SEA": 2, "NEAR_COAST": 0, "COASTLINE": 0},
        baseline={"distance_km": 300.0, "is_navigable": False, "land_share": 0.25,
                  "land_samples": 40, "samples": 160, "extra_distance_pct": 6.67,
                  "verdict": "Great-circle route crosses land and is not navigable",
                  "weighted_cost": None, "cost_saving_pct": None,
                  "zone_profile": {c: 40 for c in CLASS_NAMES}},
        cost_map_stats={}, created_utc="2026-09-20T12:00:00Z", speed_knots=14.0,
    )


def make_results():
    """Build two classification results (one COASTAL_SEA, one OPEN_SEA).

    Returns
    -------
    list of ClassificationResult
        Points with depth values and uniform class probabilities.
    """
    return [
        ClassificationResult(54.0, 10.0, 1, "COASTAL_SEA", 0.91,
                             {c: 0.25 for c in CLASS_NAMES}, -25.0),
        ClassificationResult(55.0, 12.0, 0, "OPEN_SEA", 0.98,
                             {c: 0.25 for c in CLASS_NAMES}, -180.0),
    ]
```

```
# ---------------------------------------------------------------------- export
def test_geojson_structure_and_geometry_order():
    """Route GeoJSON is a FeatureCollection with route and great-circle features in
[lon, lat] order."""
    document = route_to_geojson(make_plan())
    assert document["type"] == "FeatureCollection"
    kinds = [f["properties"]["feature_type"] for f in document["features"]]
    assert "optimal_route" in kinds and "great_circle_reference" in kinds
    route = next(f for f in document["features"]
                 if f["properties"]["feature_type"] == "optimal_route")
    # GeoJSON stores [longitude, latitude]
    assert route["geometry"]["coordinates"][0] == [10.0, 54.0]
    json.dumps(document)          # must be serialisable


def test_geojson_contains_no_infinities():
    """Serialised route GeoJSON contains no "Infinity" or "NaN" (both are invalid
JSON)."""
    payload = json.dumps(route_to_geojson(make_plan()))
    assert "Infinity" not in payload and "NaN" not in payload


def test_route_csv_has_one_row_per_waypoint():
    """Route CSV has the expected header and one row per waypoint."""
    rows = list(csv.reader(io.StringIO(route_to_csv_string(make_plan()))))
    assert rows[0][:4] == ["seq", "latitude", "longitude", "class_code"]
    assert len(rows) == 1 + 3
    assert rows[1][3] == "COASTAL_SEA"


def test_impassable_cost_is_exported_as_infinity():
    """A berth in a shore zone must not export as the internal 1e6 constant."""
    plan = make_plan()
    plan.legs[0].zone_cost = 1.0e6
    rows = list(csv.reader(io.StringIO(route_to_csv_string(plan))))
    assert rows[1][4] == "inf"
    feature = next(f for f in route_to_geojson(plan)["features"]
                   if f["properties"].get("seq") == 0)
    assert feature["properties"]["zone_cost"] is None
    assert feature["properties"]["navigable"] is False


def test_points_exports():
    """Point exports: one GeoJSON feature per point, CSV header + rows with per-class
probability columns."""
    results = make_results()
    document = classified_points_to_geojson(results)
    assert len(document["features"]) == 2
    rows = list(csv.reader(io.StringIO(points_to_csv_string(results))))
    assert len(rows) == 3
    assert "p_OPEN_SEA" in rows[0]


def test_pdf_is_written_and_looks_like_a_pdf(tmp_path):
    """PDF report is written, starts with the %PDF signature and is not trivially
small."""
    path = route_to_pdf(make_plan(), tmp_path / "report.pdf",
                        model_summary={"architecture": "ResidualMLP", "n_features": 28,
                                       "parameters": 98364, "accuracy": 0.9665,
                                       "macro_f1": 0.9665, "trained_utc": "2026-09-20"})
    assert path.exists()
    data = path.read_bytes()
    assert data.startswith(b"%PDF")
    assert len(data) > 3000


# ---------------------------------------------------------------------- storage
def test_zone_table_is_seeded(repository):
    """A new database has the zone table filled with all four class codes."""
    with repository.connect() as conn:
        codes = [r["code"] for r in conn.execute("SELECT code FROM zone")]
    assert set(codes) == set(CLASS_NAMES)


def test_route_round_trip(repository):
    """A saved route is read back with its scalar fields, ordered waypoints and
baseline."""
    plan = make_plan()
    repository.save_route(plan)
    stored = repository.get_route(plan.route_id)
```

```
    assert stored is not None
    assert stored["algorithm"] == "astar"
    assert stored["distance_km"] == pytest.approx(320.0)
    assert len(stored["waypoints"]) == 3
    assert stored["waypoints"][0]["seq"] == 0
    assert stored["baseline"]["is_navigable"] is False


def test_points_and_histogram(repository):
    """Saved points are counted and summarised correctly in the class histogram."""
    session_id = repository.create_session("t", "unit-test")
    assert repository.save_points(session_id, make_results()) == 2
    histogram = repository.class_histogram(session_id)
    assert histogram["OPEN_SEA"] == 1 and histogram["COASTAL_SEA"] == 1


def test_statistics_and_delete(repository):
    """Statistics count a stored route; deleting it removes it and resets the count."""
    plan = make_plan()
    repository.save_route(plan)
    stats = repository.statistics()
    assert stats["routes"] == 1 and stats["waypoints"] == 3
    assert "astar" in stats["by_algorithm"]
    assert repository.delete_route(plan.route_id) is True
    assert repository.get_route(plan.route_id) is None
    assert repository.statistics()["routes"] == 0


def test_cascade_delete_removes_waypoints(repository):
    """Deleting a route also deletes its waypoints (ON DELETE CASCADE)."""
    plan = make_plan()
    repository.save_route(plan)
    repository.delete_route(plan.route_id)
    with repository.connect() as conn:
        assert conn.execute("SELECT COUNT(*) FROM waypoint").fetchone()[0] == 0


def test_non_finite_values_become_null(repository):
    """inf/NaN route metrics are stored as NULL and read back as None."""
    plan = make_plan()
    plan.total_cost = float("inf")
    plan.detour_ratio = float("nan")
    repository.save_route(plan)
    stored = repository.get_route(plan.route_id)
    assert stored["total_cost"] is None
    assert stored["detour_ratio"] is None


def test_genetic_route_can_be_stored(repository):
    """A route produced by the genetic algorithm is accepted by the schema."""
    plan = make_plan()
    plan.algorithm = "genetic"
    repository.save_route(plan)
    assert repository.get_route(plan.route_id)["algorithm"] == "genetic"


def test_old_database_is_upgraded_for_the_genetic_algorithm(tmp_path):
    """A database created before the GA existed is migrated on start-up."""
    import sqlite3

    from maritime_route.storage.repository import SCHEMA_PATH, RouteRepository

    path = tmp_path / "old.db"
    old_schema = SCHEMA_PATH.read_text(encoding="utf-8").replace(
        "'astar','dijkstra','genetic','dynamic','great_circle'",
"'astar','dijkstra','great_circle'")
    with sqlite3.connect(path) as conn:
        conn.executescript(old_schema)
    repository = RouteRepository(path)
    plan = make_plan()
    plan.algorithm = "genetic"
    repository.save_route(plan)
    assert repository.get_route(plan.route_id)["algorithm"] == "genetic"
```

tests/test_model_and_api.py

```
"""Integration tests of the trained model and of the HTTP/WebSocket API.
```

模型推理与 Web 接口的集成测试（需要已训练的模型）。

Three groups of tests:

* model - the trained ``ZoneClassifierService``: probability output, plausible
  classes for known places, reported quality, determinism, batch consistency;
* routing - ``RoutePlanner`` on the real model for Kiel -> Tallinn: no land
  between the berths, correct end points, land-crossing great-circle baseline,
  A*/Dijkstra agreement and automatic lattice refinement;
* API - every group of endpoints of ``maritime_route.web.app`` through
  FastAPI's ``TestClient`` (REST requests, validation errors, exports, upload,
  offline tiles) and the ``/ws/plan`` WebSocket protocol.

The whole module is skipped (``pytestmark = requires_model``) when the trained
model files are missing.
整个模块在缺少已训练模型时跳过。
"""
```
from __future__ import annotations

import json

import numpy as np
import pytest
from fastapi.testclient import TestClient

from conftest import requires_model
from maritime_route.config import CLASS_NAMES

# Module-level marker: applies the skip condition to every test in this file.
pytestmark = requires_model


@pytest.fixture(scope="module")
def client():
    """FastAPI TestClient bound to the real application, shared by the module.

    Using the client as a context manager runs the application's start-up and
    shutdown events. Requests are served in-process, without a network socket.
    Note that routes planned here are stored in the application's configured
    database (the ``STATE`` repository), not in a temporary one.

    Yields
    ------
    fastapi.testclient.TestClient
    """
    from maritime_route.web.app import app
    with TestClient(app) as test_client:
        yield test_client


# ---------------------------------------------------------------------- model
def test_probabilities_are_a_distribution(classifier):
    """Predicted class probabilities have shape (N, 4), are non-negative and sum to 1
per point."""
    probs = classifier.predict_coordinates([54.5, 57.0, 52.5], [10.2, 19.0, 13.4])
    assert probs.shape == (3, len(CLASS_NAMES))
    np.testing.assert_allclose(probs.sum(axis=1), 1.0, rtol=1e-5)
    assert (probs >= 0).all()


def test_known_locations_are_classified_plausibly(classifier):
    """Deep Atlantic must be sea; a point deep inland must be land."""
    results = classifier.classify_coordinates(
        [58.0, 52.52], [-25.0, 13.40])          # mid-Atlantic, Berlin
    assert results[0].class_code in ("OPEN_SEA", "COASTAL_SEA")
    assert results[1].class_code in ("COASTLINE", "NEAR_COAST")


def test_model_summary_reports_trained_quality(classifier):
    """Model summary reports 28 features, a positive parameter count and test accuracy
above 90 %."""
    summary = classifier.summary()
    assert summary["n_features"] == 28
    assert summary["parameters"] > 0
    assert summary["accuracy"] > 0.90


def test_inference_is_deterministic(classifier):
    """Repeated inference on the same coordinate gives identical probabilities (eval
mode, no dropout)."""
```

```
    a = classifier.predict_coordinates([55.0], [12.0])
    b = classifier.predict_coordinates([55.0], [12.0])
    np.testing.assert_allclose(a, b, rtol=1e-6)


def test_batched_inference_matches_single(classifier):
    """Classifying points in one batch gives the same result as one call per point."""
    lat = [54.0, 55.0, 56.0, 57.0]
    lon = [10.0, 12.0, 14.0, 16.0]
    batch = classifier.predict_coordinates(lat, lon)
    single = np.vstack([classifier.predict_coordinates([a], [b]) for a, b in zip(lat,
lon)])
    np.testing.assert_allclose(batch, single, rtol=1e-5, atol=1e-6)


# ---------------------------------------------------------------------- routing
def test_planned_route_never_crosses_land(planner):
    """Every intermediate waypoint must be navigable.

    The two end points are the berths the user asked for, and a quay legitimately
    lies in a shore zone, so the invariant is stated over the interior of the route.
    首尾为用户指定的泊位，允许位于岸线区；中间航路点必须全部可航行。
    """
    plan, _ = planner.plan((54.3233, 10.1228), (59.4370, 24.7536), "astar", 0.25)
    assert plan.feasible
    assert plan.interior_land_waypoints == 0
    for leg in plan.legs[1:-1]:
        assert leg.class_code in ("OPEN_SEA", "COASTAL_SEA")
    assert plan.distance_km >= plan.great_circle_km


def test_departure_berth_may_lie_on_the_shore(planner):
    """Kiel is a port: its exact coordinates fall in a shore cell by design."""
    plan, _ = planner.plan((54.3233, 10.1228), (59.4370, 24.7536), "astar", 0.25)
    assert plan.legs[0].latitude == pytest.approx(54.3233)
    assert plan.legs[-1].latitude == pytest.approx(59.4370)
    assert plan.interior_land_waypoints == 0


def test_baseline_reports_land_crossing(planner):
    """The great-circle baseline Kiel -> Tallinn is reported as crossing land with no
weighted cost."""
    plan, _ = planner.plan((54.3233, 10.1228), (59.4370, 24.7536), "astar", 0.25)
    assert plan.baseline["is_navigable"] is False
    assert plan.baseline["land_samples"] > 0
    assert plan.baseline["weighted_cost"] is None


def test_astar_and_dijkstra_return_the_same_optimum(planner):
    """On one shared cost map A* and Dijkstra reach the same cost, A* with no more
expanded nodes."""
    plan_a, cost_map = planner.plan((54.3233, 10.1228), (59.4370, 24.7536), "astar",
0.3)
    plan_d, _ = planner.plan((54.3233, 10.1228), (59.4370, 24.7536), "dijkstra",
                             cost_map.spec.resolution_deg, reuse_cost_map=cost_map,
                             auto_refine=False)
    assert plan_a.total_cost == pytest.approx(plan_d.total_cost, rel=1e-6)
    assert plan_a.nodes_expanded <= plan_d.nodes_expanded


def test_auto_refinement_recovers_a_narrow_strait(planner):
    """At 0.5° the Danish straits are narrower than a cell; refinement must recover."""
    plan, cost_map = planner.plan((54.3233, 10.1228), (59.4370, 24.7536), "astar", 0.5)
    assert plan.feasible
    assert cost_map.spec.resolution_deg < 0.5


# ---------------------------------------------------------------------- API
def test_health_and_zones(client):
    """GET /api/health reports "ok"; GET /api/zones lists the classes in model order,
two of them navigable."""
    assert client.get("/api/health").json()["status"] == "ok"
    zones = client.get("/api/zones").json()["classes"]
    assert [z["code"] for z in zones] == CLASS_NAMES
    assert sum(1 for z in zones if z["navigable"]) == 2


def test_index_page_is_served(client):
    """GET / returns the single-page client HTML."""
    response = client.get("/")
```

```
    assert response.status_code == 200
    assert "Maritime Route Planner" in response.text


def test_classify_endpoint(client):
    """POST /api/classify returns one result per point and a histogram with the same
total."""
    response = client.post("/api/classify", json={"points": [[54.5, 10.2], [52.5,
13.4]]})
    assert response.status_code == 200
    body = response.json()
    assert body["n"] == 2
    assert sum(body["histogram"].values()) == 2


def test_classify_rejects_bad_coordinates(client):
    """POST /api/classify answers 422 for an out-of-range latitude and for a one-number
point."""
    assert client.post("/api/classify", json={"points": [[200.0, 10.0]]}).status_code ==
422
    assert client.post("/api/classify", json={"points": [[10.0]]}).status_code == 422


def test_route_endpoint_and_exports(client):
    """POST /api/route plans a feasible route that exports as GeoJSON, CSV and PDF; an
unknown format gives 400."""
    response = client.post("/api/route", json={
        "start_lat": 54.3233, "start_lon": 10.1228,
        "end_lat": 59.4370, "end_lon": 24.7536,
        "algorithm": "astar", "resolution_deg": 0.3,
    })
    assert response.status_code == 200
    plan = response.json()
    assert plan["feasible"] is True
    route_id = plan["route_id"]

    geojson = client.get(f"/api/export/{route_id}.geojson")
    assert geojson.status_code == 200
    assert json.loads(geojson.text)["type"] == "FeatureCollection"

    csv_response = client.get(f"/api/export/{route_id}.csv")
    assert csv_response.status_code == 200
    assert csv_response.text.startswith("seq,latitude,longitude")

    pdf = client.get(f"/api/export/{route_id}.pdf")
    assert pdf.status_code == 200
    assert pdf.content.startswith(b"%PDF")

    assert client.get(f"/api/export/{route_id}.docx").status_code == 400


def test_route_validation_rejects_bad_input(client):
    """POST /api/route answers 422 for a latitude above 90 and for an unknown
algorithm."""
    bad = {"start_lat": 91.0, "start_lon": 0.0, "end_lat": 0.0, "end_lon": 0.0}
    assert client.post("/api/route", json=bad).status_code == 422
    bad_alg = {"start_lat": 54.0, "start_lon": 10.0, "end_lat": 55.0, "end_lon": 11.0,
               "algorithm": "bfs"}
    assert client.post("/api/route", json=bad_alg).status_code == 422


def test_routes_are_listed_and_retrievable(client):
    """Stored routes are listed, retrievable by uid, and an unknown uid gives 404."""
    client.post("/api/route", json={
        "start_lat": 54.3233, "start_lon": 10.1228,
        "end_lat": 57.0, "end_lon": 18.0, "resolution_deg": 0.4})
    routes = client.get("/api/routes?limit=5").json()["routes"]
    assert routes
    uid = routes[0]["route_uid"]
    assert client.get(f"/api/routes/{uid}").json()["route_uid"] == uid
    assert client.get("/api/routes/does-not-exist").status_code == 404


def test_upload_geojson_and_classify(client, tmp_path):
    """POST /api/upload classifies an uploaded GeoJSON and compares against its
labels."""
    document = {
        "type": "FeatureCollection",
        "features": [
            {"type": "Feature", "properties": {"class_code": "OPEN_SEA"},
```

```
             "geometry": {"type": "Point", "coordinates": [-25.0, 58.0]}},
            {"type": "Feature", "properties": {"class_code": "COASTLINE"},
             "geometry": {"type": "Point", "coordinates": [13.40, 52.52]}},
        ],
    }
    path = tmp_path / "track.geojson"
    path.write_text(json.dumps(document), encoding="utf-8")
    with path.open("rb") as handle:
        response = client.post("/api/upload",
                               files={"file": ("track.geojson", handle,
"application/json")})
    assert response.status_code == 200
    body = response.json()
    assert body["classified"] == 2
    assert body["accuracy_vs_labels"]["n_labelled"] == 2


def test_upload_rejects_a_file_without_coordinates(client, tmp_path):
    """POST /api/upload answers 400 for a CSV without coordinate columns."""
    path = tmp_path / "bad.csv"
    path.write_text("a,b\n1,2\n", encoding="utf-8")
    with path.open("rb") as handle:
        response = client.post("/api/upload", files={"file": ("bad.csv", handle,
"text/csv")})
    assert response.status_code == 400


def test_on_board_tile_is_a_png(client):
    """Offline tile endpoint returns a PNG (checked by signature) and 400 for zoom >
10."""
    response = client.get("/api/tiles/zones/4/8/5.png")
    assert response.status_code == 200
    assert response.content[:8] == b"\x89PNG\r\n\x1a\n"
    # Zoom 14 is above the on-board chart limit (0..10).
    assert client.get("/api/tiles/zones/14/1/1.png").status_code == 400


def test_websocket_live_planning(client):
    """WebSocket protocol: hello, ping/pong, classify, then
started/progress/overlay/result for a plan."""
    with client.websocket_connect("/ws/plan") as socket:
        assert socket.receive_json()["type"] == "hello"

        socket.send_json({"action": "ping"})
        assert socket.receive_json()["type"] == "pong"

        socket.send_json({"action": "classify", "points": [[54.5, 10.2]]})
        frame = socket.receive_json()
        assert frame["type"] == "classified"
        assert frame["results"][0]["class_code"] in CLASS_NAMES

        socket.send_json({
            "action": "plan", "start_lat": 54.3233, "start_lon": 10.1228,
            "end_lat": 59.4370, "end_lon": 24.7536, "algorithm": "astar",
            "resolution_deg": 0.3, "include_overlay": True,
        })
        # Collect frame types until the result arrives; the 400-frame bound
        # keeps a protocol bug from hanging the test suite forever.
        # 限制最多读取 400 帧，避免协议错误导致测试无限等待。
        seen = set()
        for _ in range(400):
            frame = socket.receive_json()
            seen.add(frame["type"])
            if frame["type"] == "result":
                assert frame["plan"]["feasible"] is True
                break
            if frame["type"] == "error":
                pytest.fail(frame["detail"])
        assert {"started", "progress", "overlay", "result"} <= seen


def test_websocket_reports_a_bad_request(client):
    """WebSocket answers an invalid plan request and an unknown action with "error"
frames."""
    with client.websocket_connect("/ws/plan") as socket:
        socket.receive_json()
        socket.send_json({"action": "plan", "start_lat": 500.0})
        assert socket.receive_json()["type"] == "error"
        socket.send_json({"action": "teleport"})
        assert socket.receive_json()["type"] == "error"
```

```
def test_statistics_endpoint(client):
    """GET /api/statistics contains the session, point, route and waypoint counters."""
    stats = client.get("/api/statistics").json()
    assert {"sessions", "points", "routes", "waypoints"} <= set(stats)
```